

A man in a tan polo shirt is smiling and writing on a notepad in the foreground. In the background, two other men are looking at a laptop and some equipment. The setting appears to be a training or office environment.

**SIEMENS**

# SITRAIN: Training for Industry

SIMATIC TIA Portal Programming 1  
(TIA-PRO1)

[siemens.com/sitrain](https://www.siemens.com/sitrain)

# SIEMENS

## SITRAIN

Training for Industry

## SIMATIC S7

## TIA Portal Programming 1

## Course TIA-PRO1

Name: \_\_\_\_\_

Course from: \_\_\_\_\_ to: \_\_\_\_\_

Instructor: \_\_\_\_\_

Location: \_\_\_\_\_

This document was produced for training purposes. SIEMENS assumes no responsibility for its contents. The reproduction, transmission or use of this document or its contents is not permitted without express written authority. Offenders will be liable to damages.

Copyright © Siemens AG 2014. All rights, including rights created by patent grant or registration of a utility model or design, are reserved.

SITRAIN course offer on the Internet: [www.siemens.com/sitrain](http://www.siemens.com/sitrain)

Course folder Version: V13 (for STEP7 Version 13)

|    |                                                       |
|----|-------------------------------------------------------|
| 1  | The SIMATIC S7 System Family                          |
| 2  | Introduction to TIA Portal                            |
| 3  | Training Devices                                      |
| 4  | Devices and Networks                                  |
| 5  | PLC Tags                                              |
| 6  | Program Blocks and Program Editor                     |
| 7  | Binary Operations                                     |
| 8  | Digital Operations                                    |
| 9  | Data Blocks                                           |
| 10 | Distributed I/O                                       |
| 11 | Connecting an HMI Device                              |
| 12 | Functions and Function Blocks (1)                     |
| 13 | Organization Blocks                                   |
| 14 | Troubleshooting                                       |
| 15 | Integrating and Commissioning a Drive with Startdrive |
| 16 | Training and Support                                  |
| 17 |                                                       |
| 18 |                                                       |
| 19 |                                                       |
| 20 |                                                       |

# Contents

|           |                                                                                 |            |
|-----------|---------------------------------------------------------------------------------|------------|
| <b>1.</b> | <b>System Overview.....</b>                                                     | <b>1-2</b> |
| 1.1.      | SIMATIC S7 Overview .....                                                       | 1-3        |
| 1.2.      | Positioning of the Modular S7 Controllers .....                                 | 1-4        |
| 1.3.      | Overview: New, Available Modules .....                                          | 1-5        |
| 1.3.1.    | Overview of Available Central Modules .....                                     | 1-5        |
| 1.3.2.    | Overview of Available Signal Modules (Central).....                             | 1-5        |
| 1.4.      | SIMATIC S7-1200: The Modular Mini-PLC.....                                      | 1-6        |
| 1.4.1.    | SIMATIC S7-1200: Modules .....                                                  | 1-7        |
| 1.4.2.    | SIMATIC S7-1200: Installation and Mounting Positions .....                      | 1-8        |
| 1.4.3.    | SIMATIC S7-1200: Signal, Communication or Battery Board .....                   | 1-9        |
| 1.5.      | SIMATIC S7-1500: Modular Controller for the Mid to Upper Performance Range..... | 1-10       |
| 1.5.1.    | SIMATIC S7-1500: Modules .....                                                  | 1-11       |
| 1.5.2.    | SIMATIC S7-1500: Installation and Mounting Positions .....                      | 1-13       |
| 1.5.3.    | SIMATIC S7-1500: Connection Technology/Front Connector.....                     | 1-14       |
| 1.5.4.    | SIMATIC S7-1500: CPU-Display → Overview .....                                   | 1-15       |
| 1.5.5.    | SIMATIC S7-1500: CPU-Display → Menu and Colors .....                            | 1-16       |
| 1.6.      | SIMATIC S7-1200/1500: Technology Functions .....                                | 1-17       |
| 1.7.      | SIMATIC S7-1200/1500: Memory Card.....                                          | 1-18       |
| 1.8.      | Distributed I/O Systems .....                                                   | 1-19       |
| 1.8.1.    | Overview: Distributed Signal Modules .....                                      | 1-19       |
| 1.8.2.    | ET 200SP Distributed I/O System.....                                            | 1-20       |
| 1.8.2.1.  | ET 200SP: Configuration and Maximum Number of Modules .....                     | 1-20       |
| 1.8.3.    | ET 200MP Distributed I/O System .....                                           | 1-21       |
| 1.8.3.1.  | ET 200MP: Configuration and Maximum Number of Modules .....                     | 1-21       |
| 1.9.      | Additional Information .....                                                    | 1-22       |
| 1.10.     | SIMATIC S7-300: Modular Automation System .....                                 | 1-23       |
| 1.10.1.   | SIMATIC S7-300: Modules .....                                                   | 1-24       |
| 1.11.     | SIMATIC S7-400: Modular Automation System .....                                 | 1-25       |
| 1.11.1.   | SIMATIC S7-400: Modules .....                                                   | 1-26       |
| 1.11.2.   | SIMATIC S7-400: CPU Design.....                                                 | 1-27       |
| 1.12.     | SIMATIC Embedded Automation: Overview.....                                      | 1-28       |
| 1.12.1.   | SIMATIC PC-based Automation Product Overview.....                               | 1-29       |

# 1. System Overview

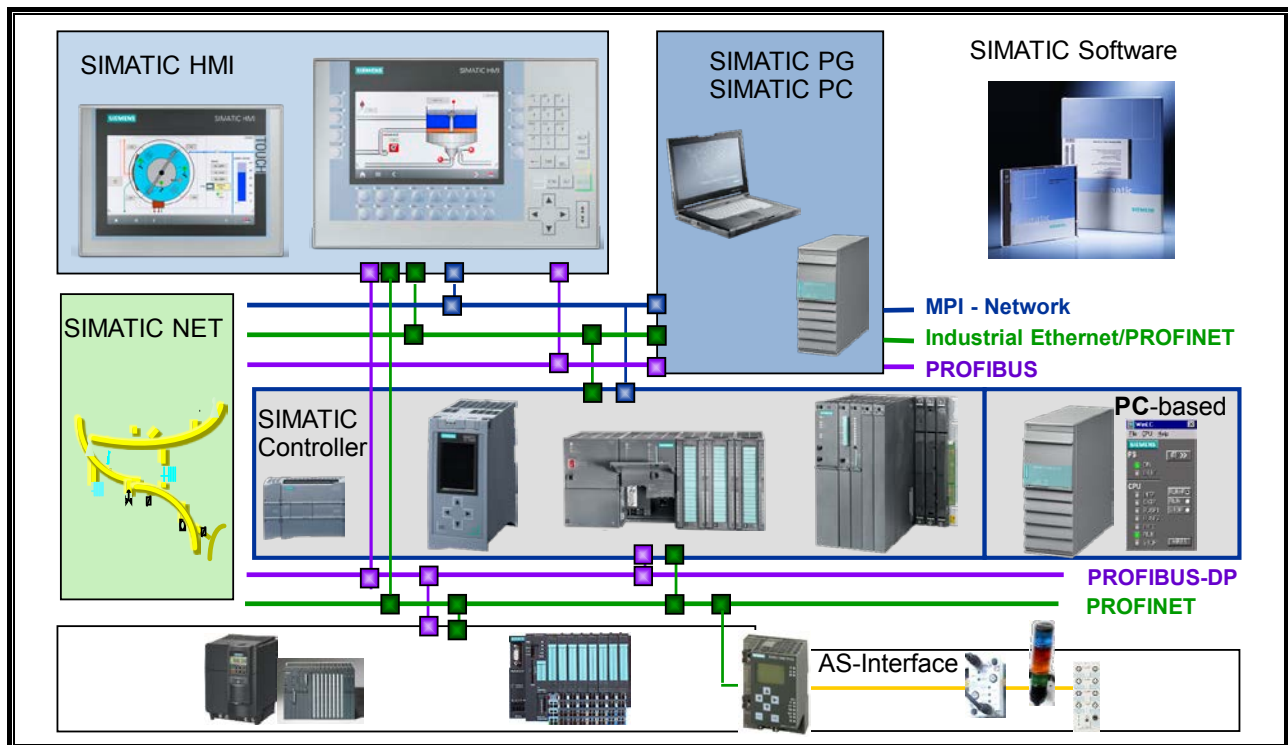
**At the end of the chapter the participant will ...**



- ... have an overview of the new SIMATIC S7-1200/1500 system family
- ... be familiar with the S7-1200 and S7-1500 automation systems
- ... have an overview of the modules available for these systems
- ... be familiar with the S7-300 and S7-400 automation systems
- ... understand the concept “Totally Integrated Automation” (T.I.A.)
- ... be familiar with the programming devices
- ... be familiar with the PC requirements for working with the TIA Portal



## 1.1. SIMATIC S7 Overview



### Introduction

For the operation of machines, equipment and processes in almost all areas of manufacturing you require control elements in addition to energy supply. It must be possible to initiate, control, monitor and end the operation of any given machine or process.

### Hard-wired Programmed Controller → PLC

In the hard-wired controllers of the past, the program logic was governed by the task-specific wiring of contactors and relays.

Today, programmable logic controllers are used to solve automation tasks. The logic stored in the program memory of an automation system does not depend on equipment design and wiring and can be modified at any time with the help of a programming device.

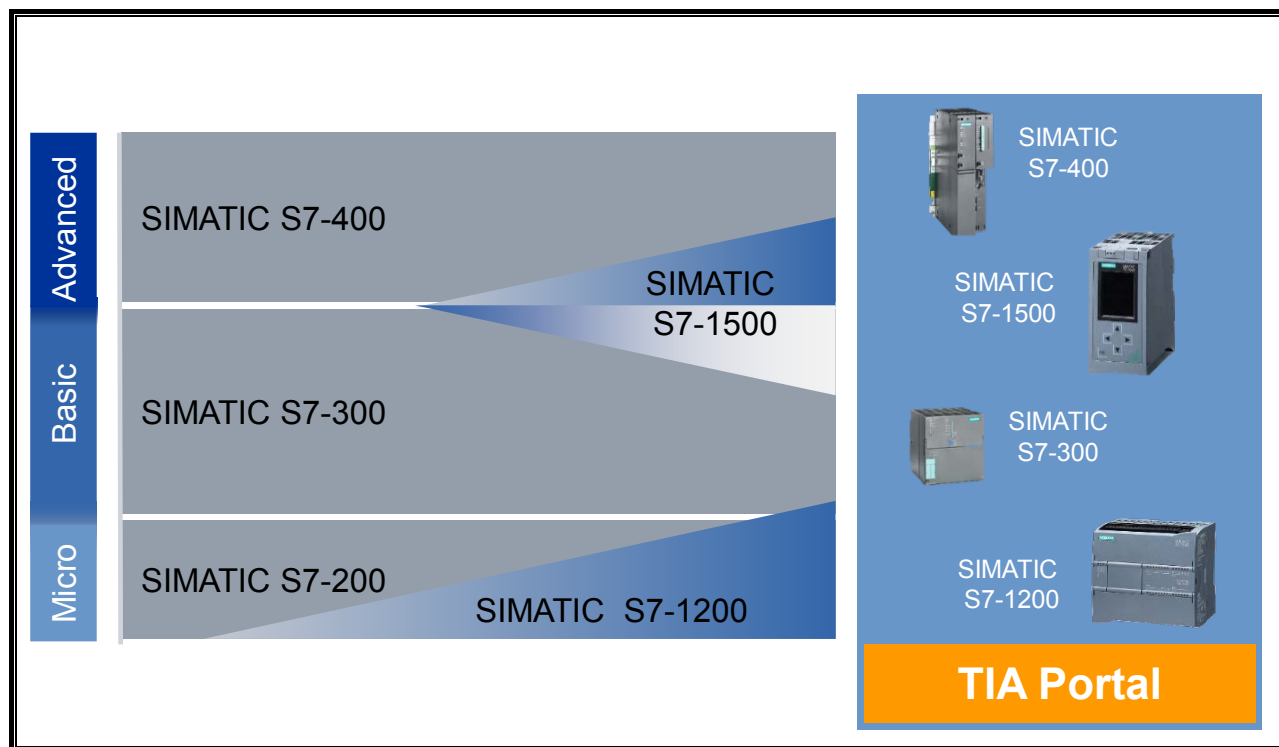
### Totally Integrated Automation

Production processes are no longer seen as individual partial processes, but rather as integral components of an entire production process.

The total integration of the entire automation environment is today achieved with the help of:

- one common software environment that integrates all components and tasks into one uniform easy to use system
- a common data management (central database)
- a common communication between all participating automation components.

## 1.2. Positioning of the Modular S7 Controllers



### SIMATIC S7

The programmable logic controllers can be divided into the micro PLC (S7-1200) and the mid/upper (S7-1500) performance ranges.

### 1.3. Overview: New, Available Modules

The S7-1200 and S7-1500 product range will be expanded in the next years such that the S7-300 and even the S7-400 can be completely replaced.

#### 1.3.1. Overview of Available Central Modules

|                   | S7-1200               | S7-1500               | S7-300 | S7-400 |
|-------------------|-----------------------|-----------------------|--------|--------|
| Standard          | ✓                     | ✓                     | ✓      | ✓      |
| Fail-safe         |                       | ✓                     | ✓      | ✓      |
| Compact           | ✓                     |                       | ✓      | ✗      |
| High availability | ✗                     | ✗                     | ✗      | ✓      |
| Technology        | ✓ different functions | ✓ different functions | ✗      | ✗      |

#### 1.3.2. Overview of Available Signal Modules (Central)

|           | S7-1200 | S7-1500 | S7-300 | S7-400 |
|-----------|---------|---------|--------|--------|
| DI/DQ     | ✓       | ✓       | ✓      | ✓      |
| AI/AQ     | ✓       | ✓       | ✓      | ✓      |
| F-DI/F-DQ |         |         | ✓      | ✗      |
| F-AI/F-AQ |         |         | ✓      | ✗      |

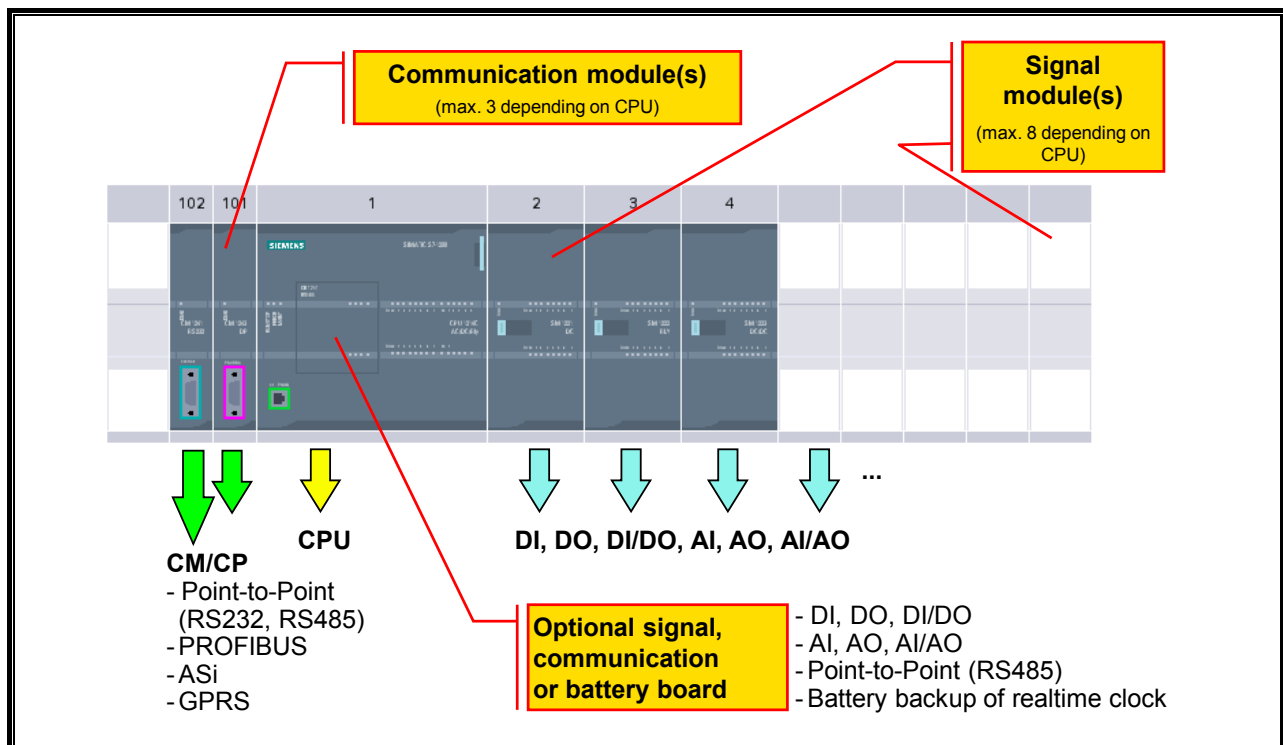
## 1.4. SIMATIC S7-1200: The Modular Mini-PLC



### Features

- Modular compact control system for the low-end performance range
- Scaled CPU range
- Extensive range of modules
- Can be expanded to up to 11 modules (depends on the CPU)
- Can be networked with PROFIBUS or PROFINET
- Slot rules
  - CM left of the CPU (number depends on the CPU)
  - SM right of the CPU (number depends on the CPU)
- "Total package" with CPU and I/O in one device
  - integrated digital and analog I/O
  - an expansion with signal board
- "Micro PLC" with integrated functions

### 1.4.1. SIMATIC S7-1200: Modules



#### Slot Rules

- CM left of the CPU (number depends on the CPU)
- Signal modules (digital, analog) right of the CPU (number depends on the CPU)

#### Signal Modules

- Digital input, output or mixed modules (24VDC, relay)
- Analog input, output or mixed modules (voltage, current, resistance, thermocouple)

#### Communication Modules (CM - Communication Module, CP - Communication Processor)

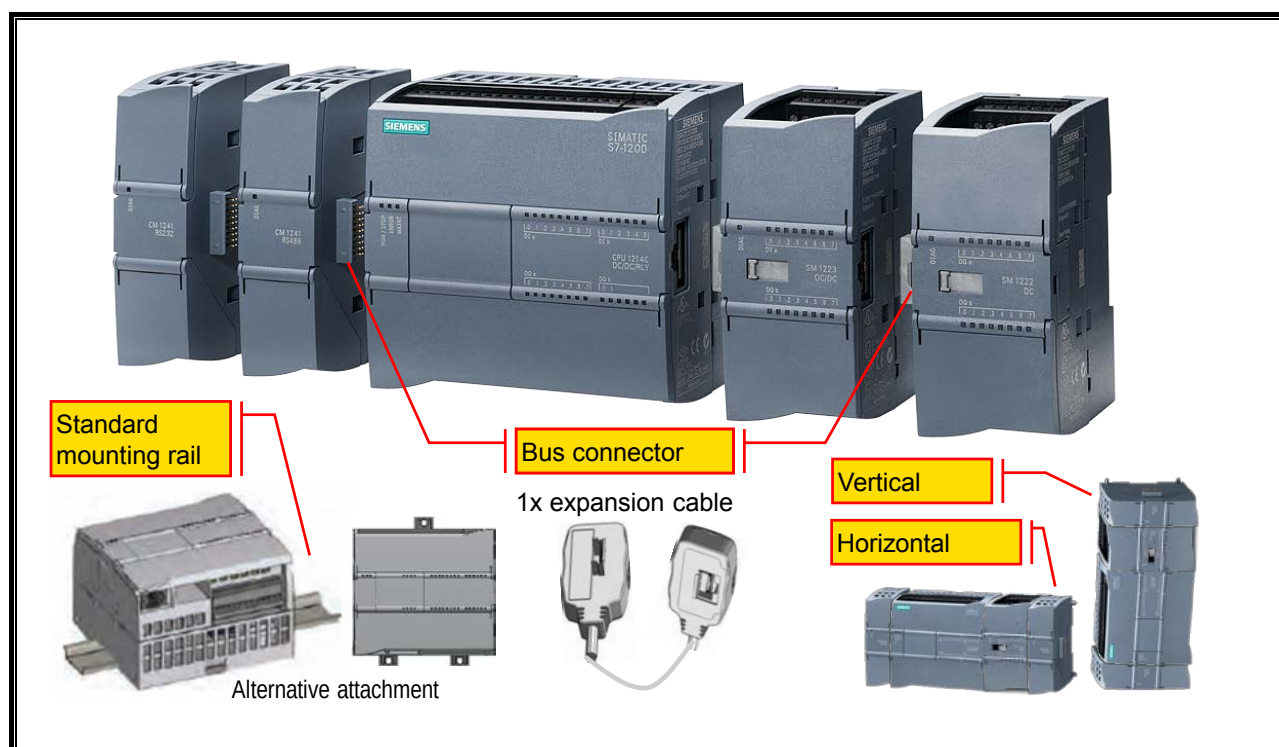
- Point-to-point connection (RS232, RS485)
- PROFIBUS
- ASi-Master
- Telecontrol (GPRS functionality)

#### Expansion Board

With this the CPU can be expanded by onboard I/O or an interface.

A battery board ensures the long-term battery backup (buffering) of the real-time clock.

## 1.4.2. SIMATIC S7-1200: Installation and Mounting Positions



### Installation

The modules are mounted on a standard mounting rail or alternatively screwed into the control cabinet.

### S7-1200 Expansion Cable

It offers additional flexibility in configuring the S7-1200 system. One expansion cable can be used for each CPU system.

- Either between the CPU and the first SM or between two SMs

### Bus Connector

It is located as a mechanical slide on the left side of the SM modules.

It is mechanically attached on the right side of the CMs/CPs.

### Mounting Positions

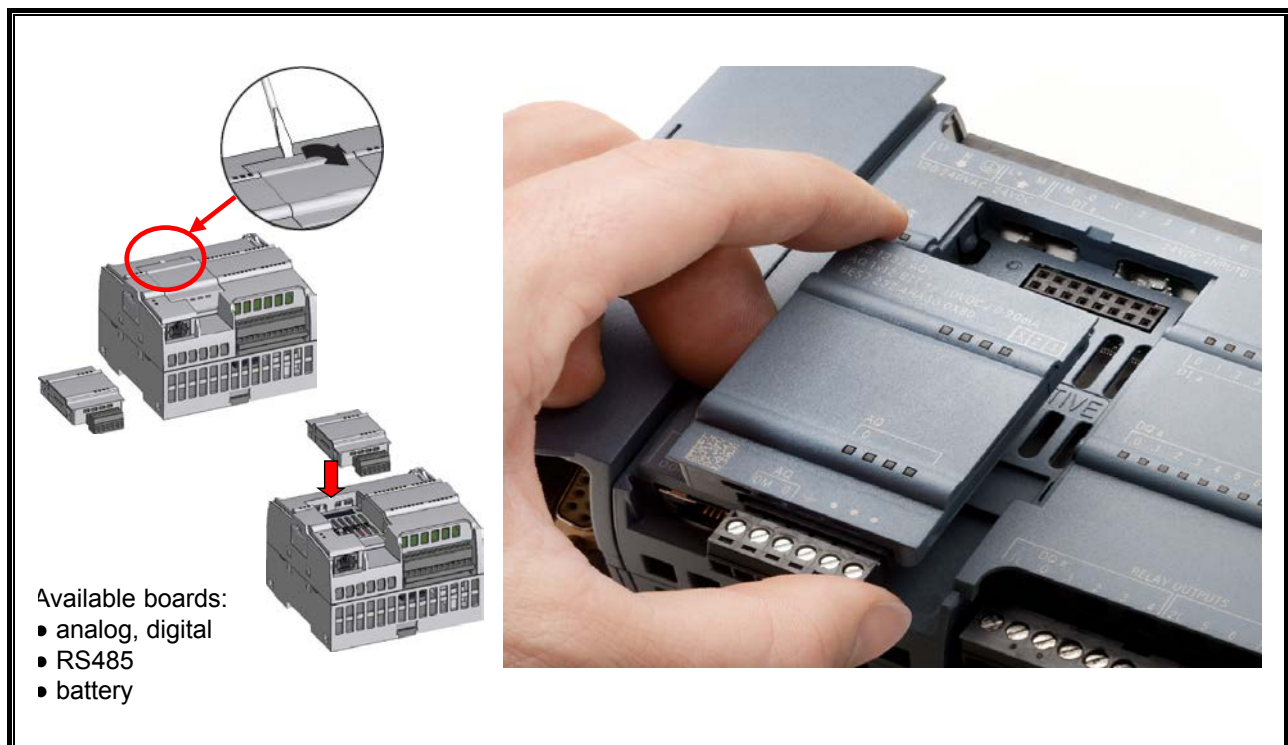
A horizontal or vertical mounting is possible.



#### Caution!

With a vertical mounting, the maximum allowed ambient temperature is 10 °C lower.

### 1.4.3. SIMATIC S7-1200: Signal, Communication or Battery Board



#### Application

These boards are used for application-specific adaptation of the CPU to the requirements of the plant. The physical size of the CPU remains unchanged.

#### Signal Board (SB)

- Digital signal board
  - only inputs
  - only outputs
  - inputs and outputs
- Analog signal board
  - only inputs
  - only outputs

#### Communication Board (CB)

- RS485 interface

#### Battery Board (BB)

A battery board (housing for CR1025 battery) ensures the long-term battery backup (buffering) of the real-time clock.

- Buffering time without battery board typically 20 days / minimum 12 days at 40°C
- Buffering time with battery board approximately 1 year

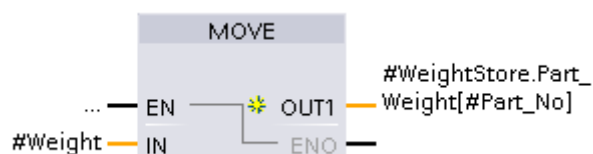


## 1.5. SIMATIC S7-1500: Modular Controller for the Mid to Upper Performance Range



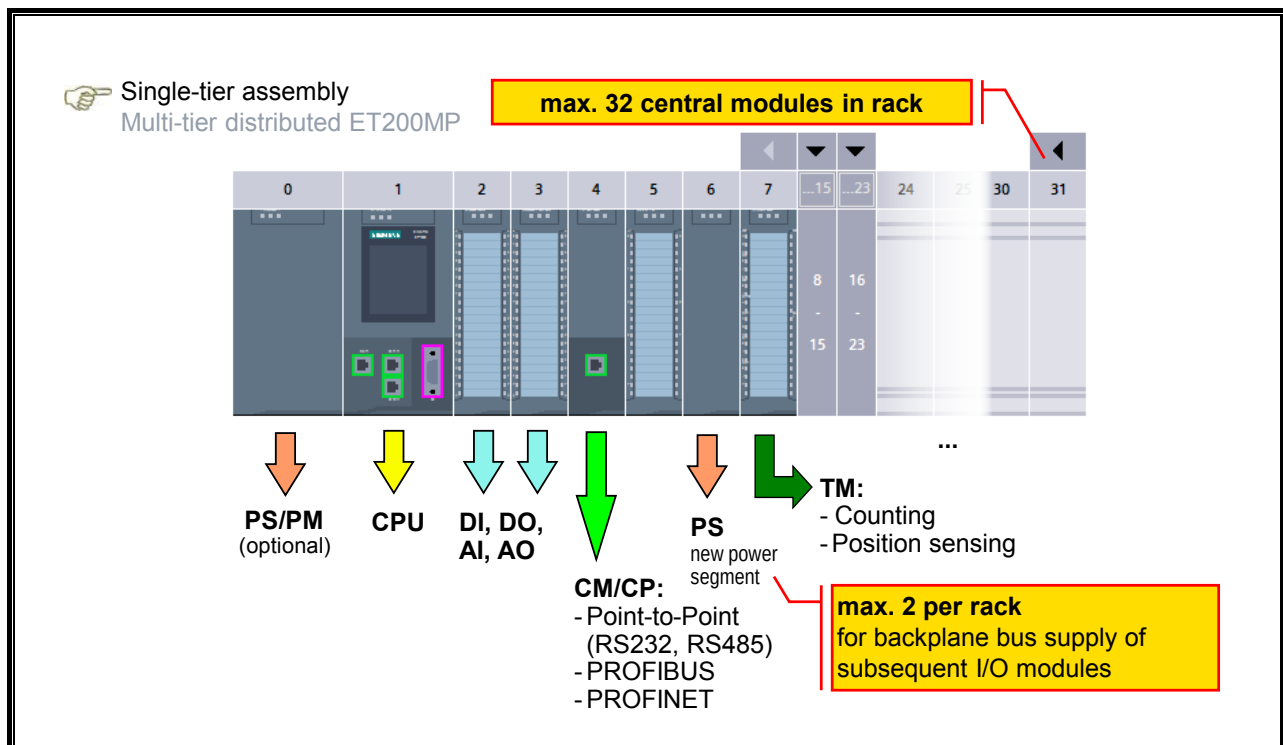
### Highlights of the SIMATIC S7-1500 System

- Maximum performance of the entire system (terminal-terminal)
  - High performance program execution in the CPU
  - High performance backplane bus
  - PROFINET interface with PROFINET IO IRT on every CPU
- Automatically activated system diagnostics, right down to the IO channel
- Trace for all CPU tags
- CPU - Display for:
  - Access to MLFB, FW-version and serial numbers
  - Commissioning (e.g. Setting the IP address, station name)
  - Diagnostics
- Simplified programming through user-friendly, new instructions in LAD/FBD/STL





### 1.5.1. SIMATIC S7-1500: Modules



#### Slot Rules

- 1x PS/PM Slot 0
- 1x CPU in Slot 1
- As of Slot 2 any

#### Signal Modules

- Digital input modules: 24VDC, 230VAC
- Digital output modules: 24VDC, 230VAC
- Analog input modules: voltage, current, resistance, thermocouple
- Analog output modules: voltage, current

#### Communication Modules (CP - Communication Processor, CM - Communication Module)

- Point-to-Point connection
- PROFIBUS
- PROFINET



CPs and CMs are both communication modules.  
CPs have, as a rule, somewhat more functionality than CMs (e.g. own web server, firewall, or the like).

#### Technology Modules (TM - Technology Module)

- Counting
- Position sensing

## Power Supply

I/O modules in the central rack of the S7-1500 require a system power supply via the backplane bus (communication connection to the CPU) and a load power supply (input or output circuits for sensors/encoders and actuators).

- **PM - Power Module → Load Power Supply**

supplies modules with 24VDC for input and output circuits as well as sensors/encoders and actuators



If the CPU is supplied 24V via a load power supply (PM), it supplies the system power supply of 12W for the first inserted I/O modules.

- **PS - Power System → System Power Supply**

supplies S7-1500 modules in the central rack via the backplane bus



Each CPU offers a system power supply of 12W for the first inserted I/O modules. Depending on the I/O modules used, further power segments have to be set up, as required.



A system power supply (PS) can also supply the load circuit for 24VDC modules in addition to the CPU.

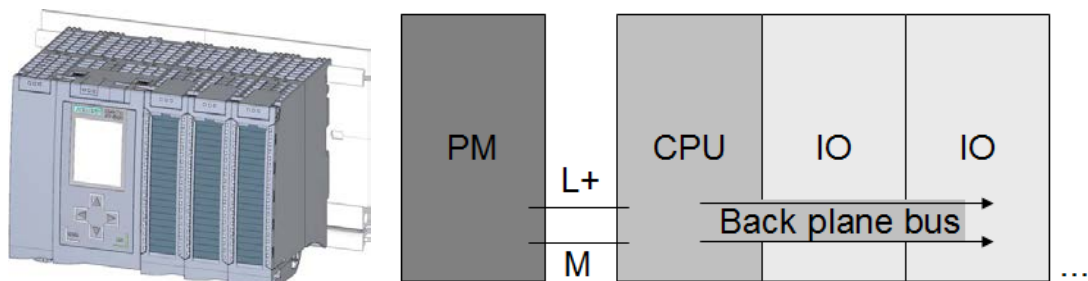
## Power Supply and Power Segments of the I/O Modules

It is necessary to set up power segments in the central rack for larger configurations or configurations with greater I/O module power requirements (as a rule, when using CP, CM, TM).

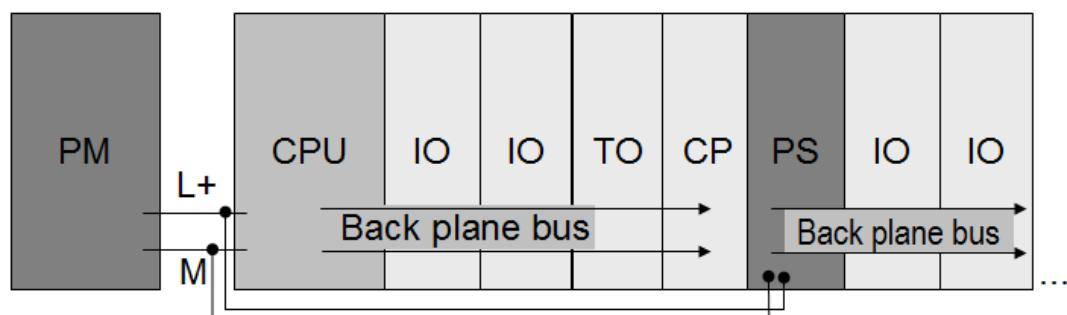
A maximum of 3 power segments can be set up per rack (1xCPU segment plus 2 more).

If the configuration includes additional power segments, additional system power supply modules (PS) are inserted to the right next to the CPU. The CPU continues to control all modules of the rack. Only the system power supply of the I/O modules is subdivided here.

## Example of a Small S7-1500 Configuration



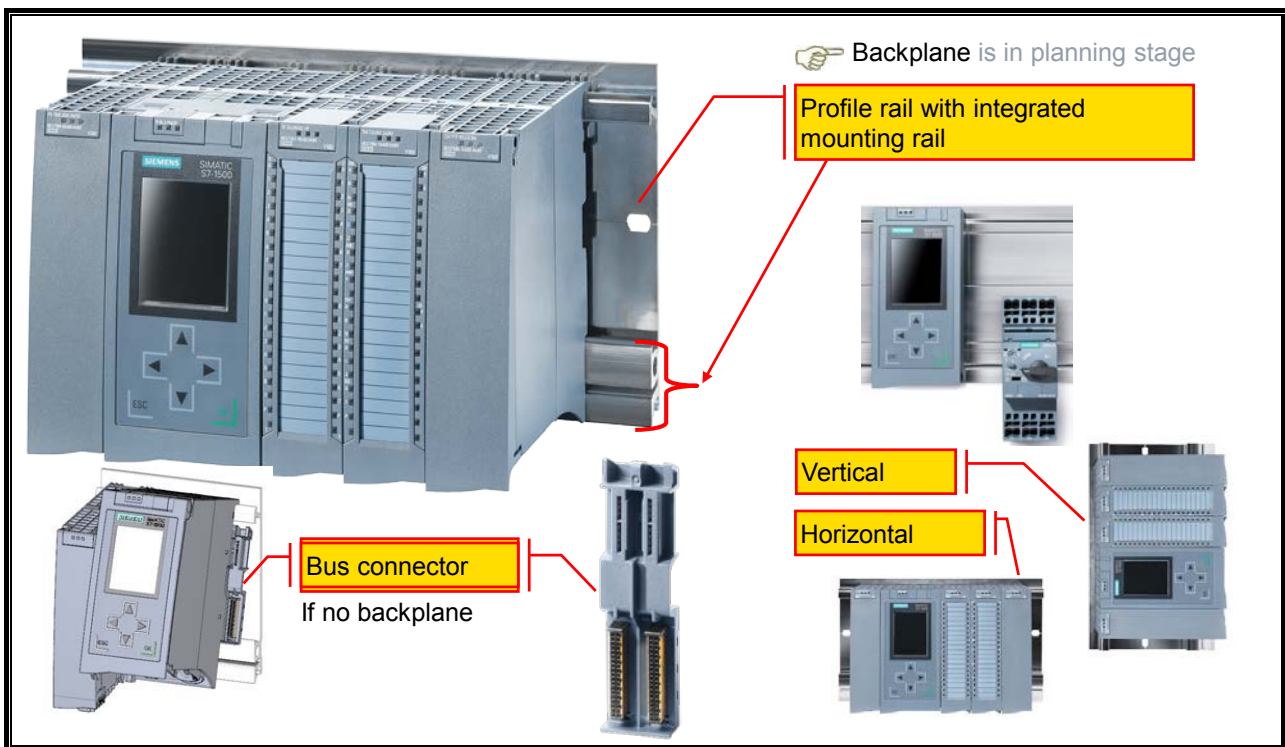
## Example of an S7-1500 Configuration with a 2nd Power Segment



## Interface Modules for Expansion Rack

A central multi-tier assembly is not planned. An expansion can be realized using the distributed ET200MP I/O system.

## 1.5.2. SIMATIC S7-1500: Installation and Mounting Positions



### Installation

The modules are mounted on an S7-1500 profile rail.

### Bus Connector

If the installation is made on the profile rail, the modules are then connected with the U-connector.

The U-connector establishes the mechanical and electrical connection between the modules and is included with every I/O module.

### Mounting Positions

A horizontal or vertical mounting is possible.



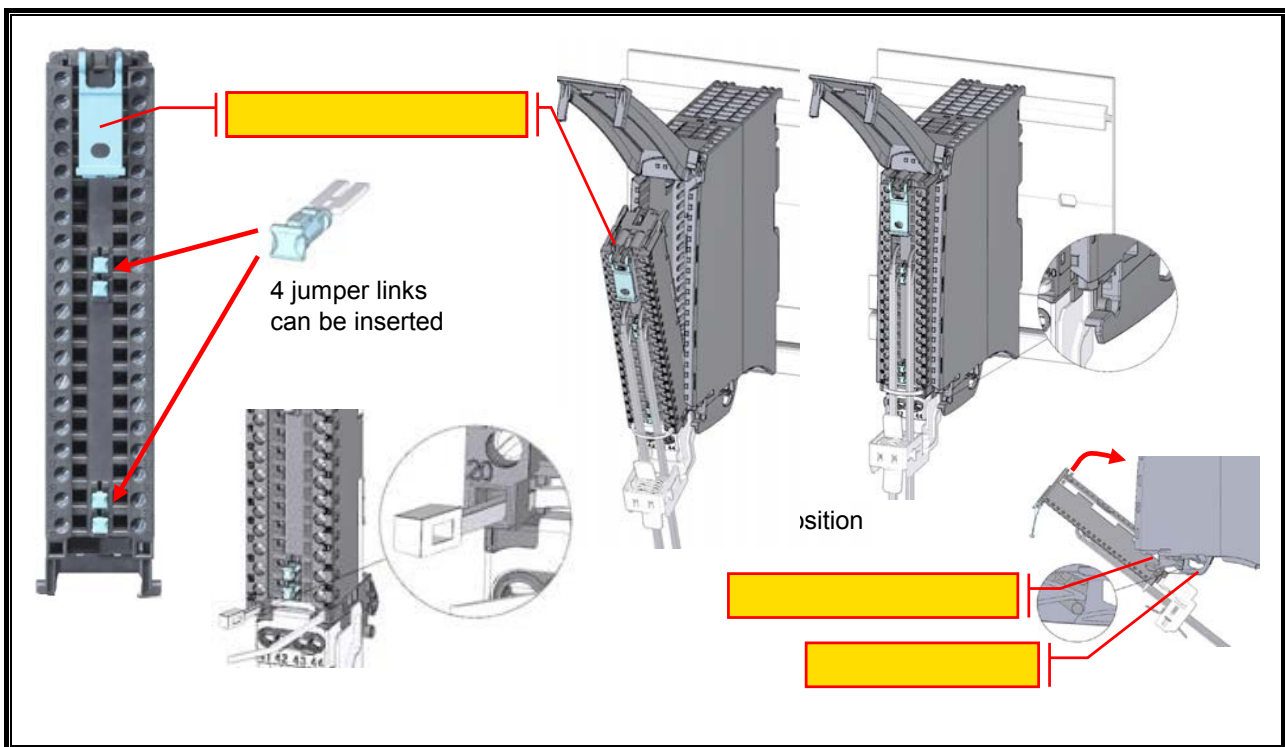
#### Caution!

With a vertical mounting, the maximum allowed ambient temperature is 20 °C lower (ambient temperature 0 to 40 °C).

### Integrated Mounting Rail

For the S7-1500 CPU, there is a profile rail on which components can also be mounted according to EN 60715. With that, even terminals, miniature circuit breakers, small contactors or similar components can be mounted in addition to the S7-1500.

### 1.5.3. SIMATIC S7-1500: Connection Technology/Front Connector



#### Properties of the Front Connector

- In each case 40 terminals
  - Clamping techniques:
    - Screw-type terminal
    - SIMATIC TOP connect
- System wiring for the connection of sensors/encoders and actuators  
 → S7-1500 front connector wired with 20 or 40 single conductors (prefabricated)



#### Prewiring Position

The front connector latches up in the front cover. In this position, the front connector still juts out of the module, but the front connector and the module are not yet electrically connected.

#### Jumper Links

The jumper links can be inserted in the front connector in four places for easier set up of load groups. That is, looping the supply voltage to several potential groups.  
*Only one connection from left to right exists!*

#### Automatic Coding of the I/O Modules

This enables a faster and safer exchange of the front connector.

#### Two Front Door Latching Positions

At the bottom of the front cover there are 2 different latching positions for different space requirements of the conductor bundle.

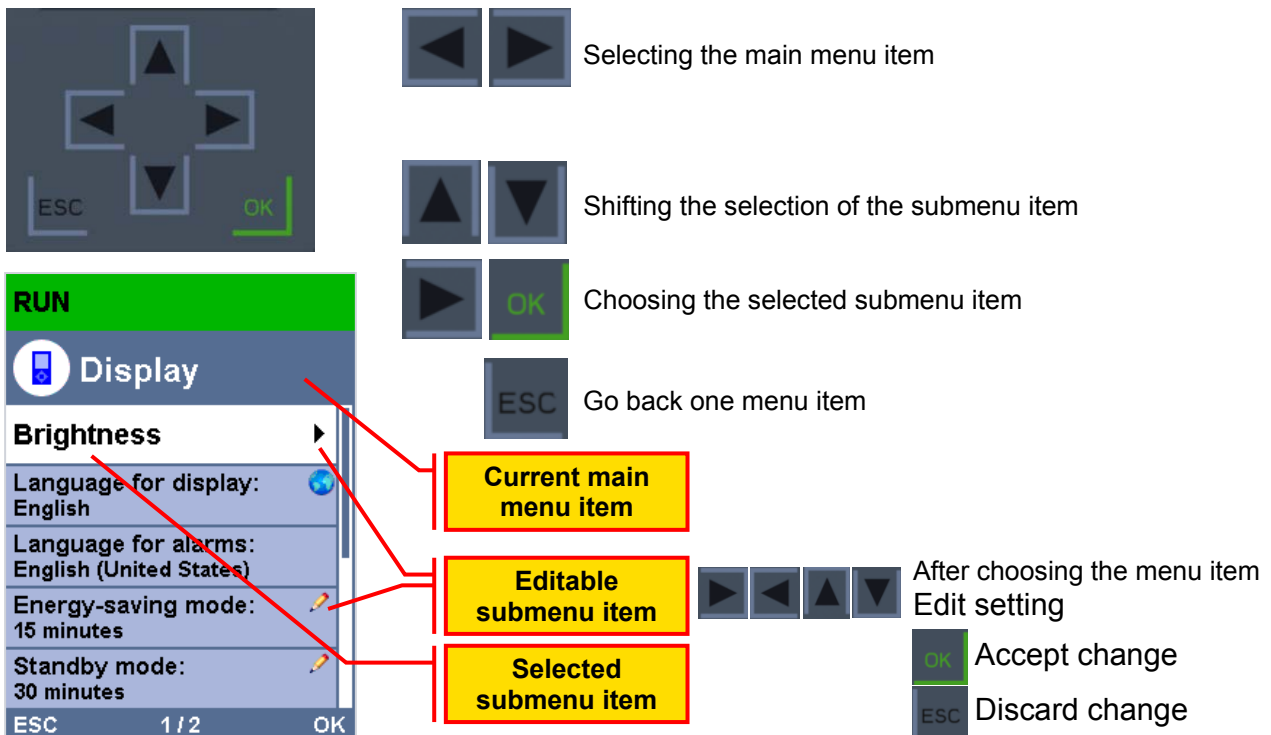
- Cable storage space that grows with the need (AWG cabling)  
 American Wire Gauge (AWG) is an American standard measure for copper wires which defines the wire strength and the allowed damping, whereby a lower AWG value represents a thicker wire. By swinging up the release latch, you can pull the front connector and remove it from the module.

### 1.5.4. SIMATIC S7-1500: CPU-Display → Overview

- Every S7-1500 CPU is delivered with a Display
- Two sizes depending on the CPU type
  - 1.36" up to CPU1513
  - 2.4" as of CPU1515
- Has its own MLFB
  - can be ordered as replacement part
- CPU can be operated without the Display
- Removal and insertion possible during running operation
  - CPU stays in RUN
- Bilingual display (menu and error/message texts)
- Language can be switched during running operation



#### Operation



### 1.5.5. SIMATIC S7-1500: CPU-Display → Menu and Colors

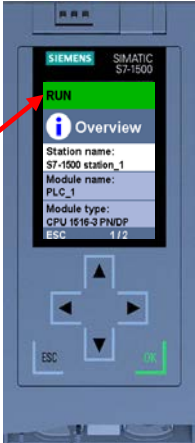


**Main menu items and their meaning:**

-  **Overview** (CPU status: name, type, MLFB, version)
-  **Diagnostics** (display diagnostics buffer, message display)
-  **Settings** (CPU settings: addresses, time, operating mode, CPU Reset, protection level, unlock Display when password set)
-  **Modules** (status information of modules in the system)
-  **Display** (display settings: brightness, display language, energy-saving times, MLFB, version)

**Colors of status information and their meaning:**

- green** RUN of CPU, RUN with pending alarm message
- yellow** STOP or CPU HOLD
- red** Error
- white** Connection setup or connection to CPU lost



Depending on the current CPU state, the color on the Display varies.

Available Display languages are the available user interface languages of STEP7.

#### Additional Symbols in the Status Information



Password is configured but not entered



Password is configured and entered

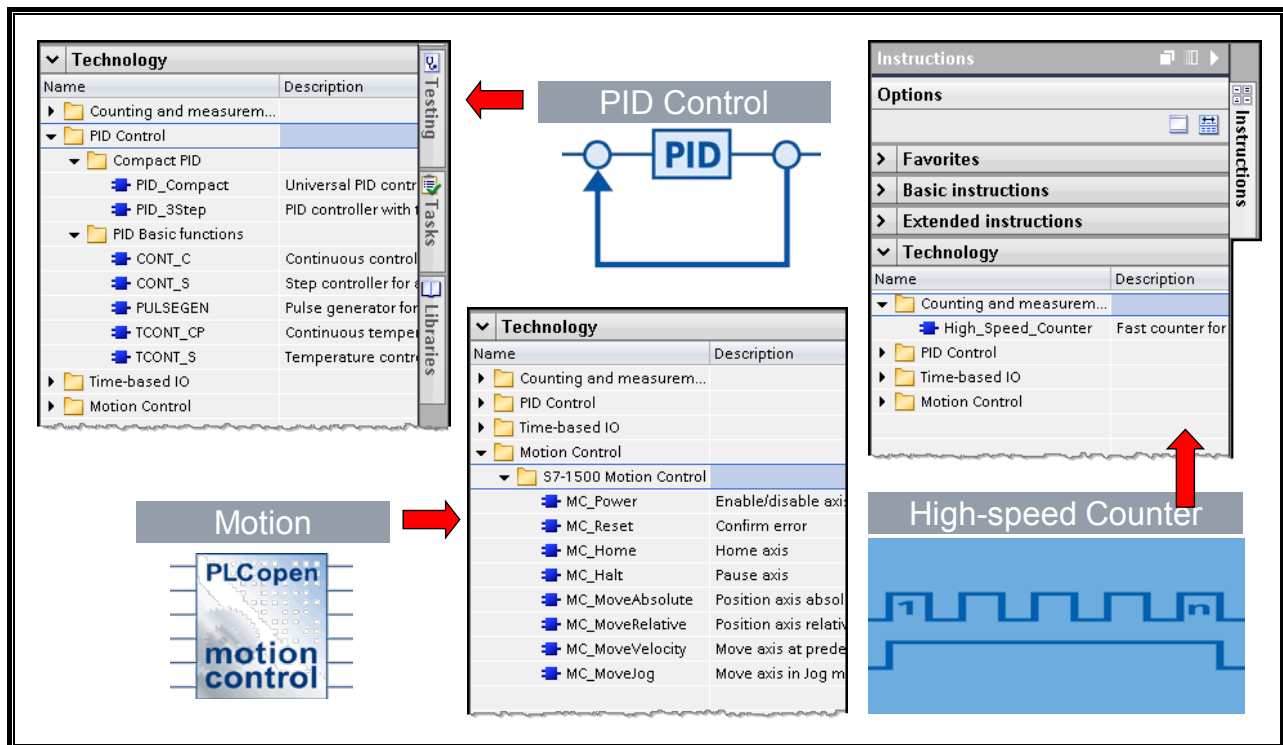


An Alarm exists



A Force job is active on the CPU

## 1.6. SIMATIC S7-1200/1500: Technology Functions



### High-speed Counters

Up to 6 high-speed counters, 3 at 100 kHz and 3 at 30 kHz are available for precise monitoring of incremental encoders, frequency counting or counting of highly frequent process events.

### Motion

To control rotary speed, position or pulse duty factor, a total of 2 PWM outputs (pulse width modulation) are available. Application examples are, for example, controlling the speed of a motor, position of a valve, or the mark-to-space ratio for a heating element. For rotary speed and position controls, a total of 2 PTO outputs at 100 kHz (pulse train output) are available.

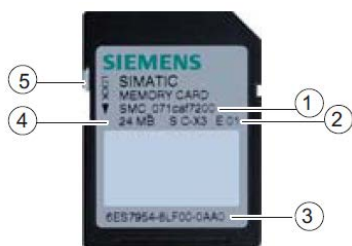
It supplies a pulse train for controlling speed and position of stepper or servo motors.

### PID

For simple closed-loop control tasks, up to 16 PID control circuits with automatic PID adjustment and tuning control panel are available.



## 1.7. SIMATIC S7-1200/1500: Memory Card



- ① Serial number of the SD card
- ② Product version
- ③ Order number
- ④ Card size
- ⑤ Slide switch for write-protect  
(must not be write-protected)

### SIMATIC Memory Card in the S7-1200 / S7-1500:

- Load memory
- Distribution of programs (only S7-1200)
- Firmware update
- Documentation
- Memory Card Binding
- Unlinked DBs
- Archiving of data
- Module exchange without PG

### Write with:

- Commercially available SD card reader
- Field PG

### Memory Card Binding – Copy Protection

The executability of the program can be bound to the serial number of the card.

### Load Memory

- S7-1500  
This has no integrated load memory and therefore it is imperative that a card is inserted.
- S7-1200  
This has an integrated load memory. Here, an inserted memory card can replace (expand) the integrated load memory or the card can be used for program updates (distribution of programs).

### Distribution of Programs ← only S7-1200

The use as Transfer card (card mode = "Transfer") is only supported by the S7-1200. Here, a program can be downloaded into the CPU without a PG if a card is inserted.

### Archiving of Data ← only S7-1500

It is possible to archive process values on the card.



The use of this functionality influences the operating life of the Memory Card



## 1.8. Distributed I/O Systems

|           |                                                                                   |           |                                                                                   |                                                                                                                                                                                                      |
|-----------|-----------------------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ET 200M   |  | ET 200MP  |  | <b>Distributed Installation</b><br>With modules of the central installation<br>- Front connector<br>- Single-wiring technology<br>- Optimized width                                                  |
| ET 200S   |  | ET 200SP  |  | <b>Distributed Installation</b><br>- Fixed wiring and multi-wiring technology<br>- Tool-free, fast connection technology<br>- Optimized size<br><br>One distributed I/O for various market segments. |
| ET 200iSP |  | ET 200iSP | To be continued                                                                   | <b>Distributed Installation</b><br>- 24 VDC and 230 VAC power supply<br><br>Functionality for process automation<br>Installation in applications for <b>Ex-Zone 1</b>                                |

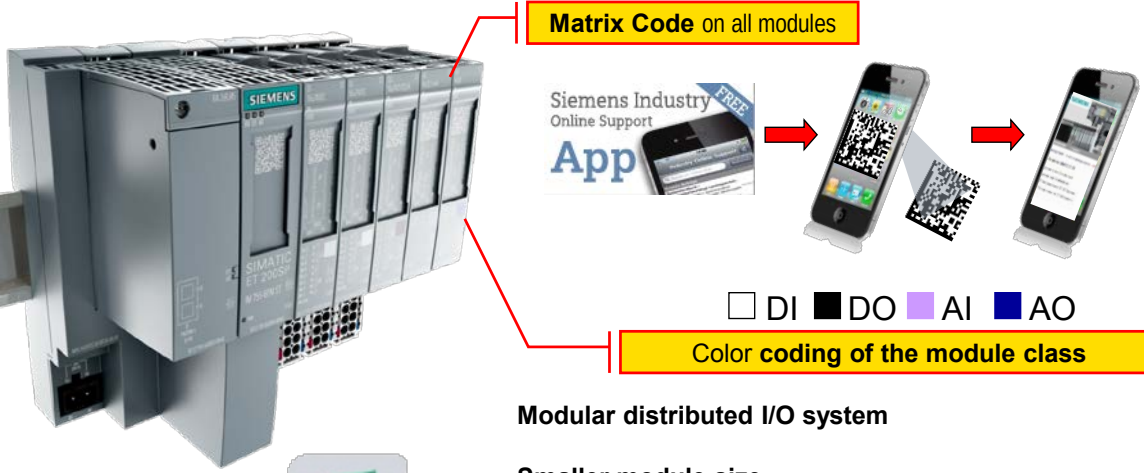
### 1.8.1. Overview: Distributed Signal Modules

|           | ET200SP | ET200MP | ET200S | ET200M | ET200pro |
|-----------|---------|---------|--------|--------|----------|
| DI/DQ     | ✓       | ✓       | ✓      | ✓      | ✓        |
| AI/AQ     | ✓       | ✓       | ✓      | ✓      | ✓        |
| F-DI/F-DQ | ✓       |         | ✓      | ✓      | ✓        |
| F-AI/F-AQ | ✗       |         | ✗      | ✓      | ✗        |

The ET200SP and ET200MP product range will be expanded in the next years such that these two product lines completely cover the applications of the ET200S and ET200M.

The ET200pro (interface modules for connection to PROFINET or PROFIBUS) in the degree of protection IP65/67 for use directly at the machine will continue to be offered.

## 1.8.2. ET 200SP Distributed I/O System



**Matrix Code on all modules**

Siemens Industry Online Support App

□ DI ■ DO ■ AI ■ AO

**Color coding of the module class**

**Modular distributed I/O system**

**Smaller module size**

- system-integrated power module
- high channel density (16-channel I/O modules)
- 50% smaller than ET 200S

**Comprehensive performance spectrum**

- stronger performance HS (high speed) modules
- PROFenergy as integrated function

PROFI<sup>®</sup> INDUSTRIAL ETHERNET  
NET

PROFI<sup>®</sup> INDUSTRIAL FIELD BUS  
BUS

PROFenergy

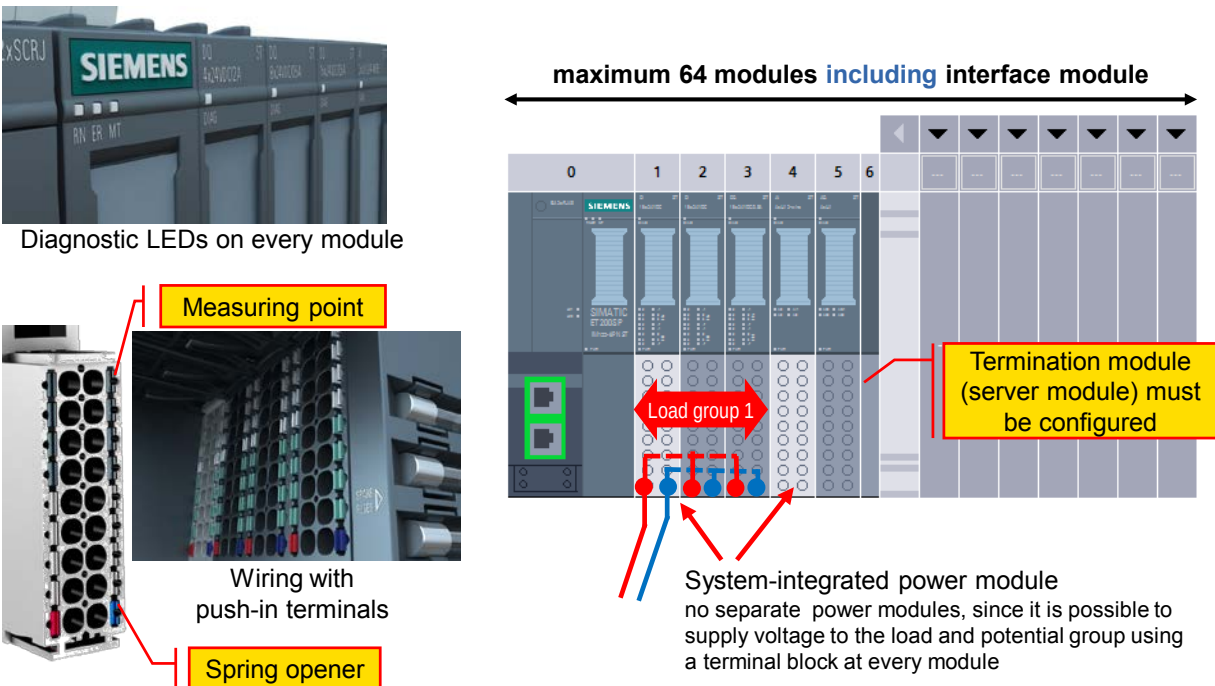
I am PROFI<sup>®</sup> PROFIsafe

### 2d Matrix Code (Data Matrix)

This code is used for identifying modules and can be photographed or decoded by SmartPhones, PDAs, iPhones and the like.

With the ET200SP modules, the code contains an Internet link to the product page of the associated module.

#### 1.8.2.1. ET 200SP: Configuration and Maximum Number of Modules



**Diagnostic LEDs on every module**

**Measuring point**

**Wiring with push-in terminals**

**Spring opener**

**maximum 64 modules including interface module**

**Load group 1**

**System-integrated power module**  
no separate power modules, since it is possible to supply voltage to the load and potential group using a terminal block at every module

**Termination module (server module) must be configured**

### 1.8.3. ET 200MP Distributed I/O System



**Modular distributed I/O system**

**S7-1500 modules can be used**

- digital/analog modules, TM, CP/CM, system power supplies (PS)
- high channel density  
e.g. 32 DI or 32 DO per module

**Comprehensive performance spectrum**

- low channel costs due to high channel density
- fast response times
- module diagnostics



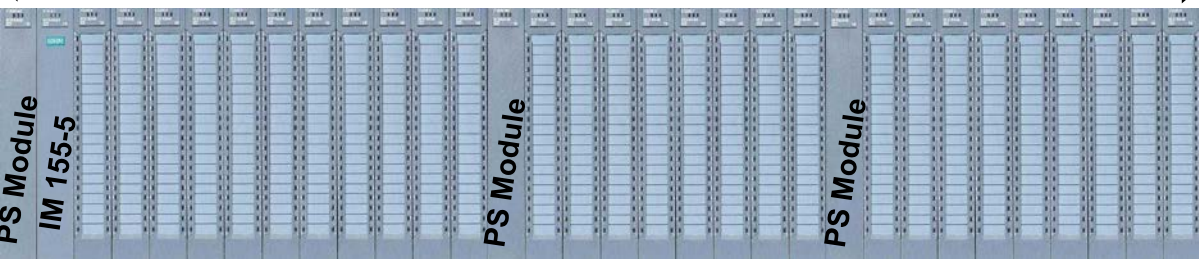



The ET200MP enables the distributed connection of S7-1500 series central I/O modules. Connection is made using an interface module.

#### 1.8.3.1. ET 200MP: Configuration and Maximum Number of Modules

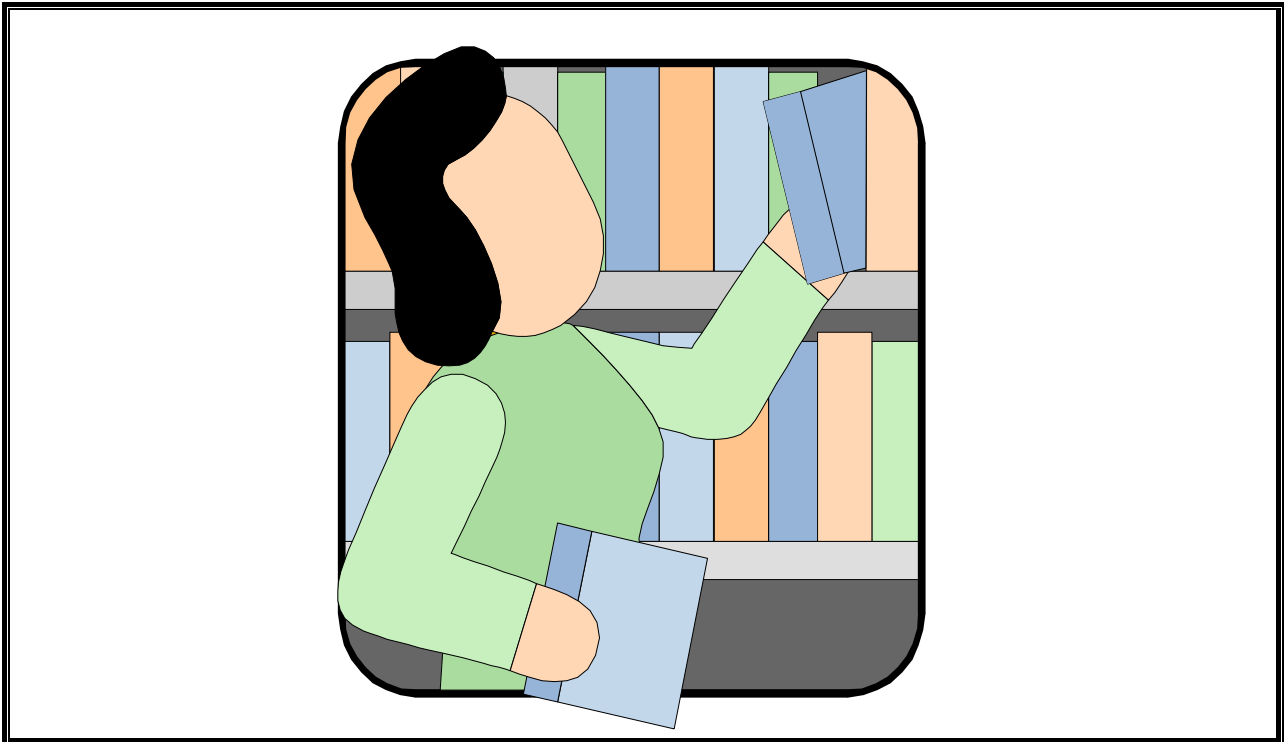
- Maximum of 32 modules
  - 1st. module = system power supply (PS)
  - 2nd. module = interface modules
  - 3rd.-32nd. module = max. of any 30 I/O modules of the S7-1500
- Formation of load groups through additional voltage supplies (similar to centralized configuration of S7-1500)

**Maximum 32 modules including power modules and interface module**

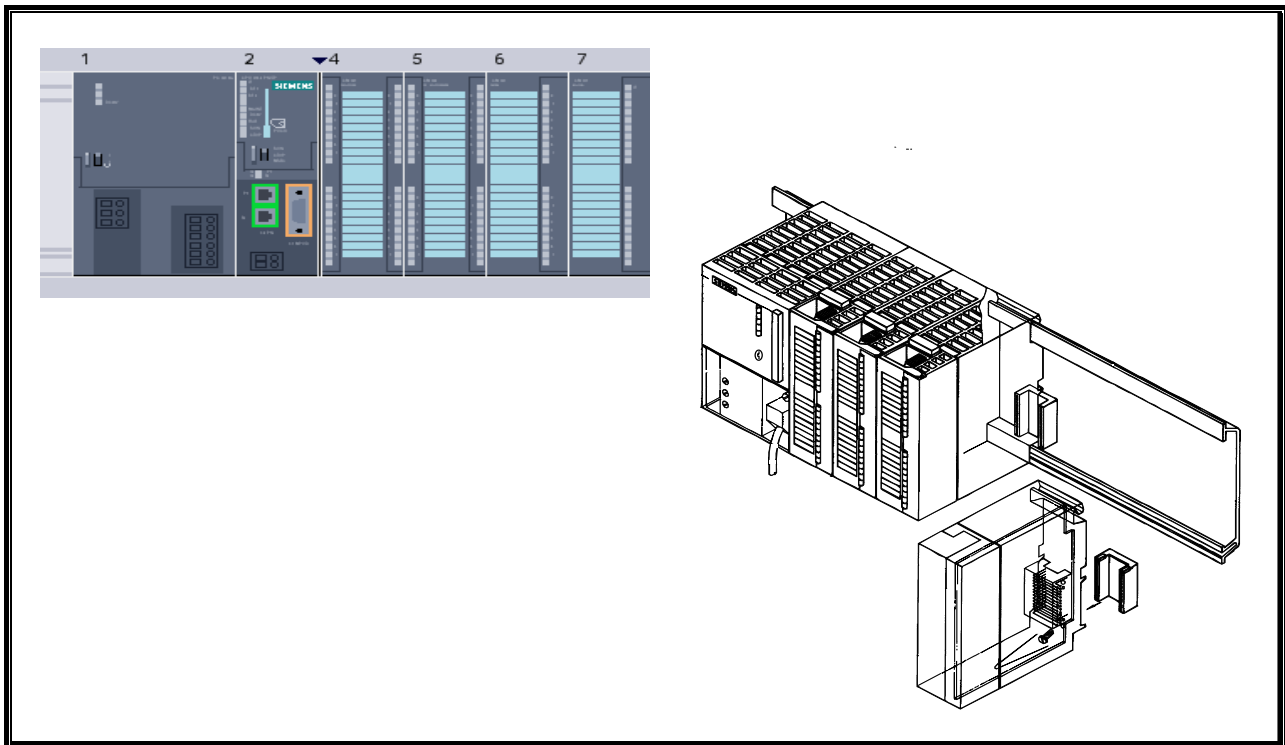


(optional)

## 1.9. Additional Information



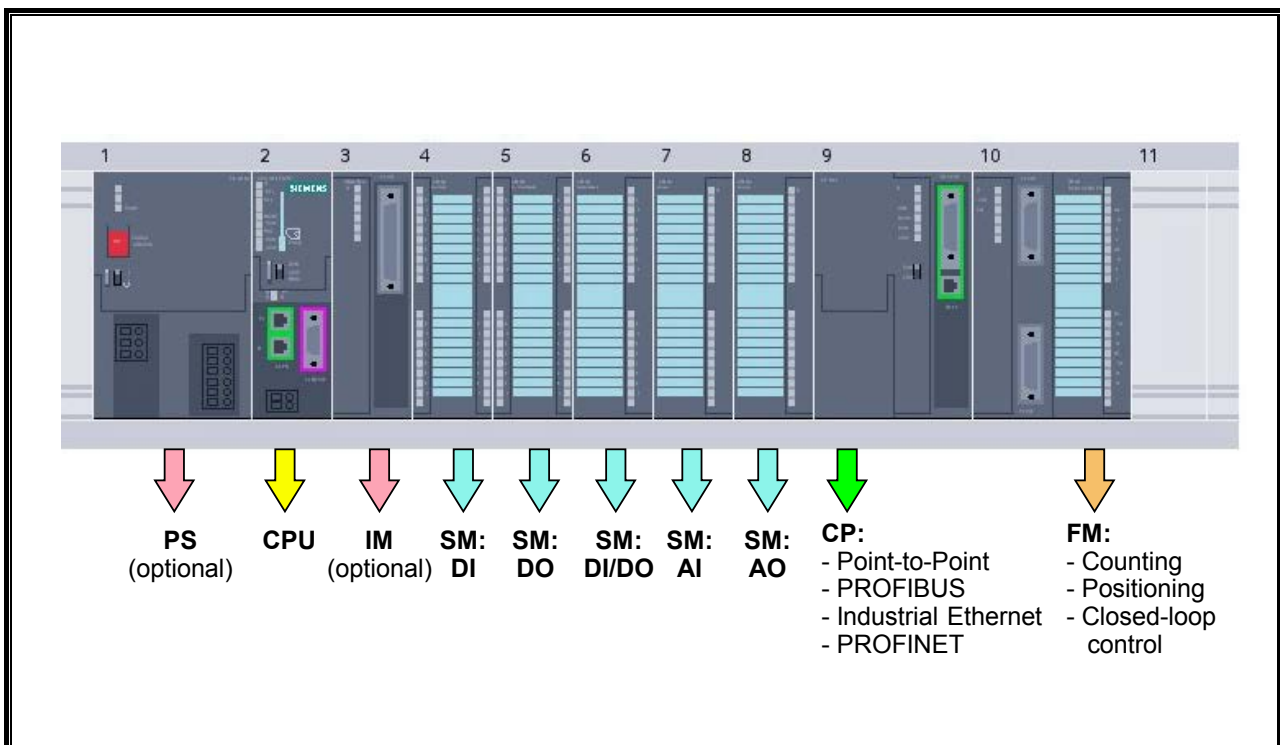
## 1.10. SIMATIC S7-300: Modular Automation System



### Features

- Modular compact control system for the lower and middle performance range
- Scaled CPU range
- Extensive range of modules
- Can be expanded to up to 32 modules
- Backplane bus integrated in the modules
- Can be networked with
  - Multipoint Interface (MPI),
  - PROFIBUS or
  - Industrial Ethernet or
  - PROFINET
- Central PG/PC connection with access to all modules
- No slot rules for I/O modules,
- Configuration and parameter setting with the help of the "Hardware Configuration" tool.

### 1.10.1. SIMATIC S7-300: Modules



#### Signal Modules (SM)

- Digital input modules: 24VDC, 120/230V AC
- Digital output modules: 24VDC, Relay
- Analog input modules: Voltage, Current, Resistance, Thermocouple
- Analog output modules: Voltage, Current

#### Interface Modules (IM)

The IM360/IM361 and IM365 make multi-tier configurations possible. The interface modules loop the bus from one tier to the next.

#### Dummy Modules (DM)

The DM 370 dummy module reserves a slot for a signal module whose parameters have not yet been assigned. A dummy module can also be used to reserve a slot for installation of an interface module at a later date.

#### Function Modules (FM)

- Counting
- Positioning
- Closed-loop control.

#### Communication Processors (CP)

- Point-to-Point connections
- PROFIBUS
- Industrial Ethernet
- PROFINET.

## 1.11. SIMATIC S7-400: Modular Automation System

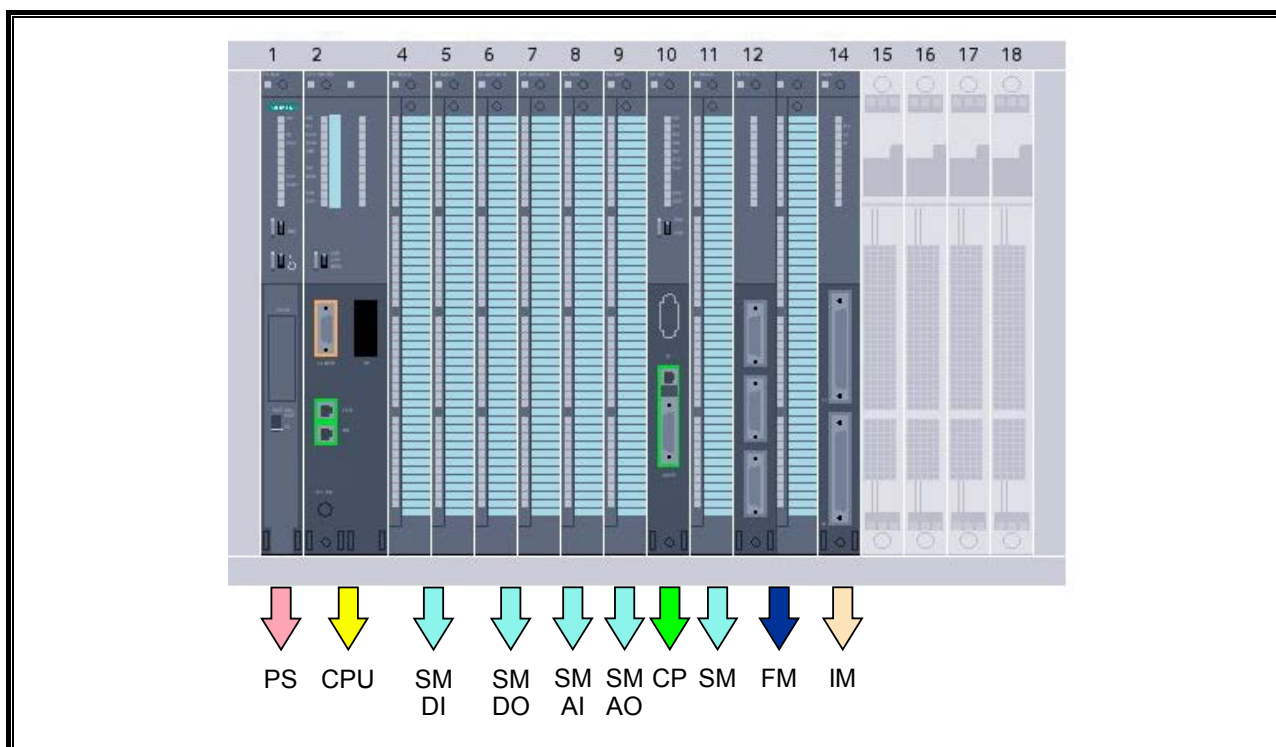


### Features

- The power PLC for the mid to upper performance range,
- Scaled CPU range
- Extensive range of modules
- Can be expanded to over 300 modules,
- Backplane bus integrated in the mounting rack
- Can be networked with
  - Multipoint interface (MPI),
  - PROFIBUS or
  - Industrial Ethernet or
  - PROFINET
- Central PG/PC connection with access to all modules,
- Only a few slot rules,
- Configuration and parameter setting with the help of the "Hardware Configuration" tool
- Multicomputing (up to four CPUs can be used in the central rack).



### 1.11.1. SIMATIC S7-400: Modules



#### Signal Modules (SM)

- Digital input modules: 24VDC, 120/230VAC
- Digital output modules: 24VDC, Relay
- Analog input modules: Voltage, Current, Resistance, Thermocouple
- Analog output modules: Voltage, Current.

#### Interface Modules (IM)

The IM460, IM461, IM463, IM467 interface modules provide the connection between various racks:

- UR1 (Universal Rack) with up to 18 modules
- UR2 (Universal Rack) with up to 9 modules
- ER1 (Expansion Rack) with up to 18 modules
- ER2 (Expansion Rack) with up to 9 modules.

#### Function Modules (FM)

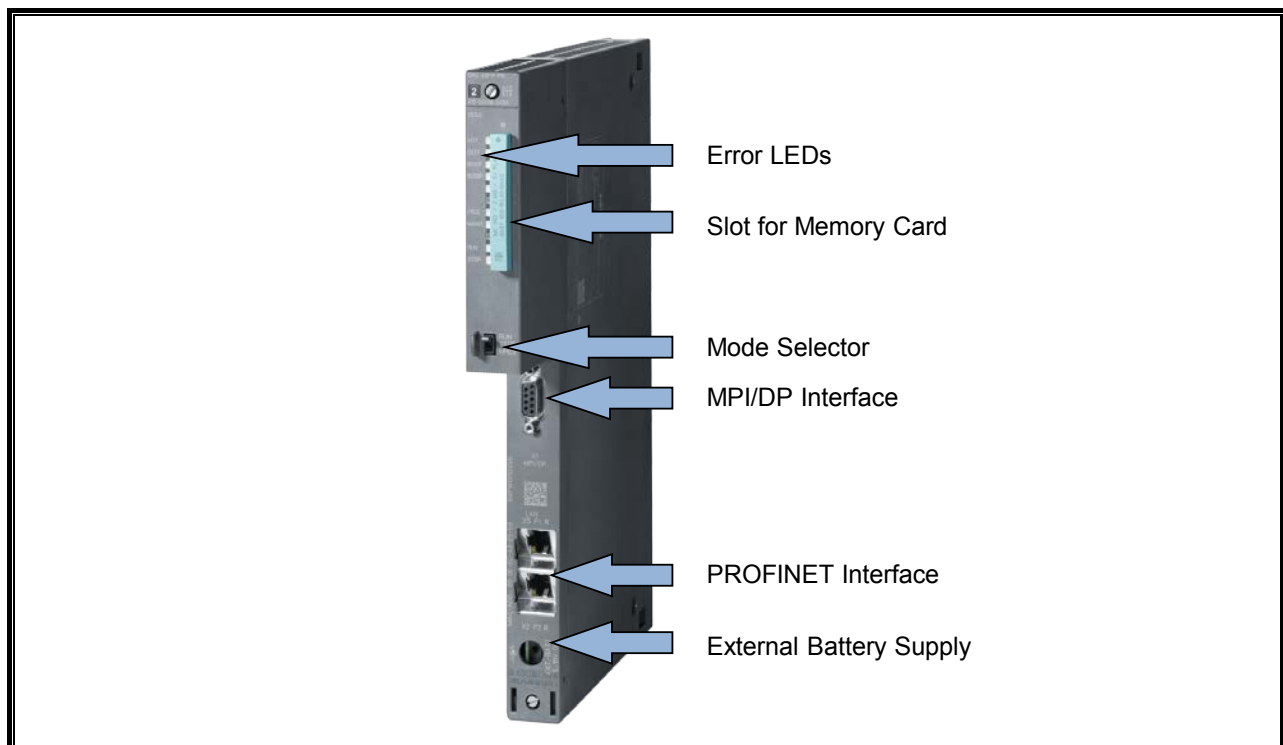
- Counting
- Positioning
- Closed-loop control.

#### Communication Processors (CP)

- Point-to-Point connections
- PROFIBUS
- Industrial Ethernet
- PROFINET.



### 1.11.2. SIMATIC S7-400: CPU Design



#### Error LEDs

LEDs for displaying the status and errors (internal and external errors) of the CPU

#### Memory Cards

With the S7-400 CPUs you can, depending on your requirements, insert RAM or flash EPROM cards as external load memory:

- RAM cards are available with a capacity of: 64KByte, 256KByte, 1MByte, 2MByte. The CPU battery backs-up the contents.
- Flash EPROM cards are available with a capacity of: 64KByte, 256KByte, 1MByte, 2MByte, 4MByte, 8MByte, 16MByte. The contents are backed-up on the integrated EEPROMs.

#### Mode Selector

- **MES** = Memory reset function (**Module RESet**)
- **STOP** = STOP mode, that is, no program execution and I/O are disabled ("OD" mode = **O**utput **D**isabled).
- **RUN** = Program is executed, read-only access possible from PG.
- **RUN-P** = Program is executed, read/write access possible from PG.

#### MPI / DP Interface

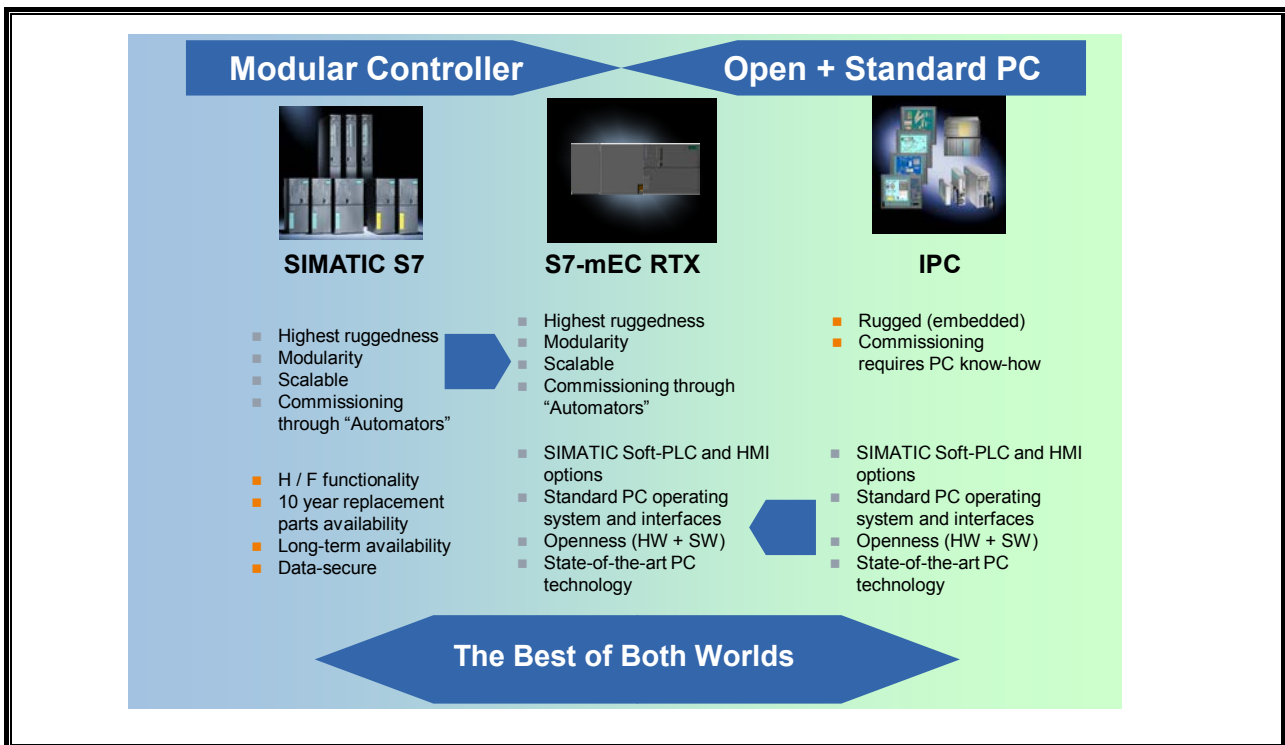
Establishing the online connection to the programming device (can be parameterized in the Hardware Configuration)

- connecting to distributed I/Os (DP)
- data exchange with other stations (S7 Communication)

#### EXT - BATT

An additional external battery socket for a 5 VDC to 15 VDC source to back-up the RAM when the power supply is being replaced.

## 1.12. SIMATIC Embedded Automation: Overview

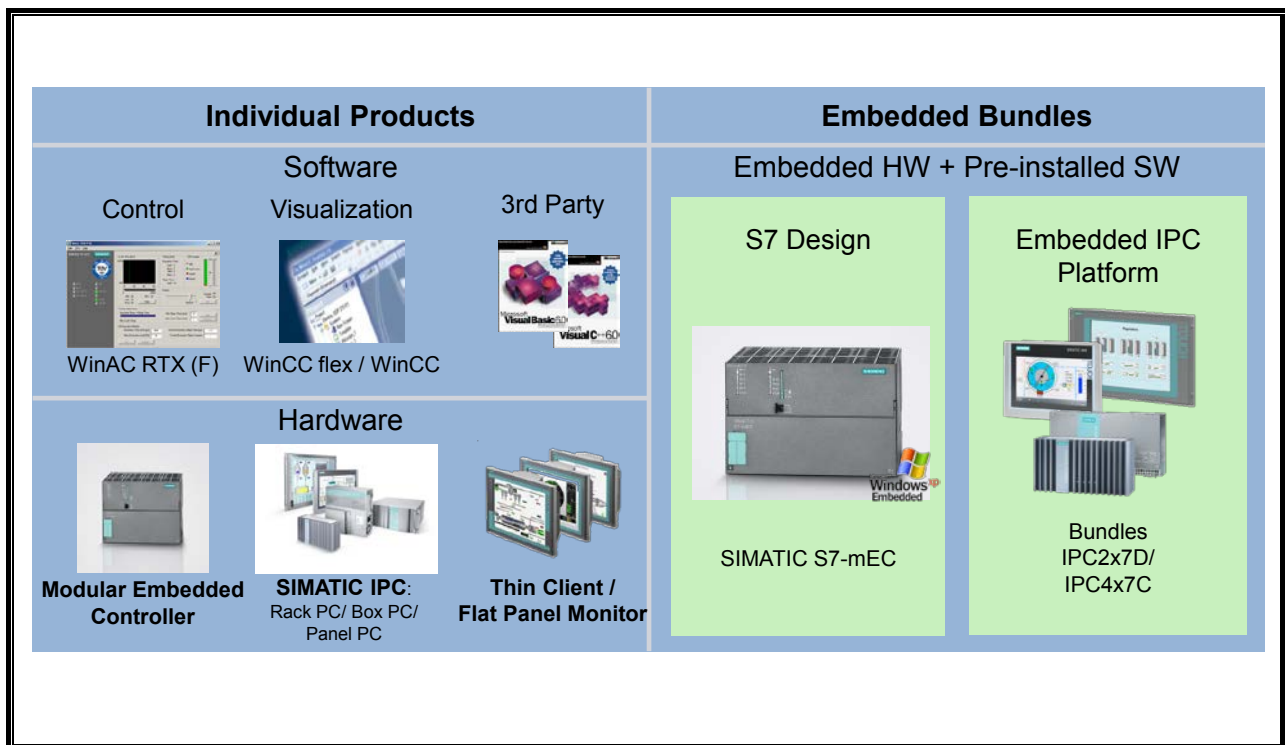


### PC-based Control

SIMATIC Embedded Automation is the implementation of various automation tasks (PLC programming, visualization, data processing, HMI services) on one common PC and software platform. Today, three fundamental directions can be derived from this:

- Modular Controllers: S7-1200, S7-300 and S7-400 controllers with their different versions and performance characteristics
- Embedded Controllers such as SIMATIC S7-mEC and SIMATIC Microbox
- SIMATIC Panel PCs with Win AC MP and Industrial PCs

### 1.12.1. SIMATIC PC-based Automation Product Overview



#### Product Overview

The picture shows an overview of the most important components that are available for PC-based Automation.

# Contents

# 2

|           |                                                                                |            |
|-----------|--------------------------------------------------------------------------------|------------|
| <b>2.</b> | <b>Engineering Software TIA Portal.....</b>                                    | <b>2-2</b> |
| 2.1.      | TIA Portal - Central Engineering Framework.....                                | 2-3        |
| 2.2.      | Scope of the Products.....                                                     | 2-4        |
| 2.3.      | STEP 7 Range of Products.....                                                  | 2-5        |
| 2.3.1.    | STEP 7 Licensing .....                                                         | 2-5        |
| 2.4.      | WinCC Range of Products.....                                                   | 2-6        |
| 2.4.1.    | WinCC Licensing .....                                                          | 2-6        |
| 2.5.      | Startdrive Range of Products and Licensing .....                               | 2-7        |
| 2.6.      | Side-by-Side Installation .....                                                | 2-8        |
| 2.7.      | Operating Systems for PC/PGs .....                                             | 2-9        |
| 2.7.1.    | Virtualization (Released Software).....                                        | 2-9        |
| 2.8.      | License Upgrade Strategy .....                                                 | 2-10       |
| 2.9.      | TIA Portal: Portal View and Project View.....                                  | 2-11       |
| 2.9.1.    | Portal View .....                                                              | 2-12       |
| 2.9.2.    | Project View .....                                                             | 2-13       |
| 2.9.3.    | TIA Portal - Settings: User Interface Language .....                           | 2-14       |
| 2.9.4.    | TIA Portal - Settings: Language, Storage Location, Layout .....                | 2-15       |
| 2.9.5.    | Window Arrangement in the Project View .....                                   | 2-16       |
| 2.9.6.    | Window Layouts Save / Manage / Use .....                                       | 2-17       |
| 2.9.7.    | Window Sections in the Working Area.....                                       | 2-18       |
| 2.9.8.    | Keeping the Editor Window in the Foreground (when Working Area is Split) ..... | 2-19       |
| 2.9.9.    | Project Tree.....                                                              | 2-20       |
| 2.9.10.   | Task Cards.....                                                                | 2-21       |
| 2.9.11.   | Inspector Window .....                                                         | 2-22       |
| 2.9.12.   | Undo and Redo .....                                                            | 2-23       |
| 2.9.13.   | Saving a Project.....                                                          | 2-24       |
| 2.9.14.   | Archiving / Retrieving a Project.....                                          | 2-25       |
| 2.10.     | Libraries .....                                                                | 2-26       |
| 2.11.     | Project Migration .....                                                        | 2-27       |
| 2.11.1.   | Project Upgrade and Compatibility Mode .....                                   | 2-28       |
| 2.12.     | Help Functions .....                                                           | 2-29       |
| 2.13.     | Additional Information .....                                                   | 2-30       |
| 2.13.1.   | Keyboard Shortcuts of the TIA Portal .....                                     | 2-31       |
| 2.13.2.   | Installation with Record Function in the Setup.....                            | 2-31       |
| 2.13.3.   | Team Engineering.....                                                          | 2-32       |
| 2.13.4.   | Inter Project Engineering .....                                                | 2-33       |
| 2.13.5.   | Update Tool.....                                                               | 2-34       |

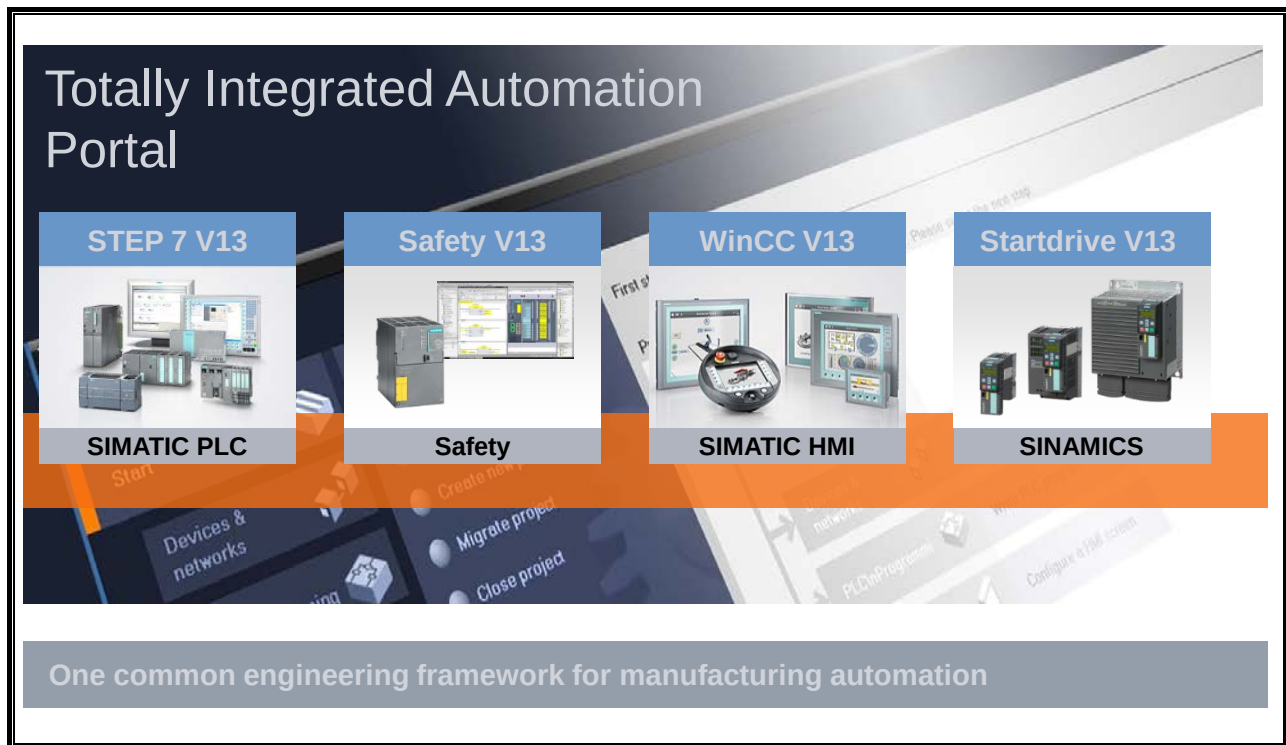
## 2. Engineering Software TIA Portal

**At the end of the chapter the participant will ...**



- ... have an overview of the scope of the engineering framework
- ... become familiar with the engineering products and their range of products
- ... become familiar with the operator interface of the framework

## 2.1. TIA Portal - Central Engineering Framework



Stand-alone software packages are limited because they lack consistency and integration.

It takes a common working environment - that is, an **engineering framework** - to achieve full integration and consistency of individual products.

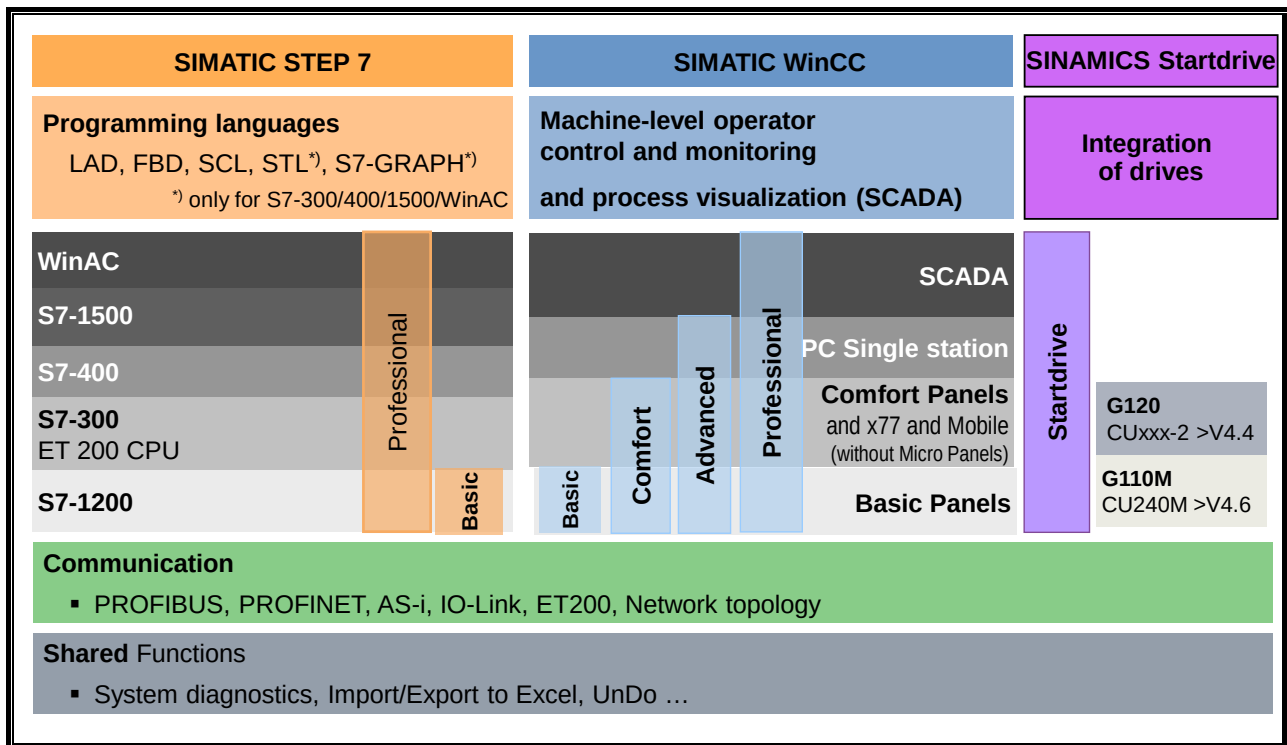
### Advantages of a Central Engineering Framework

- Uniform operator control concept for all automation tasks with common services (for example configuration, communication, diagnostics)
- Automatic data and project consistency
- Powerful libraries covering all automation objects

### The Most Important Engineering Products are:

- SIMATIC STEP 7  
for PLC programming
- SIMATIC Safety  
for programming fail-safe PLCs
- SIMATIC HMI  
for configuring process visualization
- Startdrive  
for parameterizing drives

## 2.2. Scope of the Products



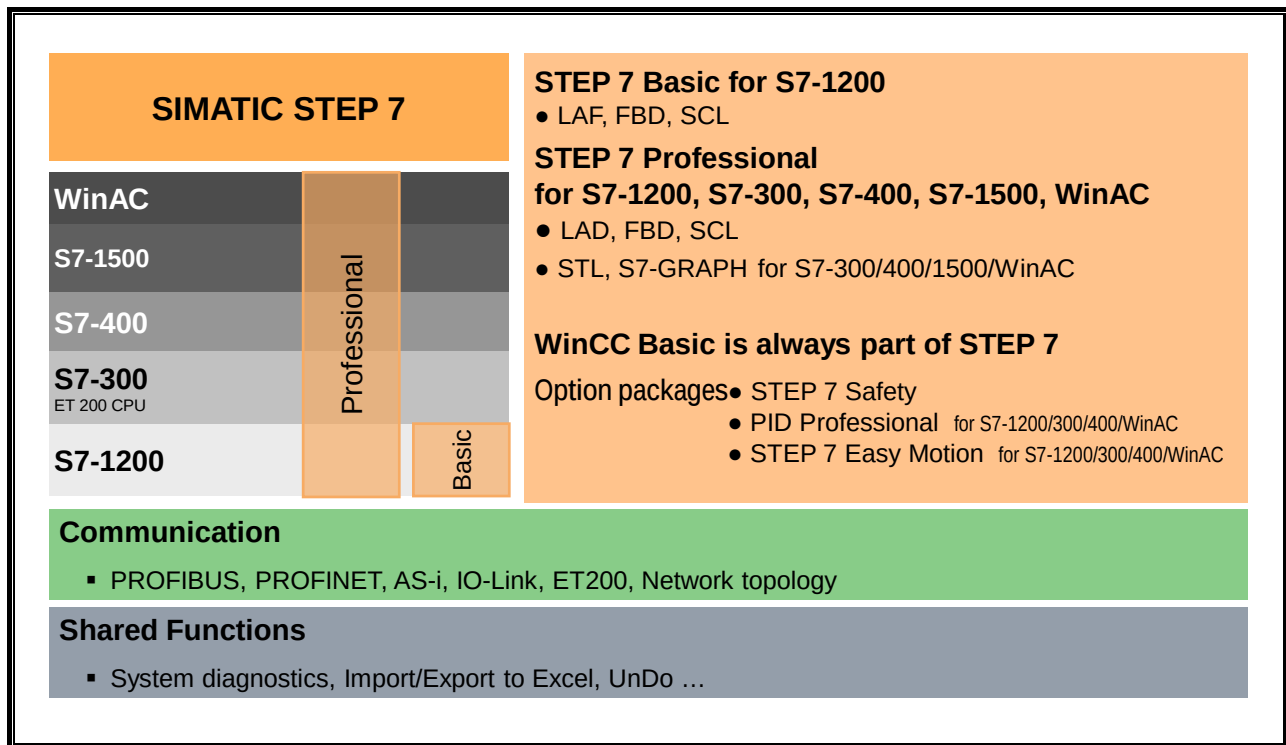
### TIA Portal

The Totally Integrated Automation Portal constitutes the working environment for an integrated engineering with SIMATIC STEP 7 V13 and SIMATIC WinCC V13.

- central engineering framework
- automatic data and project consistency
- uniform operating concept for all automation tasks
- extensive libraries for all automation objects



## 2.3. STEP 7 Range of Products

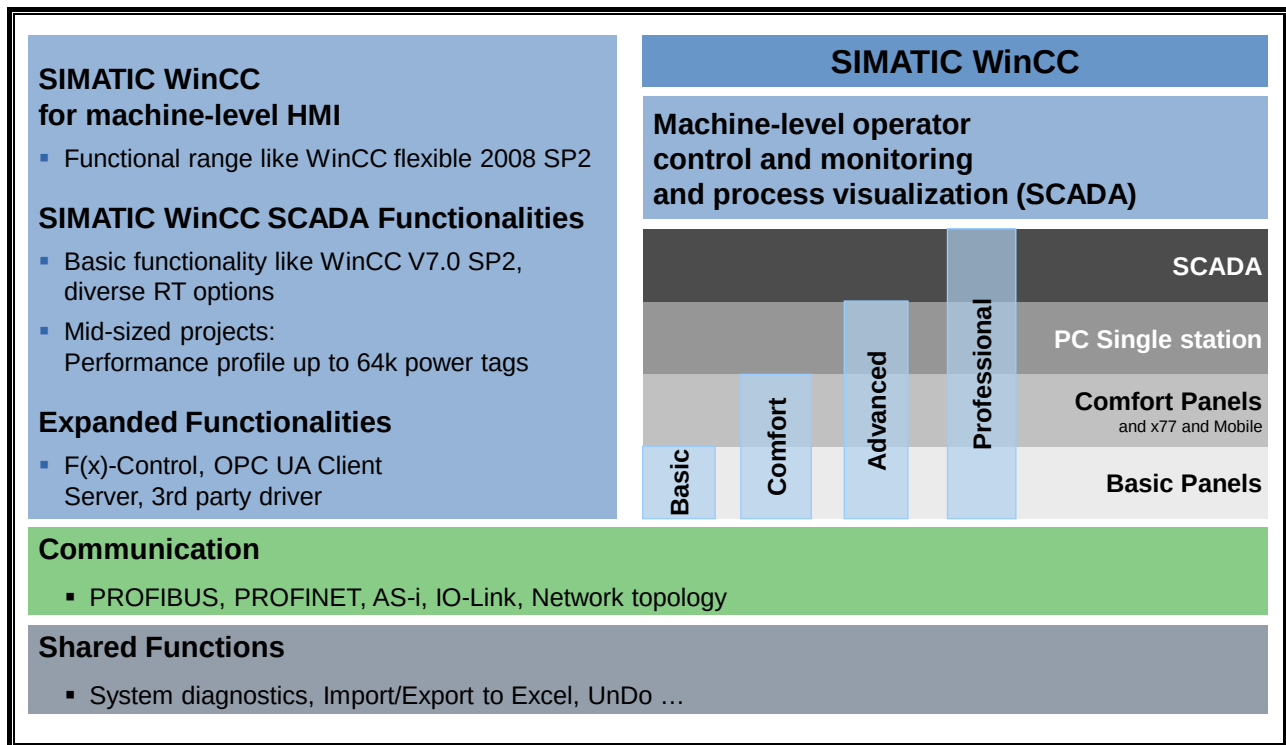


### 2.3.1. STEP 7 Licensing

|                     | Engineering System (ES)                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                        |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                     | S7-1200<br>(LAD, FBD, SCL)                                                                                                                                                                                                                                                                                          | S7-300 / S7-400 / S7-1500 / WinAC<br>(LAD, FBD, STL, SCL, S7-GGRAPH, PLCSIM)                                                                                                                                           |
| STEP 7 Basic        |  <b>Single license</b>                                                                                                                        | ---                                                                                                                                                                                                                    |
| STEP 7 Professional |  <b>Floating license</b>                                                                                                                      |                                                                                                                                                                                                                        |
|                     |  <b>Upgrade license</b>  +  generates <b>Combo license</b> | generates <b>Combo license</b><br>→ STEP 7 V5.4 (and higher)<br>and<br>→ STEP7 V13 Professional<br> parallel installation possible |

To program the controllers, only one license (engineering license) is required in the engineering. The program does not require any further licenses on the CPU (runtime license).

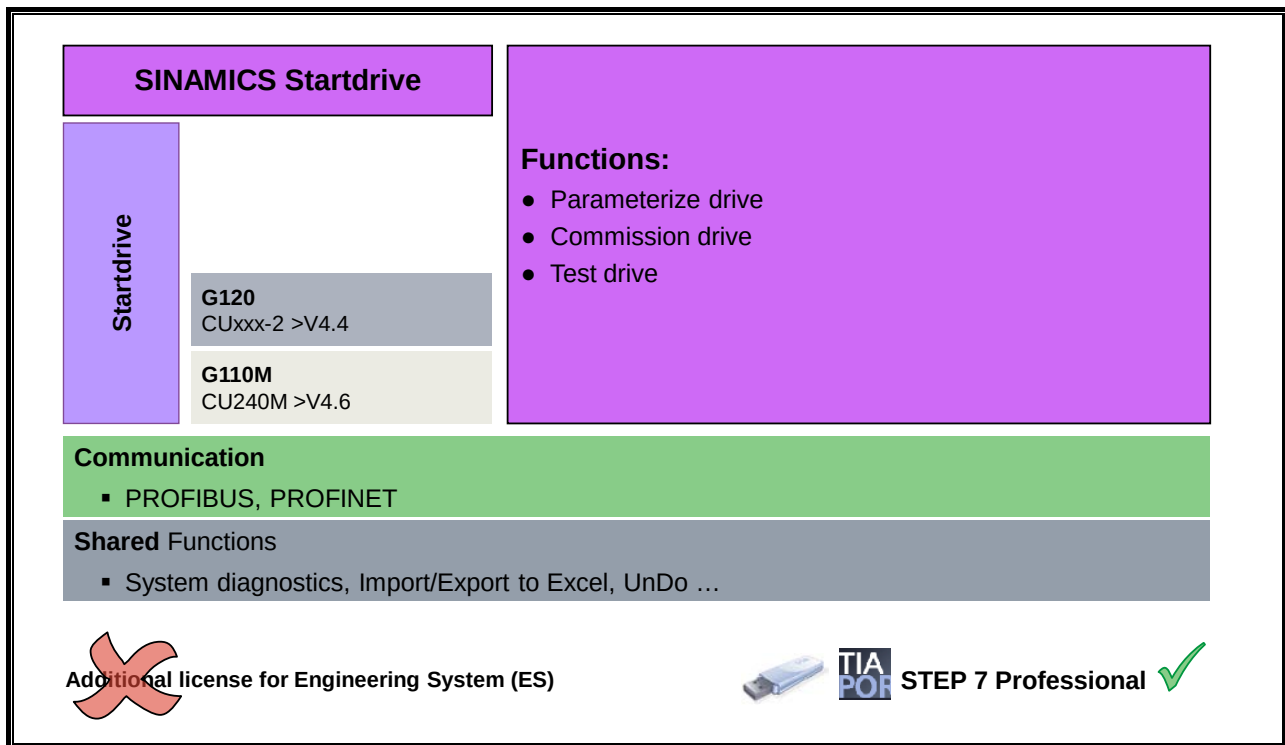
## 2.4. WinCC Range of Products



### 2.4.1. WinCC Licensing

|                    | Engineering System (ES)                                                                                                                                                                              | Runtime (RT)                                                                                                  |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| WinCC Basic        | --- (Component of STEP7)<br>Floating license                                                                                                                                                         | --- (Component of the Panels)                                                                                 |
| WinCC Comfort      | Floating license<br>Upgrade license  +<br>generates <b>Combo license</b><br>→ WinCC flexible 2008 Standard<br>and → WinCC Comfort<br>parallel installation possible                                  | --- (Component of the Panels)                                                                                 |
| WinCC Advanced     | Floating license<br>Upgrade license  +<br>generates <b>Combo license</b><br>→ WinCC flexible 2008 Advanced<br>and → WinCC Advanced<br>parallel installation possible                                 | Single license<br>Upgrade license  →                                                                          |
| WinCC Professional | Floating license<br>Upgrade license not available, since another license model exists for WinCC V7 (and higher)<br>parallel installation WinCC V7 with WinCC Professional currently not yet possible | Single license<br>Upgrade license not available, since another license model exists for WinCC V7 (and higher) |

## 2.5. Startdrive Range of Products and Licensing



With this engineering product, drives can be commissioned.

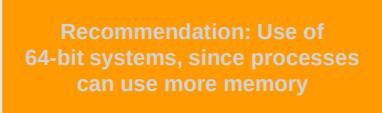
An additional license (engineering license) is not required for the parameterization and testing of drives. Operating the drives does not require any further licenses on the CU (runtime license).

## 2.6. Side-by-Side Installation

|                                |                                         | WinCC V13 |                      |              |                     |                         | STEP 7 V13 |              |
|--------------------------------|-----------------------------------------|-----------|----------------------|--------------|---------------------|-------------------------|------------|--------------|
|                                |                                         | Basic     | Comfort/<br>Advanced | Professional | Runtime<br>Advanced | Runtime<br>Professional | Basic      | Professional |
| STEP 7<br>Classic              | STEP 7 Professional 2010 SR2 (V5.5 SP2) | OK        | OK                   | OK           | OK                  | OK                      | OK         | OK           |
|                                | STEP 7 Micro/Win V4.0 SP2               | OK        | OK                   | OK           | OK                  | OK                      | OK         | OK           |
|                                | STEP 7 V1x                              | OK        | OK                   | OK           | OK                  | OK                      | OK         | OK           |
| WinCC<br>flexible<br>(classic) | WinCC flexible 2008 SP2 & Runtime       | OK        | OK                   | OK           | OK                  | OK                      | OK         | OK           |
|                                | WinCC flexible 2008 SP3 & Runtime       | OK        | OK                   | OK           | OK                  | OK                      | OK         | OK           |
| WinCC<br>(classic)             | WinCC RC V7.0 SP3                       | OK        | OK                   | x            | OK                  | x                       | OK         | OK           |
|                                | WinCC RT V7.0 SP3                       | OK        | OK                   | x            | OK                  | x                       | OK         | OK           |
|                                | WinCC RC V7.2                           | OK        | OK                   | x            | OK                  | x                       | OK         | OK           |
|                                | WinCC RT V7.2                           | OK        | OK                   | x            | OK                  | x                       | OK         | OK           |
| WinCC V1x                      | WinCC Basic/Comf/Adv V1x                | OK        | OK                   | OK           | OK                  | OK                      | OK         | OK           |
|                                | WinCC Runtime Advanced V1x              | OK        | UP                   | UP           | UP                  | OK                      | OK         | OK           |
|                                | WinCC Professional V1x                  | OK        | OK                   | x            | OK                  | x                       | OK         | OK           |
|                                | WinCC Runtime Professional V1x          | OK        | OK                   | x            | OK                  | x                       | OK         | OK           |

UP := Upgrade of the Runtime/Simulation

## 2.7. Operating Systems for PC/PGs

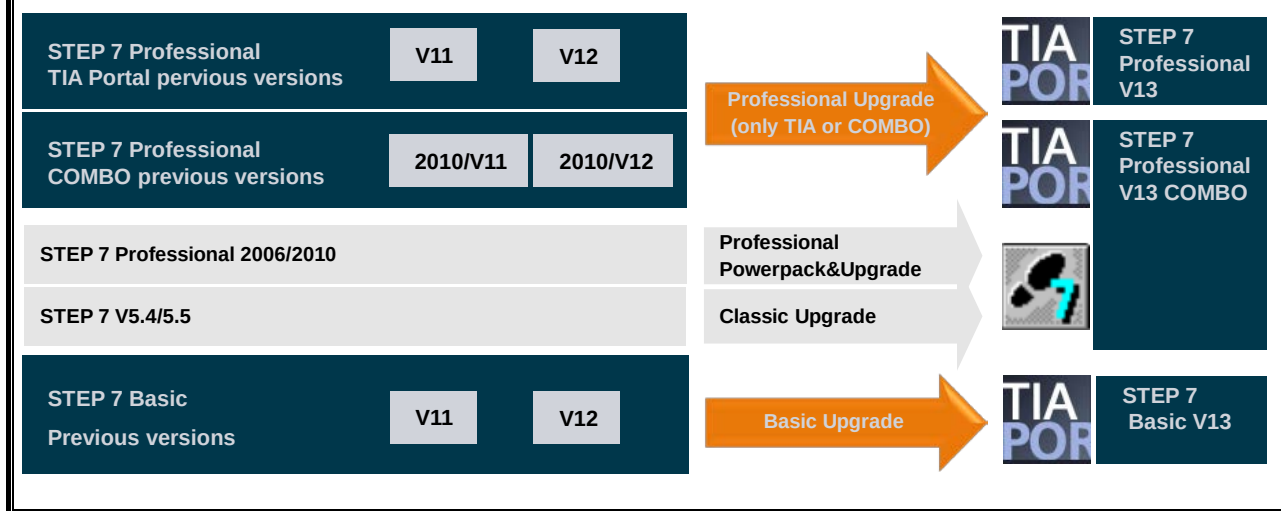
|                      |                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Engineering Software | Startdrive V13                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                 |
|                      | WinCC Comfort / Advanced / Professional V13                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                 |
|                      | STEP 7 Basic / Professional V13                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                 |
|                      |                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                 |
| Operating Systems    | 32-bit / 64-bit                                                                                                                                                                                                    | 64-bit                                                                                                                                                                                                                                                                                                                                                          |
|                      |  Windows 7 SP1 Prof. / Enterprise / Ultimate<br> |  Windows 8.1 Professional / Enterprise<br> Windows Server 2012 R2 Std.<br> Windows Server 2008 R2 Std. SP1 |

### 2.7.1. Virtualization (Released Software)

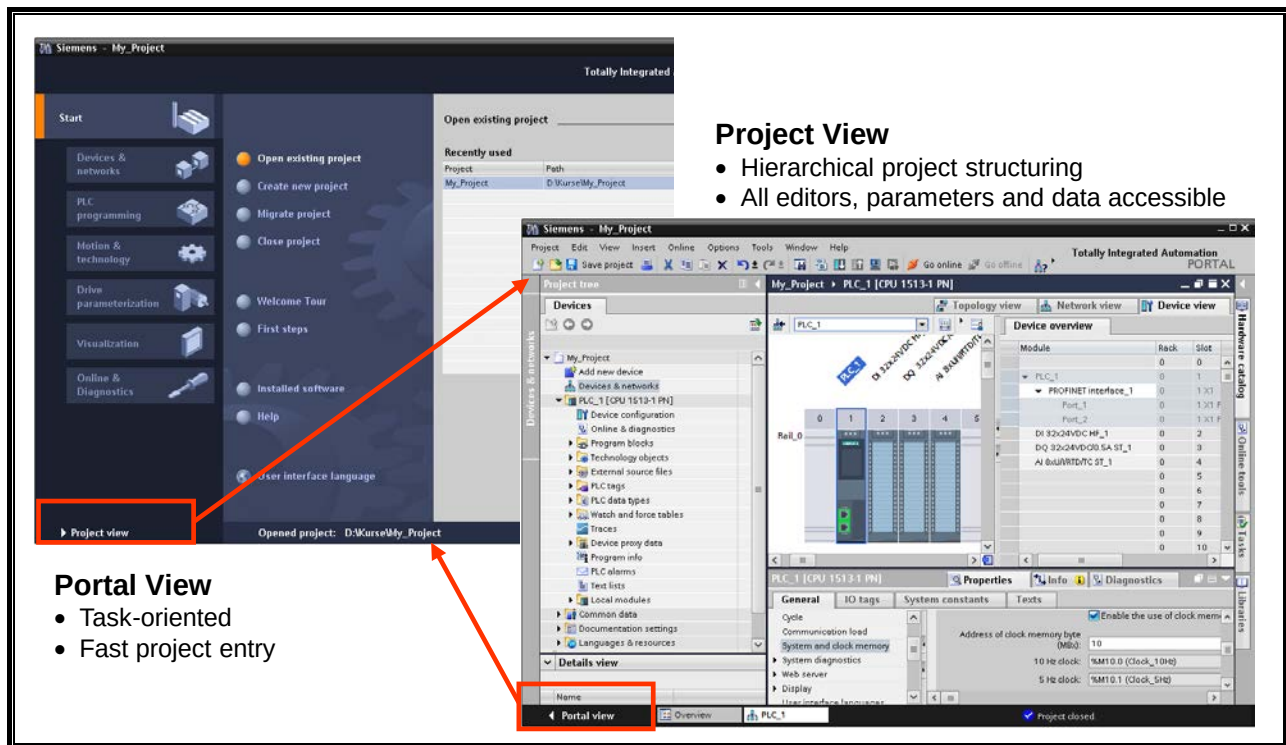
| Released Virtualization Software                                                                                                                                                                                                             | Released TIA Portal V13 Versions                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VMware Player 6                                                                                                                                                                                                                              | ✓ STEP 7 Basic/Professional V13                                                                                                                                |
| VMware Workstation 10                                                                                                                                                                                                                        | ✓ WinCC Basic/Comfort/Advanced/Professional V13                                                                                                                |
| VMware vSphere Hypervisor ESX(i) 5. 5                                                                                                                                                                                                        | ✓ Startdrive V13, Safety Advanced                                                                                                                              |
| Microsoft Windows Server 2012 R2 Hyper-V                                                                                                                                                                                                     |                                                                                                                                                                |
| VMware vSphere Hypervisor ESX(i) 5.5                                                                                                                                                                                                         | ✓ WinCC Runtime Advanced V13                                                                                                                                   |
| Microsoft Windows Server 2012 R2 Hyper-V                                                                                                                                                                                                     | ✓ WinCC Runtime Professional V13                                                                                                                               |
| Host Operating Systems                                                                                                                                                                                                                       | Guest Operating Systems                                                                                                                                        |
| <ul style="list-style-type: none"> <li>Windows 7 SP1 64-bit (Prof. / Ultimate / Enterprise)</li> <li>Windows 8.1 64-bit (Professional / Enterprise)</li> <li>Windows Server 2008 R2 Std. SP1</li> <li>Windows Server 2012 R2 Std.</li> </ul> | <ul style="list-style-type: none"> <li>Windows 7 SP1 64-bit (Prof. / Ultimate / Enterprise)</li> <li>Windows 8.1 64-bit (Professional / Enterprise)</li> </ul> |

## 2.8. License Upgrade Strategy

- One TIA Portal Upgrade in the most current version for all TIA Portal previous versions
- Upgrade can be ordered in each case as DVD and download
- Use the new TIA Selection Tool ([> Industry Mall > Configurators > Automation technology](#)) to find the right upgrade



## 2.9. TIA Portal: Portal View and Project View



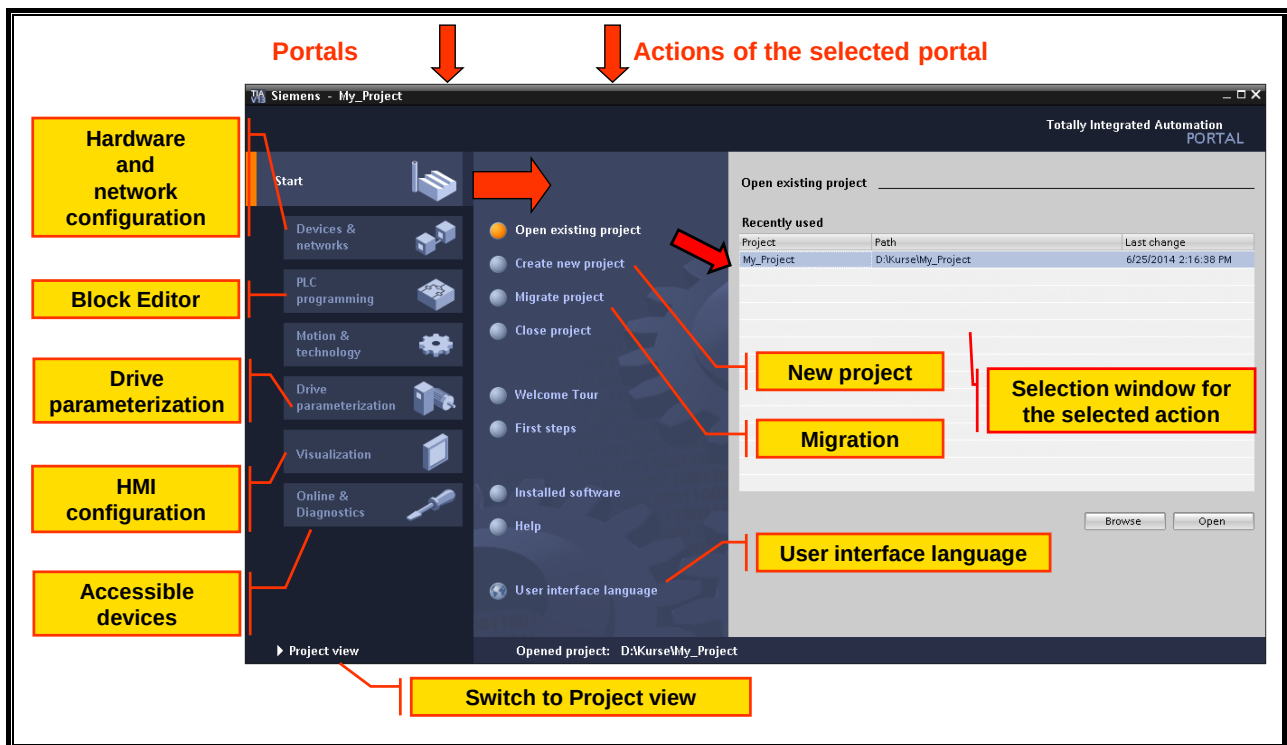
### Portal View

- Task-oriented mode of working
- Fast project entry with user guidance

### Project View

- Hierarchical structuring of the project
- The necessary editors open according to the task in hand
- All editors, parameters and data are found in one view

### 2.9.1. Portal View



#### Layout of the Portal View:

- Portals for the different tasks
- Actions for the selected portal
- Selection window for the selected action

#### Portals

Access to devices, components and their connections.

#### Actions

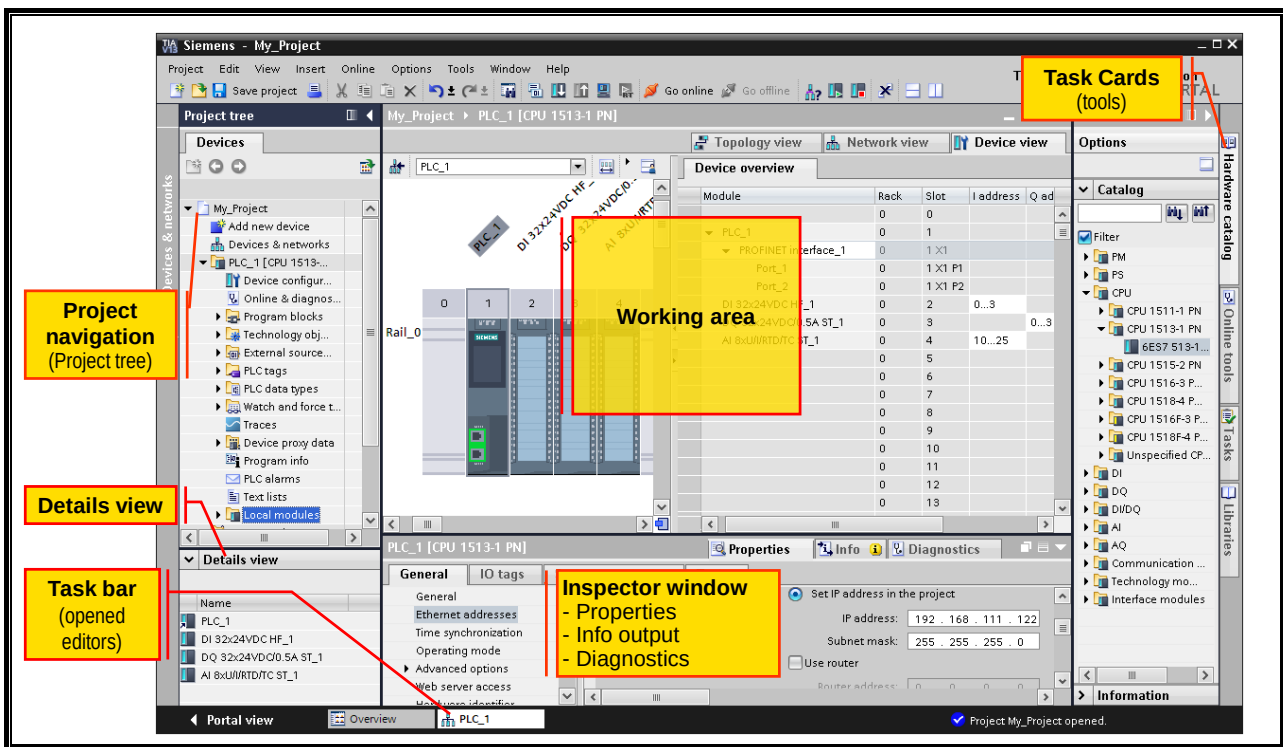
Depending on the selected portal, actions are available here that can be executed in the selected portal. Context-sensitive help is available in every portal.

#### Selection Window

The selection window is available in all portals. The content of the window adapts to your current selection.



## 2.9.2. Project View



### Project Navigation (Tree)

The Project tree contains all components and project data of an automation solution. All components can be opened from there.

### Working Area

The objects opened for editing are displayed in the working area. These objects include, for example hardware components, blocks, PLC tag tables, screens of HMI devices etc. If several objects are open at the same time, they are displayed as tabs in the task bar.

### Task Cards

These provide tools for configuring/programming. The content of the Task cards depends on the object displayed in the working area.

If a hardware station is open, the Hardware catalog, for example, is available as a Task card. If a program block is open, there is a Task card with Instructions.

### Inspector Window

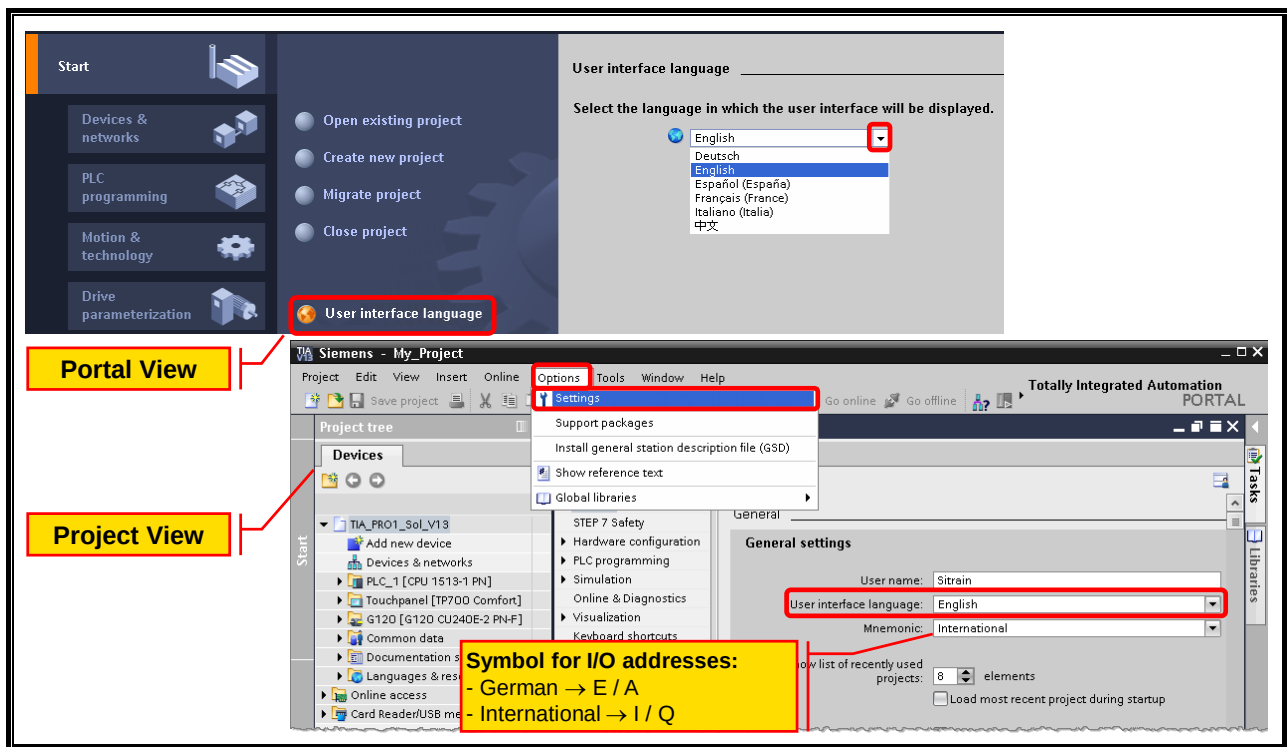
Additional information on a selected object or on executed actions is displayed in the Inspector window. The available properties of the selected objects can also be edited here (for example, properties of screens, screen objects, tags).

The Inspector window displays all system messages from the engineering, for example, those resulting from generating a project. This window should always be checked for any errors and warnings after a generation is completed.

### Details View

The Details view is a help window. Here, the elements of the configuration object selected in the Project tree are displayed. These can be used in the active working area (by dragging them to the working area using drag & drop). This enables fast access to the required objects (for example, tags).

### 2.9.3. TIA Portal - Settings: User Interface Language

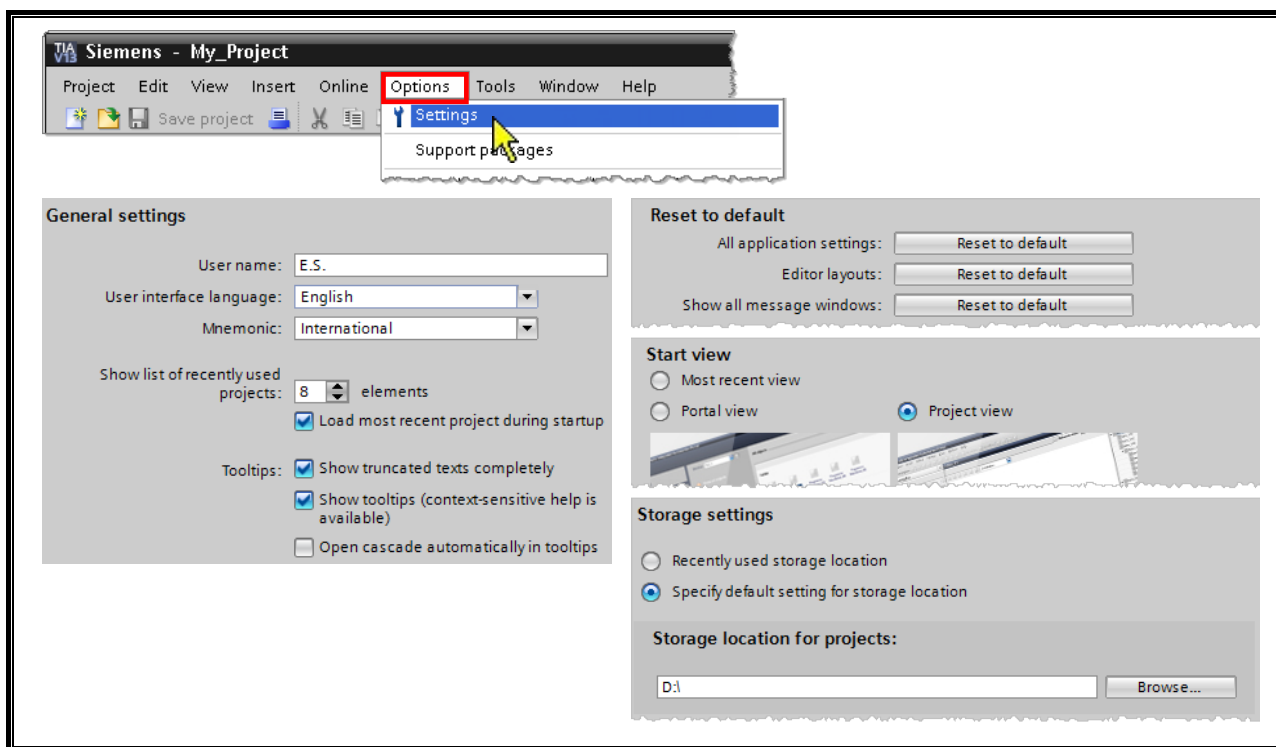


#### Available User Interface Languages

The user interface language of the TIA Portal can be changed during running operation. The following languages are available:

- German
- English
- French
- Spanish
- Italian
- Russian
- Korean
- Japanese
- Chinese (simplified)

## 2.9.4. TIA Portal - Settings: Language, Storage Location, Layout



### Language

The user interface language of the TIA Portal can be changed at any time without needing to restart. The TIA Portal always starts in the language in which it was last used.

### Storage Settings

#### Storage location for projects:

Storage location of newly created projects and their project libraries

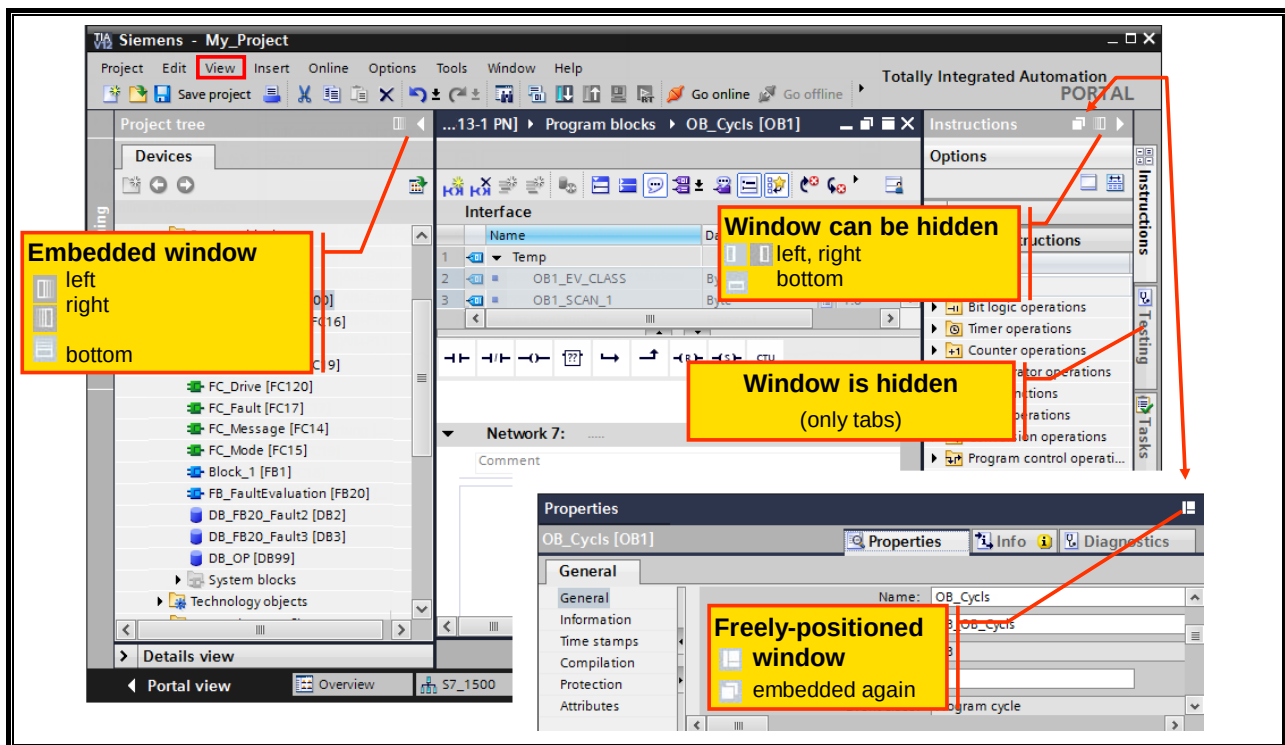
#### Storage location for libraries:

Storage location for global libraries

### Layout

If the layout is reset, the original window layout arrangement of the TIA Portal is restored.

## 2.9.5. Window Arrangement in the Project View



The positions and characteristics of windows can be configured individually. You can hide windows that are seldom required in order to enlarge the surface of the working area.

The current configuration of the engineering user interface is saved in the user profile of Windows. On saving the project, the positions and characteristics of windows are automatically saved with it.

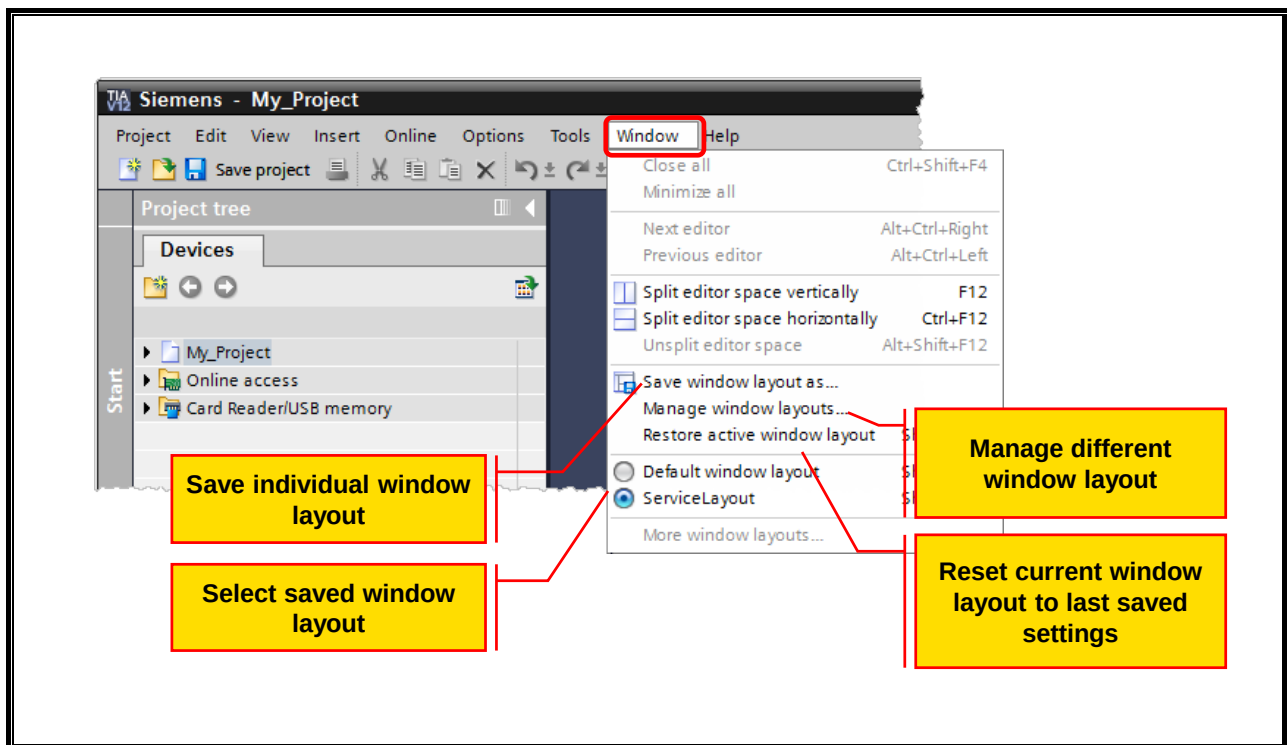
### Window Arrangement Options

- Embedded (docked) window
  - fixed location and fixed size on user interface
  - position at left, bottom or right outside of the working area is possible
  - always open, reduces the working area
- Window can be hidden
  - hidden at edge of user interface
  - position at left, bottom or right is possible, superimposed on working area when open
  - default status = window closed, and tab displayed at edge of the user interface
  - mouse click on the tab opens the window
  - closed automatically the next time there is a click outside the window area
- Freely-positioned window!! **Makes sense if a 2nd monitor is used!!**
  - can be positioned anywhere on the user interface
  - permanently covers the user interface area underneath it

Clicking the functions  in the window title bar switches between the modes "freely-positioned", "embedded" and "hidden".

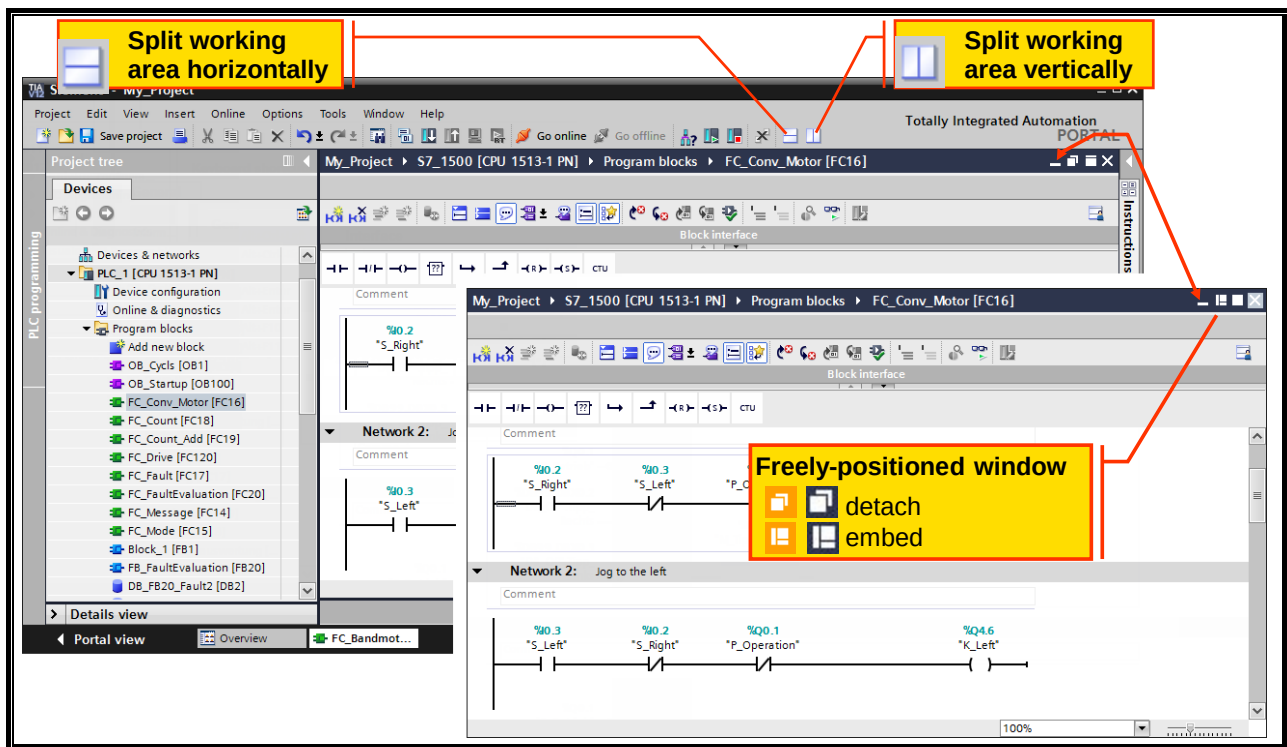
Hidden windows are opened by clicking on the tab and closed again by clicking outside the window area.

## 2.9.6. Window Layouts Save / Manage / Use



The different window arrangements of the user interface can be saved and then restored.

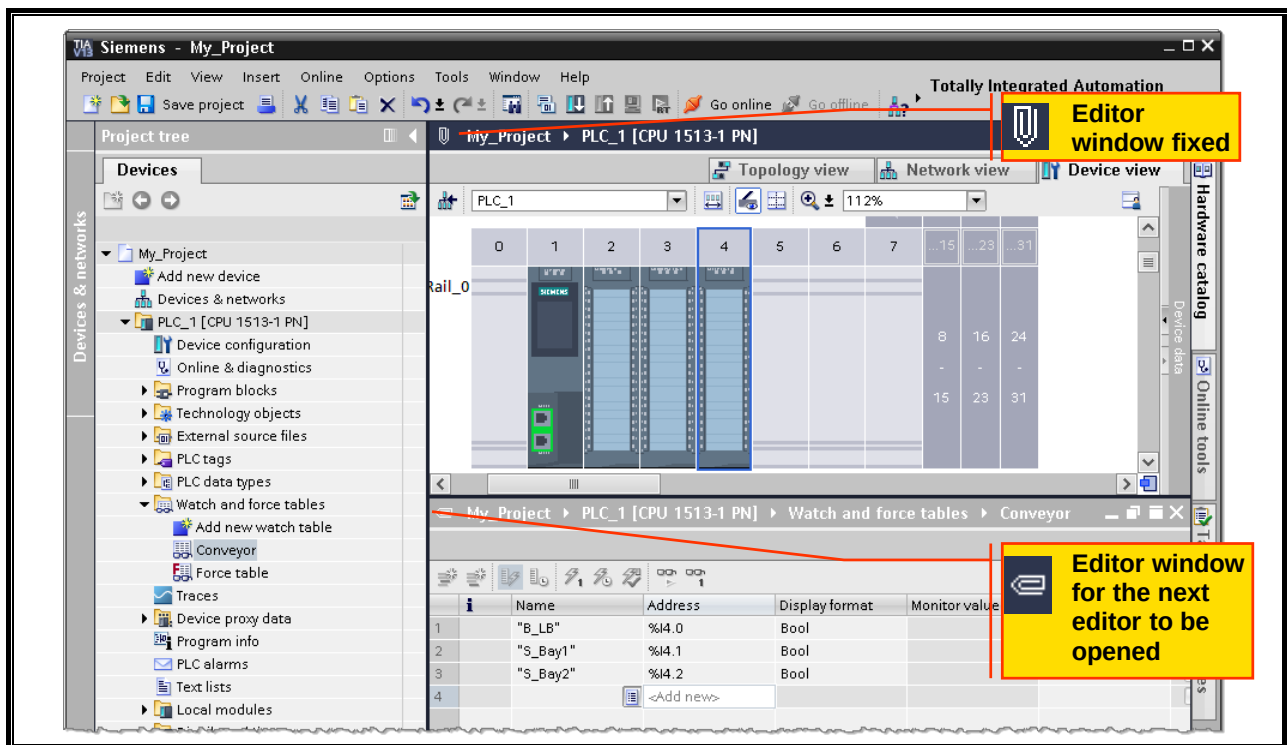
## 2.9.7. Window Sections in the Working Area



The windows of the working area can be arranged as follows:

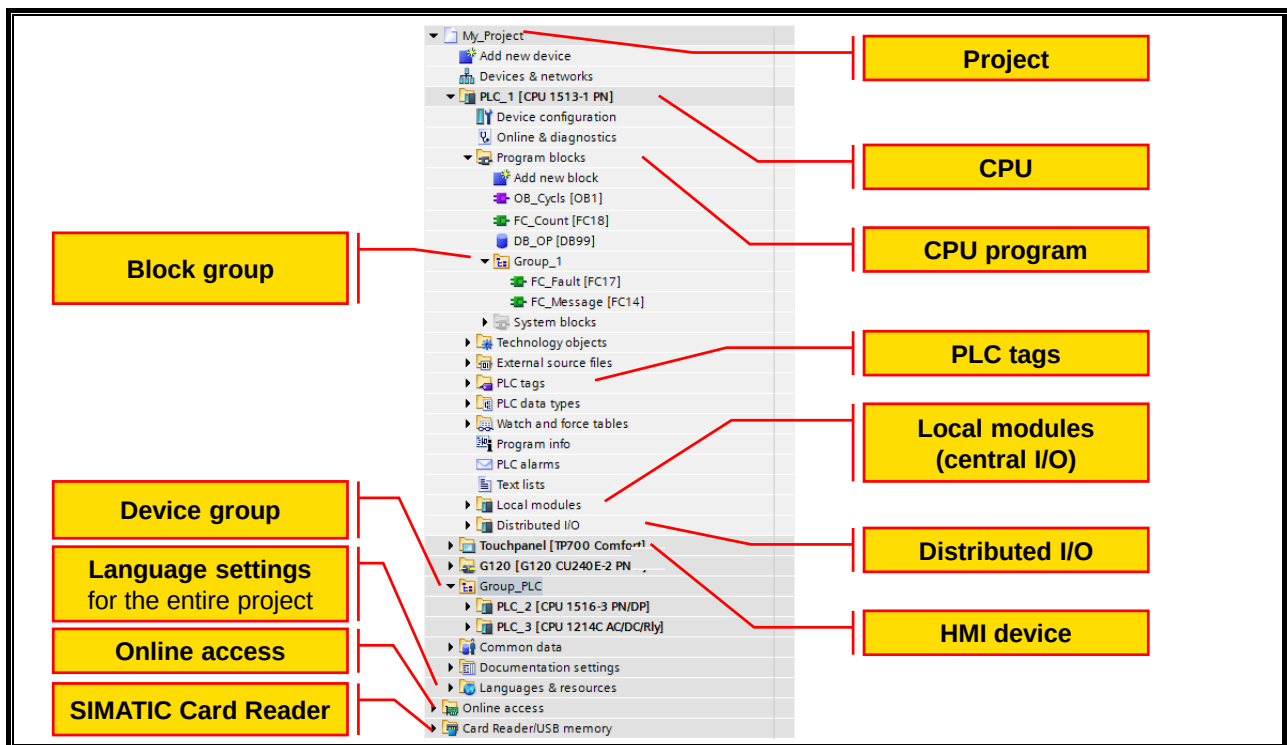
- 
 Maximize (full size) a working area  
(color depending on View online/offline)
- 
 Detach or release a window from the working area  
(color depending on View online/offline)
- 
 Embed a window in the working area once again  
(color depending on View online/offline)
-  Split the working area horizontally into two windows
-  Split the working area vertically into two windows

## 2.9.8. Keeping the Editor Window in the Foreground (when Working Area is Split)



If you work with a split working area, one of the two working areas can be fixed (attached) by clicking on the “paper-clip” so that when you open a further editor, this first one always remains fixed in the foreground and the newly opened one always becomes the second visible editor.

### 2.9.9. Project Tree



The "Project tree" window provides access to all components and project data. All components and all available objects of a project appear in the Project tree in a tree structure and can be opened from there by double-clicking on them.

The following actions can be carried out:

- adding new components (controllers, HMI devices etc.)
- editing existing components
- querying and modifying properties of existing components
- diagnosing accessible components

To improve clarity, objects (entire stations or blocks of a station) can be grouped together.

#### Hiding/Showing a Structure Section

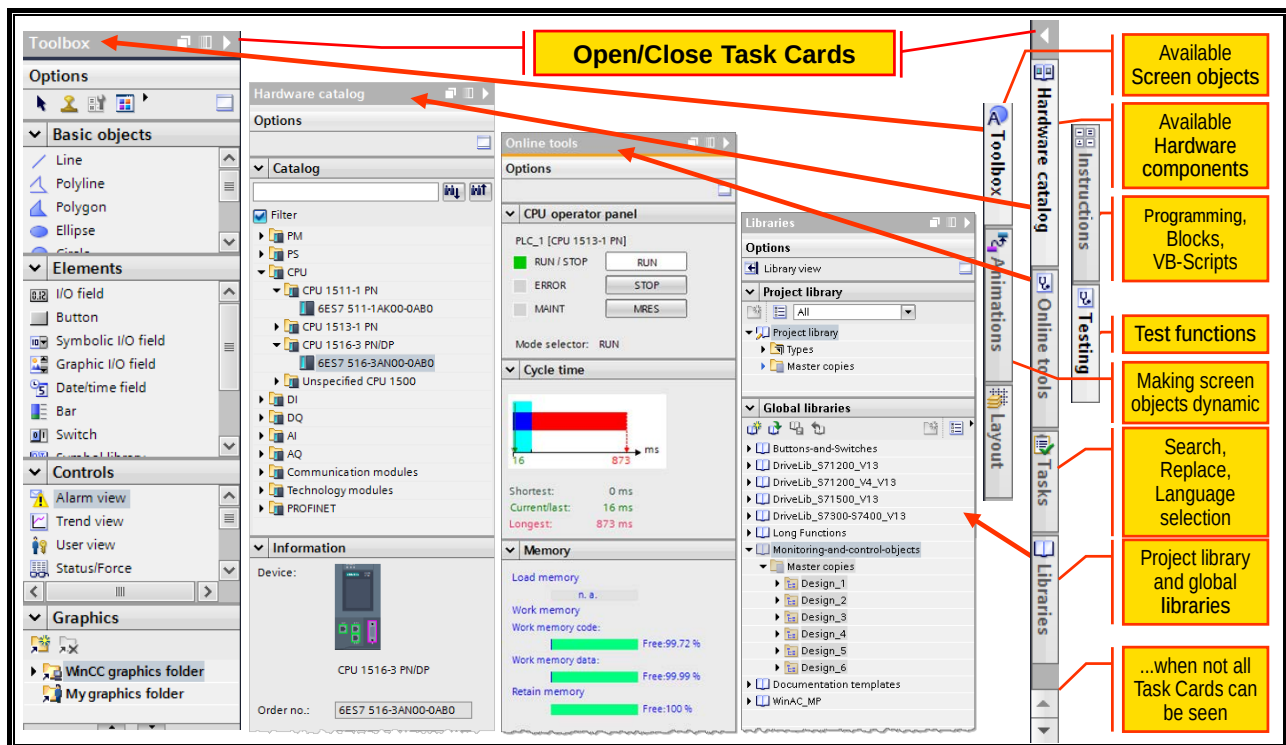
An underlying structure is indicated by the black triangle ▸. By clicking on the triangle, the underlying structure level can be shown ▸→▾ or hidden again ▾→▸.

#### Details Window

Certain contents of a selected object are displayed in the Details window. Possible contents are, for example, text lists or tags.



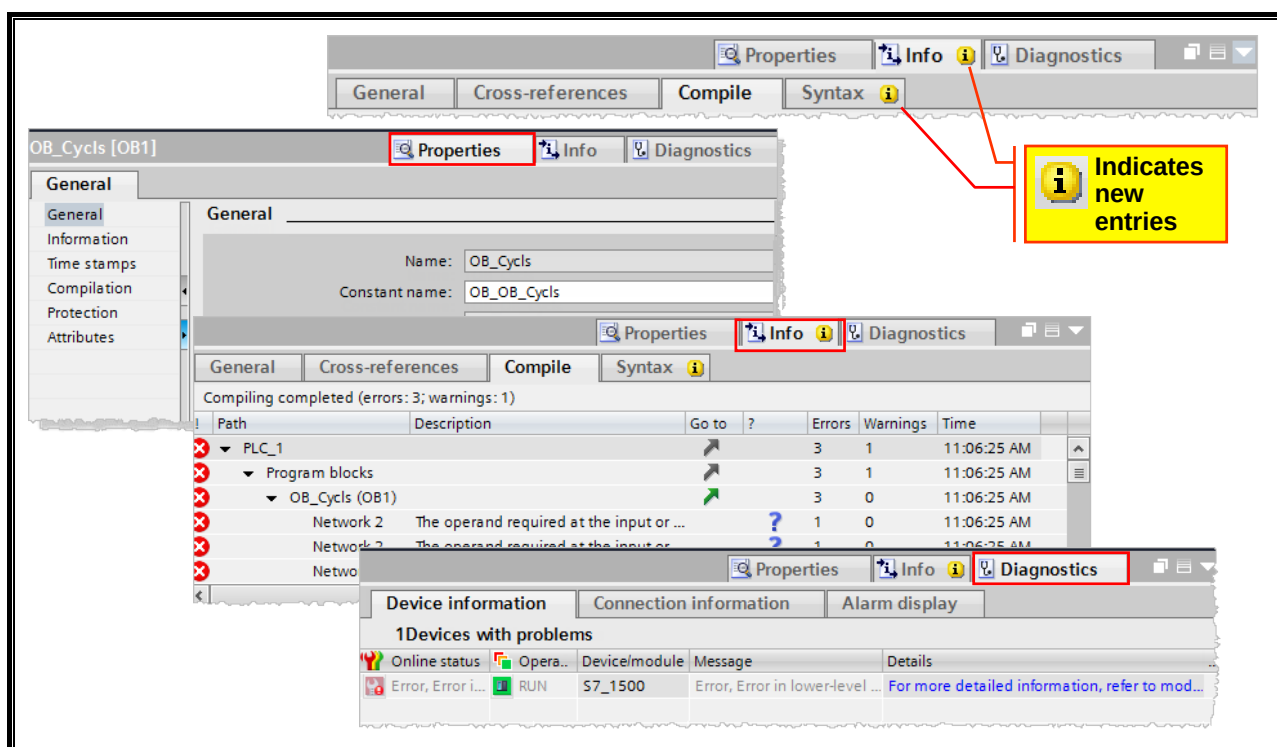
## 2.9.10. Task Cards



Which task cards are available depends on the products that have been installed and on the object currently being edited or open in the working area. If not all Task Cards are visible, the Task Card-bar can be shifted using the cursor buttons at the bottom right.

- Hardware Catalog**  
 Here, all the available hardware components such as CPUs, modules etc. can be selected.
- Instructions**  
 Instructions for programming blocks;  
 Code templates and function list wizard for script programming (VBS as well as C scripts with WinCC Professional)
- Online Tools**  
 If there is an online connection established, diagnostics and online information can be called, such as, the current cycle time of the CPU and the configuration of the load and work memory of the CPU. As well, the CPU can be switched to the STOP and RUN mode.
- Toolbox**  
 Configurable screen objects (graphics, display and operator control objects) in different panes (basic objects, elements, controls, optional customized controls, graphics)
- Animations**  
 Templates for making screen objects dynamic in different panes (movements, display, tag link for making dynamic)
- Layout**  
 Tools for adapting the presentation when designing screens during configuration of HMI devices (zoom, level assignment, grid alignment, objects outside the area)
- Tasks**  
 Here, classic editor functions are available such as finding and replacing tags, instructions etc.
- Libraries**  
 Management of the local project library and global libraries

## 2.9.11. Inspector Window



Additional information on a selected object or on actions to be executed is displayed in the Inspector window. The Inspector window consists of the following tabs:

→ can be selected by clicking the tabs

 This symbol in the tab indicates new entries.

 If errors are displayed, you can jump to the error location or into the associated editor by double-clicking on the error information.

### "Properties" Area

This tab displays the properties of the object selected in the working area and editable properties can be changed.

### "Info" Area

This is the output area of the engineering. This tab displays further information for the object selected. In addition to this, messages relating to executed actions, for example, compilation and download of blocks to the CPU are output.

"General" tab → general status output

"Cross-references" tab → display of the current locations at which the selected object is used

"Compile" tab → status display of compilation progress

"Syntax" tab → status display for invalid programming commands

### "Diagnostics" Area

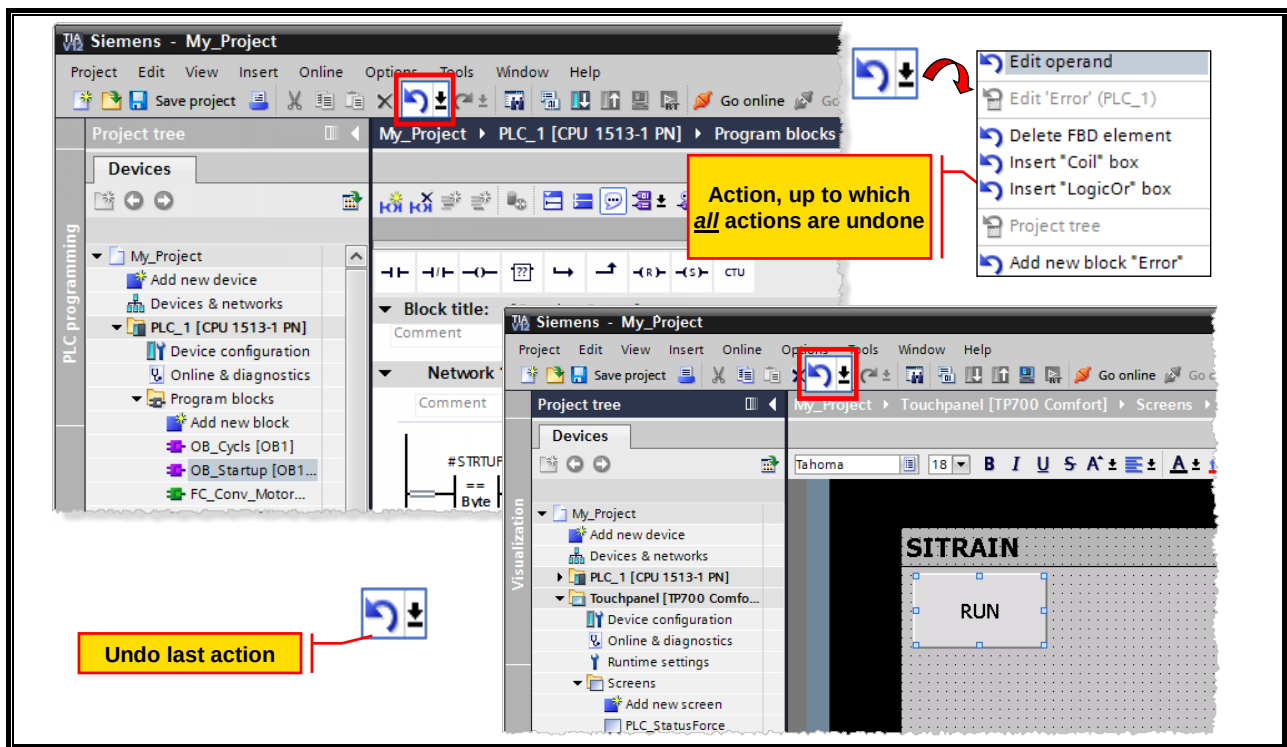
This tab displays information on system diagnostics and configured alarm events

"Alarm display" tab → Display of current CPU alarms (for example Alarm\_S)

"Device information" tab

"Connection information" tab → for components currently connected online

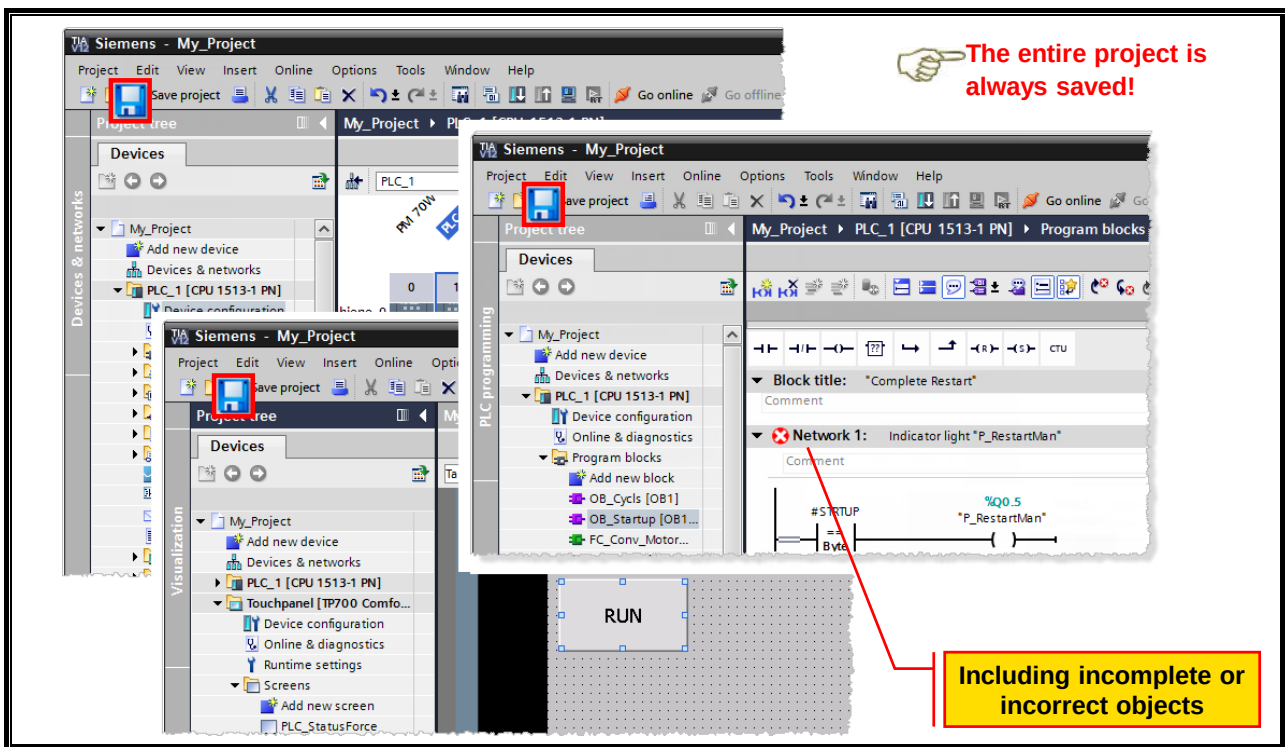
## 2.9.12. Undo and Redo



### Undo Concept of the TIA Portal

The drop-down menu shows the user in which editor the "Undo" function is executed. Closed editors are then automatically opened. Since all actions are only undone up to the selected action, the consistency of the project is ensured.

### 2.9.13. Saving a Project

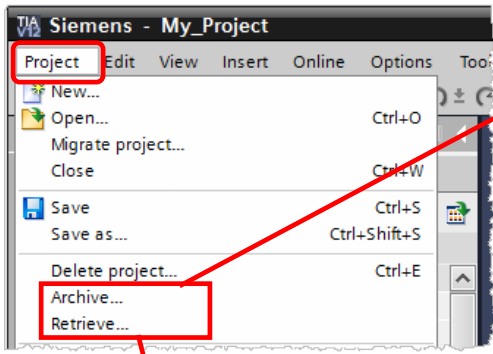


#### Save

Regardless of the object that is open in the working area, it is **always the entire project** in the current state that is saved when the Save icon is pressed, even if some objects of the project are still incomplete or faulty [incorrect] (for example, syntax faulty blocks or symbols which have not yet been assigned an absolute operand in the global symbol list).

If the project is closed without saving, all changes made or objects created during the session are discarded.

## 2.9.14. Archiving / Retrieving a Project



**Archive...**  
Prerequisite: Project is open

Save as type: TIA Portal project archives

- Reorganize project
- Delete HMI RT data
- Compress .zap13 format

**Retrieve...**

Save as type: TIA Portal project archives

- Decompress (extract) .zap13 format
- Save project
- Open project

Save as type: TIA Portal project minimized

- Reorganize project
- Normal file folder

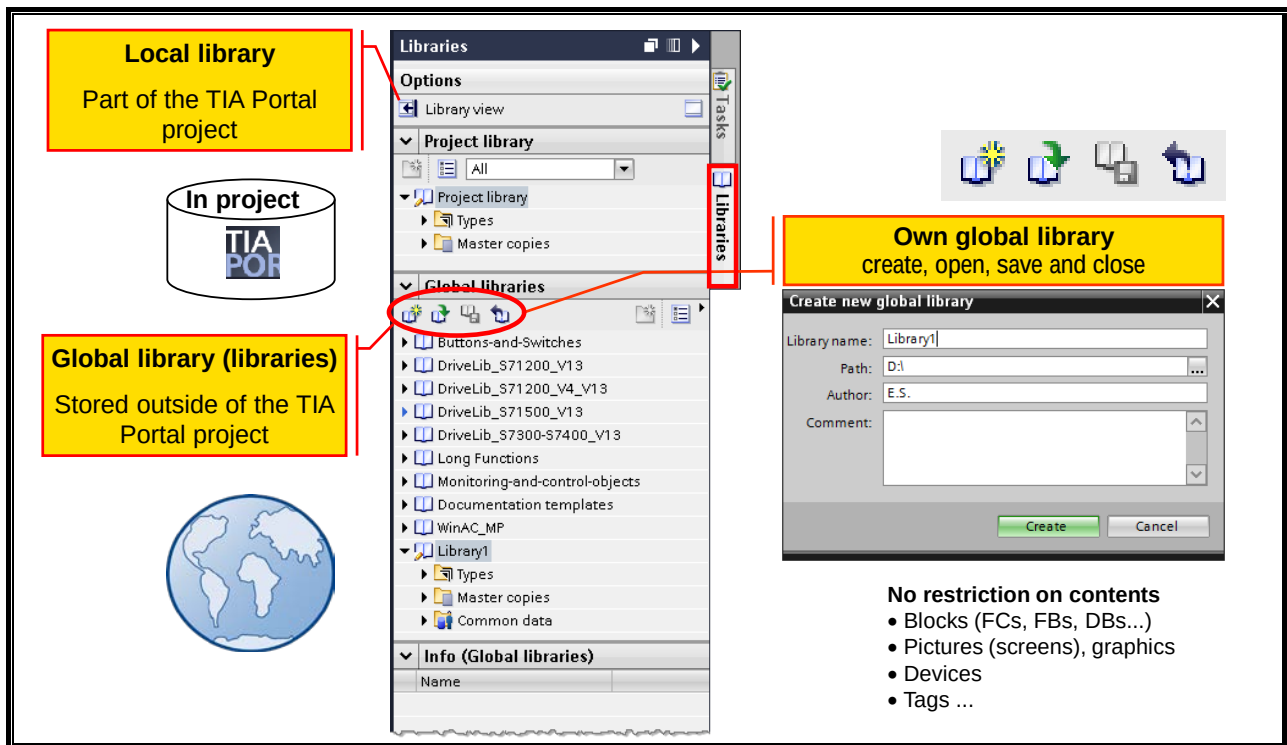
### Archiving...

optionally, compressing or creating a project archive

### Retrieving...

extracting a project archive

## 2.10. Libraries



### Project Library

Each project has its own library. Here, objects can be stored that are to be reused within the project. This project library is always opened, saved, and closed together with the current project.

### Global Libraries

Global libraries are stored independently of the projects and are used to store objects that are to be used and reused in different projects.

The area of the global libraries also contains libraries supplied with the TIA Portal that, for example, contain ready-made functions and function blocks. The supplied libraries cannot be changed.

### Library Objects

A library is a collection of any project objects:

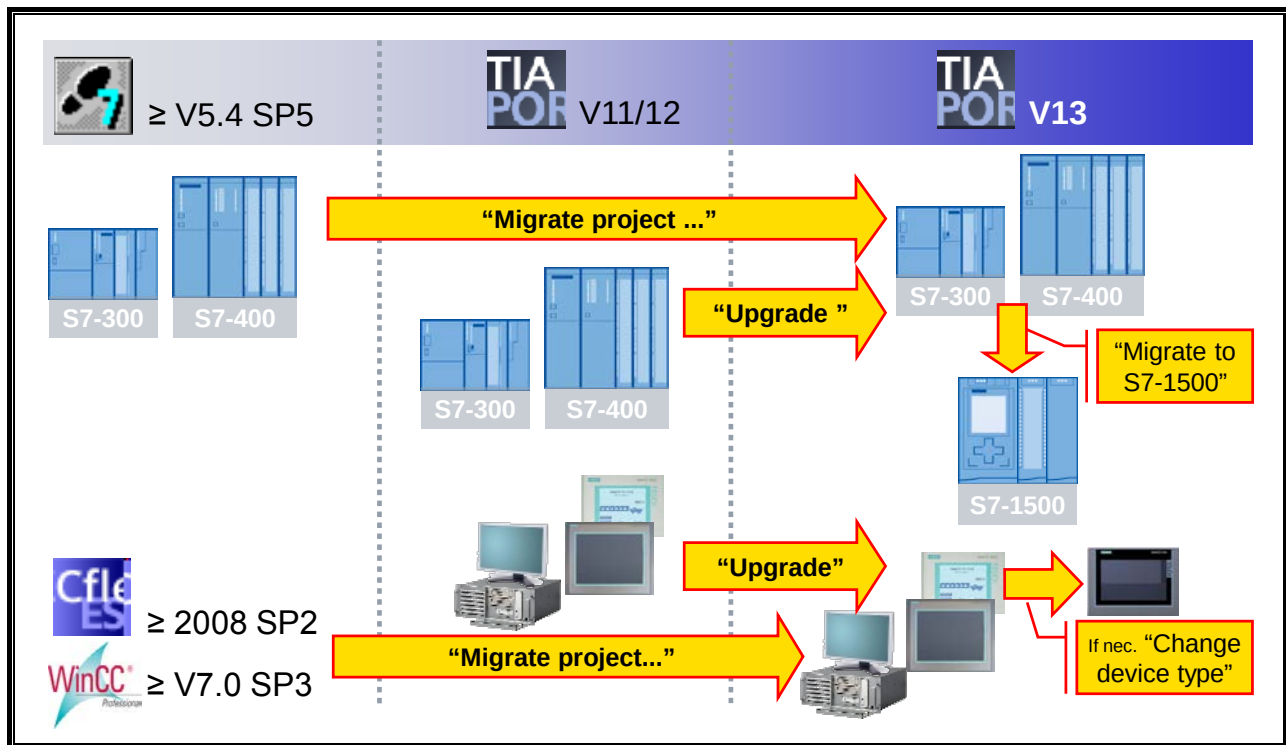
Functions (FCs), function blocks (FBs), data blocks (DBs), devices, PLC data types, watch tables, screens, graphics, faceplates, tag tables....

### Uses of Global Libraries

Library objects can either be used as a master copy or as an instance.

- Objects from the **Master copies** folder are *copied to the project when used*. If subsequent changes are made to this master copy, these changes are not made to the copies in the project.
- Objects from the **Types** folder are *copied to the Types folder of the project library when they are used and an instance (location of use) is created in the project*. These objects are then stored in the local project library. The object itself is not used in the project, rather only a reference to it. If such library objects are modified, all the instances (locations at which the object is used) in the project are also updated.

## 2.11. Project Migration



### Project Migration

- **Classic -> TIA Portal**

If projects from the classic world (older versions) are to be migrated into the TIA Portal world, the classic projects must be STEP7  $\geq V5.4$  SP5 or WinCC flexible  $\geq 2008$  SP2 or WinCC  $\geq V7.0$  SP3 versions.

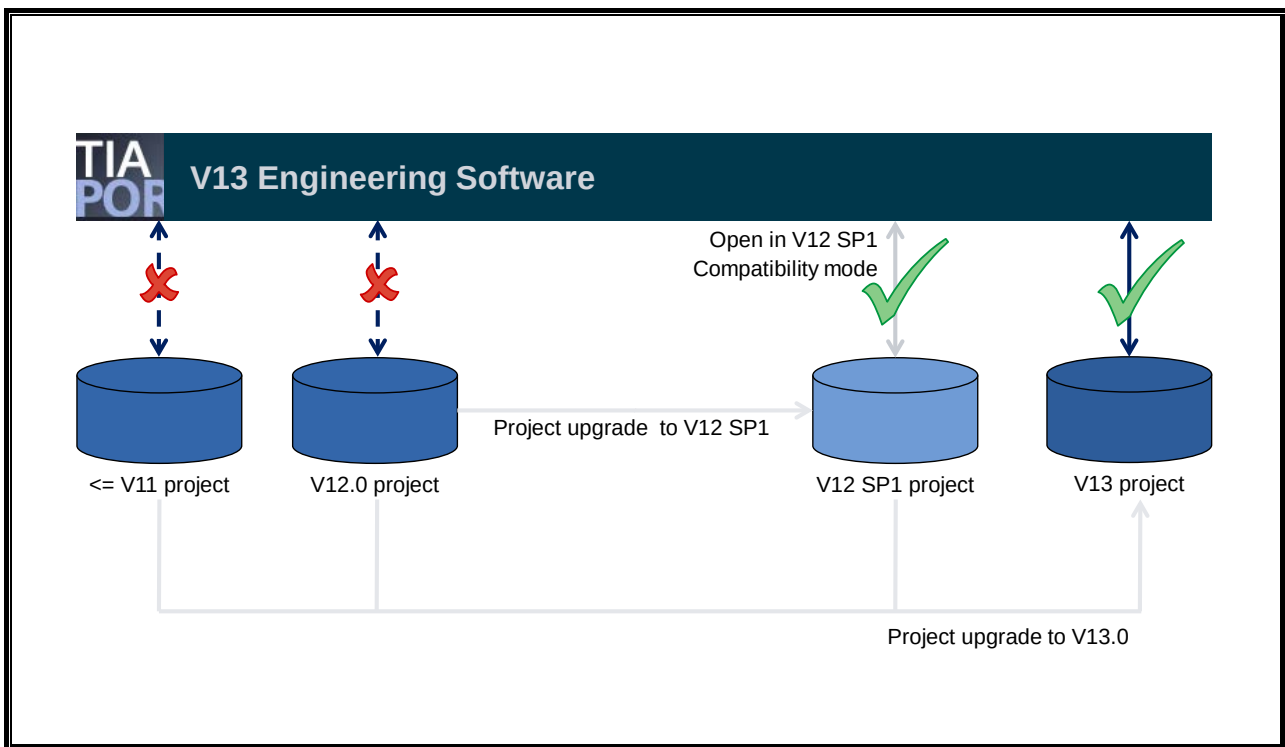
A migration of an S7-300/400 program to the hardware of the S7-1500 is possible within V13.

- **TIA Portal V11 and V12 -> TIA Portal V13**

V11 and V12 projects and libraries can be migrated without restrictions into V13 projects or V13 libraries using the "Upgrade" function.

V12 SP1 projects as well as V12 SP1 libraries can be opened and edited with TIA Portal V13 without the V12 SP1 project becoming a V13 project.

### 2.11.1. Project Upgrade and Compatibility Mode



#### Compatibility Mode

If a V12 SP1 project is opened with TIA Portal V13 and not upgraded, the project finds itself in the compatibility mode. The range of functions of the TIA-Portal V13 is then limited to the range of functions of V12 SP1. Components that were later supplied for Version V12 SP1 within the framework of a Hardware Support Package (HSP) can be added. The project remains backward compatible and can still be opened and edited with TIA-Portal V12 SP1.

Global libraries are always created in the most current format and are not backward compatible, even if the project is open in the compatibility mode.

Backward compatibility of the current product version

Projects that were saved with the current version of TIA-Portal are not backward compatible because of the expanded functionality compared to older versions. Projects that were saved with TIA-Portal V13 can only be opened with TIA-Portal V13 or newer.



## 2.12. Help Functions

The screenshot illustrates the help functions for the **P\_TRIG: Scan RLO for positive signal edge** instruction. It shows the instruction list on the left, a detailed description and parameter table on the right, and an example ladder logic diagram at the bottom.

**Instructions List:**

| Name                                | Description                              |
|-------------------------------------|------------------------------------------|
| General                             |                                          |
| Bit logic operations                |                                          |
| Normally open contact [Shift+F2]    |                                          |
| Set operand on positive signal edge |                                          |
| Set operand on negative signal edge |                                          |
| <b>P_TRIG</b>                       | <b>Scan RLO for positive signal edge</b> |

**Description:** Use the "Scan RLO for positive signal edge" instruction to query a "0" to "1" change in the signal state of the result of logic operation (RLO). The instruction compares the current signal state of the RLO with the signal state of the previous query, which is saved in an edge memory bit (<operand>). If the instruction detects a change in the RLO from "0" to "1", there is a positive signal edge. If a positive edge is detected, the output of the instruction has the signal state "1". In all other cases, the signal state at the output of the instruction is "0".

**Note:** The address of the edge memory bit must not be used more than once in the program, otherwise the bit memory is overwritten. This step influences the edge evaluation and the result is therefore no longer unique. The memory area of the edge memory bit must be located in a DB (static area for FB) or in the bit memory area.

**Parameter Table:**

| Parameter       | Declaration | Data type | Memory area   | Description                                                      |
|-----------------|-------------|-----------|---------------|------------------------------------------------------------------|
| Current RLO     | Input       | BOOL      | I, Q, M, D, L | Current RLO                                                      |
| Edge memory bit | InOut       | BOOL      | M, D          | Edge memory bit in which the RLO of the previous query is saved. |
| Output          | Output      | BOOL      | I, Q, M, D, L | Result of edge evaluation                                        |

**Example:** The following example shows how the instruction works:

```

graph LR
    TagIn_1[TagIn_1] --> AND1(( ))
    TagIn_2[TagIn_2] --> AND1
    AND1 --> P_TRIG[P_TRIG]
    P_TRIG --> TagM[Tag_M]
    TagM --> CAS1[CAS1]
    CAS1 --> JUMP((JUMP))
  
```

The RLO of the previous query is saved in the edge memory bit "Tag\_M". If a "0" to "1" change is detected in the signal state of the RLO, the program jumps to jump label CAS1.

Wide-ranging help functions are available for solving your tasks; these describe basic concepts, actions and functions.

- **Tooltip** for information on the user interface elements, for example, instructions, input boxes, buttons and symbols  
Some tooltips provide cascades with more precise information. These are activated by clicking the ► in the tooltip ▼.
- Help on the current context  
For example, on menu commands by pressing the <F1> or <Shift+F1> keys.
- In the input boxes (for example, in the Properties in the Inspector window), the **roll-out** provides information about the permitted value ranges and data types for the input.

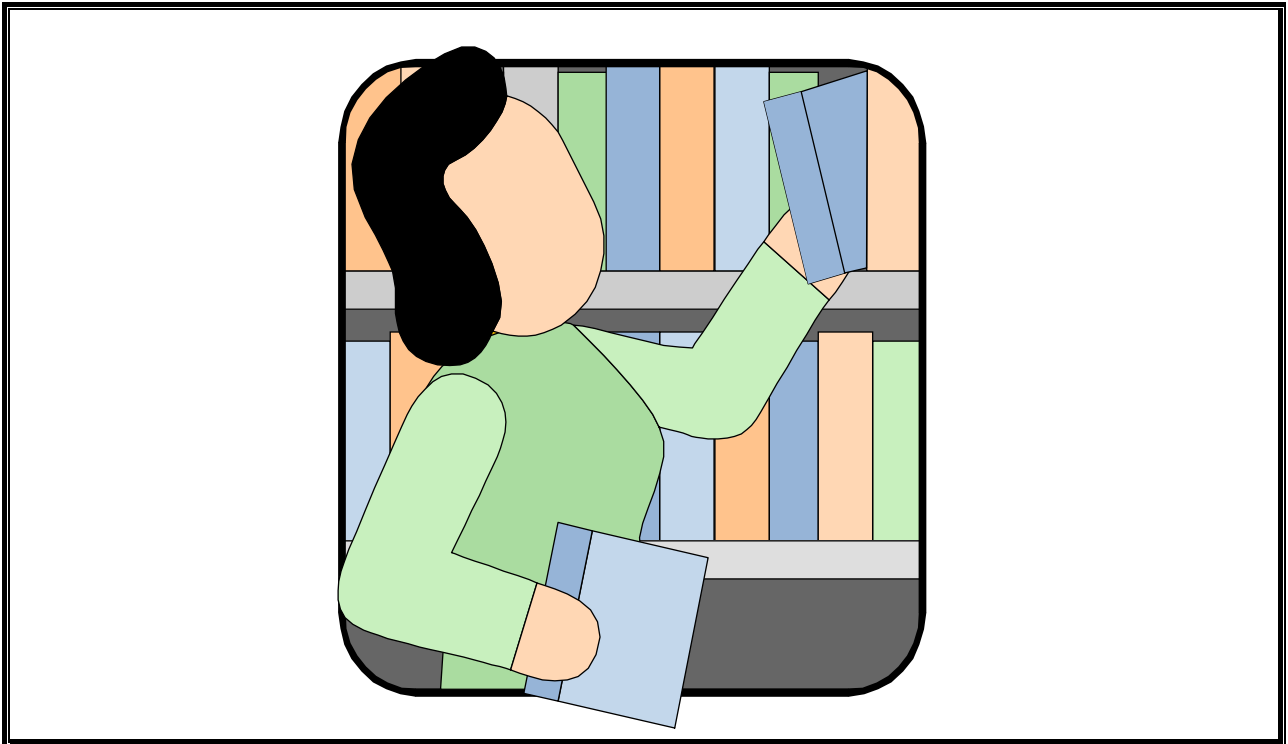
### Contents of the Help Functions

Menu function > Help > Show Help

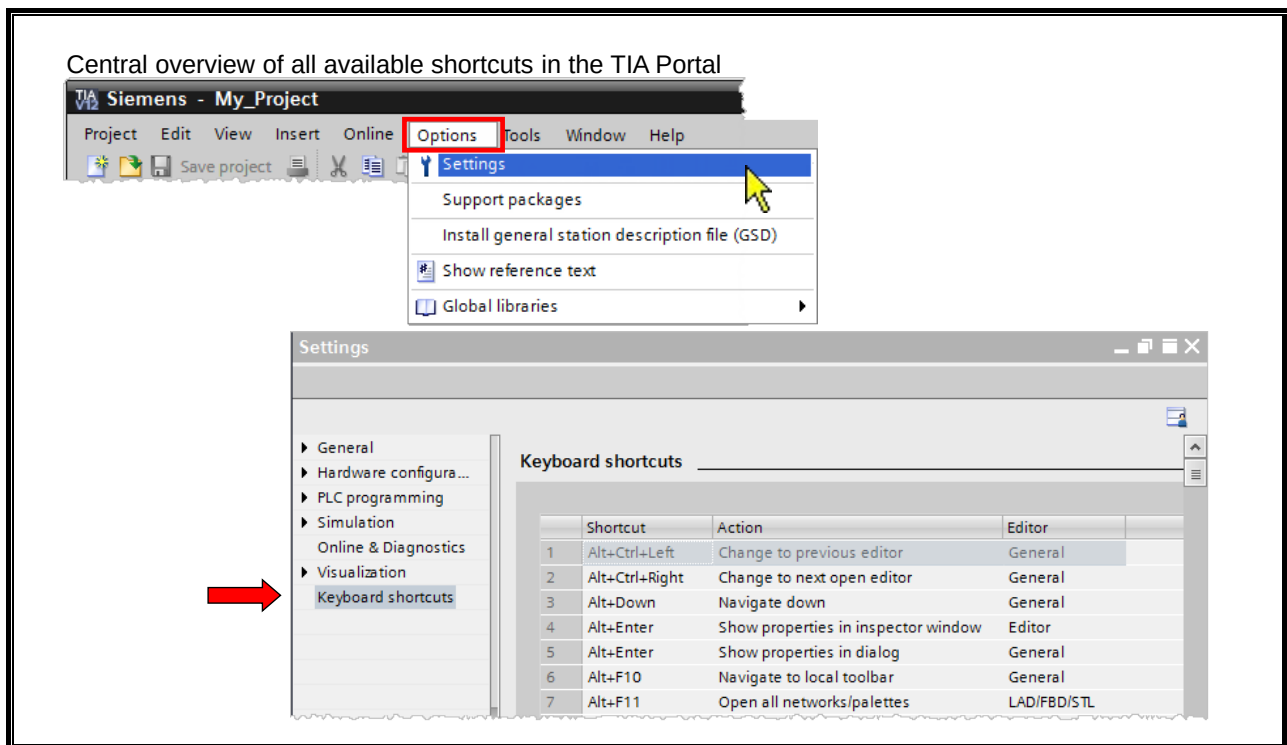
The Help describes concepts, actions and functions. It also contains reference information and examples. The Help opens up in its own window on the right side of the screen. The following functions are available in the navigation area:

- Table of contents
- Search in the index
- Full text search of the entire Help

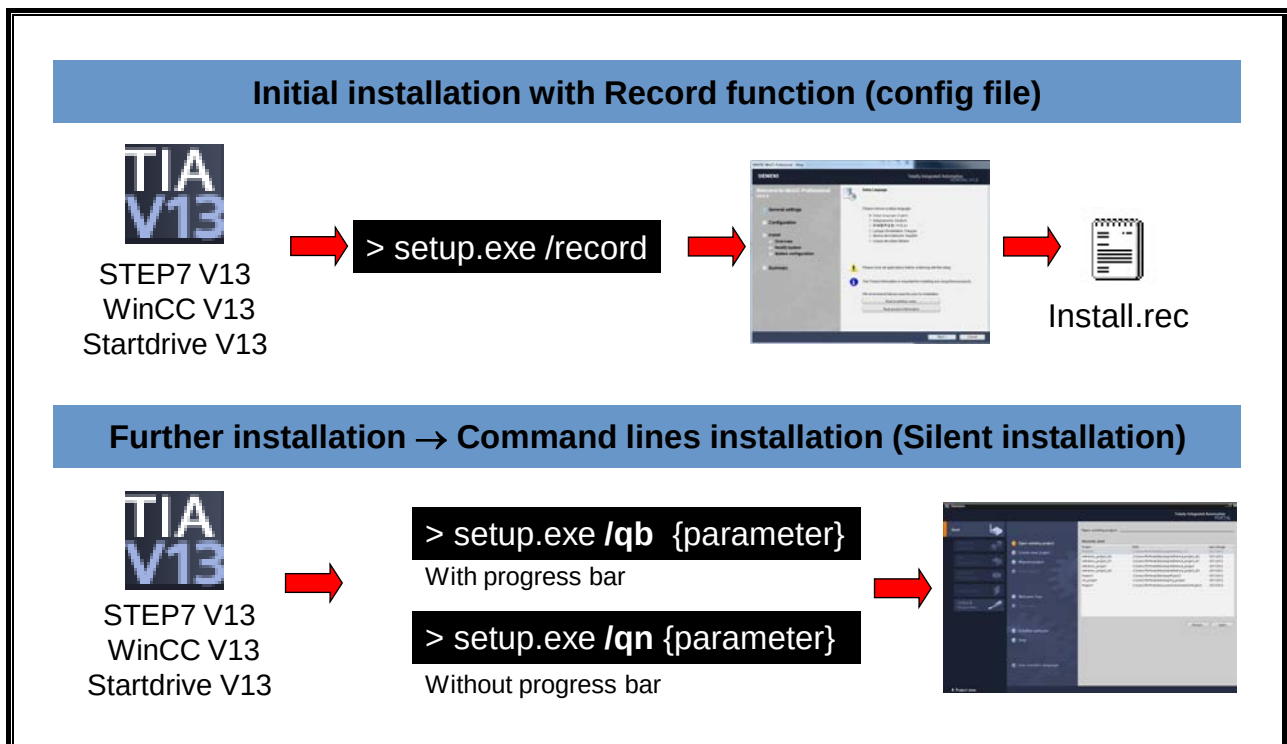
## 2.13. Additional Information



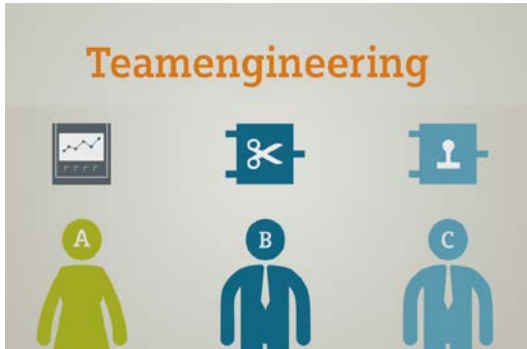
### 2.13.1. Keyboard Shortcuts of the TIA Portal



### 2.13.2. Installation with Record Function in the Setup



### 2.13.3. Team Engineering



**1. PLC proxy for HMI**

- Separate HMI configuration via imported CPU proxy (data interface for tags & DBs)

**2. Copy & Merge (co-ordinated engineering in a team)**

- Project engineers work on different project parts and namespaces.
- Manual project merging with system-supported conflict settlement (Compare)

**3. Online commissioning**

- Simultaneous online monitoring on a controller

#### Team Engineering

Team Engineering provides the opportunity of creating projects in a team and commissioning them. During commissioning, several persons together can access a CPU in parallel with up to five Engineering Systems (ES).

A particular advantage is that, in the commissioning phase, parts of a master project can be edited simultaneously and in parallel offline independent of one another. During loading, the changes of the other persons are displayed in a dialog for the "software synchronization" and – as far as possible – automatically synchronized.

Certain online functions can also be executed simultaneously and in parallel from several Engineering Systems on the commonly used CPU, such as:

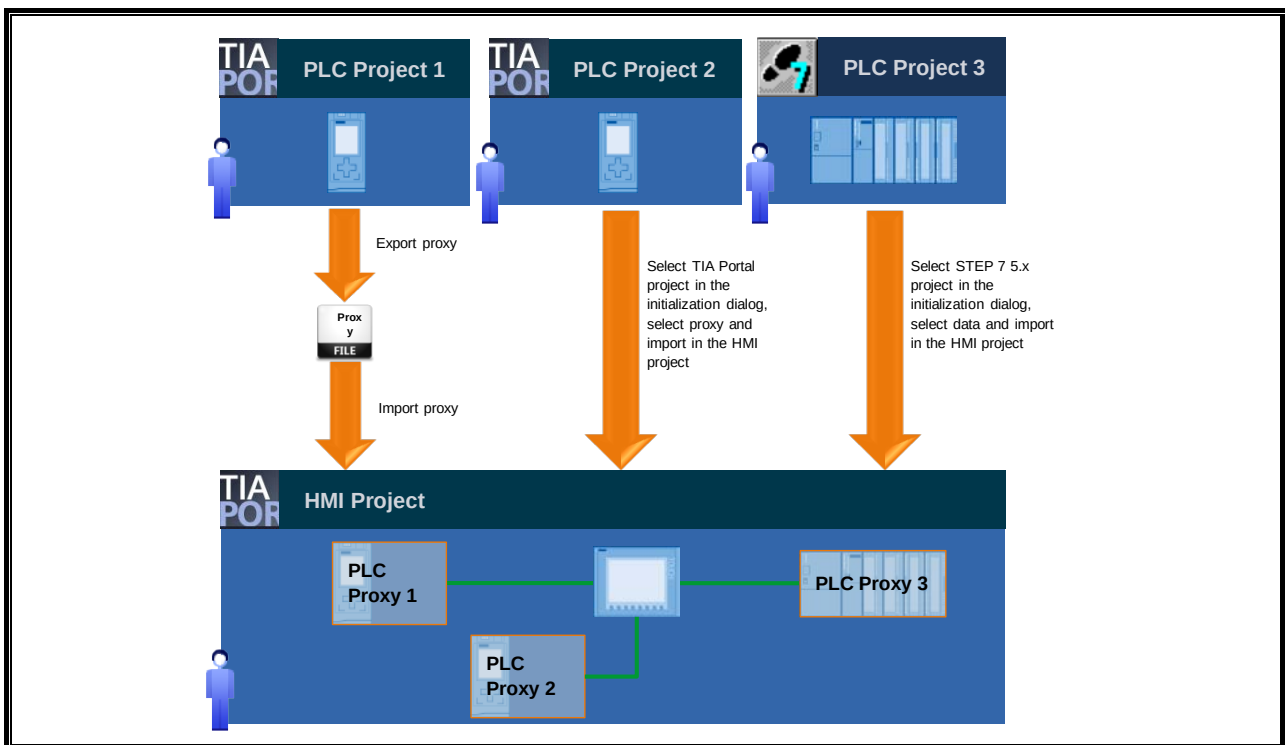
- The monitoring of blocks on the CPU
- The modifying of blocks on the CPU
- Trace functions

The following online functions cannot be simultaneously executed:

- Force: in each case only one ES can force on the CPU.
- Download: in each case only one ES can download into the CPU.

You will also find additional information on the topic of Team Engineering under [Service and Support in Siemens Industry Online Support](#) .

### 2.13.4. Inter Project Engineering



#### Inter Project Engineering (IPE? )

With the IPE (Inter Project Engineering) function, you can read in controller data from other TIA Portal projects and use them for configuration.

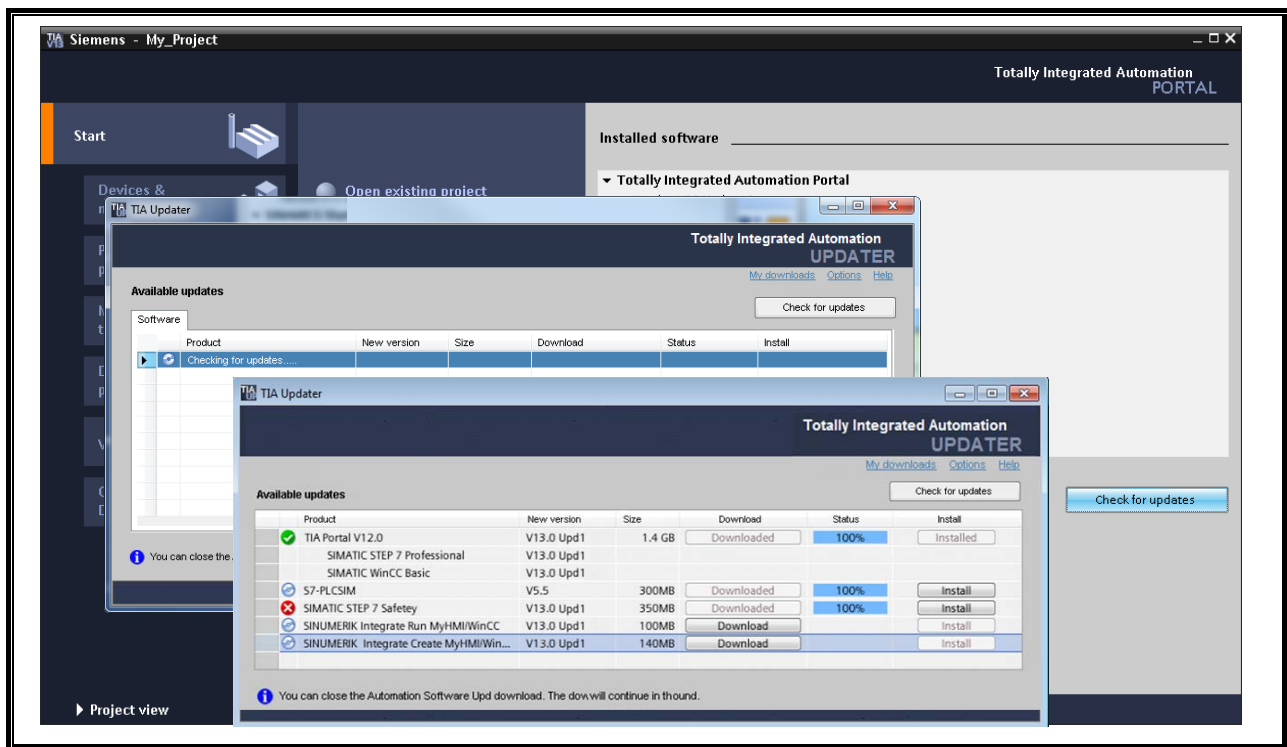
With IPE, you can convert and merge the communication of HMI configuration and PLC programming in different TIA Portal projects.

#### Requirements for IPE

To be able to adopt controller data from projects, the following requirements apply:

- The project is created with hardware configuration and PLC data which can be transferred to another project.
- It has to be ensured that the project in itself is consistent before the controller data is passed on through the device proxy data.

### 2.13.5. Update Tool



In the “Installed software” dialog you can search for updates.

- Start via: Portal view: Start > Installed software > Check for updates  
Project view: Help > Installed software > Check for updates
- Checks and informs about updates of installed software
- Download updates
- Pause / Continue downloads
- Installation of updates

# Contents

# 3

|           |                                                                        |            |
|-----------|------------------------------------------------------------------------|------------|
| <b>3.</b> | <b>Training Devices and Addressing.....</b>                            | <b>3-2</b> |
| 3.1.      | Training Area Setup with S7-1500.....                                  | 3-3        |
| 3.2.      | Configuration of the S7-1500 Training Device.....                      | 3-4        |
| 3.3.      | Configuration of the ET200SP Training Device.....                      | 3-5        |
| 3.4.      | Operating and Display Elements of the Training Device .....            | 3-6        |
| 3.5.      | Setup and Connection of the Conveyor Model.....                        | 3-7        |
| 3.5.1.    | Connection to Central I/Os of the S7-1500.....                         | 3-7        |
| 3.5.2.    | Connection to Distributed I/Os of the ET200SP .....                    | 3-7        |
| 3.6.      | Networking and IP Addresses of the Modules.....                        | 3-8        |
| 3.7.      | Training Area as Plant with Distribution Conveyor and Touchpanel ..... | 3-8        |

### 3. Training Devices and Addressing

**At the end of the chapter the participant will ...**

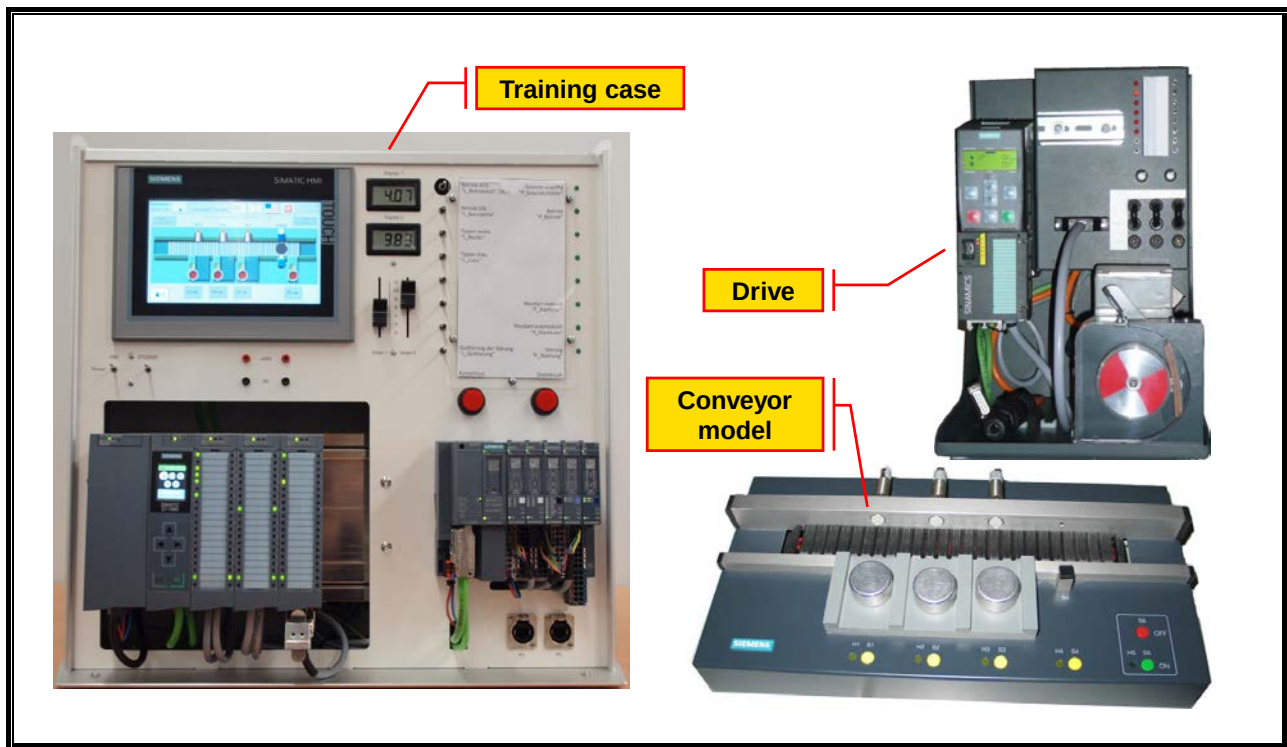


... be familiar with the configuration of the training area

... be familiar with the wiring of the training area components



### 3.1. Training Area Setup with S7-1500

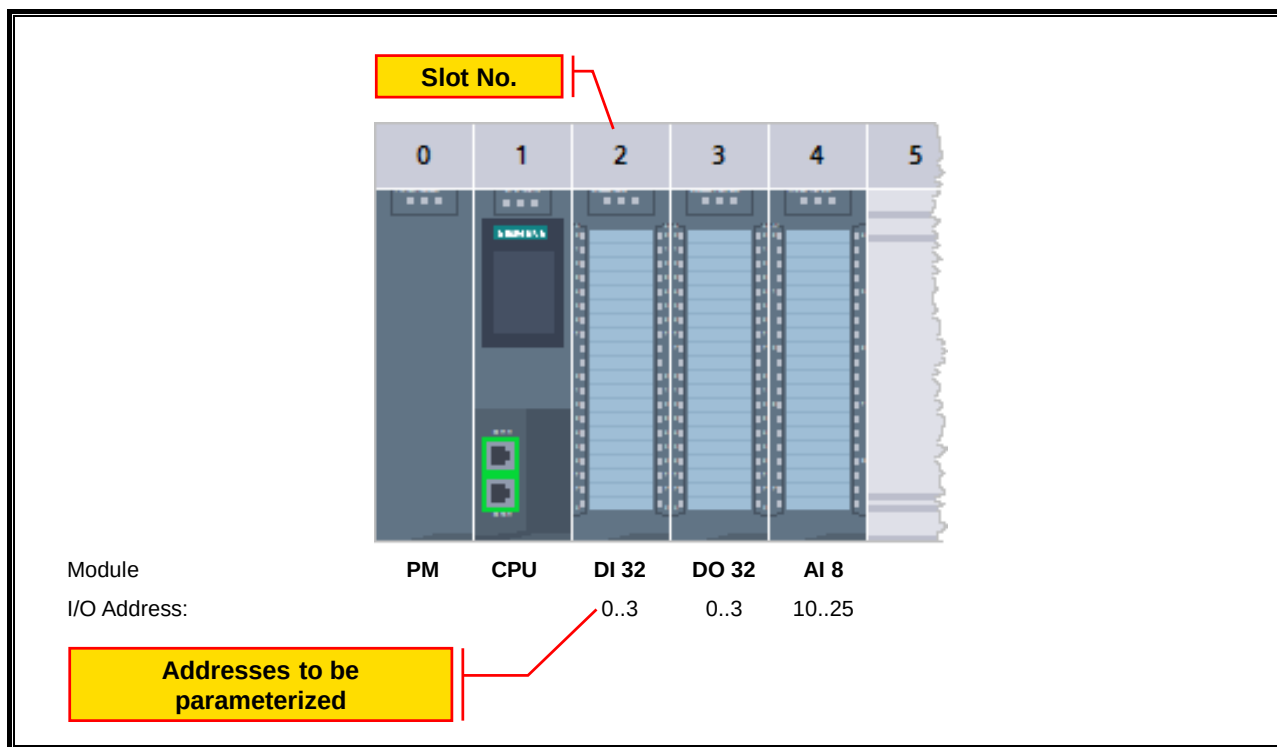


#### Components of the Training Area with S7-1500

The training area for this course contains the following components:

- SIMATIC Field PG
- Training case with S7-1500, ET200SP, touchpanel, operating elements (switches / pushbuttons and slide controls) and display elements (lights (LEDs) and voltage displays)
- Sinamics G120 drive
- Conveyor model

### 3.2. Configuration of the S7-1500 Training Device

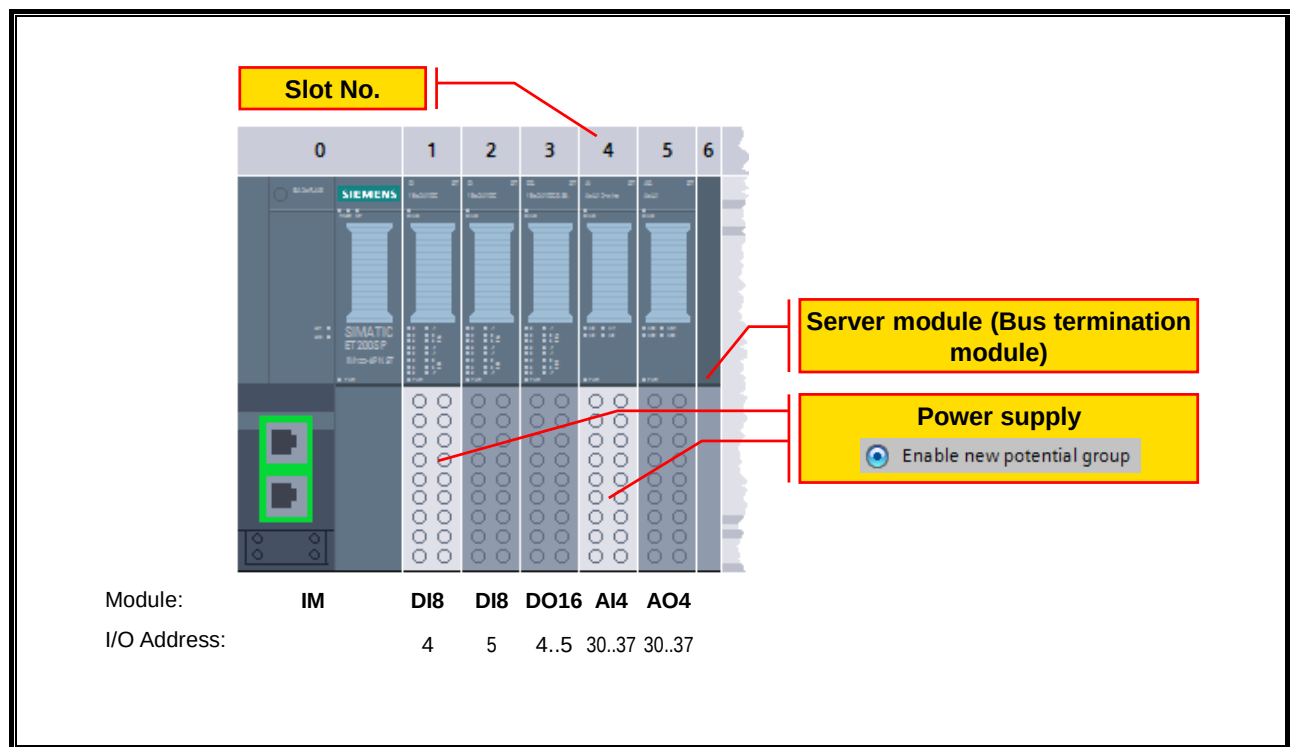


#### Addresses of the Central S7-1500 I/O Modules

Two digital 32-channel modules are available as central I/O. These are to begin as of Address =0.

Since digital channels are also available on the distributed I/O, the analog module of the central I/O is to begin as of Address =10.

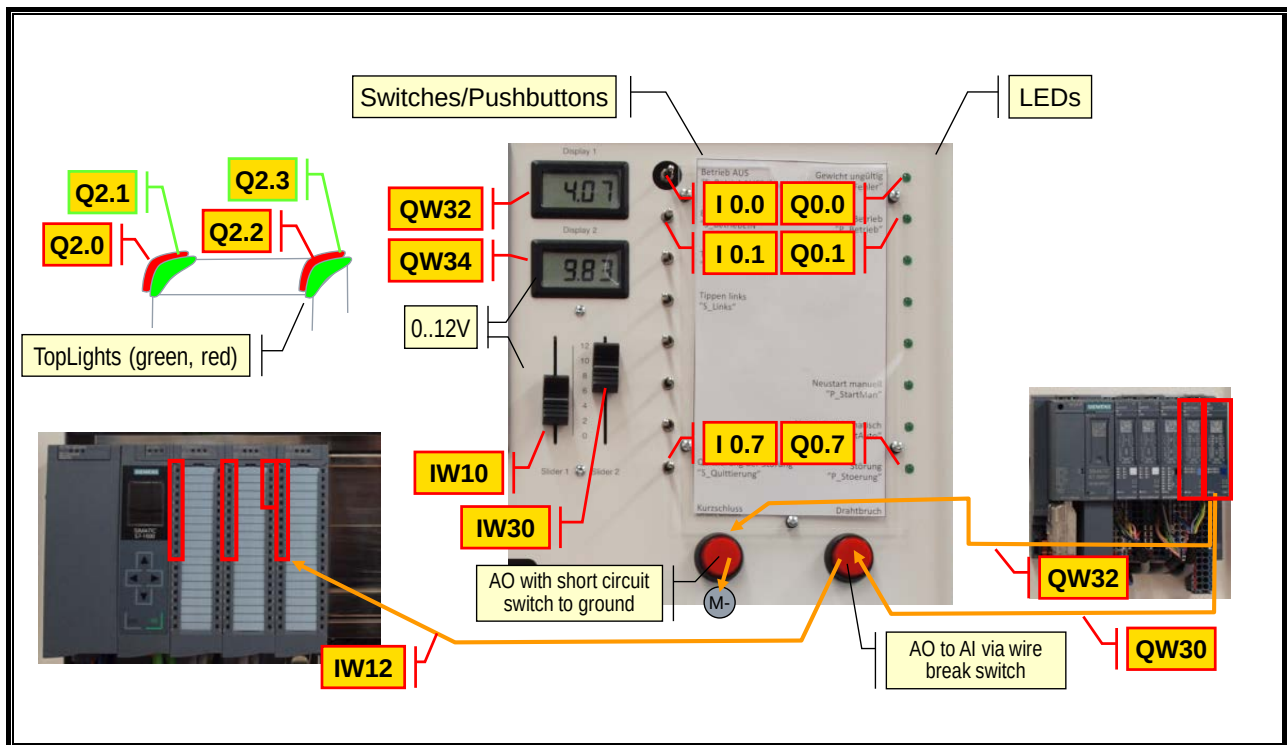
### 3.3. Configuration of the ET200SP Training Device



Three digital modules are available as distributed I/O. These are to follow the central digital I/O in the address space as of Address =4.

The analog distributed I/O is to begin as of Address =30.

### 3.4. Operating and Display Elements of the Training Device



#### Operating Elements

In addition to the touchpanel, separate operating elements are also available for operating the system:

- 8 switches or pushbuttons
- 2 potentiometers for setting or simulating analog input signals
- Wire break switch that interrupts the connection AO1 distributed I/O to AI2 central I/O
- Short circuit switch that short-circuits the AO2 of the distributed I/O to ground

#### Display Elements

In addition to the touchpanel, separate display elements are also available for the visualization of process information:

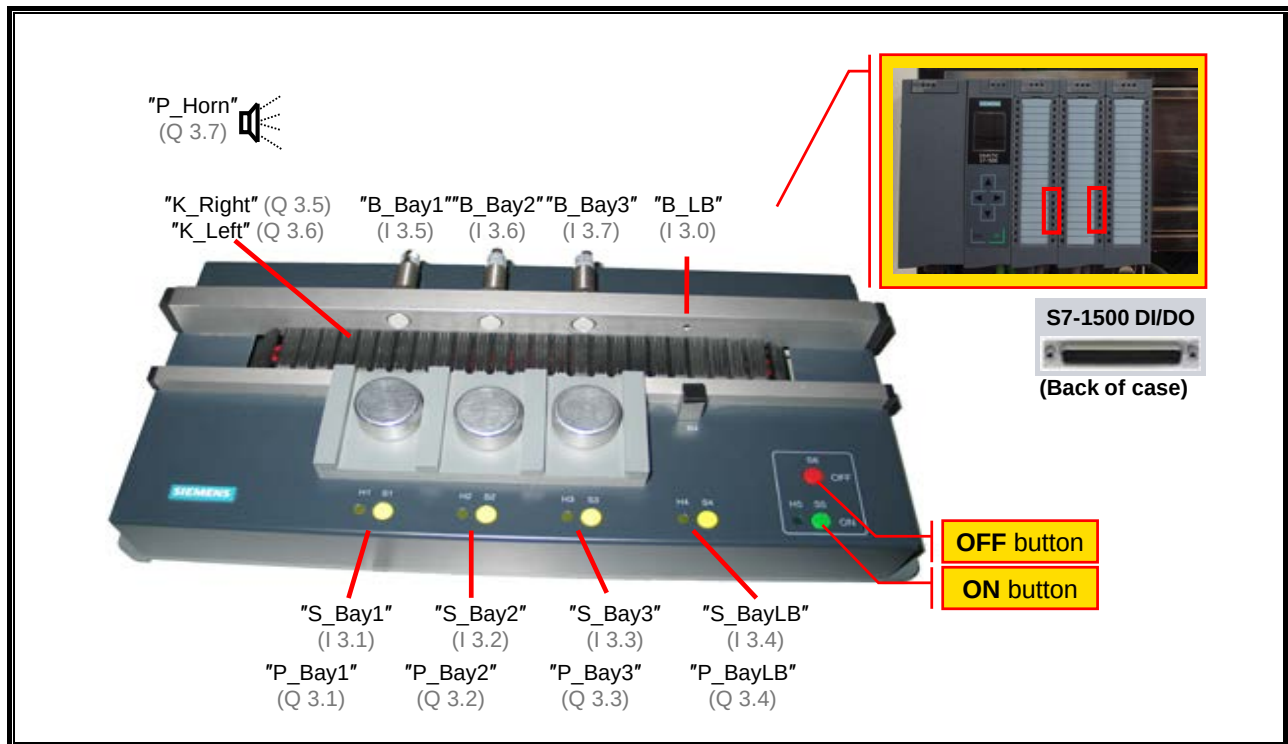
- 8 LEDs
- 2 digital voltage displays for displaying analog output signals
- On top of the training device there are, to the right and left, 2 LED bars "TopLights" (2x green, 2x red). These can be controlled by means of 4 DOs.

#### Addressing

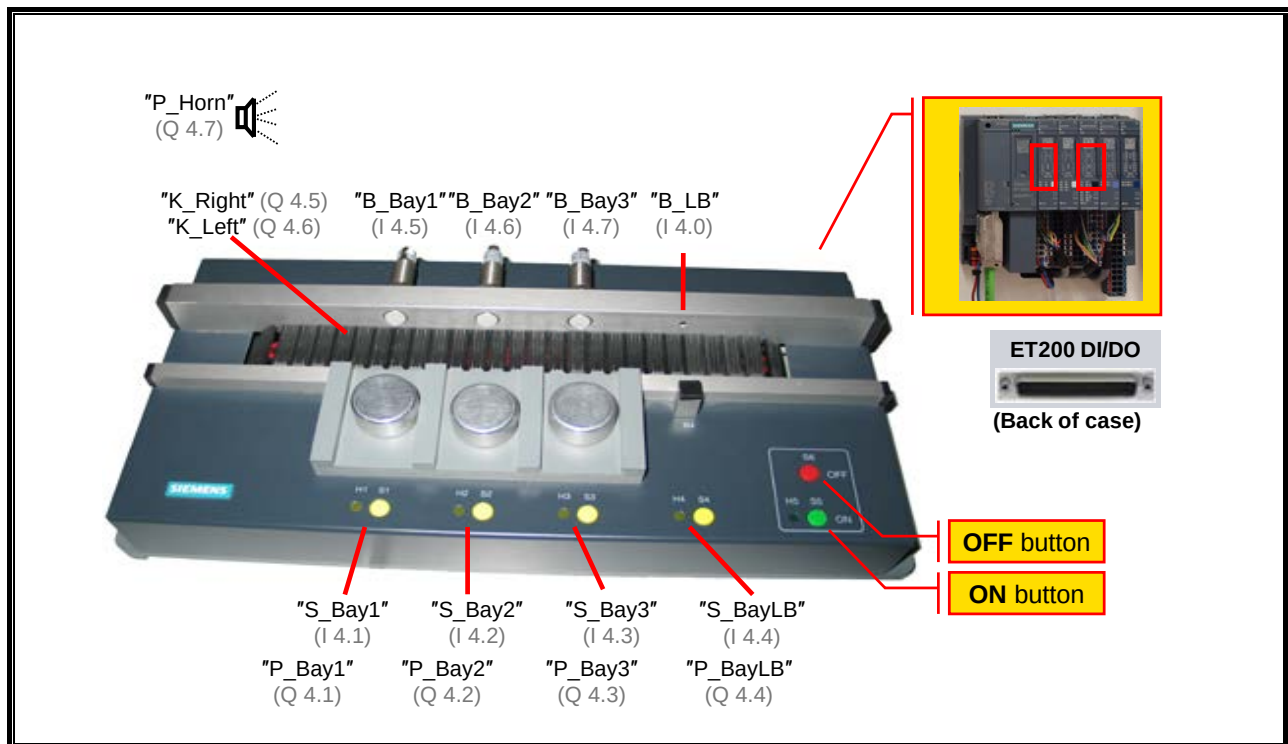
For the addressing shown in the picture, the relevant module address settings must be made in the device configuration.

### 3.5. Setup and Connection of the Conveyor Model

### 3.5.1. Connection to Central I/Os of the S7-1500



### 3.5.2. Connection to Distributed I/Os of the ET200SP





### Function Description:

The distribution conveyor is used to transport parts and can be operated in two different operating modes.

For now, the Simulator pushbuttons are used to select the operating mode and later, the associated buttons on the Touchpanel.

### Operation Switched Off "P\_Operation" (Q0.1) = Off

Now, the conveyor motor can be jogged to the right and left. For now, the Simulator pushbuttons are to be used for this, later, the associated buttons on the Touchpanel.

Later, the G120 drive will also be controlled as if it would drive the conveyor. The speed of the motor is preset through a parameter in the converter.

### Operation Switched On "P\_Operation" (Q0.1) = On

Now, parts are transported on the conveyor model from Bay 1 or 2 to the right until they are through the light barrier.

If a transport sequence takes longer than 6 seconds, the conveyor motor is automatically switched off and the fault is displayed on the Simulator as well as on the Touchpanel. Only after the fault is acknowledged with the Simulator pushbutton or on the Touchpanel, can a new transport sequence be started.

All parts that pass the light barrier when the operation is switched on ("P\_Operation" = On) are counted and weighed. The weight setting is done through the left slider "S\_Slider1". The part weights are saved in a data storage.

If the setpoint quantity (can be preset on the Touchpanel) of parts is reached, it is indicated on the conveyor model LED ("P\_BayLB") of the light barrier bay with a 1Hz flashing light. Only after the message is acknowledged with the pushbutton ("S\_BayLB") of the light barrier bay, can a new transport sequence be started.

The indicator lights at the Bays 1 and 2 show...

- Continuous light when a new part can be placed on the conveyor.
- 1Hz flashing light at the Bay at which a part is detected by the associated proximity switch, however, only as long as the conveyor has not yet been started.
- 2Hz flashing light as long as the conveyor motor is running.

The indicator light at the Light Barrier Bay shows...

- 2Hz flashing light as long as the conveyor motor is running.
- Continuous light when the SETPOINT quantity has been reached.

Later, the drive will also be controlled as if it would drive the conveyor. The speed of the motor can be set through the right slider "S\_Slider2".

# Contents

|           |                                                                                                                             |            |
|-----------|-----------------------------------------------------------------------------------------------------------------------------|------------|
| <b>4.</b> | <b>Devices &amp; Networks: Online Functions and Hardware Configuration.....</b>                                             | <b>4-3</b> |
| 4.1.      | Online Tools, Configuring and Parameterizing the Hardware .....                                                             | 4-4        |
| 4.2.      | Online Connection via Industrial Ethernet: IP Address and Subnet Mask .....                                                 | 4-5        |
| 4.2.1.    | Establishing an Online Connection: Assigning an IP Address for the PG.....                                                  | 4-6        |
| 4.2.1.1.  | Windows7 Operating System.....                                                                                              | 4-6        |
| 4.3.      | Online Access: Accessible Devices in the Portal View.....                                                                   | 4-7        |
| 4.3.1.    | Accessible Devices in the Project View: CPU Online Access: Reading-out the Diagnostics Buffer.....                          | 4-8        |
| 4.3.2.    | Accessible Devices in Project View: CPU Online Access: CPU-RUN/STOP, Memory Reset (MRES) .....                              | 4-9        |
| 4.3.3.    | Online Access to the CPU: IP Address, Name, Time, FW Update, Format Memory Card .....                                       | 4-10       |
| 4.4.      | CPU Memory Reset (MRES) using the Mode Selector Switch .....                                                                | 4-11       |
| 4.4.1.    | SIMATIC S7-1200/1500: Memory Concept for CPU Memory Reset.....                                                              | 4-12       |
| 4.4.2.    | SIMATIC S7-1200/1500: Memory Concept for CPU Reset to Factory Settings.....                                                 | 4-13       |
| 4.5.      | SIMATIC Card Reader .....                                                                                                   | 4-14       |
| 4.6.      | Components of the "Devices & Networks" Editor .....                                                                         | 4-15       |
| 4.6.1.    | Devices & Networks: Device View (Hardware Configuration) .....                                                              | 4-16       |
| 4.6.2.    | Hardware Catalog .....                                                                                                      | 4-17       |
| 4.6.3.    | Setpoint and Actual Configuration .....                                                                                     | 4-18       |
| 4.7.      | Setpoint Configuration: Creating a Hardware Station.....                                                                    | 4-19       |
| 4.7.1.    | Inserting / Deleting a Module .....                                                                                         | 4-20       |
| 4.7.2.    | Changing a Module .....                                                                                                     | 4-21       |
| 4.7.3.    | Area for Modules Not Plugged In.....                                                                                        | 4-22       |
| 4.8.      | Uploading the Actual Configuration without Parameterization into the Project (1): Inserting an Unspecified CPU .....        | 4-23       |
| 4.8.1.    | Uploading the Actual Configuration without Parameterization into the Project (2): Detecting Accessible Devices .....        | 4-24       |
| 4.8.2.    | Uploading the Actual Configuration without Parameterization into the Project (3): Reading-out the Actual Configuration..... | 4-25       |
| 4.9.      | Upload a Device as a New Station (1) (Hardware with Parameterization and Software)...                                       | 4-26       |
| 4.9.1.    | Upload a Device as a New Station (2) (Hardware with Parameterization and Software)...                                       | 4-27       |
| 4.10.     | Compiling the Hardware Configuration and Downloading it into the CPU .....                                                  | 4-28       |
| 4.11.     | CPU Properties: Ethernet Address .....                                                                                      | 4-29       |
| 4.11.1.   | CPU Properties: System and Clock Memory .....                                                                               | 4-30       |
| 4.11.2.   | CPU Properties: Maximum Cycle Time .....                                                                                    | 4-31       |
| 4.11.3.   | CPU Properties: Protection.....                                                                                             | 4-32       |
| 4.11.3.1. | Accessing a Protected CPU by Means of Password Specification .....                                                          | 4-33       |
| 4.11.3.2. | Status LEDs of the S7-1500 CPU.....                                                                                         | 4-34       |
| 4.11.4.   | Status LEDs of the Central DI/DO Modules of the S7-1500 CPU .....                                                           | 4-34       |
| 4.12.     | Task Description: Creating a Project with an S7-1500 Station.....                                                           | 4-35       |
| 4.12.1.   | Exercise 1: Deleting Old Projects .....                                                                                     | 4-36       |
| 4.12.2.   | Exercise 2: Connecting the PG and Setting the IP Address of the PG .....                                                    | 4-37       |
| 4.12.3.   | Exercise 3: Erasing the SIMATIC Memory Card of the CPU.....                                                                 | 4-38       |
| 4.12.4.   | Exercise 4: Resetting the CPU using the Mode Selector Switch.....                                                           | 4-39       |
| 4.12.5.   | Exercise 5: Determining the CPU Firmware Version and Assigning the IP Address (Node Initialization) .....                   | 4-40       |
| 4.12.6.   | Exercise 6: Creating a New Project .....                                                                                    | 4-41       |



|           |                                                                                              |      |
|-----------|----------------------------------------------------------------------------------------------|------|
| 4.12.7.   | Exercise 7: Creating (Adding) a New Device.....                                              | 4-42 |
| 4.12.8.   | Exercise 8: Reading-out the Actual Configuration.....                                        | 4-43 |
| 4.12.9.   | Exercise 9: CPU Properties: Parameterizing the Clock Memory Byte .....                       | 4-44 |
| 4.12.10.  | Exercise 10: CPU Properties: Parameterizing the Display Language and Display Protection..... | 4-45 |
| 4.12.11.  | Exercise 11: Addresses of the DI Module.....                                                 | 4-46 |
| 4.12.12.  | Exercise 12: Addresses of the DO Module.....                                                 | 4-47 |
| 4.12.13.  | Exercise 13: Addresses of the AI Module.....                                                 | 4-48 |
| 4.12.14.  | Exercise 14: Compiling the Device Configuration and Downloading it into the CPU .....        | 4-49 |
| 4.12.15.  | Exercise 15: Switching the CPU to RUN using the CPU-Display.....                             | 4-51 |
| 4.13.     | Additional Information .....                                                                 | 4-53 |
| 4.13.1.   | 'View' Settings of the "Hardware catalog" Task Card .....                                    | 4-54 |
| 4.13.2.   | Firmware Update .....                                                                        | 4-55 |
| 4.13.2.1. | Firmware Update → Offline using SIMATIC Memory Card with the Windows Explorer ....           | 4-56 |
| 4.13.2.2. | Firmware Update → Offline using STEP7 by means of Card Creation in the TIA Portal ...        | 4-57 |
| 4.13.2.3. | Firmware Update → Online using STEP 7 in the TIA Portal .....                                | 4-58 |
| 4.13.2.4. | Firmware Update → Important Facts.....                                                       | 4-59 |

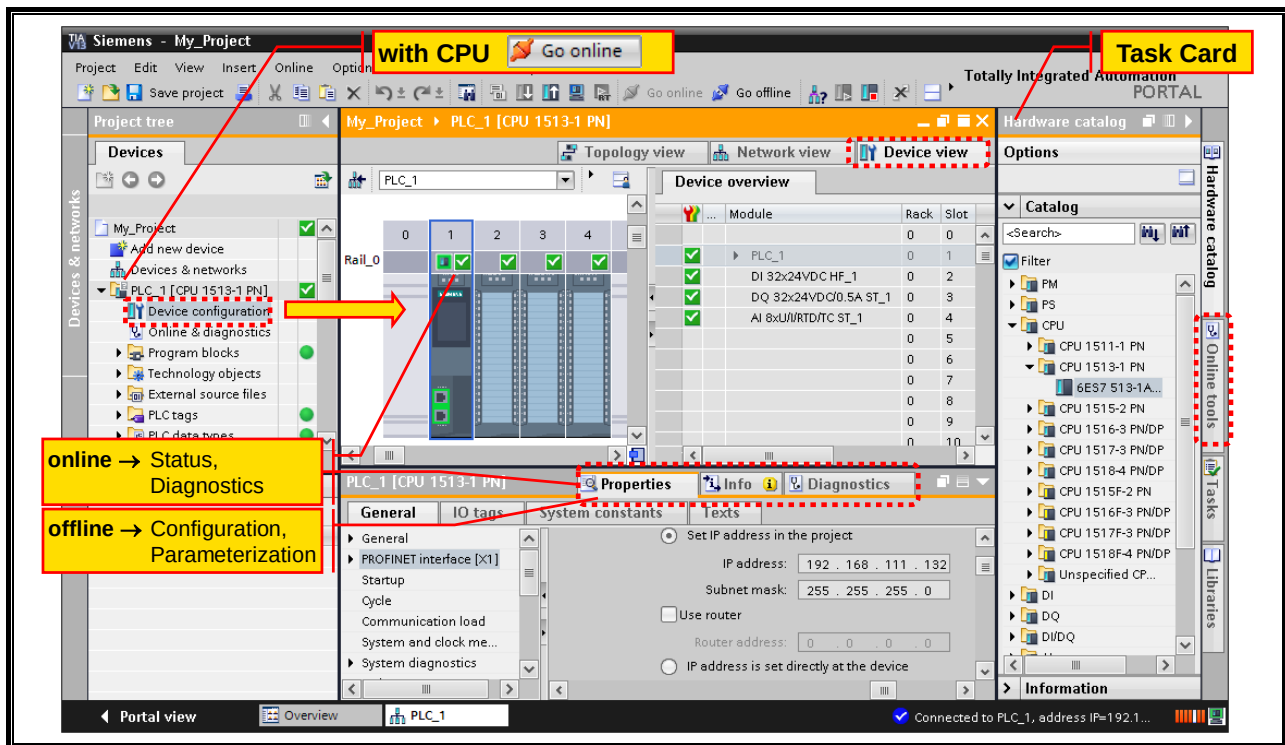
## 4. Devices & Networks: Online Functions and Hardware Configuration

**At the end of the chapter the participant will ...**



- ... be able to establish an online connection between PG and CPU via Industrial Ethernet
- ... be able to use online functions to start and stop the CPU and to reset it to factory settings
- ... be able to create and parameterize a setpoint configuration
- ... be familiar with addressing the input and output modules of an S7-1500 and be able to do it

## 4.1. Online Tools, Configuring and Parameterizing the Hardware



### Online Tools

If it is possible to establish an online connection to the CPU, diagnostics and status information of all modules can be called.

With CPUs that can be accessed online, the mode can also be controlled using the "Online tools" task card and further status information (cycle time statistics and memory load) can be called.

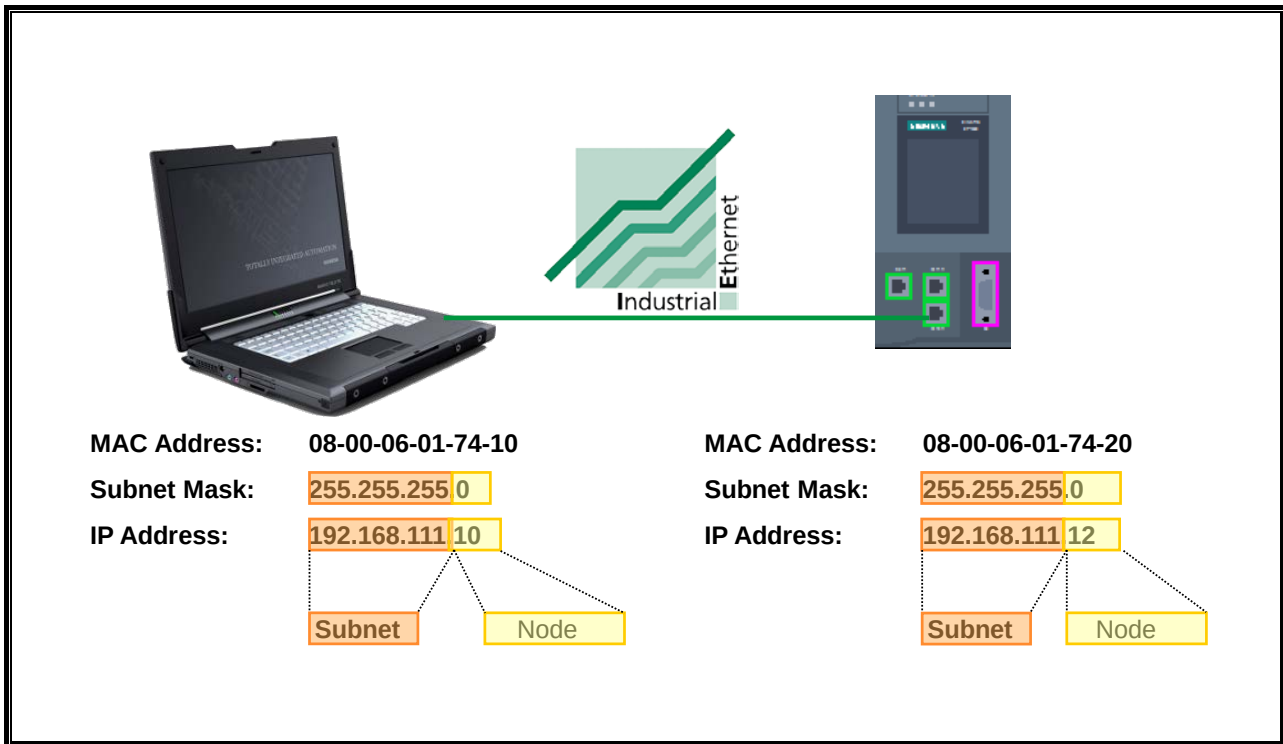
### Configuring and Parameterizing the Hardware

Almost all devices or components of an automation solution such as controllers or touchpanels can be assigned parameters. The parameter assignment of the devices and network settings required for commissioning is handled using the "Devices & networks" editor.

With this, for example, all components of an Ethernet network are assigned IP addresses via which they communicate during later operation.

But even inside the automation device, address areas of the I/O modules must be specified and the cycle monitoring time of the CPU must be set, for example.

## 4.2. Online Connection via Industrial Ethernet: IP Address and Subnet Mask



### Internet Protocol

The Internet Protocol (IP) is the basis for all TCP/IP networks. It creates the so-called datagrams (data packets specially tailored to the Internet protocol) and handles their transport within the local subnet or their "routing" (forwarding) to other subnets.

### IP Addresses

IP addresses are not assigned to a specific computer, but rather to the network interfaces of the computer. A computer with several network connections (for example routers) must therefore be assigned an IP address for each connection.

IP addresses consist of 4 bytes. With the dot notation, each byte of the IP address is expressed by a decimal number between 0 and 255. The four decimal numbers are separated by dots (see picture).

### MAC Address

Every Ethernet interface is assigned a fixed address by the manufacturer that is unique worldwide. This address is referred to as the hardware or MAC address (Media Access Control). It is stored on the network card and uniquely identifies the Ethernet interface in a local network. Cooperation among the manufacturers ensures that the address is unique worldwide.

### Subnet Mask

The subnet mask specifies which IP addresses in the local network can be reached. It separates the IP address into the network and device part.

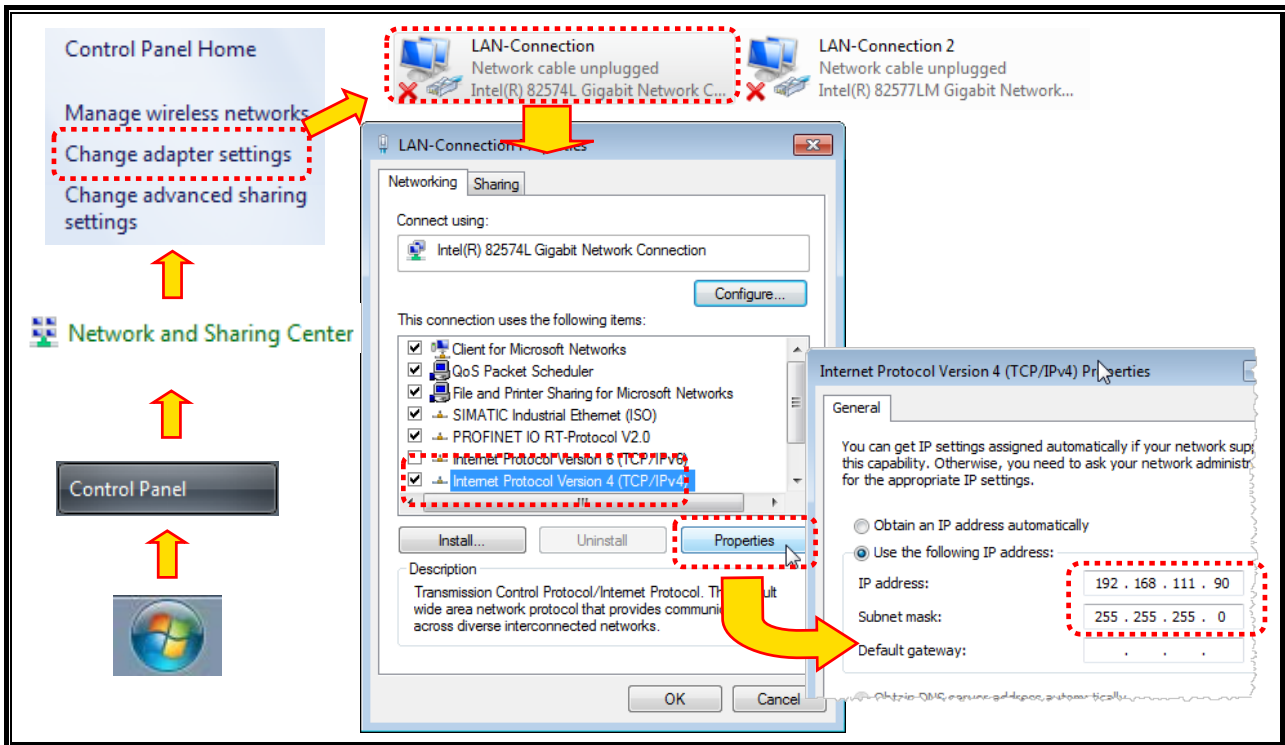
Only IP addresses whose network part is the same can be reached.

For example: Subnet mask = **255.255.255.0** and IP address = **192.168.111.10**

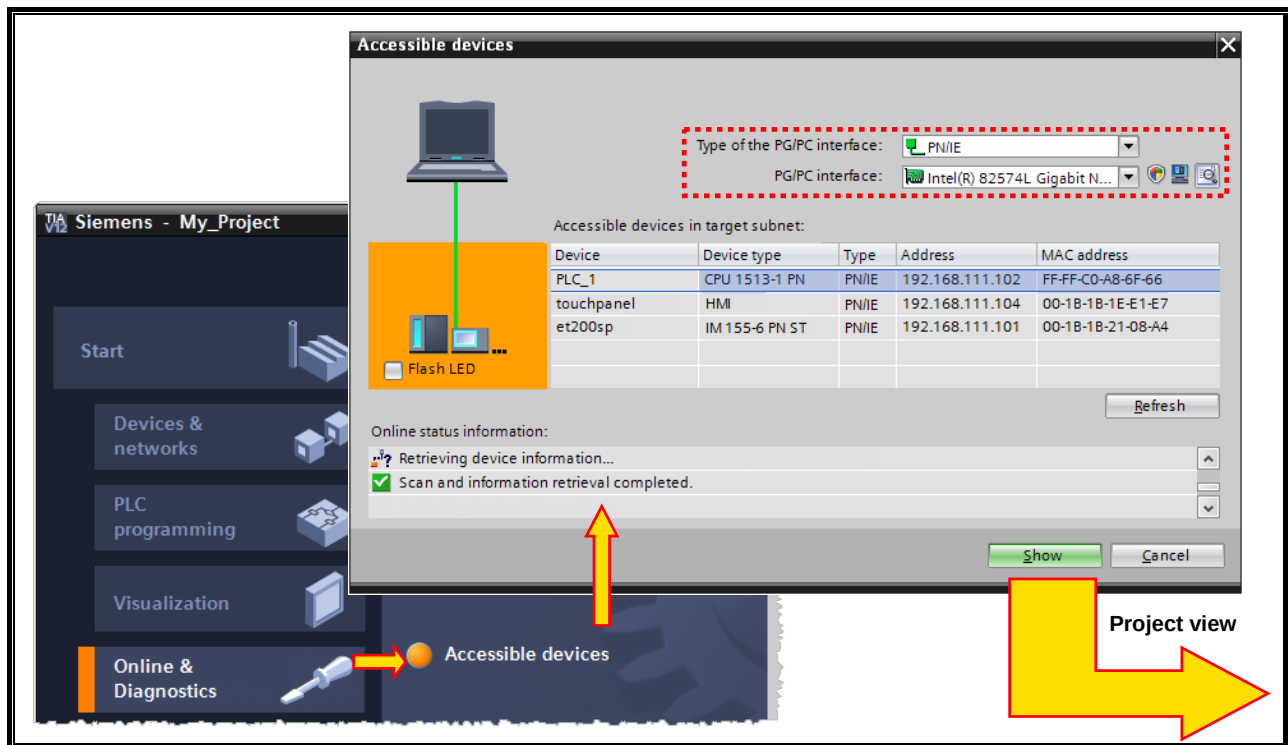
reachable IP addresses: **192.168.111.1** to **192.168.111.254**

## 4.2.1. Establishing an Online Connection: Assigning an IP Address for the PG

### 4.2.1.1. Windows7 Operating System



### 4.3. Online Access: Accessible Devices in the Portal View



#### Accessible Devices in the Portal View

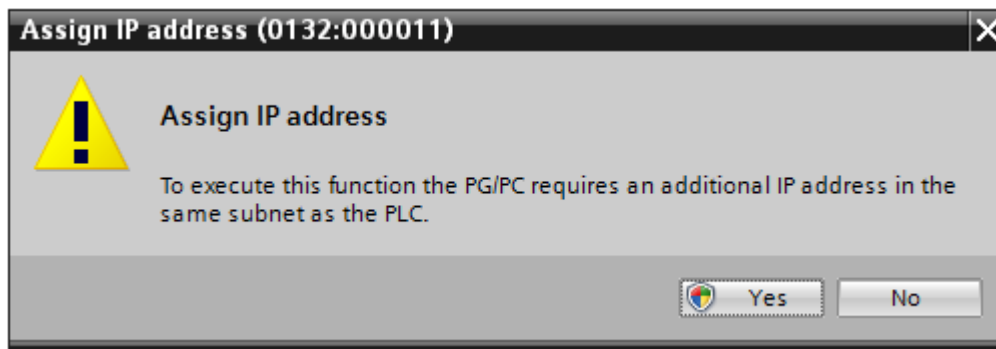
This function provides the option of fast access (for example for service purposes) even when there is no offline project data for the target systems on the PG.

All accessible, programmable modules (CPUs, FMs CPs, HMI devices) are listed in the Portal view, even if they are located in other subnets.

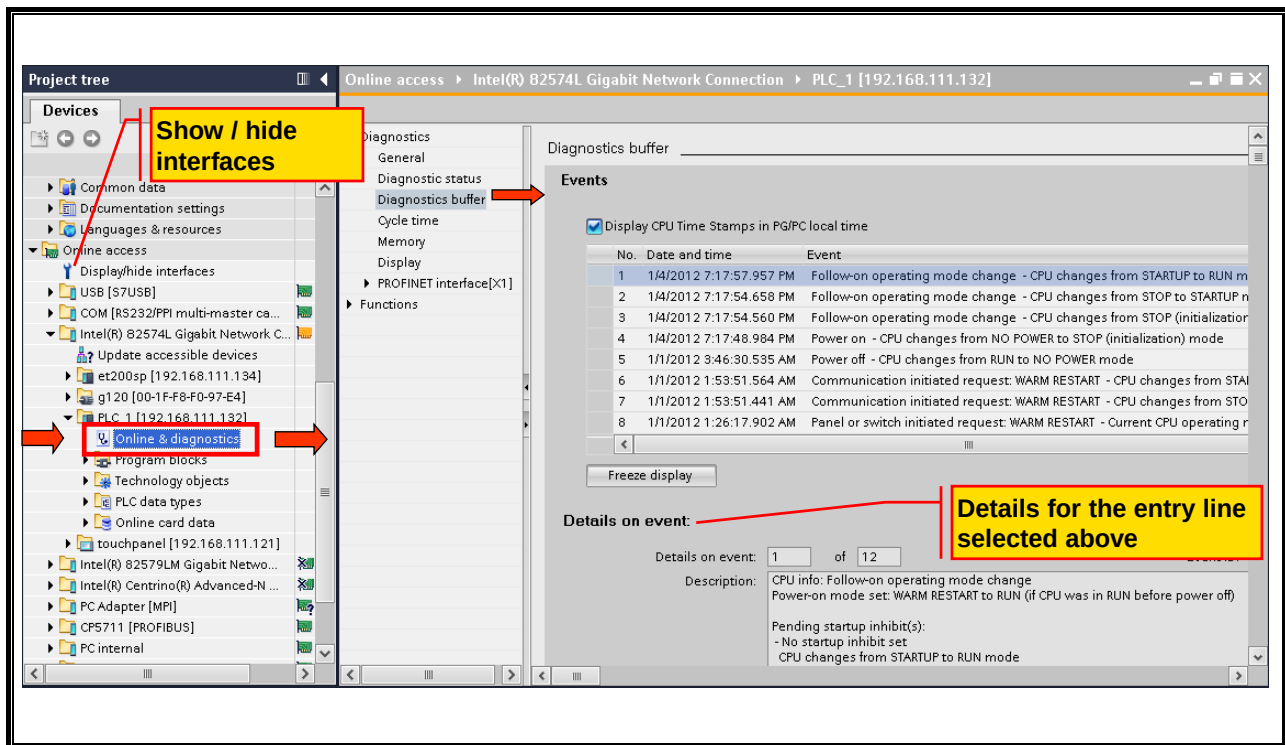
#### Access Online Functions → Button

Whenever there is an attempt to access a module online with the "Show" button and this is located in a different subnet from the PG, a dialog opens asking whether an additional IP address should be assigned.

Following confirmation, an additional IP address is assigned to the PG that is located in the same subnet as the address of the CPU. After that, all online functions can be used.



### 4.3.1. Accessible Devices in the Project View: CPU Online Access: Reading-out the Diagnostics Buffer



#### Online Access to the CPU

If the PG and the target system (for example CPU) are located in the same subnet, various Online & diagnostics functions are available in the "Accessible devices" function.

- in the working area of the TIA Portal
- in the "Online tools" task card

#### Diagnostics Buffer

The diagnostics buffer is a buffered memory area on the CPU organized as a circular buffer. It contains all diagnostics events (error alarms, diagnostics interrupts, start-up information etc.) of the CPU in the order in which they occurred. The highest entry is the last event to occur.

All events can be displayed on the programming device in plain language and in the order in which they occurred.



The size of the diagnostics buffer depends on the CPU. As well, not all of the diagnostics buffer is buffered with PowerOFF (only a part is retentive).

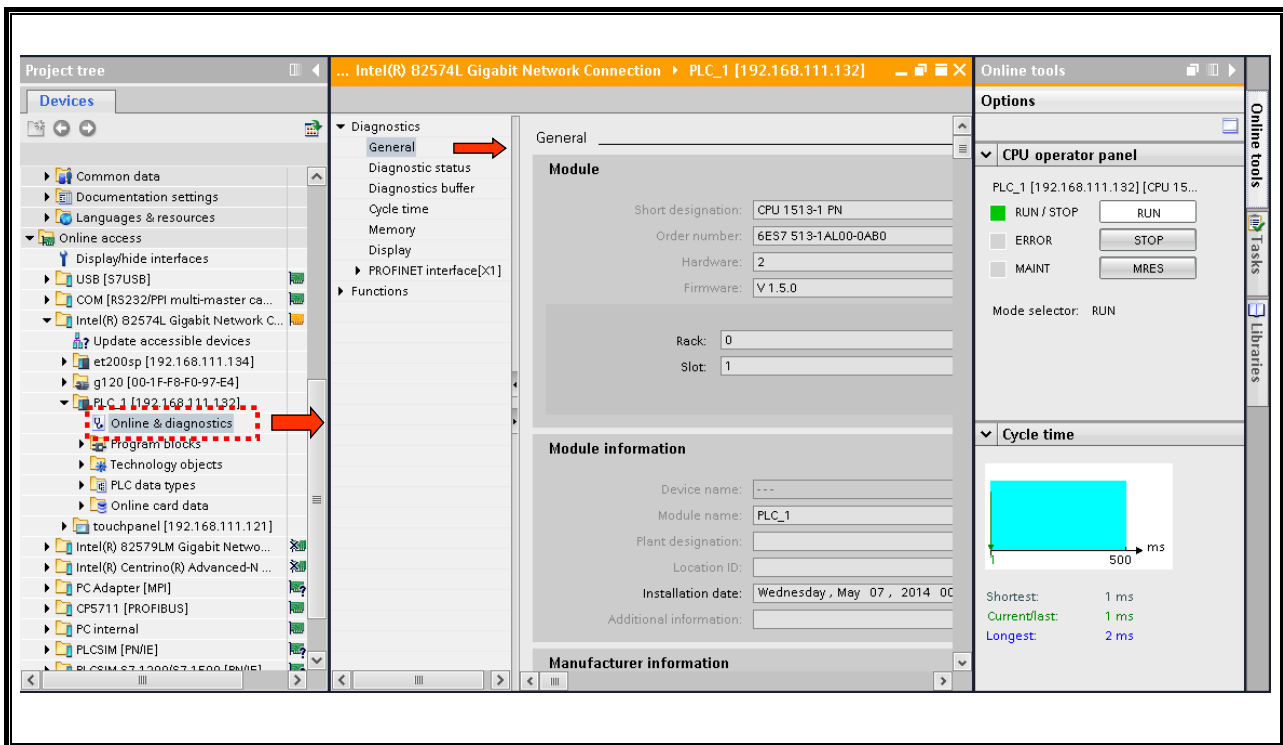
- Number of entries, 1000 to 3200
- Of that, retentive 250 to 500

#### Details on Event

Some additional information is also provided for the selected event in the "Details on event" box:

- Event name and number
- Additional information depending on the event, such as, the address of the instruction that caused the event etc.

### 4.3.2. Accessible Devices in Project View: CPU Online Access: CPU-RUN/STOP, Memory Reset (MRES)



#### CPU Operator Panel: Mode Selector Switch

The operating mode of the CPU can be changed.

- **RUN → STOP:**  
If there is a change from RUN to STOP, the CPU terminates the running user program.
- **STOP → RUN:**  
If there is a change from STOP to RUN, the CPU performs a restart.

#### Cycle Time:

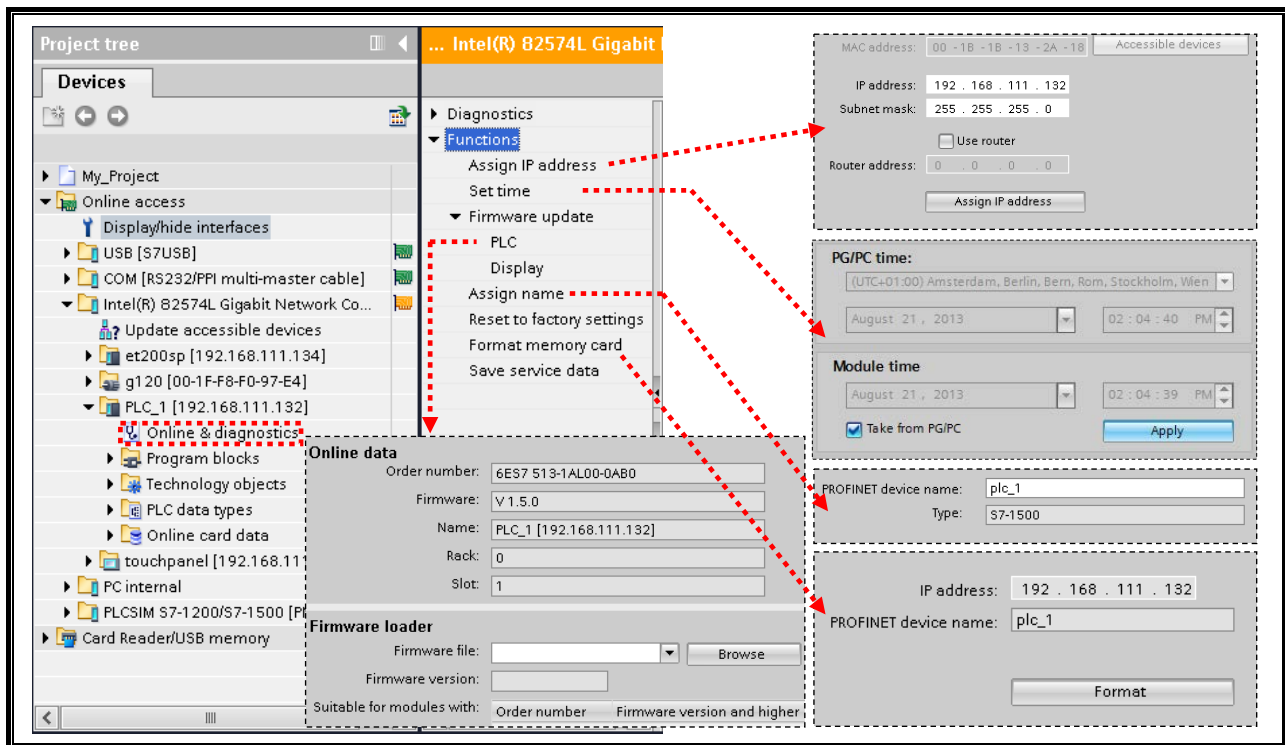
"Shortest", "Current" and "Longest" are the cycle times since the last CPU restart

#### With a Memory Reset (MRES), a CPU reset is carried out:

- All user data (even the retentive) is deleted (delete work memory)  
(process images, memory bits, timers, counters, all program/data blocks)
- Retained are: parameter assignment of the X1 (Ethernet) interface, the retentive part of the diagnostics buffer, time-of-day



### 4.3.3. Online Access to the CPU: IP Address, Name, Time, FW Update, Format Memory Card



- **Set Time (of Day)**  
Each S7 CPU has a real-time clock that can be set here.
- **Assign IP Address**  
As long as no IP address has been specified already by a hardware configuration that was downloaded earlier, this can be assigned or modified here (this function is also available when the PG/PC and the CPU are not assigned to the same subnet).
- **Reset to Factory Settings**  
Unlike the "memory reset", all the memory areas of the CPU (work, load and retentive memory, diagnostics buffer and time) are deleted. Optionally (see dialog in the picture), the IP address can also be deleted so that the CPU then only has a MAC address (Media Access Control).
- **Format Memory Card**  
The CPU memory card can also be deleted in the CPU via this online function. After that, the CPU only has its IP address. All other data (including the device configuration) is deleted. The card cannot be deleted in the card reader via the Project tree. Device configuration and blocks have a gray background, that is, are write-protected (only status information or open with a double-click).
- **Assign Name**  
In PROFINET, each device must be assigned a unique device name that is stored retentively on the device. The device name identifies a distributed I/O module (PROFINET IO) and allows module replacement without a PG/PC.
- **Firmware Update**  
Here the firmware version of the CPU can be updated. Under "Diagnostics -> General", the current firmware version of the CPU is displayed in the module data.



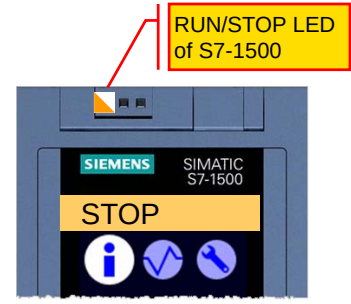
- **Attention!**  
If both components have to be updated, then update the Display first and only after that the CPU.

## 4.4. CPU Memory Reset (MRES) using the Mode Selector Switch



1. Set the **mode selector switch** to **STOP**
2. Press the mode selector switch in the **MRES** position until the **RUN/STOP LED** has flashed 2x slowly  
then let go again
3. Press the mode selector switch in the **MRES** position until the **RUN/STOP LED** begins to flash quickly  
then let go again
4. Set the **mode selector switch** to **RUN**  
A CPU restart is carried out

**within 3 sec !!!**



**Result:**

With inserted PROGRAM card  
→ **Memory Reset**

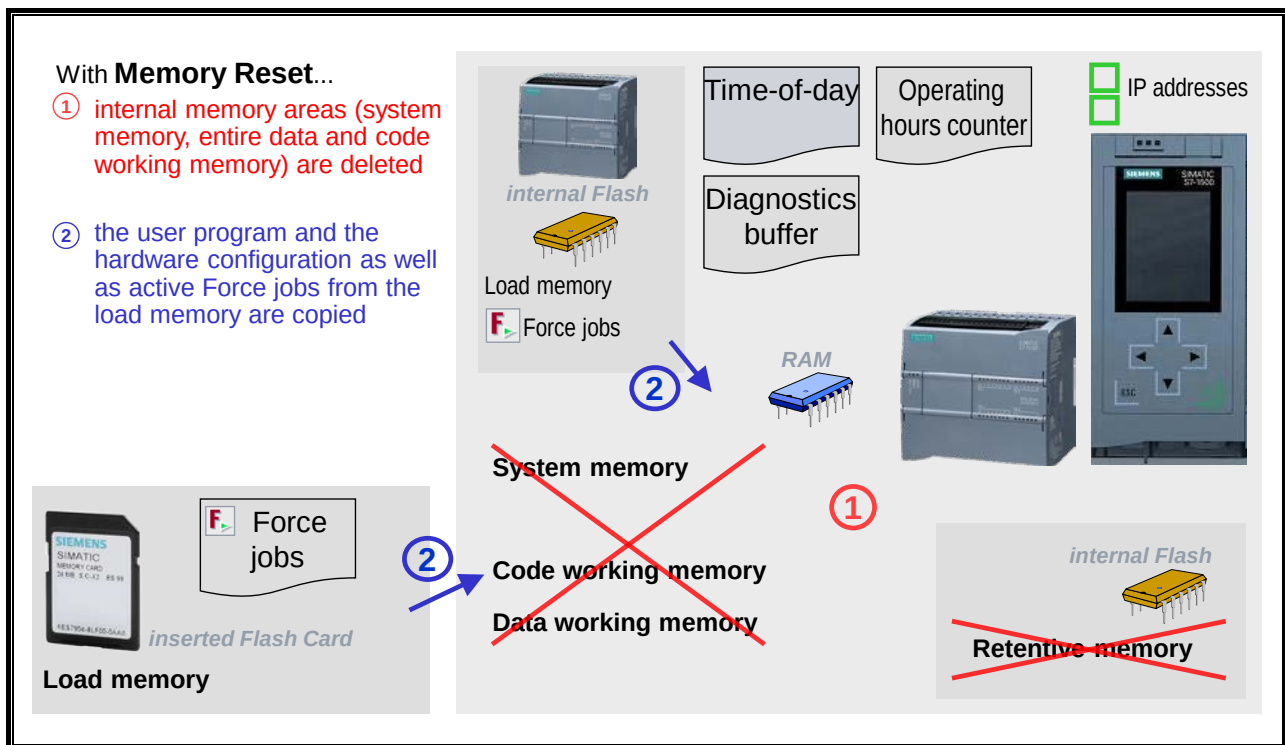
Without inserted card  
→ **Reset to factory settings**

### Particular Feature for CPU Memory Reset (MRES) using the Mode Selector Switch:

- **when Memory Card (MC) is inserted => Memory Reset**
  - All user data is deleted (work, retentive memory) (process image, bit memories, counters, timers, all program/data blocks)
  - Retained are: parameter assignment of the X1 (Ethernet) interface, the retentive part of the diagnostics buffer
  - The CPU copies all load memory data relevant for execution (memory card) into the internal RAM work memory. (Data relevant for execution: device configuration, program blocks, data blocks).
- **when no Memory Card (MC) is inserted => Reset to factory settings**
  - All memory areas of the CPU (work, retentive memory, diagnostics buffer, time-of-day) and the IP address are deleted.

After the MC is inserted, the load memory data relevant for execution is reloaded into the internal RAM work memory from the memory card:  
Device configuration (with IP address), program blocks, data blocks

#### 4.4.1. SIMATIC S7-1200/1500: Memory Concept for CPU Memory Reset



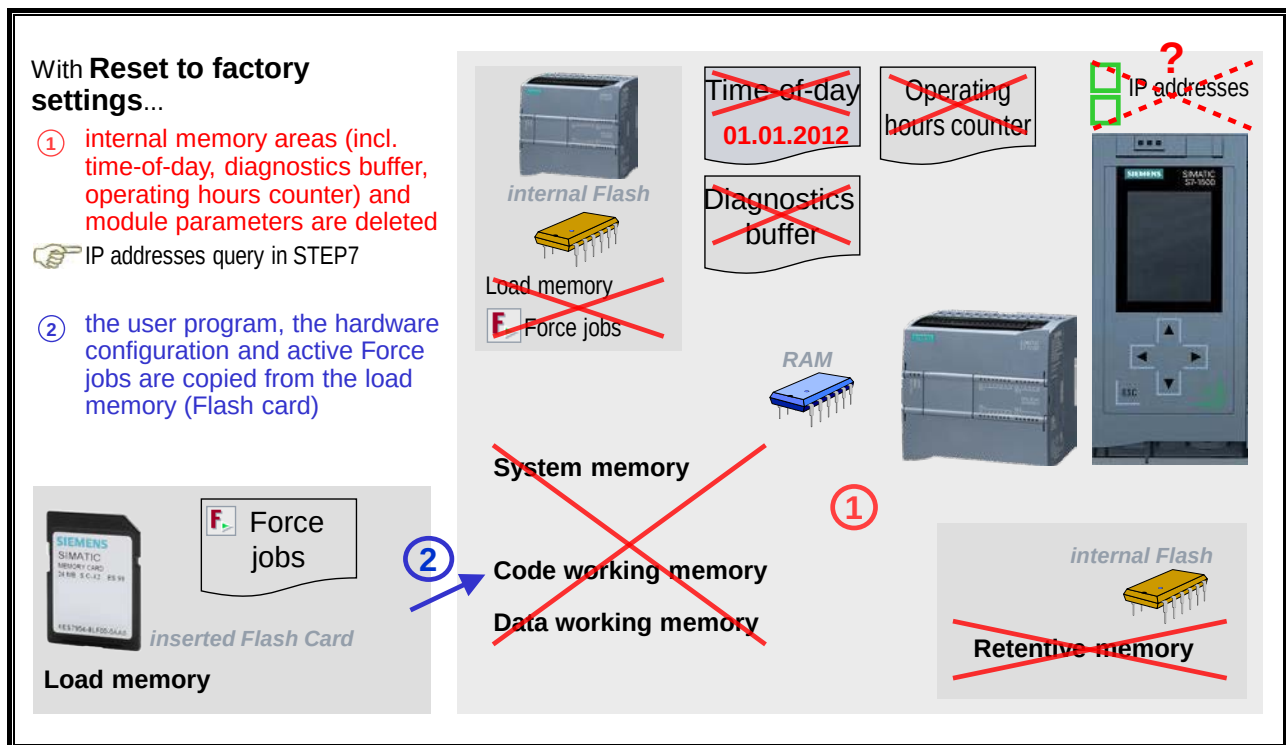
##### CPU Memory Reset

- **What to do:**
  - STEP7 online function → MRES in "CPU operator panel" of "Test" and "Online tools" Task Cards
  - Display (only S7-1500) → Main menu "Settings", submenu "Memory reset"
  - CPU mode selector switch (with inserted memory card)
- **Impact**
  - The entire RAM work memory is deleted, that is, all user data (process images, bit memories, counters, timers, all program/data blocks, even the retentive ones)
  - After that, the CPU copies all data relevant for execution into the internal RAM work memory from the memory card. (Data relevant for execution: device configuration, program blocks, data blocks....).
  - Retained are: parameter assignment of the X1 (Ethernet) interface, the retentive part of the diagnostics buffer, operating hours counter, CPU time-of-day

##### Caution:

Active Force jobs are stored in the load memory and after a memory reset and subsequent STOP-RUN transition are once again active.

#### 4.4.2. SIMATIC S7-1200/1500: Memory Concept for CPU Reset to Factory Settings



##### CPU Reset to Factory Settings

- **What to do:**
  - STEP7 online function → MRES in "CPU operator panel" of "Test" and "Online tools" Task Cards
  - Display (only S7-1500) → Main menu "Settings", submenu "Memory reset" → Factory Defaults
  - Mode selector switch (only without memory card)
- **Impact**
  - The entire RAM work memory is deleted, that is, all user data (process images, bit memories, counters, timers, all program/data blocks, even the retentive ones, parameter assignment of the X1 (Ethernet) interface, the retentive part of the diagnostics buffer)

If a memory card is inserted (or is already inserted), the CPU copies all data relevant for execution into the internal RAM work memory from the memory card. (Data relevant for execution: device configuration incl. IP address, program blocks, data blocks....).

##### Note:

With an S7-1500, active Force jobs are once again active after a reset to factory settings and a subsequent STOP-RUN transition.

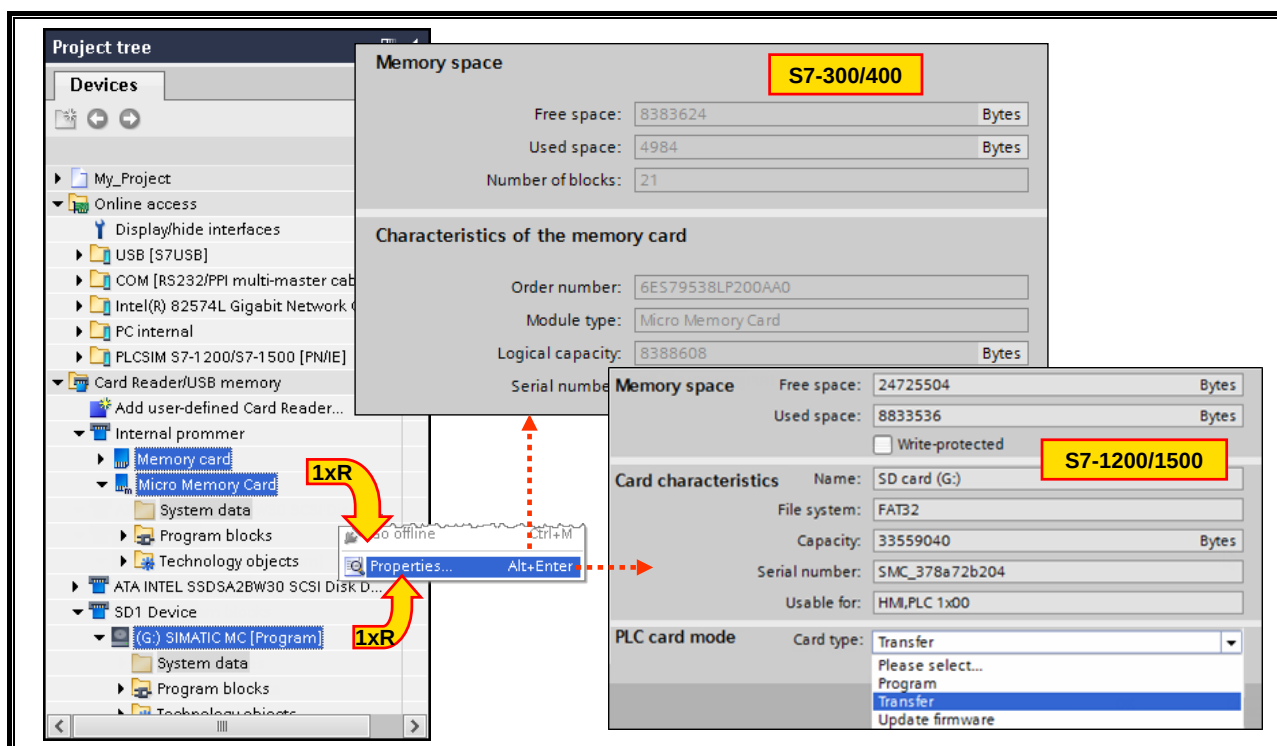
With an S7-1200, the resetting to factory settings is not possible with an inserted Flash card (that is, with an external load memory). The internal load memory is deleted as well as the active Force jobs that are stored in the internal Flash.

##### Particular Feature with the Use of an S7-1200 with Flash Card as Load Memory:

Since the resetting to factory settings with an inserted Flash card is not possible with an S7-1200, this must be removed beforehand.

After subsequent insertion of the card, necessary CPU memory reset or power off and a STOP-RUN transition, the active Force jobs which were stored on the Flash card are once again active.

#### 4.5. SIMATIC Card Reader



## SIMATIC Card

The SIMATIC Memory Card of an S7-1200 is an SD memory card pre-formatted by Siemens. It can be read and written with the Windows Explorer but under no circumstances can it be formatted with it!

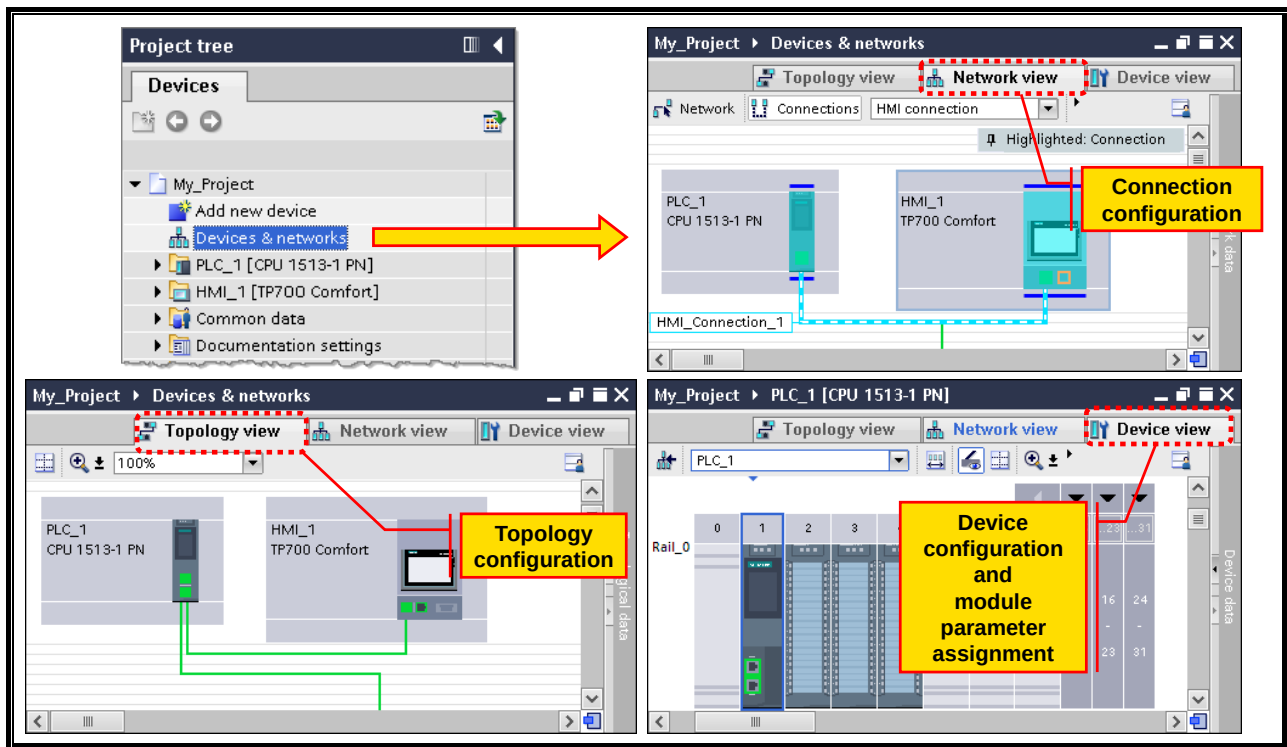
The SIMATIC Micro Memory Card (MMC) of an S7-300 is a memory card formatted by Siemens. It cannot be read with Windows and under no circumstances can it be formatted with Windows!

**Card type of the SIMATIC Card for S7-1200/1500:**

The SIMATIC card is used as a Program card or a Transfer card or for Firmware Updates. Before the relevant data is stored on the SD card, the card type must be selected as shown in the picture.

- **SIMATIC card as Program card:**  
The card contains all configuration and parameterization data for the station as well as the entire user program with documentation. During operation, the card must remain inserted in the CPU because it is used as a replacement for the internal CPU load memory which remains unused.
- **SIMATIC card as Transfer card (only for S7-1200):**  
The card contains the same data as a Program card but it doesn't have to remain inserted during operation. After inserting the card and subsequent Power ON, all data is copied into the internal load memory of the CPU. Then the card has to be removed and a restart has to take place.
- **SIMATIC card to Update Firmware:**  
The SIMATIC card contains the files required for a firmware update. After execution (instructions are included as a Text file) the SIMATIC card must be removed.

## 4.6. Components of the "Devices & Networks" Editor



### Components of the Devices & Networks Editor

The "Devices & networks" editor consists of a Device, Network and Topology view.

#### Network View

The network view is used for networking devices.

- Connection configuration

#### Device View

The device view is used for configuring devices.

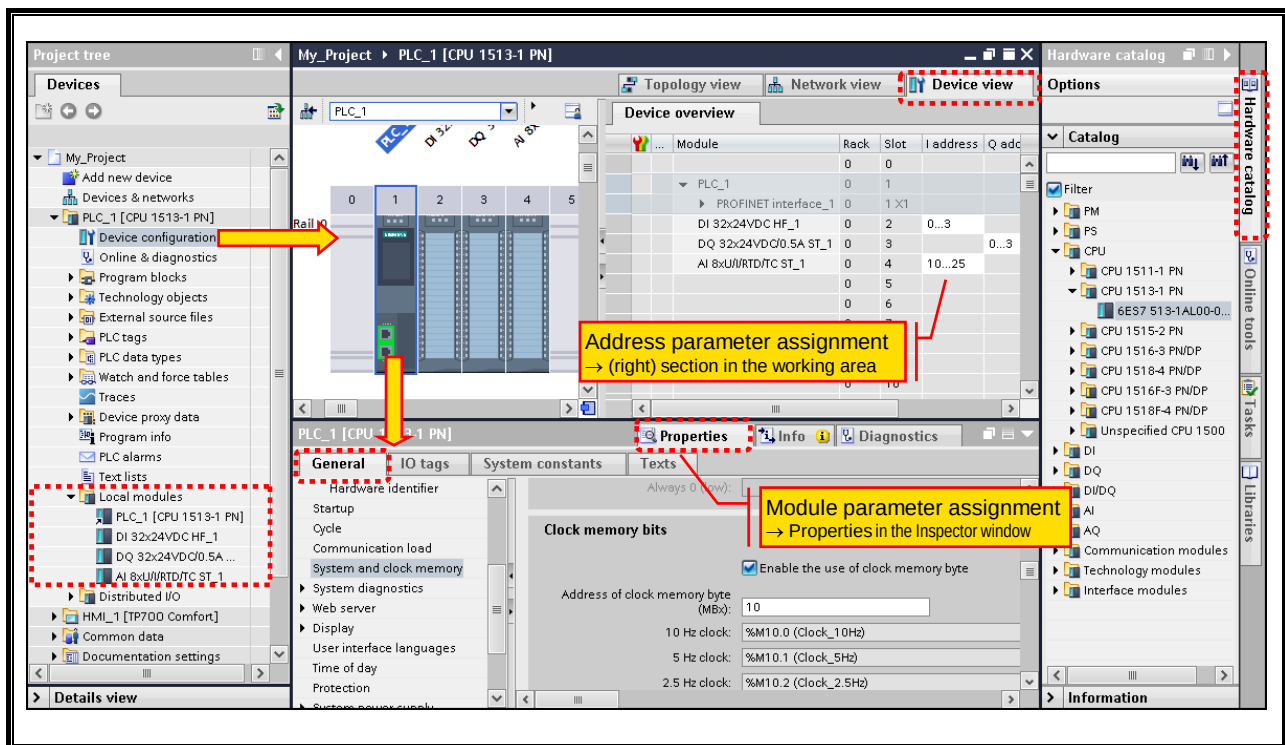
- Hardware configuration
- Module parameter assignment

#### Topology View

The topology view is used to determine the physical structure of networks.

- Configuration of the interface assignment and the relationship between devices

#### 4.6.1. Devices & Networks: Device View (Hardware Configuration)



##### Creating/Editing a Hardware Configuration

- "Devices & networks" editor → select module → select "Device view" tab
- Open the "Device configuration" editor of the module from the Project tree

##### Components of the "Device configuration" Editor

- **"Device view" section** in the working area  
This editor is made up of 2 parts. The right section can be opened/closed using the



buttons or the splitting of the two sections can be dragged to the desired width using the mouse.

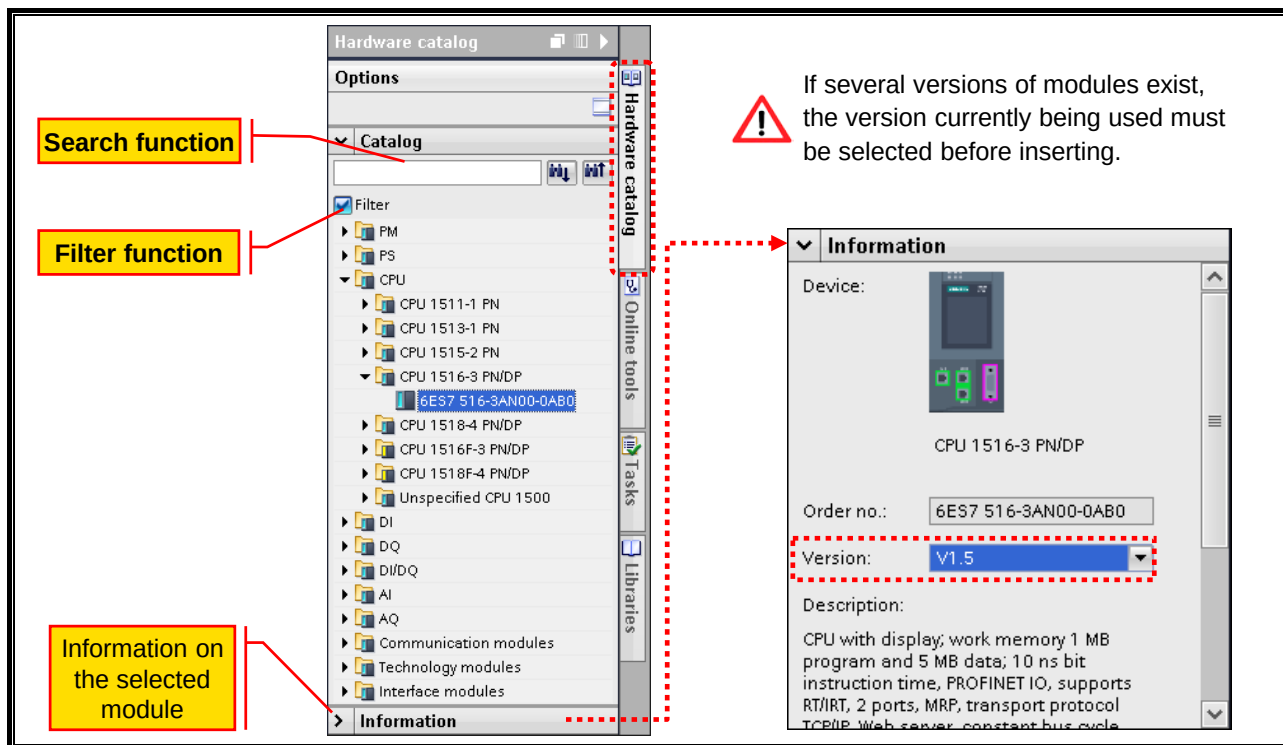
- left area = module configuration
- right area = address parameter assignment of the configured modules
- **"Properties" tab** in the Inspector window  
This tab is used to assign parameters to the module selected in the working area. Here, all the properties or parameters of the selected module are displayed and can also be modified. In the left-hand part of the Properties tab, there is a navigation section in which the parameters are arranged in groups.
- **"Hardware catalog" task card**  
Module catalog for the configuration (module grouping) in the working area

##### Project tree → "Local modules"

In the Project tree, the modules along with their parameter assignments (for example, addresses) are stored under the relevant device in the "Local modules" folder.



## 4.6.2. Hardware Catalog



The Hardware catalog contains all devices and hardware components in a tree structure. From the catalog, selected devices or modules can be dragged to the graphic work area of the "Devices & networks" editor.

### Search Function

This allows a convenient search for specific hardware components. The search also includes the module description texts.

### Filter Function

☒ Filter enabled

Only modules that match the current context (match the working area) are displayed.

☐ Filter disabled

All existing objects of the catalog are displayed

### Contents of the Hardware Catalog

- Network view → only objects that can be networked
- Device view → all modules that belong to the current device in the working area

When changing from the Network to the Device view, the view of the filtered objects is adapted to the device currently selected in the Network view.

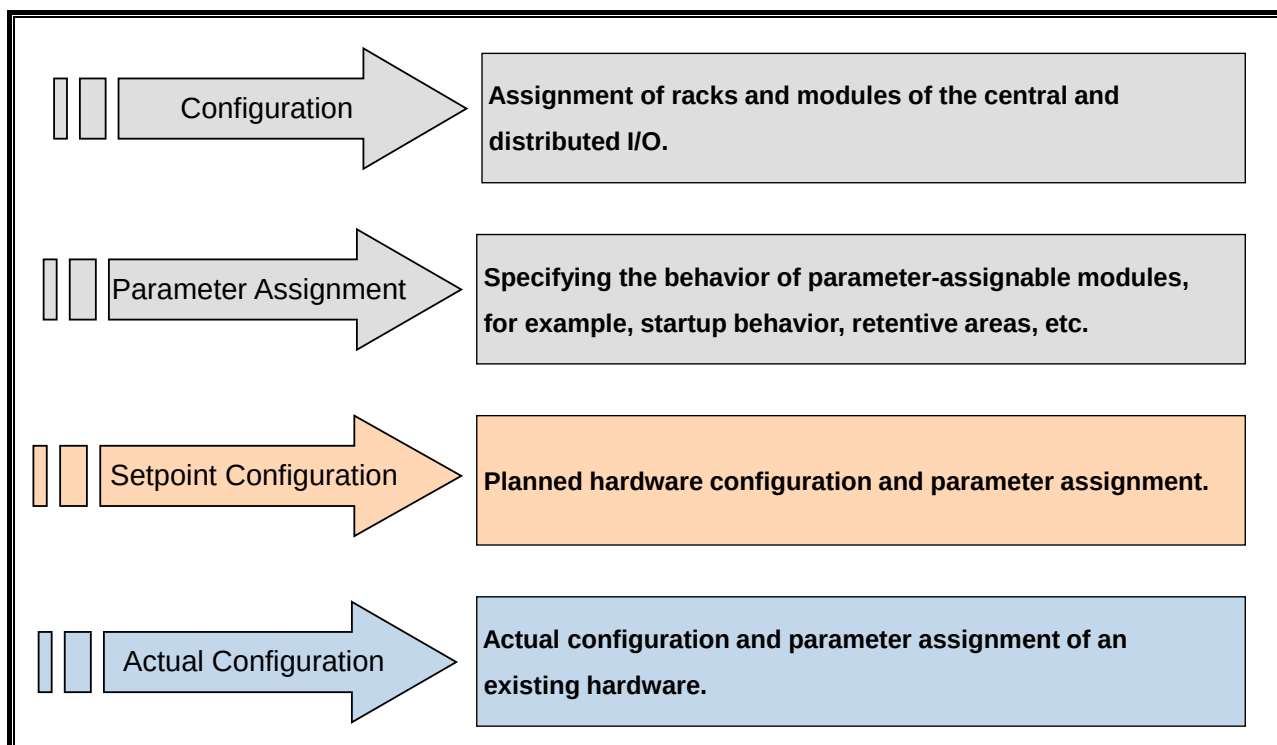
### Information

The "Information" pane shows detailed information about the objects selected in the catalog.

- Schematic diagram
- Name
- Order number
- **Version number**



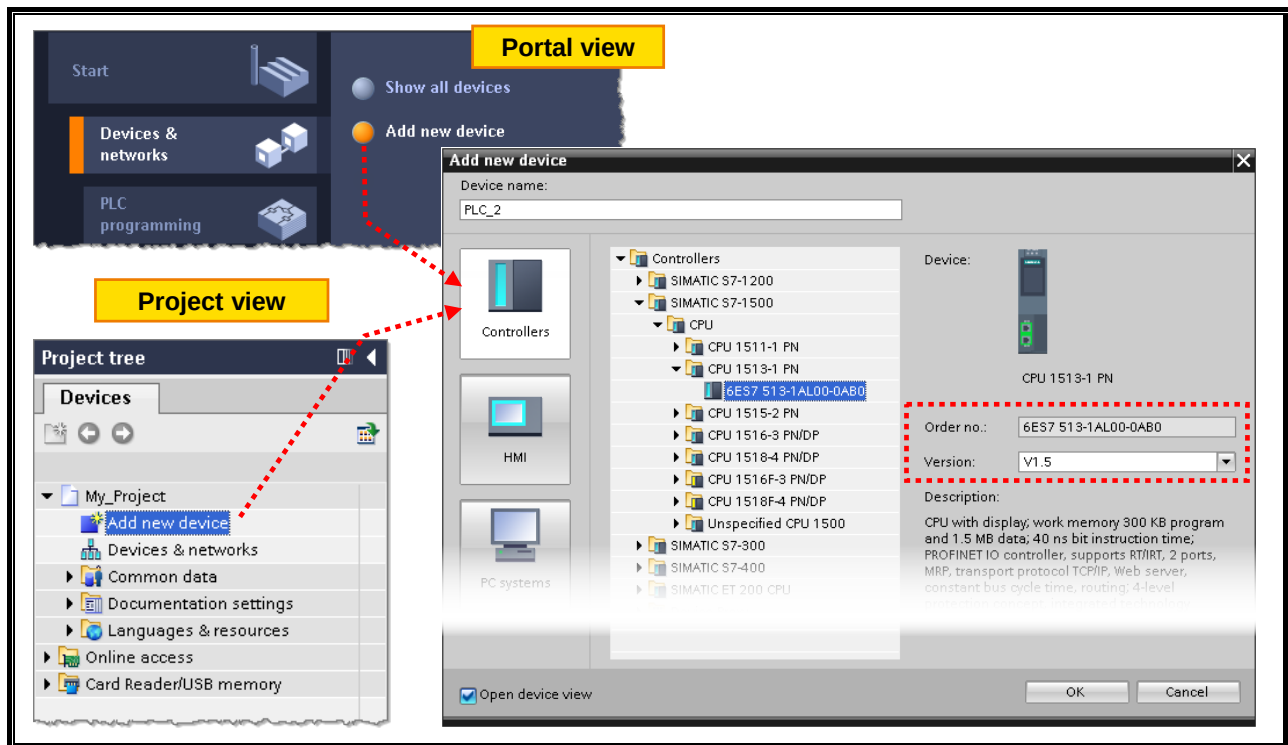
### 4.6.3. Setpoint and Actual Configuration



#### Setpoint and Actual Configuration

When you configure a system, a setpoint configuration is created. It contains a hardware station with the planned modules and the associated parameters. The PLC system is assembled according to the setpoint configuration. During commissioning, the setpoint configuration is downloaded to the CPU.

## 4.7. Setpoint Configuration: Creating a Hardware Station

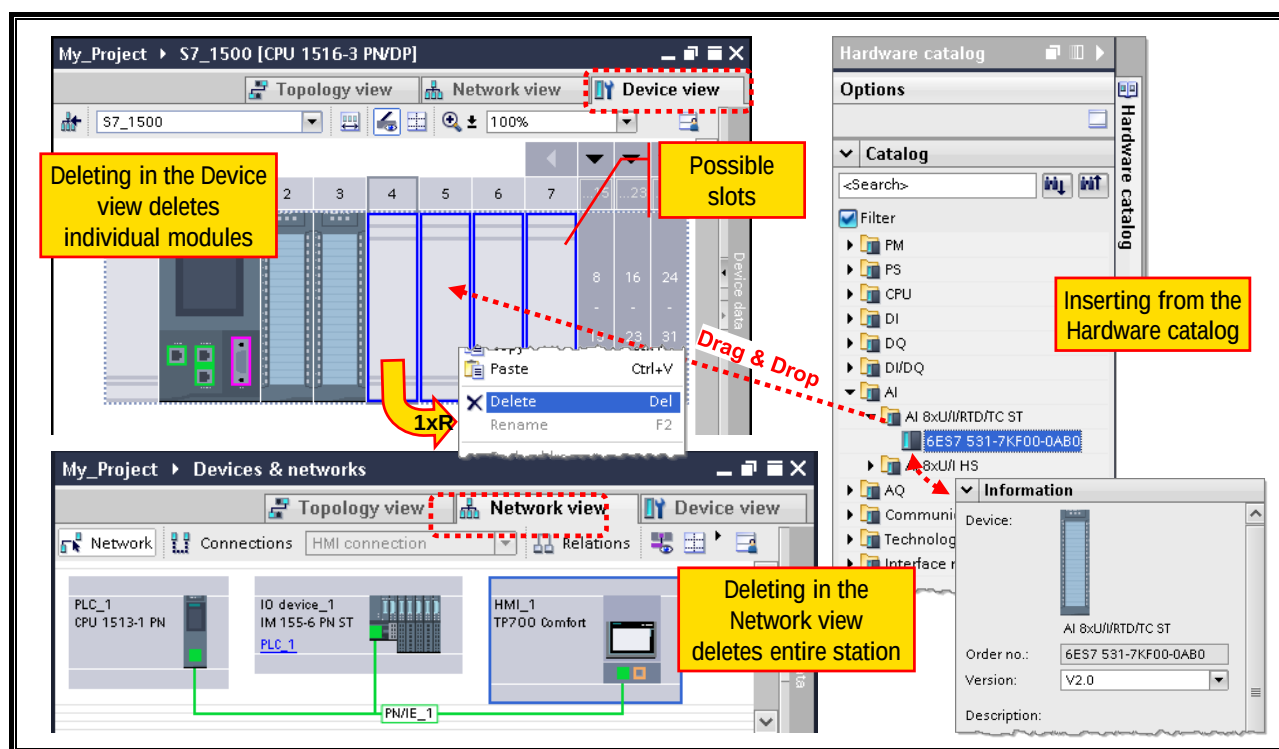


### Device Configuration

With the device configuration, the user specifies the arrangement of modules in the rack.

When a new device is created, a suitable rack is also created automatically. The selected device is inserted into the first permitted slot in the rack. Regardless of the method selected, the added device is visible in the Device view and the Network view of the "Devices & networks" editor.

### 4.7.1. Inserting / Deleting a Module



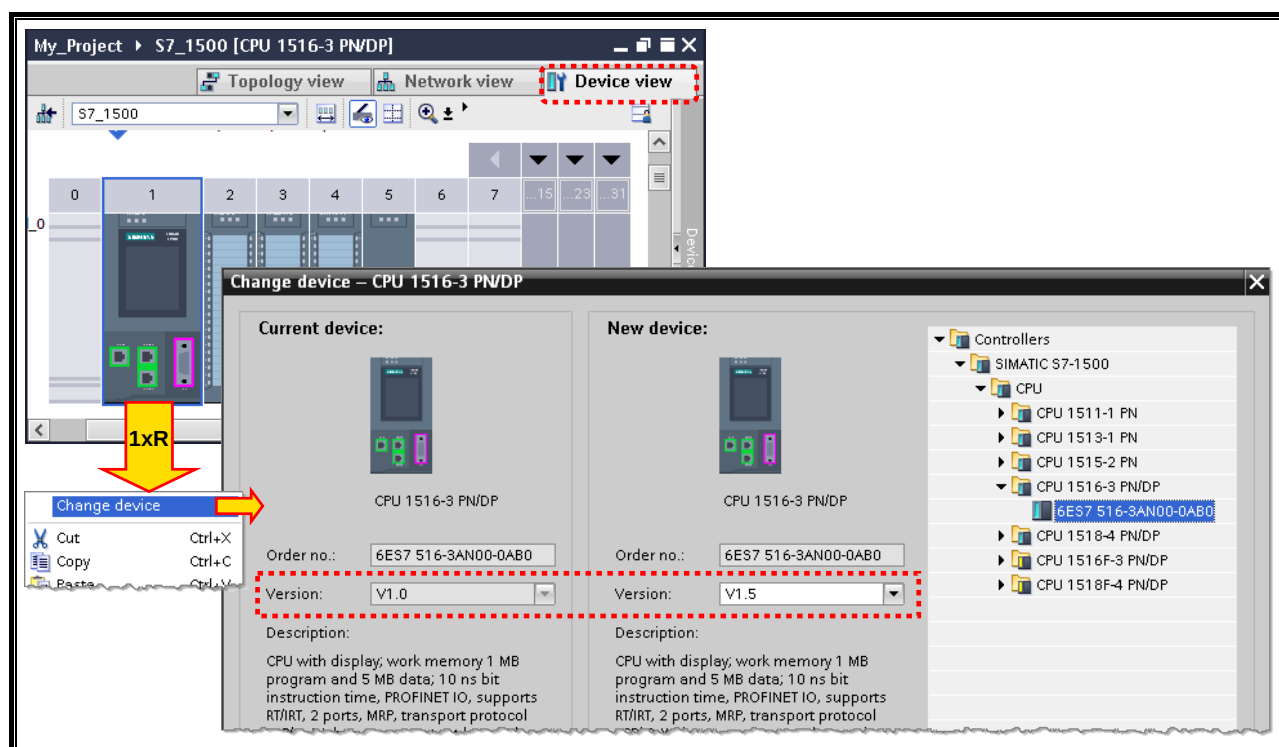
#### Inserting a Module

A new module can also be inserted by dragging and dropping it between two modules that have already been plugged in. To do this, the new module is dragged above and between the two already existing modules. An insert mark then appears, via which the new module can be inserted. All modules that have already been inserted are moved one slot to the right; if there are too many modules, the excess modules are moved to the "area for modules not plugged in".

#### Deleting a Module

Deleted hardware components are removed from the system and assigned addresses are released again.

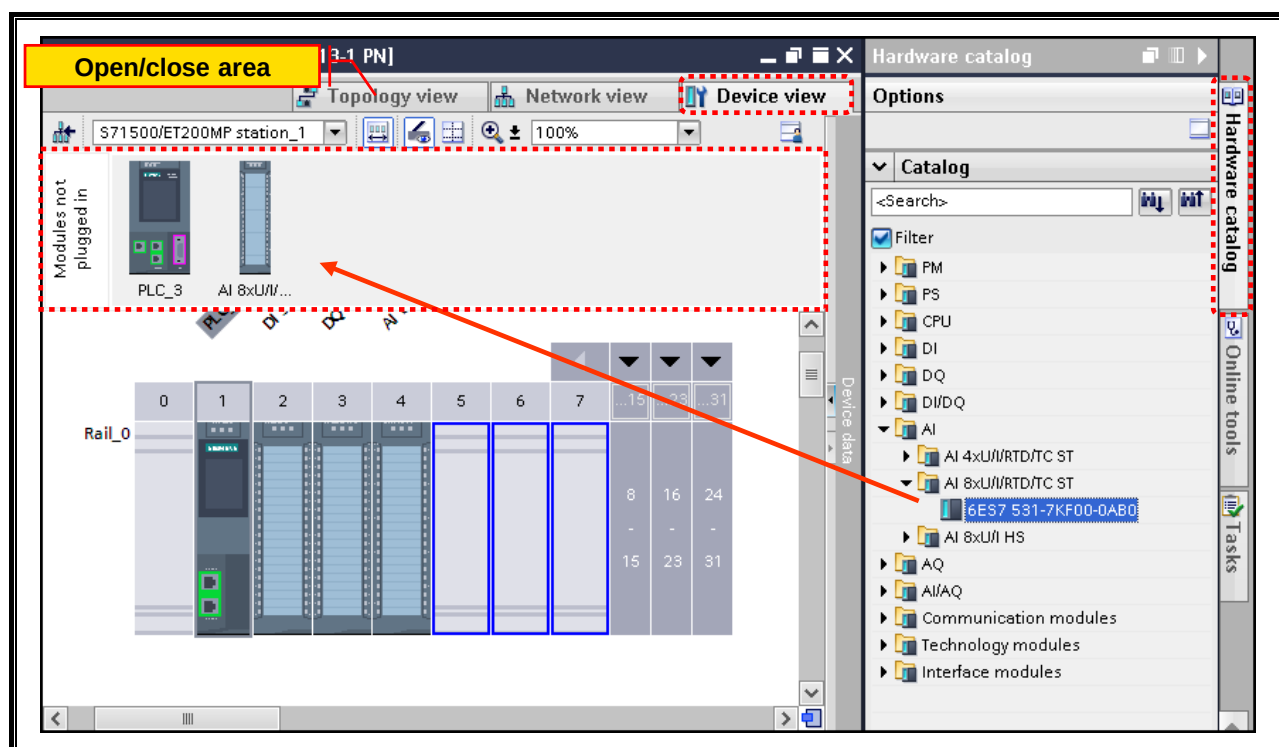
## 4.7.2. Changing a Module



### Changing a Module

Compared to deleting and then inserting a new module, the advantage of changing is that when a module is changed (replaced), all the parameters of the old module are adopted on the new module. A module exchange can, for example, then be necessary when the CPU version in the offline project is to be adapted to the CPU version (online) following a firmware update. Hardware components can only be exchanged if the components are compatible.

### 4.7.3. Area for Modules Not Plugged In



#### Area for Modules Not Plugged In

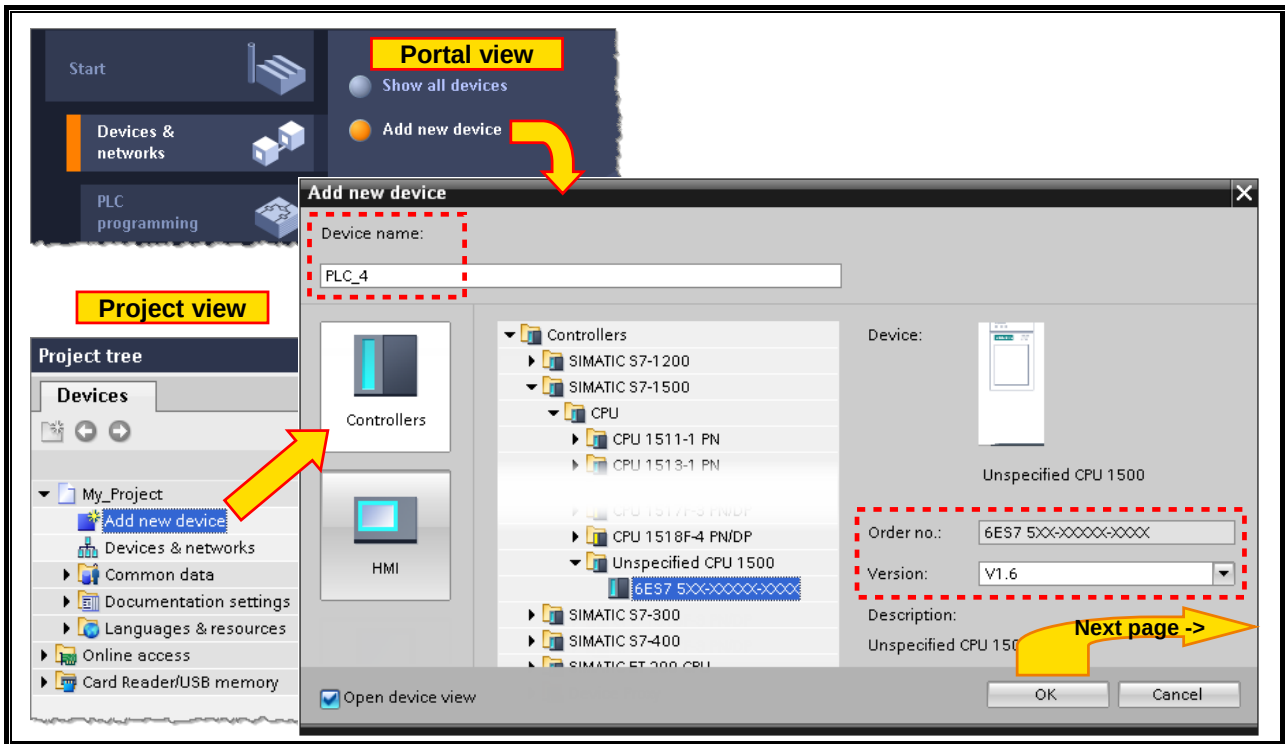
Modules that are intended to be assigned to a device, for example as the result of a copy action, but for which the corresponding rack does not have a free compatible slot are moved automatically to the "area for modules not plugged in".

Modules are added to the area for modules not plugged in under the following conditions:

- When a module is copied or moved within the Project tree.
- When a module is dragged to a device in the Network view, even though there is no compatible slot free on its rack.
- In the Device view, when a module is directly moved into the area from the rack or Hardware catalog.

These modules are ignored when downloading to the target system. There is therefore no consistency check for modules in the "area for modules not plugged in".

## 4.8. Uploading the Actual Configuration without Parameterization into the Project (1): Inserting an Unspecified CPU

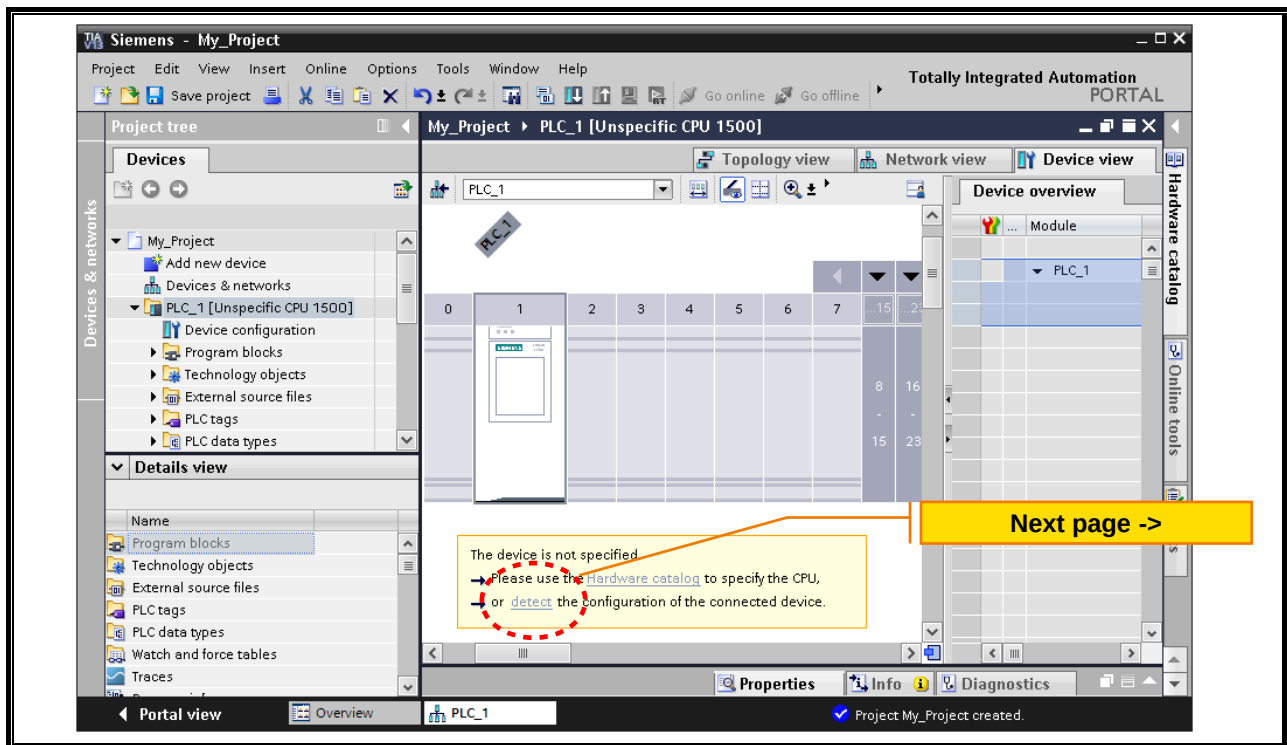


### Uploading the Actual Configuration into the PG

The available configuration (without module parameters!) can be read out from an actually existing station.

This is then necessary, for example, when no matching offline project exists on the PG. After the actual configuration has been read out, the modules can be assigned parameters, saved and reloaded into the CPU. For this, an "Unspecified CPU" must be created in the offline project as a first step.

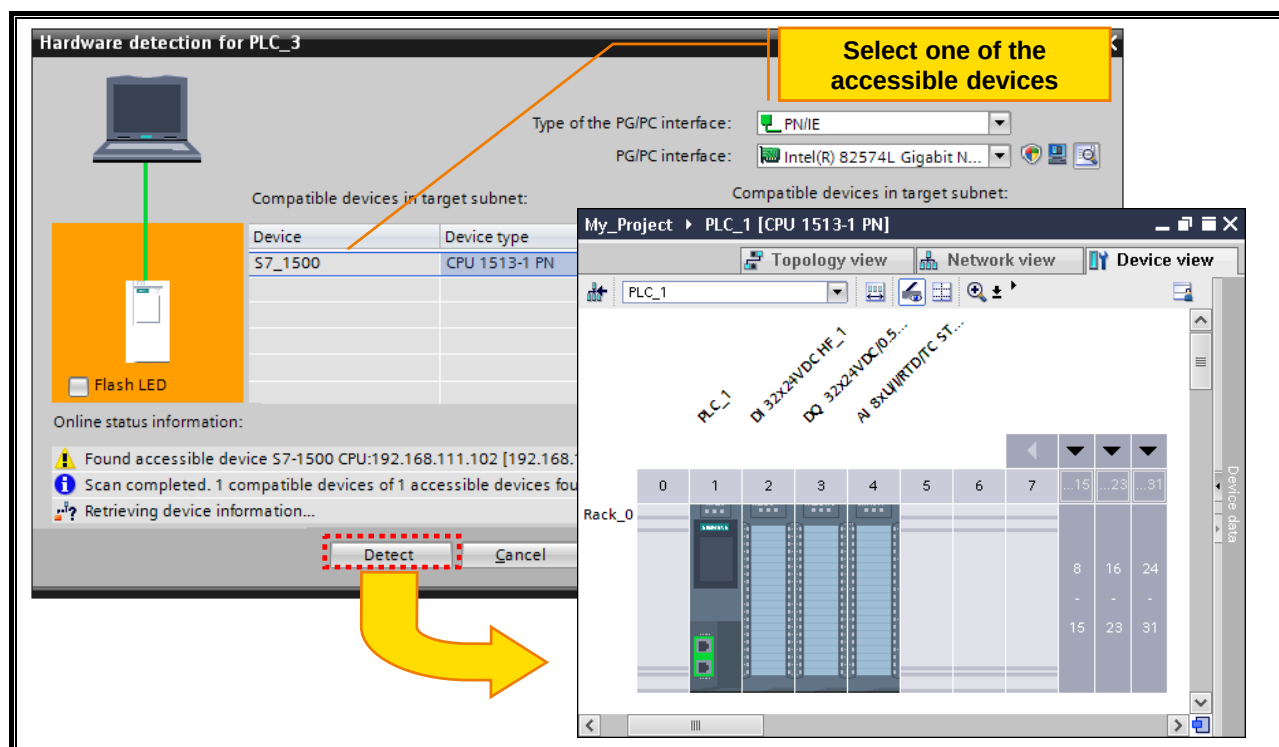
#### 4.8.1. Uploading the Actual Configuration without Parameterization into the Project (2): Detecting Accessible Devices



#### Detecting the Online Station

As soon as the "Unspecified CPU" is created, all accessible devices can be detected in the dialog shown using "detect".

#### 4.8.2. Uploading the Actual Configuration without Parameterization into the Project (3): Reading-out the Actual Configuration



#### Uploading the Actual Configuration

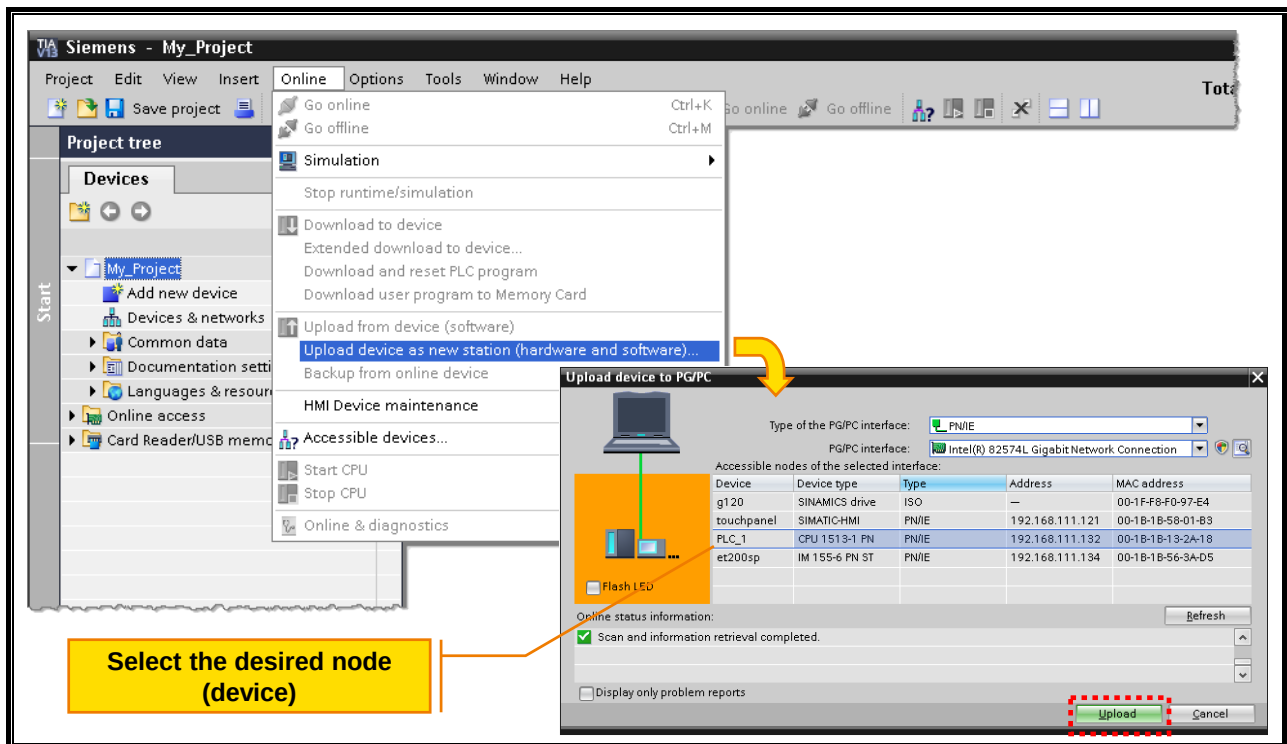
The actual configuration of the selected device is read out and stored in the offline/project.

Note:

During read-out, only the hardware configuration is uploaded, no hardware parameterization and also no S7 program blocks.



## 4.9. Upload a Device as a New Station (1) (Hardware with Parameterization and Software)



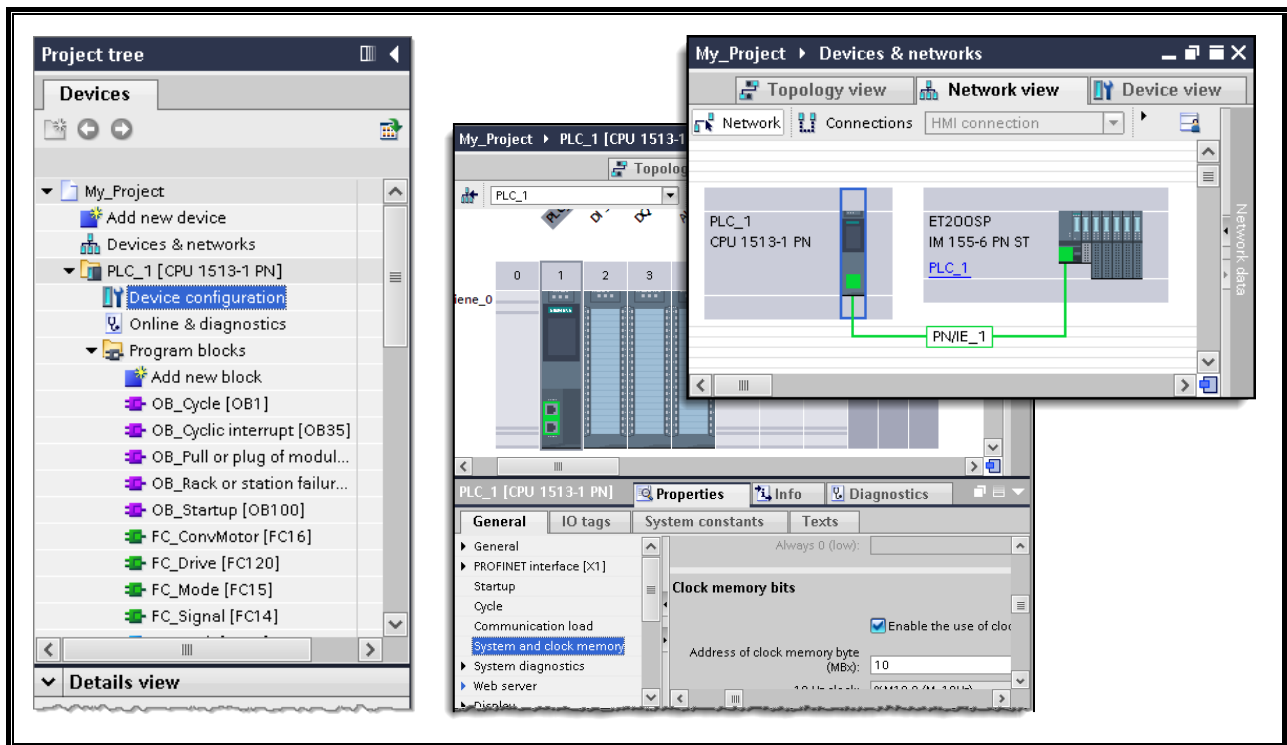
Unlike the option “Upload actual configuration without parameterization”, an already existing configuration of an S7 station can be read out using the function “Upload device as new station”.

This is then necessary when the appropriate offline station **doesn't** exist on the PG. After reading out the S7 station, the hardware as well as the program can be adjusted or modified, saved and downloaded into the CPU.

### Requirement:

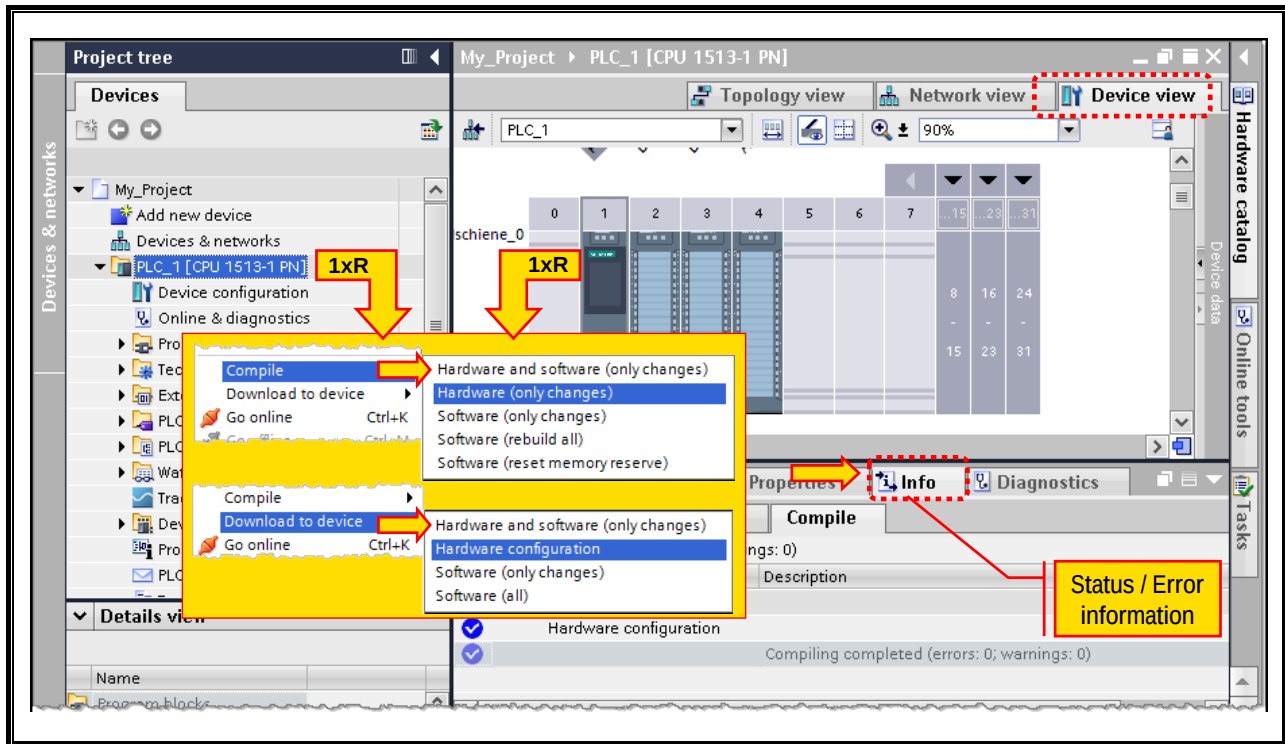
The station already has a configuration.

#### 4.9.1. Upload a Device as a New Station (2) (Hardware with Parameterization and Software)



After the upload of the device, the entire station (central and distributed hardware with parameterization, the entire program with comments and symbols) is available to the user offline.

## 4.10. Compiling the Hardware Configuration and Downloading it into the CPU

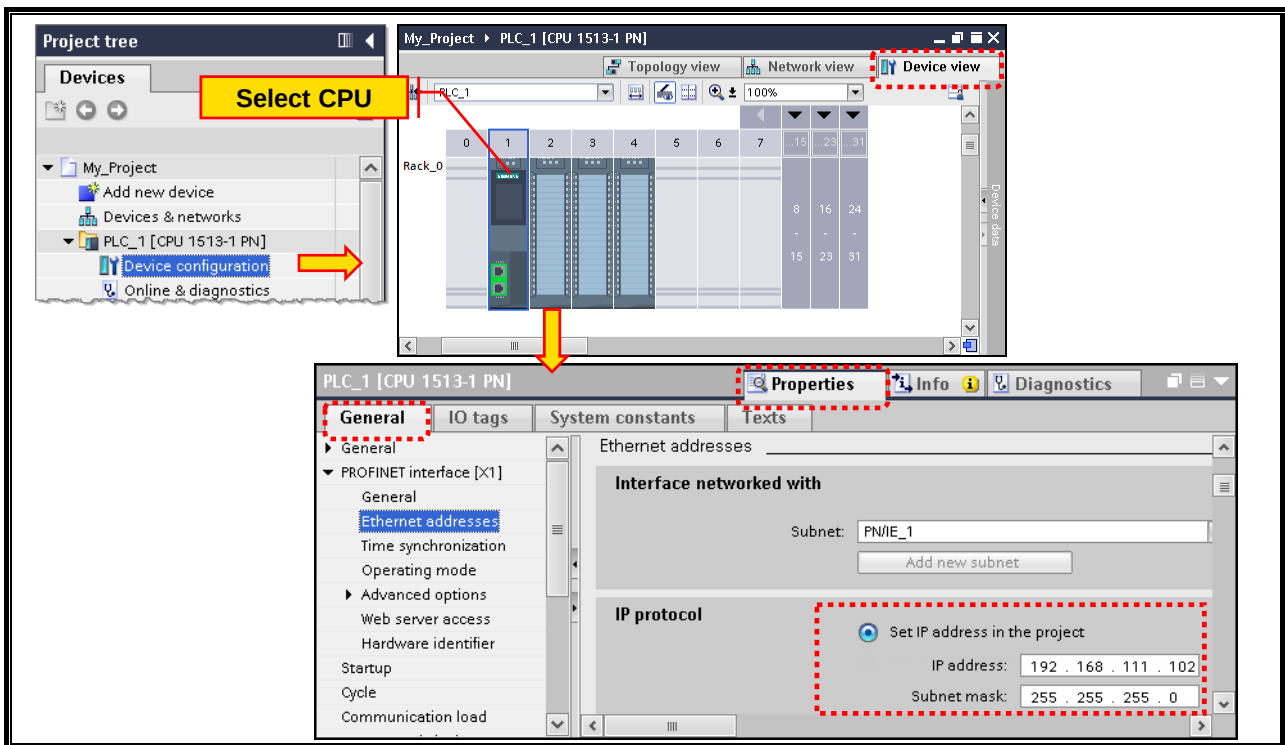


### Compiling / Downloading the Hardware Configuration

The following components of a hardware station can be compiled and downloaded:

- All  
The entire hardware configuration and parameter assignment as well as the entire user program are compiled/downloaded.
- Hardware/Hardware configuration  
Only the entire hardware configuration and parameter assignment are compiled/downloaded.
- Software (only changes)  
Only the changed blocks of the user program are compiled/downloaded
- Software (rebuild all/download)  
All blocks of the user program are compiled/downloaded.
- Software (reset memory reserve)  
All data blocks are reset to start values (loading state). The current data in the CPU is lost, even if a memory reserve (for loading a DB structure change with value retention) is set in the properties of the data block.

## 4.11. CPU Properties: Ethernet Address



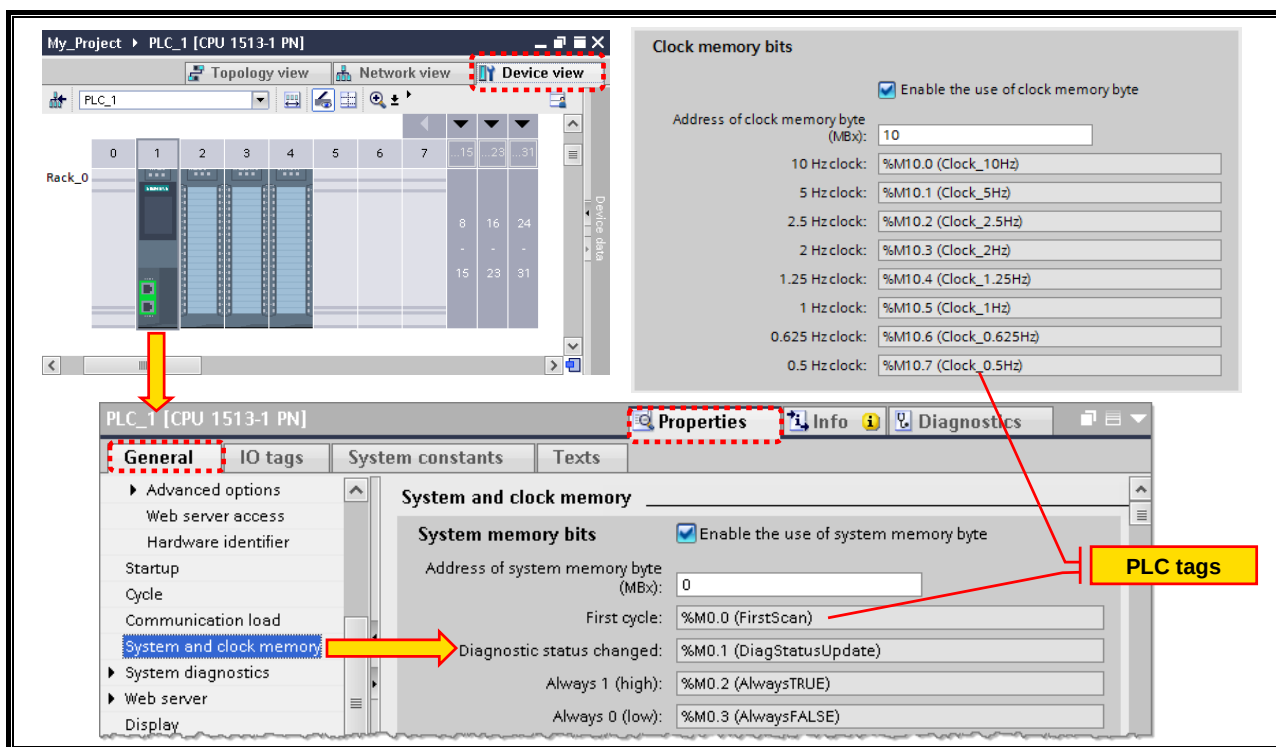
### PROFINET Interface

Regardless of whether the "Devices & networks" editor is in the Device or Network view, if the CPU is selected, the settings of the CPU PROFINET interface can be made in the Inspector window in the "Properties" tab.



If an online connection needs to be established between the programming device and CPU, both devices must be assigned the same subnet mask and the IP addresses must be located in the same subnet.

#### 4.11.1. CPU Properties: System and Clock Memory



A PLC tag is automatically created for each available system or clock memory bit.

#### Clock Memory (8 bits)

These are memory bits whose binary state is changed periodically by the operating system of the CPU with a pulse-pause ratio of 1:1. The various frequencies are shown in the picture. Clock memory (bits) are used to trigger actions periodically. For example, to make an indicator light flash



#### Attention!

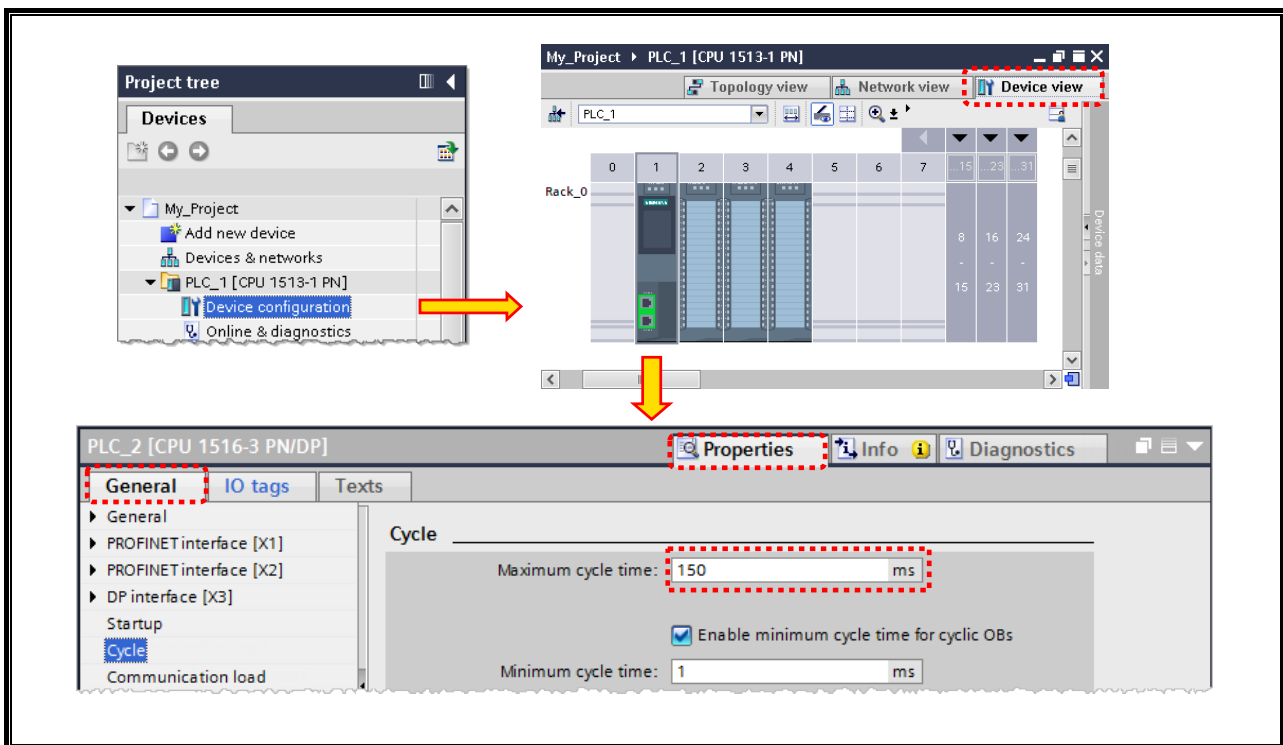
Clock memory (bits) are not synchronized with the CPU cycle; in other words, with long cycle times, the state of the clock memory (bits) can change more than once within one cycle.

#### System Memory (4 bits)

These are memory bits that provide system status information.

- "FirstScan" =1 in the first CPU cycle; otherwise =0
- "DiagStatusUpdate" =1, if a problem exists; otherwise =0
- One static 0 and 1 memory bit each

### 4.11.2. CPU Properties: Maximum Cycle Time



#### Cycle Time

This is the time that the CPU requires to cyclically execute the entire program once. Since parts of the user program can also be processed conditionally and the program execution can also be interrupted (for example, by diagnostics interrupts, time interrupts, hardware interrupts etc.), the cycle time is not the same in every cycle.

#### Maximum Cycle Time

The operating system monitors the runtime of the program for the configured upper limit. If the runtime of the program is longer than the set cycle monitoring time

- the operating system calls the associated error OB (OB80)
- the operating system enters the event in the diagnostics buffer
- the operating system indicates the error on the error LED of the CPU
- S7-1200  
The CPU remains in RUN mode even if no OB80 is programmed and loaded. If the runtime of the program is more than twice as long as the set cycle monitoring time (2xMaxCycleTime error), the CPU changes to STOP mode without calling OB80.
- S7-1500  
If an OB80 is programmed and loaded, the CPU remains in RUN mode, otherwise it changes to STOP.

With the RE\_TRIGR instruction, the monitoring of the cycle time can be retrIGGERED or reset to 0.

#### Minimum Cycle time

The minimum cycle time is the minimum time that should pass for the one-time execution of the cyclic user program and the updating of the associated I/O. The start of the next CPU cycle is delayed if this time has not yet expired.

### 4.11.3. CPU Properties: Protection

**Full access** ⇒ No protection for STEP7 and HMI

**Read access** ⇒ Write protection for STEP7  
⇒ No protection for HMI

**HMI access** ⇒ Write and read protection for STEP7  
⇒ No protection for HMI

**No access** ⇒ Write and read protection for STEP7  
⇒ Write and read protection for HMI

PLC\_2 [CPU 1516-3 PN/DP]

Properties Info Diagnostics

General IO tags Texts

Protection

Select the access level for the PLC.

Access allowed for:

| Access level                                                     | Access |      |       | Access permission |              |
|------------------------------------------------------------------|--------|------|-------|-------------------|--------------|
|                                                                  | HMI    | Read | Write | Password          | Confirmation |
| <input type="radio"/> Full access (no protection)                | ✓      | ✓    | ✓     | ****              | ****         |
| <input type="radio"/> Read access                                | ✓      | ✓    |       | ****              | ****         |
| <input type="radio"/> HMI access                                 | ✓      |      |       | ****              | ****         |
| <input checked="" type="radio"/> No access (complete protection) |        |      |       |                   |              |

1 2 3

#### Protection Levels

With the following protection levels, the access rights (read / write) of the programming device to the CPU are specified:

- **Full access (no protection):** Default setting  
Read and write access is always permitted.
- **Read access:** → Write protection  
Read-only access possible. No data can be changed in the CPU and no blocks or modified hardware configuration or parameter assignment can be downloaded to the CPU.
- **HMI access:** → Write and read protection for STEP7  
No write or read access is possible from the engineering. Only the CPU type and identification data can be displayed in the Project tree under "Accessible devices". It is not possible to display online information or blocks under "Accessible devices" without entering a password.
- **No access (complete protection):** → General write and read protection for STEP7 and HMI  
Now, write and read access is also not possible for those HMI devices that do not have a configured password in the connection.

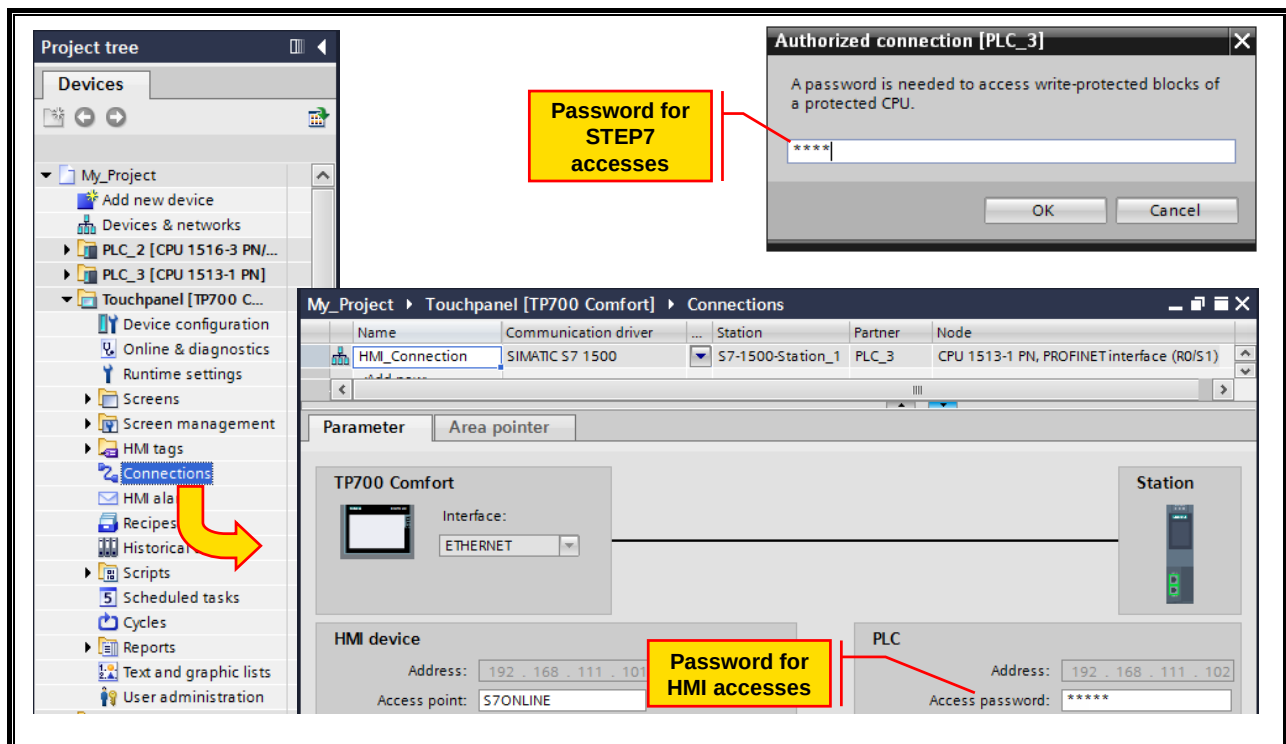
#### Access Permitted through Passwords

In the example shown, "No access (complete protection)" is selected, whereby STEP7 and HMI devices can neither read-access nor write-access the CPU.

The above explained protection levels can, however, be lifted again with passwords:

- By specifying a password **3** an HMI device can once again read-access and write-access the CPU. For STEP7, neither read-access nor write-access is possible
- By specifying a password **2** an HMI device can once again read-access and write-access the CPU and for STEP7, only a read-access is permitted, not a write-access
- By specifying a password **1** a read-access and a write-access of the CPU is possible for both an HMI device as well as for STEP7.

#### 4.11.3.1. Accessing a Protected CPU by Means of Password Specification



##### Password Query for STEP7 Accesses

After the TIA Portal is started, the CPU password is queried once (see picture top right) for STEP7 online accesses to a CPU protected by password (e.g. monitor/modify variables, block download....).

##### Password Query for HMI Accesses

In order to receive access rights to a CPU protected by password, the HMI device must log on to the CPU by means of a password when runtime is started. This password must be specified in the connection configuration of the HMI device (see picture bottom right).

The following functions are generally excluded from the (write/read) protection:

- Reading diagnostics data
- Identification via the "Accessible devices" function.



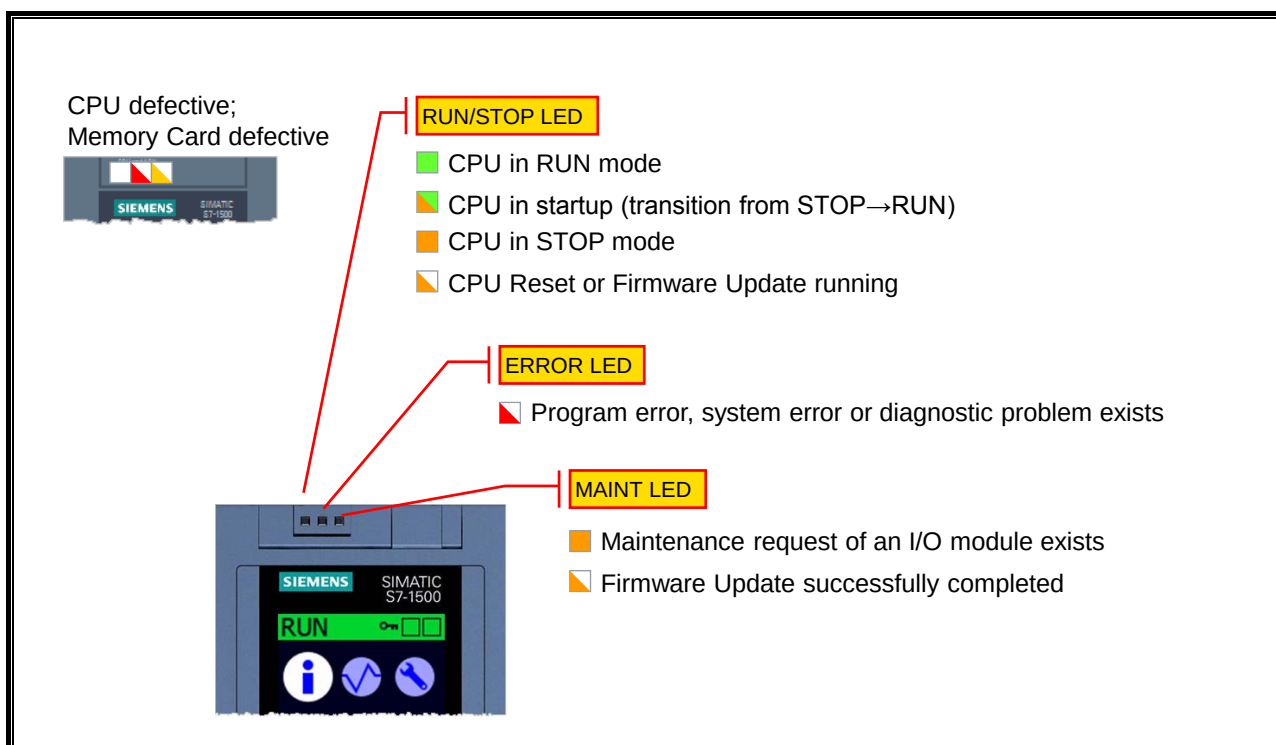
##### Attention!

The password is stored on the CPU in encrypted form and cannot be deleted.

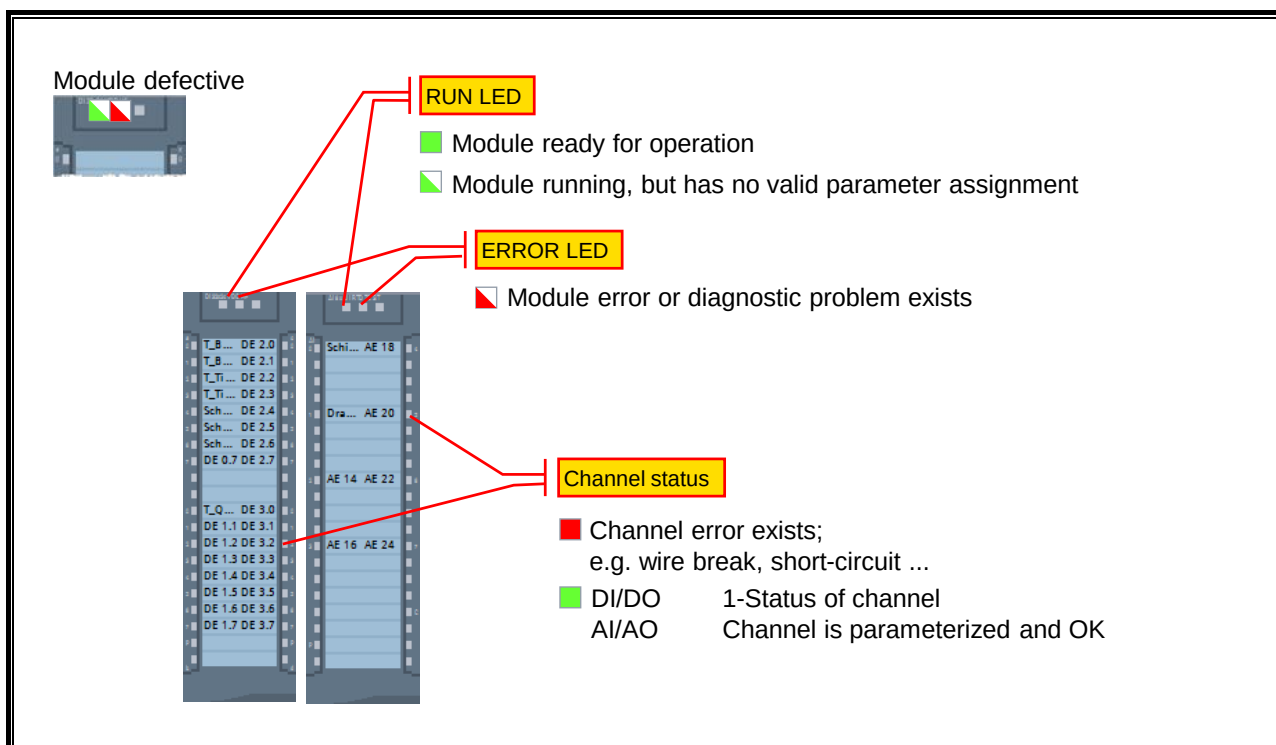
The functions protected by password can only be carried out by one PG/PC at a time. A further PG/PC cannot log on.



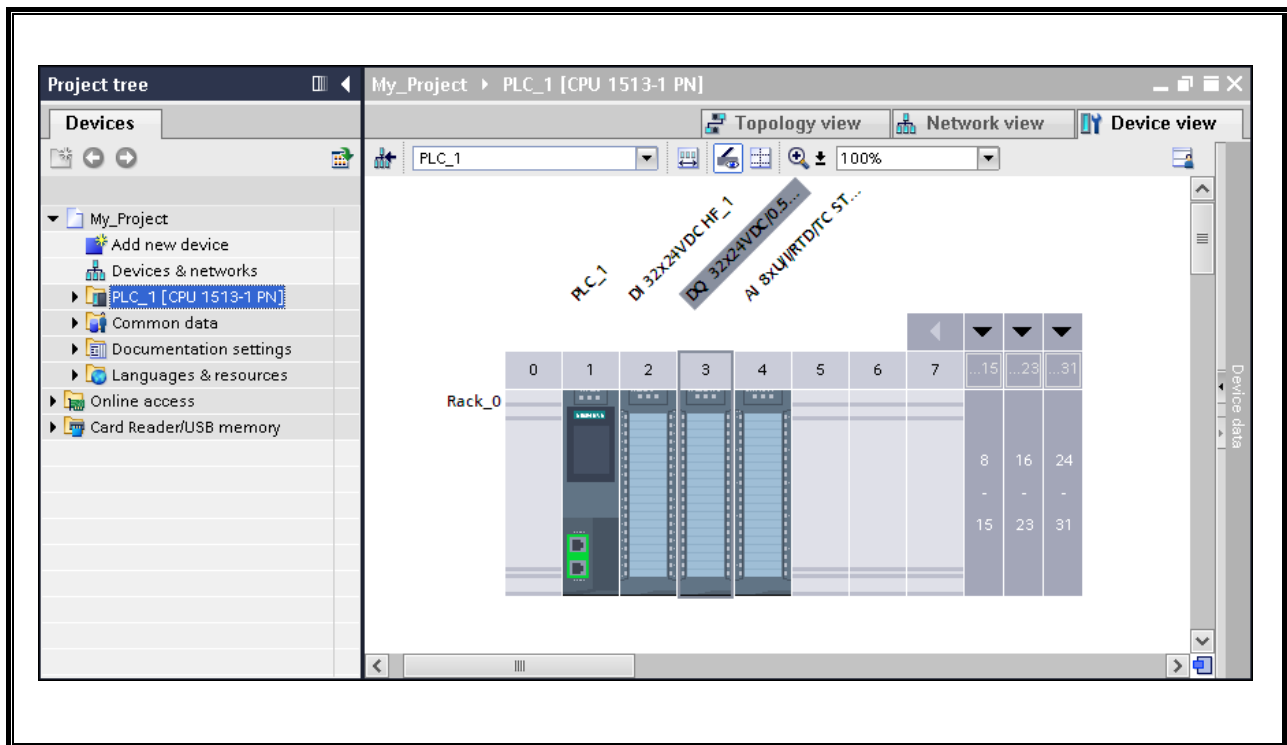
#### 4.11.3.2. Status LEDs of the S7-1500 CPU



#### 4.11.4. Status LEDs of the Central DI/DO Modules of the S7-1500 CPU



## 4.12. Task Description: Creating a Project with an S7-1500 Station

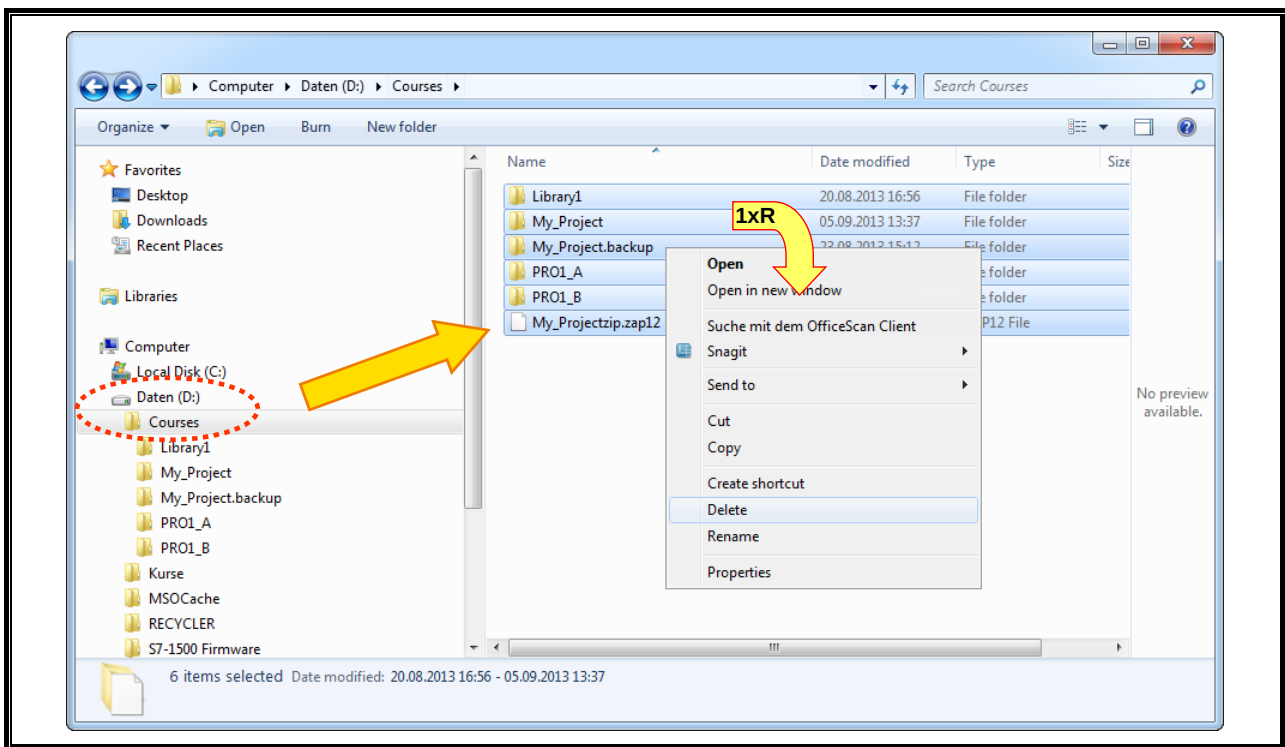


### Task Description:

A project with the name "My\_Project" is to be created. It is to contain an S7-1500 station whose configuration is to correspond exactly to that of your training device.

Furthermore, the modules are to be assigned parameters in such a way that the input and output addresses match those shown in the chapter "Training Devices".

### 4.12.1. Exercise 1: Deleting Old Projects



#### Task

Delete the TIA Portal projects on the PG.

#### What to Do

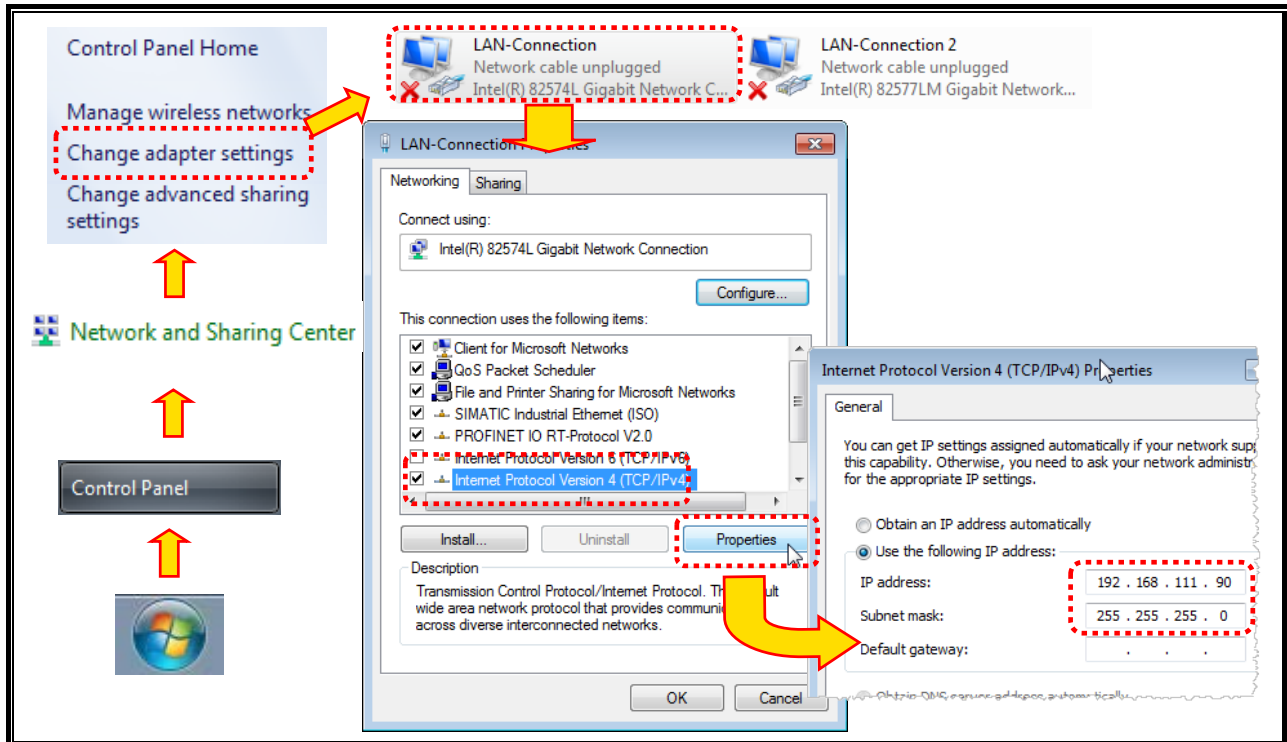
1. Start the Windows Explorer
2. In the directory *D:\Courses*, delete all projects.

#### Note

Projects that are to be deleted must to be closed!

## 4.12.2. Exercise 2: Connecting the PG and Setting the IP Address of the PG

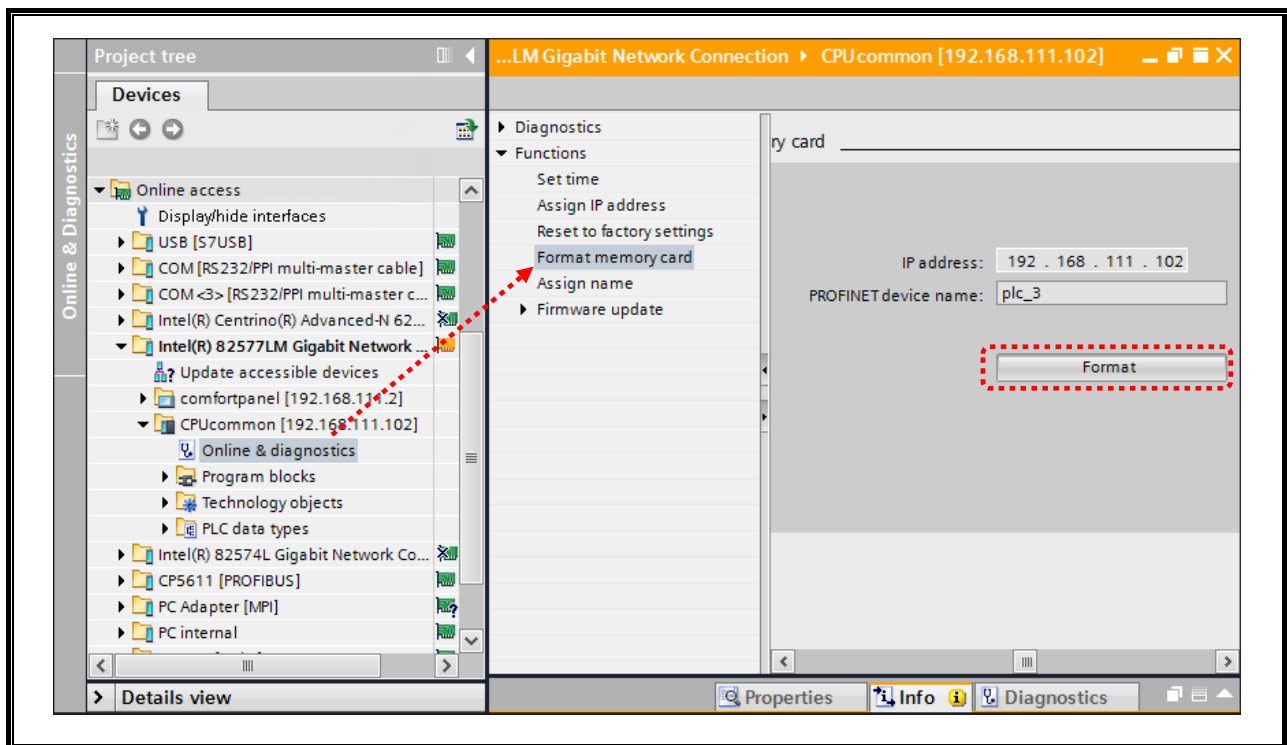
### PG with Windows 7



### Task and What to Do:

1. Connect the Ethernet interface "Intel(R) 82574L" of the PG to the "P2" connection on the training device using an Ethernet cable.
2. Assign the IP address 192.168.111.90 and the subnet mask 255.255.255.0 to this PG interface. Proceed as shown in the picture.

### 4.12.3. Exercise 3: Erasing the SIMATIC Memory Card of the CPU



#### Task

In order to completely erase the CPU, the SIMATIC Memory Card of the CPU must first be erased. This can be carried out as follows:

- with the Windows Explorer (SD card is inserted in the PG Card Reader)
- with the TIA Portal (SD card is inserted in the PG Card Reader)
- with the TIA Portal (SD card is inserted in the CPU)

#### What to Do:

1. Check whether the SD card is inserted in the CPU.
2. In the Project view under the "Intel(R) 82574L" interface, display all "Accessible devices"
3. Under the S7-1500 station, activate "Online & diagnostics" (see picture)
4. There under "Functions", activate "Format memory card" (see picture)

#### Note

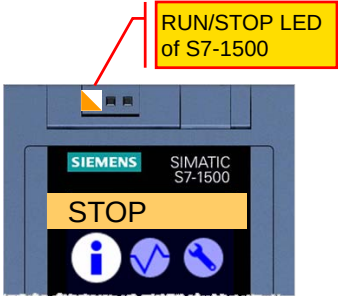
Should there be a password - that is unknown to you - stored on the CPU, it is only possible to erase the SD card if it is inserted in the PG Card Reader.

#### 4.12.4. Exercise 4: Resetting the CPU using the Mode Selector Switch



1. Set the **mode selector switch** to **STOP**
2. Press the mode selector switch in the MRES position until the RUN/STOP LED has flashed 2x slowly  
then let go again
3. Press the mode selector switch in the MRES position until the RUN/STOP LED begins to flash quickly  
then let go again
4. Set the **mode selector switch** to **RUN**  
A CPU restart is carried out

within 3 sec !!!



#### Task

In the last exercise you erased the SD of the CPU. Now, the work memory of the CPU is also to be erased. For this, a memory reset has to be carried out.

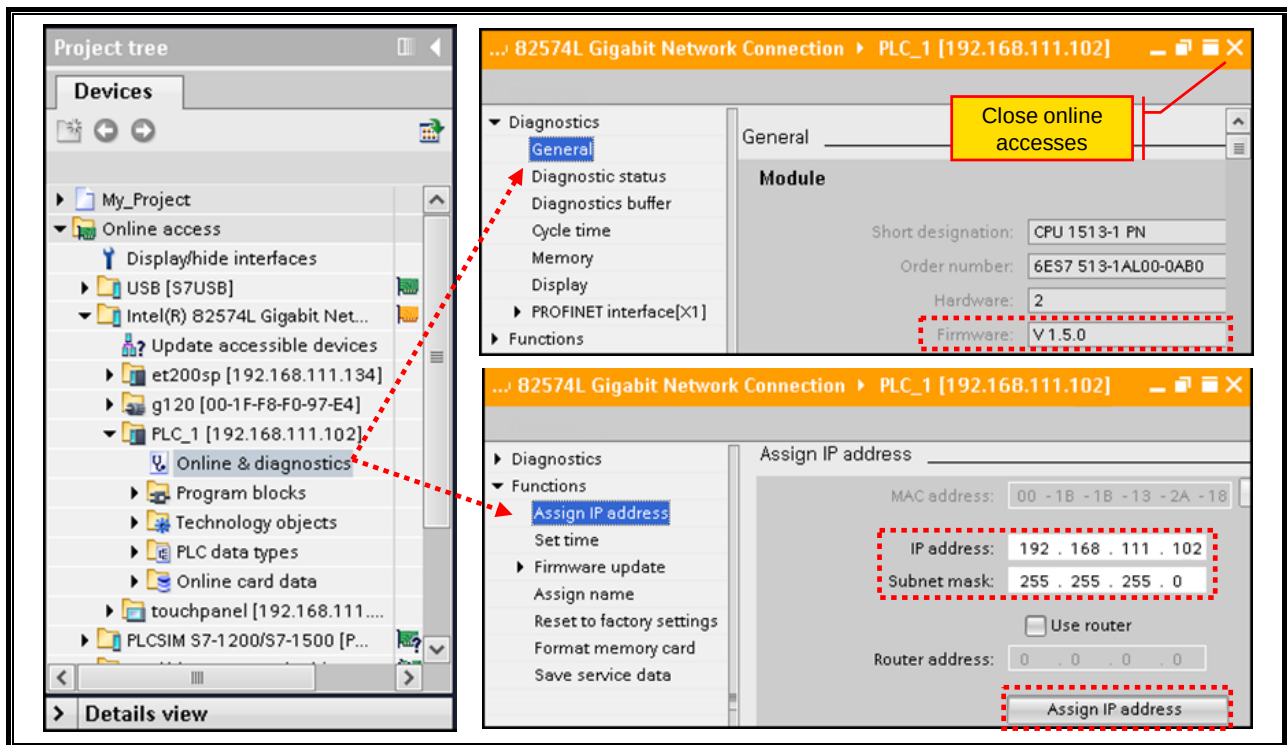
#### What to Do

1. Carry out the memory reset directly on the CPU following the steps shown in the picture.
2. Carry out a CPU restart by switching the mode selector switch from STOP to RUN.

#### Result:

- The CPU remains in the STOP state because a user program isn't loaded.
- The I/O modules indicate with a flashing green light that they are not parameterized.

#### 4.12.5. Exercise 5: Determining the CPU Firmware Version and Assigning the IP Address (Node Initialization)



#### Task

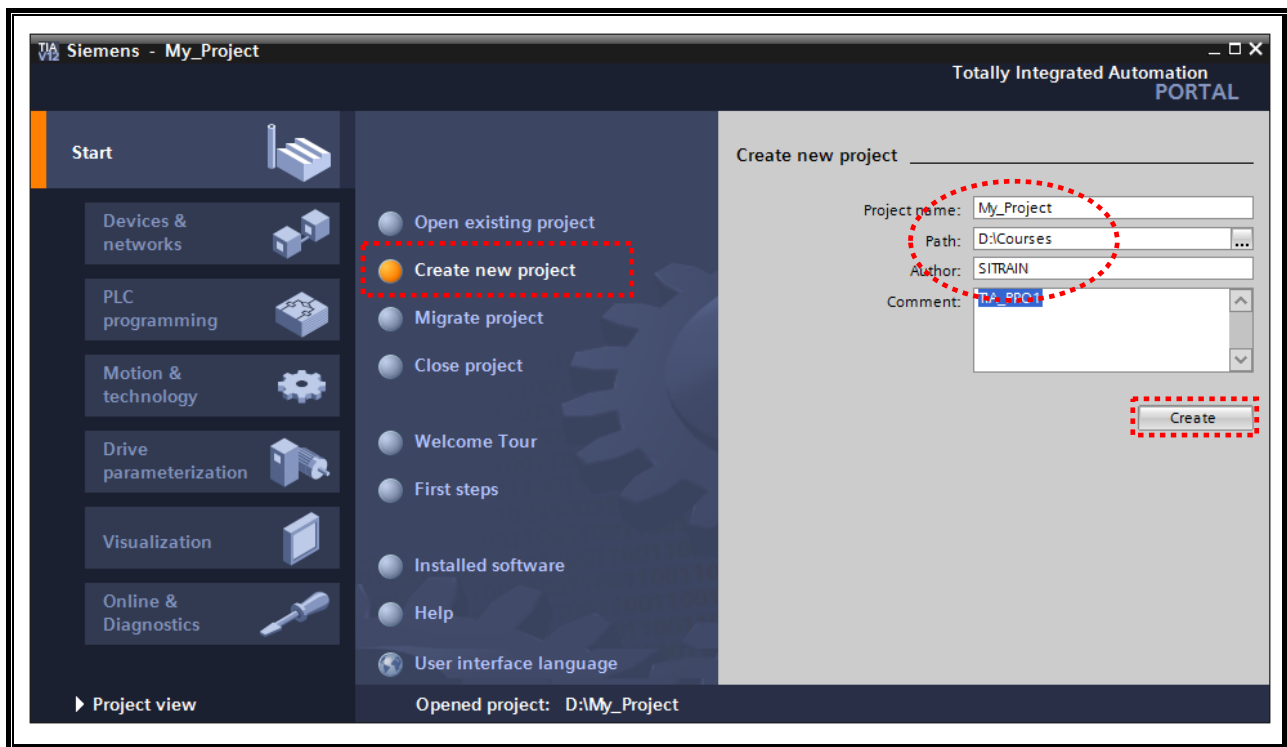
In order to later be able to create an S7-1500 station in the offline project which exactly matches your training controller, you are to determine online the Firmware version of the CPU and make note of it.

Furthermore, you are to check the IP address of the PROFINET interface of the CPU and, if necessary, set it.

#### What to Do:

1. In the Project tree under the interface "Intel(R) 82574L", display all "Accessible devices" (see picture)
2. Under the S7-1500 station, activate "Online & diagnostics" (see picture)
3. There, under "General" you can see the Firmware version and make note of it.
4. Under "Functions --> Assign IP address", check the IP address and correct it, if necessary.
5. In the working area, close the online accesses (see picture)

#### 4.12.6. Exercise 6: Creating a New Project



#### Task

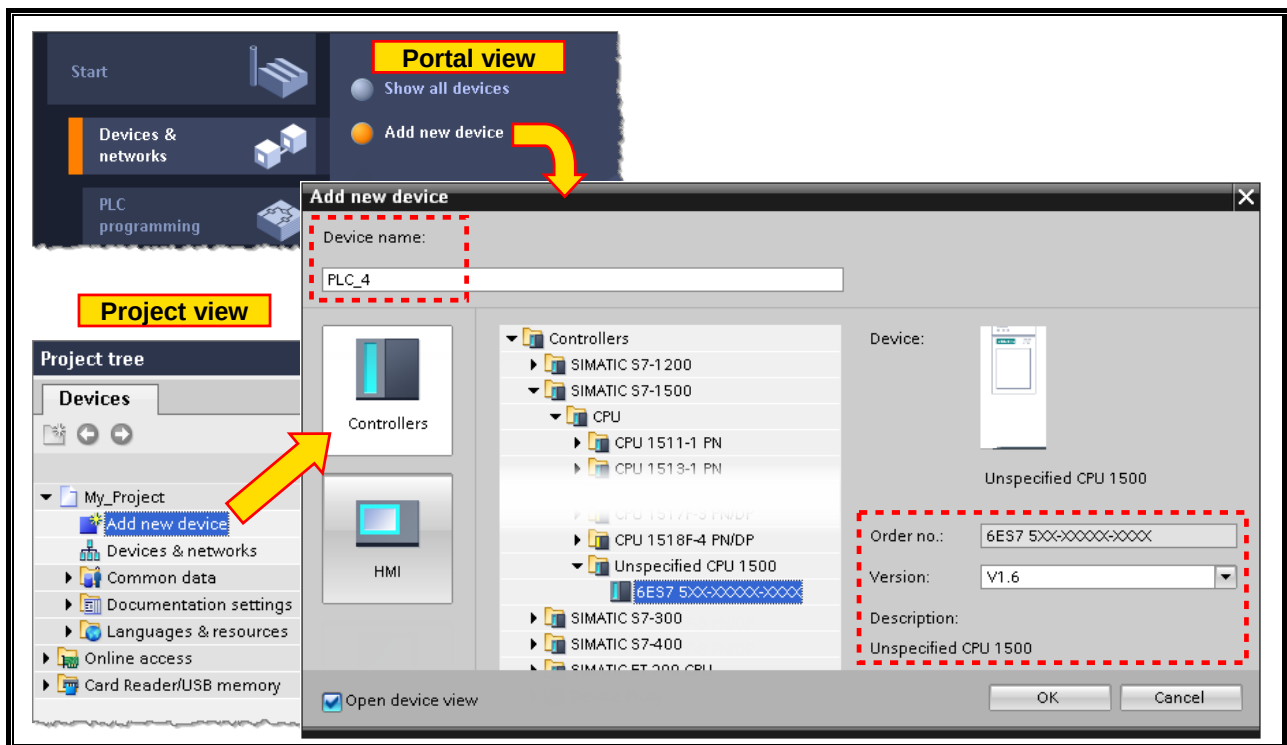
A new project with the name "My\_Project" is to be created.

#### What to Do

1. In the Portal view, activate "Create new project".
2. Enter the project name and the given path as shown in the picture and "Create" the project.



#### 4.12.7. Exercise 7: Creating (Adding) a New Device



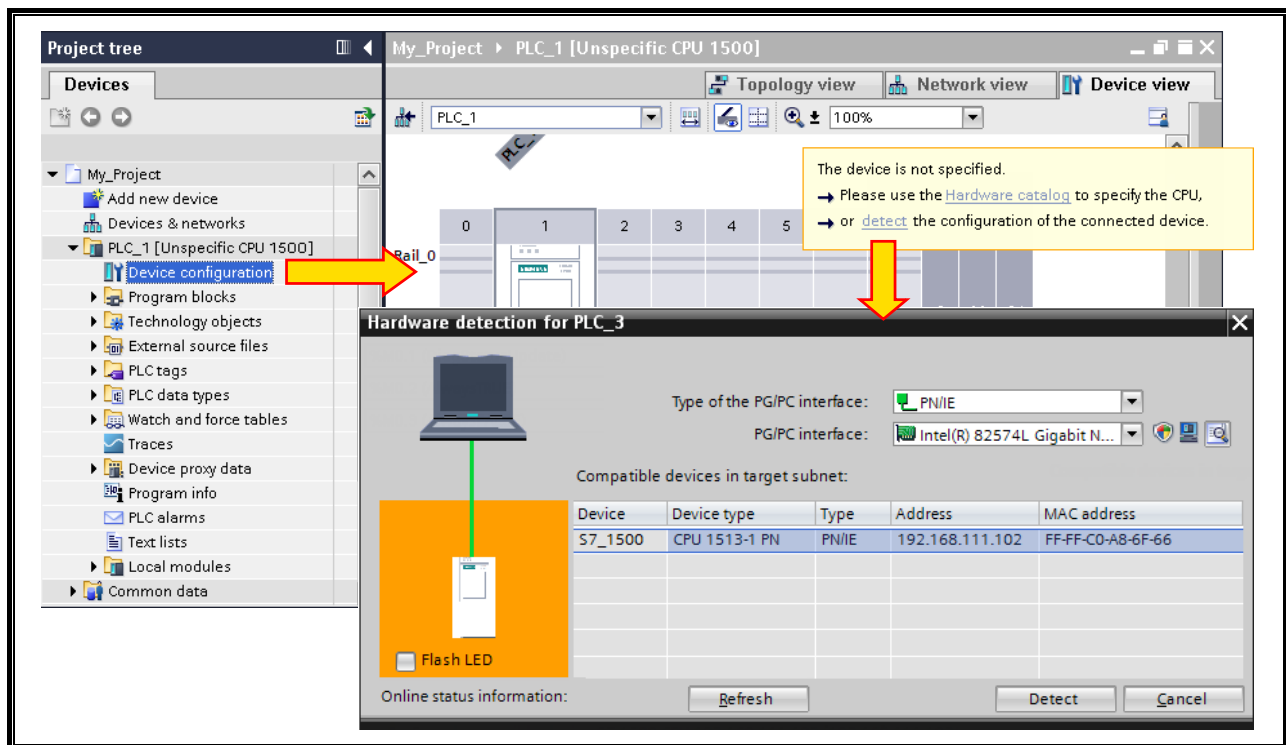
#### Task

As a "new device" create (add) an S7-1500 station with an unspecified CPU.

#### What to Do

1. Activate the menu item: "Add new device".
2. As device, select an S7-1500 station with unspecified CPU with the relevant Firmware version
3. Give the device a device name.

#### 4.12.8. Exercise 8: Reading-out the Actual Configuration



#### Task

You are to read out the actual configuration of the modules from the CPU and save it in your project in the still "unspecified station".

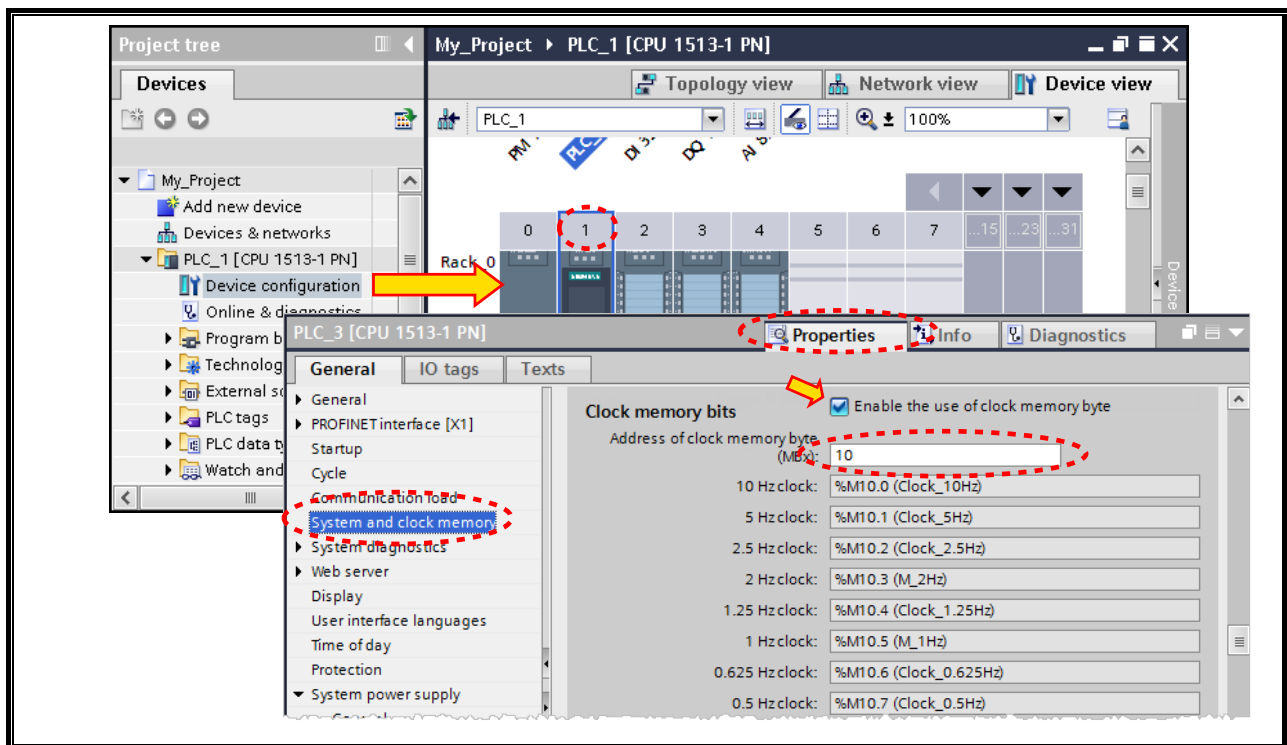
#### What to Do

1. Switch to the Project view.
2. Double-click on the "Device configuration" of the S7-1500 station with unspecified CPU. The Device view of the station opens with the message "The device is not specified" and the possibility of "detecting" the device.
3. Activate "'Detect' the (hardware) configuration of the connected device".
4. In the "Hardware detection" dialog, select the CPU of your station from the "Compatible devices in target subnet" list and load the configuration into the project via "Detect".
5. Save your project.

#### Result

The Device view shows the read-out configuration.

#### 4.12.9. Exercise 9: CPU Properties: Parameterizing the Clock Memory Byte



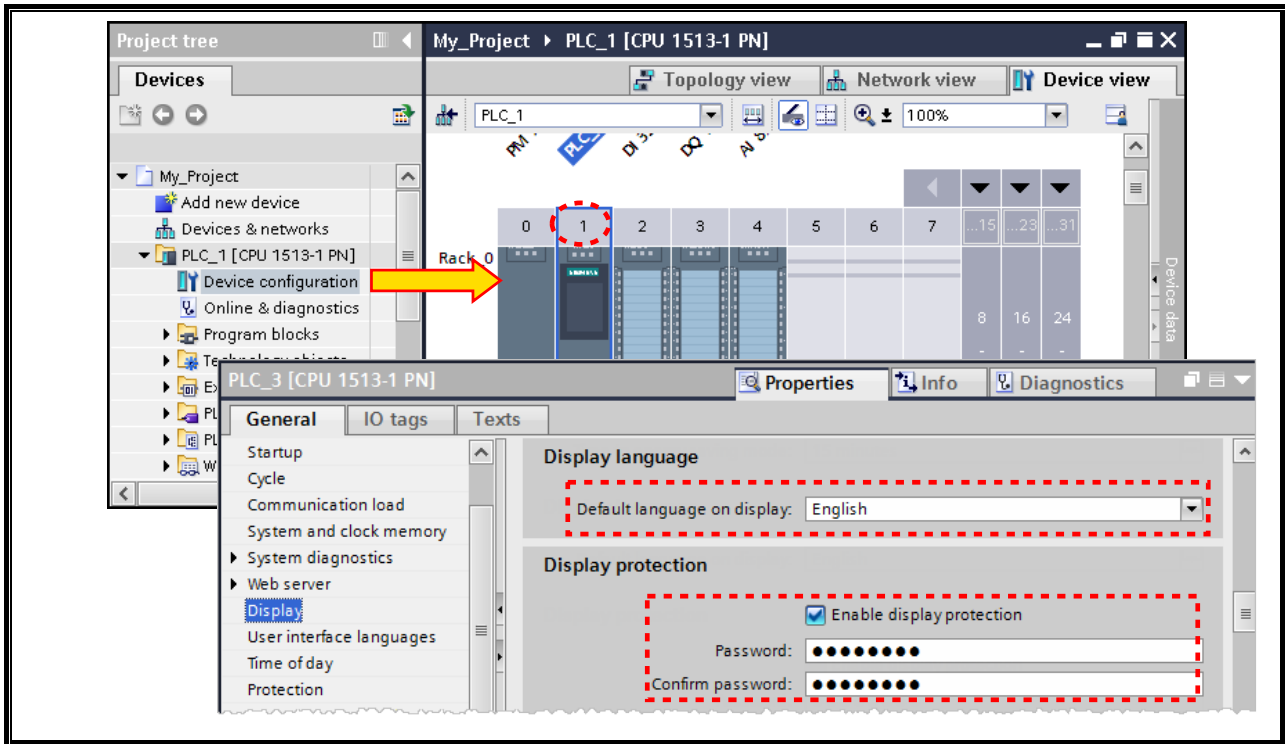
#### Task

In the CPU Properties, parameterize memory byte 10 as a clock memory byte.

#### What to Do

1. In the "Inspector window", select the "Properties" tab and there "System and clock memory".
2. Enable the "..use of clock memory byte" and parameterize MB10.

#### 4.12.10. Exercise 10: CPU Properties: Parameterizing the Display Language and Display Protection



##### Task

In the CPU Properties, parameterize the display language of the CPU-Display and the display protection.

##### What to Do

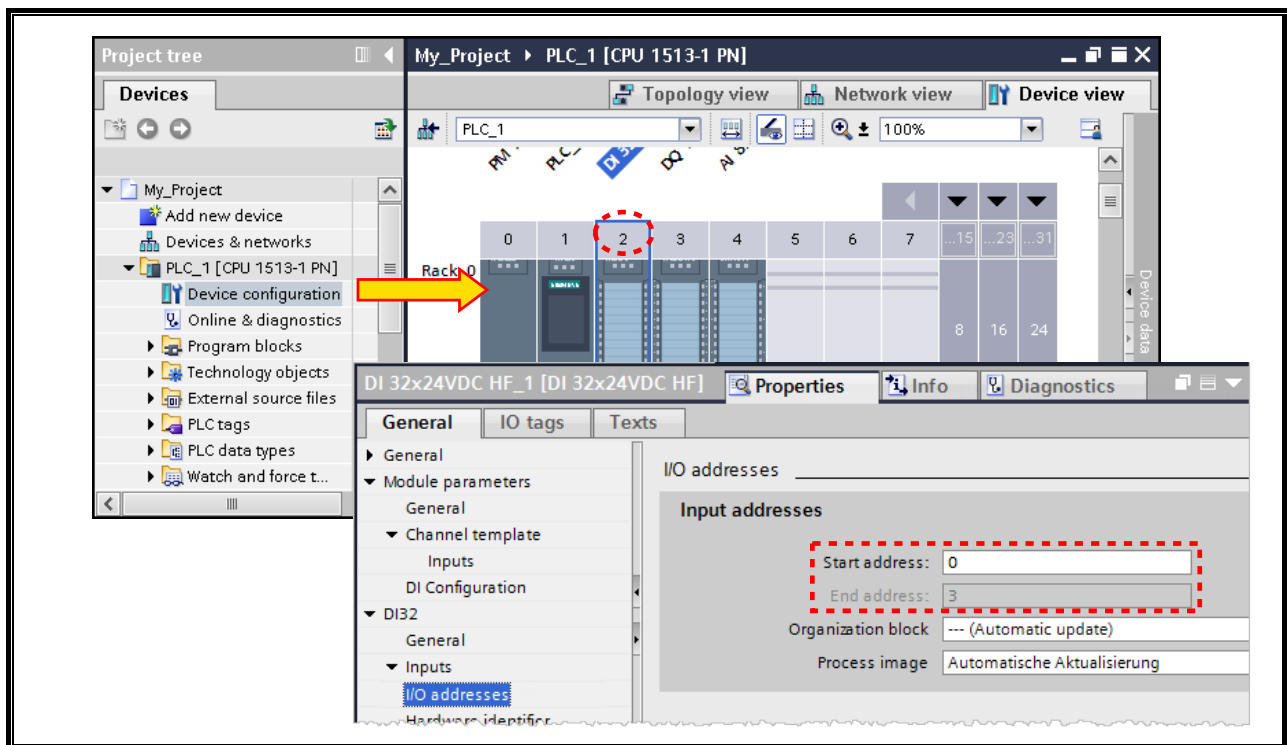
1. In the "Inspector window", select the "Properties" tab and there "Display".
2. Set the display language to "English".
3. Enable the display protection and enter a password

##### Note on Password Assignment:



Upper and lower case is not relevant, since only the letters A to Z and digits 0 to 9 can be selected when making entries on the Display.  
Since there is no Display keypad, it is recommended that you select a simple (possibly only numerical) password.

#### 4.12.11. Exercise 11: Addresses of the DI Module



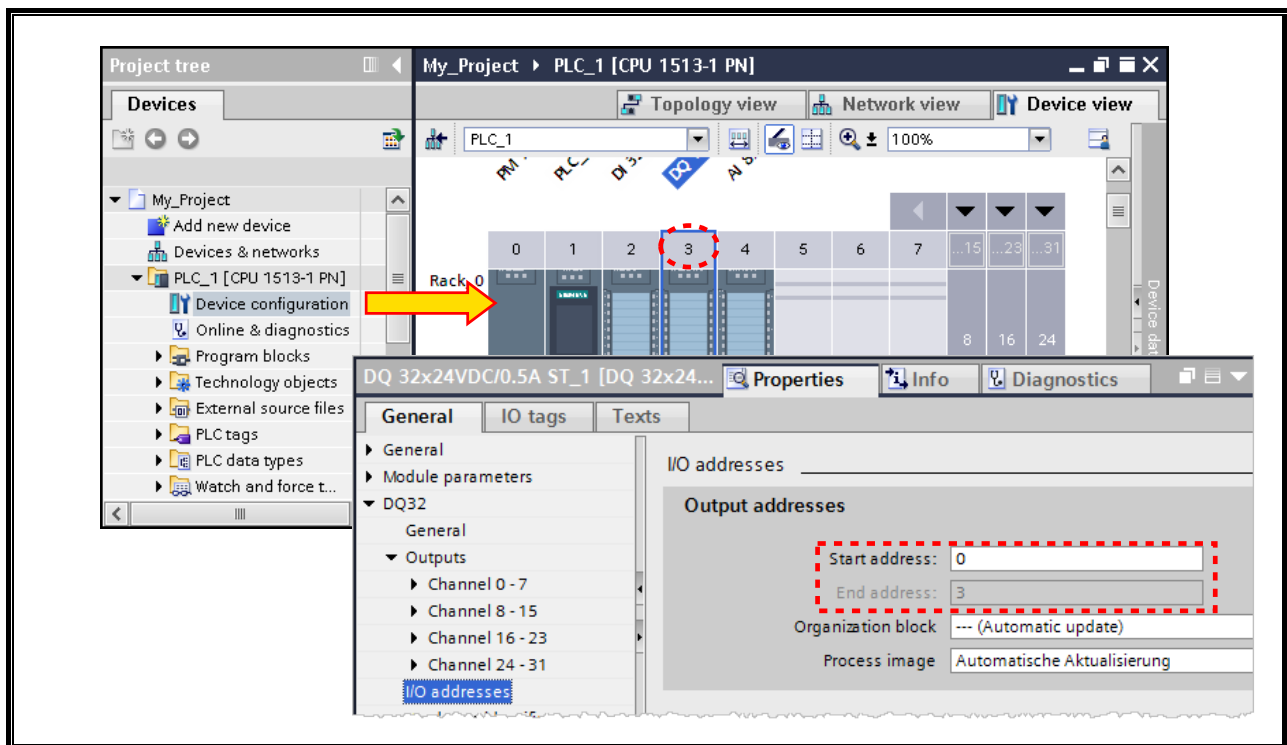
#### Task

Parameterize the I/O addresses of the DI module as shown in the picture.

#### What to Do

1. In the Device view, select the DI module (see picture).
2. In the "Inspector window", activate the "Properties" tab and there under "DI32" the "I/O addresses".
3. In the dialog box that appears, enter the I/O address 0 shown in the picture.

#### 4.12.12. Exercise 12: Addresses of the DO Module



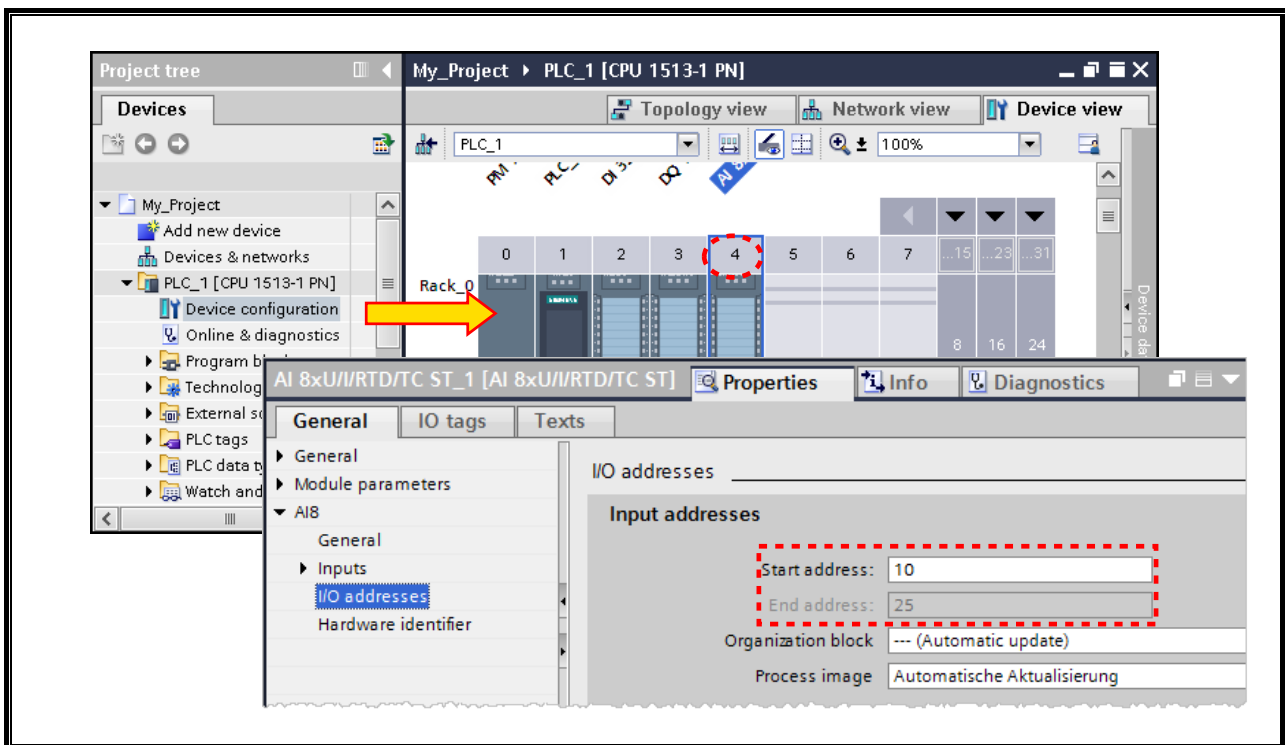
#### Task

Parameterize the I/O addresses of the DO module as shown in the picture.

#### What to Do

1. In the Device view, select the DO module (see picture).
2. In the "Inspector window", activate the "Properties" tab and there under "DO32" the "I/O addresses".
3. In the dialog box that appears, enter the I/O address 0 shown in the picture.

### 4.12.13. Exercise 13: Addresses of the AI Module



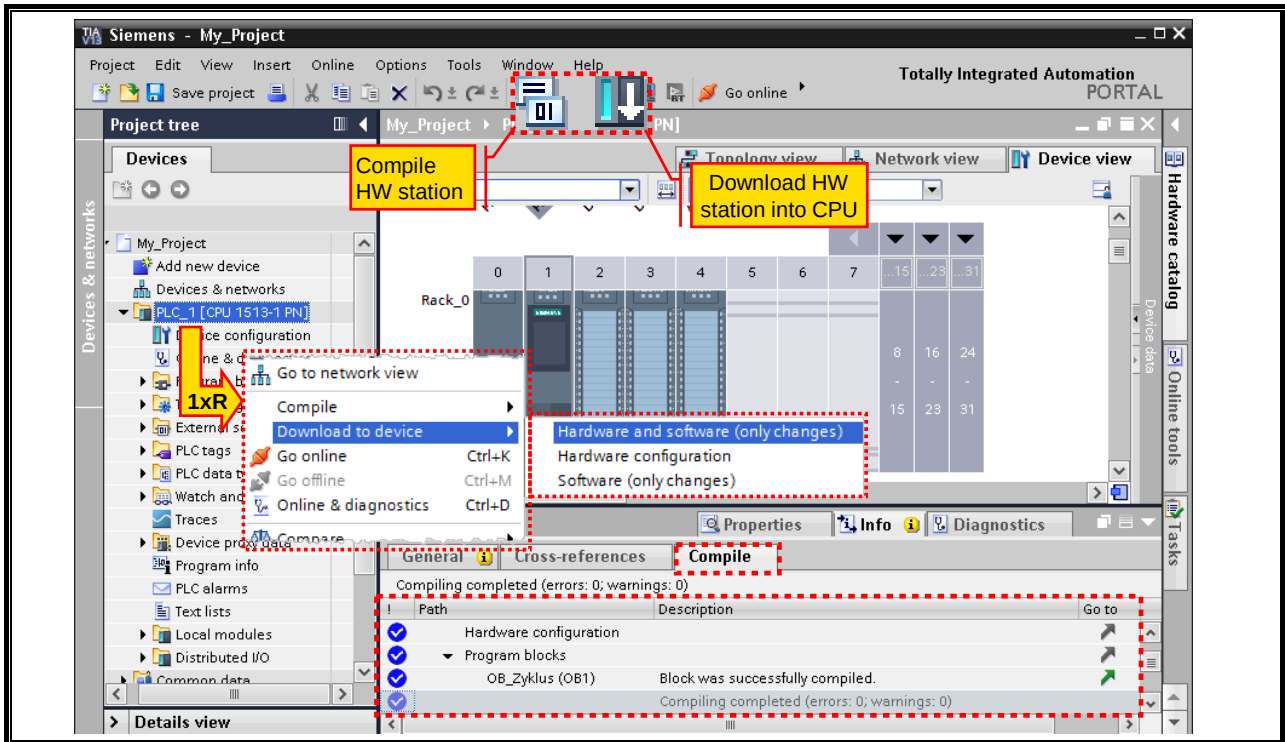
#### Task

Parameterize the I/O addresses of the AI module as shown in the picture.

#### What to Do

1. In the Device view, select the AI module (see picture).
2. In the "Inspector window", activate the "Properties" tab and there under "AI8" the "I/O addresses".
3. In the dialog box that appears, enter the I/O address 10 shown in the picture.

#### 4.12.14. Exercise 14: Compiling the Device Configuration and Downloading it into the CPU



#### Task

The configuration and parameterization of the S7-1500 hardware station is to be compiled and then downloaded into the CPU. So that the CPU can switch to the RUN state after loading, the software or one CPU program must also be loaded.

At the moment, your CPU program only consists of the automatically created block "Main" (OB1), which doesn't yet contain any instructions, but is sufficient so that the CPU switches to the RUN state during a restart.

#### What to Do

1. In the Project view, select your S7-1500 station.



2. Compile the HW-Station (either via the context menu (right-click on the station) or via the button shown here on the left (see also picture) and in the Inspector window in the "Info" tab, check whether the compilation was completed without errors.

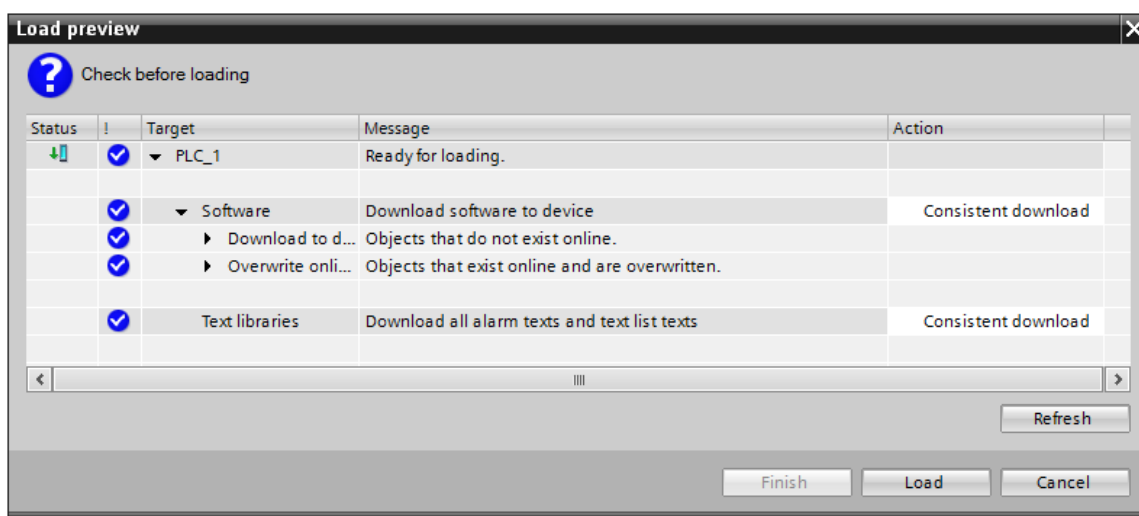


3. After an error-free compilation, download the HW-Station complete with hardware and software into the CPU (either via the context menu (right-click on the station) or via the button shown here on the left (see also picture)

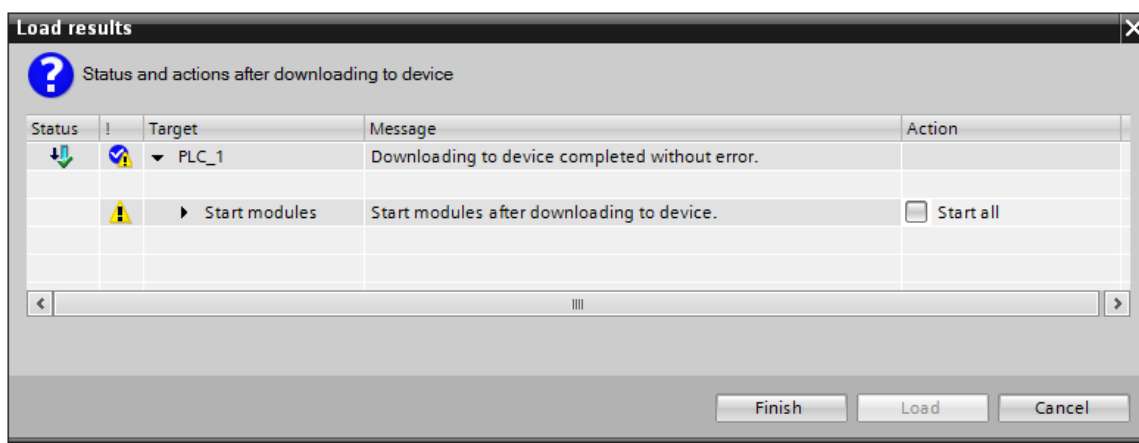
Fill-in the dialog that appears as follows and conclude it with "Load":

**Continued on the next page**

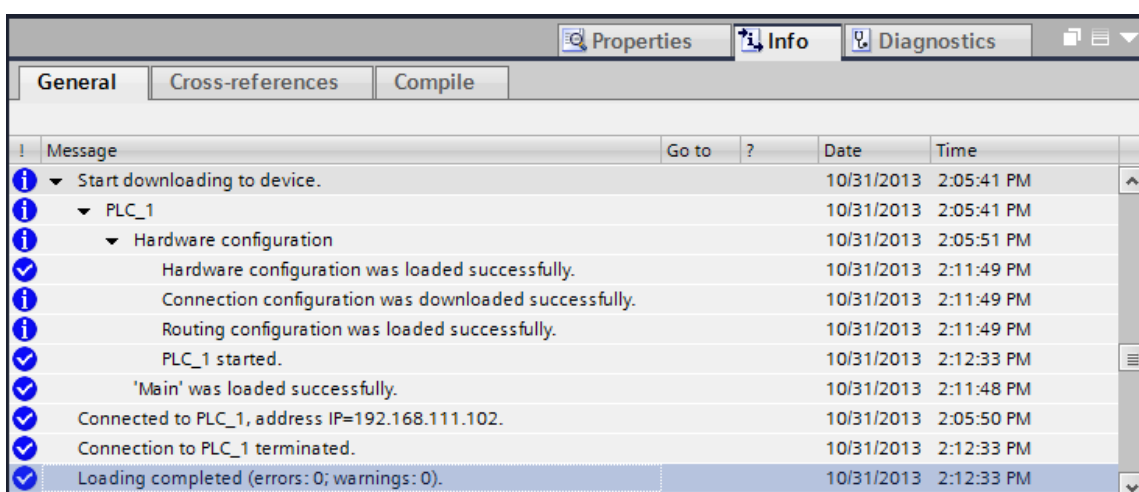




After you have activated "Load", another dialog appears which you fill-in as follows and then conclude with "Finish".

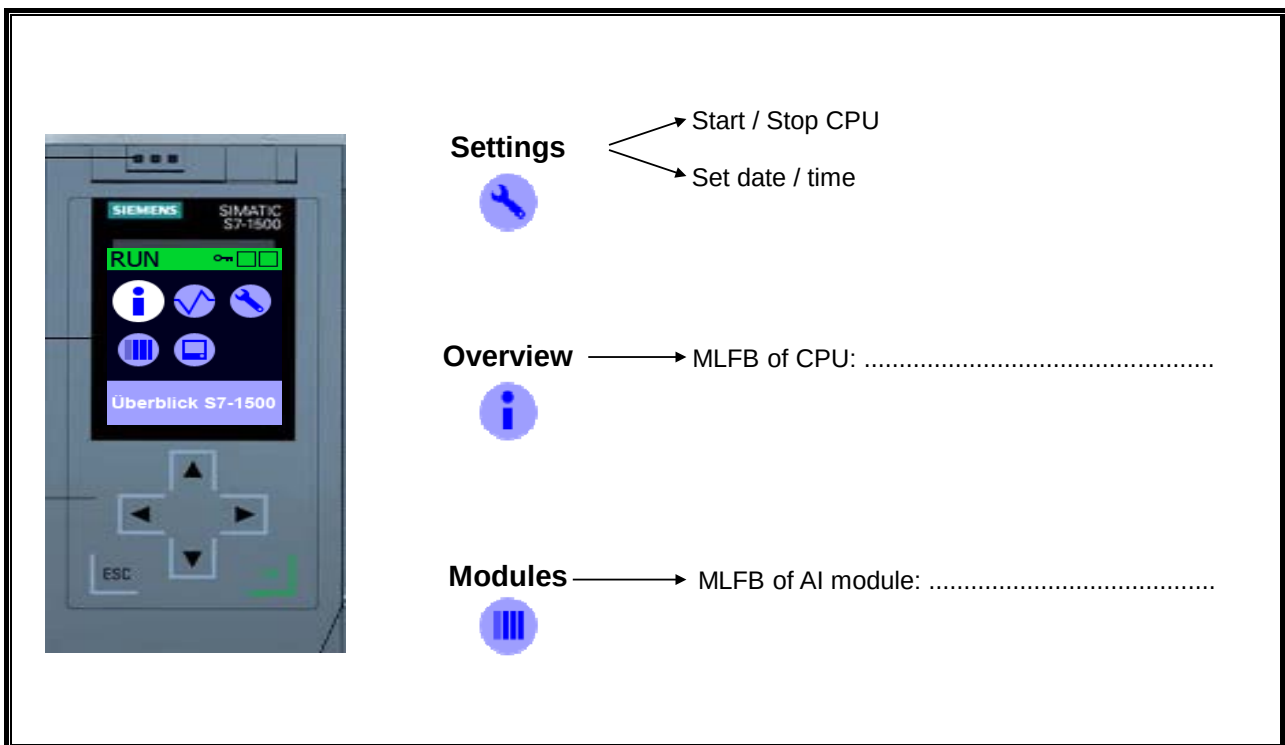


- In the "Inspector window" under "Info -> General" check the result of the hardware configuration download:



- Save your project.

#### 4.12.15. Exercise 15: Switching the CPU to RUN using the CPU-Display



#### Task

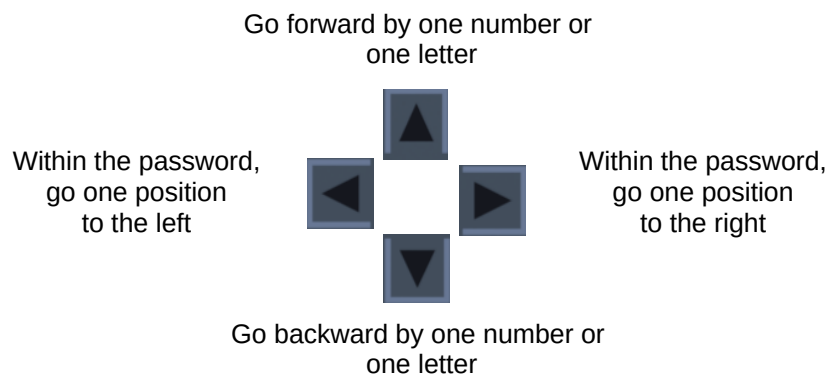
Using the Display, the CPU is to be stopped and restarted and the CPU date and time is to be set. Furthermore, you are to determine the order numbers (MLFBs) of the CPU and the central AI module.

You navigate the Display menus using the cursor keys, activate a menu using OK and exit the menu using ESC.

#### What to Do

1. Use the Display (menu "Settings", see picture) to stop the CPU and then restart it. To do so, you must first enter the Display password that you configured in the hardware configuration in the CPU Properties.

You enter the Display password as follows:

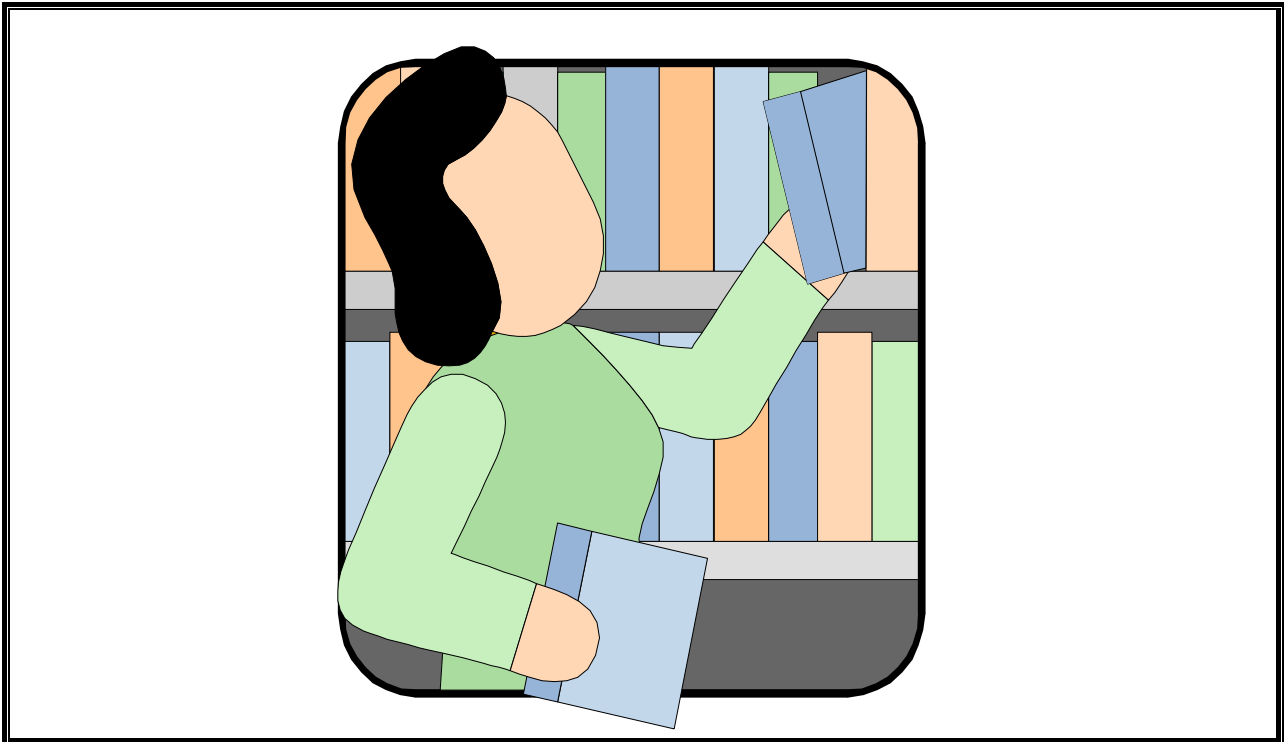


Conclude the password entry with "OK".

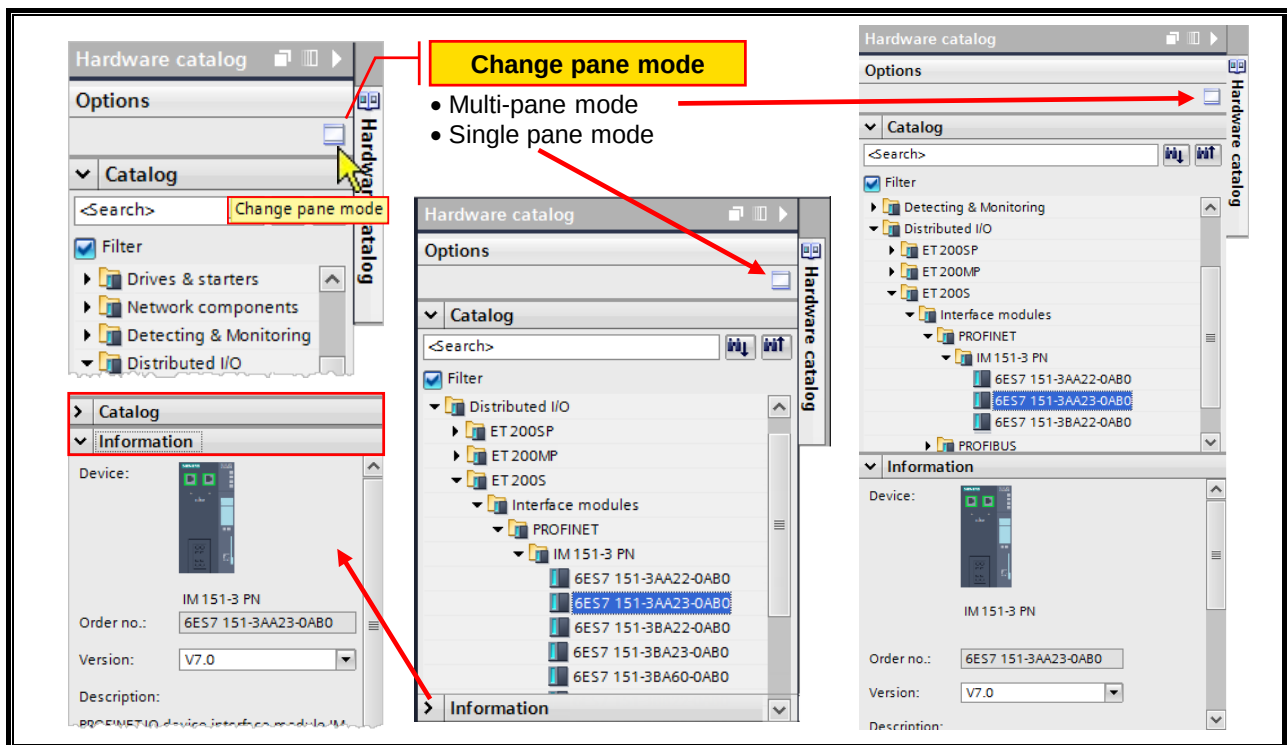
**Continued on the next page**

2. Use the Display (menu "Settings", see picture) to set the date and time of the CPU. Check your settings with the programming device via  
*Online accesses -> Intel(R) 82574L -> [your station] -> Online & diagnostics -> Functions -> Set time*
3. Use the Display (menu "Overview", see picture) to research the order number (MLFB) of the CPU and enter it in the picture (on the previous page).
4. Use the Display (menu "Modules", see picture) to research the order number (MLFB) of the analog input module (AI) and enter it in the picture (on the previous page).
5. Bring the Display back to its main menu.

## **4.13. Additional Information**



#### 4.13.1. 'View' Settings of the "Hardware catalog" Task Card



You can choose between two pane modes:

- **Single pane mode:**  
There is only one pane open at a time. If a different pane is selected, the previously open pane is closed automatically.
- **Multi-pane mode:**  
Several panes can be open at the same time.

#### Setting for the Device Configuration

Since there is generally more than one version of a module when configuring the devices (CPUs, I/O modules), the required version must be selected.

Since this additional information on the modules selected in the catalog is shown in the "Information" pane, it is recommended that the **multi-pane mode** is set here.

#### 4.13.2. Firmware Update

Firmware of the modules can be updated

→ Display and modules (CPU, TM, SM, CP, CM, distributed I/O)

**Possibilities of a Firmware Update:**

- Offline using SIMATIC Memory Card with the Windows Explorer
- Offline using STEP 7 by means of card creation in the TIA Portal
- Online using STEP 7 in the TIA Portal (distributed I/O exclusively online)
- SIMATIC Webserver of the CPU (in the planning stages)

**Approach:**

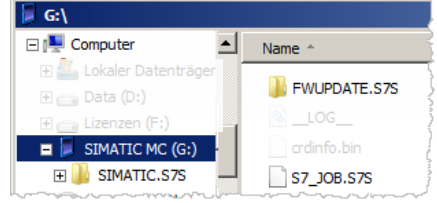
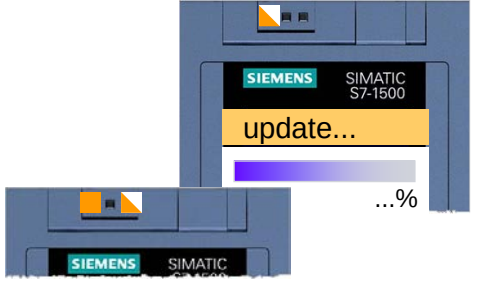
- 1** Download the new Firmware file from the Service & Support pages.  
<http://support.automation.siemens.com>
- 2** ...depending on selected possibility of an update  
...depending on loaded Firmware files

Procuring the new Firmware is always the first step.

The next steps depend on the selected update possibility.

## 4.13.2.1. Firmware Update → Offline using SIMATIC Memory Card with the Windows Explorer

- 3 Save the Firmware files   on the SIMATIC Memory Card in the Windows Explorer  
 delete first, in case there is data on the card
- 4 Switch off the power supply of the CPU  
Insert the card into the card slot of the CPU  
and then switch the power supply of the CPU back on
- 5 **During the update**, the Display shows a progress bar and the **left LED of the CPU flashes orange**
- 6 **After the update**, the status is shown in the Display as a message and the **left LED is orange and the right LED flashes orange**
- 7 Remove the card
- 8 Switch off the power supply of the CPU, delete data on the card, re-insert card and switch the power supply of the CPU back on

When the CPU or the Display is updated, Step  is necessary in any case. However, this is also recommended at the end of every module update.

**Status of the Update in the Display (only S7-1500)**

Only the S7-1500 has a Display.

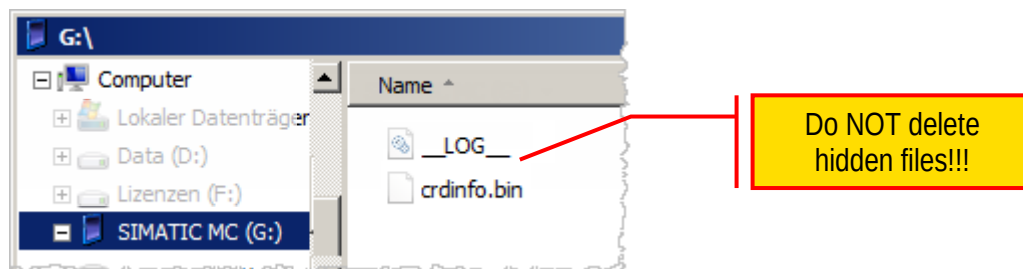
**Status of the Update with LEDs**

If the **middle LED (red, error)** lights up, the update was not successfully completed or it could not be carried out.

**Necessary Firmware Update Files**

 FWUPDATE.S7S  
 S7\_JOB.S7S

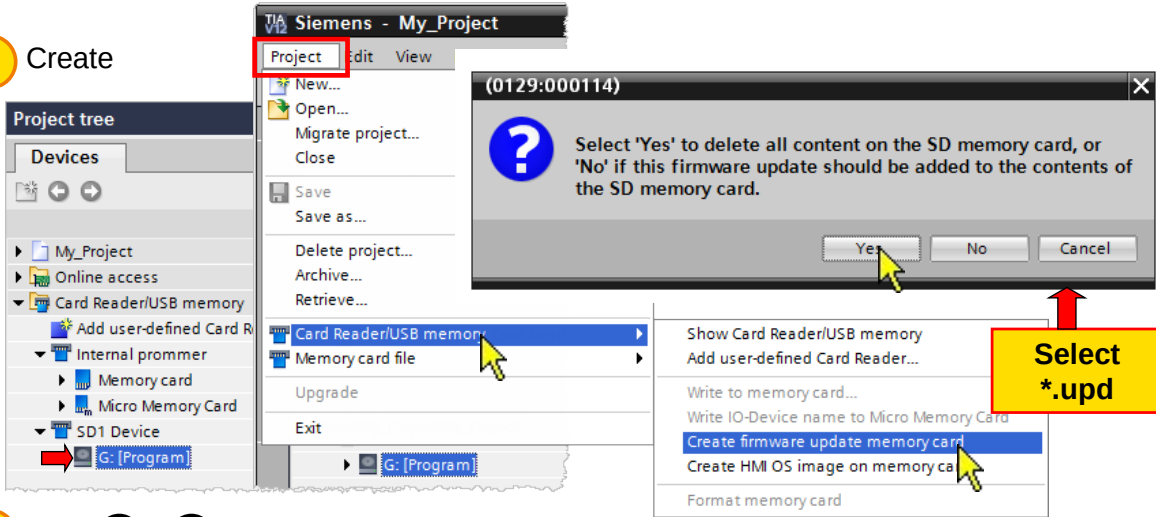
Directory with FW files (FWUPDATE.S7S) and data ID file (S7\_JOB.S7S)

**Empty Card (Windows Explorer)**

## 4.13.2.2. Firmware Update → Offline using STEP7 by means of Card Creation in the TIA Portal

**3** Save the Firmware file `MLFB_ausgabe.upd` on the computer  
 ☞ Card does NOT have to be deleted first in case there is data on the card

**4** Create



The screenshot shows the 'Project' menu with options like 'New...', 'Open...', 'Migrate project...', 'Close', 'Save', 'Save as...', 'Delete project...', 'Archive...', 'Retrieve...', 'Card Reader/USB memory', 'Memory card file', 'Upgrade', and 'Exit'. A sub-menu for 'Card Reader/USB memory' is open, showing options like 'Show Card Reader/USB memory', 'Add user-defined Card Reader...', 'Write to memory card...', 'Write IO-Device name to Micro Memory Card', 'Create firmware update memory card', 'Create HMI OS image on memory card', and 'Format memory card'. A dialog box titled '(0129:000114)' is displayed, asking: 'Select 'Yes' to delete all content on the SD memory card, or 'No' if this firmware update should be added to the contents of the SD memory card.' The 'Yes' button is highlighted. A yellow callout box with a red arrow points to the 'Create firmware update memory card' option in the sub-menu, with the text 'Select \*.upd'.

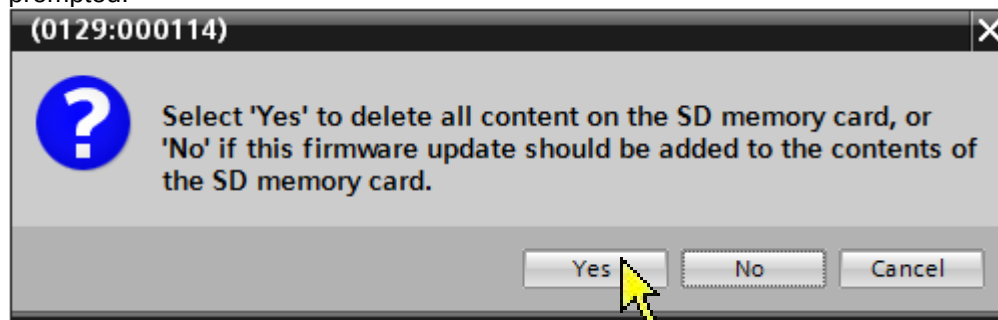
**5** Step **6**... **8** as for "Offline using SIMATIC Memory Card with the Windows Explorer"

Necessary Firmware Update File e.g. `6ES7 521-1BL00-0AB0 R04.08.04.upd`

FW file (MLFB\_ausgabe.upd)

## Card Contents

The SD card for the update does not necessarily have to be deleted first. During creation you are prompted:



- **Yes**  
Card is deleted and update card is created
- **No**  
The update is only added to the card contents and the card type is set to "Update firmware". This can only be done for updating the Display since the program is active and CPU + I/O modules cannot be updated in this state.  
 ☞ For updating the CPU or the Display, an empty card is necessary in order to conclude the update (see Step **8** ).  
 For that reason, it is generally recommended to delete the "Program" card or to use a separate card for updates.



## 4.13.2.3. Firmware Update → Online using STEP 7 in the TIA Portal

3 Save the Firmware file MLFB\_ausgabe.upd on the computer

4 Establish an online connection to the CPU

5 Select \*.upd

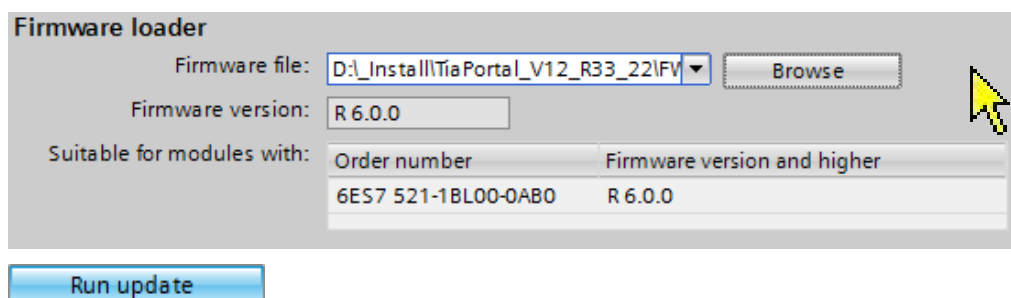
6 Run update

7 Restart CPU

Distributed I/O without memory card is exclusively update-able online

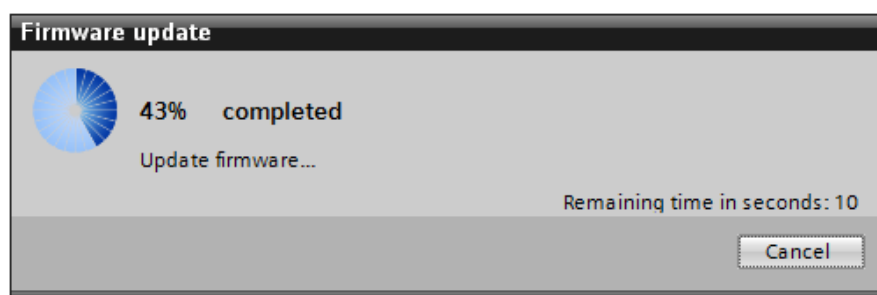
CPU should be switched to STOP for updates of I/O modules

Before running the update, the appropriate \*.upd must be selected.



Before running an update of I/O modules, the CPU should be switched to STOP, if the active user program uses this module.

Wait until update is completed...



## CPU Update

Advantage: The current "Program" card is not changed.

**Attention!**

The retentiveness is however deleted, that is, the program is then reset to the loading state.

#### **4.13.2.4. Firmware Update → Important Facts**

During an update, the supply voltage must **not** fail or be removed.

Distributed I/O without memory card is exclusively update-able online using STEP7.

After updating the CPU or the Display (only S7-1500), the CPU should be switched off, an empty memory card inserted in the CPU and the CPU switched back on.

It is possible to do a **simultaneous update** of several modules offline using the memory card.

→ Respective Firmware update files are stored in the folder for Firmware update

#### **The Sequence is Relevant if Several Module Types are Updated**

1. Update of the Display Firmware (only S7-1500)
2. Update of the central I/O modules
3. Update of the distributed I/O modules
4. Update of the Boot loader of the CPU
5. Update of the CPU

# Contents

|           |                                                              |            |
|-----------|--------------------------------------------------------------|------------|
| <b>5.</b> | <b>PLC Tags.....</b>                                         | <b>5-2</b> |
| 5.1.      | Global and Local Tags.....                                   | 5-3        |
| 5.2.      | Meaning of Variables and Data Types.....                     | 5-4        |
| 5.3.      | PLC-Tags and Constants.....                                  | 5-5        |
| 5.3.1.    | PLC Tags in the Device View .....                            | 5-7        |
| 5.3.2.    | Details View of PLC Tags .....                               | 5-8        |
| 5.3.3.    | Finding / Replacing / Sorting PLC Tags.....                  | 5-9        |
| 5.3.4.    | Error Indication in the PLC Tag Table .....                  | 5-10       |
| 5.3.5.    | Copy & Paste PC Tags to Excel .....                          | 5-11       |
| 5.4.      | Using a PLC Tag as an Operand.....                           | 5-12       |
| 5.4.1.    | Absolute and Symbolic Addressing .....                       | 5-13       |
| 5.4.2.    | Renaming / Rewiring PLC Tags.....                            | 5-14       |
| 5.4.3.    | Defining (Declaring) Tags while Programming .....            | 5-15       |
| 5.4.4.    | Monitoring PLC Tags .....                                    | 5-16       |
| 5.4.5.    | Retentiveness of PLC Tags .....                              | 5-17       |
| 5.4.6.    | HMI Access to PLC Tags.....                                  | 5-18       |
| 5.4.7.    | Exercise 1: Copying the PLC Tag Table from the Library ..... | 5-19       |
| 5.4.8.    | Exercise 2: Creating a Conveyor Model Tag Table.....         | 5-20       |
| 5.4.9.    | Exercise 3: Monitoring the PLC Tag Table "Conveyor" .....    | 5-21       |

## 5. PLC Tags

**At the end of the chapter the participant will ...**

- ... be familiar with the meaning and use of tags
- ... be familiar with the difference between local and global tags
- ... be familiar with the difference between absolute and symbolic addressing
- ... be able to declare a PLC tag table
- ... be able to rename and rewire PLC tags
- ... be able to create global data blocks
- ... be able to declare tags in data blocks and be able to use them in the program
- ... be familiar with the properties of optimized data blocks and be able to use them



## 5.1. Global and Local Tags

|                                      | Global Tags                                                                                                                                                                         | Local Tags                                                                                                                                                                  |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Validity range                       | <ul style="list-style-type: none"> <li>• valid throughout the entire CPU i.e. all blocks have access</li> <li>• the name of the tag must be unique within the entire CPU</li> </ul> | <ul style="list-style-type: none"> <li>• only valid in the block in which they have been declared</li> <li>• the name of the tag must be unique within the block</li> </ul> |
| Operands                             | <ul style="list-style-type: none"> <li>• Inputs</li> <li>• Outputs</li> <li>• Bit memories</li> <li>• Tags in data blocks</li> <li>• SIMATIC Timers / Counters</li> </ul>           | <ul style="list-style-type: none"> <li>• Temporary tags (in all logic blocks)</li> <li>• Static tags (only in FBs)</li> </ul>                                               |
| Location of declaration (definition) | <ul style="list-style-type: none"> <li>• PLC tag table</li> <li>• Global data blocks</li> </ul>                                                                                     | <ul style="list-style-type: none"> <li>• Declaration part of block</li> </ul>                                                                                               |
| Symbolic Representation              | <ul style="list-style-type: none"> <li>• Global tags are presented in quotation marks<br/>Example: "Max"</li> </ul>                                                                 | <ul style="list-style-type: none"> <li>• Local tags are presented preceded by #<br/>Example: #Max</li> </ul>                                                                |

### Validity Range of Tags

Tags that are declared in the PLC tag table or in a global data block can be addressed by all CPU program blocks. For that reason, these tags are called global tags.

Tags and parameters that are declared in the declaration part of a logic block are local operands; they can only be used in the statement part of the same block.

## 5.2. Meaning of Variables and Data Types

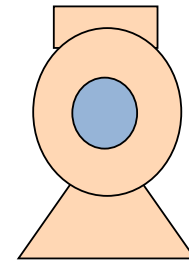
**Variables represent the abstraction of reality and permit you to save and later continue to process values.**

**By declaring a variable, the following properties are defined:**

- Symbolic name
- Data type
- Validity range
- Memory area

**The data type establishes:**

- The possible value range (e.g. INT: -32 768 to +32 767)
- The permitted instructions (e.g. math instructions: +I, -I)
- Data types abstract from the underlying representation of the bits in the memory



**Actual\_speed: REAL**

**Set\_speed: REAL**

**Disturbance: BOOL**

**Enable: BOOL**

### "Traditional" PLC Operands

In traditional PLC programming, PLC memory addresses were accessed directly by specifying the operand area (for example: M= bit memory, I=inputs, etc.), the access length (for example: B=byte, W=word, etc.) and by specifying the byte/(bit) address. These memory areas addressed via absolute addresses could be used within the program for different purposes, for example, as integer (e.g. DINT), as floating-point number (e.g. REAL) or simply as a collection of individual signals (e.g. WORD). Up until now, it was up to the programming person to make a mental note of the format and the purpose of the individual memory locations. Accordingly the programming was prone to errors.

### The Meaning of Variables

Next to commands, variables are the most important elements of a programming system. Their task is to save values in a program so that they can be further processed at a later time. The value of a variable can be saved "anywhere" in the PLC memory.

The data represents an abstraction of reality in which irrelevant properties of objects are ignored.

### Data Types

It is often quite difficult to decide how data is to be represented and the available possibilities quite often restrict the choice. On the one hand, the object properties the data describe must be correctly reflected. On the other hand, it must also be possible to carry out the instructions necessary for process with the data.

The data type determines which values are accepted by data and which instructions can be carried out with these values. The data type uniquely defines

- the possible value range
- the allowed instructions

### 5.3. PLC-Tags and Constants

The screenshot displays the SIMATIC Manager interface. On the left is the 'Project tree' showing the project structure. The main window is titled '...PU 1516-3 PN/DP' and contains two tabs: 'Tags' and 'User constants'. The 'Tags' tab is active, showing a table of 'STEP7 classic symbols'.

|    | Name               | Data type | Address | Retain                              |
|----|--------------------|-----------|---------|-------------------------------------|
| 1  | S_Left             | Bool      | %I0.3   |                                     |
| 2  | S_OperationOFF     | Bool      | %I0.0   |                                     |
| 3  | M_2Hz              | Bool      | %M10.3  | <input checked="" type="checkbox"/> |
| 4  | M_Auto_Right       | Bool      | %M16.4  |                                     |
| 5  | M_Conv_Fault       | Bool      | %M17.0  |                                     |
| 6  | OD_Conv_Monitoring | Timer     | %T17    |                                     |
| 7  | S_Acknowledge      | Bool      | %I0.7   |                                     |
| 8  | S_OperationON      | Bool      | %I0.1   |                                     |
| 9  | S_Right            | Bool      | %I0.2   |                                     |
| 10 | P_Fault            | Bool      | %Q0.7   |                                     |
| 11 | P_Operation        | Bool      | %Q0.1   |                                     |

Below the 'STEP7 classic symbols' table is the 'PLC tags' table, which is also visible. It has tabs for 'Tags', 'User constants', and 'System constants'. The 'Tags' tab is active, showing a table with the following data:

|   | Name      | Tag table         | Data type | Value | Comment                  |
|---|-----------|-------------------|-----------|-------|--------------------------|
| 1 | max_time  | Default tag table | Time      | T#6s  | max. transportation time |
| 2 | <Add new> |                   |           |       |                          |

#### PLC Tags and Constants

The PLC tag table contains the declaration (definition) of CPU-wide valid and thus global tags and constants. For each CPU added in the project, a PLC tag table is automatically created. The PLC tag table contains one tab each for tags, user constants and system constants.

Tags are operands with changeable content used in the user program. Constants are operands with content that cannot be changed. When a tag or constant is declared in the PLC tag table, the symbolic name (for example, "M\_Jog\_RIGHT"), the data type (e.g. Bool) and the absolute address (for example, M16.2) is defined. No absolute address is required for constants.

#### Creating Tags with Group Function

By clicking on the "Fill" symbol in the lower right corner of the cell and then dragging it down, tags are automatically created (comparable to Excel).

It is possible to automatically create tags through the "Name" and "Address" columns. The new tags are created with the name of the current tag appended by a consecutive number. From a tag with the name "T\_Station", new tags are then created with the names "T\_Station\_1", "T\_Station\_2" etc.

## User Constants

A constant defines an unchangeable data value. During program execution, constants can be read by various program elements, but they cannot be overwritten. Changing the constant value while the program is running is not possible.

In TIA Portal, it is possible to declare symbolic names for constants so as to make static values available in the program under one name. These symbolic constants are valid throughout the CPU. The declaration of the constants is made in the "User constants" tab of the PLC tag table.

## System Constants

Every hardware object, for example, a module or even transfer areas of I-Devices, has a hardware ID which is automatically assigned by STEP 7 during the configuration of a hardware component.

In S7-1500 and even in S7-1200, these hardware IDs have the same function as the diagnostic addresses of hardware components in the S7-300/400. Even when the I/O addresses of modules are changed, their hardware ID remains the same. When I/O addresses are changed, the instructions that were addressed through a hardware ID do not have to be adjusted.

## Hardware ID in the System Constants

As well, entries are made in the PLC tags ("System constants" tab) by STEP 7 for every configured hardware component. These entries are read-only. In addition to the integer value, name, hardware data type and in the comment the path of the HW component are also assigned.

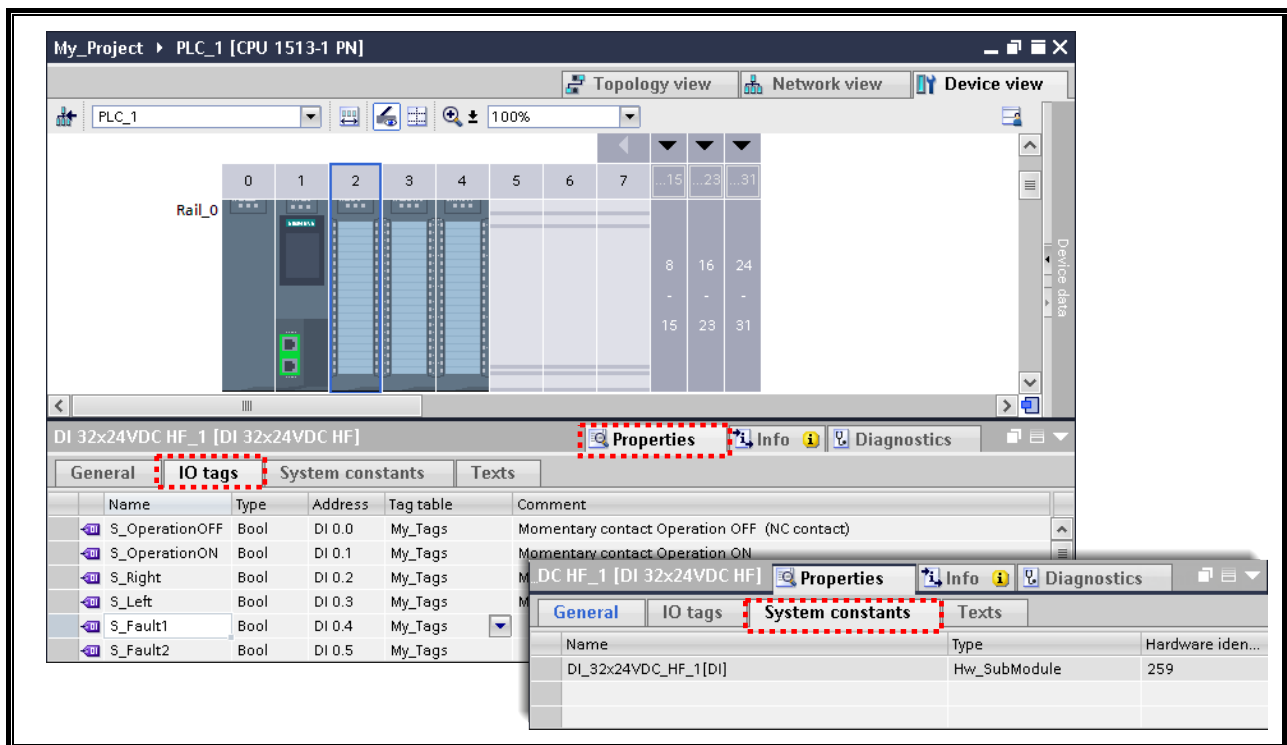
The name corresponds to the editable name in the properties of the component (Inspector window).

Recommendation: To simplify the identification of hardware objects, assign, as far as possible, "speaking" names for the hardware objects during configuration of the hardware. The names should take into consideration the association to a system part or to a station.

|    | Name                                                                                                          | Data type    | Value | Comment |
|----|---------------------------------------------------------------------------------------------------------------|--------------|-------|---------|
| 40 |  PLC_1[Exec]               | Hw_SubModule | 52    |         |
| 41 |  PROFINET_interface_1      | Hw_Interface | 64    |         |
| 42 |  Port_1[PN]                | Hw_Interface | 65    |         |
| 43 |  Port_2[PN]                | Hw_Interface | 66    |         |
| 44 |  DI_32x24VDC_HF_1[DI]      | Hw_SubModule | 259   |         |
| 45 |  DQ_32x24VDC_0.5A_ST_1[DO] | Hw_SubModule | 260   |         |
| 46 |  AI_8xU_I_RTD_TC_ST_1[AI]  | Hw_SubModule | 261   |         |
| 47 |  PM_70W_120_230VAC         | Hw_SubModule | 277   |         |



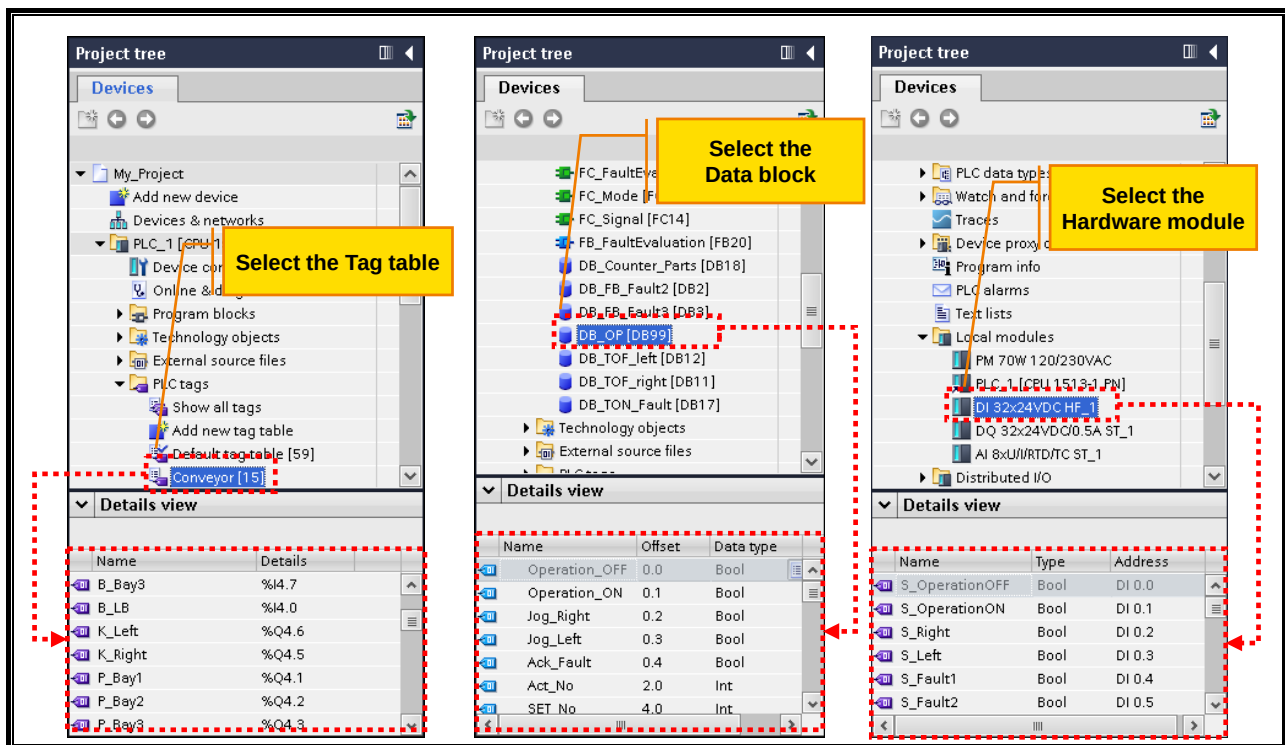
### 5.3.1. PLC Tags in the Device View



The PLC tags of inputs and outputs can be declared and changed in the Device view. For this, in the Properties in the Inspector window, open the IO tags tab.

In addition, the system constants of the selected hardware are displayed in the System constants tab.

### 5.3.2. Details View of PLC Tags



The Details View shows the... :

- ...tags of the selected tag table
- ...tags of the selected data block
- ...channels of the selected local modules and their tags

### 5.3.3. Finding / Replacing / Sorting PLC Tags

Click on column name to sort

|    | Name               | Data type | Address | Retain | Visible ... | Accessible from HMI | Comment                                      |
|----|--------------------|-----------|---------|--------|-------------|---------------------|----------------------------------------------|
| 1  | S_Left             | Bool      | %I0.3   |        |             |                     | Momentary contact Jog conveyor left          |
| 2  | S_OperationOFF     | Bool      | %I0.0   |        |             |                     | Momentary contact Operation OFF (NC contact) |
| 3  | M_2Hz              | Bool      | %M10.3  |        |             |                     | Memory bit - flashing frequency 2 Hz         |
| 4  | M_Auto_Right       | Bool      | %M16.4  |        |             |                     | Memory bit conveyor RIGHT in auto mode       |
| 5  | M_Conv_Fault       | Bool      | %M17.0  |        |             |                     | Memory bit Conveyor fault                    |
| 6  | OD_Conv_Monitoring | Timer     | %T17    |        |             |                     | Timer, Conveyor monitoring in Auto mode      |
| 7  | S_Acknowledge      | Bool      | %I0.7   |        |             |                     | Momentary contact Fault acknowledgement      |
| 8  | S_OperationON      | Bool      | %I0.1   |        |             |                     | Momentary contact Operation ON               |
| 9  | S_Right            | Bool      | %I0.2   |        |             |                     | Momentary contact Jog conveyor right         |
| 10 | P_Fault            | Bool      | %Q0.7   |        |             |                     | Indicator light Conveyor fault               |
| 11 | P_Operation        | Bool      | %Q0.1   |        |             |                     | Indicator light Operation ON                 |
| 12 | M_aux_Open         | Bool      | %M15.1  |        |             |                     | Edge auxiliary memory bit System ON          |
| 13 | M_Jog_Right        | Bool      | %M16.2  |        |             |                     | Memory bit Jog conveyor RIGHT                |
| 14 | M_Weightstore_full | Bool      | %M8.1   |        |             |                     | Memory bit Weight store full                 |
| 15 | M_aux_Stop         | Bool      | %M16.1  |        |             |                     | Edge auxiliary memory bit conveyor stop      |
| 16 | M_1Hz              | Bool      | %M10.5  |        |             |                     |                                              |
| 17 | P_RestartAuto      | Bool      | %Q0.6   |        |             |                     | Indicator light for automatic warm restart   |
| 18 | M_WeightOK         | Bool      | %M35.0  |        |             |                     | is equal to 1 when weight ok                 |
| 19 | P_RestartMan       | Bool      | %Q0.5   |        |             |                     | Indicator light for manual warm restart      |
| 20 | P_VoltageDisplay1  | Int       | %QW32   |        |             |                     | Voltage Display 1                            |
| 21 | S_Slider1          | Int       | %IW10   |        |             |                     | Slider 1                                     |
| 22 | P_WeightFault      | Bool      | %Q0.0   |        |             |                     | Indicator light - weight invalid             |
| 23 | M_aux_LS           | Bool      | %M16.0  |        |             |                     | Edge auxiliary memory bit light barrier      |
| 24 | M_PartNo_even      | Bool      | %M8.5   |        |             |                     | Memory bit actual part no is even            |

Find and replace options:

- Find: M\_Jog\_Right
- Whole words only: ☐
- Match case: ☐
- Find in substructures: ☐
- Find in hidden texts: ☐
- Use wildcards: ☐
- Use regular expressions: ☐
- Whole document: ☐
- From current position: ☒
- Selection: ☐
- Down: ☐
- Up: ☒
- Find: [Find]
- Replace with: [Replace]
- Replace: [Replace]
- Replace all: [Replace all]

#### Finding / Replacing

In the PLC tag table, tags can be found and replaced via the "Tasks" task card. Dummies can also be used (? for one character, \* for several characters).

Example of "Find and Replace":

Assign byte address 4 to all outputs with byte address 8:

Find: Q 8. Replace with: Q 4.

#### Sorting

By clicking on one of the column names "Name", "Data type" or "Address", the tags are sorted alphabetically or according to address (ascending or descending) depending on the column.

### 5.3.4. Error Indication in the PLC Tag Table

The screenshot shows the 'STEP7 classic symbols' table in the SIMATIC TIA Portal. The table lists 19 tags with their names, data types, addresses, and other properties. Several annotations highlight errors:

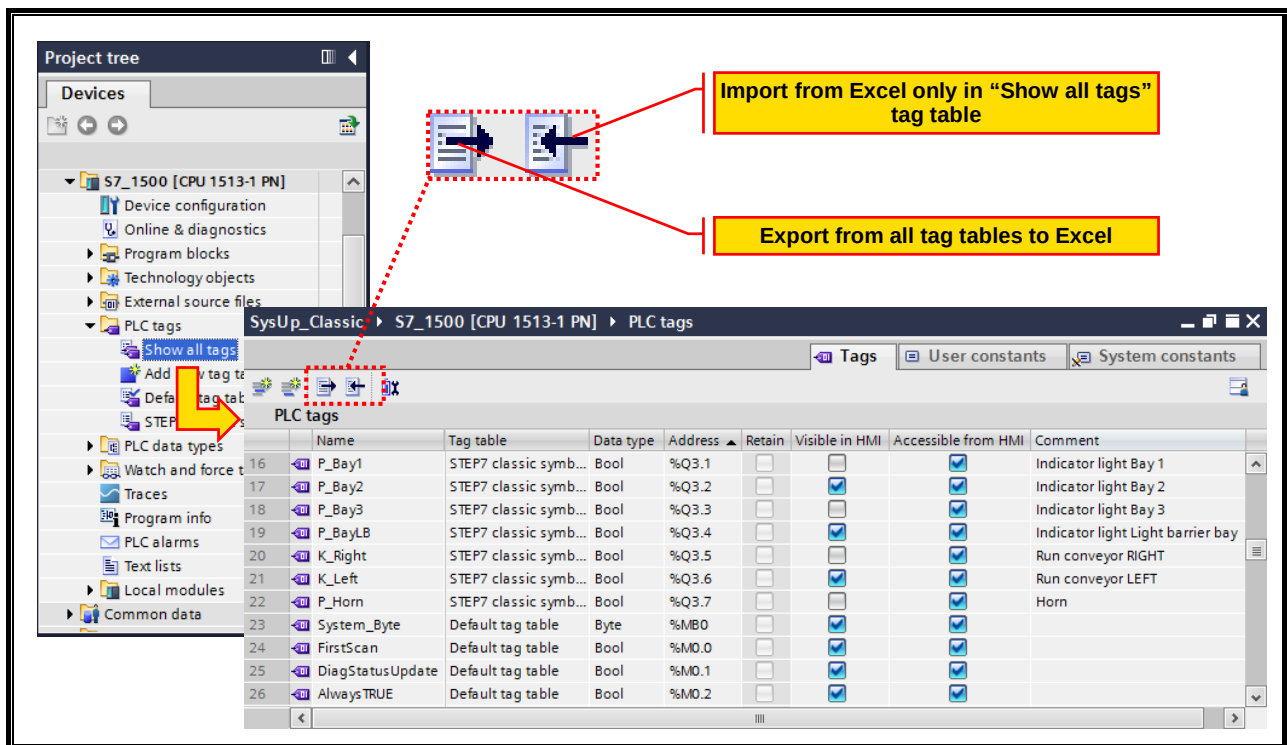
- Row 3:** The tag name 'M\_WeightOK(1)' is highlighted with a red dashed box. A yellow callout box states: "If the name already exists in the table, a "(1)" is automatically added."
- Row 6:** The address '%M16.4' is highlighted in yellow. A yellow callout box states: "This absolute address is used twice!" (referring to rows 5 and 6).
- Row 12:** The address '%A3.7' is highlighted in pink. A yellow callout box states: "The absolute address does not correspond to the data type of the tag!" (since the data type is Bool, not Char).

|    | Name               | Data type | Address | Retain | Visible in HMI | Accessible from HMI                 | Comment                                 |
|----|--------------------|-----------|---------|--------|----------------|-------------------------------------|-----------------------------------------|
| 1  | QD_Conv_Monitoring | Timer     | %T17    |        |                | <input checked="" type="checkbox"/> | Timer, Conveyor monitoring in Auto mode |
| 2  | M_WeightOK         | Bool      | %M35.0  |        |                | <input checked="" type="checkbox"/> |                                         |
| 3  | M_WeightOK(1)      | Bool      | %M35.1  |        |                | <input checked="" type="checkbox"/> |                                         |
| 4  | M_Conv_Fault       | Bool      | %M17.0  |        |                | <input checked="" type="checkbox"/> | Memory bit conveyor fault               |
| 5  | M_Auto_Right       | Bool      | %M16.4  |        |                | <input checked="" type="checkbox"/> | Memory bit conveyor RIGHT in auto mode  |
| 6  | M_Jog_Right        | Bool      | %M16.4  |        |                | <input checked="" type="checkbox"/> | Memory bit jog conveyor RIGHT           |
| 7  | M_aux_Stop         | Bool      | %M16.1  |        |                | <input checked="" type="checkbox"/> | stop                                    |
| 8  | M_aux_Op_on        | Bool      | %M15.1  |        |                | <input checked="" type="checkbox"/> | Edge auxiliary memory bit System ON     |
| 9  | M_1Hz              | Bool      | %M10.5  |        |                | <input checked="" type="checkbox"/> | Memory bit - flashing frequency 1 Hz    |
| 10 | M_2Hz              | Bool      | %M10.3  |        |                | <input checked="" type="checkbox"/> | Memory bit - flashing frequency 2 Hz    |
| 11 | M_WeightStore_full | Bool      | %M8.4   |        |                | <input checked="" type="checkbox"/> |                                         |
| 12 | P_Horn             | Char      | %A3.7   |        |                | <input checked="" type="checkbox"/> |                                         |
| 13 | K_Left             | Bool      | %Q3.6   |        |                | <input checked="" type="checkbox"/> |                                         |
| 14 | K_Right            | Bool      | %Q3.5   |        |                | <input checked="" type="checkbox"/> | Run conveyor RIGHT                      |
| 15 | P_BayLB            | Bool      | %Q3.4   |        |                | <input checked="" type="checkbox"/> | Indicator light Light barrier bay       |
| 16 | P_Bay3             | Bool      | %Q3.3   |        |                | <input checked="" type="checkbox"/> | Indicator light Bay 3                   |
| 17 | P_Bay2             | Bool      | %Q3.2   |        |                | <input checked="" type="checkbox"/> | Indicator light Bay 2                   |
| 18 | P_Bay1             | Bool      | %Q3.1   |        |                | <input checked="" type="checkbox"/> | Indicator light Bay 1                   |
| 19 | P_Fault            | Bool      | %Q0.7   |        |                | <input checked="" type="checkbox"/> | Indicator light Conveyor fault          |

#### Syntax Check

With every entry, there is a syntax check in which existing errors are displayed in RED, or for warnings in YELLOW. A still faulty PLC tag table can be saved but as long as it is still faulty, the program cannot be compiled and downloaded into the CPU.

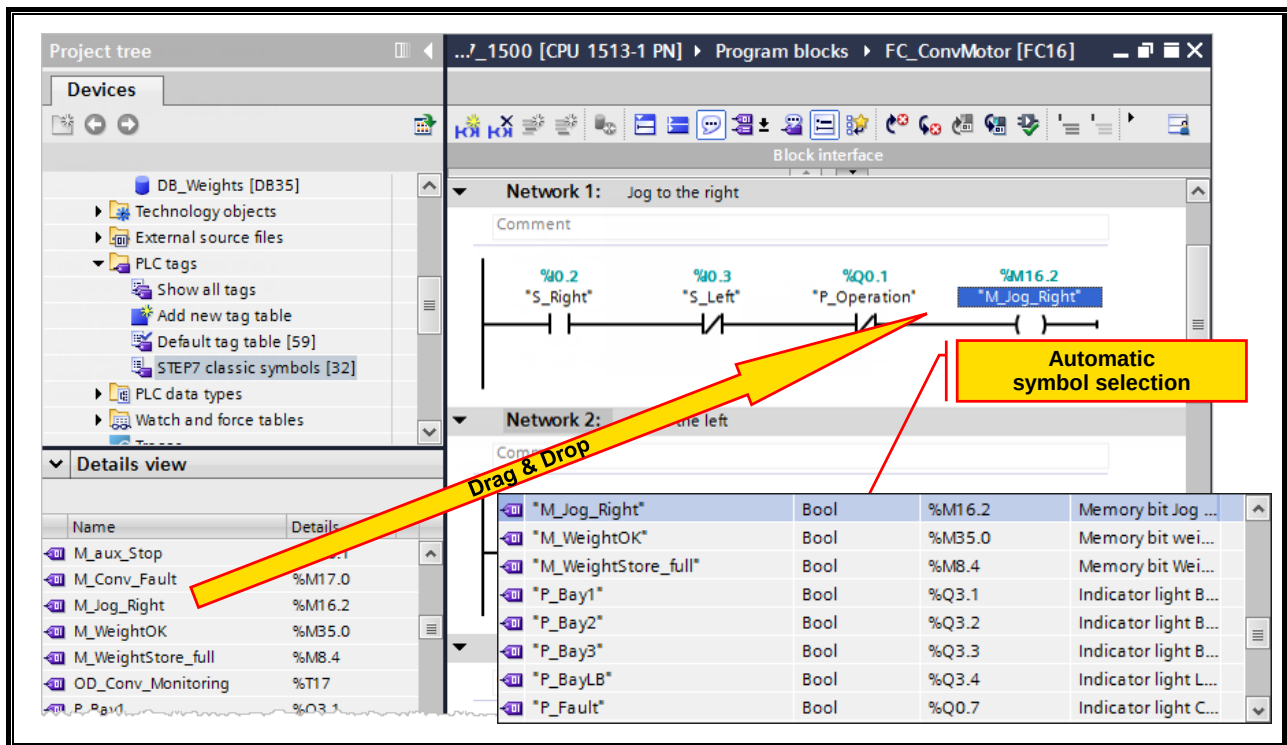
### 5.3.5. Copy & Paste PC Tags to Excel



#### Copy & Paste from and to Excel

The Windows Copy & Paste function can be used to easily copy individual or several tags from a PLC tag table or a data block to Excel to further process it/them there and then to copy it/them back from Excel to the PLC tag table or the data block.

## 5.4. Using a PLC Tag as an Operand



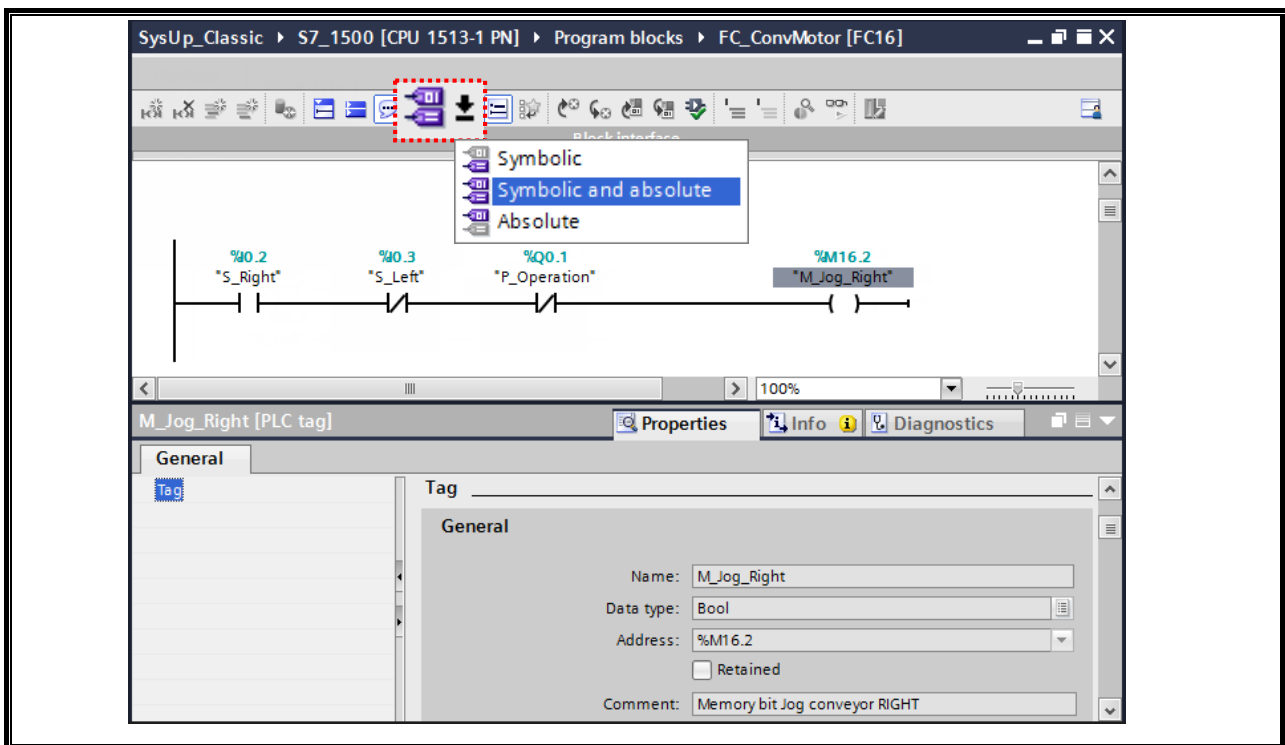
### Using a Tag as Operand

During programming, the name of the tag can be entered, or, it can be selected from the automatic symbol selection as well as the Details view.

### Symbol Selection

When operands are selected, after the first letter of an operand name has been entered, a selection of all the operands whose name start with the entered letter and are of the corresponding data type is displayed. All the operands that are valid for this block are displayed. These are all global tags (also those that are declared in data blocks), local tags (temporary and static), the parameters of the block as well as global and local constants.

### 5.4.1. Absolute and Symbolic Addressing



#### Absolute and Symbolic Addressing

All global tags (such as, inputs, outputs, bit memories) have both an absolute and a symbolic address. You can define which is to be displayed or with which is to be programmed (see picture).

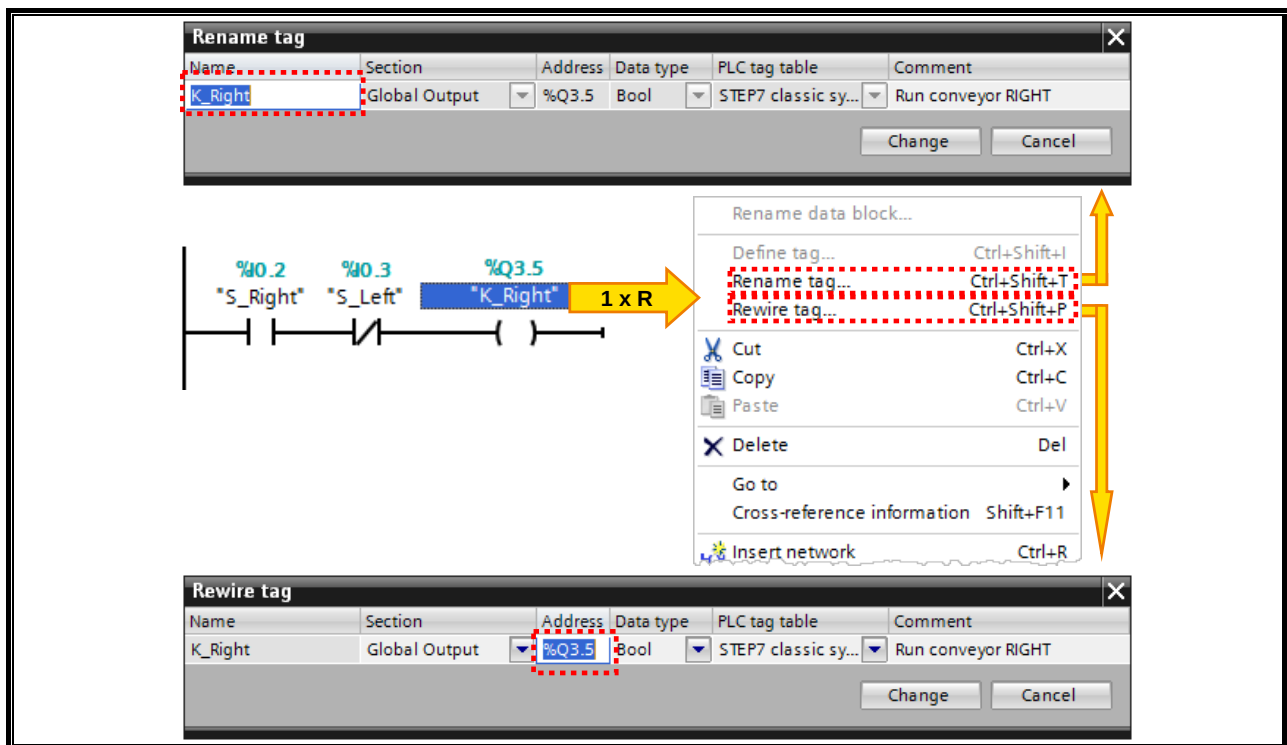
When you use a symbolic address (for example, "M\_Jog\_RIGHT") to which an absolute address has not yet been assigned, you can save the block but you cannot compile and download it into the controller.

When you use an absolute address (for example, M16.2), it is automatically assigned a symbolic default address (for example, "Tag\_1") which you can change.

#### Properties

If a block or the PLC tag table is open in the working area and a tag is selected (highlighted) there, then all details are displayed in the "Properties" tab in the Inspector window.

### 5.4.2. Renaming / Rewiring PLC Tags



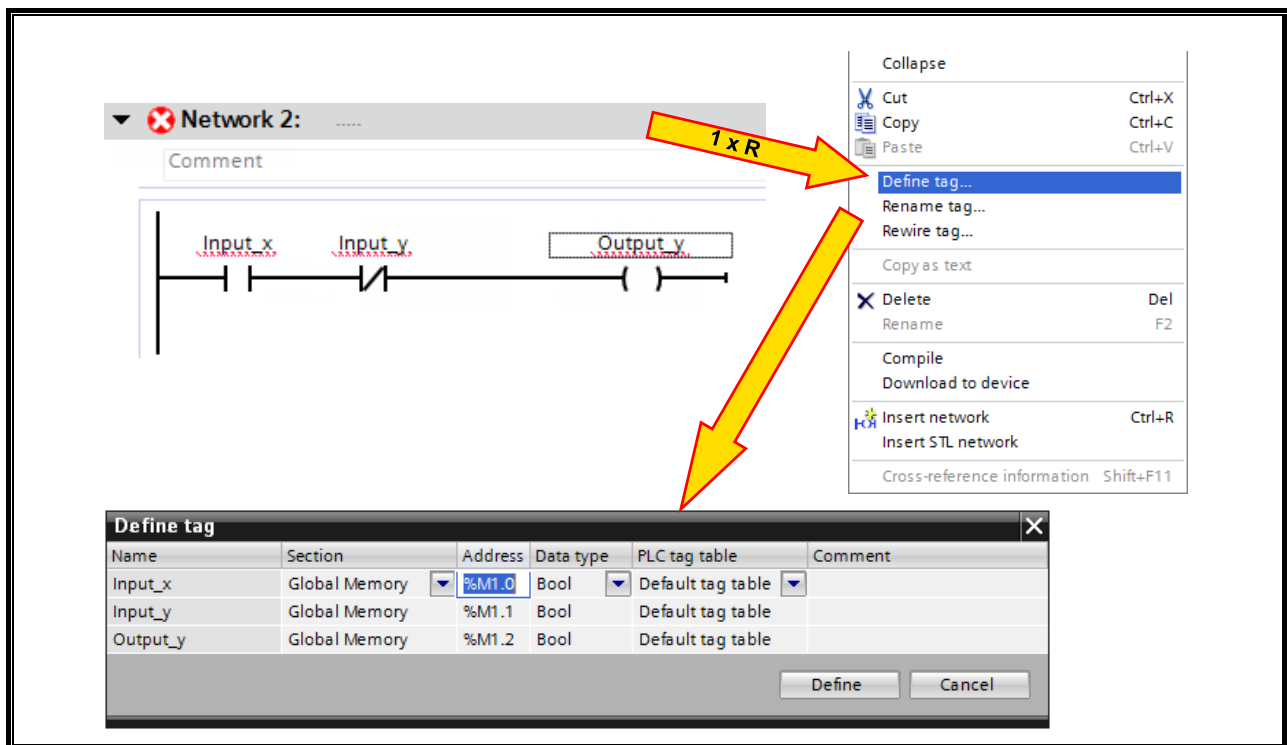
#### Renaming and Rewiring Tags

Tags can be renamed or rewired directly in the PLC tag table or as shown in the picture using the Blocks Editor. The changes are immediately adopted in the PLC tag table and affect the entire program.

- **Rename:**  
Change the tag name, while the absolute address remains unchanged.
- **Rewire:**  
Change the associated absolute address, while the name remains unchanged.



### 5.4.3. Defining (Declaring) Tags while Programming



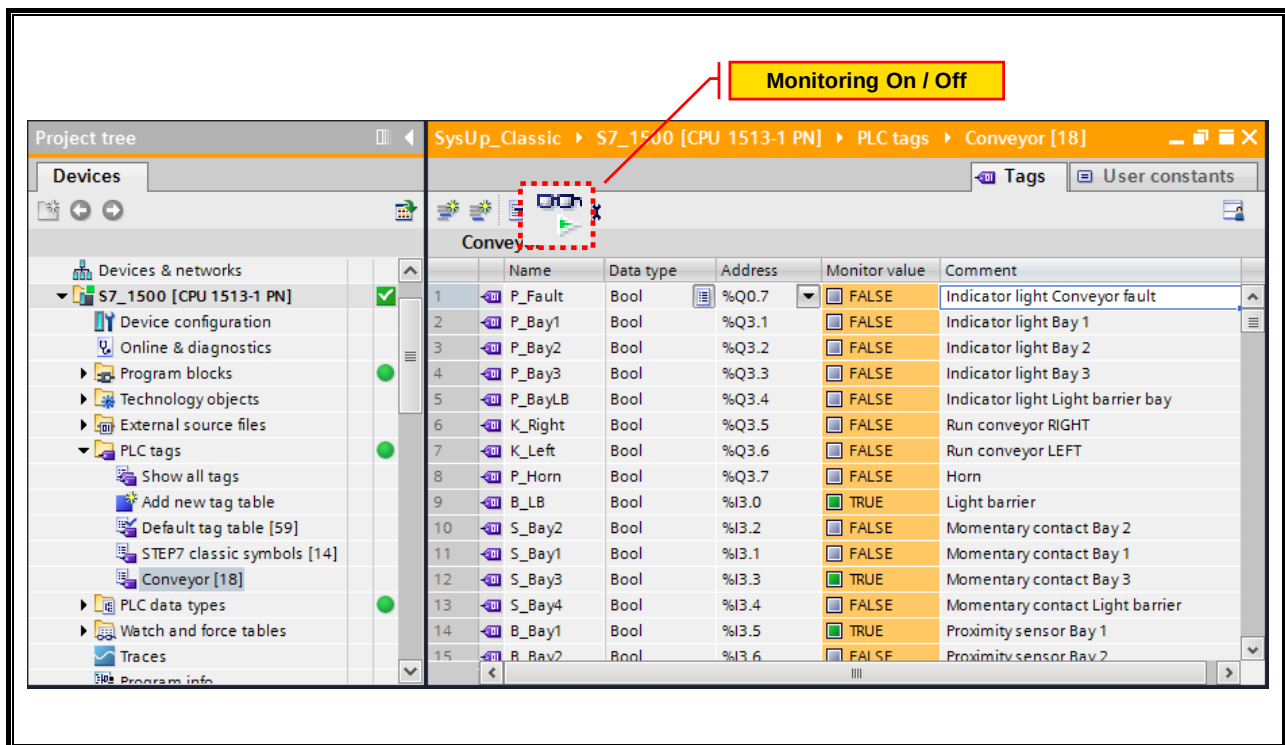
#### Defining (Declaring) Tags while Programming

If, during programming, unknown tags are used, they can be defined later-on network-by-network.

Advantage:

In the dialog that appears, only addresses are offered that are unused up to that point. In this way, for example, errors are avoided such as the using of bits that belong to an already used word (overlapping access).

#### 5.4.4. Monitoring PLC Tags



#### Monitoring PLC Tags

PLC tags can be monitored directly through the PLC tag table. In so doing, the "Monitor value" shows the current value of the tags in the CPU.

### 5.4.5. Retentiveness of PLC Tags

**Default tag table**

| Name          | Data type | Address | Retain                              | Visible in HMI                      | Access...                           | Comment          |
|---------------|-----------|---------|-------------------------------------|-------------------------------------|-------------------------------------|------------------|
| K_Left        | Bool      | %Q3.6   | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | Run conveyor LE  |
| P_Horn        | Bool      | %Q3.7   | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | Horn             |
| System_Byte   | Byte      | %MB0    | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |                  |
| FirstScan     | Bool      | %M0.0   | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |                  |
| Clock_1.25Hz  | Bool      | %M10.4  | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |                  |
| M_1Hz         | Bool      | %M10.5  | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |                  |
| Clock_0.625Hz | Bool      | %M10.6  | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |                  |
| Clock_0.5Hz   | Bool      | %M10.7  | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |                  |
| M_aux_On      | Bool      | %M15.1  | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | edge auxiliary m |
| M_aux_Stop    |           |         |                                     |                                     |                                     |                  |
| M Jog_Right   |           |         |                                     |                                     |                                     |                  |
| M_Auto_Right  |           |         |                                     |                                     |                                     |                  |
| M_Conv_Fault  |           |         |                                     |                                     |                                     |                  |
| <Add new>     |           |         |                                     |                                     |                                     |                  |

**Retain memory**

Number of memory bytes starting at MB0: 12

Number of SIMATIC timers starting at T0: 0

Number of SIMATIC counters starting at C0: 0

Currently available retain memory (bytes): 90700

**Example:**  
MB0 ... MB11 are retentive

**S7-1500:**

- Retentiveness (retain) can be set for memory bytes, timers and counters

**S7-1200:**

- Retentiveness (retain) can only be set for memory bytes

#### Retentive (Retain) Memory

The S7-1500 CPUs have a retentive memory for storing retentive data when the power is switched OFF. The size of the retentive memory is documented in the technical data of the CPU.

The utilization of the retentive memory of the configured CPU is shown offline in the Project tree under "Program info > Resources of..." or online in the Project tree under "Online & diagnostics > Diagnostics > Memory".

When you define data as retentive, their contents are retained after a power failure, during CPU start-up and during loading of a modified program.

You can define the following data or objects as retentive:

Memory bytes, timers, counters

Tags of global data blocks

Tags of instance data blocks of a function block

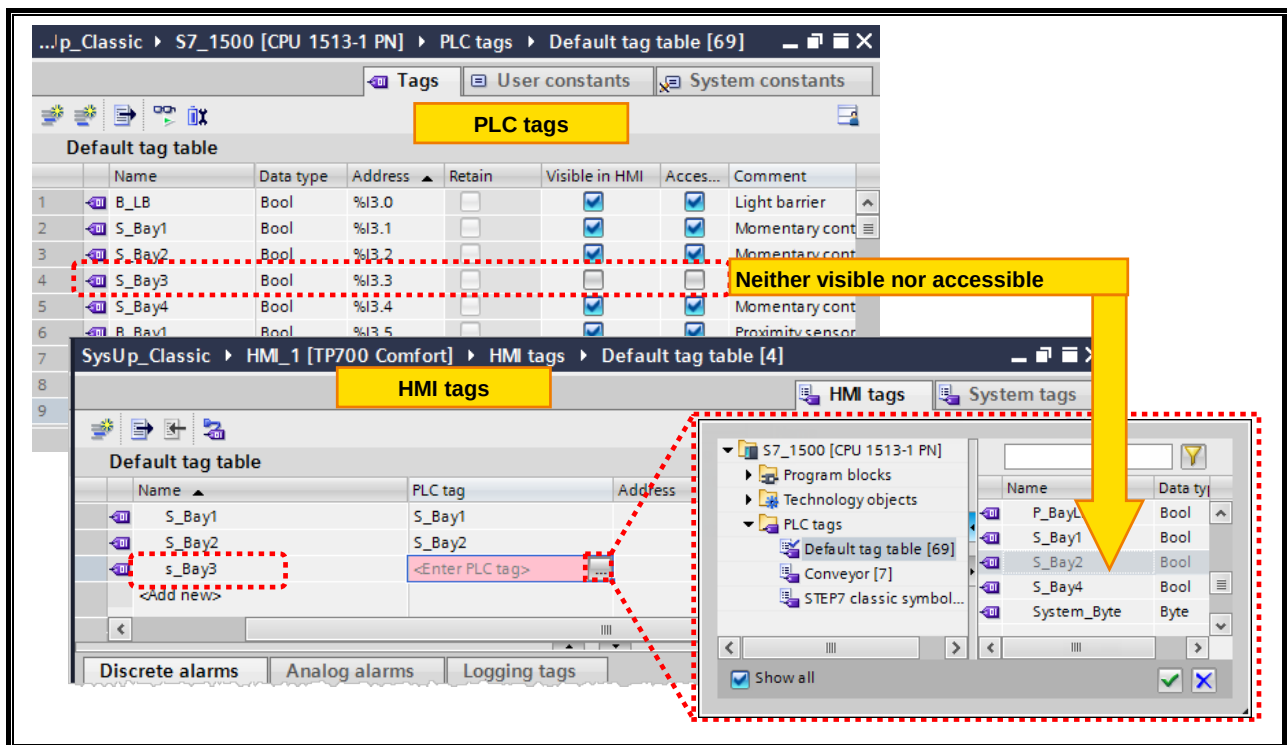
Certain tags of technology objects are always retentive, for example, adjustment values of absolute value encoders.

#### Memory Bytes, Timers, Counters

In the S7-1500, the number of retentive memory bytes, timers and counters can be defined in the PLC tag table through the "Retain" button.

In the S7-1200, only the number of retentive memory bytes can be defined in the PLC tag table through the "Retain" button.

### 5.4.6. HMI Access to PLC Tags

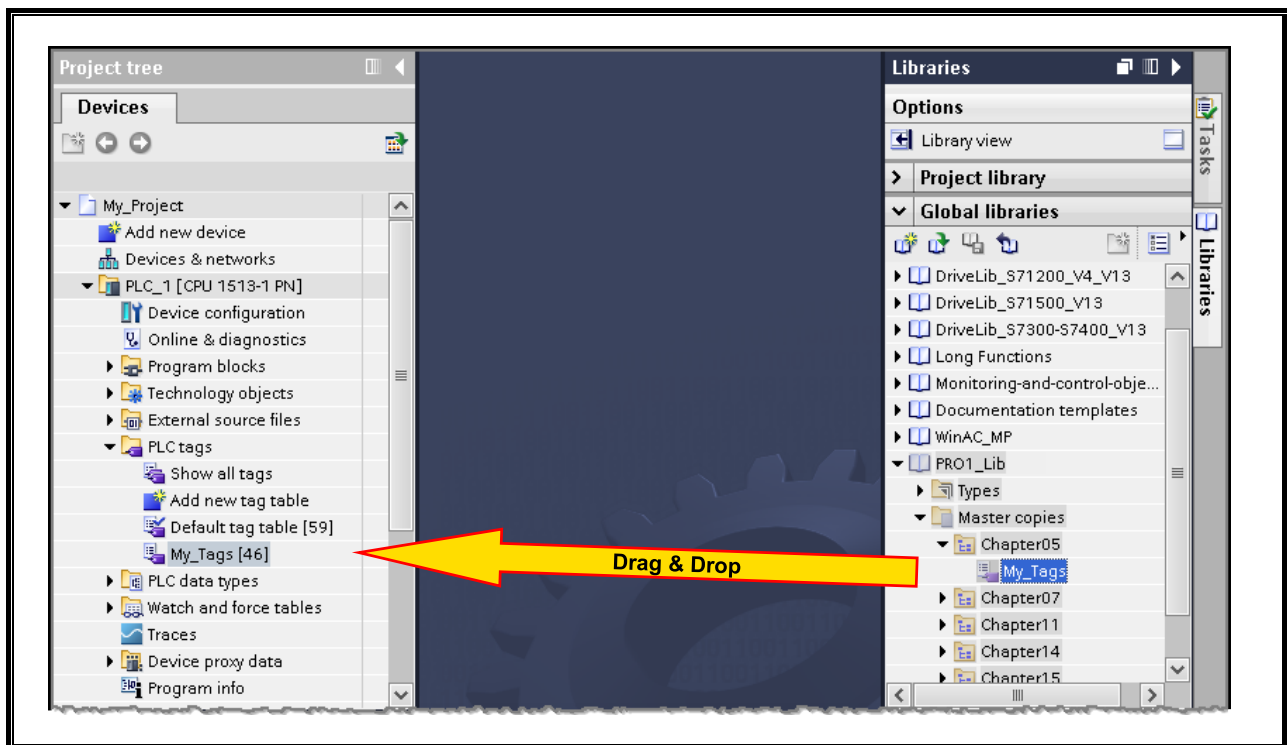


#### HMI Tag Access

Here, protective mechanisms can be declared with whose help unwanted accesses to PLC tags from HMI devices can be prevented:

- "Visible in HMI":**  
 During HMI configuration, only PLC tags with the attribute "Visible in HMI" can be selected. This filter function can, however, be disabled in the selection dialog shown by activating "Show all".
- "Accessible from HMI" (only S7-1200 and S7-1500):**  
 The HMI device can only access the PLC tags which have the attribute "Accessible from HMI" online. This protective function integrated in the S7-1200/1500 operating system ensures that the HMI device does not overwrite certain tags. Tags which are not "Accessible from HMI" are accordingly also not "Visible in HMI".

### 5.4.7. Exercise 1: Copying the PLC Tag Table from the Library



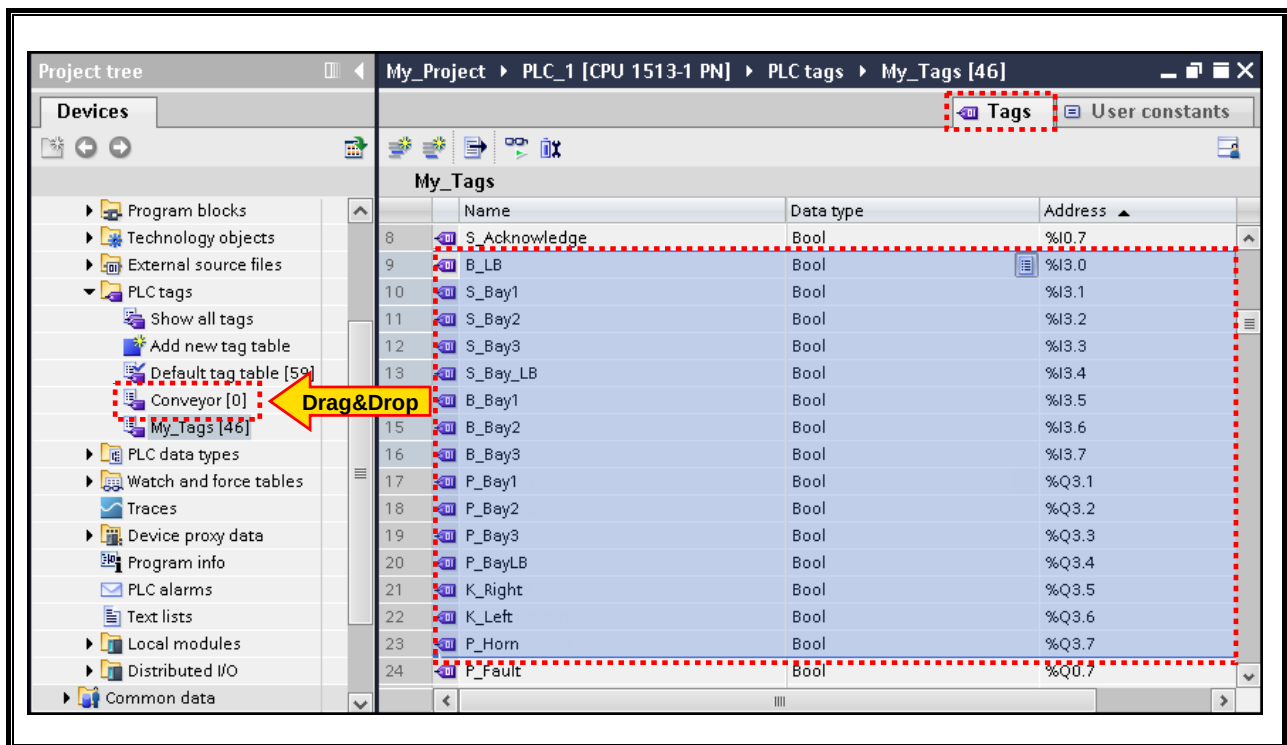
#### Task

You are to copy the prepared PLC tag table "My\_Tags" from the "PRO1\_Lib" library into your own project.

#### What to Do

1. In "Global libraries", open the library C:\Archives\TIA\_Portal\PRO1\_Lib"
2. Use drag and drop to copy the PLC tag table "My\_Tags" from the library into your project.
3. Save your project.

### 5.4.8. Exerciser 2: Creating a Conveyor Model Tag Table



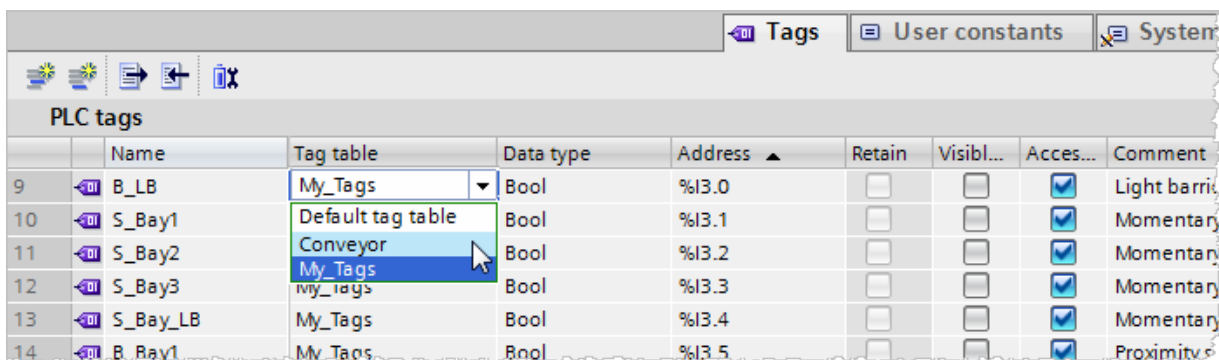
#### Task

A separate PLC tag table "Conveyor" is to be created for the inputs and outputs to which the sensors and actuators of the conveyor model are connected.

#### What to Do

1. Using "Add new tag table", create the new PLC tag table "Conveyor".
2. Open the PLC tag table "My\_Tags" and move the inputs and outputs into the newly created PLC tag table "Conveyor" using drag & drop (mouse pointer on tag icons, see picture).
3. Save your project.

**Note:** As an alternative, for every tag you can define in which tag table it is to be saved. You do this in the tag table "Show all tags" via the column "Tag table".



### 5.4.9. Exercise 3: Monitoring the PLC Tag Table "Conveyor"

**Conveyor**

| Name       | Address | Monitor value | Comment                 |
|------------|---------|---------------|-------------------------|
| 1 B_LB     | %I3.0   | TRUE          | Light barrier           |
| 2 S_Bay1   | %I3.1   | FALSE         | Momentary contact Bay 1 |
| 3 S_Bay2   | %I3.2   | FALSE         | Momentary contact Bay 2 |
| 4 S_Bay3   |         |               |                         |
| 5 S_BayLB  |         |               |                         |
| 6 B_Bay1   |         |               |                         |
| 7 B_Bay2   |         |               |                         |
| 8 B_Bay3   |         |               |                         |
| 9 P_Bay1   |         |               |                         |
| 10 P_Bay2  |         |               |                         |
| 11 P_Bay3  |         |               |                         |
| 12 P_BayLB |         |               |                         |
| 13 K_Right |         |               |                         |
| 14 K_Left  |         |               |                         |
| 15 P_Horn  |         |               |                         |

Labels on PLC rack:

"K\_Right" (Q 3.5) "B\_Bay1" (I 3.5) "B\_Bay2" (I 3.6) "B\_Bay3" (I 3.7) "B\_LB" (I 3.0)

"S\_Bay1" (I 3.1) "S\_Bay2" (I 3.2) "S\_Bay3" (I 3.3) "S\_BayLB" (I 3.4)

"P\_Bay1" (Q 3.1) "P\_Bay2" (Q 3.2) "P\_Bay3" (Q 3.3) "P\_BayLB" (Q 3.4)

#### Task and What to Do

1. Open the newly created PLC tag table "Conveyor" and activate the function "Monitoring"
2. On the conveyor model, press the Bay pushbuttons and check whether Status '1' or "TRUE" is displayed at the corresponding inputs.

## 6

## Contents

|           |                                                                                    |            |
|-----------|------------------------------------------------------------------------------------|------------|
| <b>6.</b> | <b>Program Blocks and Program Editor .....</b>                                     | <b>6-2</b> |
| 6.1.      | Plant Description: The Conveyor Model as a Distribution Conveyor .....             | 6-3        |
| 6.2.      | Types of Program Blocks .....                                                      | 6-4        |
| 6.3.      | Structured Programming .....                                                       | 6-5        |
| 6.4.      | Process Images .....                                                               | 6-6        |
| 6.5.      | Cyclic Program Execution .....                                                     | 6-7        |
| 6.6.      | Adding a New Block .....                                                           | 6-8        |
| 6.6.1.    | Block Properties: Programming Language, Time Stamps .....                          | 6-9        |
| 6.6.2.    | Block Properties: IEC Check .....                                                  | 6-10       |
| 6.6.3.    | Block Properties: Know-how Protection .....                                        | 6-11       |
| 6.6.4.    | Other Block Attributes .....                                                       | 6-12       |
| 6.6.5.    | Block Editor Settings .....                                                        | 6-13       |
| 6.6.6.    | Block Networks .....                                                               | 6-14       |
| 6.6.7.    | Block Programming .....                                                            | 6-15       |
| 6.6.8.    | Programming an Instruction using "Empty Box" .....                                 | 6-16       |
| 6.6.9.    | Closing / Saving / Rejecting a Block .....                                         | 6-17       |
| 6.6.10.   | Block Groups .....                                                                 | 6-18       |
| 6.6.11.   | Block Calls .....                                                                  | 6-19       |
| 6.6.12.   | Compiling a Block .....                                                            | 6-20       |
| 6.6.13.   | Downloading Blocks into the CPU .....                                              | 6-21       |
| 6.6.14.   | Downloading Blocks into the CPU: S7-1500 - Memory Concept .....                    | 6-22       |
| 6.6.15.   | Monitoring a Block .....                                                           | 6-23       |
| 6.6.16.   | Deleting Blocks .....                                                              | 6-24       |
| 6.6.17.   | "Upload" Blocks "from Device" (Upload into Project) .....                          | 6-25       |
| 6.6.18.   | Comparing Blocks .....                                                             | 6-26       |
| 6.7.      | Task Description: Jogging the Conveyor Motor .....                                 | 6-27       |
| 6.7.1.    | Exercise 1: Adding the "FC_ConvMotor" (FC16) Block .....                           | 6-28       |
| 6.7.2.    | Exercise 2: Programming the "FC_ConvMotor" (FC16) Block .....                      | 6-29       |
| 6.7.3.    | Exercise 3: Adapting the OB1 Properties .....                                      | 6-30       |
| 6.7.4.    | Exercise 4: Calling "FC_ConvMotor" (FC16) in OB1 .....                             | 6-31       |
| 6.7.5.    | Exercise 5: Compiling the Program, Downloading it into the CPU and Saving it ..... | 6-32       |
| 6.7.6.    | Exercise 6: Monitoring "FC_ConvMotor" (FC16) .....                                 | 6-33       |



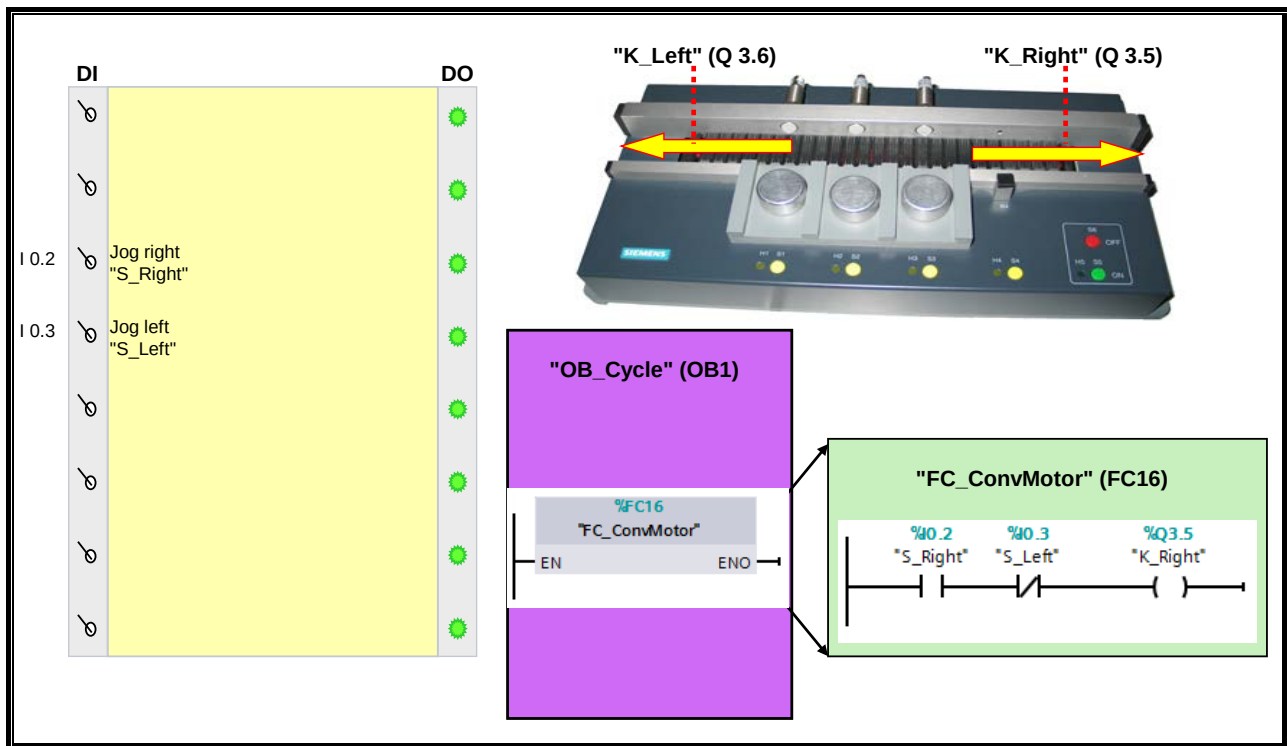
## 6. Program Blocks and Program Editor

**At the end of the chapter the participant will ...**

- ... be familiar with the different S7 block types
- ... be familiar with the principle of "structured programming"
- ... be familiar with the meaning of process images (PII, PIQ)
- ... be able to explain the principle of cyclic program execution
- ... be familiar with the LAD and FBD presentation formats and be able to select them
- ... be able to edit, save and load a block using the editor
- ... be able to carry out a simple program test with "Monitor block"



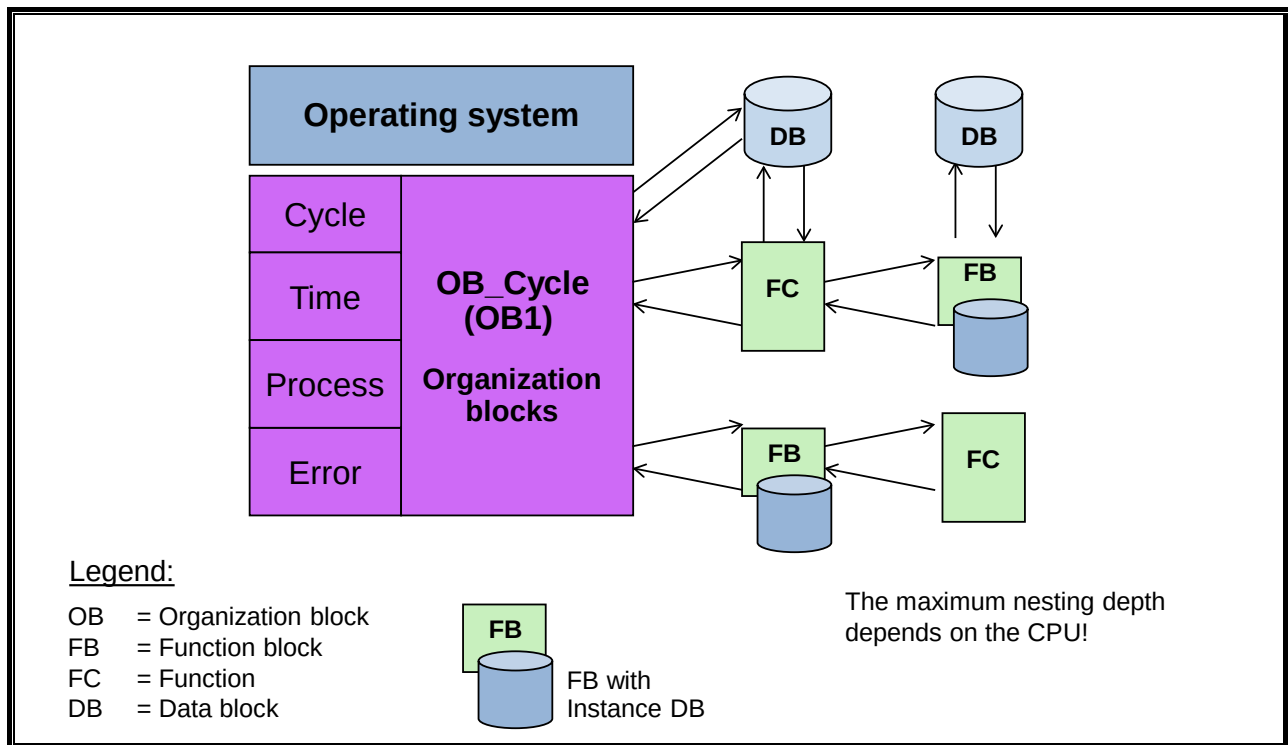
## 6.1. Plant Description: The Conveyor Model as a Distribution Conveyor



### Conveyor Model as Distribution Conveyor

The conveyor motor can be jogged to the right and left using the pushbuttons "S\_Right" (I 0.2) and "S\_Left" (I 0.3).

## 6.2. Types of Program Blocks



### Blocks

The automation system provides various types of blocks in which the user program and the related data can be stored. Depending on the requirements of the process, the program can be structured in different blocks. You can use the entire operation set in all blocks (FB, FC and OB).

### Organization Blocks (OBs)

Organization blocks (OBs) form the interface between the operating system and the user program. The entire program can be stored in OB1 that is cyclically called by the operating system (linear program) or the program can be divided and stored in several blocks (structured program).

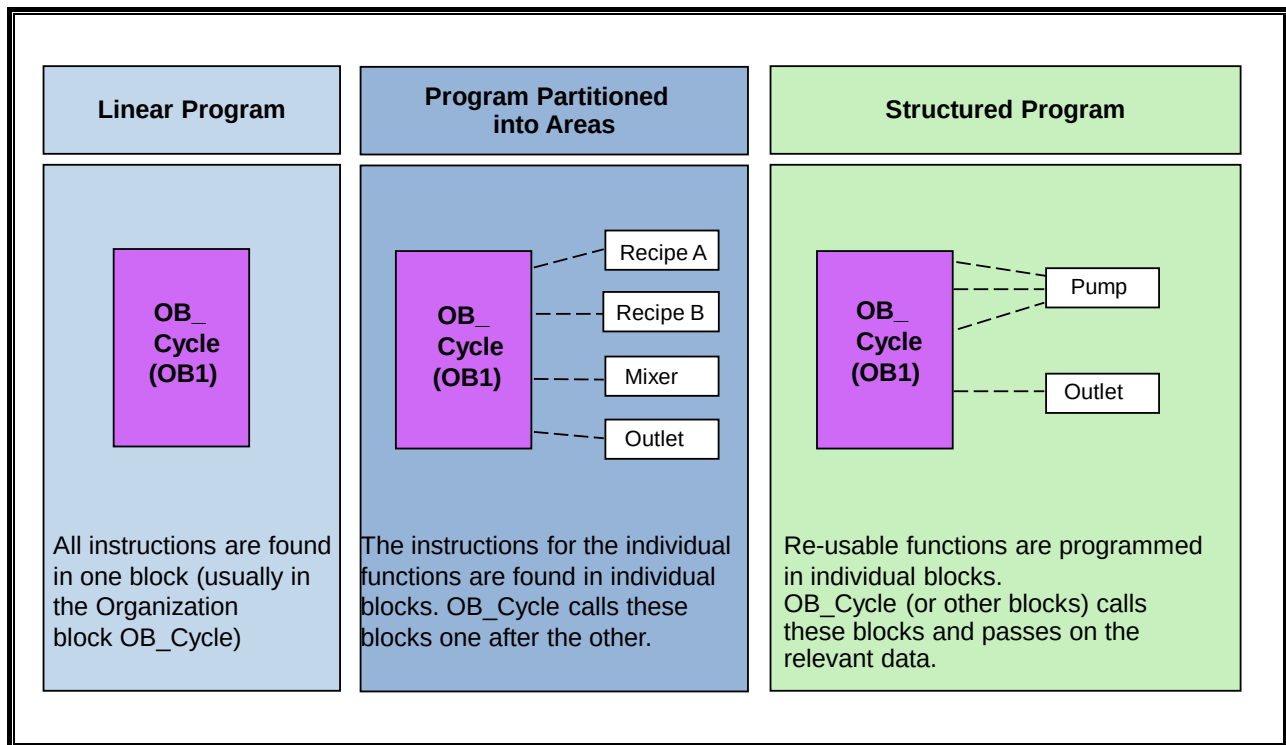
### Functions (FCs)

A function (FC) contains a partial functionality of the program. It is possible to program functions as parameter-assignable so that when the function is called it can be assigned parameters. As a result, functions are also suited for programming frequently recurring, complex partial functionalities such as calculations.

### Function Blocks (FBs)

Basically, function blocks offer the same possibilities as functions. In addition, function blocks have their own memory area in the form of instance data blocks. As a result, function blocks are suited for programming frequently recurring, complex functionalities such as closed-loop control tasks.

## 6.3. Structured Programming



### Linear Program

The entire program is found in one continuous program block. This model resembles a hard-wired relay control that was replaced by an automation system (programmable logic controller). The CPU processes the individual instructions one after the other.

### Partitioned Program

The program is divided into blocks, whereby every block only contains the program for solving a partial task. Further partitioning through networks is possible within a block. You can generate network templates for networks of the same type. Normally, the OB 1 organization block contains instructions that call the other blocks in a defined sequence.

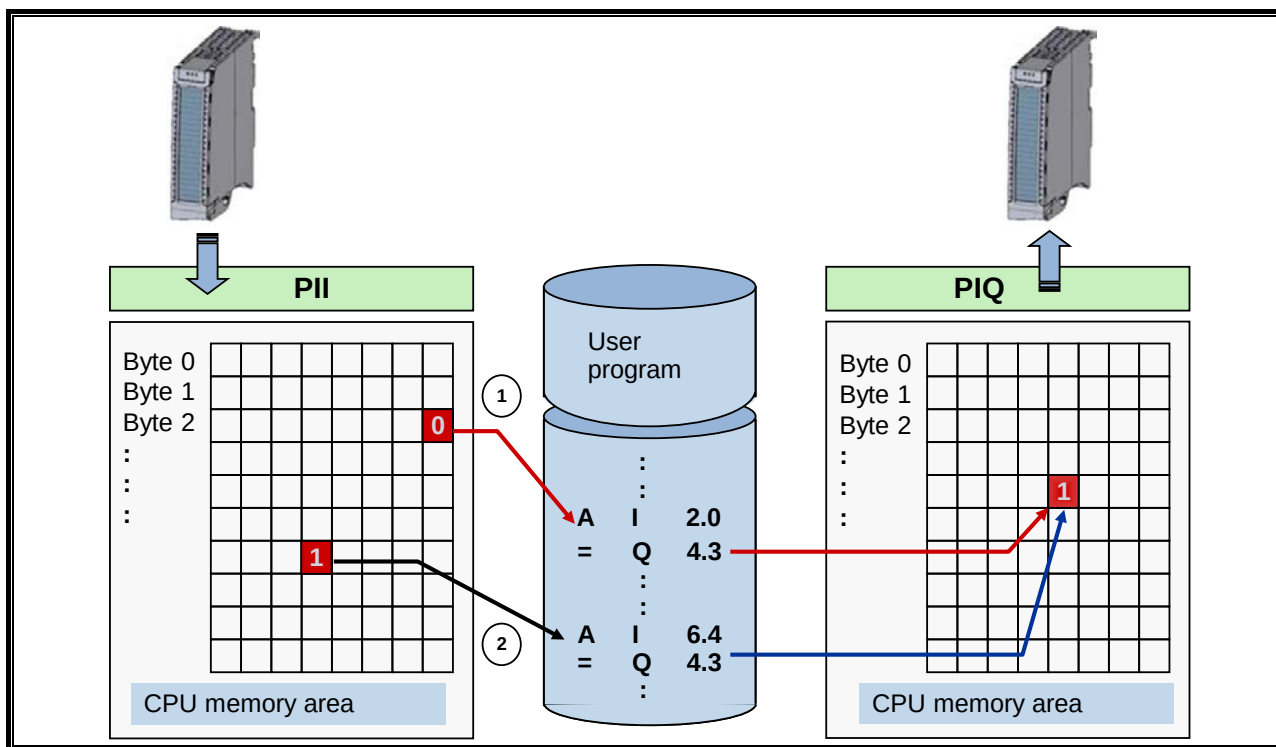
### Structured Program

A structured program contains parameter-assignable blocks that are set up in such a way that they can be used universally. When a parameter-assignable block is called, it is passed current parameters (for example, the actual addresses of inputs and outputs as well as parameter values).

Example:

- A "pump block" contains instructions for the control of a pump.
- The program blocks, which are responsible for the control of special pumps, call the "pump block" and provide it with information about which pump is to be controlled with which parameters.

## 6.4. Process Images



### Process Images

For the storage of **all** digital input and output states, the CPU has reserved memory areas: the process-image input table (PII) and the process-image output table (PIQ). During program execution, the CPU accesses these memory areas exclusively. It does not access the digital input and output modules directly.

#### PII

The Process-Image Input table (PII) is the memory area in which the states of all digital inputs are stored. At the beginning of the cycle, the digital input modules read-in to the PII. When an input is linked, the state of this input stored in the PII is linked. This state cannot change within a cycle since the PII is only updated or read-in at the beginning of a cycle. This guarantees that when there are multiple queries of the input in one cycle, the same result is always supplied.

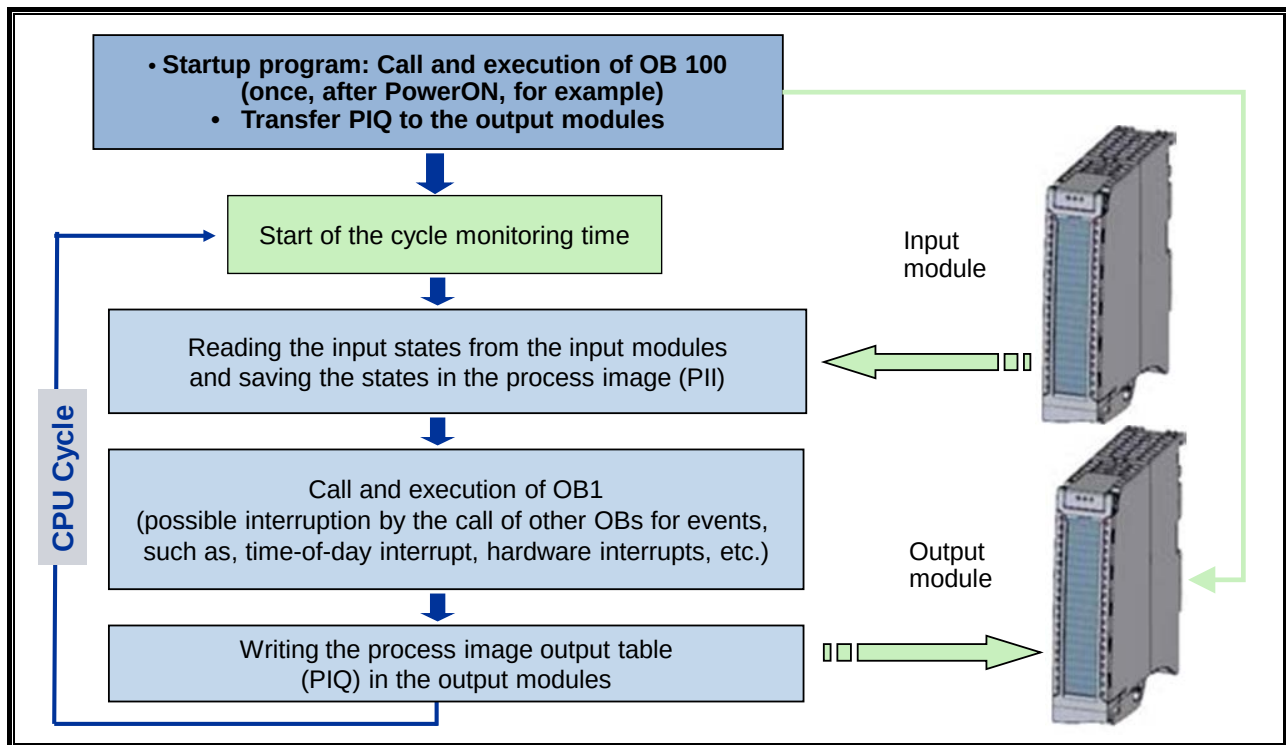
#### PIQ

The Process-Image Output table (PIQ) is the memory area in which the states of all digital outputs are stored. The PIQ is output to the digital output modules at the end of the cycle. Outputs can be assigned as well as queried in the program.

### "Classic Error": Double Assignment

If an output is assigned a state in several locations in the program, then only the state that was assigned last is transferred to the particular output module. As a rule, these types of double assignments are programming errors.

## 6.5. Cyclic Program Execution



### Restart

When you switch on or switch from STOP --> RUN, the CPU carries out a complete restart (with OB100). During restart, the operating system deletes all non-retentive bit memories and starts the cycle monitoring time.

### Cyclic Program Execution

Cyclic program execution occurs in an endless loop. After the execution of a program cycle is completed, the execution of the next cycle occurs automatically. In every program cycle, the CPU carries out the following steps.

- The CPU scans the states of the input signals and updates the process image inputs.
- The CPU sequentially processes the instructions of the user program and so works directly with the process images, not with the inputs and outputs of the input / output modules.
- The CPU transfers the output states from the process image outputs to the output modules.

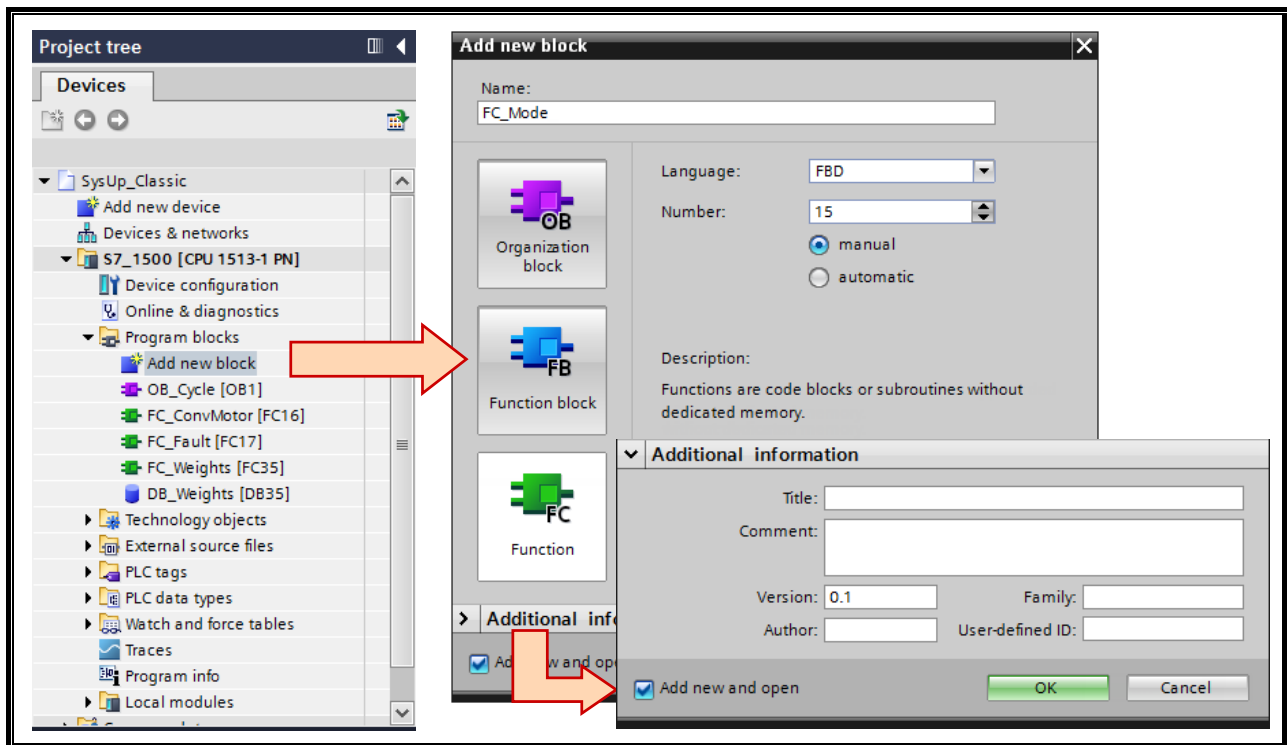
### Cycle and Cycle Monitoring Time

The time that the CPU requires for the execution of the complete program cycle, is the cycle time which is monitored for time by the CPU operating system. If the cycle time exceeds the cycle monitoring time defined in the CPU properties by more than double, the CPU goes into the STOP state.

### Using the Process Image or Deselecting it for Individual Modules

In the properties of the modules you can select whether the input values are to be automatically adopted in the PII or whether the output values are to be automatically written with values from the PIQ.

## 6.6. Adding a New Block

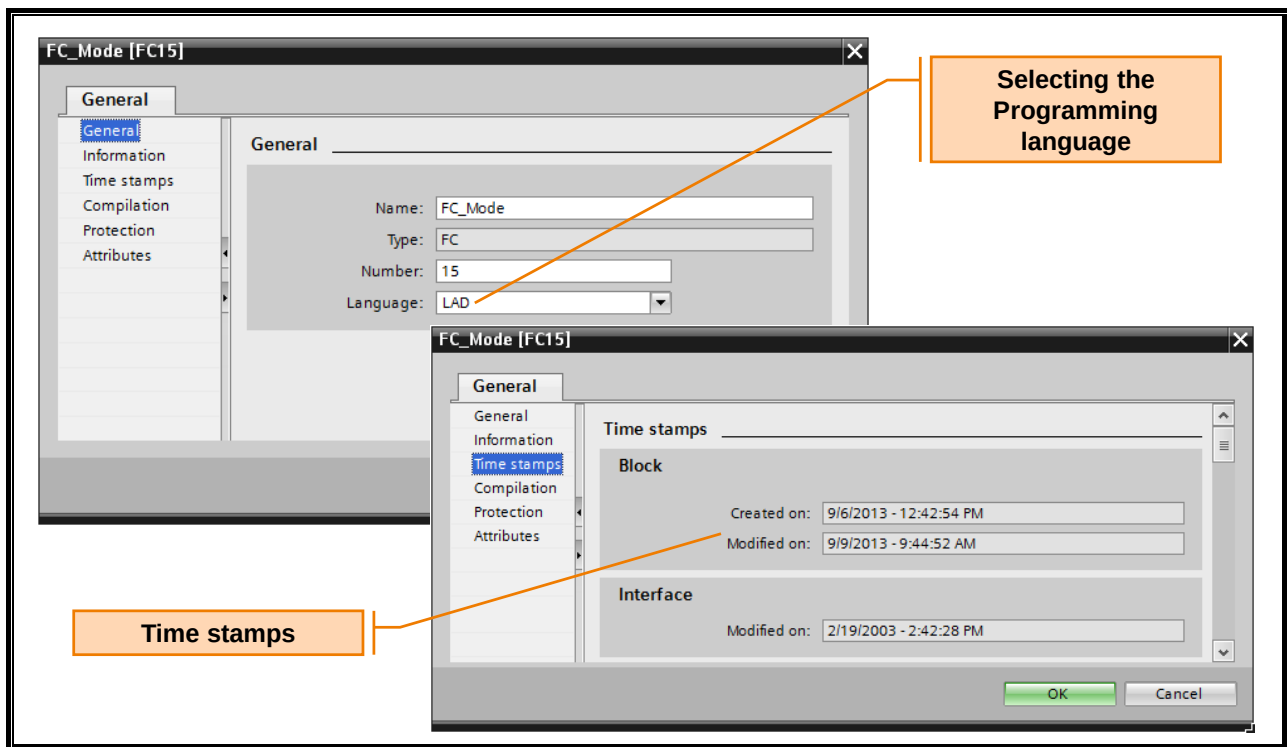


### Inserting a Block

A new block is created as shown in the picture. When you create a block, the type of block (OB, FB, FC or DB), the programming language, the symbolic name and number, among other things, must be defined. The block numbers can also be assigned automatically or manually.

Under "Additional information", the block can be documented in more detail, among other things, with a Version number and an Author.

### 6.6.1. Block Properties: Programming Language, Time Stamps



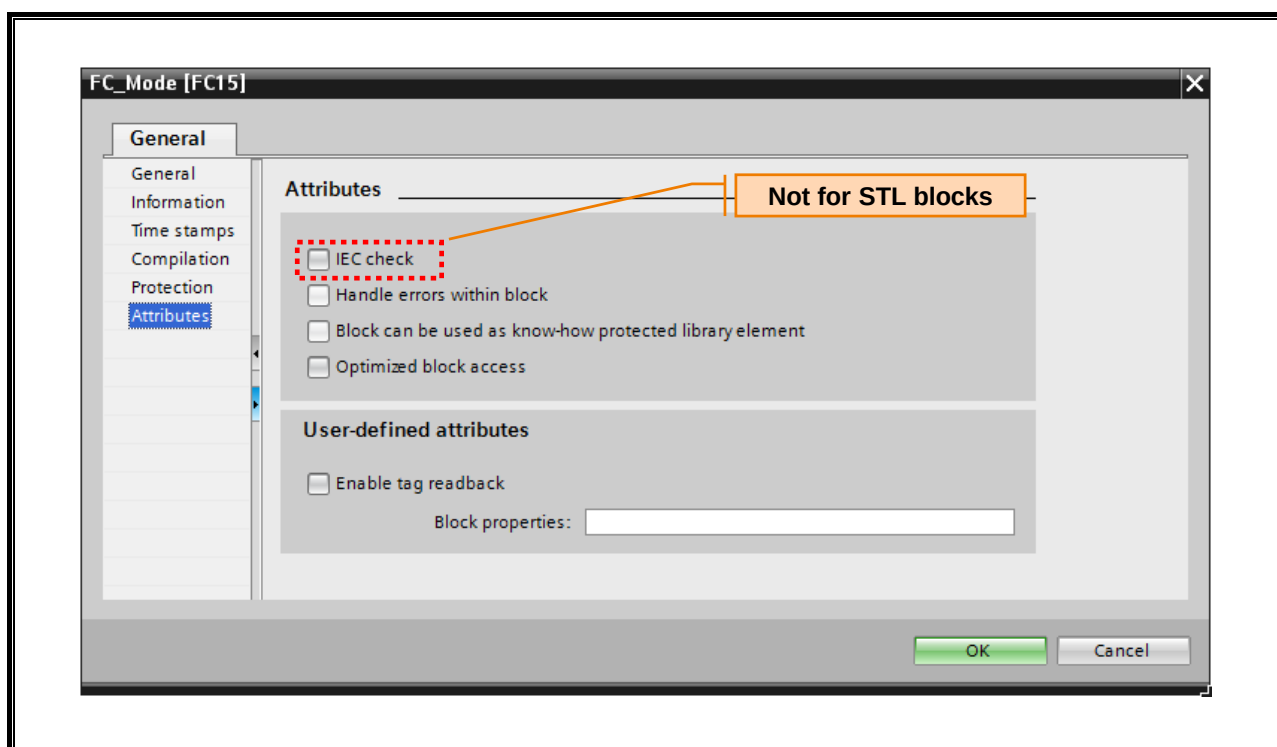
#### Properties

Each block has certain properties that you can display and edit. These properties are used to:

- Identify the block
- Display the memory requirements and the compilation status of the block
- Display the time stamp
- Display the reference information
- Specify the access protection
- Display and change the programming language



## 6.6.2. Block Properties: IEC Check



### With IEC Check (Default)

When IEC check is set, the following rules apply:

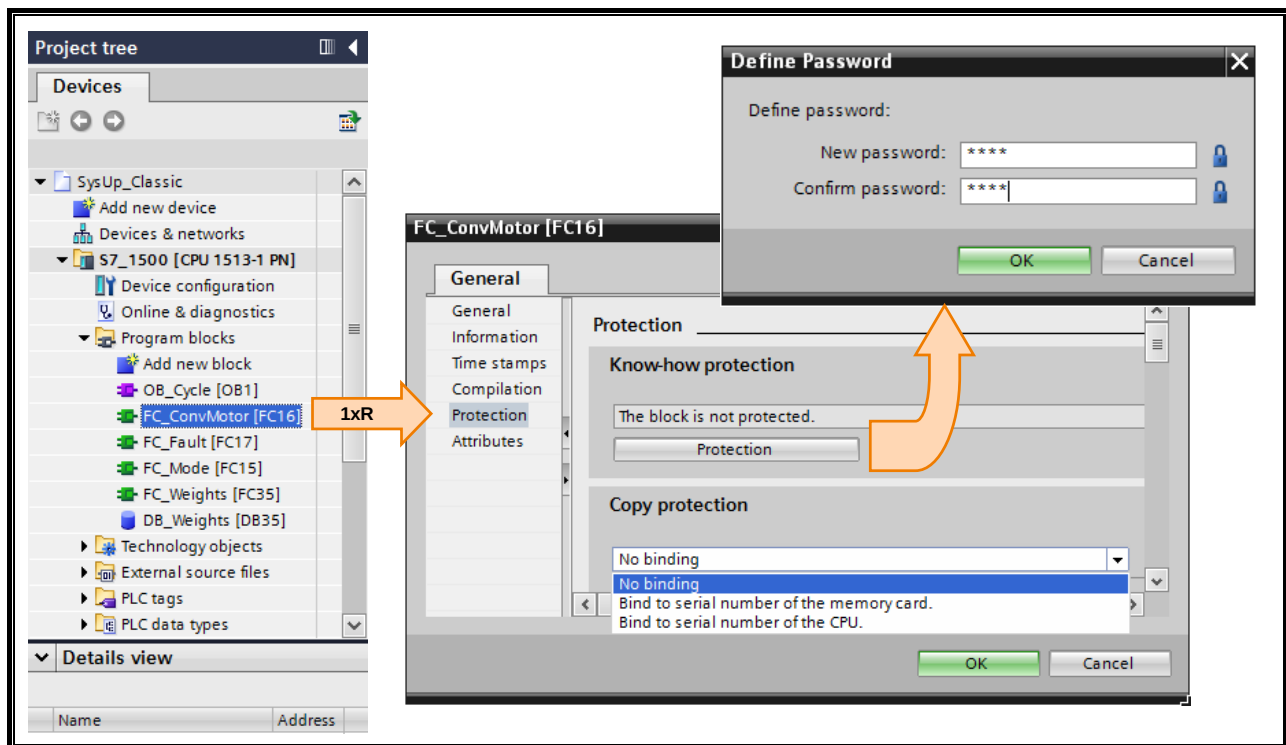
- The implicit conversion (see chapter "Digital Operations") from BOOL into other data types is not possible.
- Only the data types REAL, BYTE, WORD, DINT, INT, SINT, UDINT, UINT, USINT, TIME, DT, STRING and CHAR can be implicitly converted.
- The bit length of the source data type must not exceed the bit length of the target data type. An operand of the data type WORD cannot, for example, be specified at a parameter when the parameter expects the data type BYTE.

### Without IEC Check

When IEC check is not set, the following rules apply:

- The implicit conversion from BOOL into other data types is not possible.
- Only the data types REAL, LREAL, BYTE, WORD, DWORD, SINT, INT, DINT, USINT, UINT, UDINT, TIME, DTL, TOD, DATE, STRING and CHAR can be implicitly converted.
- The bit length of the source data type must not exceed the bit length of the target data type. An operand of the data type DWORD cannot, for example, be specified at a parameter when the parameter expects the data type WORD.
- At InOut parameters, the bit length of a specified operand must match the programmed bit length of the respective parameter.
- Note: Implicit Conversion without IEC Check
- The programming editor identifies operands that are implicitly converted with a gray rectangle. A dark gray rectangle signals that an implicit conversion without the loss of accuracy is possible, for example, when you convert the data type SINT to INT. A light gray rectangle signals that an implicit conversion is possible but, during runtime, errors could occur. If, for example, you convert the data type DINT to INT and an overflow results, the enable output ENO is set to "0".

### 6.6.3. Block Properties: Know-how Protection



#### Know-how Protection

Blocks can be protected against unauthorized access with a password. With a know-how protected block, only the following data can be read:

- Transfer parameters Input, Output, InOut, Return
- Block title
- Block comment
- Block properties
- Cross references with the information which global tags are used

Just like unprotected blocks, know-how protected blocks can also be copied, called, downloaded into the CPU and deleted. The code of the block, however, is protected against unauthorized reading and changing.

#### Copy Protection

In addition to the know-how protection you can also generate a copy protection in which you define to which memory card or CPU (each identified through a serial number) the block can be transferred.

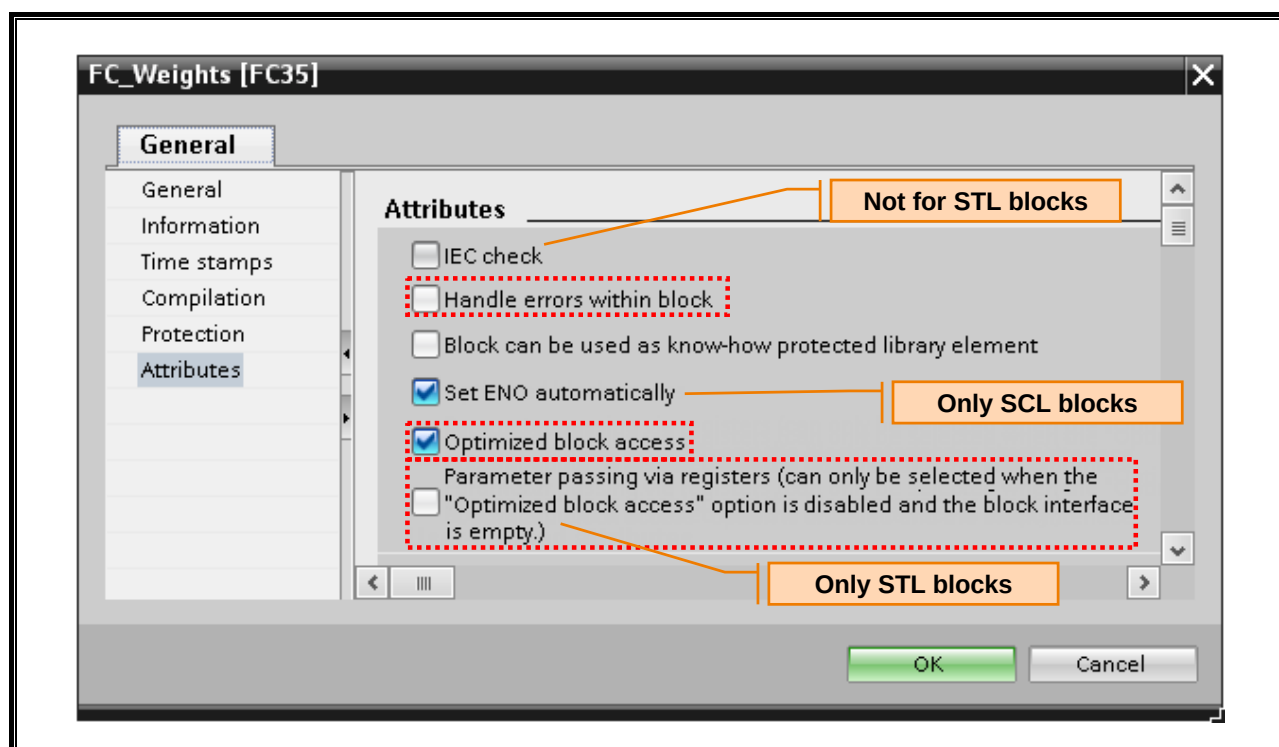
#### Note:

When downloading a know-how protected block into the CPU, the know-how protection is also loaded in the CPU, but not the retrieval information. This results in the following consequence: if a know-how protected block is uploaded from the CPU into an offline project, this block can no longer be opened offline even when the correct password is given. A block comparison between the offline and the online version of the block is, however, still possible with the correct password.

#### Caution!

If you forget the password, it is no longer possible to access the block.

#### 6.6.4. Other Block Attributes



##### Handle Errors within Block

Similar to the mechanism of error masking in S7-300/400, S7-1200/1500 allows for local error handling by means of the instructions GET\_ERROR (query errors locally within the block) or GET\_ERROR\_ID (query error ID locally within the block). By calling one of these instructions in a block, the block is given the attribute "Handle errors within block". If a programming error (for example, access to a non-existing data word) occurs during the execution of the block, no entry is made in the diagnostics buffer and the programming error organization block OB121 is not called.

##### Attribute "Optimized Block Access"

The "Optimized block access" attribute can be assigned to all logic and data blocks. It has the following effects for the different blocks:

##### for Data Blocks (DB):

- The variables of the data block can only be accessed symbolically
- The retentive behavior of the variables can be adjusted variable-by-variable
- With S7-1200, these DBs are created memory-optimized
- With S7-1500, these DBs are created access-optimized

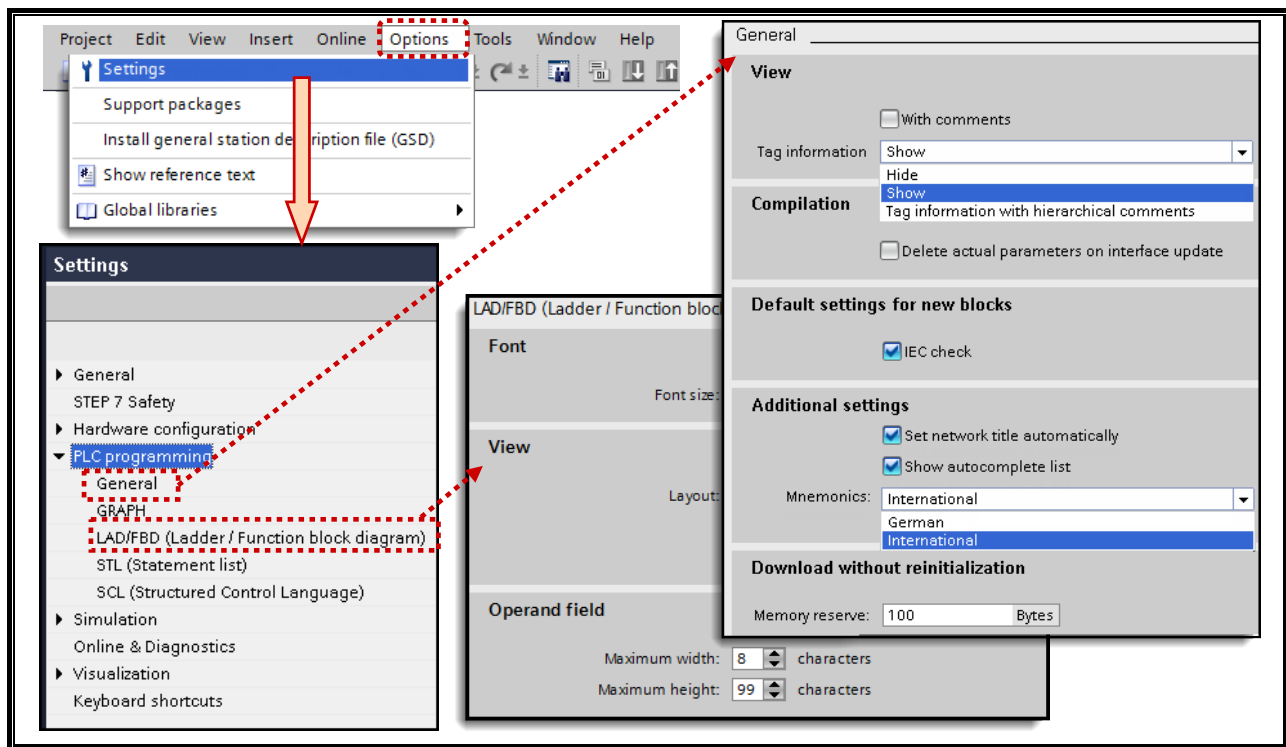
##### for Logic Blocks (OB, FC, FB):

- Indirect symbolic access to complex variables
- No access to CPU registers
- No indirect addressing via pointer with absolute addresses

##### Parameter Passing via Registers (Only for STL blocks of the S7-1500)

With S7-300/400, it is possible for block calls using the instructions UC and CC to use the registers (accumulators, address register and data block register) for passing values from block to block. With S7-1200/1500, these registers are non-existent. For STL instructions that explicitly access these registers (only S7-1500), these are emulated. If values are to be passed from block to block via these emulated registers, the attribute "Parameter passing via registers" must be activated (only STL blocks).

## 6.6.5. Block Editor Settings

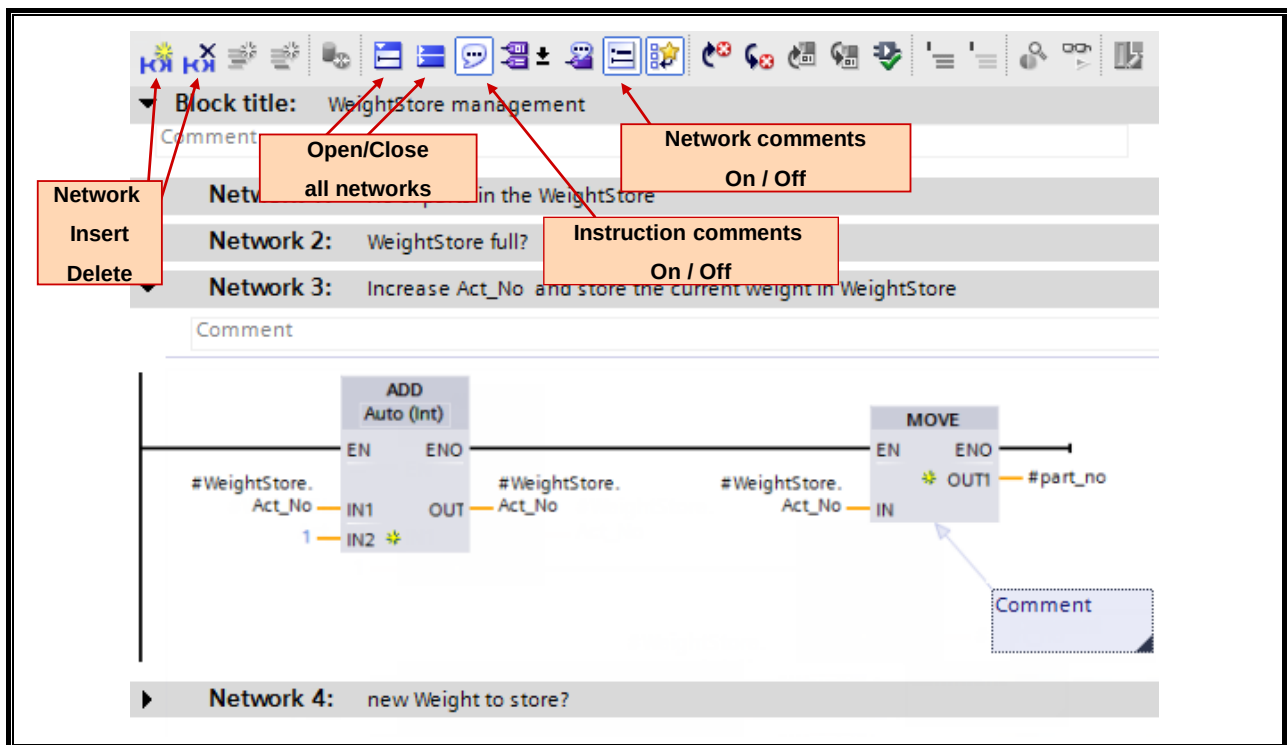


### Block Editor Settings

With the settings, you merely define how a block is to be presented when it is opened. In the editor, you can make changes to the presentation (such as showing and hiding comments) at any time.

- **Compilation**  
When "Delete actual parameters on interface update" is activated, the calls of parameterized blocks are automatically adjusted if, within the block, parameters are deleted after the fact.
- **IEC check**  
Only variables with the correct data type can be used. If an operation requires a variable of the data type INT, no variable of the data type WORD can be used even if the dimension (16 bits) is the same.
- **Download without reinitialization**  
Defines the size of the memory reserve for all newly created blocks in the project that support the function "Download without reinitialization"
- **Mnemonics**  
Setting the syntax for the programming language: German (e.g. E for Eingang (Input)) or International (e.g. I for Input).
- **Layout**  
When "With absolute information" is activated the absolute addresses of global operands are also displayed.
- **Operand field**  
Setting the maximum width and height of function block diagram and ladder diagram symbols.

### 6.6.6. Block Networks

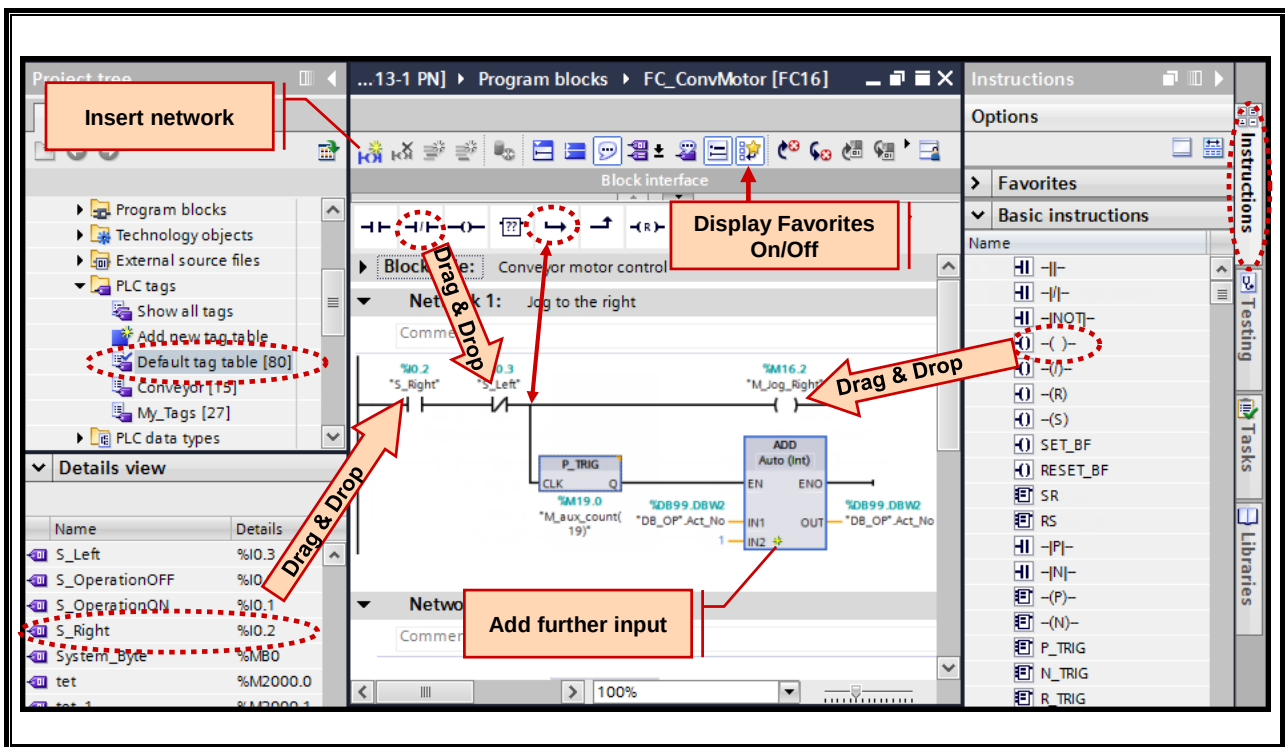


#### Networks

Just as the entire user program is subdivided into individual blocks, the individual blocks in turn are made up of individual networks. The division of a block into networks is individually designed by the user.

Each network can be given a network label and a comment. Within the networks, the individual instructions can be given instruction comments.

### 6.6.7. Block Programming



#### Block Programming

The instructions within a block can be programmed as follows:

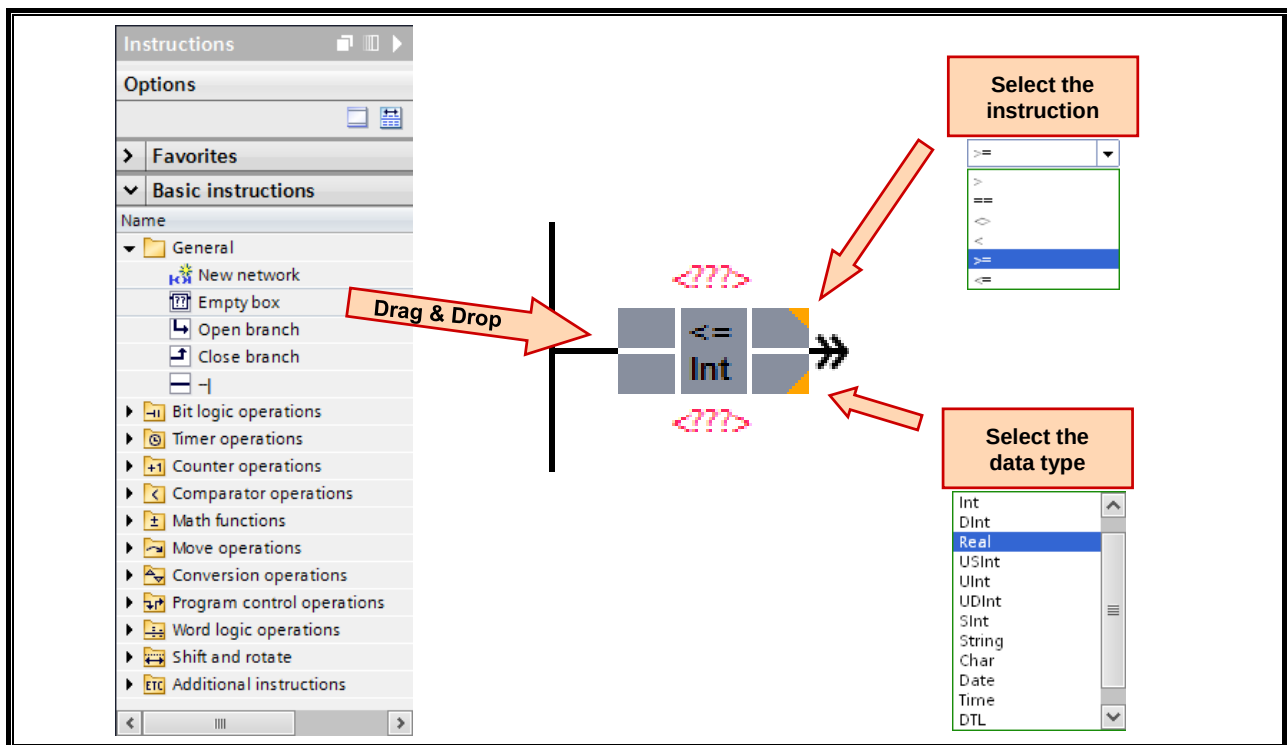
- using drag & drop from the Favorites or the Instructions catalog to anywhere in the program
- by first selecting the location in the program and then double-clicking on the desired instruction in the Favorites or the Instructions catalog

Operands can be entered with an absolute or a symbolic address. If the tag table is highlighted (not opened!) in the Project tree, tags (variables) can also be pulled from the Details view using drag & drop to the appropriate location in the program.

#### Favorites

Frequently used LAD (FBD) elements are available in the symbol bar which can be expanded individually using drag & drop from the Instructions catalog.

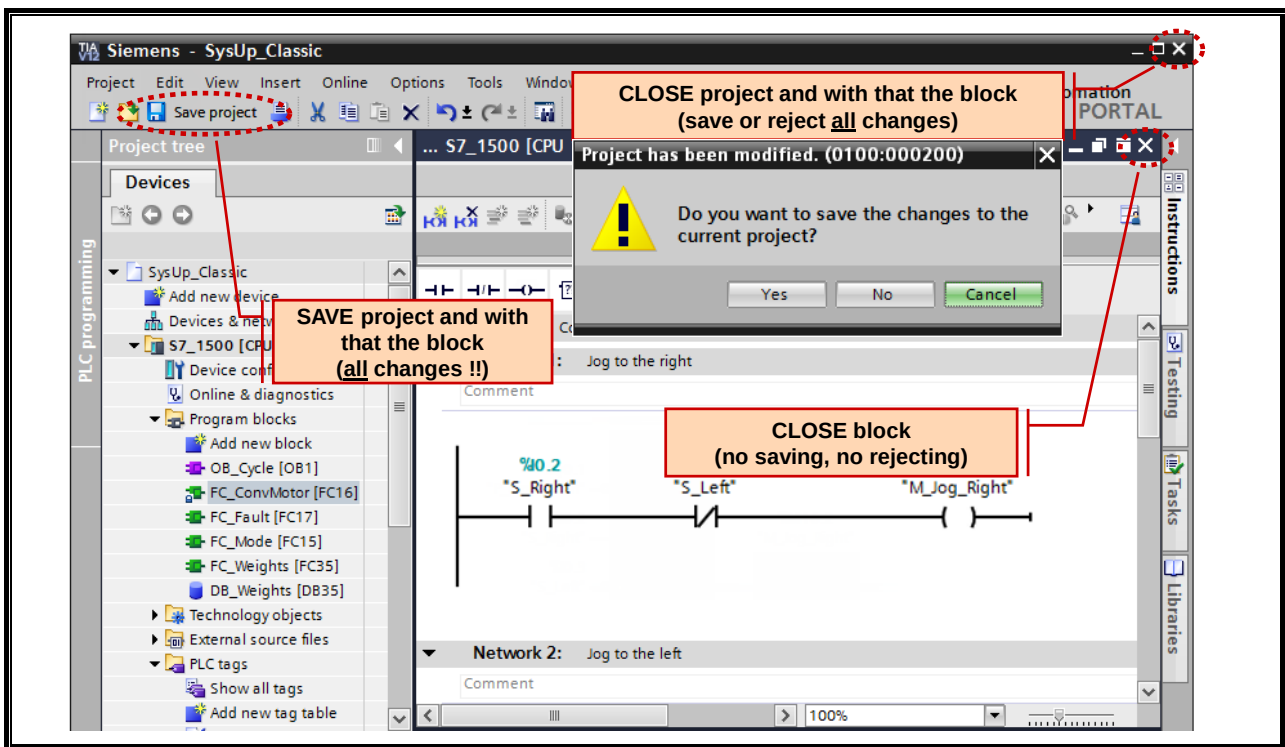
### 6.6.8. Programming an Instruction using "Empty Box"



#### Programming an Instruction using Empty Box

An instruction can also be programmed in the so-called "Empty box". First, the empty box is pulled into the network from the "Instructions" task card using drag & drop and then a selection is made on the empty box as to which instruction is to be used with which data type.

### 6.6.9. Closing / Saving / Rejecting a Block



#### Closing a Block

By clicking on the  symbol in the title bar, the block is merely closed. Changes are neither rejected nor are they saved on the hard drive!

#### Saving a Block



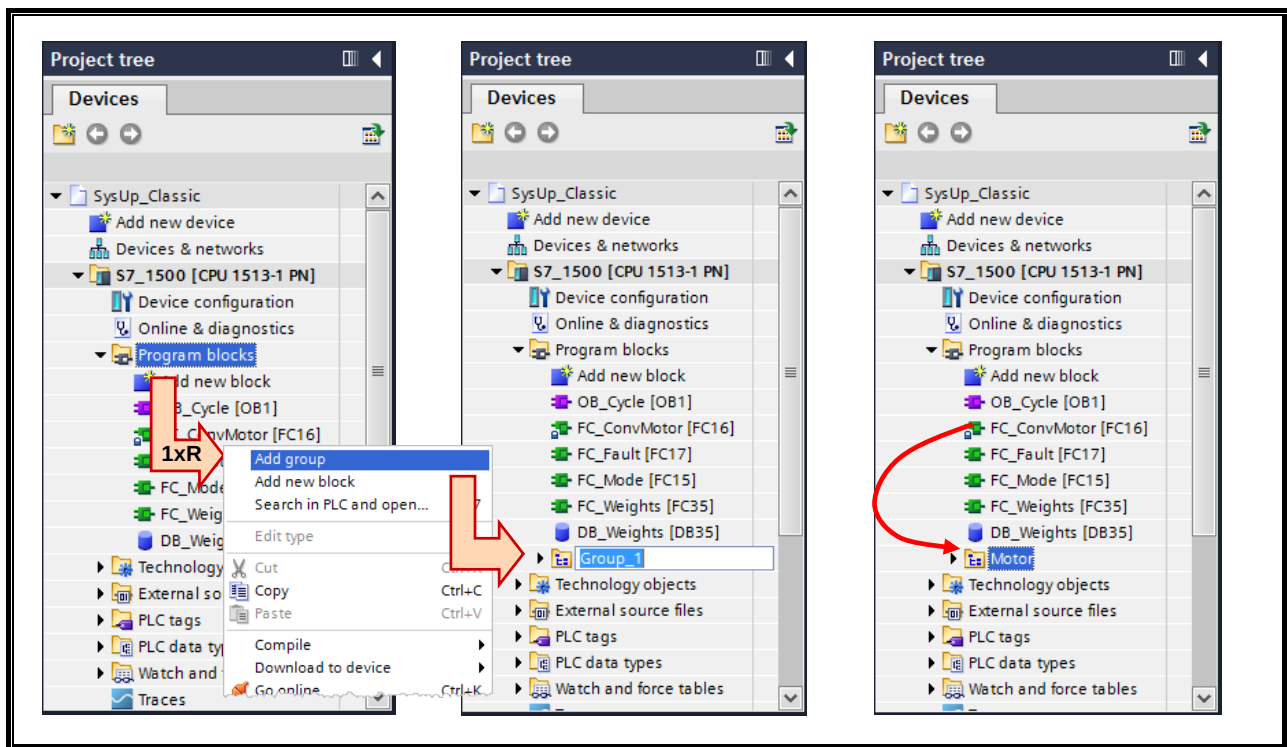
By using "Save project" the entire project, and with that also the block, is saved on the hard drive. All changes made to the project are saved.

#### Rejecting a Block

It is only possible to reject block changes by closing the entire project without saving. All changes made in the project are rejected.



## 6.6.10. Block Groups

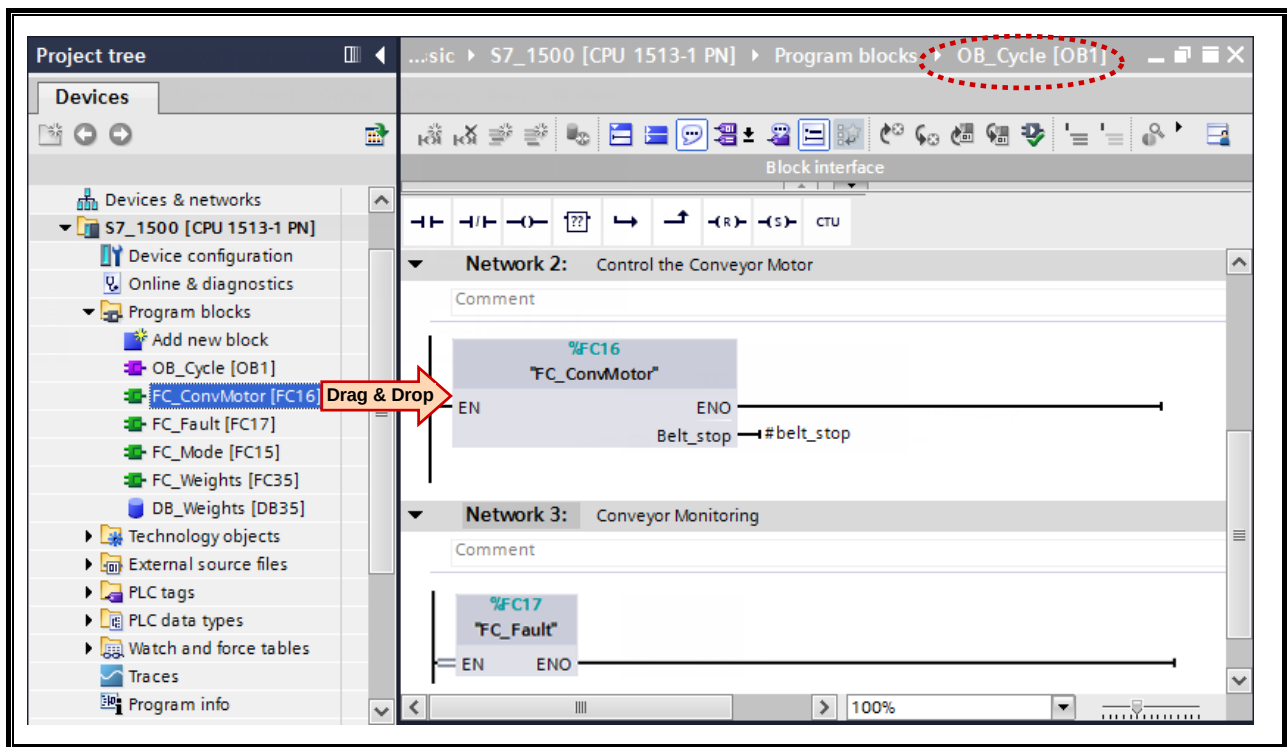


### Block Groups

To achieve more clarity, large programs with many blocks can be divided into different block groups. The groupings can, for example, be related to the structure of the system to be controlled. Even if the blocks are managed in different groups, each block must have a unique symbolic name. Regardless of the groupings, the sum of all blocks represents the user program of the controller.

The blocks can simply be shifted between the block groups using drag & drop.

### 6.6.11. Block Calls

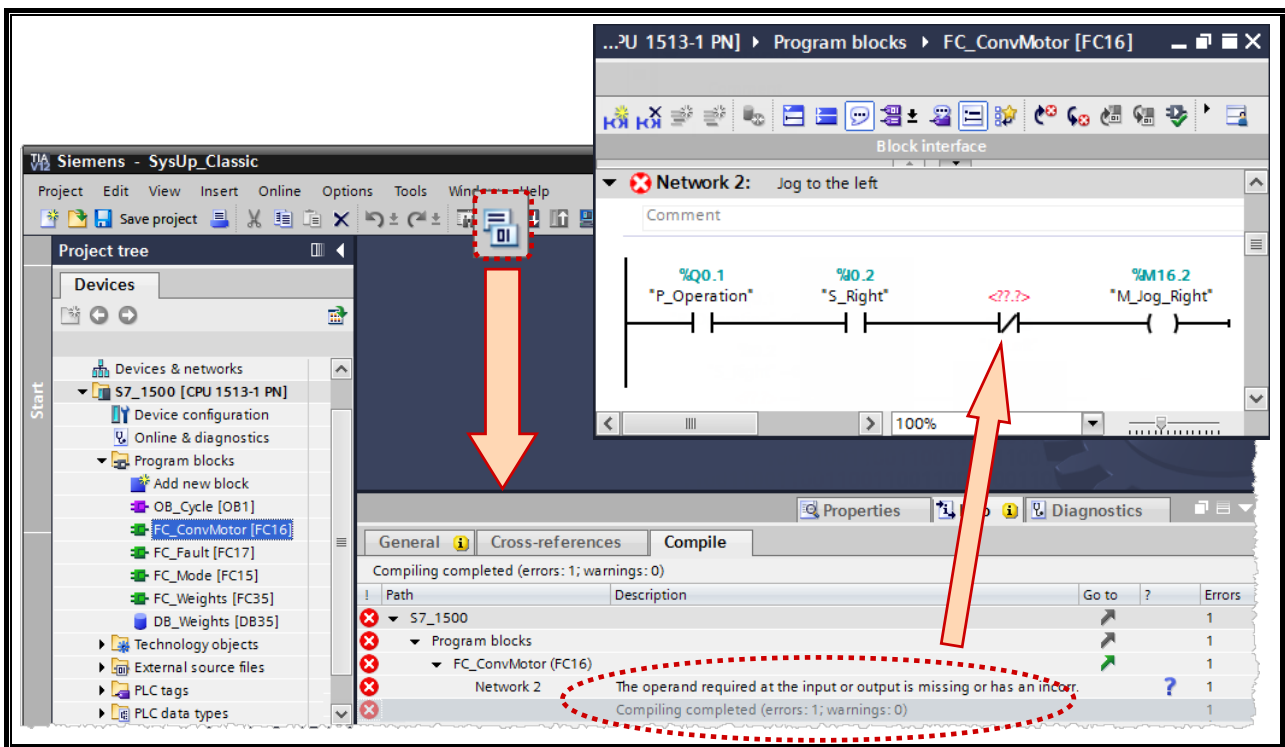


#### Block Calls

If one block calls another block, the instructions of the called block are executed. Only when the execution of the called block is completed, is the execution of the calling block taken up again and execution continues with the instruction that follows the block call.

The block call can be programmed using drag & drop or copy & paste.

## 6.6.12. Compiling a Block

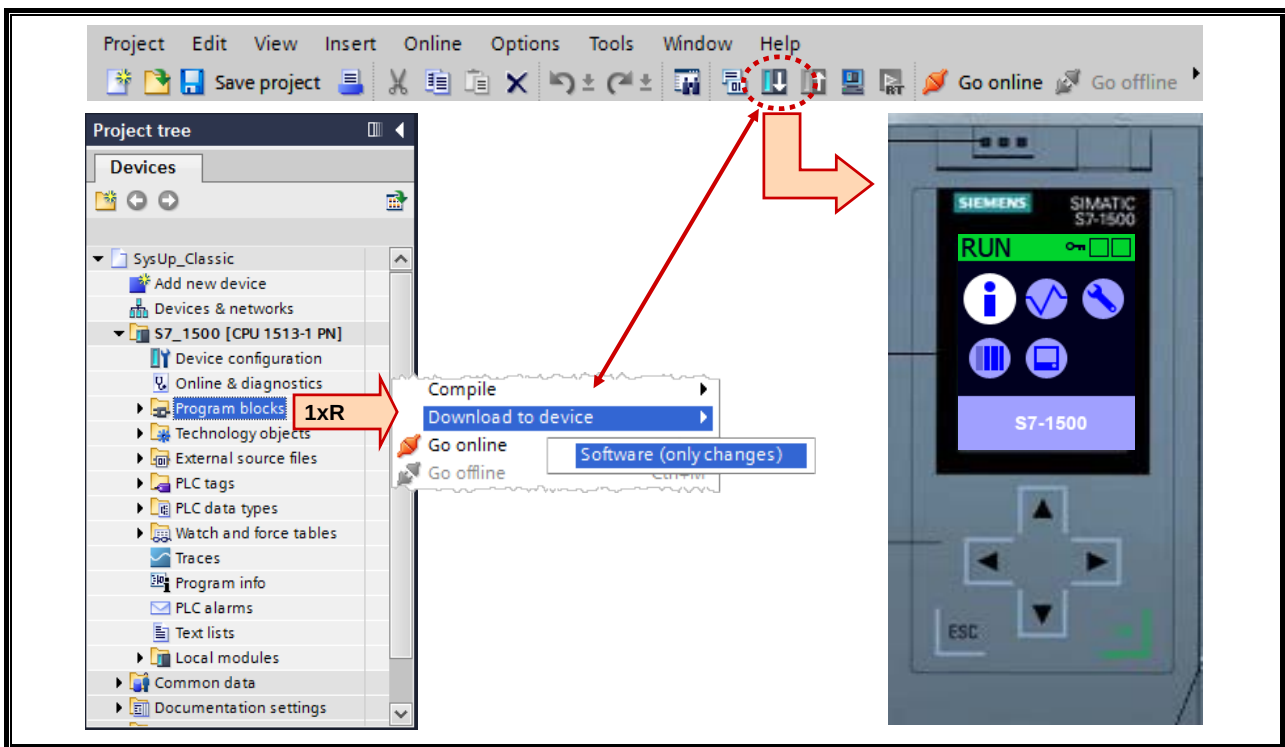


### Compiling a Block

Whatever is selected (highlighted) in the Project tree is compiled (in the example shown only "FC\_ConvMotor" FC16 is compiled). Individual blocks, the entire program ("Program blocks" is selected) or the entire station with software and hardware ("Station" is selected) can be compiled.

In the Inspector window "Info -> Compile", the status of the compilation is displayed. If errors occurred during compilation, you can jump directly from the error entry to the error location by double-clicking.

### 6.6.13. Downloading Blocks into the CPU



#### Downloading into the CPU

The project data which is downloaded into the devices is divided into hardware and software project data:

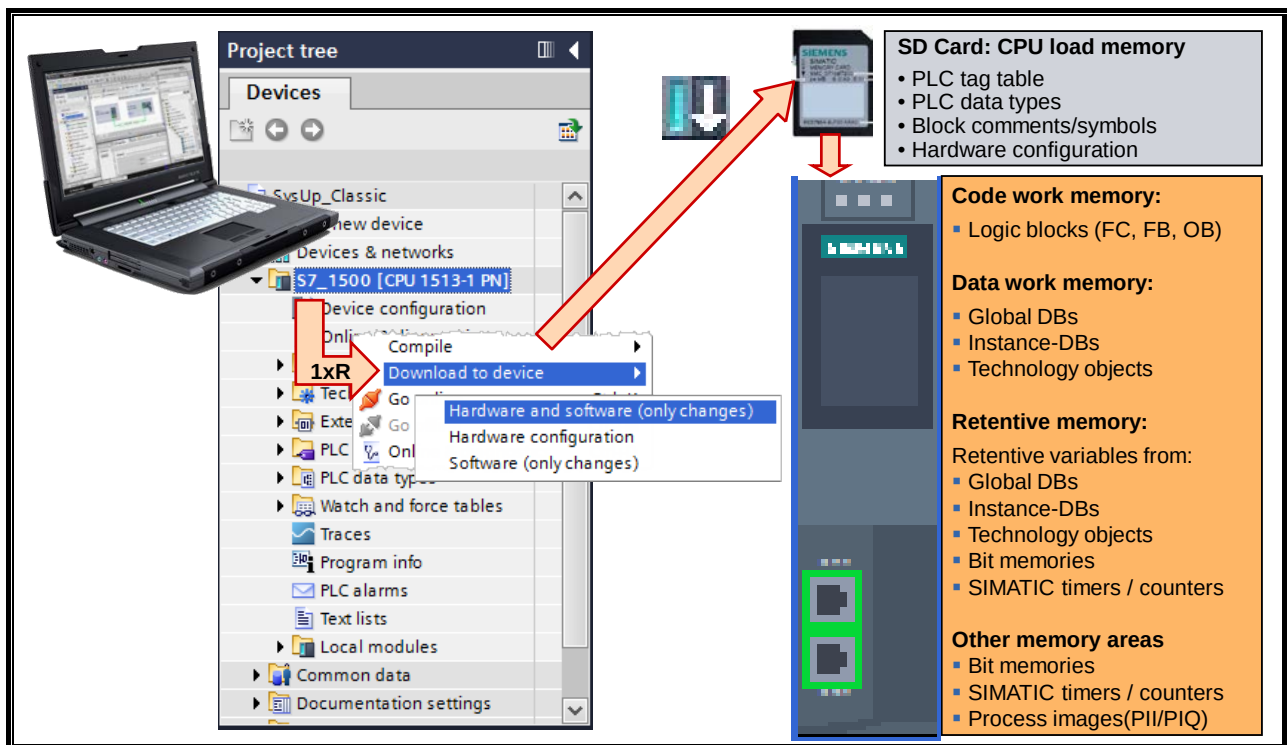
- Hardware project data results from configuring the hardware, networks, and connections.
- Software project data involves the blocks of the user program. The first time you download, the entire software project data is loaded. In subsequent downloads, only new and modified software project data is downloaded.

#### What is to be Downloaded?

**Selection via: Context menu of device > Download to device**

- |                                         |                                                                                                            |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------|
| – Hardware and Software (only changes): | Download all new and modified software project data as well as the new and modified hardware configuration |
| – Hardware configuration:               | Download the entire hardware configuration                                                                 |
| – Software (only changes):              | Download all new and modified software project data                                                        |

## 6.6.14. Downloading Blocks into the CPU: S7-1500 - Memory Concept



### Memory Areas of the CPU

The SD Card is the load memory of the CPU. Accordingly, an inserted SD Card is absolutely necessary for operating the CPU.

The data of the entire station is stored on the SD Card, that is, the complete S7 program including documentation, PLC tags and data types as well as the complete hardware configuration including distributed I/O and parameter assignments. In addition, other files can also be found on the SD Card, such as, recipes, HMI backups.....

When downloading the S7 program into the CPU, all blocks are always first loaded into the load memory from where the CPU automatically copies the parts of the blocks that are relevant for execution into the work memory.

### Work Memory

The work memory is a volatile RAM memory which cannot be expanded. It is divided into two areas:

- Code work memory which contains the parts of the logic blocks relevant for execution
- Data work memory which contains the data of the data blocks and technology objects relevant for execution.

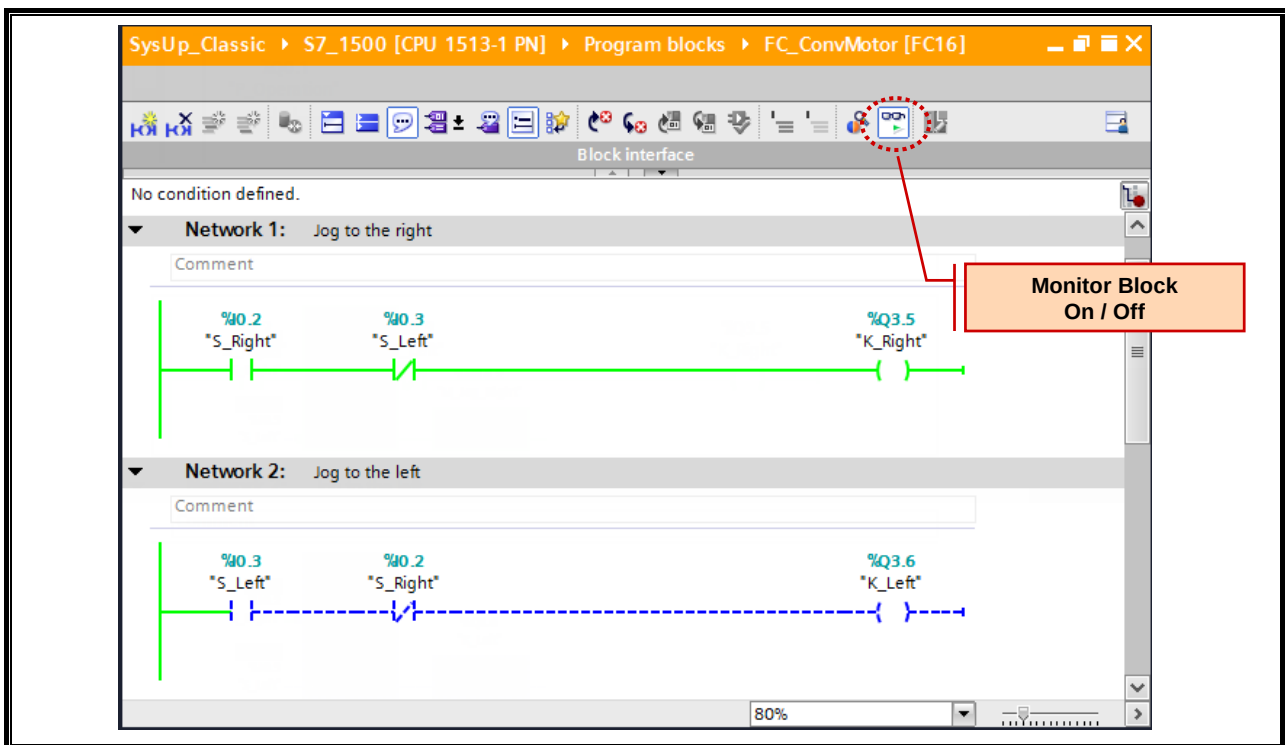
### Retentive Memory

The retentive memory is a non-volatile memory for saving the variables declared as retentive whose value is retained during a power failure.

The contents of the retentive memory is only erased by a

- memory reset
- resetting of the CPU to factory settings

### 6.6.15. Monitoring a Block



#### Monitoring Blocks

The test function *Monitor block* is used to follow the program execution within a block. The statuses or contents of the operands used in the block at the time of program execution are displayed on the monitor.

#### Monitoring

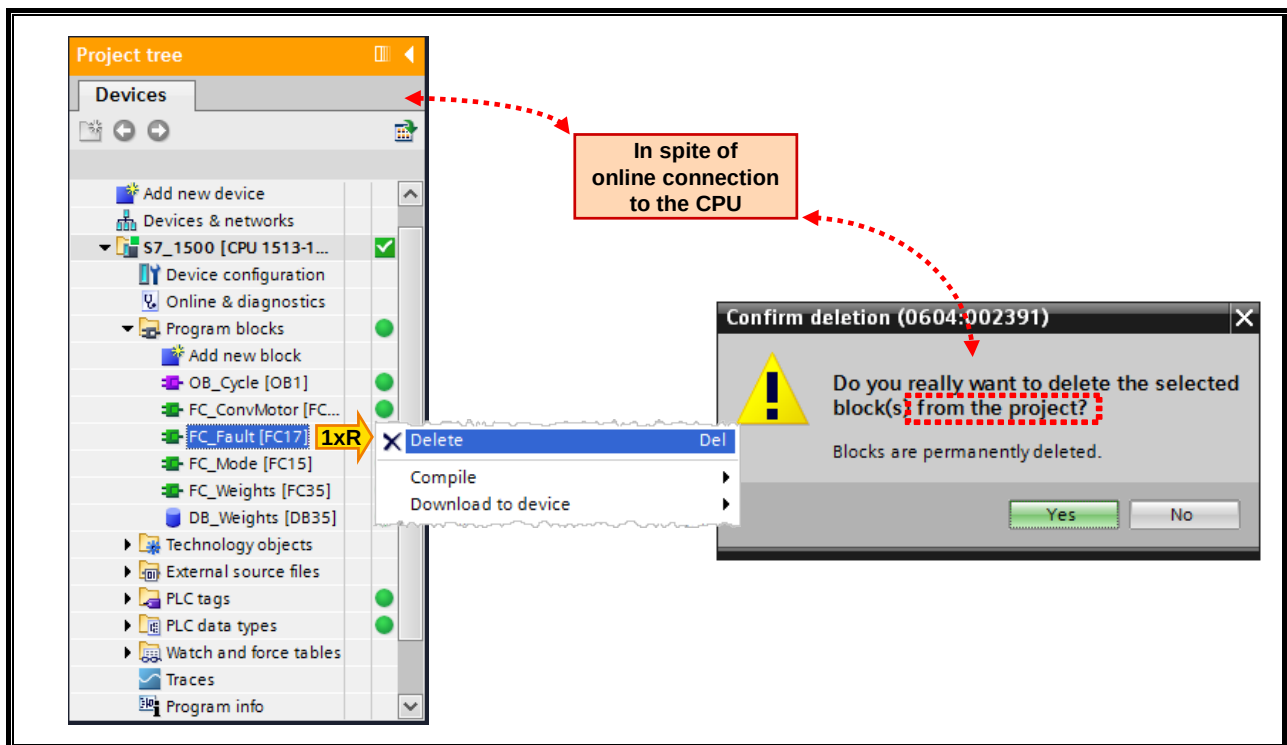
Blocks can only be monitored if an online connection to the CPU exists. Furthermore, the offline block must be identical to the online block.

In test mode, the statuses of operands and LAD / FBD elements are represented by different colors. You can make the settings for this via *Options* → *Settings*:

Examples:

- Status fulfilled → "Element is represented with a green color"
- Status not fulfilled → "Element is represented with a blue color"

## 6.6.16. Deleting Blocks

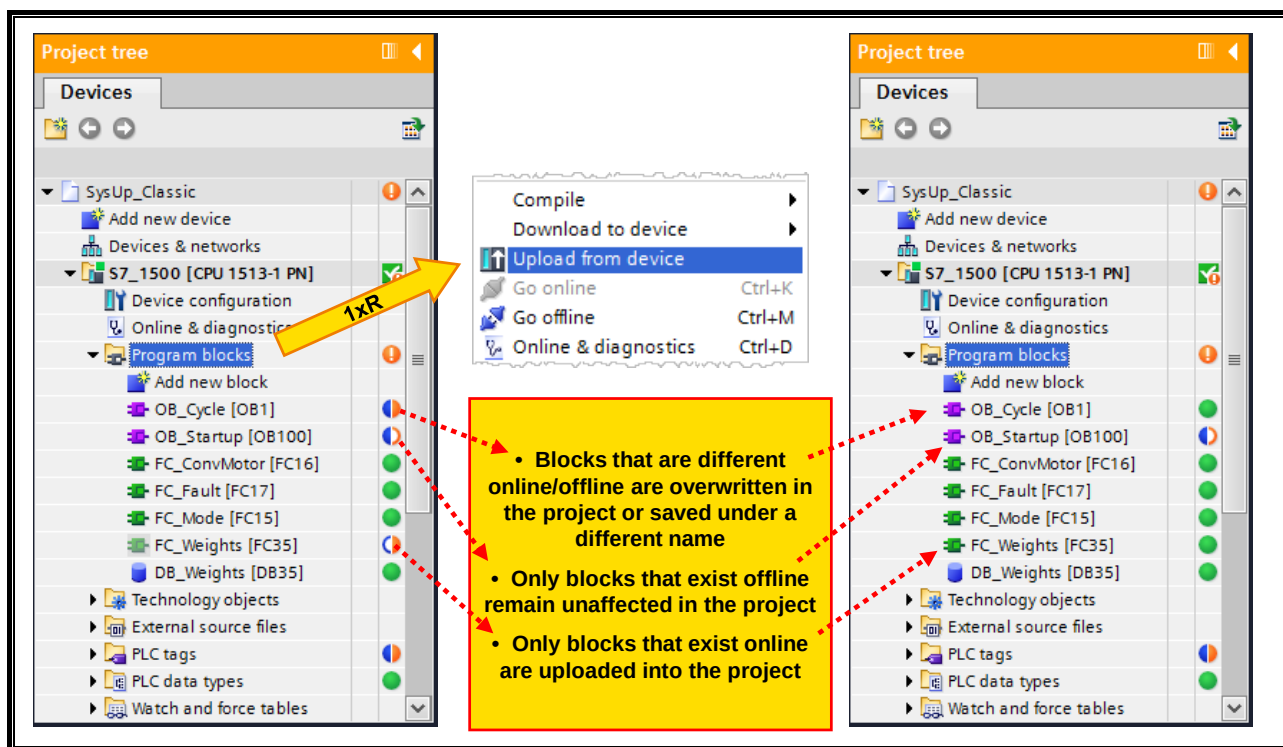


### Deleting Blocks

Block cannot be deleted online directly in the CPU. If a block (even with an existing online connection) is selected and "Delete" is activated, the dialog shown in the picture appears with the question of whether the block is to be deleted offline.

With subsequent, consistent loading of the entire program, the blocks which only exist in the CPU are deleted there online.

### 6.6.17. "Upload" Blocks "from Device" (Upload into Project)



#### Uploading Blocks into the Project:

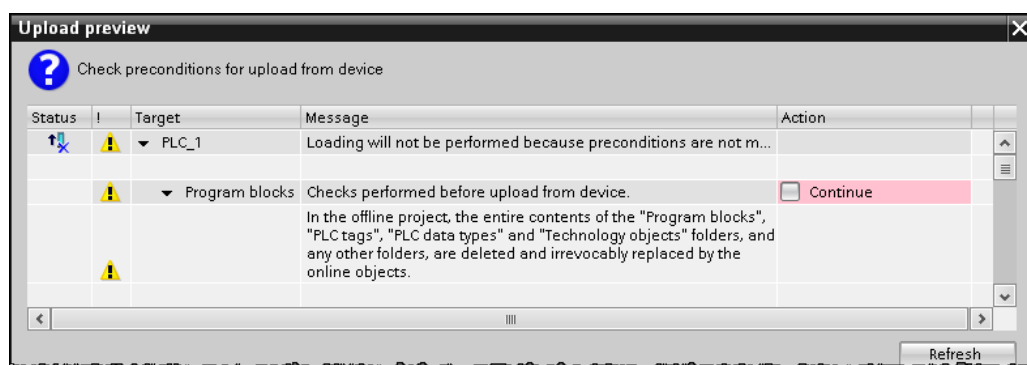
With "Upload from device", individual blocks or the entire CPU program including technology objects, PLC tag tables and PLC data types can be uploaded into the project from the CPU.

#### If the Blocks folder or individual blocks is/are selected, then...

- ...blocks that only exist online in the CPU are uploaded into the offline project.
- ...blocks that are different online / offline are either overwritten in the offline project with the uploaded blocks, or the uploaded blocks are additionally saved under a different name (but with the name number!) in the project. (selectable in the Upload dialog)
- ...blocks that only exist offline are not influenced.

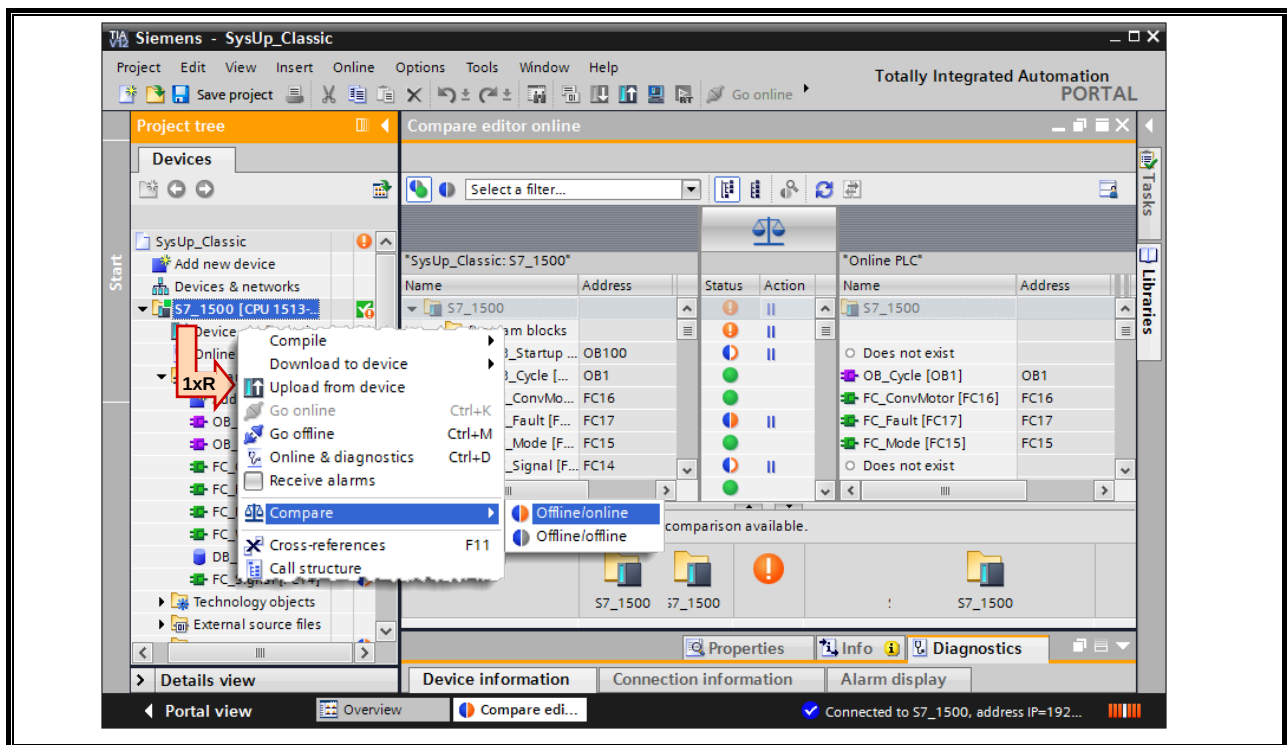
#### If the Station is selected, then...

- ...offline all blocks, PLC data types, technology objects and symbols are **deleted (!)** and the online blocks, PLC data types, technology objects and symbols are uploaded into the project.





## 6.6.18. Comparing Blocks



### Types of Comparison

In principle, there are two different types of comparison:

Online/Offline comparison:

- The objects in the project are compared with the objects of the relevant device. For this, an online connection to the device is necessary.

Offline/Offline comparison:

- Either the objects of two devices within a project, from different projects or a device and a library are compared.

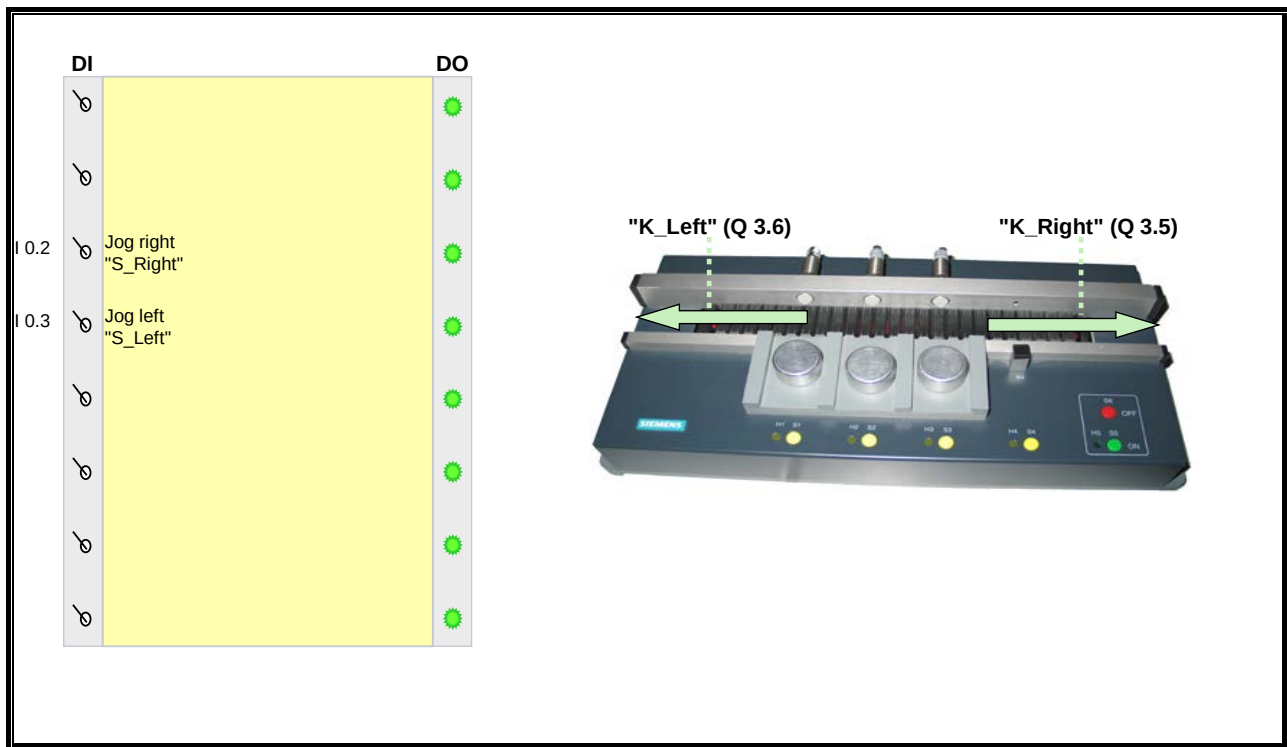
### Symbols of the Results Display

The result of the comparison is presented by means of symbols.

The following table shows the symbols for the comparison results of an Online/Offline comparison:

| Symbol | Meaning                                                                 |
|--------|-------------------------------------------------------------------------|
| !      | Folder contains objects whose online and offline versions are different |
| ?      | Comparison result is unknown                                            |
| ●      | Online and offline versions of the object are identical                 |
| ●      | Online and offline versions of the object are different                 |
| ●      | Object only exists offline                                              |
| ●      | Object only exists online                                               |

## 6.7. Task Description: Jogging the Conveyor Motor

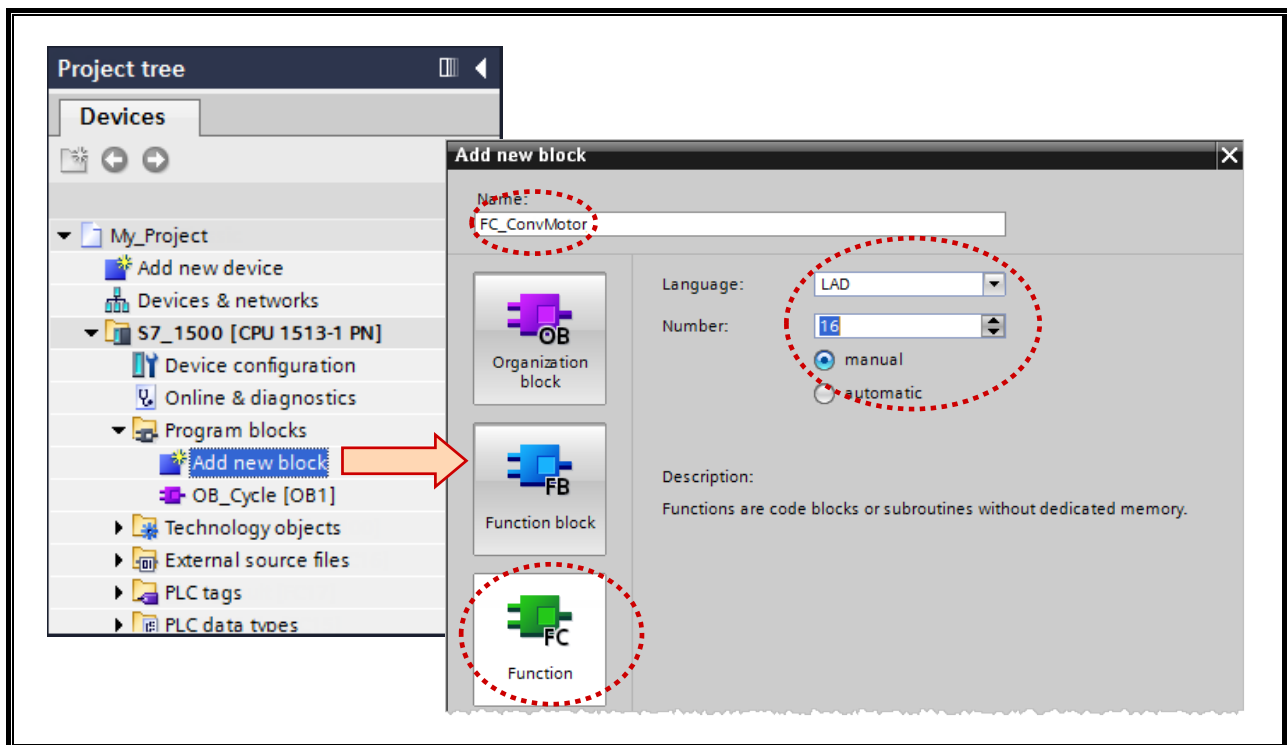


### Task Description

Using the simulator pushbutton "S\_Right" (I 0.2), you should be able to jog the conveyor motor to the RIGHT "K\_Right" (Q 3.5). Using the simulator pushbutton "S\_Left" (I 0.3), you should be able to jog the conveyor motor to the LEFT "K\_Left" (Q 3.6).

If both "pushbuttons" are pressed simultaneously, then the conveyor motor may not start in either direction (Lock-out!).

### 6.7.1. Exercise 1: Adding the "FC\_ConvMotor" (FC16) Block



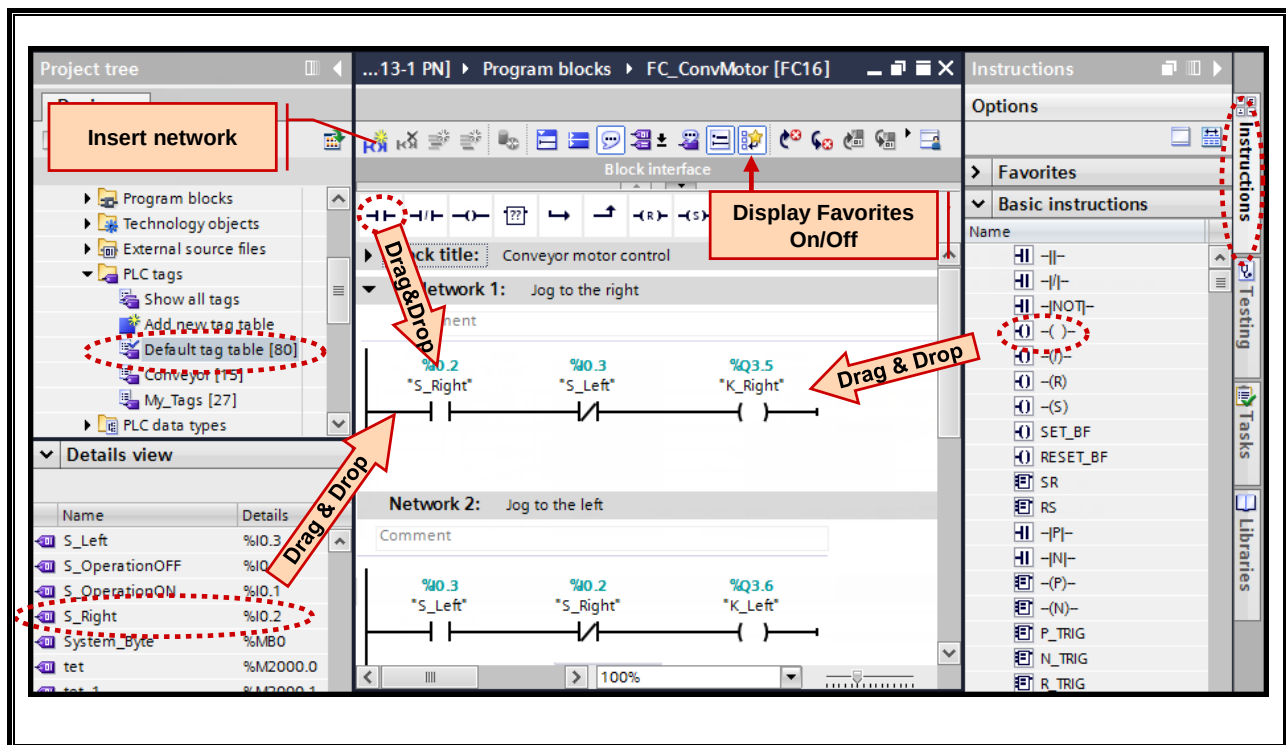
#### Task

Create "FC\_ConvMotor" (FC16) as a new block in which you will then program the required jog operation of the conveyor motor in the next exercise.

#### What to Do

1. In the "Program blocks" container, double-click on "Add new block".
2. In the dialog that appears, make the entries as shown in the picture above.

### 6.7.2. Exercise 2: Programming the "FC\_ConvMotor" (FC16) Block



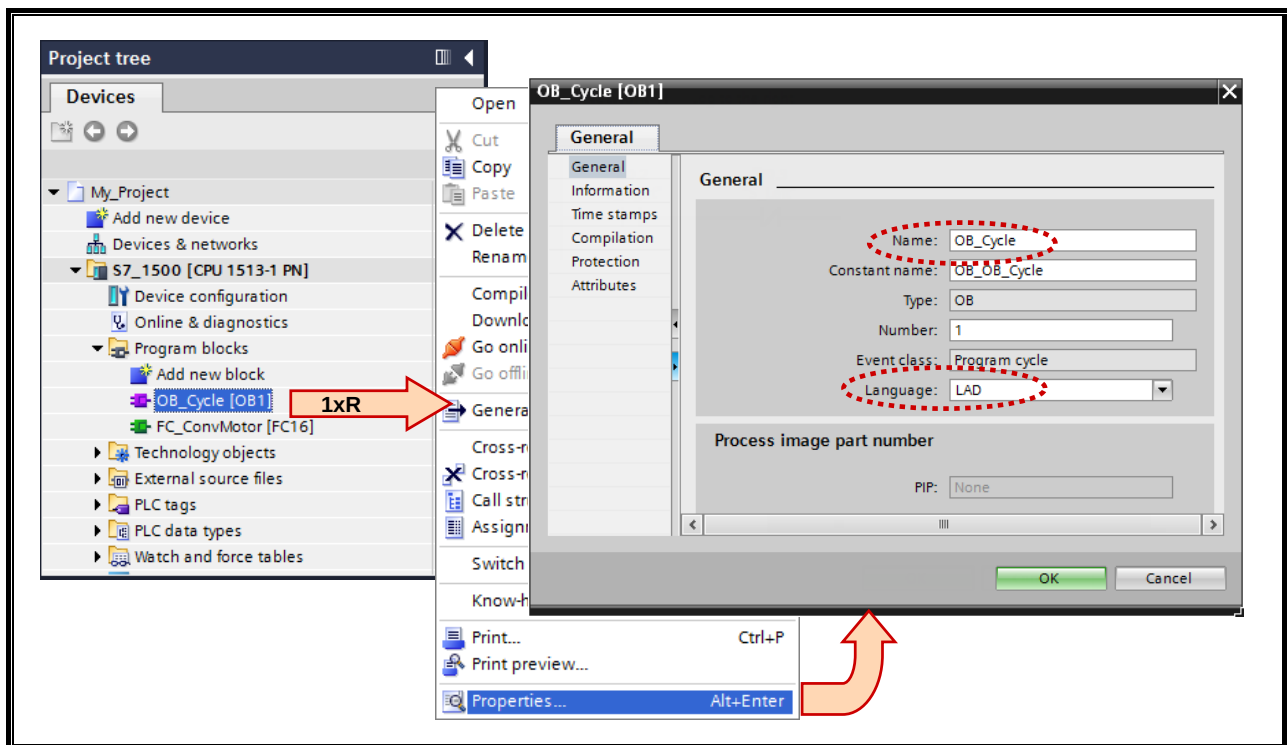
#### Task

Program the jog operation of the conveyor motor as shown in the picture.

#### What to Do

1. Program an AND logic operation by pulling it from the Favorites into the network using drag & drop.
2. Program an instruction by pulling it from the "Instructions" task card to the AND logic operation using drag & drop.
3. Above the instruction, enter the output "K\_Right" (Q3.5) as the operand (you can enter the symbol as well as the absolute address)
4. In the Project tree, select (don't open) the Default tag table and from the "Details view" drag the tag "S\_Left" (I 0.3) as the operand to the AND logic operation. Do exactly the same for "S\_Right" (I 0.2).
5. Label both the block and the network.
6. Add a new network and there program a corresponding logic operation for jogging the conveyor motor to the left.
7. Close the block.
8. Save your project.

### 6.7.3. Exercise 3: Adapting the OB1 Properties

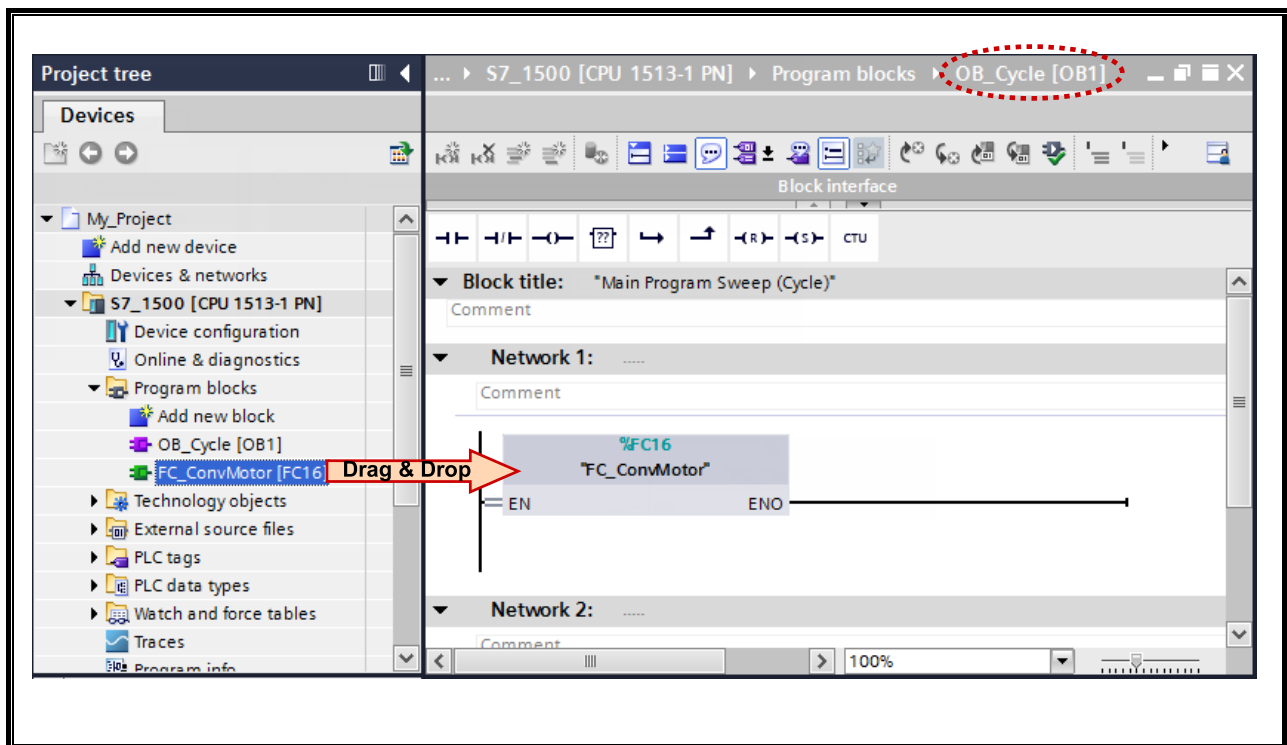


#### Task Description

The OB1 properties are to be adapted as shown in the picture.

The symbolic name is to be changed and the programming language LAD is to be set. Proceed as shown in the picture.

#### 6.7.4. Exercise 4: Calling "FC\_ConvMotor" (FC16) in OB1



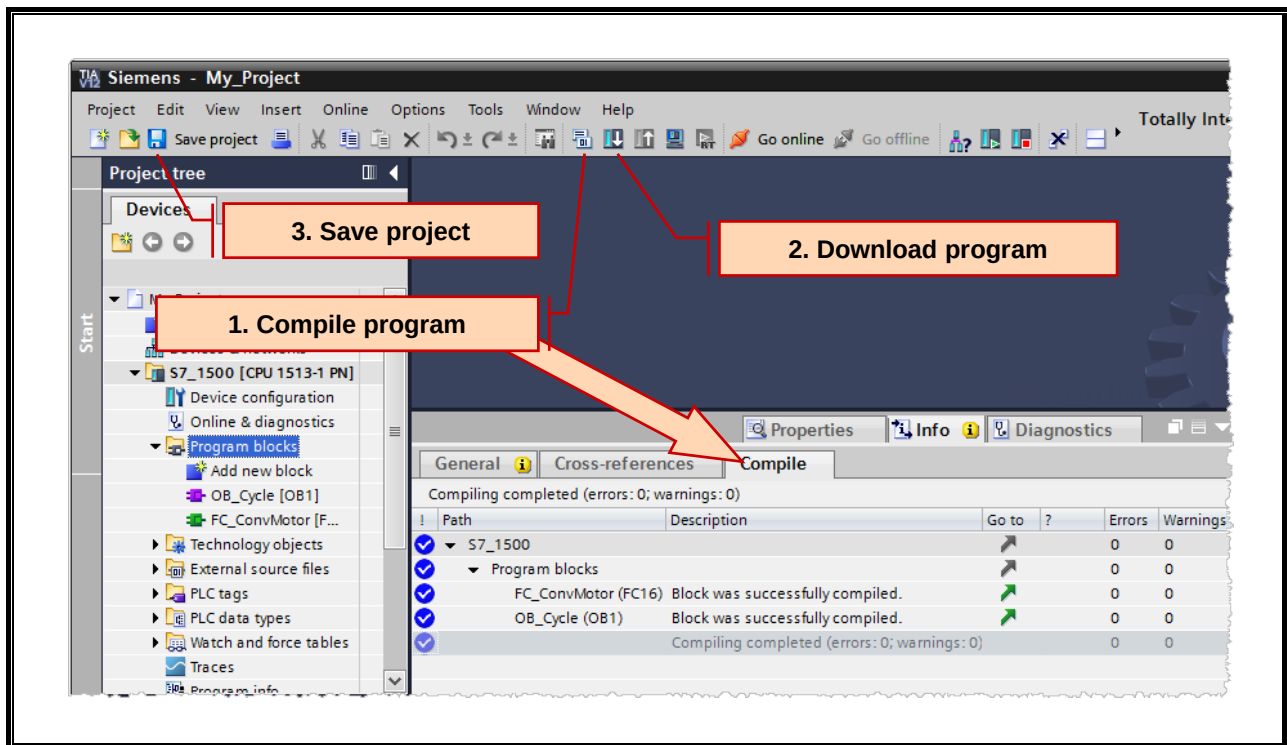
#### Task

So that the newly created block "FC\_ConvMotor" (FC16) can be executed cyclically, its call must be programmed in OB1.

#### What to Do

1. Open the block "OB\_Cycle" (OB1) by double-clicking on it.
2. Program the call of "FC\_ConvMotor" (FC16) as shown in the picture using drag & drop.
3. Edit a block title and a label for Network 1.
4. Close the block.
5. Save your project.

### 6.7.5. Exercise 5: Compiling the Program, Downloading it into the CPU and Saving it



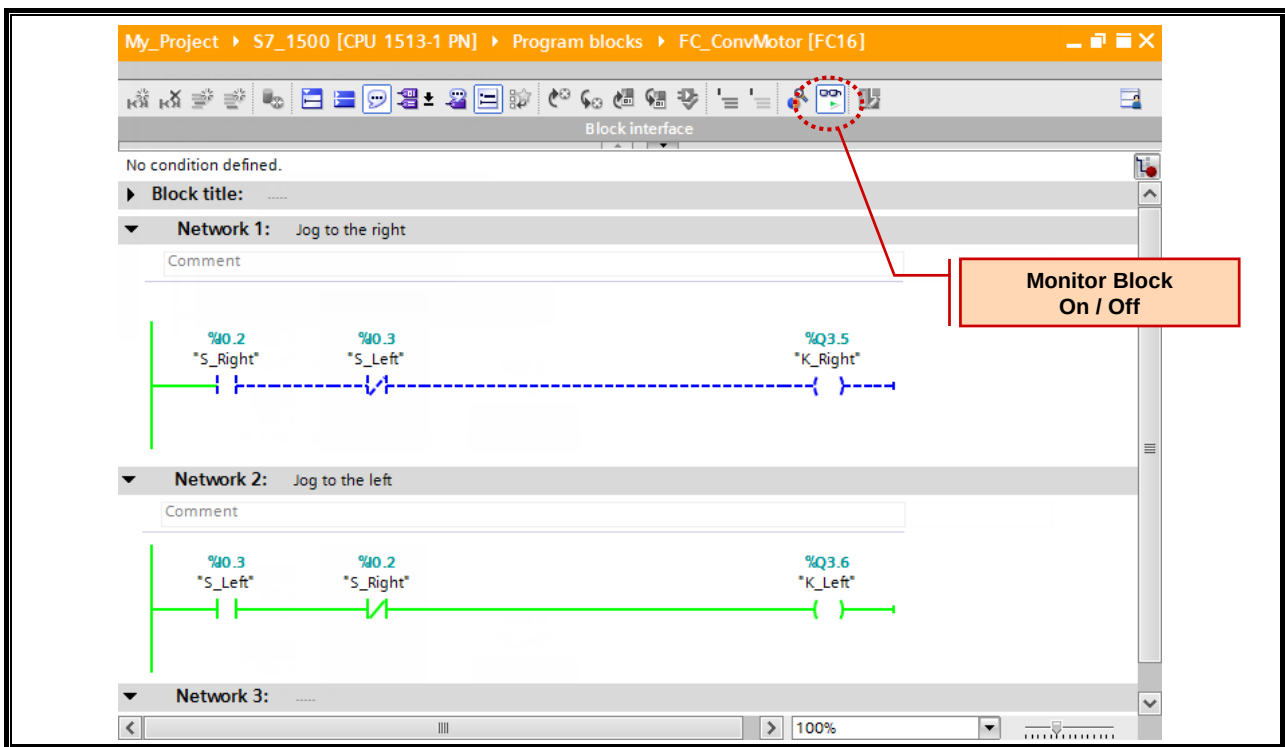
#### Task

The newly programmed blocks are to be compiled, downloaded into the CPU and saved offline in the project data storage.

#### What to Do

1. To compile the entire program (OB1 and FC16) and to download it into the CPU, select the "Program blocks" container in the Project tree.
2. Carry out the steps shown in the picture and check the program functioning by pressing the simulator pushbuttons "S\_Right" (I 0.2) and "S\_Left" (I 0.3).

### 6.7.6. Exercise 6: Monitoring "FC\_ConvMotor" (FC16)



#### Task

You are to monitor the program execution of the newly programmed block "FC\_ConvMotor" (FC16). To do so, press the simulator pushbuttons "S\_Right" (I 0.2) and "S\_Left" (I 0.3) and interpret the statuses shown on the PG screen.

#### What to Do

1. In the Editor, open the block "FC\_ConvMotor" (FC16).
2. Activate the test function "Monitor block" (see picture).
3. Alternately and also simultaneously press the simulator pushbuttons "S\_Right" (I 0.2) and "S\_Left" (I 0.3) and interpret the signal statuses of the operands shown.



# Contents

|           |                                                                                        |            |
|-----------|----------------------------------------------------------------------------------------|------------|
| <b>7.</b> | <b>Binary Operations .....</b>                                                         | <b>7-2</b> |
| 7.1.      | Task Description: The Conveyor Model as Distribution Conveyor .....                    | 7-3        |
| 7.2.      | Binary Logic Operations: AND, OR.....                                                  | 7-4        |
| 7.2.1.    | Sensors and Check Symbols.....                                                         | 7-5        |
| 7.2.2.    | First Check, Signal State, Result of Check and Result of Logic Operation.....          | 7-6        |
| 7.2.3.    | Theory Exercise 1: Sensor and Check Symbols .....                                      | 7-7        |
| 7.2.4.    | Binary Logic Operations: Exclusive OR (XOR).....                                       | 7-8        |
| 7.3.      | Assignment, Set, Reset, NOT .....                                                      | 7-9        |
| 7.3.1.    | Flip Flops.....                                                                        | 7-10       |
| 7.4.      | Task Description: "FC_Mode" (FC15).....                                                | 7-11       |
| 7.4.1.    | Exercise 2: Programming the "FC_Mode" (FC15) Block.....                                | 7-12       |
| 7.4.2.    | Task Description: Parts Transportation when "P_Operation" (Q0.1) is Switched On .....  | 7-13       |
| 7.4.3.    | Exercise 3: Expanding "FC_ConvMotor" (FC16).....                                       | 7-14       |
| 7.5.      | Task Description: Parts Transportation THROUGH the Light Barrier .....                 | 7-15       |
| 7.5.1.    | Signal - Edge Evaluation.....                                                          | 7-16       |
| 7.5.2.    | RLO - Edge Evaluation .....                                                            | 7-17       |
| 7.5.3.    | Exercise 4: Integrating an Edge Evaluation in "FC_ConvMotor" (FC16).....               | 7-18       |
| 7.6.      | Task Description: Controlling the Indicator Lights, Commissioning "FC_Signal" (FC14).. | 7-19       |
| 7.6.1.    | Exercise 5: Commissioning "FC_Signal" (FC14).....                                      | 7-20       |
| 7.7.      | Additional Information .....                                                           | 7-21       |
| 7.7.1.    | Additional Exercise 6: Optimizing "FC_Mode" (FC 15).....                               | 7-22       |
| 7.7.2.    | Jump Instructions JMP, JMPN, RET.....                                                  | 7-23       |

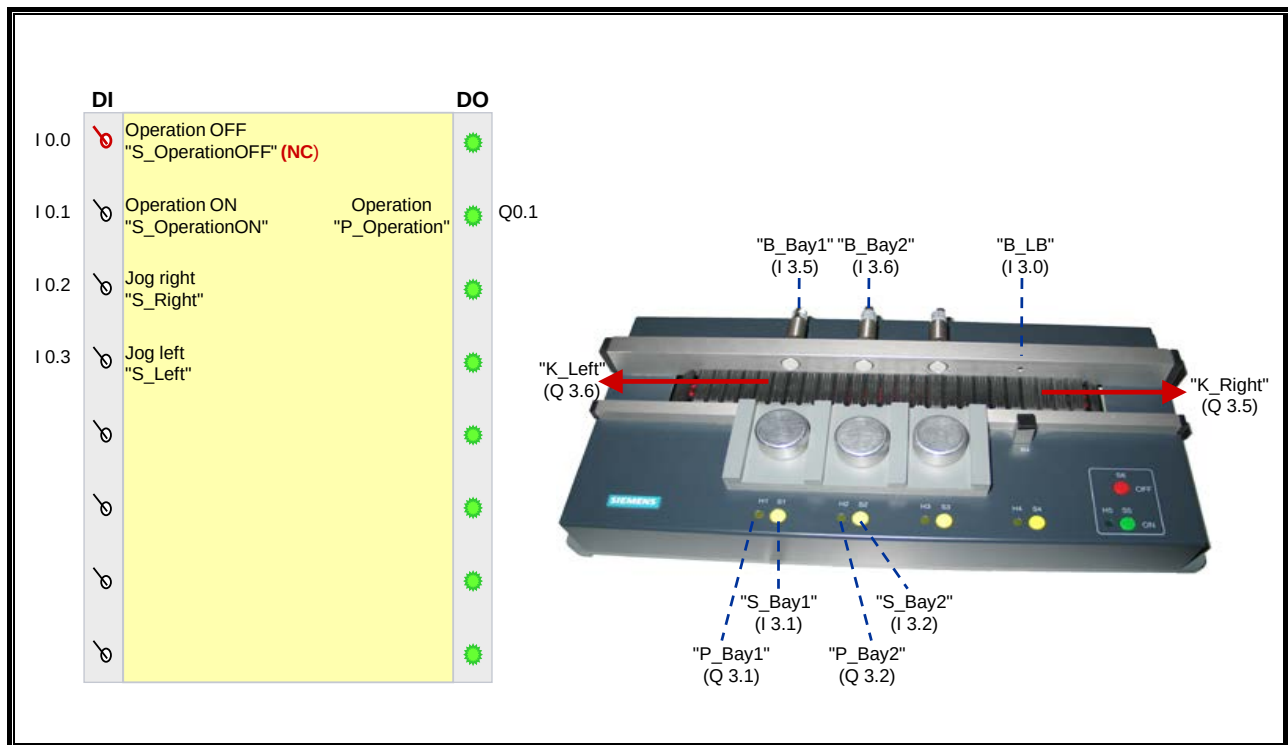
## 7. Binary Operations

**At the end of the chapter the participant will ...**

- ... understand the difference between 'real' NC contacts and NO contacts connected in the hardware, and programmed check symbols
- ... be able to explain the terms Result of Logic Operation (RLO) and Status (STAT)
- ... be able to program basic binary logic operations
- ... be able to carry out simple troubleshooting



## 7.1. Task Description: The Conveyor Model as Distribution Conveyor



### Function Description

The distribution conveyor is used to transport parts from Bay 1 or 2 to the light barrier bay. Operation (Simulator LED "P\_Operation", Q0.1) can be switched on via the simulator pushbutton "S\_OperationON" (I 0.1) and switched off via the simulator pushbutton "S\_OperationOFF" (I 0.0, NC).

- When "P\_Operation" (Q0.1) is switched off...  
...the conveyor motor can be jogged to the right via the simulator pushbutton "S\_Right" (I 0.2) and jogged to the left via the simulator pushbutton "S\_Left" (I 0.3).

- When "P\_Operation" (Q0.1) is switched on...  
...parts are transported from Bay 1 or 2 through to the light barrier. For this, the part must be placed on the conveyor at exactly one of the two bays and the associated bay pushbutton pressed.

The indicator lights at Bays 1 and 2 show...

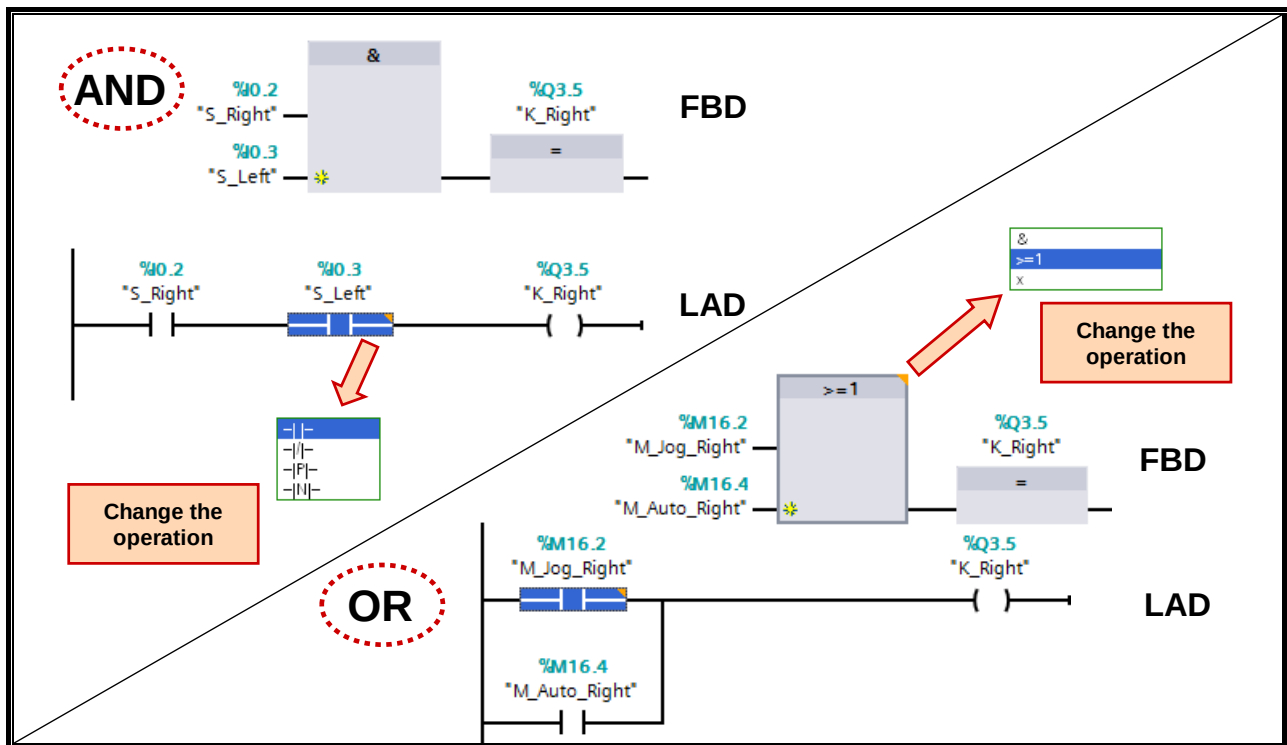
...a continuous light when a new part can be placed on the conveyor (conveyor motor is stopped and both proximity sensors are free)

...a 1Hz flashing light at the bay on which a part is detected by the associated proximity sensor, however, only as long as the conveyor has not yet been started (if parts are placed on the conveyor at both proximity sensors, neither indicator light can light up)

...a 2Hz flashing light as long as the conveyor motor is running.

The indicator light at the light barrier bay shows a 2Hz flashing light as long as the conveyor motor is running and continuous light when the setpoint quantity has been reached.

## 7.2. Binary Logic Operations: AND, OR



### AND and OR Logic Operations

With the AND and OR logic operations, basically all binary operands can be queried, even outputs. Instead of individual operands, the results of other logic operations can also be further logically linked. Also, the logic operations can also be combined.

All inputs of the logic operations can be programmed as check for signal state or Status '0' and '1', regardless of whether a hardware NO contact or NC contact is connected in the process.

### AND Logic Operation

For an AND logic operation, the result of logic operation (RLO) = '1', when all input signals have Status '1'.

### OR Logic Operation

For an OR logic operation, the result of logic operation (RLO) = '1', when at least one input signal has Status '1'.

### 7.2.1. Sensors and Check Symbols

| Process                                                                                         |                                                                                                 |                           | Interpretation in the PLC Program |                                                                                                           |                 |                                                                                                             |                 |
|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------|-----------------|
| The sensor is a ...                                                                             | The sensor is ...                                                                               | Voltage present at input? | Signal state at input             | Check for signal state "1"                                                                                | Result of check | Check for signal state "0"                                                                                  | Result of check |
|                                                                                                 |                                                                                                 |                           |                                   | Symbol / Instruction                                                                                      |                 | Symbol / Instruction                                                                                        |                 |
| NO contact<br> | activated      | yes                       | 1                                 | LAD:<br><br>"NO contact" | "Yes" 1         | LAD:<br><br>"NC contact" | "No" 0          |
|                                                                                                 | not activated  | no                        | 0                                 | FBD:<br>                 | "No" 0          | FBD:<br>                 | "Yes" 1         |
| NC contact<br> | activated      | no                        | 0                                 | STL:<br>AI x.y                                                                                            | "Yes" 1         | STL:<br>AN I x.y                                                                                            | "No" 0          |
|                                                                                                 | not activated  | yes                       | 1                                 |                                                                                                           |                 |                                                                                                             |                 |

#### Sensors of the Process

The use of normally open or normally closed contacts for the sensors in a controlled process depends on the safety regulations for that process. Normally closed contacts are always used for limit switches and safety switches, so that dangerous conditions do not arise if a wire break occurs in the sensor circuit. Normally closed contacts are also used for switching off machinery for the same reason.

#### Check Symbols of the Program

In LAD, a symbol with the name "NO contact" is used for checking for signal state "1" and a symbol with the name "NC contact" to check for signal state "0". It makes no difference whether the process signal "1" is supplied by an activated NO contact or a non-activated NC contact.

The "NO contact" symbol delivers the result of check or result of logic operation (RLO) "1" when the checked operand has state or status "1".

The "NC contact" symbol delivers the result of check or result of logic operation (RLO) "1" when the checked operand has state or status "0".

## 7.2.2. First Check, Signal State, Result of Check and Result of Logic Operation

|          | Example 1    |                 |                           |             | Example 2    |                 |                           |             | Example 3    |                 |                           |             |
|----------|--------------|-----------------|---------------------------|-------------|--------------|-----------------|---------------------------|-------------|--------------|-----------------|---------------------------|-------------|
|          | Signal State | Result of Check | Result of Logic Operation | First Check | Signal State | Result of Check | Result of Logic Operation | First Check | Signal State | Result of Check | Result of Logic Operation | First Check |
| ...      |              |                 |                           |             |              |                 |                           |             |              |                 |                           |             |
| = M 3.4  |              |                 |                           |             |              |                 |                           |             |              |                 |                           |             |
| A I 1.0  | 0            |                 |                           |             | 1            |                 |                           |             | 1            |                 |                           |             |
| AN I 1.1 | 0            |                 |                           |             | 1            |                 |                           |             | 0            |                 |                           |             |
| A M 4.0  | 0            |                 |                           |             | 1            |                 |                           |             | 1            |                 |                           |             |
| = Q 8.0  |              |                 |                           |             |              |                 |                           |             |              |                 |                           |             |
| = Q 8.1  |              |                 |                           |             |              |                 |                           |             |              |                 |                           |             |
| A I 2.0  | 0            |                 |                           |             | 1            |                 |                           |             | 0            |                 |                           |             |

### Signal State

A logic operation is made up of a series of instructions to check (query) the states of signals (inputs (**I**), outputs (**Q**), bit memories (**M**), timers (**T**), counters (**C**) or data bits (**D**)) and instructions to set Q,M,T,C or D.

### Result of Check (Query)

When the program is executed, the result of check (query) is obtained. If the check condition is fulfilled, the result of check is "1". If the check condition is not fulfilled, the result of check is "0".

### First Check (Scan)

The first check (scan) that follows an RLO limiting operation (such as S, R, CU, = ...) or the first check in a logic string is called a First Check (FC) since the result of this check - regardless of the last RLO - is accepted as the new RLO.

### Result of Logic Operation

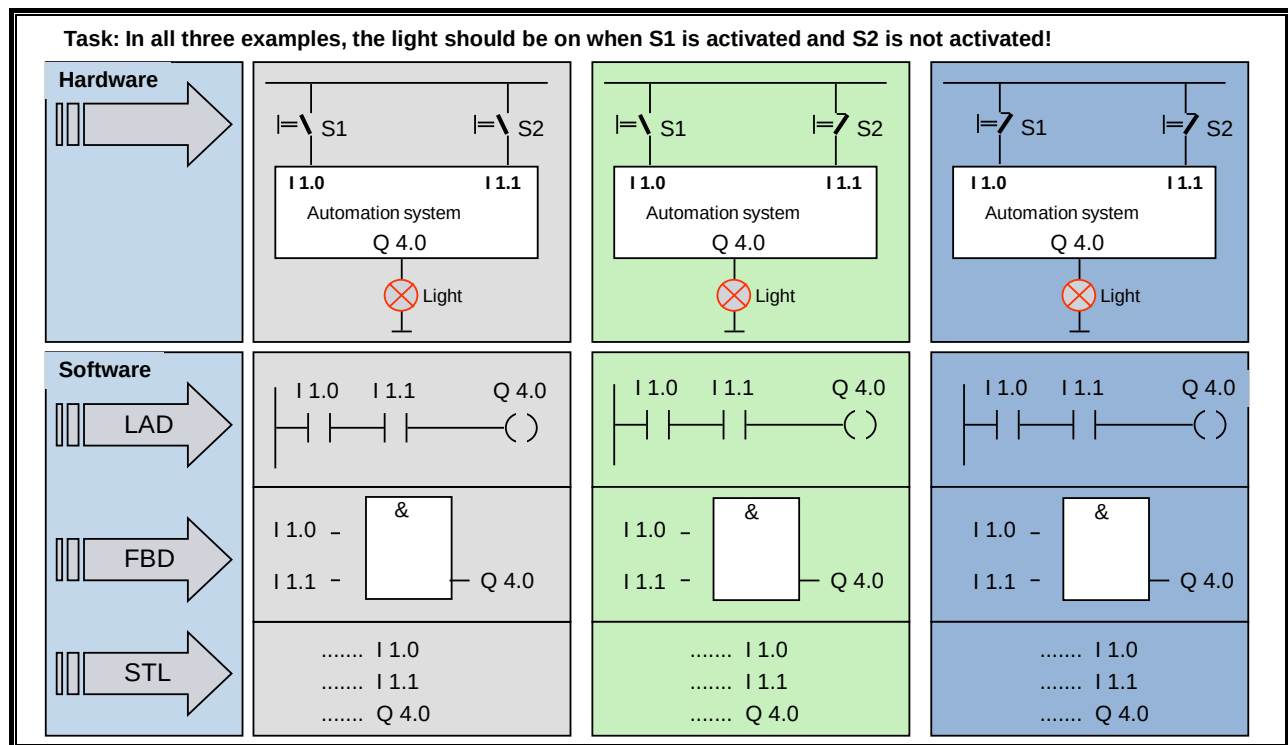
When the next check instructions are executed, the result of logic operation is gated with the result of check and a new RLO is obtained.

When the last check instruction in a logic operation has been executed, the RLO remains the same. A number of instructions using the same RLO can follow.

### Note

The result of the first check (scan) is stored without being subjected to a logic operation. Therefore, it makes no difference whether you program the first check with an AND or an OR instruction in STL. To convert your program to one of the other programming languages, you should, however, always program using the correct instruction.

### 7.2.3. Theory Exercise 1: Sensor and Check Symbols



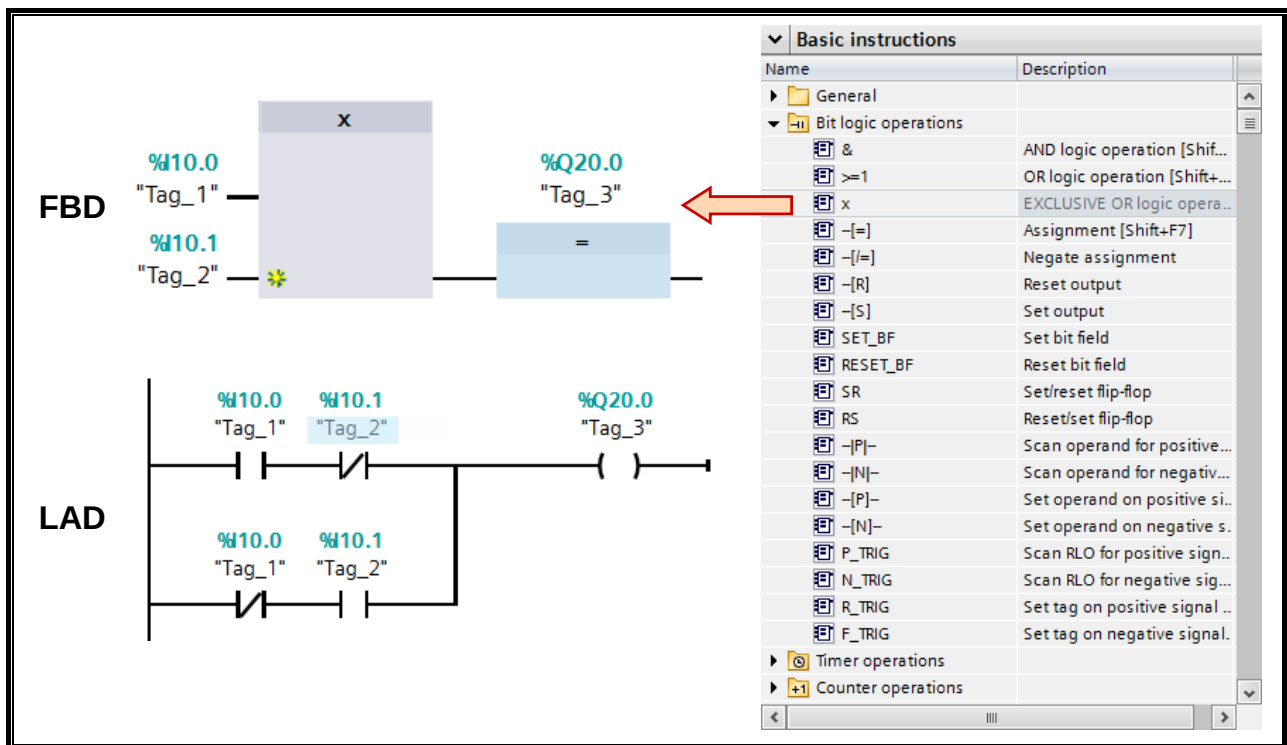
#### Task

Complete the programs above to obtain the following functionality: When switch S1 is activated and switch S2 is not activated, the light should be ON in all three cases.

#### Note

The terms "NO contact" and "NC contact" have different meanings depending on whether they are used in the process-hardware-context or as check symbols in the software.

### 7.2.4. Binary Logic Operations: Exclusive OR (XOR)



#### XOR Logic Operation

With the XOR logic operation, basically all binary operands can be queried, even outputs. Instead of individual operands, the results of other logic operations can also be further logically linked. Also, the logic operations can also be combined.

All inputs of the logic operations can be programmed as check for signal state or Status '0' and '1', regardless of whether a hardware NO contact or NC contact is connected in the process.

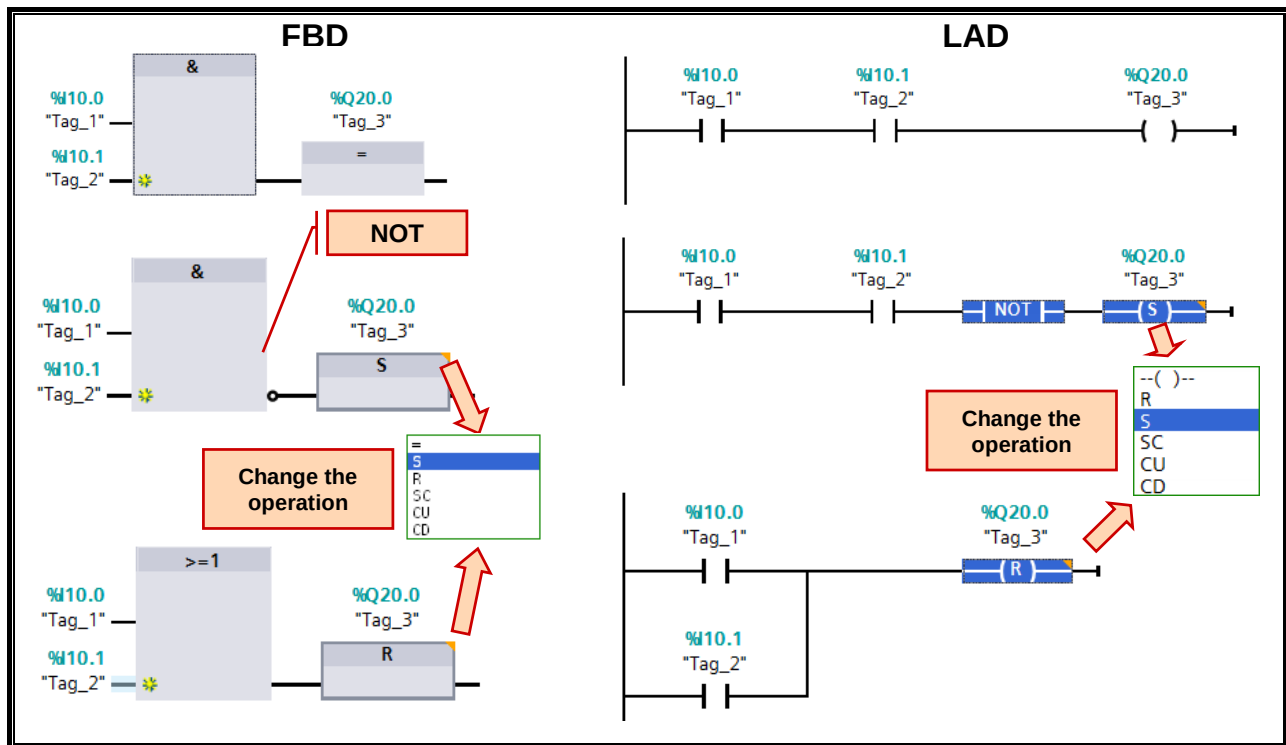
- For an XOR logic operation with 2 inputs, the result of logic operation (RLO) = '1', when one and only one of the two input signals has Status '1'.
- For an XOR logic operation with more than 2 operands, the RLO ...
  - = '1', when an uneven number of checked operands has Status '1'
  - = '0', when an even number of checked operands has Status '1'.

#### XOR in the Programming Languages FBD and LAD

In the LAD programming language, there is no explicit XOR logic operation. It must be generated by programming the discrete instructions shown in the picture above.



### 7.3. Assignment, Set, Reset, NOT



#### Assignment

With an assignment, the specified operand is always assigned the current RLO as status. The assigned RLO remains available after the assignment and can be assigned to a further operand or it can be further logically linked.

#### Set

If RLO = "1", the specified operand is assigned Status '1'; if RLO = "0", the status of the operand remains unchanged.

#### Reset

If RLO = "1", the specified operand is assigned Status '0'; if RLO = "0", the status of the operand remains unchanged.

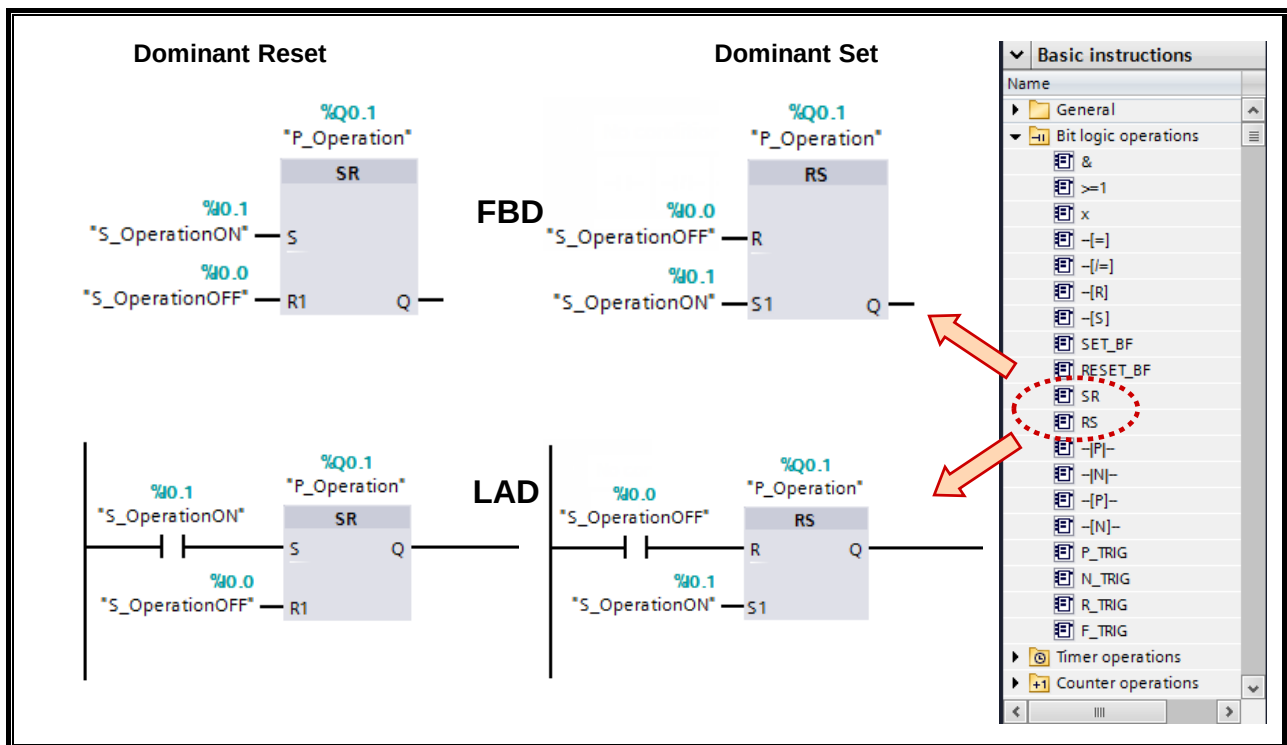
#### NOT

The NOT instruction inverts the result of logic operation (RLO).

If, in the example shown, the RLO of the AND logic operation = '1', the NOT instruction inverts it to RLO '0' and the Set instruction is not executed (the status of "Tag\_3" (Q20.0) then remains unchanged).

If the RLO of the AND logic operation = '0', the NOT instruction inverts it to RLO '1' and the Set instruction is executed ("Tag\_3" (Q20.0) is assigned Status '1').

### 7.3.1. Flip Flops



#### Memory Function "Flip Flop"

A flip flop has a Set input and a Reset input. The operand is set or reset, depending on which input has an RLO = "1".

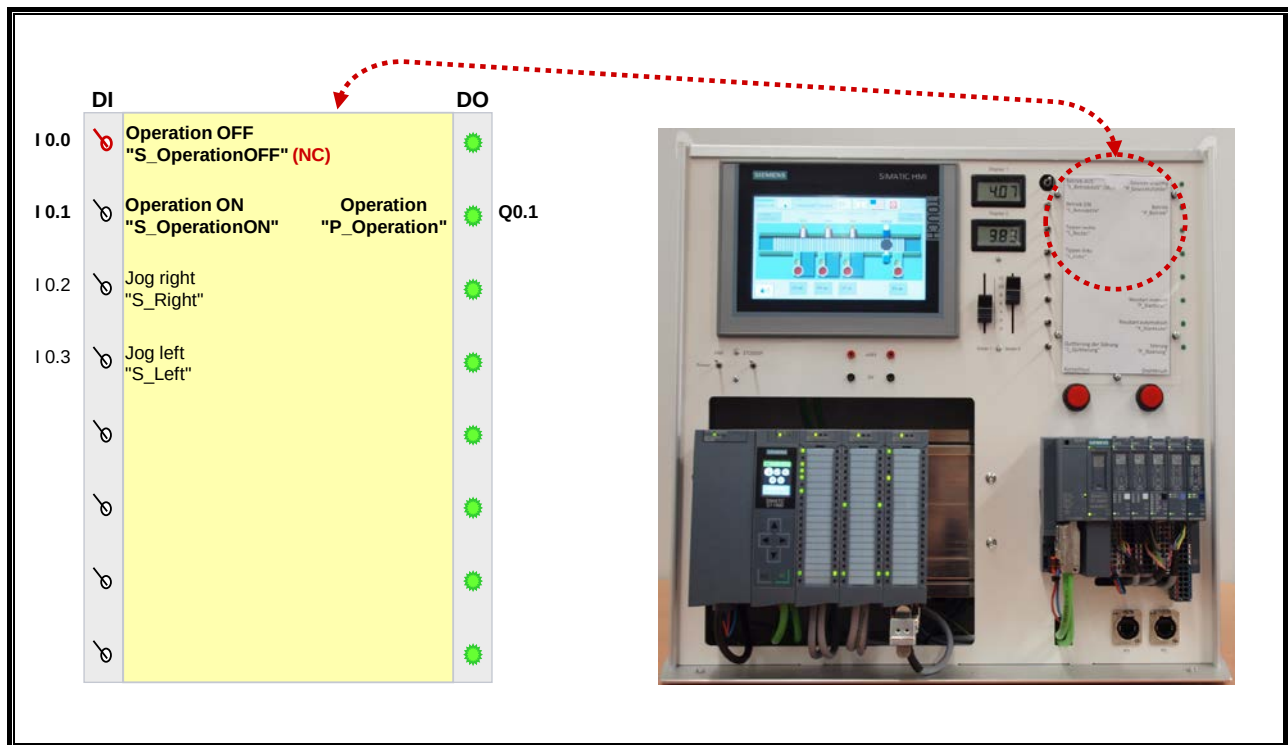
#### Priority (Dominancy)

If there is a signal state or RLO = "1" at both inputs at the same time, the priority of the operation decides whether the operand is set or reset. For that reason, there are different symbols for the Dominant Set and Dominant Reset memory functions in LAD and FBD.

#### Note

With a CPU restart, all outputs are reset. That is, they are overwritten with Status '0'.

## 7.4. Task Description: "FC\_Mode" (FC15)

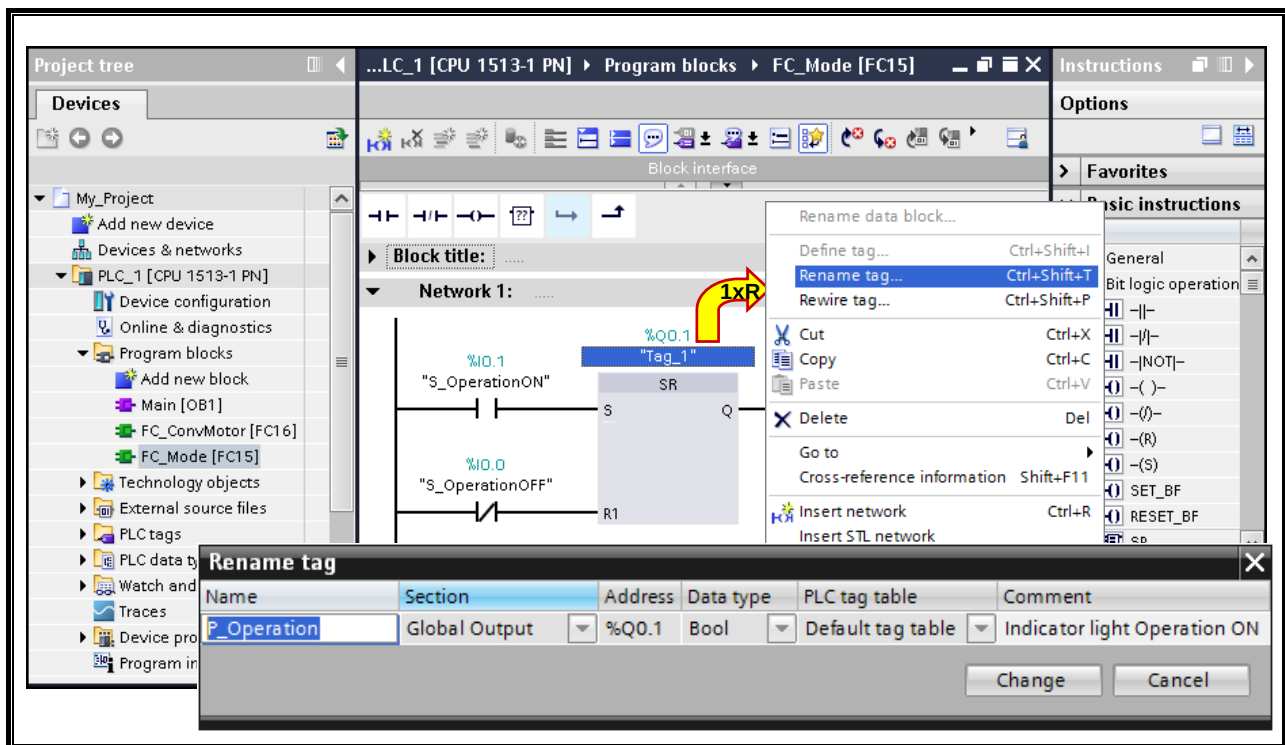


### Task Description

A mode section for the system is to be programmed in "FC\_Mode" (FC15), which controls the indicator light "P\_Operation" (Q0.1). This is then to be linked in "FC\_ConvMotor" (FC16) as a further condition for jogging the conveyor motor.

- Function of the mode section in "FC\_Mode" (FC15):  
The system or "P\_Operation" (Q0.1) is switched on via the simulator pushbutton "S\_OperationON" (I 0.1) and switched off via the simulator pushbutton "S\_OperationOFF" (I 0.0, NC).
- Integrate "P\_Operation" (Q0.1) in "FC\_ConvMotor" (FC16):  
The "Jogging the conveyor motor" programmed in FC 16 is now only to be possible when "P\_Operation" (Q0.1) is switched off.

### 7.4.1. Exercise 2: Programming the "FC\_Mode" (FC15) Block



#### Task

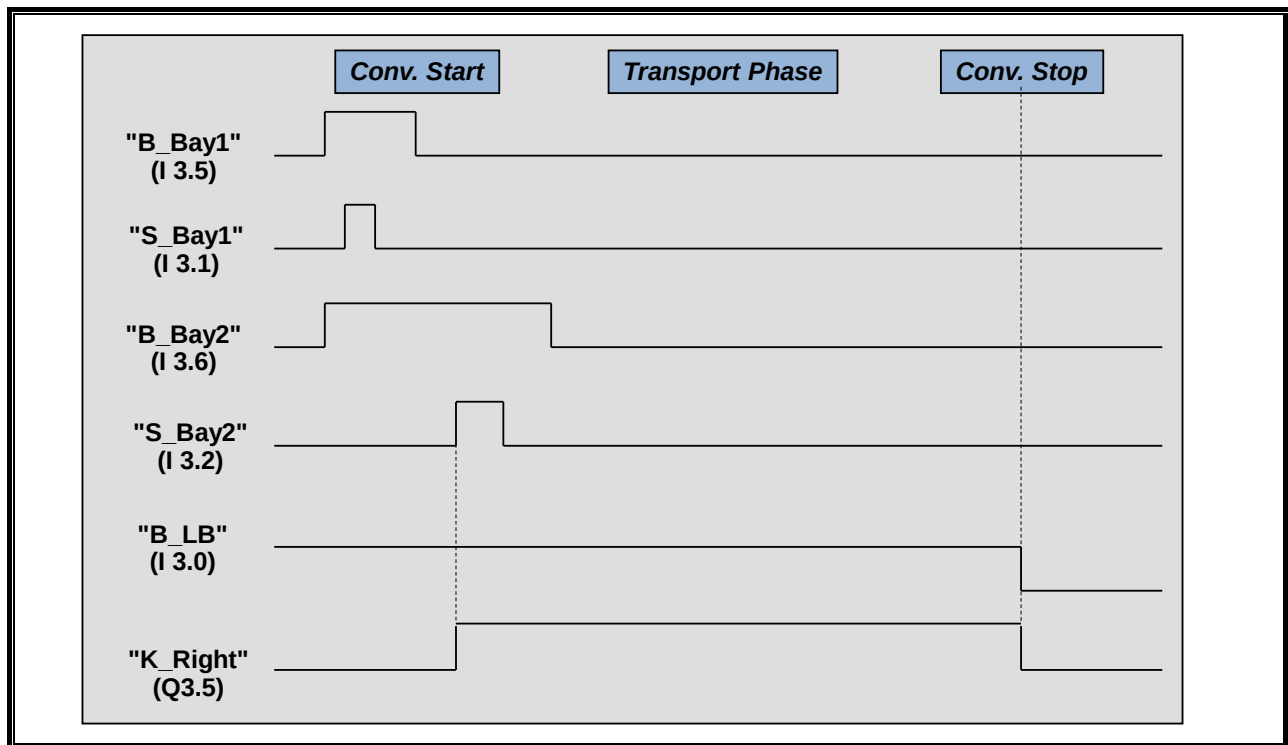
In "FC\_Mode" (FC15) which is now to be created, program a mode section for the distribution conveyor:

The operation or the indicator light "P\_Operation" (Simulator LED Q0.1) is switched on via the simulator pushbutton "S\_OperationON" (I 0.1) and is switched off via the simulator pushbutton "S\_OperationOFF" (I 0.0, NC).

#### What to Do

1. Create the new block "FC\_Mode" (FC15).
2. Open the "Instructions" Task Card, program the instructions shown in the picture using drag & drop and use the absolute addresses shown.
3. Rename the tags used as shown in the picture.
4. In "FC\_ConvMotor" (FC16), link the simulator LED "P\_Operation" (Q0.1) logically in such a way that the conveyor motor can only be jogged when this is switched off.
5. Download the modified blocks into the CPU and check the program function.

#### 7.4.2. Task Description: Parts Transportation when "P\_Operation" (Q0.1) is Switched On

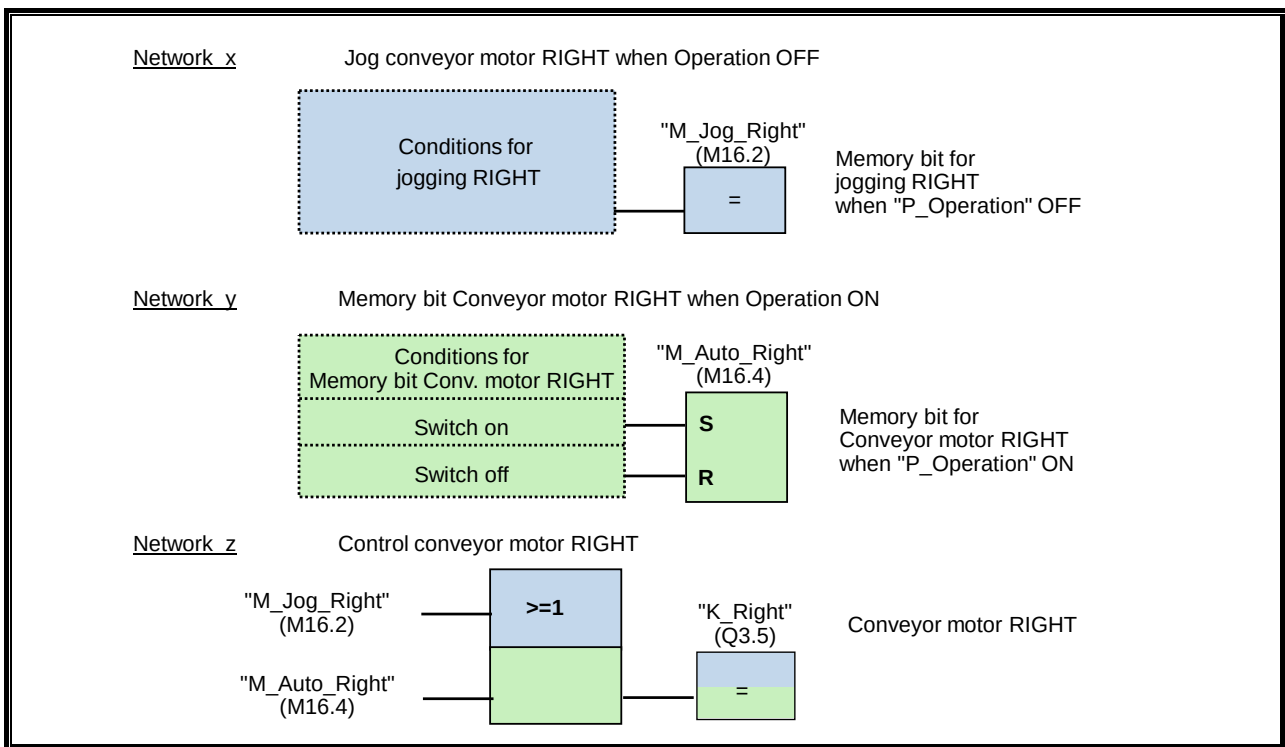


##### Task Description:

When "P\_Operation" (Q0.1) is switched ON, parts are to be transported from Bay 1 or Bay 2 to the light barrier "B\_LB" (I 3.0). The precondition is that a part is only placed on the conveyor at one of the two bays. If parts are placed on the conveyor at both bays, no transport sequence can be started.

- The conveyor motor "K\_Right" (Q3.5) is started when
  - on Bay 1 the proximity sensor "B\_Bay1" (I 3.5) is occupied AND on Bay 2 the proximity sensor "B\_Bay2" (I 3.6) is NOT occupied AND the pushbutton at Bay 1 "S\_Bay1" (I 3.1) is pressed
  - OR
  - on Bay 2 the proximity sensor "B\_Bay2" (I 3.6) is occupied AND on Bay 1 the proximity sensor "B\_Bay1" (I 3.5) is NOT occupied AND the pushbutton at Bay 2 "S\_Bay2" (I 3.2) is pressed.
- The conveyor motor "K\_Right" (Q3.5) is stopped when
  - the part has reached the light barrier "B\_LB" (I 3.0)
  - OR
  - "P\_Operation" (Q0.1) is switched off.

### 7.4.3. Exercise 3: Expanding "FC\_ConvMotor" (FC16)



#### Task

Expand the "FC\_ConvMotor" (FC16) block with the previously described functions.

#### Solution Notes

The conveyor motor "K\_Right" (Q3.5) must now be controlled under two conditions:

When "P\_Operation" (Q0.1) is switched off while jogging RIGHT (in the picture Network x) OR when "P\_Operation" (Q0.1) is switched on under the conditions described in the task (in the picture Network y).

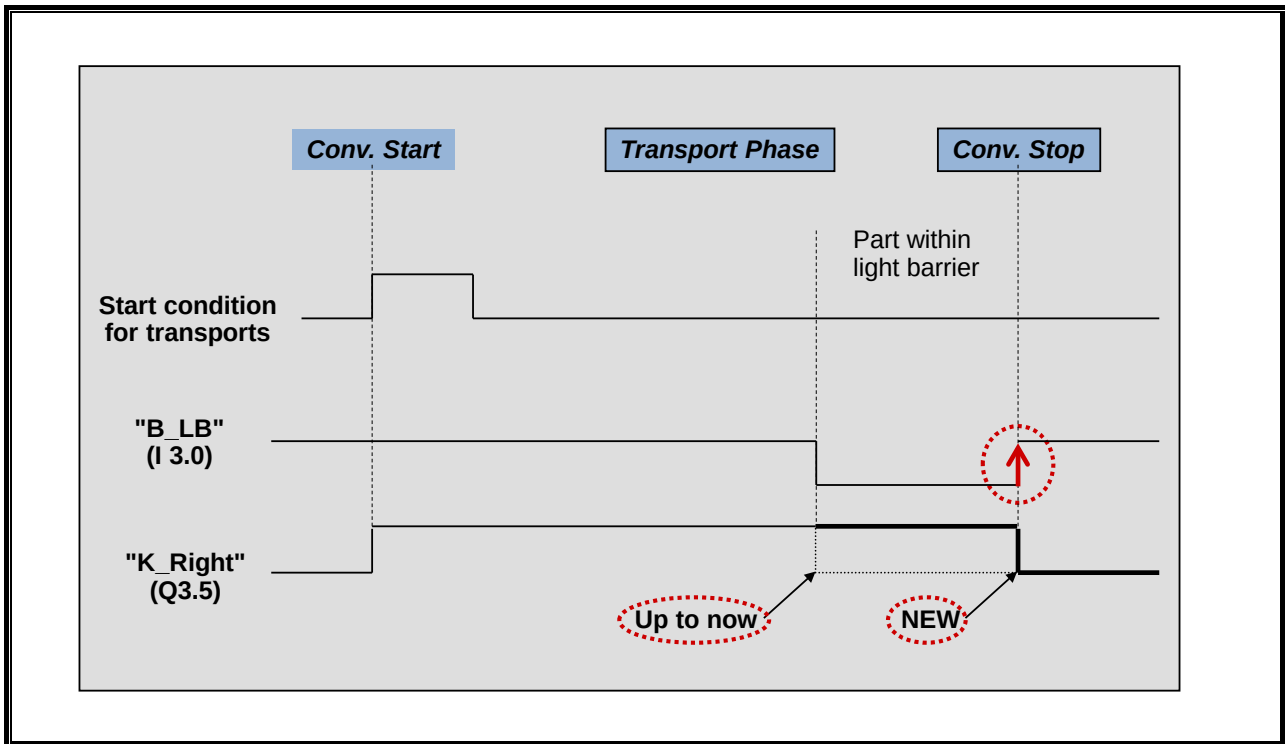
If, in both Network x and y, the result of logic operation of the conditions were each assigned to the output "K\_Right" (Q 3.5), an error in the form of a double assignment would occur. Jogging the motor RIGHT in manual mode (Network x) would no longer function, since the state assigned to the output here would then be overwritten in Network y.

The problem can be solved by programming a memory bit for each condition or by first assigning the results of the logic operations to a memory bit in both Network x and y. These are then used in the Network to control the conveyor motor.

#### What to Do

1. Open the "FC\_ConvMotor" (FC16) block.
2. Program the required functions in new networks (see picture). Use the memory bits shown in the picture and provide them with the symbols shown.
3. Modify the already existing network "Jog Right" as described in the solution notes.
4. Download the expanded block into the CPU and check the program function.

## 7.5. Task Description: Parts Transportation THROUGH the Light Barrier



### Function Up to Now in FC\_ConvMotor

When "P\_Operation" (Q0.1 = '0') is switched off, you can jog the conveyor motor "K\_Right" (Q3.5) using the simulator pushbuttons "S\_Right" (I 0.2) and "S\_Left" (I 0.3).

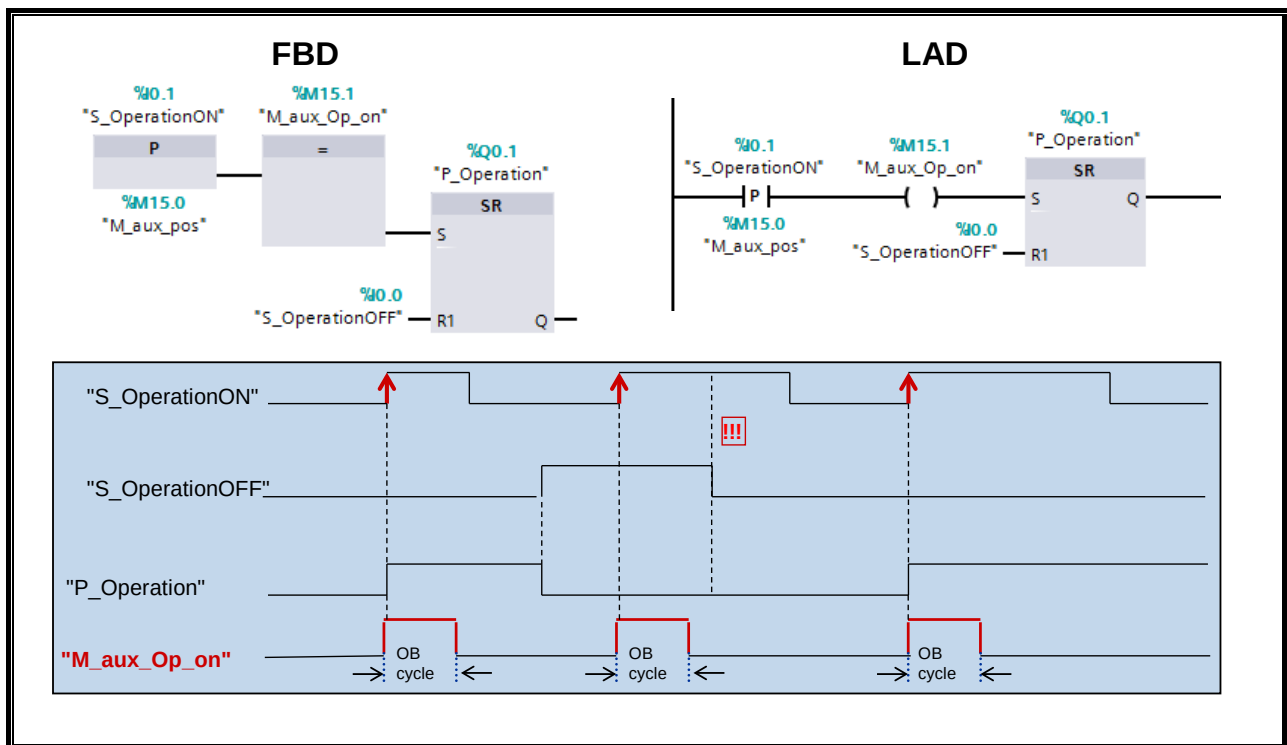
When "P\_Operation" (Q0.1) is switched on, the conveyor motor "K\_Right" (Q 3.5) is switched on when a part is placed on the conveyor exactly in front of one of the proximity sensors of Bay 1 or 2, and the occupied bay's pushbutton is pressed.

The conveyor motor "K\_Right" (Q3.5) is stopped when the part has **reached** the light barrier or "P\_Operation" (Q0.1) is switched off.

### Task Description:

The function of the "FC\_ConvMotor" (FC16) to control the conveyor motor when "P\_Operation" (Q0.1) is switched on is to remain fundamentally unchanged. However, the conveyor motor is only to stop when the part has **passed through** the light barrier.

### 7.5.1. Signal - Edge Evaluation



#### Signal Edge Evaluation (P, N)

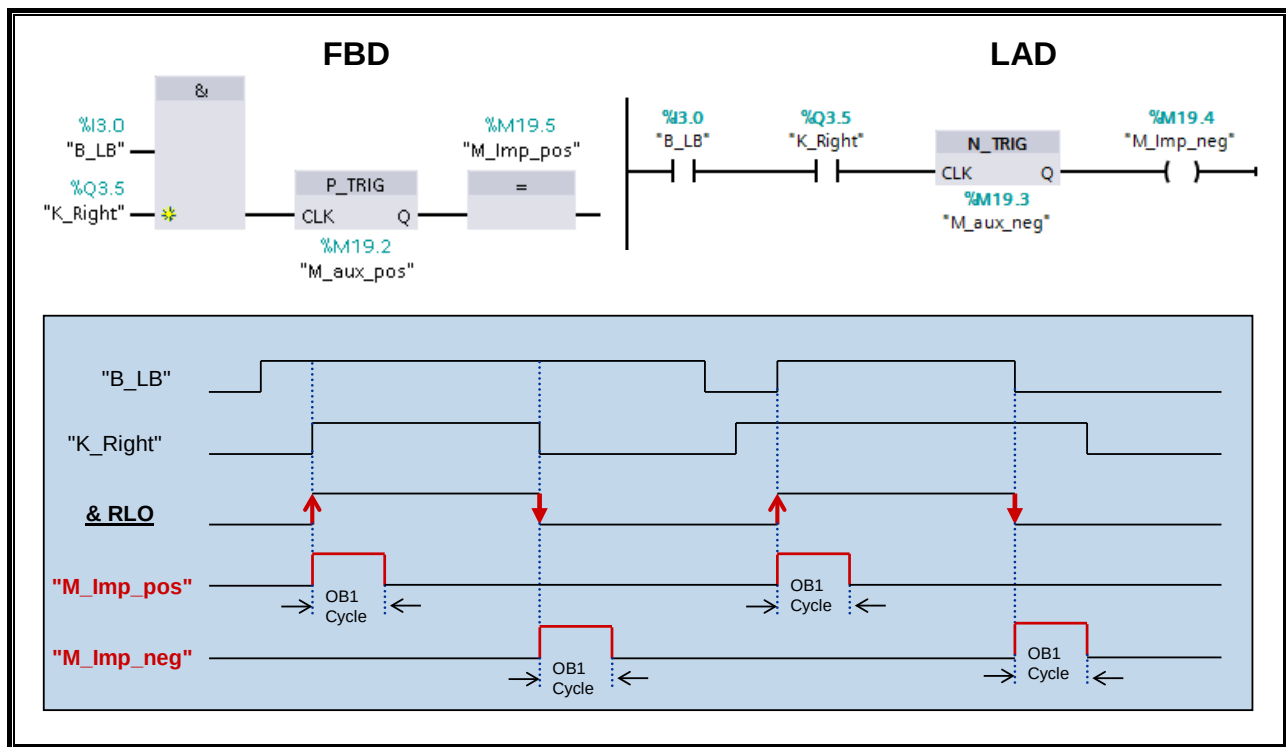
With a signal edge evaluation it is possible to detect the status change of an individual operand (in the example `"S_OperationON"`) from '0' to '1' (rising or positive edge) or from '1' to '0' (falling or negative edge). If this is the case, the instruction supplies RLO '1' as the result, which can be further logically linked (in the example as set condition) or assigned to another operand (for example, a bit memory) as status. In the following cycle, the instruction then once again supplies '0' as the result even if `"S_OperationON"` still has Status '1'.

The instruction compares the current status of the operand `"S_OperationON"` with its status in the previous program cycle. This status is stored in a so-called edge memory bit for this (in the example `"M_aux_Pos"`). It must be ensured that the status of this edge memory bit is not overwritten at another location in the program. For each edge evaluation, a separate edge memory bit must be used accordingly, even then when the same operand (in the example, `"S_OperationON"`) is evaluated once again, for example, in another block!

As well, the edge is assigned to a pulse memory bit (`"M_aux_OP_on"`) in the example. This has the advantage that the edge of the operand is only queried once in the program. If, in the further course of the program, the edge of the operand must once again be evaluated, no further edge memory bit is required but rather the state of the pulse memory bit can be queried.



## 7.5.2. RLO - Edge Evaluation



### RLO Edge Evaluation (N\_TRIG, P\_TRIG)

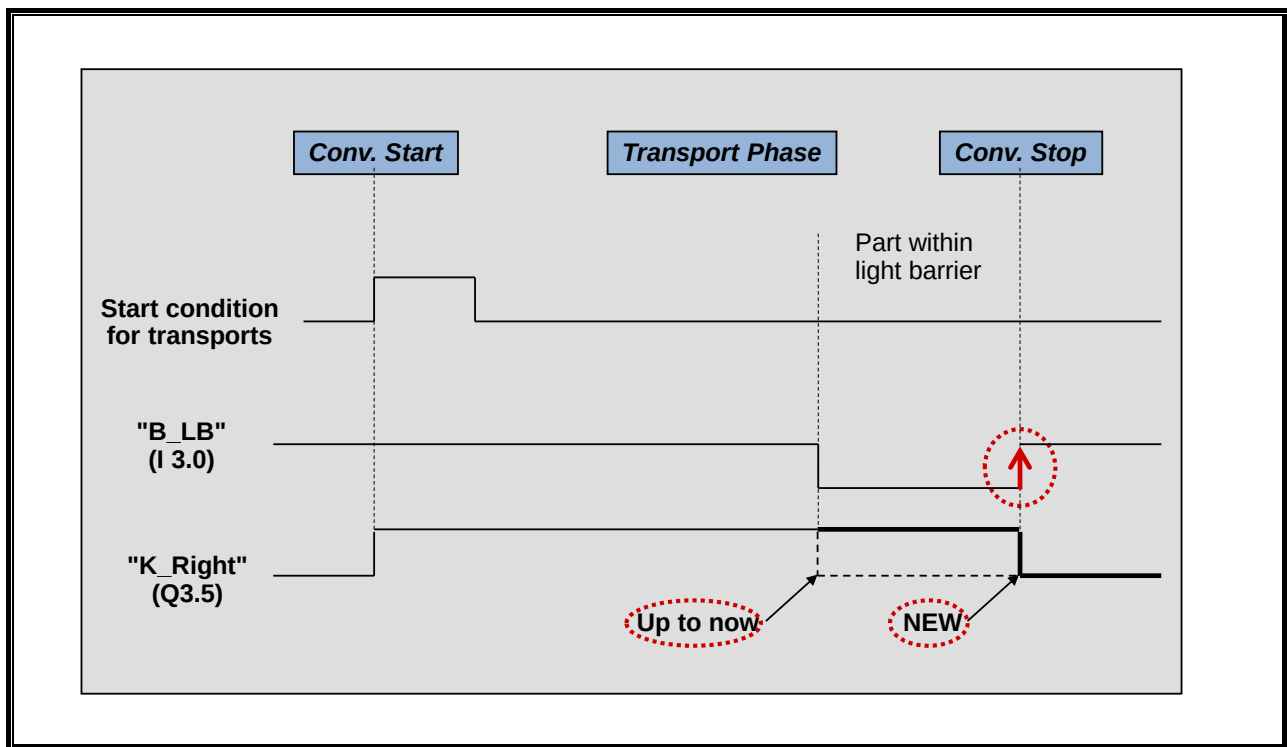
With an RLO edge evaluation it is possible to detect the status change of an individual operand or the RLO of a logic operation from '0' to '1' (rising or positive edge) or from '1' to '0' (falling or negative edge). If this is the case, both edge evaluations supply RLO '1' as a result to their output for the duration of one cycle. In the following cycle, the instructions then once again supply RLO '0' as a result, even if the status or the RLO of the operand or the logic operation has not changed.

The instructions compare the current status of the operand or the RLO of the logic operation with its status in the previous program cycle which is stored in a so-called edge memory bit for this (in the example, `"M_aux_pos"` or `"M_aux_neg"`). It must be ensured that the status of this edge memory bit is not overwritten at another location in the program. For every edge evaluation, a separate edge memory bit must be used accordingly, even then when the same operand is evaluated once again, for example, in another block!

On the examples in the picture:

- With the instructions `P_TRIG` and `N_TRIG`, the result or the RLO of the edge evaluation is available on the right at the output. It can be further logically linked or assigned to an operand (in the example, it is assigned to the operand `"M_imp_pos"` or `"M_imp_neg"`).

### 7.5.3. Exercise 4: Integrating an Edge Evaluation in "FC\_ConvMotor" (FC16)



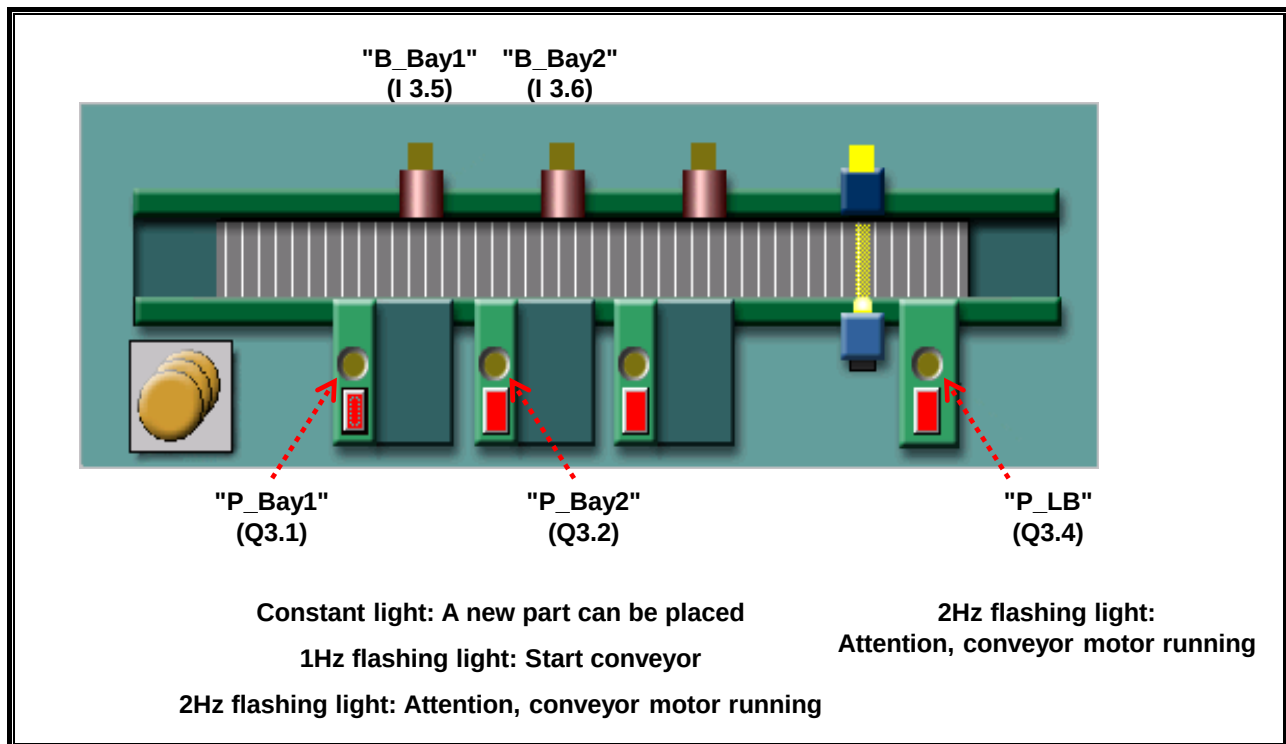
#### Task:

When "P\_Operation" (Q0.1 = '1') is switched on, the parts are to be transported from Bay 1 or 2 THROUGH the light barrier.

#### What to Do:

1. Program the necessary changes in "FC\_ConvMotor" (FC16), by now linking the result of the edge evaluation as the reset condition for M16.4 (bit memory for conveyor motor RIGHT) instead of the light barrier signal "B\_LB" (I 3.0) itself. For the necessary edge evaluation of the light barrier signal use the bit memory "M\_aux\_LB" (M16.0) as an edge memory bit.
2. Download the modified "FC\_ConvMotor (FC16) block into the CPU and check the program function.

## 7.6. Task Description: Controlling the Indicator Lights, Commissioning "FC\_Signal" (FC14)



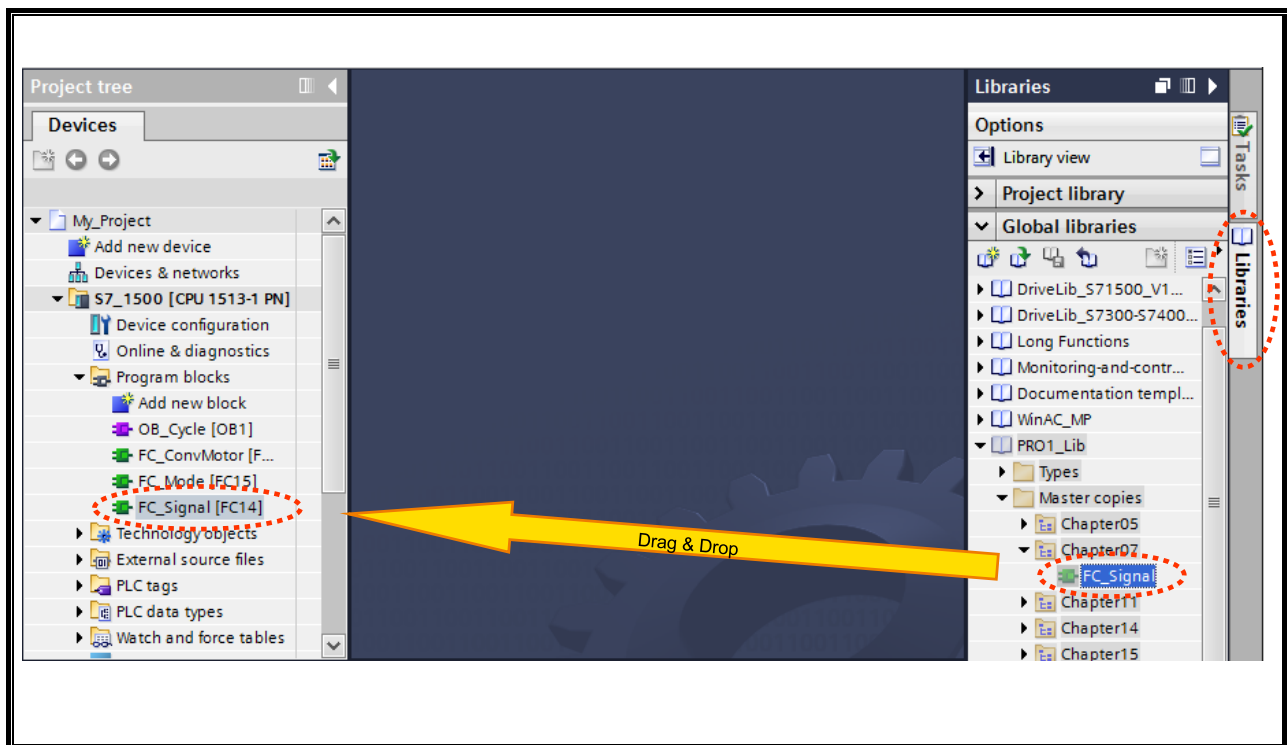
### Task Description

When "P\_Operation" (Q0.1) is switched on, the indicator lights "P\_Bay1" (Q3.1) and "P\_Bay2" (Q3.2) are to be controlled as follows:

- They show
  - a constant light when a new part can be placed on the conveyor (conveyor motor is stopped and both proximity switches are free)
  - 1Hz flashing light at the bay where the associated proximity switch detects a part, but only as long as the conveyor has not yet been started (if parts are placed on the conveyor at both proximity switches, neither indicator light must light up).
  - 2Hz flashing light as long as the conveyor motor is running
- The indicator light at the light barrier bay "P\_LB" (Q3.4) shows 2Hz flashing light as long as the conveyor motor is running.

The described functions are currently already programmed in the "FC\_Signal" (FC14) block, which is stored in the "PRO1\_Lib" global library. The block is to be commissioned and completed by you in the next exercise.

### 7.6.1. Exercise 5: Commissioning "FC\_Signal" (FC14)



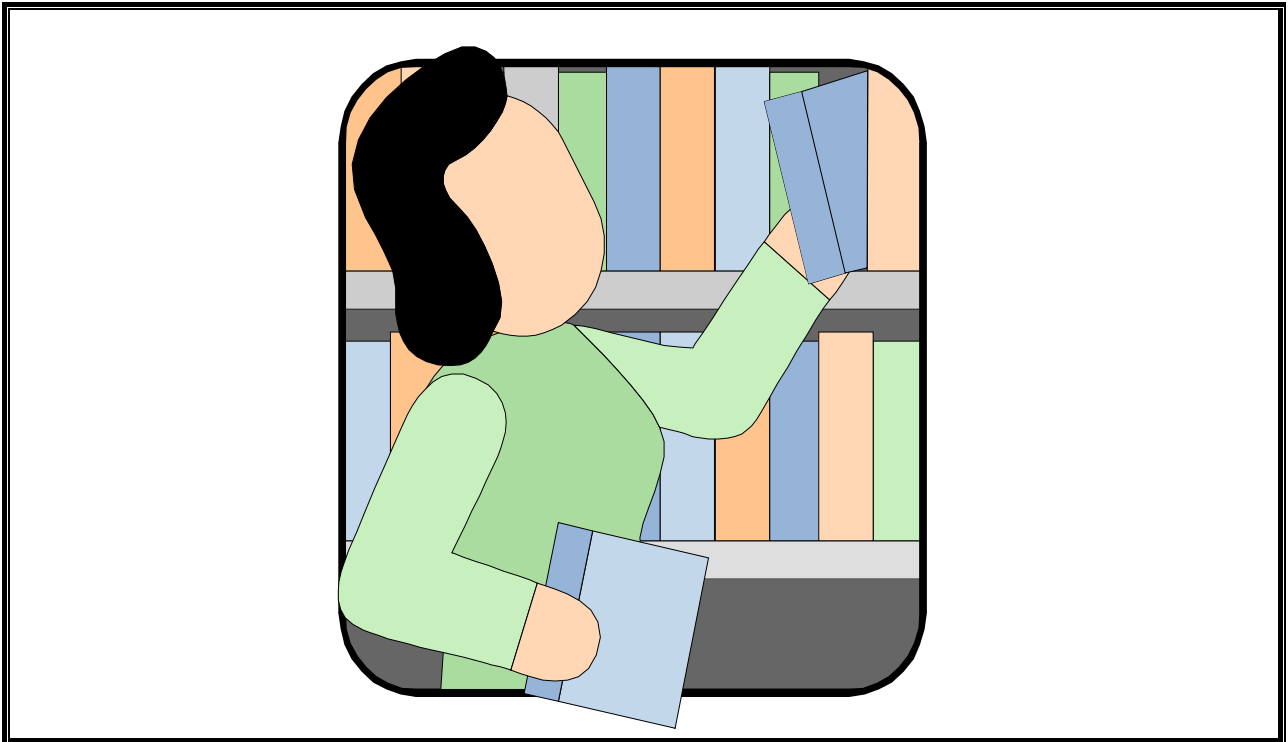
#### Task

The "FC\_Signal" (FC14) block is to be copied from the "PRO1\_Lib" global library into the project, then commissioned and completed.

#### What to Do

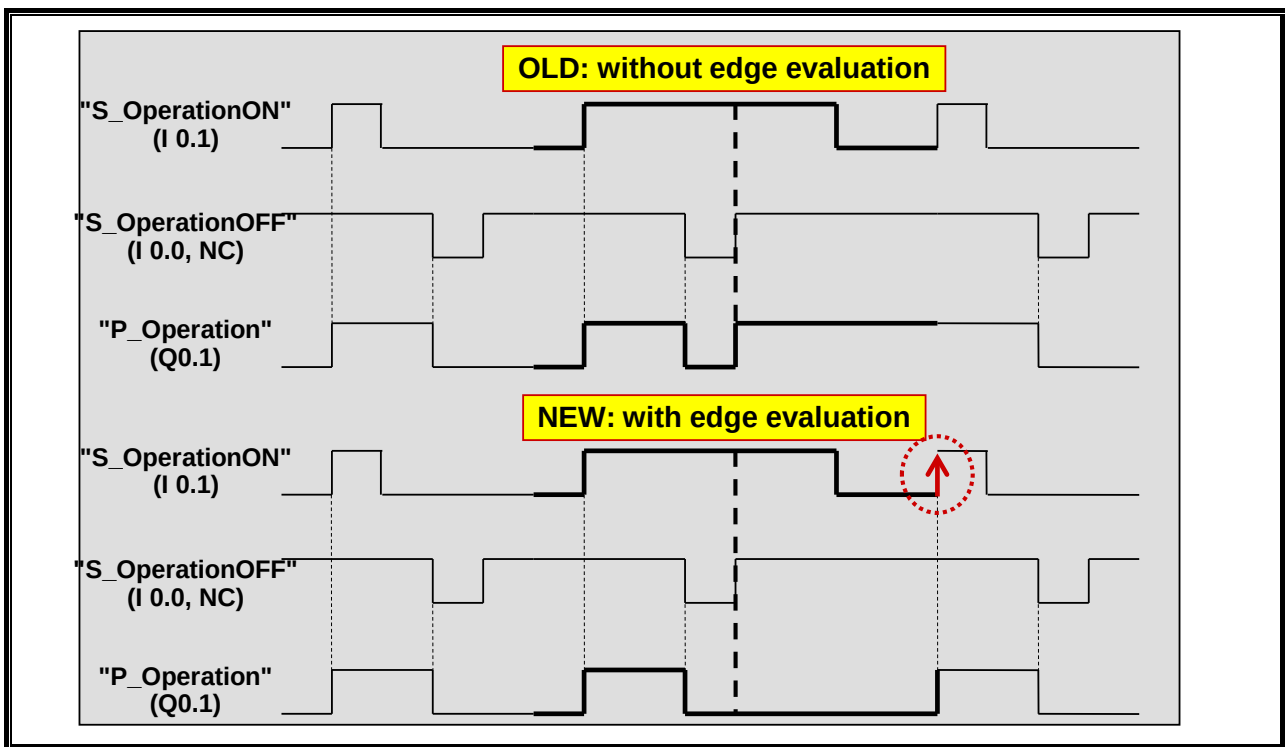
1. Using drag & drop, copy the "FC\_Signal" (FC14) block into the "Program blocks" container from the "PRO1\_Lib" global library (as shown in the picture).
2. Complete the block so that the indicator lights "P\_Bay1" (Q3.1) and "P\_Bay2" (Q3.2) show the required signals.
3. Program the call of "FC\_Signal" (FC14) in "OB\_Cycle" (OB1).
4. Download all modified blocks into the CPU and test the program function.
5. Save your project.

## 7.7. Additional Information



**Note** The following pages contain either further information or are for reference to complete a topic.  
For more in-depth study we offer advanced courses and self-learning mediums.

### 7.7.1. Additional Exercise 6: Optimizing "FC\_Mode" (FC 15)



#### Function Up to Now of "FC\_Mode" (FC15)

"P\_Operation" (Q0.1) is switched on with the simulator pushbutton "S\_OperationON" (I 0.1) and switched off with the simulator pushbutton "S\_OperationOFF" (I 0.0, **NC**). If you press both pushbuttons simultaneously, the operation remains switched on or is switched off if currently on. If, however, both pushbuttons are pressed and while pressing the ON pushbutton, the OFF pushbutton is let go, the operation switches back on without the ON pushbutton having to be pressed again (see picture, upper function diagram "OLD: without edge evaluation").

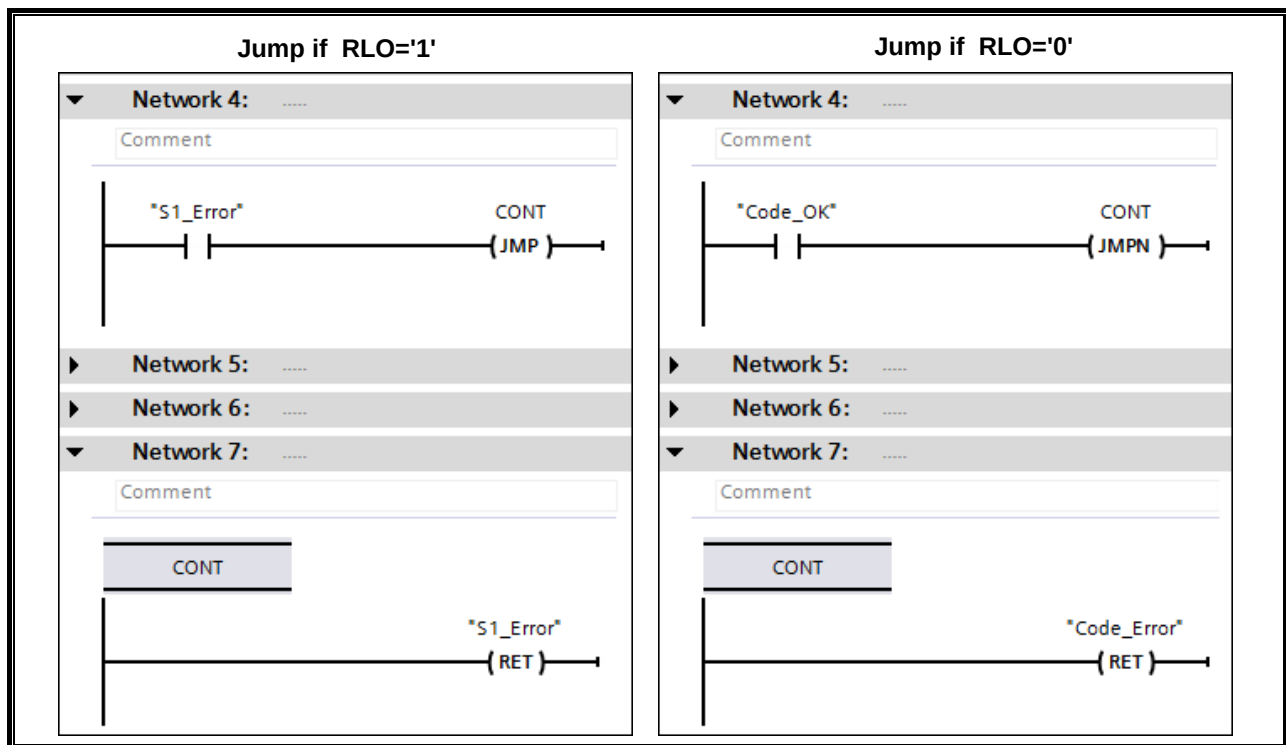
#### Task:

Expand the functionality of "FC\_Mode" (FC15) using edge evaluation so that the ON pushbutton must be pressed every time the operation is switched on (see picture, lower function diagram "NEW: with edge evaluation"). The criteria for switching on the system is no longer to be the pressed ON pushbutton or its "1" signal, but the function of pressing or the "positive edge" of the ON pushbutton signal.

#### What to Do:

1. In the set condition for "P\_Operation" (Q0.1), insert an edge evaluation of the pushbutton "S\_OperationON" (I 0.1). For the edge evaluation, use the bit memory "M\_aux\_Op\_on" (M15.1) as edge memory bit.
2. Download the modified "FC\_Mode" (FC15) block into the CPU and check whether it fulfils the desired functions!

### 7.7.2. Jump Instructions JMP, JMPN, RET



#### Jump Instructions JMP and JMPN

With the jump instructions JMP and JMPN, the linear execution of the program can be interrupted within a block and continued in another network. With the jump instruction, a Label is specified which also identifies the target network. The specified label must be located in the same block and be unique. Each label can be jumped to from several locations. The jump can take place in networks with higher (forwards) or lower numbers (backwards).

- **JMP:**  
If RLO = '1', the jump into the target network is executed; if RLO = '0', the jump is not executed and the linear program execution continues.
- **JMPN:**  
If RLO = '0', the jump into the target network is executed; if RLO = '1', the jump is not executed and the linear program execution continues.

#### End Block Execution RET

With the instruction RET the program execution of the entire block is ended. The program execution is then continued in the calling block with the instruction that follows the call of this block.

# Contents

# 8

|           |                                                                                                   |            |
|-----------|---------------------------------------------------------------------------------------------------|------------|
| <b>8.</b> | <b>Digital Operations .....</b>                                                                   | <b>8-2</b> |
| 8.1.      | Acquiring, Processing and Outputting Data.....                                                    | 8-3        |
| 8.2.      | Task Description: Counting the Transported Parts.....                                             | 8-4        |
| 8.3.      | Integer (INT, 16-Bit Integer) Data Type .....                                                     | 8-5        |
| 8.3.1.    | Double Integer (DINT, 32-Bit Integer) Data Type .....                                             | 8-6        |
| 8.3.2.    | REAL (Floating-point Number, 32 Bit) Data Type.....                                               | 8-7        |
| 8.3.3.    | Data Types and Display Formats.....                                                               | 8-8        |
| 8.4.      | Counters: CTU, CTD, CTUD.....                                                                     | 8-9        |
| 8.4.1.    | Timer / Counter Instance Data Blocks .....                                                        | 8-10       |
| 8.4.2.    | Counter Function: Inputs.....                                                                     | 8-11       |
| 8.4.3.    | Counter Function: Outputs.....                                                                    | 8-12       |
| 8.4.4.    | Exercise 1: Counting the Transported Parts Programming "FC_Counting" (FC18).....                  | 8-13       |
| 8.5.      | Task Description: Timed Monitoring of the Transport Sequences.....                                | 8-14       |
| 8.5.1.    | Timer Function TON .....                                                                          | 8-15       |
| 8.5.2.    | Timer Function TON (ON Delay) Pulse Diagram.....                                                  | 8-16       |
| 8.5.3.    | Exercise 2: Timed Monitoring of the Transports Programming "FC_Fault" (FC17) .....                | 8-17       |
| 8.6.      | Task Description: Counting the Transported Parts using Addition in "FC_Counting_Add" (FC19) ..... | 8-18       |
| 8.6.1.    | Basic Mathematical Functions: Addition .....                                                      | 8-19       |
| 8.6.2.    | Basic Mathematical Functions: Comparison Operations.....                                          | 8-20       |
| 8.6.3.    | Value Assignment of a Variable.....                                                               | 8-21       |
| 8.6.4.    | Exercise 3: Counting the Transported Parts using Addition in "FC_Counting_Add" (FC19).....        | 8-22       |
| 8.7.      | Additional Information .....                                                                      | 8-23       |
| 8.7.1.    | Additional Exercise 4: Counting the Conveyor Faults Expanding "FC_Fault" (FC17).....              | 8-24       |
| 8.7.2.    | Additional Exercise 5: Timely Lock-out of the Conveyor Motor Jogging .....                        | 8-25       |
| 8.7.3.    | Digital Logic Operations.....                                                                     | 8-26       |
| 8.7.4.    | Application Example: Digital Edge Evaluation .....                                                | 8-27       |
| 8.7.5.    | Comparator Operations: IN_RANGE, OUT_RANGE .....                                                  | 8-28       |
| 8.7.6.    | Date and Time-of-day: RD_SYS_T .....                                                              | 8-29       |



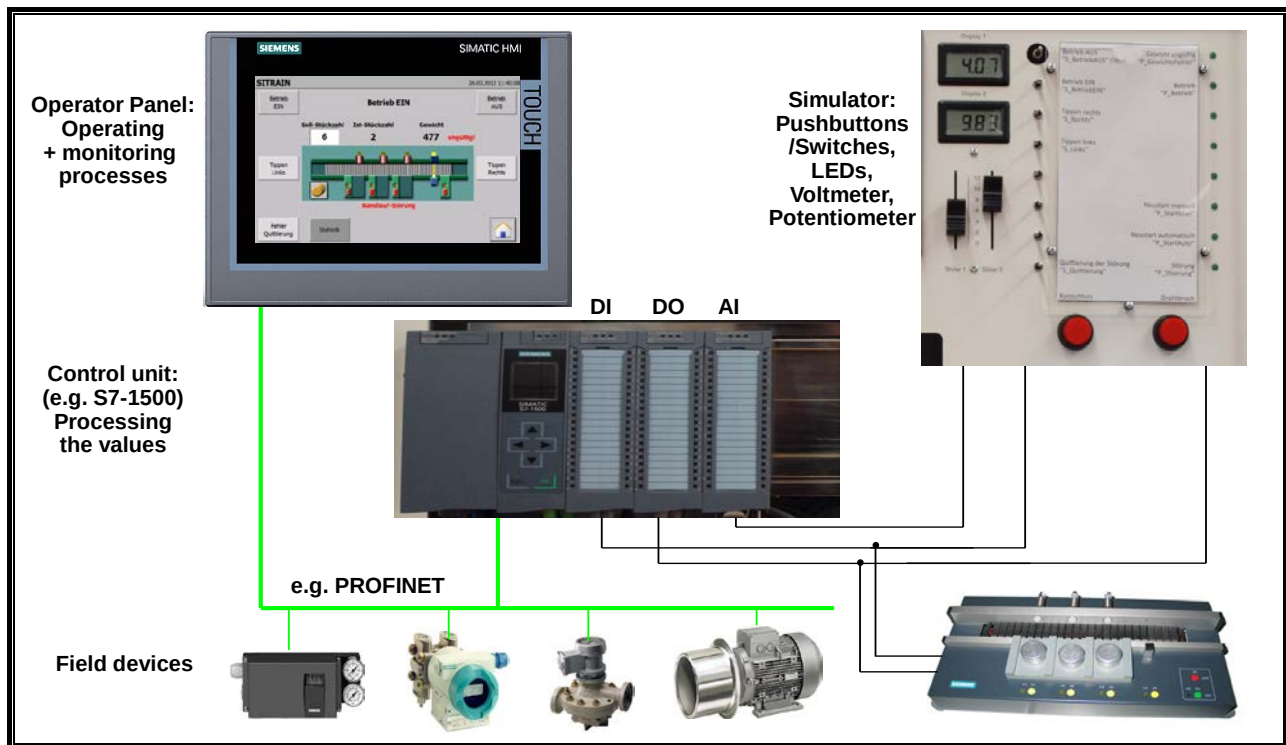
## 8. Digital Operations

At the end of the chapter the participant will...

- ... be familiar with the different data types
- ... be familiar with the different counter and timer functions
- ... be able to use and program counter and timer functions and be able to select a suitable data type
- ... be able to use basic mathematical operations



## 8.1. Acquiring, Processing and Outputting Data



### Binary/Digital Processing

True logic control systems are recognizable in the fact that they exclusively process binary data. The performance of today's control computer, as well as tasks in the areas of data processing, quality control, among others, has increased the importance of digital data processing using PLCs. Digital process variables can be found in all areas of open-loop control - such as in connected devices for process operating and monitoring or in the control of field devices.

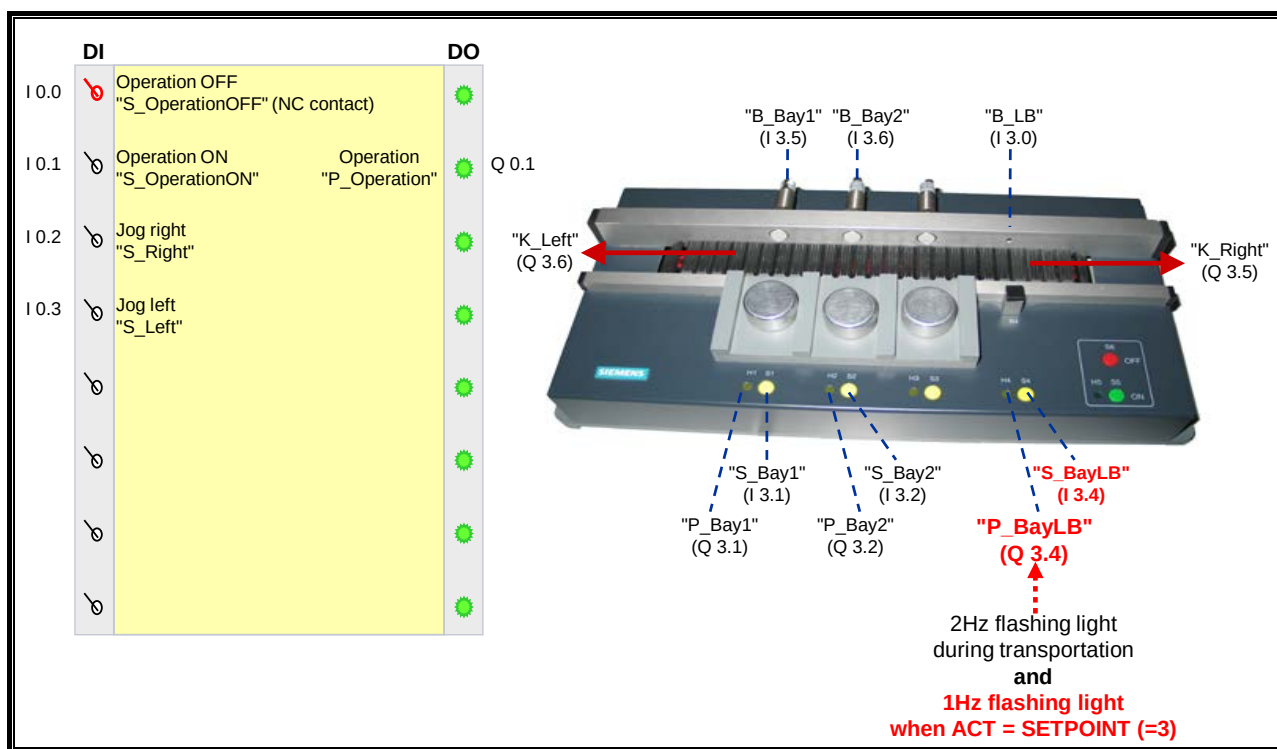
### Operating and Monitoring

The goal of process monitoring is to provide the operator with up-to-the-minute information about the working machine or system quickly, concisely and clearly as well as the opportunity to intervene and control and influence the process. While in the past mostly simple, that is, "dumb" input and output devices, such as 7-segment displays and thumbwheel buttons were used to display and enter digital values, today "intelligent" operating and monitoring devices are frequently connected to a PLC. Depending on the type of device connected, different number formats for the coding of data are used to transmit data between device and PLC, as well as for storing and processing data in the PLC.

### Field Devices

Today as well, field devices that acquire process data or that control the process are supplied directly with digital variables through field bus systems. The connection of field devices, such as drives or weighing systems, using analog input and output modules is becoming more and more a thing of the past.

## 8.2. Task Description: Counting the Transported Parts



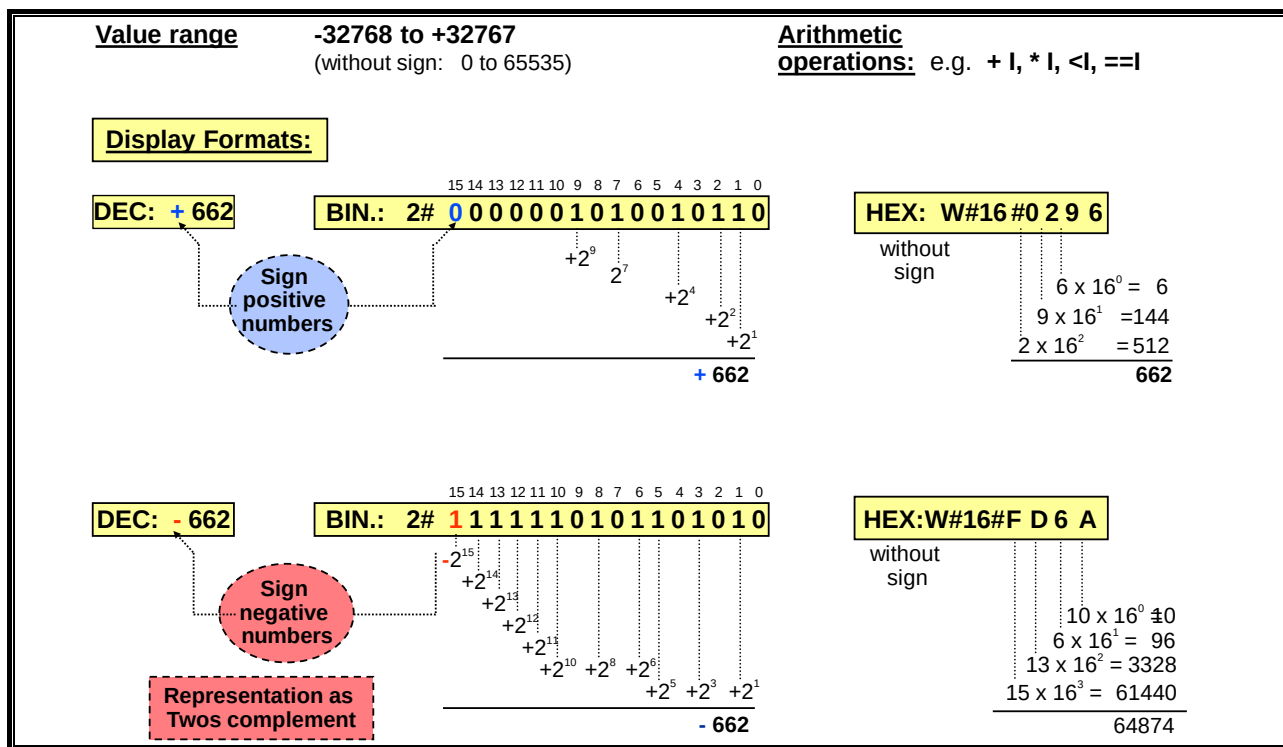
### Function Up Till Now

When "P\_Operation" (Q0.1) is switched on, parts are transported from Bay 1 or Bay 2 through the light barrier. The transport sequence starts as soon as a part is placed on Bay 1 or Bay 2 and the associated bay's pushbutton is pressed and it ends as soon as the part has passed the light barrier.

### Task

- When "P\_Operation" (Q0.1) is switched on, the transported parts are to be counted as soon as they have passed through the "B\_LB" (I 3.0) light barrier ("B\_LB" 0 → 1). The number of transported parts (ACTUAL quantity) is to be recorded with a counter and stored in the variable "MW\_ACT" (MW20).
- The counter is reset to 0 when ACT=SETPOINT is acknowledged via the pushbutton "S\_BayLB" (I 3.4) and when "P\_Operation" (Q0.1) is **switched on**.
- The indicator light "P\_BayLB" (Q3.4) shows a 1Hz flashing light when the actual quantity has reached the setpoint quantity of 3 (= a new part must not be placed on the conveyor -> lock-out in FC14) and no further transport sequence can be started (-> lock-out in FC16)), and 2Hz flashing light during parts transportation.

### 8.3. Integer (INT, 16-Bit Integer) Data Type



#### Integer (16-Bit) Data Type

An *Integer* data type value is a whole number value, that is, a value without a decimal point. SIMATIC S7 stores *Integer* data type values with sign in 16 bit code. This results in the value range shown in the picture above. As well, SIMATIC S7 provides arithmetic operations for processing Integer values.

#### Decimal

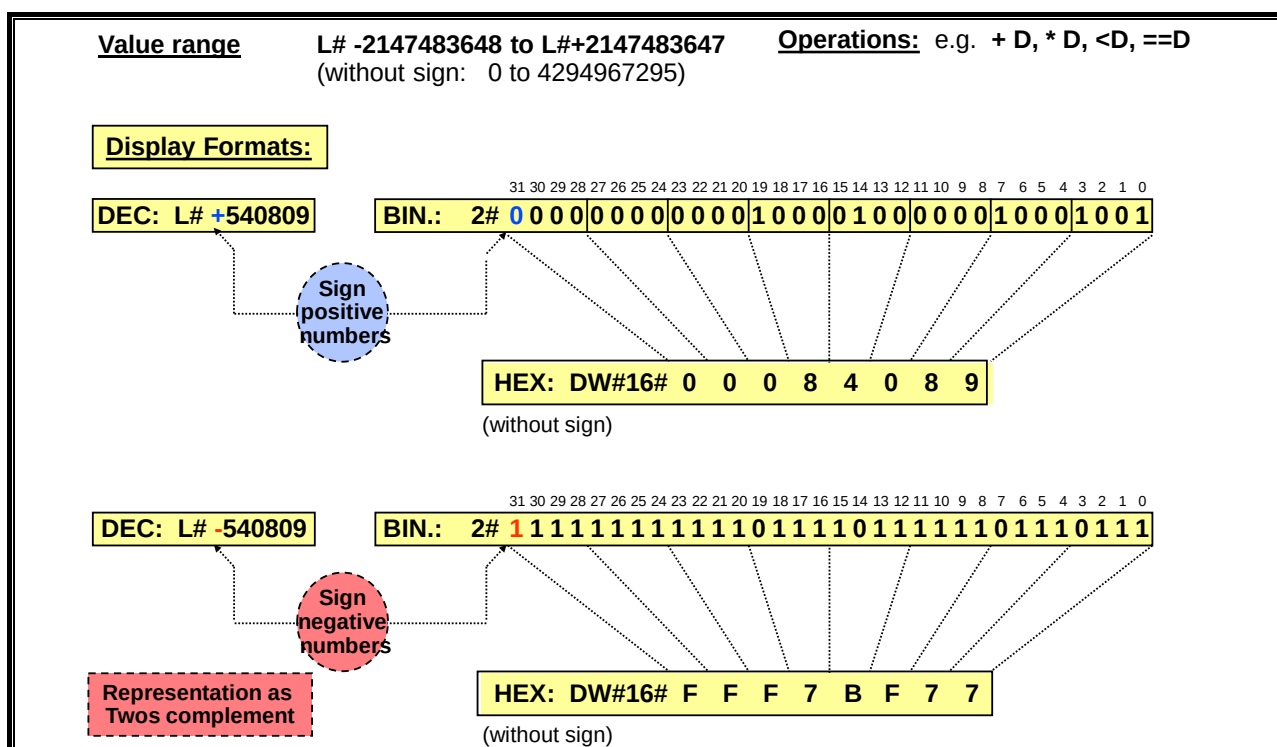
STEP7 uses the *Decimal* (not BCD!) display format to specify the constants of the *Integer* data type with sign and without explicit format description. The use of constant Integer values in the *Binary* and *Hexadecimal* display formats is possible in principle, but because of the poor legibility, they are more or less not suitable. For this reason, the syntax of STEP7 provides the specification of Integer values only in the decimal display format.

#### Binary

In a digital computer system, all values are stored in a binary-coded form. Only the digits 0 and 1 are available in the binary number system. Base 2 of this number system results from the number of available digits. Accordingly, the value of every position of a binary number results from a power of Base 2. This is also expressed in the format specification 2#....

Negative values are represented as binary numbers in twos complement. In this representation, the most significant bit (bit no. 15 for the Integer data type) has the value - 215. Since this value is greater than the sum of all residual values, this bit also has the sign information. That is, if this bit = 0, then the value is positive; if the bit is = 1, then the value is negative. The conversion of a binary number into a decimal number is made by adding the values of the positions that have a 1 (see picture).

### 8.3.1. Double Integer (DINT, 32-Bit Integer) Data Type



#### Double Integer (32-Bit Integer)

SIMATIC S7 stores *Double Integer* data type values with sign as 32 bit code. This results in the value range shown in the picture above. As well, SIMATIC S7 provides arithmetic operations for processing DINT values.

#### Decimal

STEP7 uses a decimal number (not BCD!) to specify a constant of the *Double Integer* data type with sign and the format **L#** for "long" (double word, 32 bit). When a value smaller than -32768 or greater than 32767 is specified, the format **L#** is automatically added. For negative numbers smaller than -32768, the user must specify the format as **L# -** (for example: L# -32769). This is imperative if the value is to be further processed arithmetically as a double integer since you would otherwise work with false values (value + sign)!

#### Hexadecimal

The hexadecimal number system provides 16 different digits (0 to 9 and A to F). This results in Base 16 of this numbers system. Accordingly, the value of every position of a hexadecimal number results from a power of Base 16.

Hexadecimal numbers are specified with the format **W#** for the dimension (W = word = 16 bit) or **DW#** (DW = double word = 32 bit) and **16#** for identifying the basic numbering system. The number of specifiable bits is variable from 1 to 8 whereby missing (unspecified) bits are filled with leading zero digits.

The digits A to F correspond to the decimal values 10 to 15. The value 15 is the last value that can be binary-coded - without sign - with 4 bits. Out of this correlation, the simple conversion of a binary number into a hexadecimal number and vice versa can be obtained. In this way, four binary bits each can easily make up one digit of a hexadecimal number.

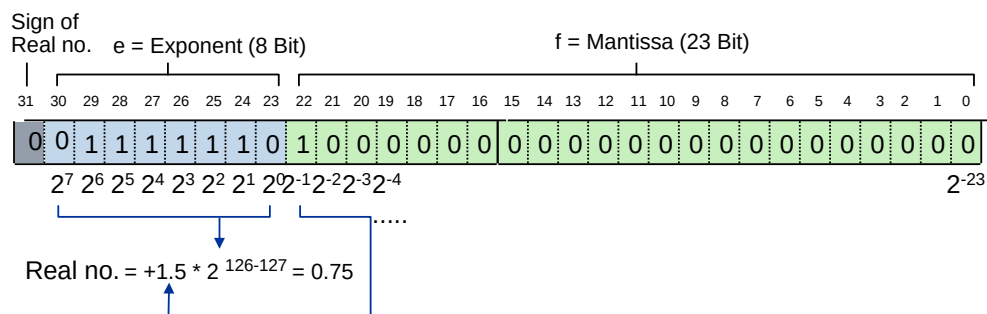
### 8.3.2. REAL (Floating-point Number, 32 Bit) Data Type

**Value range**  $-3.402823 \cdot 10^{+38}$  to  $-1.175495 \cdot 10^{-38}$ , 0.0,  $+1.175495 \cdot 10^{-38}$  to  $+3.402823 \cdot 10^{+38}$

**Operations:** e.g. + R, \* R, <R, ==R  
sin, acos, ln, exp, SQR

**General format of a Real number** = (Sign) • (1.f) • ( $2^{e-127}$ )

**Example:** 0.75



#### Real

The previously described INT and DINT data types are used to store whole number values with sign. Accordingly, only operations that supply a whole number value as the result can be performed with these values. In cases where analog process variables such as voltage, current, and temperature have to be processed, it becomes necessary to use *Real* values (real numbers, "decimal numbers"). In order to be able to represent such values, binary digits have to be defined whose value is less than 1 (power of base 2 with negative exponent).

#### Real Format

In order to be able to form the greatest possible value range within a defined memory capacity (for SIMATIC S7: double word, 32 bit) (see picture), you must be able to select the decimal point position as required. Early on, IEEE defined a format for floating-point numbers. This format was laid down in IEC 61131 and was included in STEP 7. This format makes it easy to process a variable decimal point position. In a binary coded floating-point number, a portion of the binary digits contain the mantissa (23 Bit) and the rest contain the exponent (8 Bit) and the sign of the floating-point number. When you specify real values, you do so without specifying the format. After you enter a constant real value (for example: 0.75), the Editor automatically makes a conversion to scientific notation (for example: 7.5000e-001).

#### Application

Floating-point numbers are used for "analog value processing", among other things. A great advantage of floating-point numbers is in the number of operations possible with such numbers. These include, in addition to the standard operations such as: +, -, \*, / also instructions such as sin, cos, exp, ln, etc, that are used mainly in closed-loop control algorithms.

### 8.3.3. Data Types and Display Formats

|    | i | Name                                                          | Address | Display format       | Monitor value                            | Modify value |                                     | Comment |
|----|---|---------------------------------------------------------------|---------|----------------------|------------------------------------------|--------------|-------------------------------------|---------|
| 1  |   |                                                               | %IW2    | DEC                  | 1                                        |              | <input type="checkbox"/>            |         |
| 2  |   |                                                               | %IW2    | Bin                  | 2#0000_0000_0000_0001                    |              | <input type="checkbox"/>            |         |
| 3  |   | // BOOL                                                       |         |                      |                                          |              |                                     |         |
| 4  |   |                                                               | %I2.7   | Bool                 | <input type="checkbox"/> FALSE           |              | <input type="checkbox"/>            |         |
| 5  |   |                                                               | %I2.0   | Bool                 | <input type="checkbox"/> FALSE           |              | <input type="checkbox"/>            |         |
| 6  |   | "B_LB"                                                        | %I3.0   | Bool                 | <input checked="" type="checkbox"/> TRUE |              | <input type="checkbox"/>            |         |
| 7  |   | // Monitoring IW4 Hexadecimal; Decimal and Binary data format |         |                      |                                          |              |                                     |         |
| 8  |   |                                                               | %IW4    | Hex                  | 16#1234                                  |              | <input type="checkbox"/>            |         |
| 9  |   |                                                               | %IW4    | Bin                  | 2#0001_0010_0011_0100                    |              | <input type="checkbox"/>            |         |
| 10 |   |                                                               | %IW4    | DEC                  | 4660                                     |              | <input type="checkbox"/>            |         |
| 11 |   |                                                               |         |                      |                                          |              |                                     |         |
| 12 |   | "MW_ACT"                                                      | %MW20   | DEC+/-               | 1234                                     | 1234         | <input checked="" type="checkbox"/> |         |
| 13 |   | "MW_ACT"                                                      | %MW20   | Hex                  | 16#04D2                                  |              | <input type="checkbox"/>            |         |
| 14 |   | "MW_ACT"                                                      | %MW20   | Bin                  | 2#0000_0100_1101_0010                    |              | <input type="checkbox"/>            |         |
| 15 |   | // MW20 high and low-Byte                                     |         |                      |                                          |              |                                     |         |
| 16 |   |                                                               | %MB20   | Hex                  | 16#04                                    |              | <input type="checkbox"/>            |         |
| 17 |   |                                                               | %MB21   | Hex                  | 16#D2                                    |              | <input type="checkbox"/>            |         |
| 18 |   | // "Test_2" Floating-Point Number (REAL) and Hexadecimal      |         |                      |                                          |              |                                     |         |
| 19 |   | "Test"                                                        | %MD80   | DEC                  | 1234                                     | 1234         | <input checked="" type="checkbox"/> |         |
| 20 |   | "Test"                                                        | %MD80   | Hex                  | 16#0000_04D2                             |              | <input type="checkbox"/>            |         |
| 21 |   |                                                               |         |                      |                                          |              |                                     |         |
| 22 |   | Invalid value !!!<br>"accessed in between"                    | %MW83   | Hex                  | 16#D244                                  |              | <input type="checkbox"/>            |         |
| 23 |   |                                                               |         |                      |                                          |              |                                     |         |
| 24 |   | "Test_2"                                                      | %MD84   | Floating-point nu... | 1234.0                                   | 1234.0       | <input checked="" type="checkbox"/> |         |
| 25 |   | "Test_2"                                                      | %MD84   | Hex                  | 16#449A_4000                             |              | <input type="checkbox"/>            |         |

#### Display Formats

Different display formats can be selected in both the "Monitor / Modify Variables" and the "Monitor (Block)" test functions to display variables or register contents. Basically, every variable can be monitored with several display format options. Depending on the variable's data type, it becomes apparent that monitoring with the appropriate display format makes more sense.

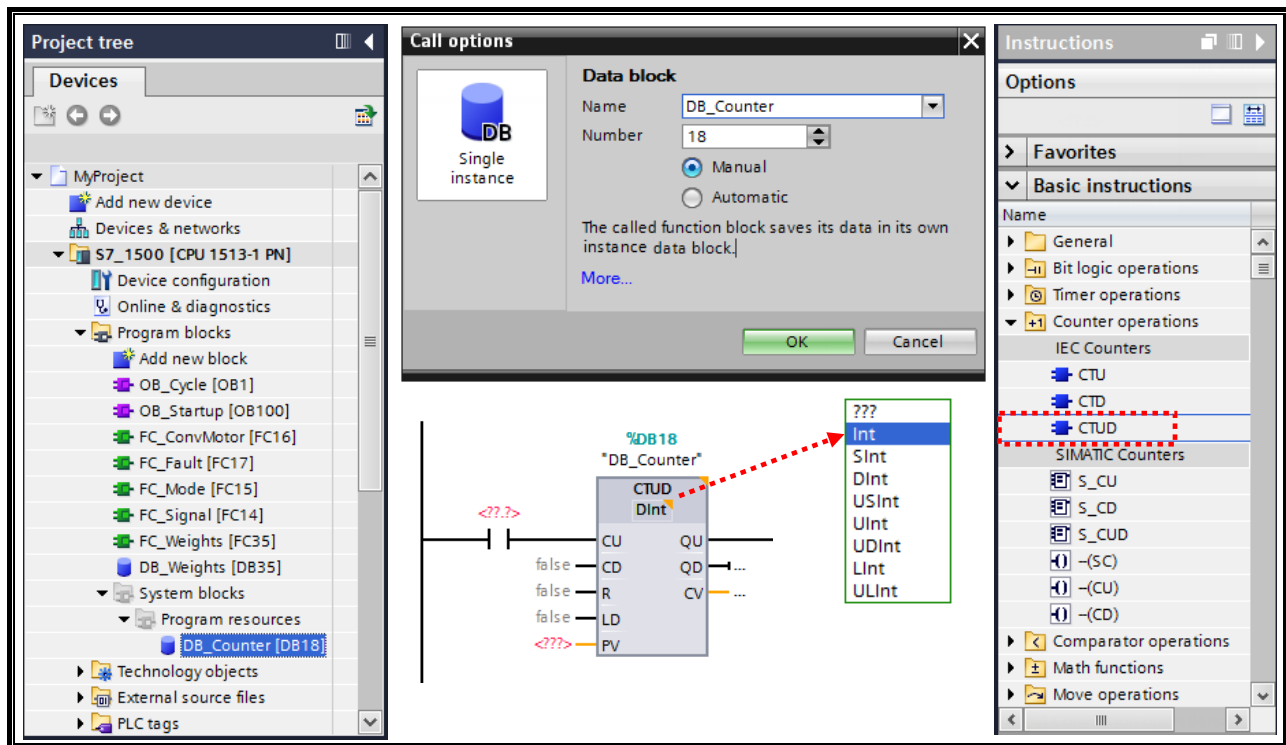
- **BOOL:** Display a single bit (only possible for a variable of the BOOL data type)
- **BIN:** Display the individual bits of a variable (makes sense for variables of the BYTE, WORD, DWORD data types)
- **HEX:** Display the contents of a variable as hexadecimal number (BCD) (makes sense for variables of the BYTE, WORD, DWORD data types)
- **DEC:** Display the contents of a variable as decimal number (not BCD!) with sign (makes sense for variables of the INT, DINT data types)
- **FLOATING Point:** Display the contents of a variable as floating-point number (makes sense for variables of the REAL data type)

#### Addressing

The SIMATIC S7 memory is universally byte-oriented. Accordingly, memory word MW 20, for example, contains the memory bytes MB 20 (high byte) and MB 21 (low byte, see picture), the memory double-word MD 80, the memory bytes MB 80, 81, 82 and 83.

With absolute accesses to variables (such as, with `L MD 80`), you must make sure that the dimension of the access (here `MD...`) as well as the address (always equal to the address of the high byte, here 80) is correct. Through an inadvertent "accessing in between", an invalid value would be loaded (such as, with `L MW 83`, see picture). Such errors can be avoided with the symbolic addressing of variables (here `L "MD_DINT"`).

## 8.4. Counters: CTU, CTD, CTUD



### Counters

Counters are used to count events, record quantities, etc. There are up counters and down counters as well as counters that can count in both directions.

### Value Range of a Counter

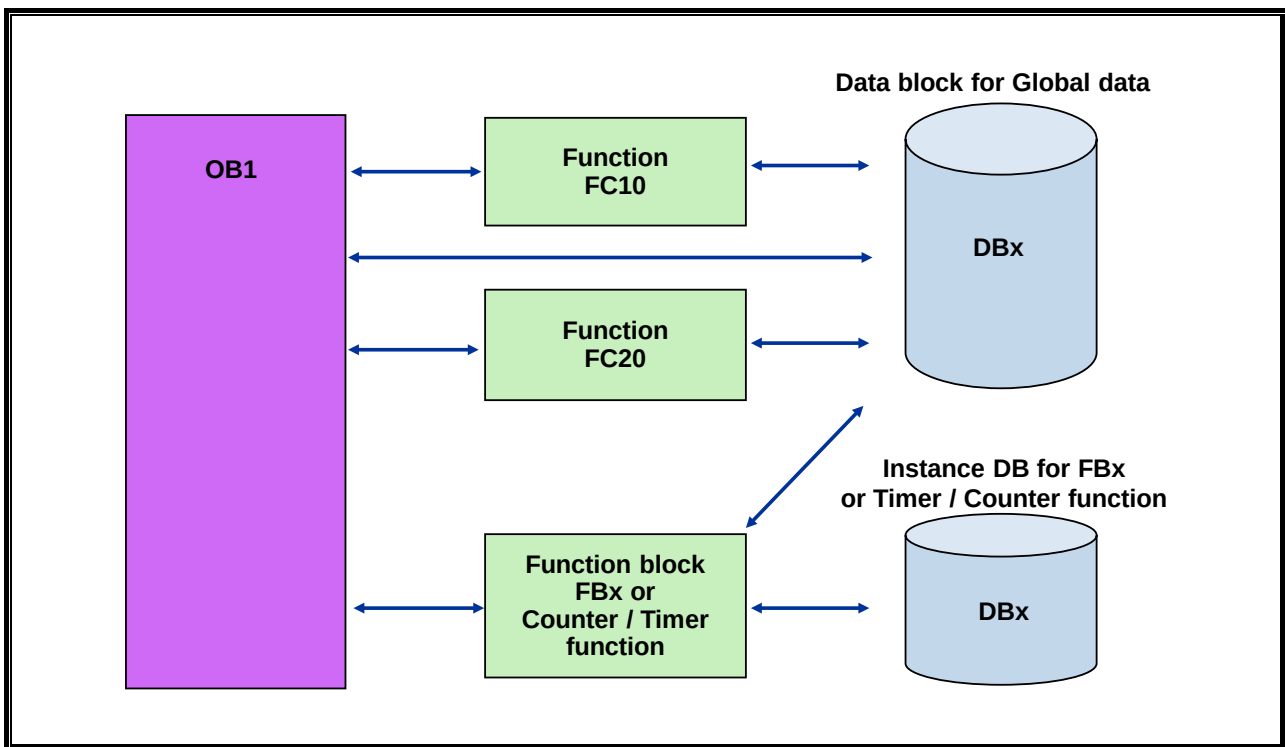
The count or value range of a counter depends on its data type (see picture) which is always an integer. The various selectable Integer data types merely differentiate themselves in their value range and thus determine the count range of the counter.

### Instance Data Block

In addition to internally required variables, the counter also stores its current counter value in a so-called instance data block which must be specified when programming a counter. The specified instance data block is automatically generated by the Editor with exactly the internal structure that the counter requires. The user has no further programming effort with this data block other than having to download it into the CPU.



### 8.4.1. Timer / Counter Instance Data Blocks



#### Data Blocks

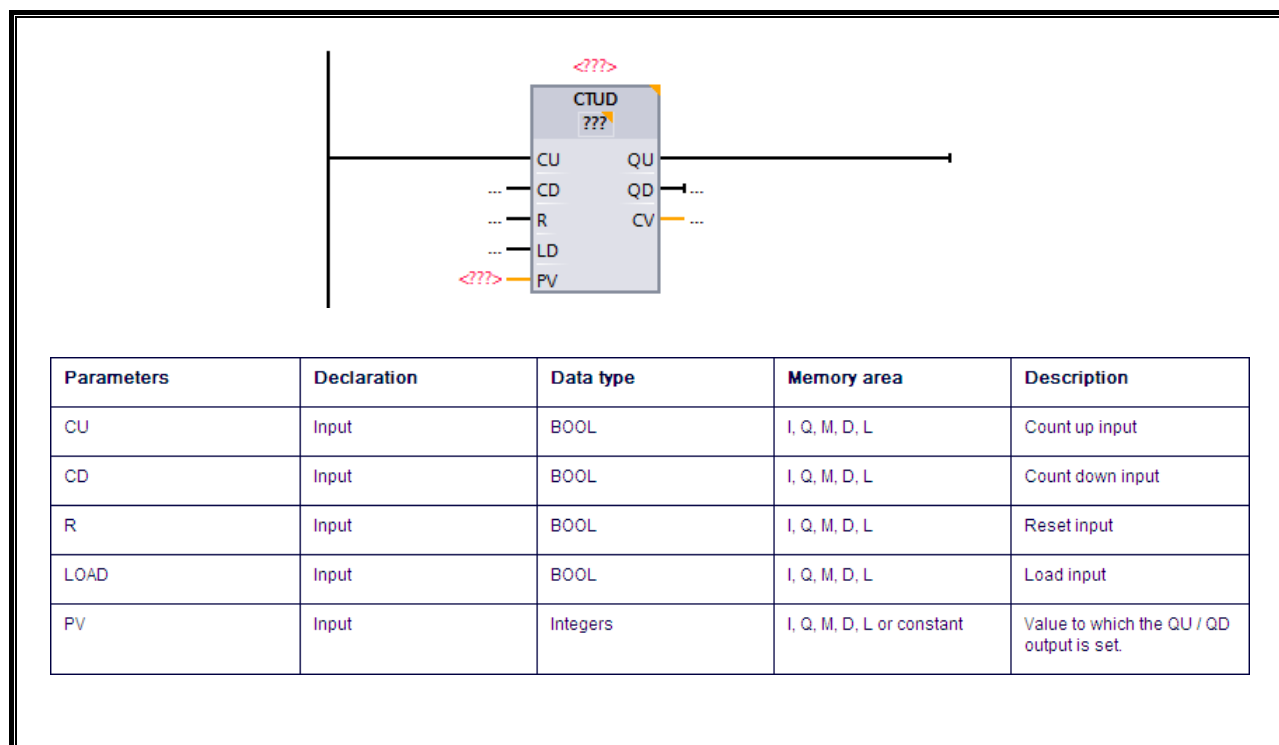
Data blocks are used for storing user data. They occupy memory space in the user memory of the CPU. Data blocks contain variable data (such as numeric values) with which the user program works. The user program can access the data in a data block with bit, byte, word or double-word operations. Access can be made symbolically or absolutely.

#### Area of Application

Data blocks are used with two different purposes:

- Global data blocks: These contain information that all the logic blocks in the user program can access.
- Instance data blocks: These are always assigned to a particular FB or functions such as counters and timers. The data of these instance DBs should only be processed by the associated FB or counter / timer function.
- Creating DBs
- Global DBs are created either with the Program Editor or according to a previously created "user-defined data type". Instance data blocks are generated when a function block or a counter / timer function is called.

### 8.4.2. Counter Function: Inputs



#### Input CU and CD

With a positive edge at input CU, the current count is increased by one; with a positive edge at input CD, the current count is decreased by 1. If a positive edge is detected at both inputs simultaneously or in the same cycle, the current count remains unchanged. If the upper or lower limit of the specified data type is reached, the count is no longer increased or decreased for a positive edge at CU or CD.

#### Input R

The input R acts statically, that is, as long as RLO '1' is at input R, the count is set to 0 and rising edges or RLO '1' at the inputs CU, CD and LOAD have no effect on the current count.

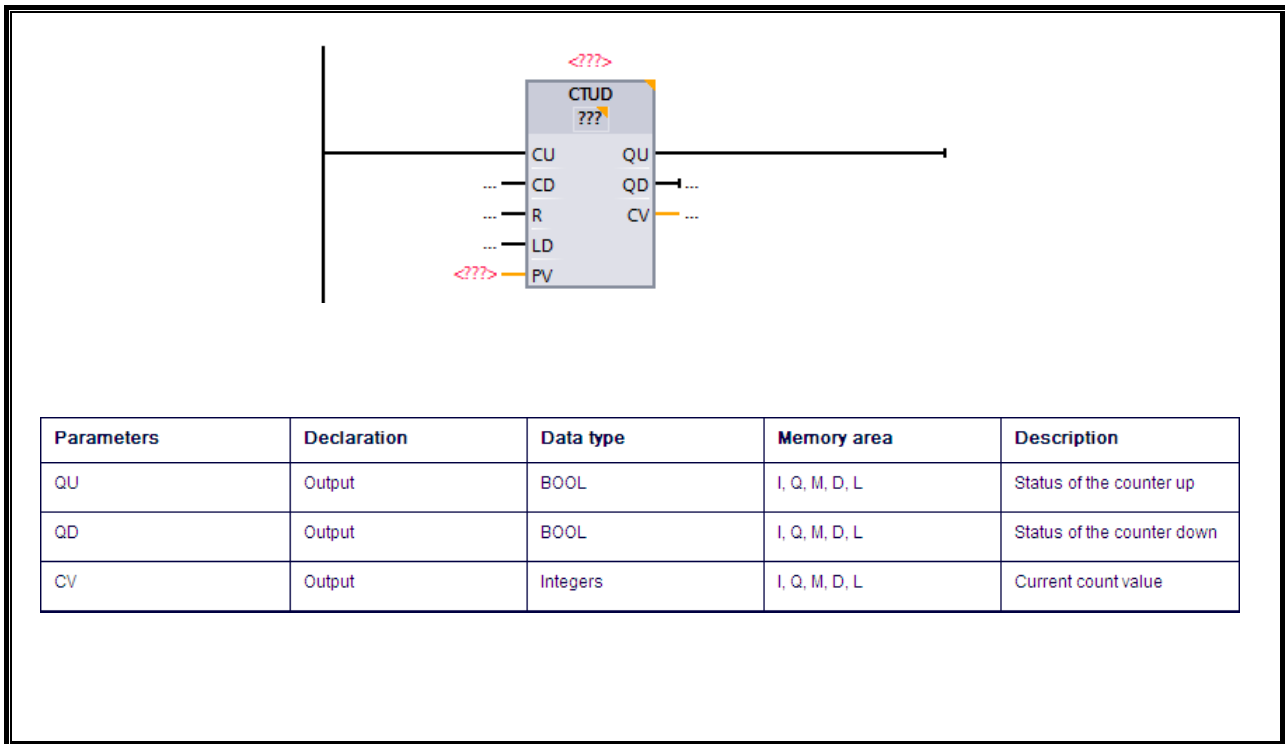
#### Input LD (Load, only for down counters)

The input LD acts statically, that is, as long as RLO '1' is at input LOAD, the current count is set to the value that is passed to the input PV, and positive edges at the inputs CU and CD have no effect on the count.

#### Input PV

The value to which the count is to be set must be passed to the input PV as long as RLO '1' is at input LD. The variable or constant passed to the input must be compatible with the data type of the counter.

### 8.4.3. Counter Function: Outputs



#### Output QD (Only for down counters)

The current status of the down counter can be checked at the output QD. As long as the current count is less than or equal to zero, the output QD has Status '1', otherwise, Status '0'.

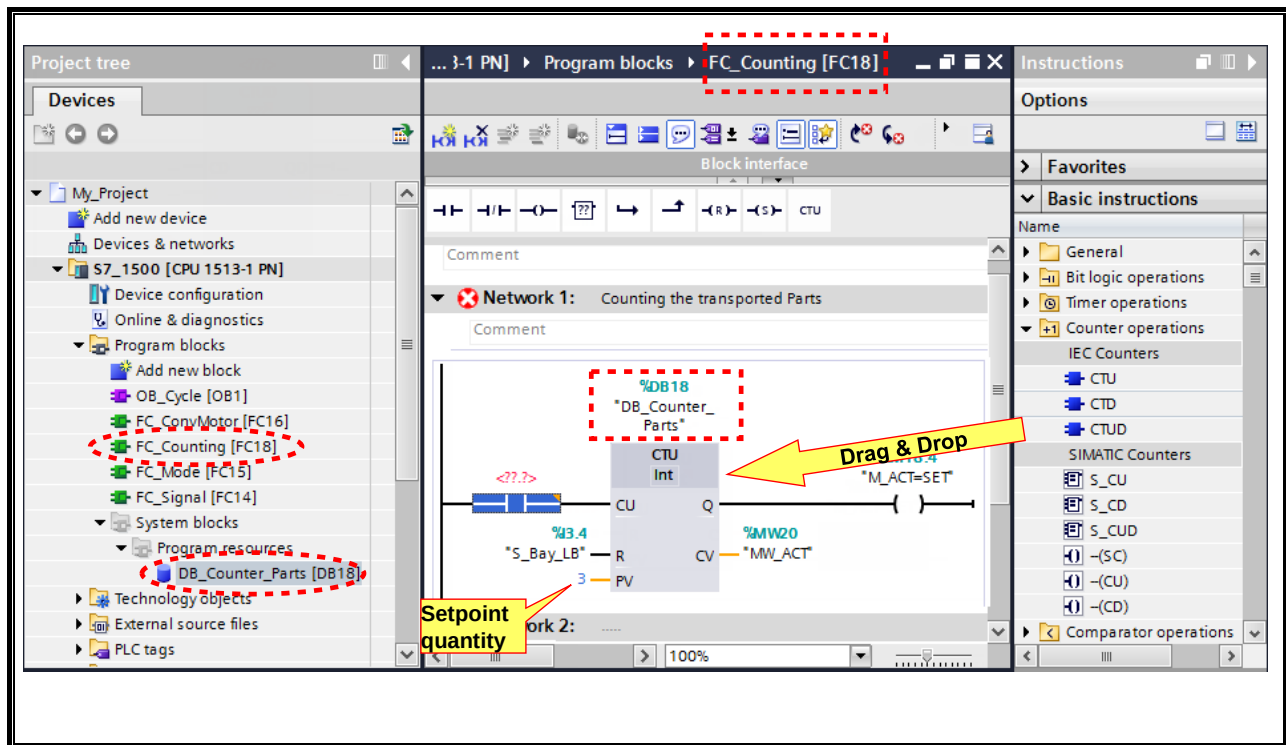
#### Output CV

The current count is output at output CV. The variable passed to the output must be compatible with the data type of the counter.

#### Output QU

The current status of the up counter can be checked at the output QU. As long as the current count is greater than or equal to the value of the parameter PV, the output QU has Status '1', otherwise, Status '0'.

#### 8.4.4. Exercise 1: Counting the Transported Parts Programming "FC\_Counting" (FC18)



#### Task

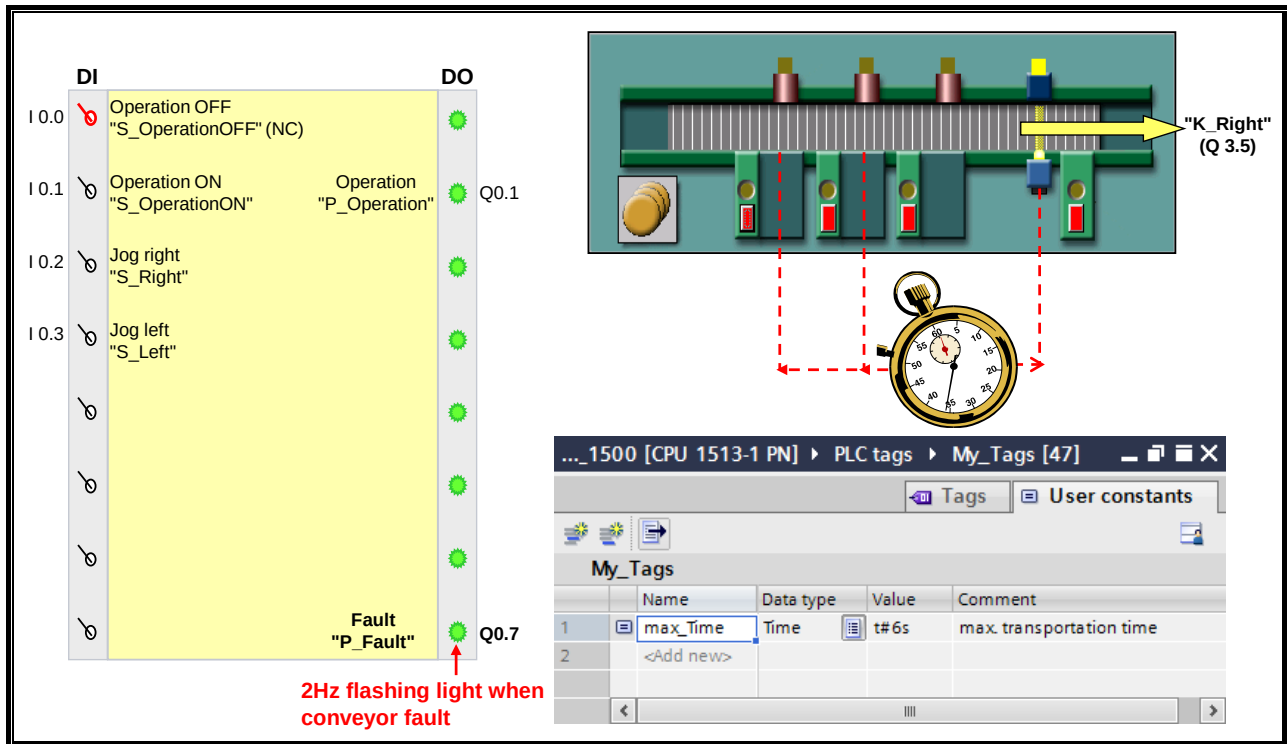
When "P\_Operation" (Q0.1) is switched on, the transported parts are to be counted and the ACTUAL quantity is to be stored in the variable "MW\_ACT" (MW20).

If the ACTUAL quantity has reached the SETPOINT quantity of 3, the indicator light "P\_BayLB" (Q3.4) shows a 1Hz flashing light. As long as this is not acknowledged via "S\_BayLB" (I 3.4), the indicator lights "P\_Bay1" (Q3.1) and "P\_Bay2" (Q3.2) are dark and no new part transportation can be started.

#### What to Do

1. Add the new block "FC\_Counting" (FC18).
2. In "FC\_Counting" (FC18), program the required functions. Use an IEC Counter to count the parts. Pass "DB\_Counter\_Parts" (DB18) as the instance data block to this IEC Counter. At the PV input, parameterize the setpoint quantity 3.
3. Program the call of "FC\_Counting" (FC18) in "OB\_Cycle" (OB1). Download all blocks into the CPU and check the program function.
4. In the blocks "FC\_Signal" (FC14) and "FC\_ConvMotor" (FC16), program the required lock-outs with "M\_ACT=SET" (M18.4).
5. Download all modified blocks into the CPU and check the program function.
6. Save your project.

## 8.5. Task Description: Timed Monitoring of the Transport Sequences



### Function Up Until Now

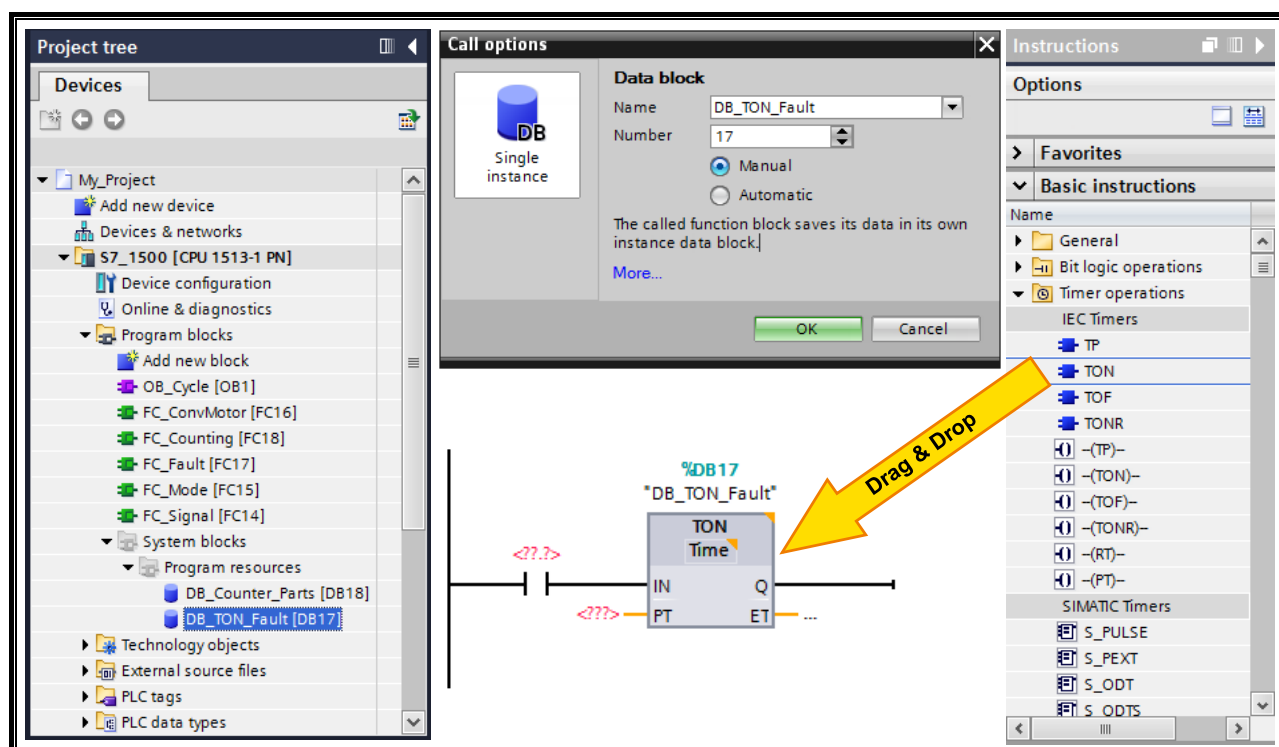
When "P\_Operation" (Q0.1) is switched on, parts are transported from Bay 1 or 2 until they are through the light barrier. A transport sequence starts as soon as a part is placed on the conveyor at Bay 1 or 2 and the associated bay pushbutton is pressed. The transport sequence ends as soon as the part has passed the light barrier.

### Task Description

The automatic transport sequences are to be monitored for time. The monitoring is to function as follows:

- If a transport sequence takes longer than the 6 second monitoring time, there is a fault and the conveyor motor is automatically switched off.
- A fault is displayed with a 2Hz flashing light on the simulator LED "P\_Fault" (Q0.7).
- A fault can be acknowledged via the simulator pushbutton "S\_Acknowledge" (I 0.7).
- As long as there is an unacknowledged fault, the indicator lights "P\_Bay1" (Q3.1) and "P\_Bay2" (Q3.2) are dark and no new transport sequence can be started.

### 8.5.1. Timer Function TON



#### Data Type TIME

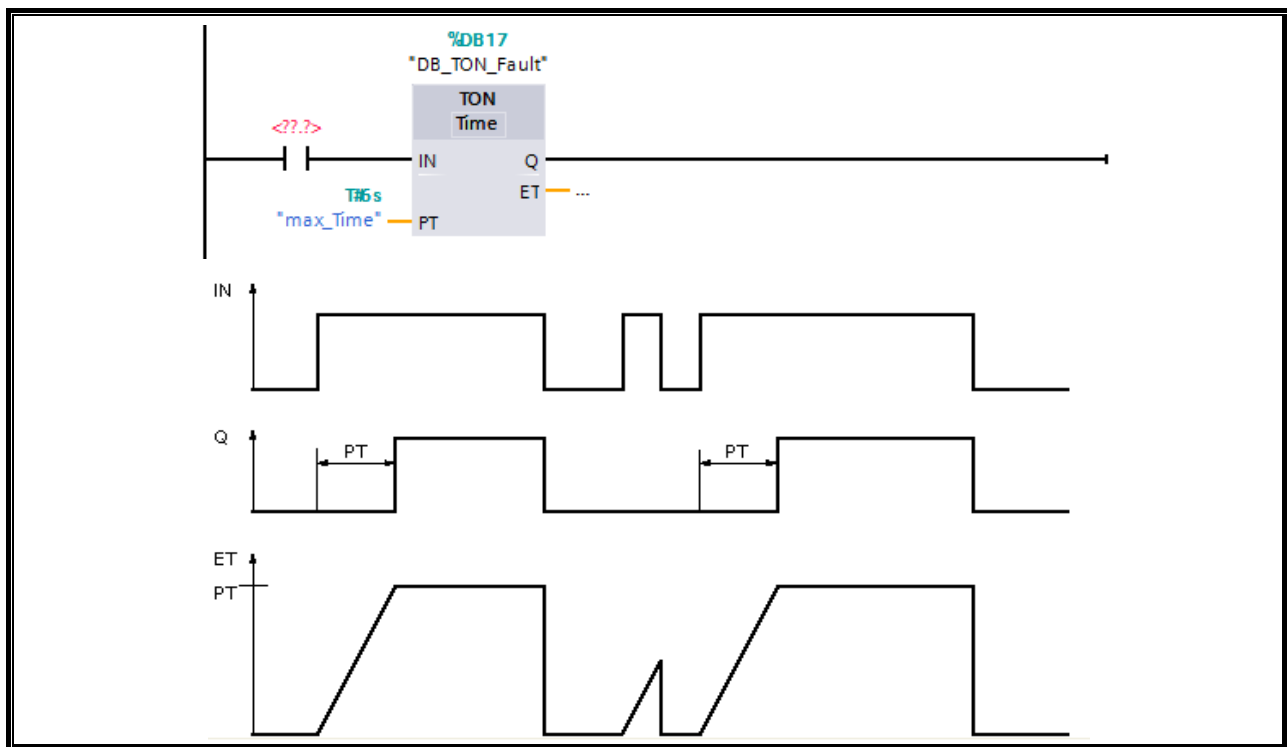
The contents of a variable or constant of the data type TIME is interpreted as an integer number in milliseconds and stored in the memory as a 32-bit integer with sign. The representation contains information for days (d), hours (h), minutes (m), seconds (s) and milliseconds (ms). Examples:

| Length (bits) | Format                 | Value range                                          | Examples of value input                                              |
|---------------|------------------------|------------------------------------------------------|----------------------------------------------------------------------|
| 32            | Time period with sign: | T# -24d20h31m23s648ms<br>to<br>T# +24d20h31m23s647ms | T#10d20h30m20s630ms,<br>TIME#10d20h30m20s630ms,<br>10d20h30m20s630ms |

#### Data Block

In addition to internally required variables, the timer function also stores the current already expired time in a data block which must be specified when programming the timer function. The specified data block is automatically generated by the Editor with exactly the internal structure that the timer function requires. The user has no further programming effort with this data block other than having to download it into the CPU.

### 8.5.2. Timer Function TON (ON Delay) Pulse Diagram

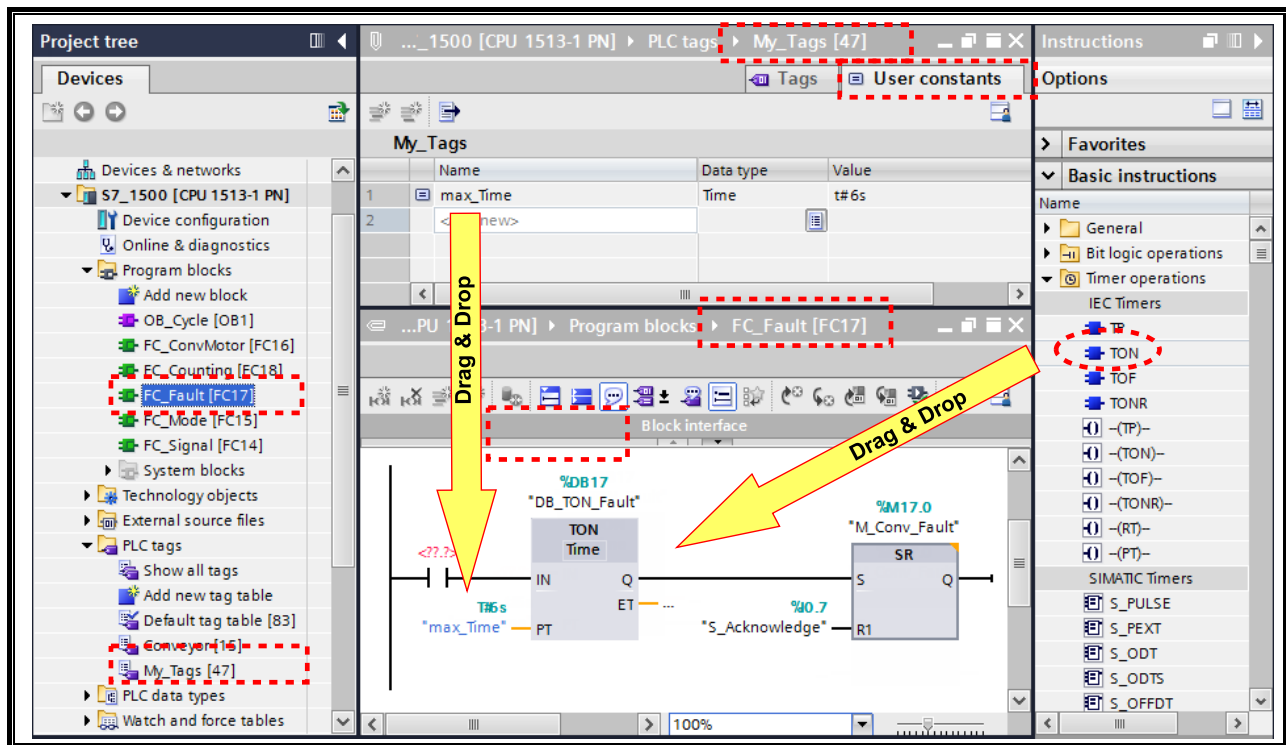


#### TON

The timer function TON (Timer on delay, "ON Delay") is started with a rising edge at input IN. So that the time expires, RLO must continue to be '1'. The timer function supplies a '1' signal at output Q, as soon as the specified time (variable or constant of data type TIME) at input PT has expired and as long as the start signal at input IN still exists. The already expired time can be queried at output ET by passing a variable of data type TIME.

| Parameters | Declaration | Data type | Memory area               | Description                                                                  |
|------------|-------------|-----------|---------------------------|------------------------------------------------------------------------------|
| IN         | Input       | BOOL      | I, Q, M, D, L             | Start input                                                                  |
| PT         | Input       | TIME      | I, Q, M, D, L or constant | Duration of the on delay.<br>The value of the PT parameter must be positive. |
| Q          | Output      | BOOL      | I, Q, M, D, L             | Output that is set when the time PT expires.                                 |
| ET         | Output      | TIME      | I, Q, M, D, L             | Current time value                                                           |

### 8.5.3. Exercise 2: Timed Monitoring of the Transports Programming "FC\_Fault" (FC17)



#### Task

The automatic transport sequences are to be monitored for time as previously described. If a transport sequence takes longer than 6 seconds, the conveyor motor is automatically switched off and the fault is displayed with a 2Hz flashing light on the simulator LED "P\_Fault" (Q0.7). As long as a fault is not acknowledged (I 0.7) no new transport sequence can be started.

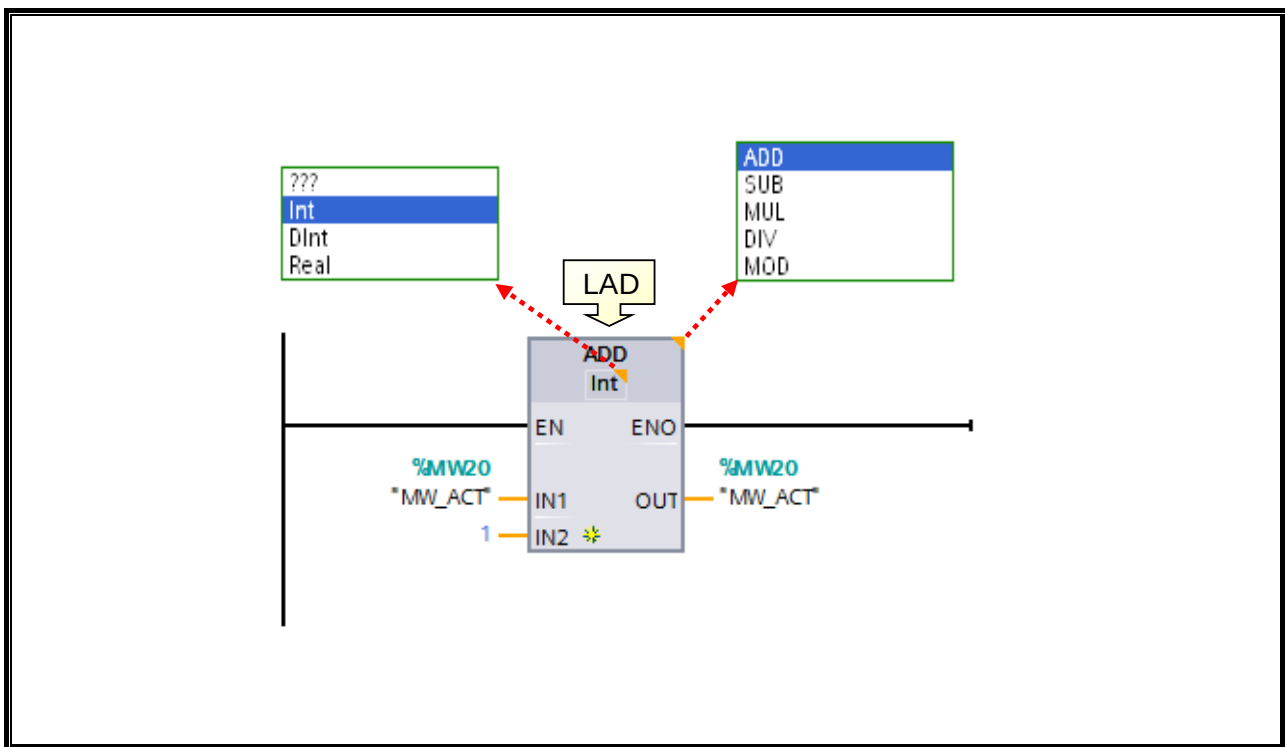
#### What to Do

1. In the PLC tag table "My\_Tags", declare the user constant "max\_Time" of the data type *Time* with a value *t#6s* as shown in the picture.
2. Program the new block "FC\_Fault" (FC17) with the necessary functions:
  - At input IN, program the relevant start conditions (the timer must be started when an automatic transport sequence starts)
  - Pass the data block "DB\_TON\_Fault" (DB17) as the instance-DB to the IEC timer function TON and as a time duration, the user constant "max\_Time" (see picture).
  - In case the maximum transportation time is exceeded, set the bit memory "M\_Conv\_Fault" (M17.0), in order to be able to further logically link it in other blocks later on.
  - Program the control of the simulator LED "P\_Fault" (Q0.7)
3. Program the call of "FC\_Fault" (FC17) in "OB\_Cycle" (OB1).
4. In the "FC\_ConvMotor" (FC16) block, program the required switching off the conveyor motor when there is a conveyor fault.
5. In "FC\_Signal" (FC14), program the relevant lock-outs so that the indicator lights "P\_Bay1" (Q3.1) and "P\_Bay2" (Q3.2) are dark when a fault exists.
6. Download all modified blocks into the CPU and check the program function.
7. Save your project.





### 8.6.1. Basic Mathematical Functions: Addition



#### Arithmetic Operations

S7-1500 has a series of arithmetic operations for the processing of variables of the arithmetic data types, such as, integer (I), double integer (DI) and real (R).

#### Inputs and Outputs of the LAD/FBD Elements:

- **EN**

The execution of the operation can be determined as follows via the EN input:

- EN is not connected: The operation is always (regardless of the RLO) executed
- Logic operation at EN is fulfilled (RLO = 1): The operation is executed
- Logic operation at EN is not fulfilled (RLO = 0): the operation is not executed

- **IN1 / IN2**

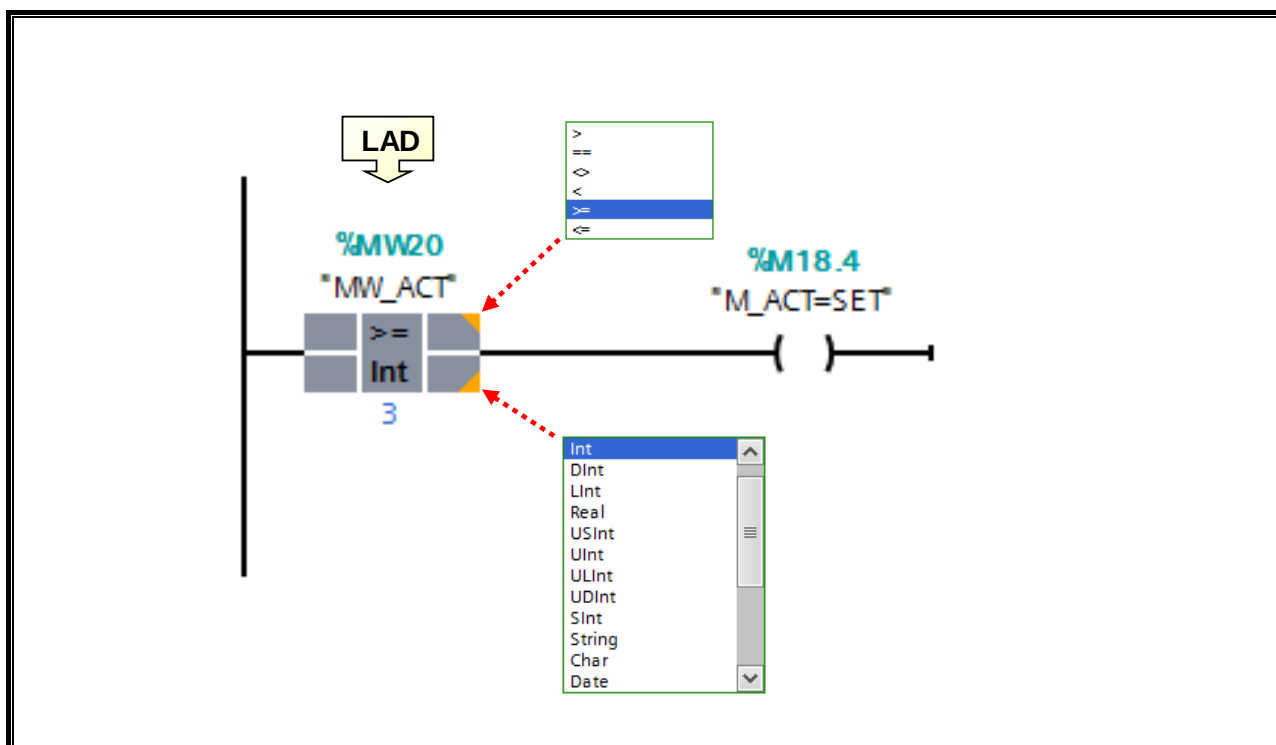
The arithmetic calculation is applied to the values delivered to IN1 and IN2 and the result is output to OUT.

ENO / OUT

- **ENO**

- ENO = 0 because RLO=0 at EN input
  - OUT not written
  - The variable delivered to OUT is not written, that is, it keeps its original value
- ENO = 0 because error occurred
  - OUT contains invalid value
  - The variable delivered to OUT is overwritten with an invalid value
- ENO = 1 (instruction was executed without error):
  - OUT contains result
  - The variable delivered to OUT is overwritten with the result

## 8.6.2. Basic Mathematical Functions: Comparison Operations



### Comparison Function

With the comparison instructions, the following pairs of numerical values can be compared:

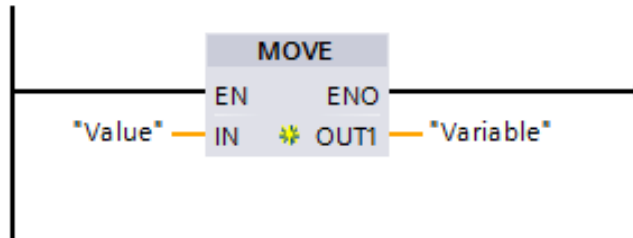
- all variations of integers
- all variations of floating-point numbers (Real = IEEE floating-point numbers)
- all variations of TIME data types

If the result of the comparison is "true", then the RLO of the operation is "1", otherwise it is "0". The input IN1 is compared with IN2 according to the selected type of comparison:

- |      |     |                             |      |
|------|-----|-----------------------------|------|
| • == | IN1 | is equal to                 | IN2  |
| • <> | IN1 | is not equal to             | IN2  |
| • >  | IN1 | is greater than             | IN2  |
| • <  | IN1 | is less than                | IN2  |
| • >= | IN1 | is greater than or equal to | IN2  |
| • <= | IN1 | is less than or equal to    | IN2. |

### 8.6.3. Value Assignment of a Variable

| Parameter | Data type                                     | Memory area                   | Description         |
|-----------|-----------------------------------------------|-------------------------------|---------------------|
| EN        | BOOL                                          | <a href="#">I, Q, M, D, L</a> | Enable input        |
| ENO       | BOOL                                          | I, Q, M, D, L                 | Enable output       |
| IN        | All elementary data types, DTL, STRUCT, ARRAY | I, Q, M, D, L or constant     | Source value        |
| OUT1      | All elementary data types, DTL, STRUCT, ARRAY | I, Q, M, D, L                 | Destination address |

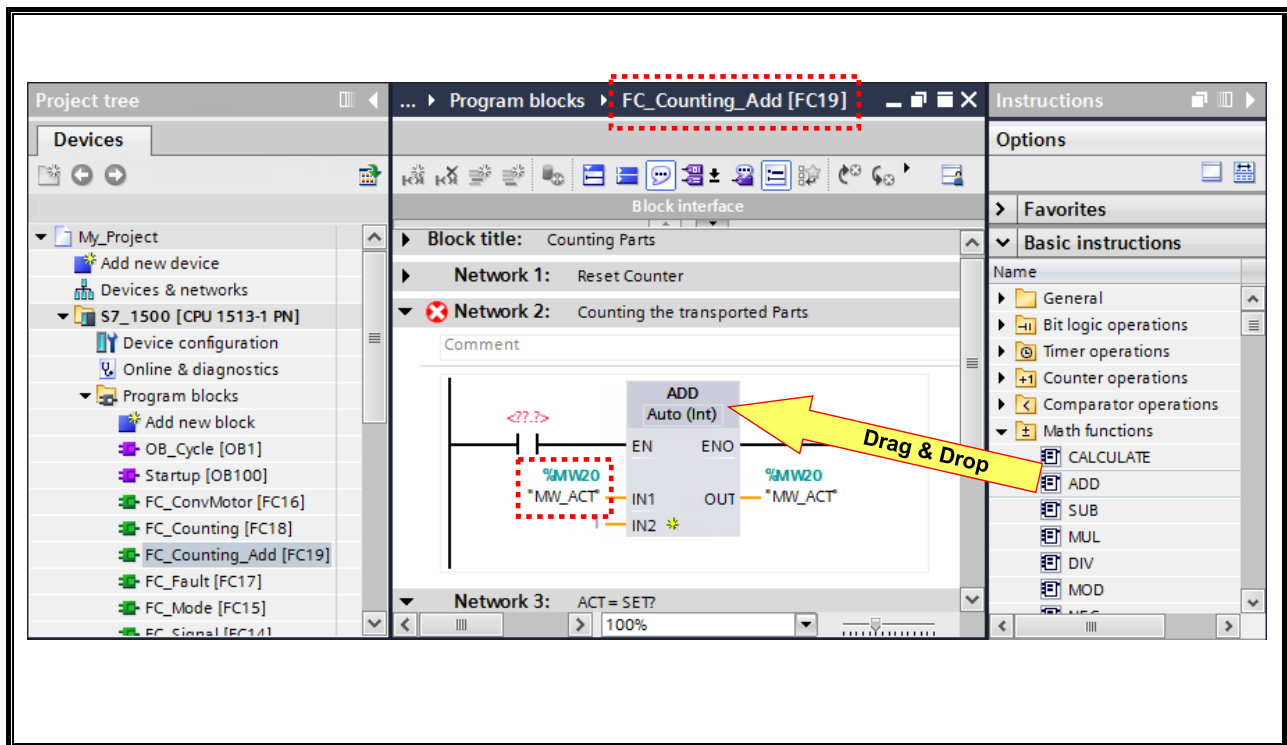


#### MOVE

With the "MOVE" operation, the content of the operand at the input IN is transferred to the operand at the output OUT1.

The operation is only executed if the signal state is "1" at the enable input EN. In this case, the ENO output also has signal state "1".

#### 8.6.4. Exercise 3: Counting the Transported Parts using Addition in "FC\_Counting\_Add" (FC19)



##### New Function

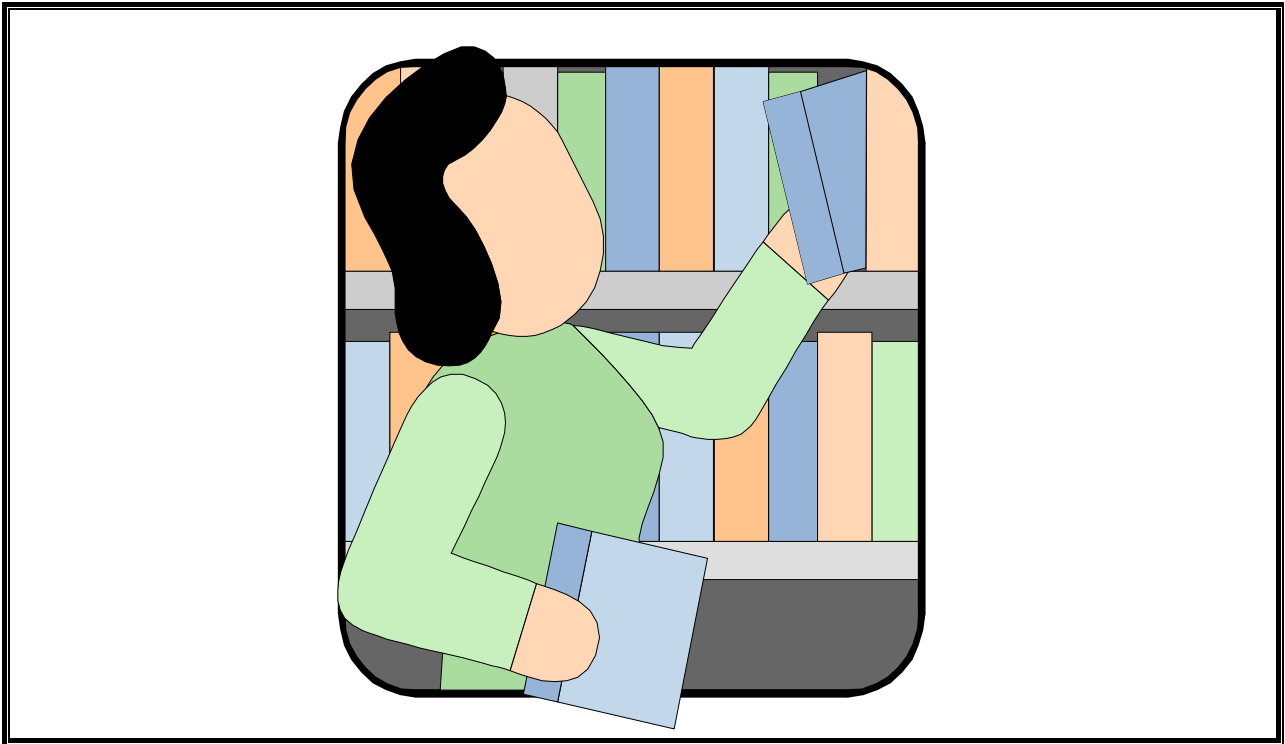
The counting function up until now is to remain unchanged, only the program implementation is to be changed as follows:

The counting of the transported parts is no longer to be done with an IEC counter, but by addition.

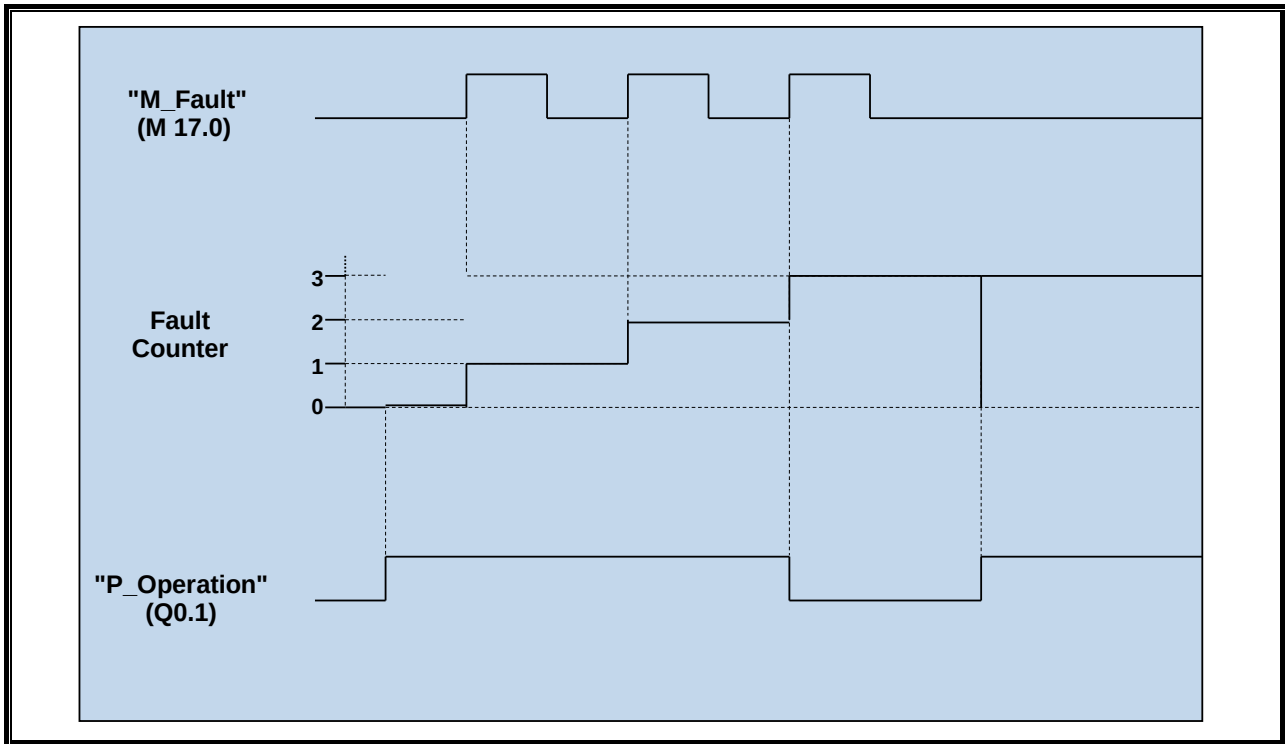
##### What to Do:

1. Program the new block "FC\_Counting\_Add" (FC19) with the relevant arithmetic operations. Save the current quantity of transported parts in the memory word "MW\_ACT" (MW20).
2. In OB1, instead of the old "FC\_Counting" (FC18), call the new "FC\_Counting\_Add" (FC19).
3. Save your project.

## **8.7. Additional Information**



### 8.7.1. Additional Exercise 4: Counting the Conveyor Faults Expanding "FC\_Fault" (FC17)



#### Function Up Until Now

When "P\_Operation" (Q0.1) is switched on, the transport sequences are monitored for time. If a transport sequence takes longer than the monitoring time of 6 seconds, there is a conveyor fault and the conveyor motor is automatically switched off (logically linked in "FC\_ConvMotor" (FC16)).

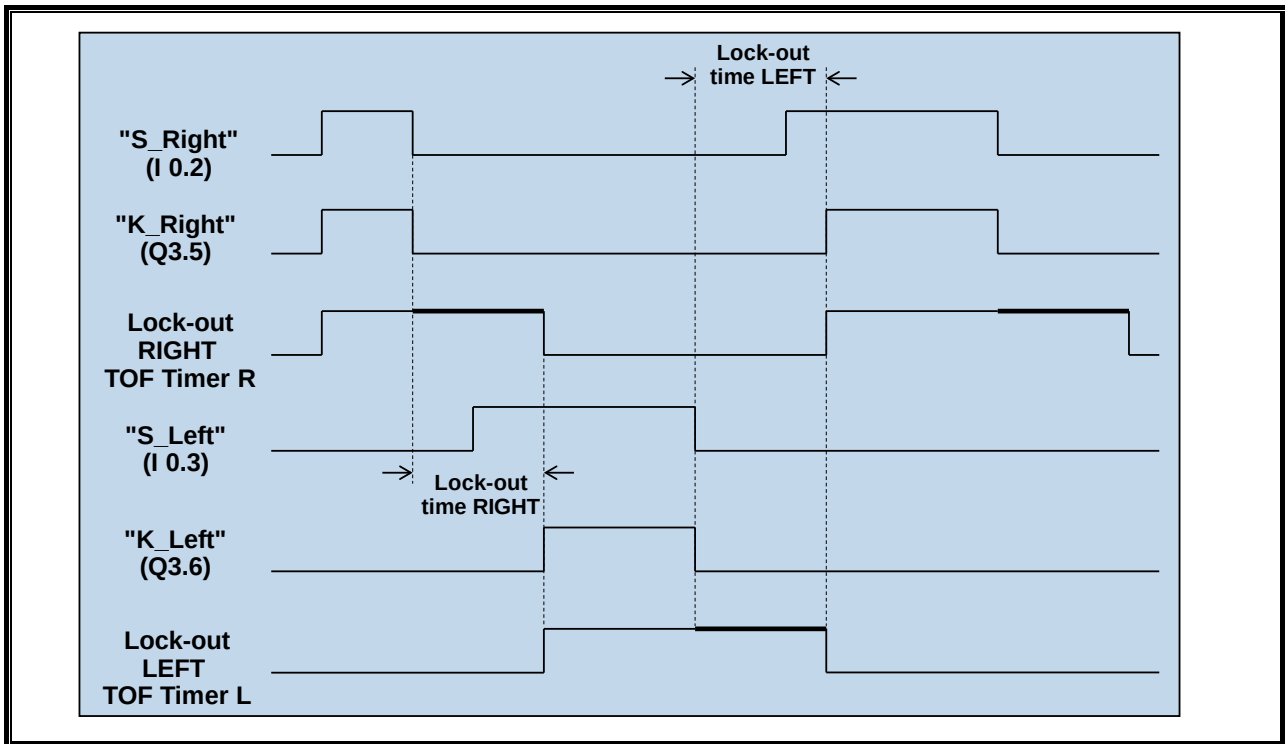
#### Task:

When "P\_Operation" (Q0.1) is switched on, the conveyor faults are to be counted. After 3 conveyor faults have occurred, "P\_Operation" (Q0.1) is to be switched off for safety reasons. To start a new transport sequence, the fault (as already programmed) must be acknowledged and "P\_Operation" (Q0.1) must be switched on once more.

#### What to Do:

1. In "FC\_Fault" (FC17), in a new network, program the counting of the conveyor faults. The counter counts up 1 every time a conveyor fault occurs ("M\_Fault" (M17.0) = "1").
2. In "FC\_Fault" (FC17), pass the value at counter output Q to any bit memory you like.
3. In "FC\_Mode" (FC15), program the switching off (reset) of "P\_Operation" (Q0.1) after three conveyor faults. For this, use the bit memory to which you passed the counter output Q in "FC\_Fault" (FC17).

### 8.7.2. Additional Exercise 5: Timely Lock-out of the Conveyor Motor Jogging



#### Function Up Until Now

When the indicator light of "P\_Operation" (Q0.1) is switched off, the conveyor motor can be jogged to the RIGHT and LEFT using the simulator pushbuttons "S\_Right" (I 0.2) and "S\_Left" (I 0.3).

#### Task:

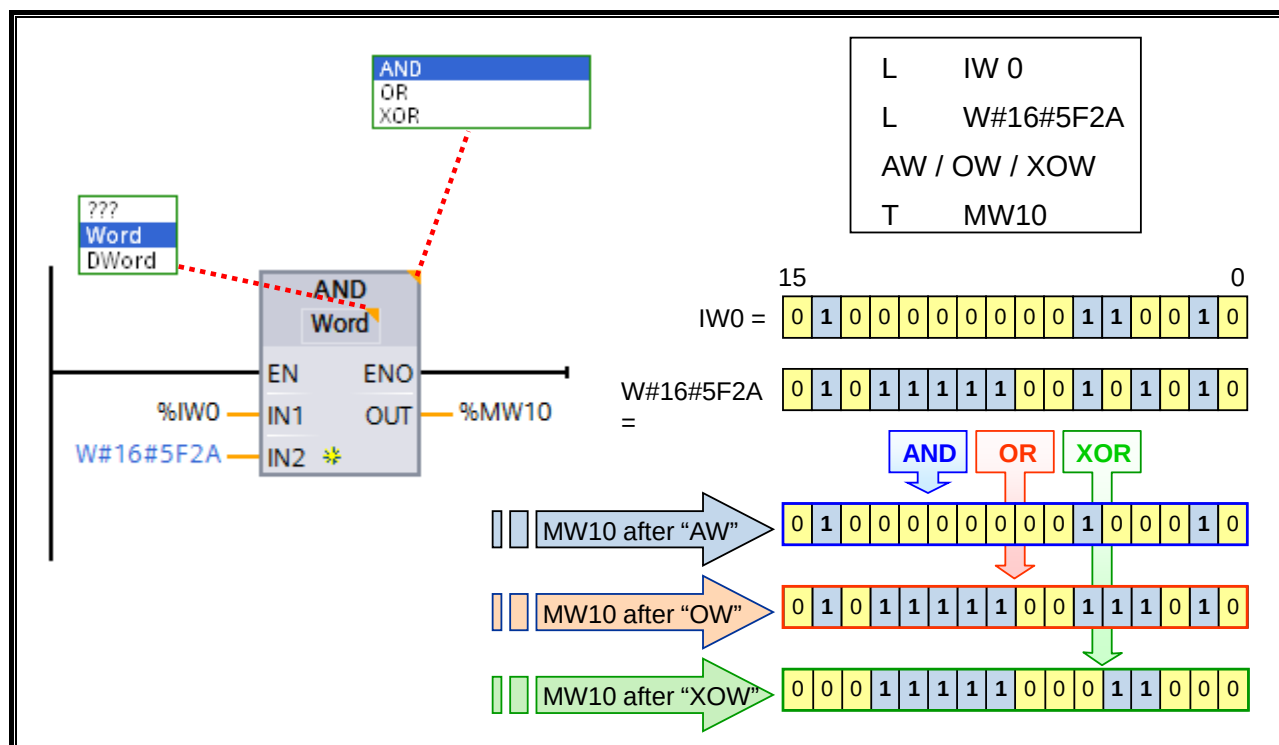
In order to avoid too great a load change, it should only be possible to jog the conveyor motor in the opposite direction after it has been jogged to the RIGHT or to the LEFT after a lock-out time of 2 seconds (see picture). If, for example, the motor has been jogged to the RIGHT, then it can only be jogged back to the LEFT after the lock-out time of 2 seconds has expired.

#### What to Do:

1. In "FC\_ConvMotor" (FC16) program two TOF timers (Off Delay) as the lock-out timers RIGHT and LEFT and assign one bit memory each to the Timer result Q.
2. Gate these bit memories to the jog conditions.
3. Download the modified "FC\_ConvMotor" (FC16) into the CPU and check the program function.



### 8.7.3. Digital Logic Operations



#### WAND\_W

The "AND Word" instruction gates (combines) the two digital values at inputs IN1 and IN2 bit-by-bit in accordance with the AND truth table. The result of the AND operation is stored at output OUT. The instruction is executed when EN = 1.

Example: Masking out the 4th decade of the thumbwheel buttons:

|           |   |      |      |      |      |
|-----------|---|------|------|------|------|
| IW2       | = | 0100 | 0100 | 1100 | 0100 |
| W#16#0FFF | = | 0000 | 1111 | 1111 | 1111 |
| MW30      | = | 0000 | 0100 | 1100 | 0100 |

#### WOR\_W

The "OR Word" instruction gates (combines) the two digital values at inputs IN1 and IN2 bit-by-bit in accordance with the OR truth table. The result of the OR operation is stored at output OUT. The instruction is executed when EN = 1.

Example: Setting bit 0 in MW32:

|           |   |      |      |      |      |
|-----------|---|------|------|------|------|
| MW32      | = | 0100 | 0010 | 0110 | 1010 |
| W#16#0001 | = | 0000 | 0000 | 0000 | 0001 |
| MW32      | = | 0100 | 0010 | 0110 | 1011 |

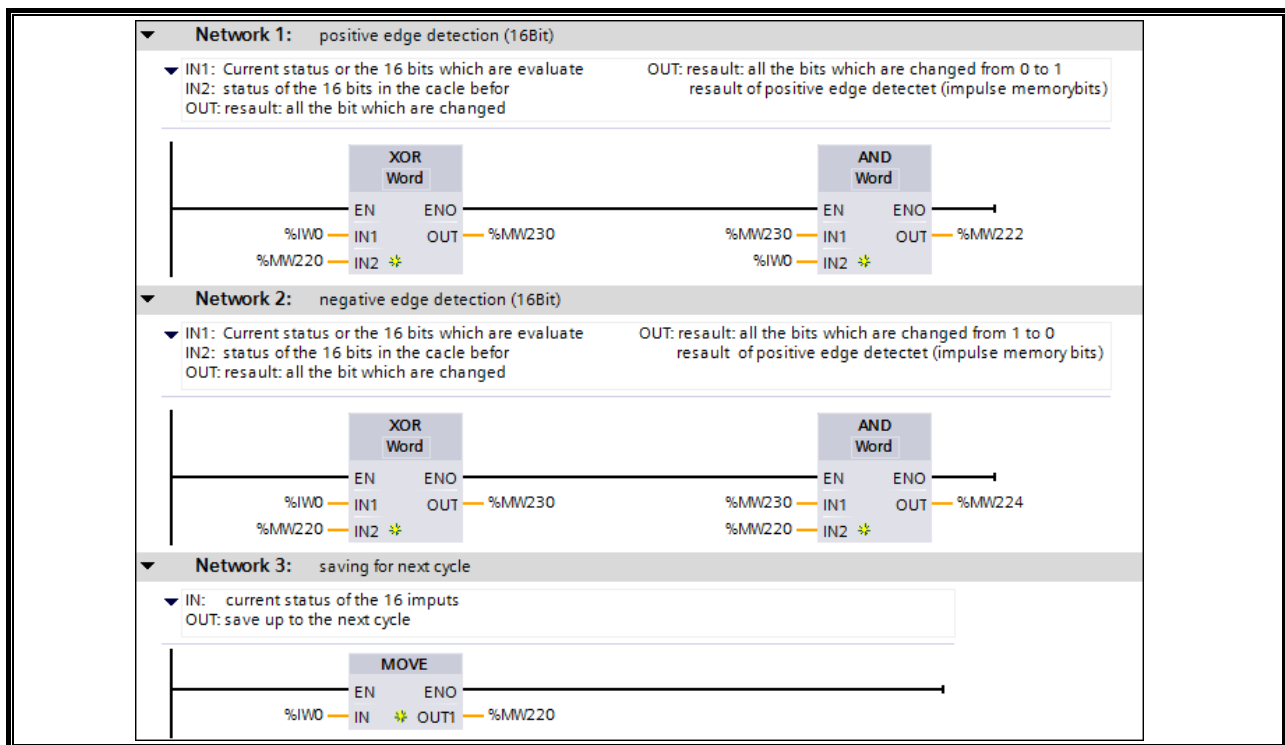
#### WXOR\_W

The "Exclusive OR Word" instruction gates (combines) the two digital values at inputs IN1 and IN2 bit-by-bit in accordance with the XOR truth table. The result of the XOR operation is stored at output OUT. The instruction is executed when EN=1.

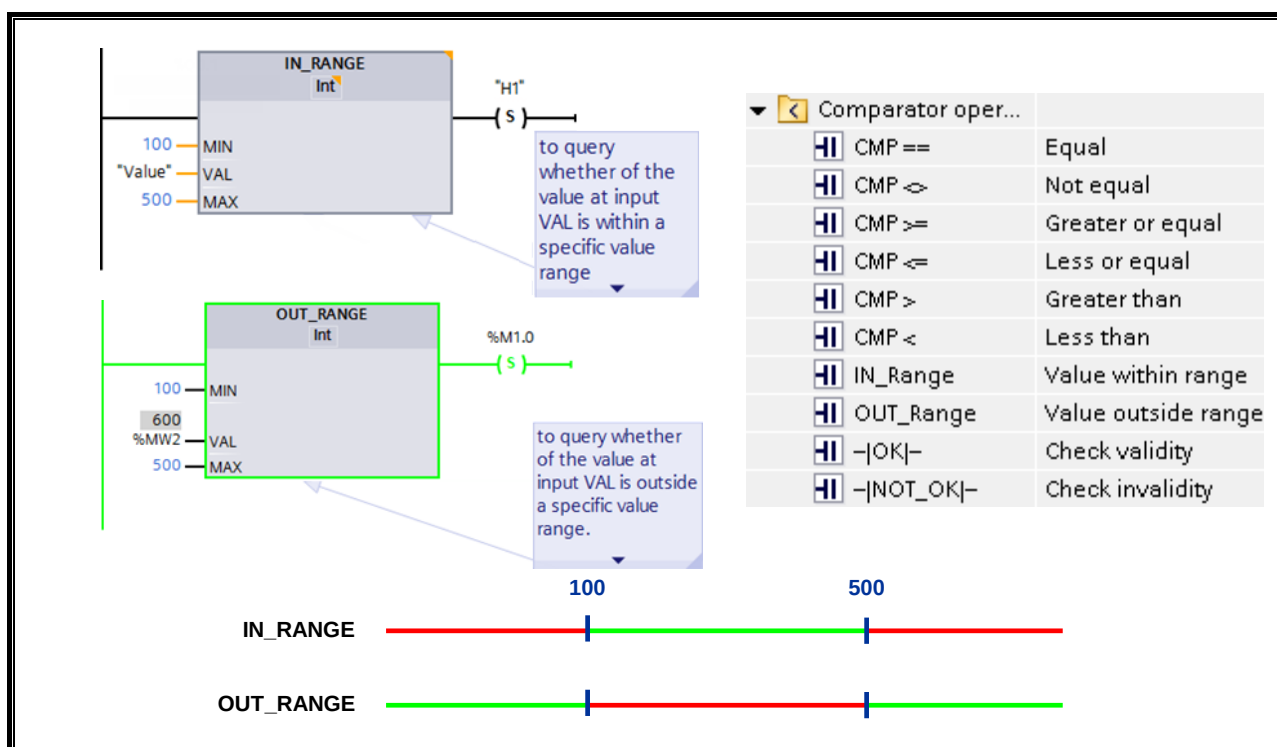
Example: Detecting signal changes in IW0:

|      |   |      |      |      |      |
|------|---|------|------|------|------|
| IW0  | = | 0100 | 0100 | 1100 | 1010 |
| MW28 | = | 0110 | 0010 | 1011 | 1001 |
| MW24 | = | 0010 | 0110 | 0111 | 0011 |

### 8.7.4. Application Example: Digital Edge Evaluation



### 8.7.5. Comparator Operations: IN\_RANGE, OUT\_RANGE



#### IN\_RANGE

You can use the "Value within range" operation to query whether the value at the VAL input is within a specific value range. You specify the limits of the value range with the MIN and MAX parameters. When the query is processed, the "Value within range" operation compares the value at the VAL input to the values of the MIN and MAX parameters and sends the result to the box output. If the value at the VAL input satisfies the comparison  $\text{MIN} \leq \text{VAL} \leq \text{MAX}$ , the box output has signal state "1", otherwise signal state "0".

The comparison function can only be executed if the values to be compared are of the same data type and the box output is interconnected.

#### OUT\_RANGE

You can use the "Value outside range" operation to query whether the value at the VAL input is outside of a specific value range. You specify the limits of the value range with the MIN and MAX parameters. When the query is processed, the "Value outside range" operation compares the value at the VAL input to the values of the MIN and MAX parameters and sends the result to the box output. If the value at the VAL input satisfies the comparison  $\text{MIN} > \text{VAL}$  or  $\text{VAL} > \text{MAX}$ , the box output has signal state "1", otherwise signal state "0".

The comparison function can only be executed if the values to be compared are of the same data type and the box output is interconnected.

### 8.7.6. Date and Time-of-day: RD\_SYS\_T

The screenshot displays the SIMATIC TIA Portal interface. On the left, a ladder logic diagram shows the **RD\_SYS\_T** instruction. It is triggered by a normally open contact labeled **"B4"**. The instruction has an **EN** (Enable In) input, an **ENO** (Enable Out) output, a **RET\_VAL** (Return Value) output, and an **OUT** (Output) output. The **OUT** output is connected to a variable named **time\_of\_day**. The **RET\_VAL** output is connected to a variable named **"Error"**. The **DTL** (Data Type List) is set to **DTL#2013-11-2...**. The **OUT** output is also connected to a variable named **16#0000**.

On the right, the **Extended instructions** panel is open, showing the **RD\_SYS\_T** instruction under the **Clock functions** category. The **RD\_SYS\_T** instruction is selected, and its properties are displayed in the **Watch and force tables** section. The **Name** is **"DB\_Test".Date\_an...**, the **Display format** is **DATE\_AND...**, and the **Monitor value** is **DT#2013-09-12-10:50:49.706**.

#### RD\_SYS\_T

With **RD\_SYS\_T**, you can read out the current date and the current time of the CPU. The read-out data is provided in DTL format at the **OUT** output of the instruction. The provided value does not include information about the local time zone or daylight savings time. At the **RET\_VAL** output, you can query whether errors have occurred during execution of the instruction.

| Byte | Component   | Data type | Value range                                                                   |
|------|-------------|-----------|-------------------------------------------------------------------------------|
| 0    | Year        | UINT      | 1970 to 2554                                                                  |
| 1    |             |           |                                                                               |
| 2    | Month       | USINT     | 0 to 12                                                                       |
| 3    | Day         | USINT     | 1 to 31                                                                       |
| 4    | Day of week | USINT     | 1(Sunday) to 7(Saturday)<br>The weekday is not considered in the value entry. |
| 5    | Hour        | USINT     | 0 to 23                                                                       |
| 6    | Minute      | USINT     | 0 to 59                                                                       |
| 7    | Second      | USINT     | 0 to 59                                                                       |
| 8    | Nanoseconds | UDINT     | 0 to 999 999 999                                                              |
| 9    |             |           |                                                                               |
| 10   |             |           |                                                                               |
| 11   |             |           |                                                                               |

# Contents

|           |                                                                              |            |
|-----------|------------------------------------------------------------------------------|------------|
| <b>9.</b> | <b>Data Blocks.....</b>                                                      | <b>9-2</b> |
| 9.1.      | Data Blocks and their Usage .....                                            | 9-3        |
| 9.2.      | Meaning of Variables and Data Types.....                                     | 9-4        |
| 9.2.1.    | Overview: Data Types in STEP 7 .....                                         | 9-5        |
| 9.2.2.    | Elementary Data Types .....                                                  | 9-6        |
| 9.2.3.    | Data Types for Timers, Date and Time-of-day .....                            | 9-7        |
| 9.2.4.    | Complex Data Types .....                                                     | 9-9        |
| 9.3.      | Creating a Global Data Block .....                                           | 9-10       |
| 9.3.1.    | DB Attributes: "Optimized Block Access" and "Only Store in Load Memory"..... | 9-11       |
| 9.3.2.    | Editing, Saving, Monitoring a Data Block.....                                | 9-12       |
| 9.3.3.    | Default, Start and Monitoring Values.....                                    | 9-13       |
| 9.3.4.    | Retentiveness, Download DB into the CPU / Upload from the CPU .....          | 9-14       |
| 9.3.5.    | Downloading Changed Data Blocks into the CPU.....                            | 9-15       |
| 9.3.6.    | HMI Access.....                                                              | 9-16       |
| 9.4.      | Copy & Paste from / to Microsoft Excel .....                                 | 9-17       |
| 9.5.      | Exercise 1: Creating Data Block "DB_OP" (DB99).....                          | 9-18       |
| 9.5.1.    | Exercise 2: Using DB Variables (Tags) in the Program.....                    | 9-19       |
| 9.6.      | Additional Information .....                                                 | 9-20       |
| 9.6.1.    | Example of a Variable of the Data Type ARRAY .....                           | 9-21       |
| 9.6.2.    | Example of a Variable of the Data Type STRUCTURE.....                        | 9-22       |

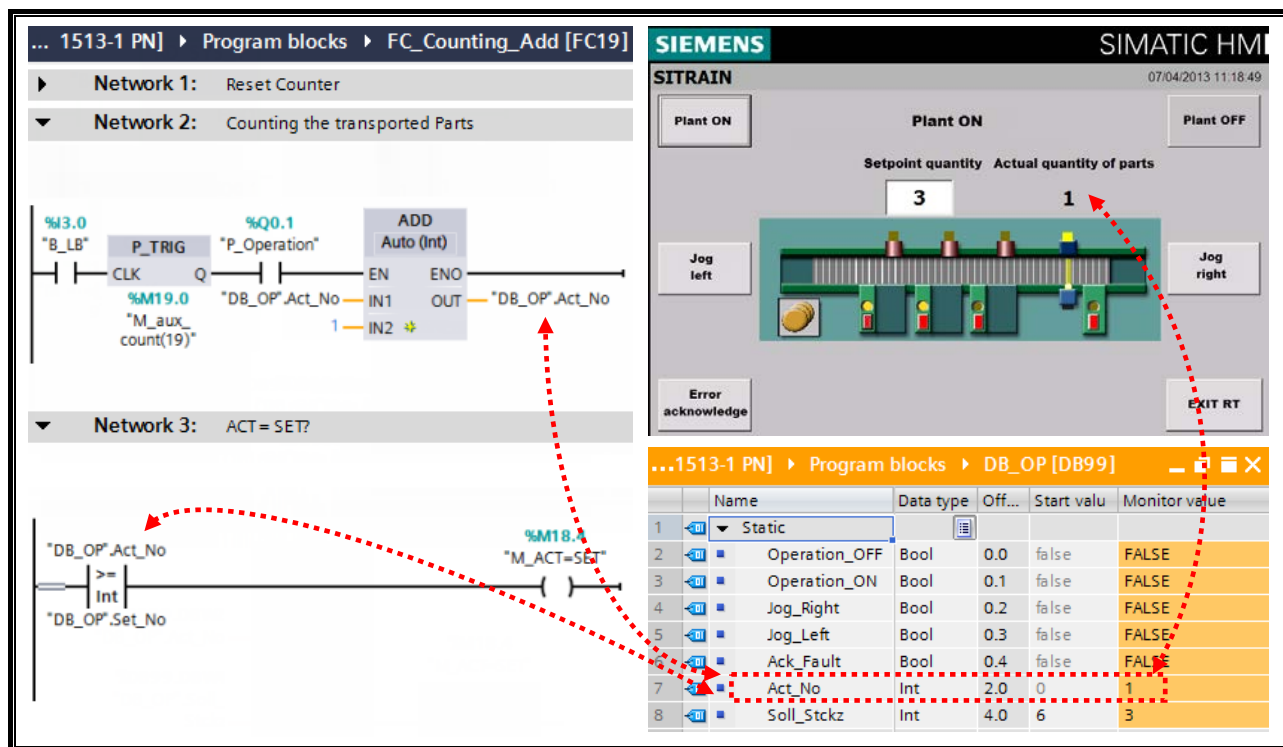
## 9. Data Blocks

**At the end of the chapter the participant will ...**

- ... understand the purpose of global data blocks
- ... be familiar with elementary and complex data types
- ... be able to edit a data block with variables and use this in the program
- ... be familiar with the possibilities for addressing data block variables
- ... be able to monitor a data block



## 9.1. Data Blocks and their Usage



### Data Blocks and their Usage

Data blocks contain variables for storing user data and accordingly occupy memory space in the work memory of the CPU. Since the CPU program as well as HMI devices can read and write access the data block variables (tags), data blocks are frequently also used as an "interface" between the CPU and HMI devices.

### Area of Application

You can use data blocks in different ways, depending on their contents. You differentiate between:

- **Global data blocks:** These contain information that all the logic blocks in the user program can access. Often, global data blocks are also used as an interface to HMI devices (see picture)
- **Instance data blocks:** These are always assigned to a particular FB. The data of these instance DBs should only be processed by the associated FB. Instance data blocks are dealt with in more detail in the chapter "Functions and Function Blocks".

### Creating DBs

Global DBs are created either with the Program Editor or according to a previously created user-defined data type (UDT). Instance data blocks are generated by the Editor according to a function block.

## 9.2. Meaning of Variables and Data Types

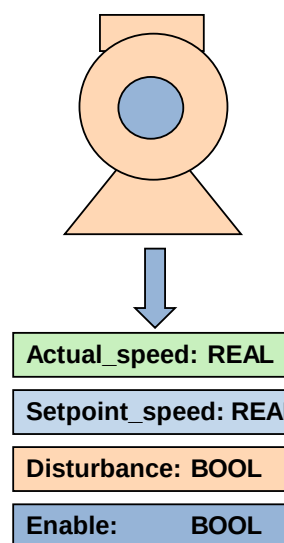
**Data types characterize the basic properties of data**

- Continuous area: e.g. Actual speed
- "yes/no" properties: e.g. Disturbance

**The data type establishes:**

- The possible value range (INT: -32 768 to +32 767, etc.)
- The allowed instructions (math instructions: +, -, etc.)
- Data types abstract from the underlying representation of the bits in the memory

**Variables permit you to save and later continue to process values**



### "Traditional" PLC Operands

In traditional PLC programming, PLC memory addresses were accessed directly by specifying the operand area (for example: M= bit memory, I=inputs, etc.), the access length (for example: B=byte, W=word, etc.) and by specifying the byte/ (bit) address. These memory areas addressed via absolute addresses could be used within the program for different purposes, for example, as integer (e.g. DINT), as floating-point number (e.g. REAL) or simply as a collection of individual signals (e.g. WORD). Up until now, it was up to the programming person to make a mental note of the format and the purpose of the individual memory locations. Accordingly the programming was prone to errors.

### The Meaning of Variables

Next to commands, variables are the most important elements of a programming system. Their task is to save values in a program so that they can be further processed at a later time. The value of a variable can be saved "anywhere" in the PLC memory.

The data represents an abstraction of reality in which irrelevant properties of objects are ignored.

### Data Types

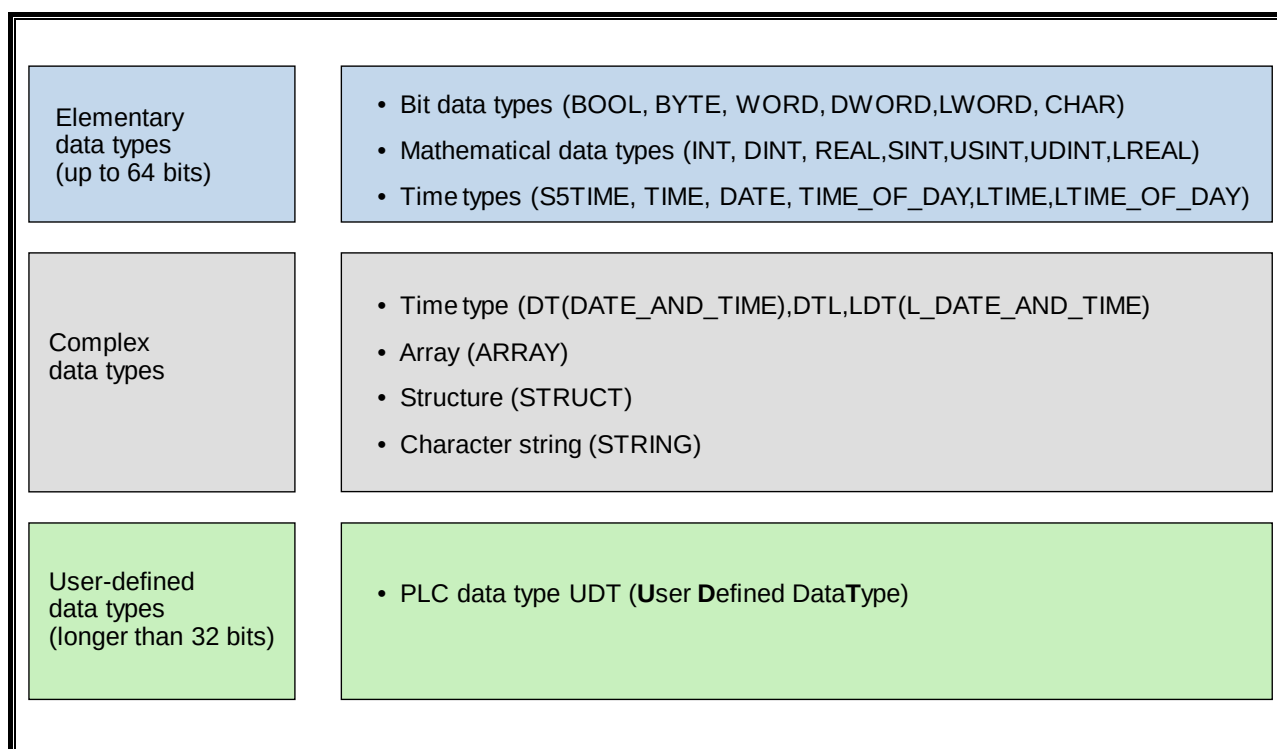
It is often quite difficult to decide how data is to be represented and the available possibilities quite often restrict the choice. On the one hand, the object properties the data describe must be correctly reflected. On the other hand, it must also be possible to carry out the instructions necessary for process with the data.

The data type determines which values are accepted by data and which instructions can be carried out with these values. The data type uniquely defines

- the possible value range
- the allowed instructions



### 9.2.1. Overview: Data Types in STEP 7



#### Elementary Data Types

Elementary data types are predefined in accordance with IEC 61131-3. They always have a length less than or equal to 64 bits and can be further processed with elementary STEP 7 instructions.

#### Complex Data Types

Complex data types contain data structures that can be made up of elementary and/or complex data types. Complex data types can be used for the declaration of variables only in global data blocks and within blocks for the declaration of local variables (TEMP, STAT) as well as parameters (IN, OUT and INOUT). Variables of complex data types cannot be completely processed with elementary instructions (such as, A, O, L, T, +I) but only the individual components of the elementary data type.

#### PLC Data Types

PLC data types are data types defined by you that are used as templates for declaring parameters and variables of complex data types (e.g. structure variables). UDTs are created with the Data Block Editor and contain a data structure that is made up of elementary and/or complex data types. In the declaration of a variable according to data type UDTx, a structure variable is created whose inner data structure is defined by a UDT. UDTs can be used for the declaration of variables in global data blocks and within blocks for the declaration of local variables (TEMP, STAT) as well as parameters (IN, OUT and INOUT).

## 9.2.2. Elementary Data Types

| Data type | Length (in bits) | Constants        | Variables |
|-----------|------------------|------------------|-----------|
| BOOL      | 1                | 1 or 0           | I 1.0     |
| BYTE      | 8                | B#16#A9          | MB70      |
| WORD      | 16               | W#16#12AF        | MW72      |
| DWORD     | 32               | DW#16#ADAC1EF5   | QD40      |
| LWORD     | 64               | LW#16#5F52DE8B   |           |
| CHAR      | 8                | ' w '            | DBB4      |
| INT       | 16               | 123              | #Value    |
| DINT      | 32               | L#65539          | MD80      |
| REAL      | 32               | 1.2 or 34.5E-12  | DBD60     |
| SINT      | 8                | +/-50            | MB24      |
| USINT     | 8                | 50               | MB24      |
| UDINT     | 32               | 4875678          | DBD64     |
| LREAL     | 64               | LREAL#1.0e-5     |           |
| LINT      | 64               | LINT#+1543258759 |           |

### BOOL, BYTE, WORD, DWORD, CHAR

Variables of the data type BOOL consist of one bit. Variables of the data types BYTE, WORD, DWORD are bit sequences of 8, 16 or 32 bits. The individual bits are not evaluated in these data types. Special forms of these data types are the BCD numbers and the count value as it is used in conjunction with the count function as well as the data type CHAR which represents a character in the ASCII code.

### INT, DINT, REAL

Variables of these data types represent numbers that can be used in mathematical operations.

### Extensions of INT, DINT, REAL, WORD

#### U - Unsigned

Variables with the extension U represent an integer without sign.

**Data types:** USINT, UINT, ULINT, UDINT

#### S - Short

Variables with the extension S represent an integer with a length of 8 bits.

**Data types:** SINT, USINT

#### L - Long

Variables with the extension L represent a number with a length of 64 bits of the data type.

**Data types:** LWORD, LINT, ULINT, LREAL

### 9.2.3. Data Types for Timers, Date and Time-of-day

| Data type                   | Length (in bits) | Example                           |
|-----------------------------|------------------|-----------------------------------|
| <b>Timers</b>               |                  |                                   |
| S5TIME                      | 16               | S5T#5s                            |
| TIME                        | 32               | T#2h46m30s630ms                   |
| LTIME                       | 64               | LT#11350d20h25m14s830ms652μs315ns |
| <b>Date and Time-of-day</b> |                  |                                   |
| DATE                        | 32               | D#1984-01-01                      |
| TIME_OF_DAY (TOD)           | 32               | TOD#18:15:18:999                  |
| L_TIME_OF_DAY (LTOD)        | 64               | LTOD#10:20:30.400_365_215         |
| DATE_AND_TIME (DT)          | 64               | DT#1984-01-21-13-45-56-567-3      |
| DATE_AND_LTIME (LDT)        | 64               | LDT#2008-01-01-18:00:30:25        |
| DTL                         | 96               | DTL#1984-01-01-18:00:30:250       |

#### S5TIME

Variables of the data type S5TIME are required for specifying time values in timer functions (S5-timing elements). You specify the time in hours, minutes, seconds or milliseconds. You can enter the timer values with an underline (1h\_4m) or without an underline (1h4m).

#### TIME

A variable of the data type TIME (duration in [ms]) occupies a double word. This variable is used, for example, for specifying time values in IEC timer functions. The contents of the variable are interpreted as a DINT number in milliseconds and can be either positive or negative (for example: T#1s=L#1 000, T#24d20h31m23s647ms = L#2147486470).

#### LTIME

Just like TIME, LTIME represents a duration whereby the duration is saved with nanosecond resolution in 64 bits. This means that in variables of the data type LTIME, longer durations with greater resolution can be saved.

#### DATE

A variable of the data type DATE is stored in a word in the form of an unsigned integer. The contents of the variable represent the number of days since 01.01.1990 (for example: D#2003-10-31 = W#16#13BB).

#### TOD

The data type TOD (TIME\_OF\_DAY) occupies a double word and stores the number of milliseconds since the beginning of the day (0:00 o'clock) as an unsigned integer.

#### LTOD

Variables of the data type LTOD occupy two double words and store the number of nanoseconds since the beginning of the day.

**DT**

The data type DATE\_AND\_TIME represents a point in time consisting of the date and the time-of-day. Instead of DATE\_AND\_TIME, the abbreviation DT can also be used.

| Byte                   | Contents                                | Range of values                                                                                 |
|------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------|
| 0                      | Year                                    | 0 to 99<br>(Years 1990 to 2089)<br>BCD#90 = 1990<br>...<br>BCD#0 = 2000<br>...<br>BCD#89 = 2089 |
| 1                      | Month                                   | BCD#0 to BCD#12                                                                                 |
| 2                      | Day                                     | BCD#1 to BCD# 31                                                                                |
| 3                      | Hour                                    | BCD#0 to BCD#23                                                                                 |
| 4                      | Minute                                  | BCD#0 to BCD#59                                                                                 |
| 5                      | Second                                  | BCD#0 to BCD#59                                                                                 |
| 6                      | The two most significant digits of MSEC | BCD#0 to BCD#999                                                                                |
| 7 (4MSB) <sup>1)</sup> | The least significant digit of MSEC     | BCD#0 to BCD#9                                                                                  |
| 7 (4LSB) <sup>2)</sup> | Weekday                                 | BCD#1 to BCD#7<br>BCD#1 = Sunday<br>...<br>BCD#7 = Saturday                                     |

**LDT**

The data type LDT consists of the data types DATE and LONGTIME and stores the date and time in nanoseconds since 1.1.1970.

**DTL**

The data type DTL has a length of 12 bytes and, like LDT, stores information on date and time-of-day precise to the nanosecond since 1.1.1970, only in a pre-defined structure.

**Advantage:** the individual values (day, hour, etc.) are easier to read out.

| Byte   | Component  | Data type | Range of values                                                               |
|--------|------------|-----------|-------------------------------------------------------------------------------|
| 0 – 1  | Year       | UINT      | 1970 to 2554                                                                  |
| 2      | Month      | USINT     | 1 to 12                                                                       |
| 3      | Day        | USINT     | 1 to 31                                                                       |
| 4      | Weekday    | USINT     | 1(Sunday) to 7(Saturday)<br>The weekday is not considered in the value entry. |
| 5      | Hour       | USINT     | 0 to 23                                                                       |
| 6      | Minute     | USINT     | 0 to 59                                                                       |
| 7      | Second     | USINT     | 0 to 59                                                                       |
| 8 – 11 | Nanosecond | UDINT     | 0 to 999999999                                                                |

## 9.2.4 Complex Data Types

| Data type                                                                                          | Length (in bits)              | Example                                                                               |
|----------------------------------------------------------------------------------------------------|-------------------------------|---------------------------------------------------------------------------------------|
| <b>STRING</b><br>(character string with max. 254 characters)                                       | 8 * (number of characters +2) | `This is a string`<br>`SIEMENS`                                                       |
| <b>ARRAY</b><br>(Group of components of the same data type)                                        | user-defined                  | Measured values: ARRAY[1..20] of INT                                                  |
| <b>STRUCT</b><br>(Structure, Group of components of different data types)                          | user-defined                  | Motor: STRUCT<br>Speed : INT<br>Current : REAL<br>END_STRUCT                          |
| <b>UDT</b><br>(User Defined Data Type = "Template" consisting of elementary or complex data types) | user-defined                  | UDT as block      UDT as array element                                                |
|                                                                                                    |                               | STRUCT<br>Speed : INT<br>Current : REAL<br>END_STRUCT      Drive: ARRAY[1..4] of UDT1 |

### Complex Data Types

Complex data types (arrays and structures) consist of groups of elementary or complex data types.

They enable you to create data types suitable for your problem with which you can structure large quantities of data and process it symbolically.

Complex data types cannot be processed directly with STEP 7 instructions all at once. Only one component at a time can be processed.

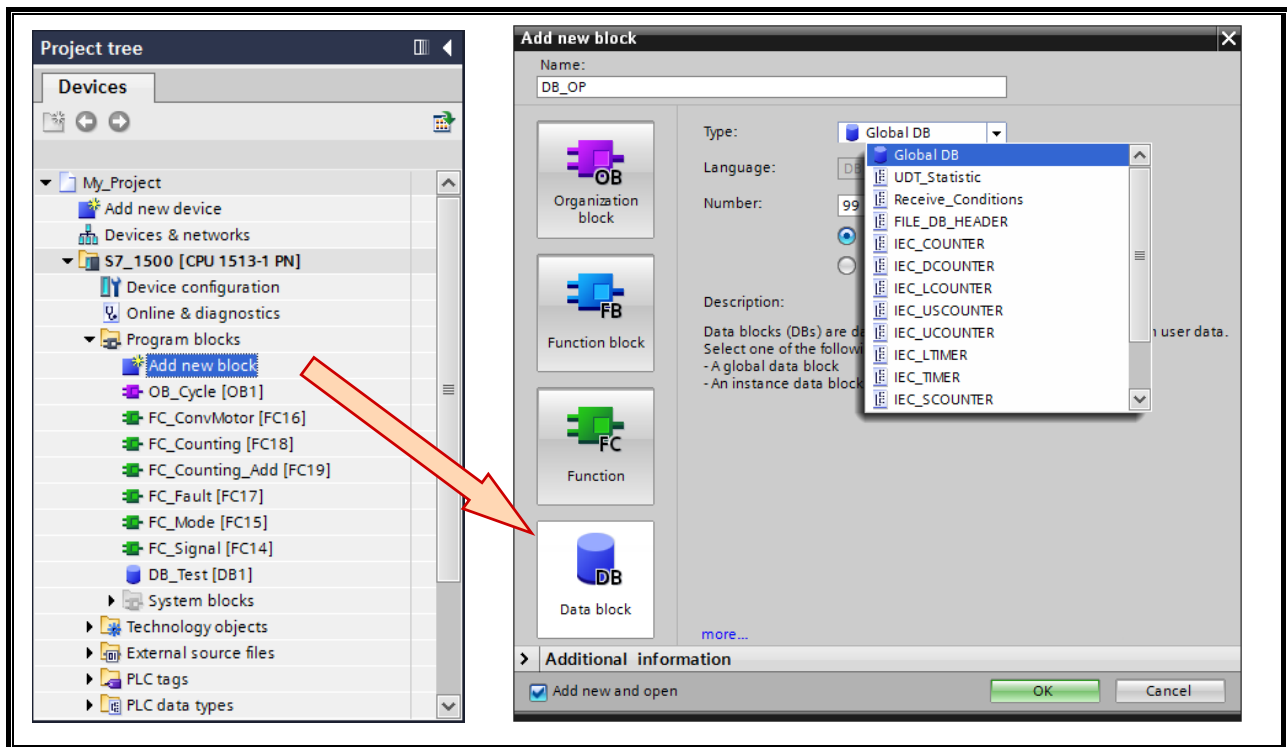
Complex data types are predefined. The data type DATE\_AND\_TIME has a length of 64 bits. The lengths of the data types ARRAY, STRUCT and STRING are defined by the user.

Variables with complex data types can only be declared within global data blocks and as parameters or local variables of logic blocks.

### User-defined Data Type (UDT)

User-defined data types represent self-defined structures. This structure is stored in UDT blocks (UDT1 to UDT65535) and can be used as a "template" in another variable's data type. You can save typing time when you input a data block if you need the same structure several times.

### 9.3. Creating a Global Data Block



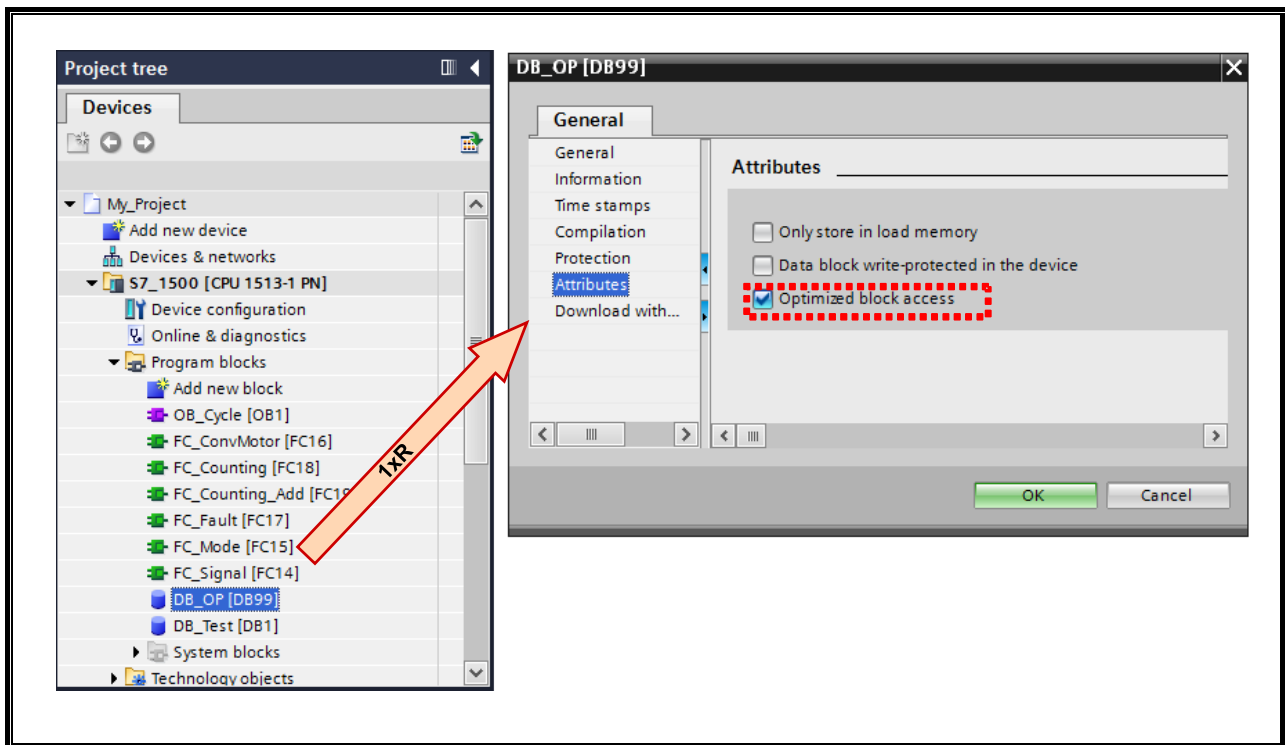
#### Creating a Data Block

Data blocks are created in the same way as logic blocks.

In creating a data block, you can choose which type the data block is to be or what purpose it is to serve:

- Global DB for storing global data or for creating global variables
- Instance DB or "private memory area" for a user function block or a special "instruction", behind which a system function block (SFB) is ultimately hidden.

### 9.3.1. DB Attributes: "Optimized Block Access" and "Only Store in Load Memory"



#### Attribute "Optimized Block Access"

The "Optimized block access" attribute has the following impact on data blocks:

- The variables (tags) of the data block can only be accessed symbolically
- The retentive behavior of the variables (tags) can be adjusted tag-by-tag
- With S7-1200, these DBs are created memory-optimized
- With S7-1500, these DBs are created access-optimized

#### Attribute "Only Store in Load Memory" ("unlinked")

This attribute means that when the data block is downloaded into the CPU, it is only loaded into the load memory of the CPU (read-only) and from there is not automatically adopted in the work memory.

### 9.3.2. Editing, Saving, Monitoring a Data Block

**DB\_Weights**

|    | Name           | Data type              | Offset | Default value | Start value | Snapshot | Monitor value | Retain                              | Visible in HMI                      |
|----|----------------|------------------------|--------|---------------|-------------|----------|---------------|-------------------------------------|-------------------------------------|
| 1  | Static         | "UDT_Statistic"        | 0.0    | 10            |             |          | 10            | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 2  | Weight_Store1  | "UDT_Statistic"        | 0.0    | 10            |             |          | 0             | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 3  | Act_No         | Int                    | 2.0    | 0             | 0           | 0        | 0             | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 4  | Part_Weight    | Array [1 .. 10] of Int | 4.0    |               |             |          |               | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 5  | Weight_Store2  | "UDT_Statistic"        | 24.0   |               |             |          |               | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 6  | Max_No         | Int                    | 0.0    | 10            | 10          | 10       | 10            | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 7  | Act_No         | Int                    | 2.0    | 0             | 0           | 5        | 8             | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 8  | Part_Weight    | Array [1 .. 10] of Int | 4.0    |               |             |          |               | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 9  | Part_Weight[1] | Int                    | 0.0    | 0             | 0           | 236      | 236           | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 10 | Part_Weight[2] | Int                    | 2.0    | 0             | 0           | 286      | 286           | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 11 | Part_Weight[3] | Int                    | 4.0    | 0             | 0           | 388      | 388           | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 12 | Part_Weight[4] | Int                    | 6.0    | 0             | 0           | 160      | 160           | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 13 | Part_Weight[5] | Int                    | 8.0    | 0             | 0           | 212      | 212           | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 14 |                |                        |        |               |             | 0        | 282           | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 15 |                |                        |        |               |             | 0        | 379           | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 16 |                |                        |        |               |             | 0        | 379           | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| 17 |                |                        |        |               |             |          |               | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |

#### Reset Start Values

This means that all start values are to be overwritten or initialized with the default values.

#### Visible in HMI

A PLC tag with the attribute "not visible" is not shown in HMI dialogs in the tag selection. By changing the option, these tags can also be included in the tag selection.

#### Retain

This is explained in more detail on one of the following pages.

- for DBs with "Optimized block access", the retentiveness can be set tag-specific
- for DBs without "Optimized block access", the retentiveness can only be set for all tags.

#### Offset

The offset denotes the absolute address of a variable (tag) within a DB. In the STEP7 program, the use of the symbolic address or name is preferable since it is easier to read and less prone to errors than the absolute addressing. There are also tasks, however, that can only be solved from a program standpoint with the use of absolute addresses (-> indirect addressing).

The absolute addresses of variables within the data block are always displayed in S7-300/400; in S7-1200 and S7-1500, only if the data block was created without the option "optimized block access".

#### Start Value

This is the value with which the CPU starts program execution, after the data block has been loaded into the CPU.

#### Snapshot

With a snapshot, the "monitoring values" or the current values at the end of cycle x online are transferred from the CPU into the offline project.



### 9.3.3. Default, Start and Monitoring Values

**Offline:**  
  
 Default values  
+  
Start values  
+  
Snapshot values

**Online:**  
  
 In the load memory:  
Default + Start values  
  
 In the work memory:  
Monitor values

My\_Project > S7\_1500 [CPU 1513-1 PN] > Program blocks > DB\_OP [DB99]

DB\_OP

|   | Name          | Data type | Default value | Start value | Snapshot | Monitor value |
|---|---------------|-----------|---------------|-------------|----------|---------------|
| 1 | Static        |           |               |             |          |               |
| 2 | Operation_OFF | Bool      | false         | false       | false    | FALSE         |
| 3 | Operation_ON  | Bool      | false         | false       | false    | FALSE         |
| 4 | Jog_Right     | Bool      | false         | false       | false    | FALSE         |
| 5 | Jog_Left      | Bool      | false         | false       | false    | FALSE         |
| 6 | Ack_Fault     | Bool      | false         | false       | false    | FALSE         |
| 7 | Act_No        | Int       | 0             | 0           | 1        | 2             |
| 8 | Set_No        | Int       | 0             | 6           | 3        | 3             |

Default values

Start values

Monitor values

Snapshot

#### Default Value

Default values cannot be edited within global data blocks. Only the default values of structure elements within PLC data types (UDTs) and the default values of parameters and static variables within FBs can be edited.

If a variable of the data type Structure is declared according to the PLC data type (UDTx) within a global DB, the default values edited in the UDT are then displayed in the data block (cannot be edited) and adopted as (editable) start values.

Within an FB, their default values also have to be specified in the declaration of parameters and static variables. In the instance DBs of the FB, these default values (cannot be edited) are then displayed and adopted as (editable) start values.

#### Start Value

In declaring a variable, the default value of a variable is automatically adopted as the start value; however, this value can be overwritten at any time and loaded into the controller. After a data block has been loaded into the CPU, the CPU starts the program execution with exactly this value.

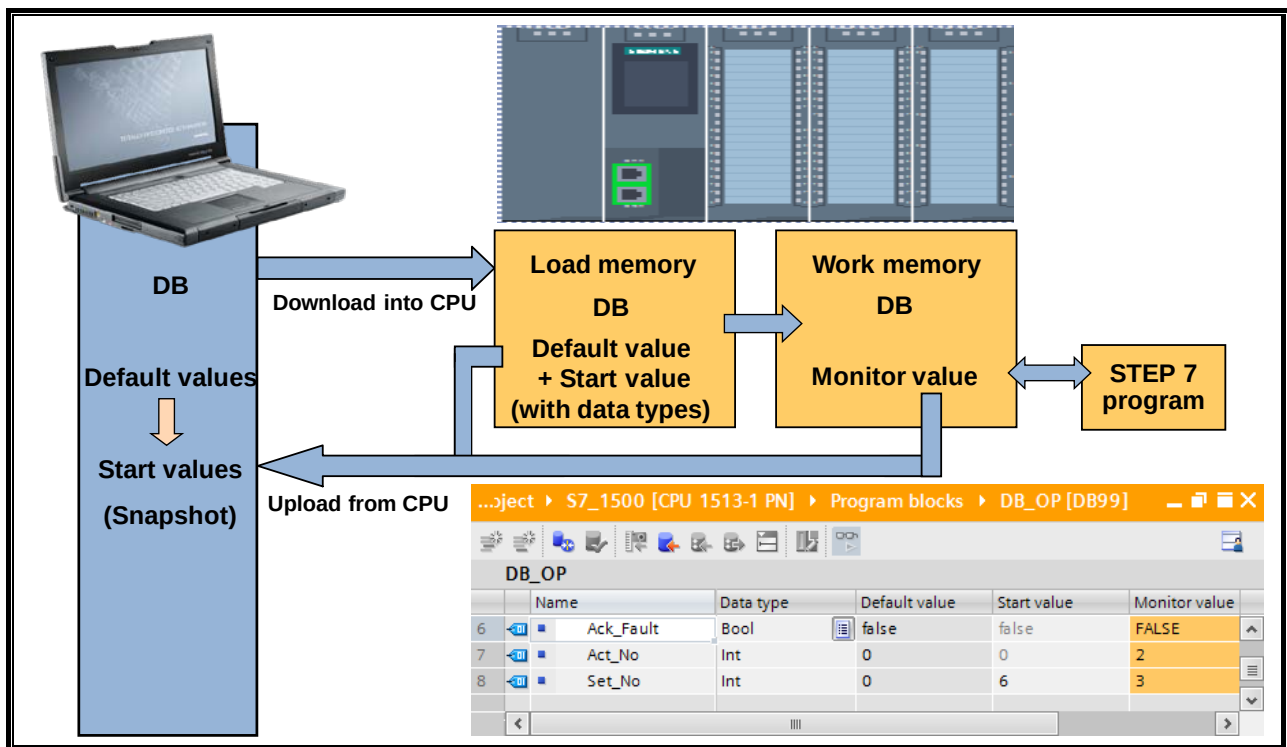
#### Monitoring Value

The monitoring value of a variable is the current value that the variable presently has in the work memory of the CPU.

#### Snapshot

The value "Snapshot" is a monitoring value for an already passed point in time x, at which the current monitoring values are read out of the CPU and stored offline as the values of the "Snapshot".

### 9.3.4. Retentiveness, Download DB into the CPU / Upload from the CPU



#### Download

In downloading a data block from the PG into the CPU, the default and start values as well as the data types of the variables are transferred into the load memory of the CPU. From the load memory, the start values are then automatically adopted as monitoring values in the work memory of the CPU. The STEP 7 program works with the monitoring values of the work memory.

#### Upload

In uploading a data block from the CPU into the PG, the default values from the load memory and the current monitoring values from the work memory are transferred into the PG. In doing so, the current monitoring values are adopted offline as start values.

#### Retentiveness (Retain)

With retentive variables, the monitoring values (actual values) are retained after a CPU restart; non-retentive variables are reset to their start values.

Non-retentive variables occupy work memory but no retentive memory and are thus reset to start values from the load memory with every CPU restart (Power→Off→On or after every STOP – RUN).

### 9.3.5. Downloading Changed Data Blocks into the CPU

The diagram illustrates the process of downloading changed data blocks into the CPU, showing three scenarios:

- Start value changed and DB downloaded in CPU:** This scenario shows a table where the start value of 'Value1' is changed from 0 to 1. The monitor value is 23. A red arrow points to the 'Start value' column with the text 'Start value changed and DB downloaded in CPU'. A red dashed circle around the 'Monitor value' column is labeled 'No initialization'.
- Structural change:** This scenario shows a table where the name of the third data block is changed from 'Value2' to 'Result'. The start value is 0 and the monitor value is 0. A red arrow points to the 'Name' column with the text 'Structural change: Name change: Download DB in CPU is only possible with re-initialization'. A red dashed circle around the 'Monitor value' column is labeled 'Initialization of all tags'.
- Restart carried out:** This scenario shows a table where the start value of 'Value1' is changed from 1 to 1 (no change) and the start value of 'Value2' is changed from 2 to 2 (no change). The monitor values are 1 and 45. A red arrow points to the 'Start value' column with the text 'Restart carried out'. A red dashed circle around the 'Monitor value' column is labeled 'Initialization of non-retentive tags'.

#### Downloading Data Blocks with Re-initialization

No matter whether a DB was created with standard or optimized block access, a change of start values is not a structural change. Therefore, it can be done and downloaded into the CPU without a re-initialization of the data block variables (tags) being necessary.

On the other hand, after structural changes, it is only possible to download the data block by re-initializing all tags.

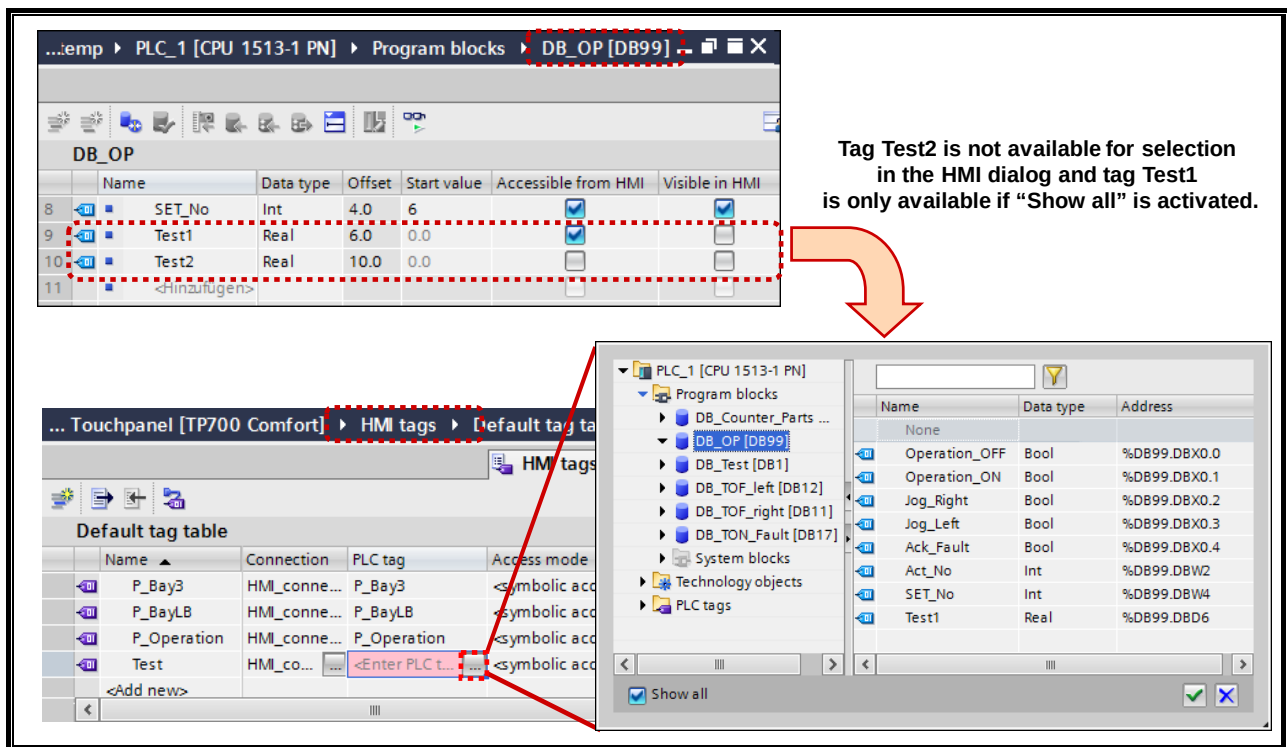
Structural changes are:

- Name changes
- Changes of the retentive behavior
- Adding / removing tags

#### DB Re-initialization during Restart

No matter whether a DB is created with standard or optimized block access, all monitor values (actual values) of non-retentive tags are overwritten with their start values when the CPU is restarted.

### 9.3.6. HMI Access



Tag Test2 is not available for selection in the HMI dialog and tag Test1 is only available if "Show all" is activated.

| Name         | Data type | Offset | Start value | Accessible from HMI                 | Visible in HMI                      |
|--------------|-----------|--------|-------------|-------------------------------------|-------------------------------------|
| SET_No       | Int       | 4.0    | 6           | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |
| Test1        | Real      | 6.0    | 0.0         | <input checked="" type="checkbox"/> | <input type="checkbox"/>            |
| Test2        | Real      | 10.0   | 0.0         | <input type="checkbox"/>            | <input type="checkbox"/>            |
| <Hinzufügen> |           |        |             |                                     |                                     |

| Name          | Data type | Address      |
|---------------|-----------|--------------|
| None          |           |              |
| Operation_OFF | Bool      | %DB99.DBX0.0 |
| Operation_ON  | Bool      | %DB99.DBX0.1 |
| Jog_Right     | Bool      | %DB99.DBX0.2 |
| Jog_Left      | Bool      | %DB99.DBX0.3 |
| Ack_Fault     | Bool      | %DB99.DBX0.4 |
| Act_No        | Int       | %DB99.DBW2   |
| SET_No        | Int       | %DB99.DBW4   |
| Test1         | Real      | %DB99.DBD6   |

#### HMI Access to Tags in DBs

The attributes "Visible in HMI" and "Accessible from HMI" are protective mechanisms used to prevent unwanted accesses to tags by HMI devices:

- "Visible in HMI":

In the HMI configuration, only tags with the attribute "Visible in HMI" can be selected. This filter function can, however, be overridden by activating "Show all" in the selection dialog shown.

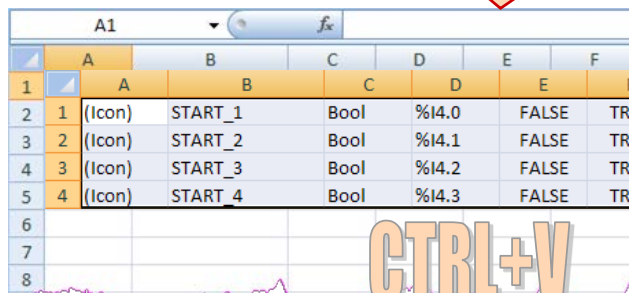
- "Accessible from HMI":

The attribute "Visible in HMI" can only be set when the attribute "Accessible from HMI" is set. However, by activating "Show all", this filter function cannot be overridden.

## 9.4. Copy & Paste from / to Microsoft Excel

### Copy & Paste from / to Excel:

- From the PLC tag table
- From the data blocks



|   | A | B              | C    | D     | E     | F  |
|---|---|----------------|------|-------|-------|----|
| 1 |   |                |      |       |       |    |
| 2 | 1 | (Icon) START_1 | Bool | %I4.0 | FALSE | TR |
| 3 | 2 | (Icon) START_2 | Bool | %I4.1 | FALSE | TR |
| 4 | 3 | (Icon) START_3 | Bool | %I4.2 | FALSE | TR |
| 5 | 4 | (Icon) START_4 | Bool | %I4.3 | FALSE | TR |
| 6 |   |                |      |       |       |    |
| 7 |   |                |      |       |       |    |
| 8 |   |                |      |       |       |    |



my\_project > S7\_1500 [CPU 1516-3 PN/DP] > PLC tags > D

Export to Excel

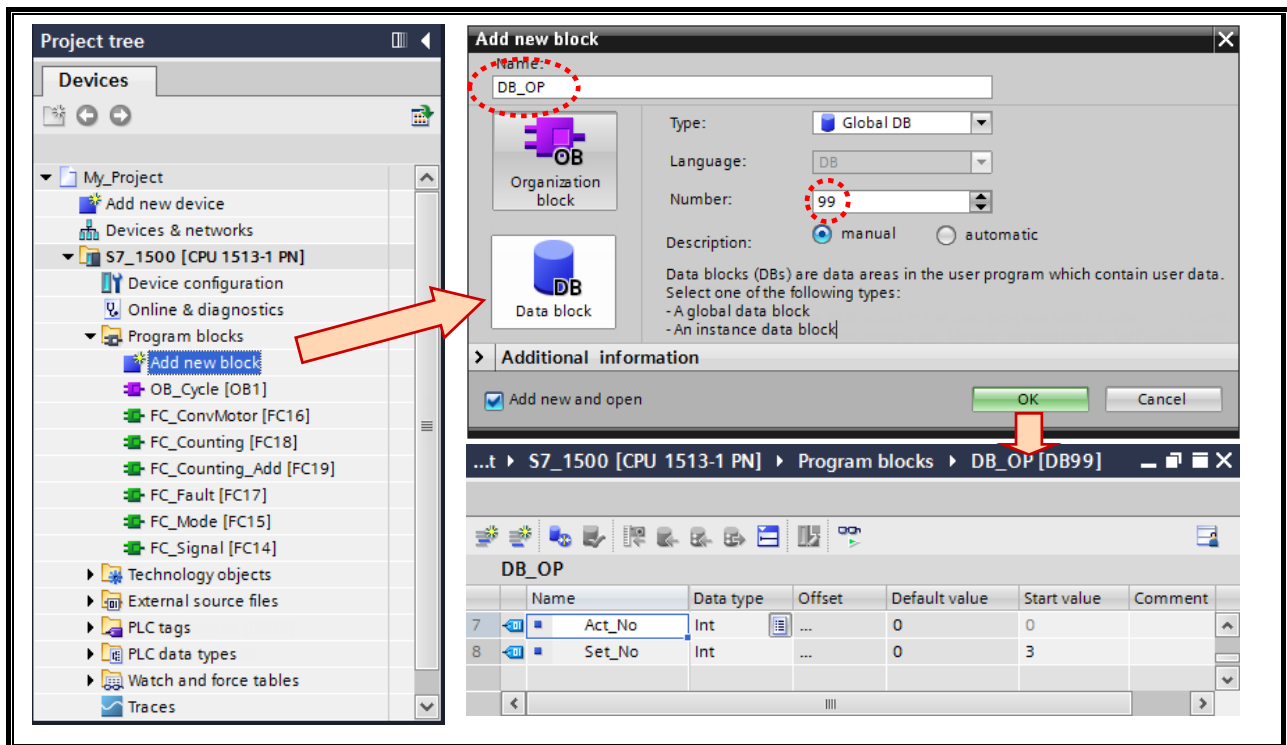
Default tag table

|  | Name    | Data type | Add |
|--|---------|-----------|-----|
|  | START_1 | Bool      | %I4 |
|  | START_2 | Bool      | %I4 |
|  | START_3 | Bool      | %I4 |
|  | START_4 | Bool      | %I4 |

### Copy & Paste from and to Excel

The Windows Copy & Paste function can be used to easily copy individual or several tags from a PLC tag table or a data block to Excel to further process it/them there and then to copy it/them back from Excel to the PLC tag table or the data block.

## 9.5. Exercise 1: Creating Data Block "DB\_OP" (DB99)



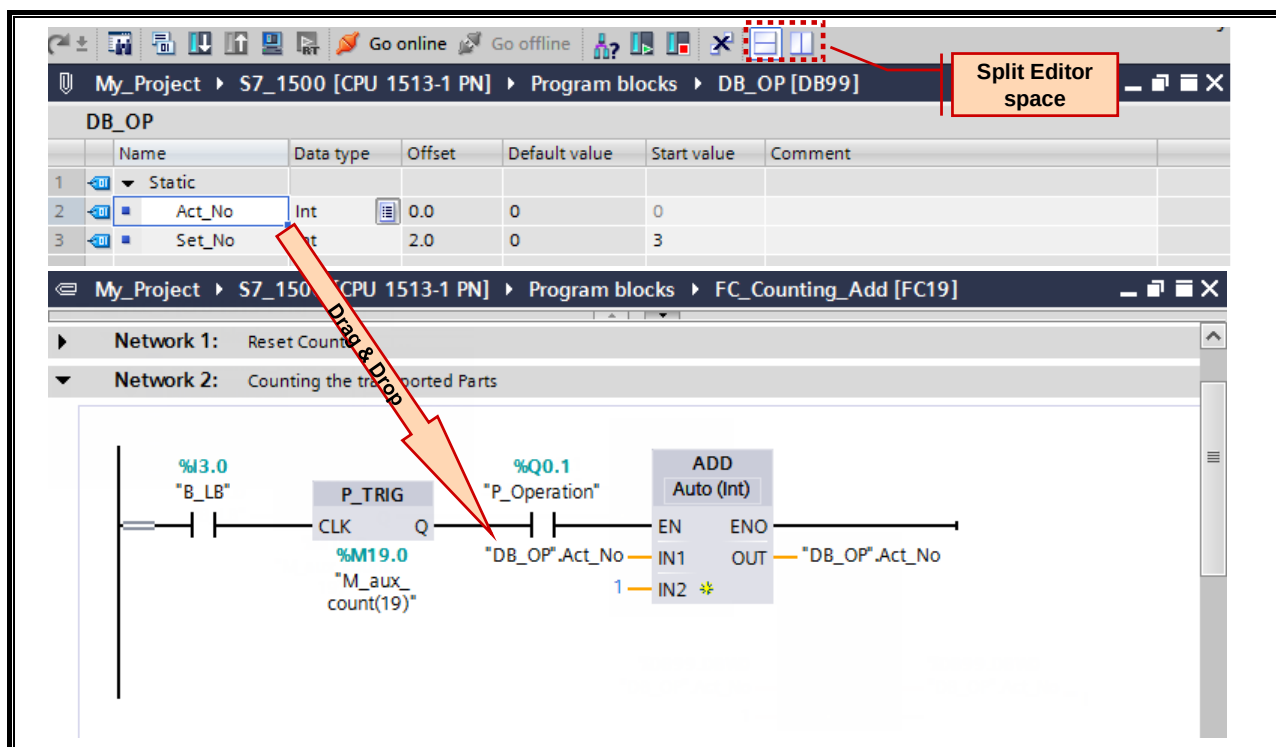
### Task:

The data block "DB\_OP" (DB99) is to be created with the variables (tags) shown in the picture. The variables are to be used later in the STEP7 program and are also to serve as an interface to the touchpanel.

### What to Do:

1. Create the new "DB\_OP" (DB99) as a Global DB.
2. Declare the variables as shown in the picture above. Give the variable "Set\_No" the Start value 3.
3. Save your project.

### 9.5.1. Exercise 2: Using DB Variables (Tags) in the Program



#### Function Up Until Now:

The transported parts in "FC\_Counting\_Add" (FC19) are counted by adding up as soon as they have passed through the light barrier.

The current count or the actual quantity is stored in memory word "MW\_ACT" (MW 20) and the setpoint quantity is preset with constant 3 in the program.

If the actual quantity has reached the setpoint quantity, it is indicated on the conveyor model indicator light "P\_BayLB" (Q3.4) with a 1Hz flashing light and no new transport sequence can be started. By acknowledging this signal with the conveyor model pushbutton "S\_BayLB" (I 3.4), "MW\_ACT" (MW20) is overwritten with 0 and further transport sequences can be started.

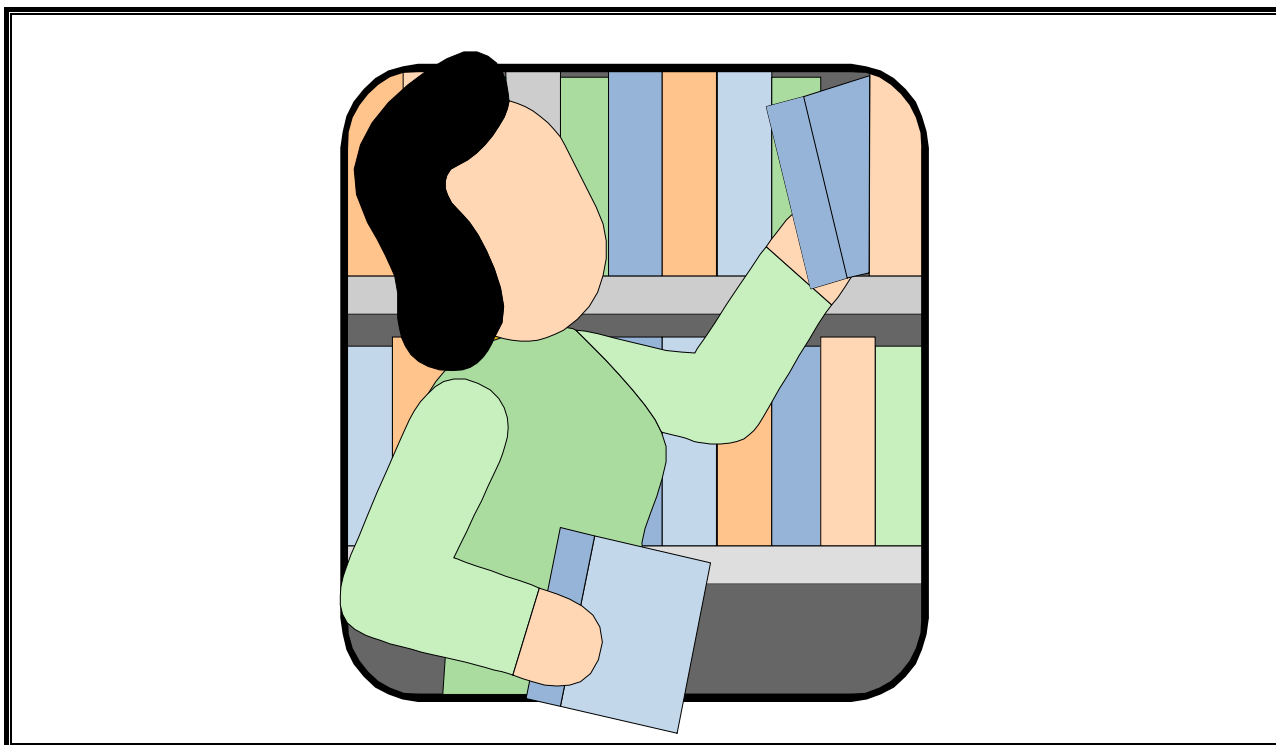
#### Task

The function of "FC\_Counting\_Add" (FC19) remains unchanged, however, the actual quantity is no longer to be stored in the memory variable "MW\_ACT" (MW20) used so far and the setpoint quantity is no longer to be preset with constant 3. Instead, the data block variables (tags) "DB\_OP".Act\_No (DB99.DBW0) and "DB\_OP".Set\_No (DB99.DBW2) are to be used.

#### What to Do:

1. In "FC\_Counting\_Add" (FC19), in all networks, replace the memory variable "MW\_ACT" (MW20) and the setpoint quantity preset with constant 3 with the relevant data block variables (tags) using drag & drop (see picture). Also, split the working area horizontally.
2. Transfer all blocks into the CPU and check the program function.
3. Monitor the data block "DB\_OP" (DB99) while the ACTUAL quantity increases.
4. Save your project.

## 9.6. Additional Information

**Note**

The following pages contain either further information or are for reference to complete a topic.  
For more in-depth study we offer advanced courses and self-learning mediums.



### 9.6.1. Example of a Variable of the Data Type ARRAY

Array with the name "Measuring\_point"  
(several elements of the same data type)

|    | Name                | Data type            | Default value | Start value | Snapshot | Retain                              | Visible in HMI                      | Comment |
|----|---------------------|----------------------|---------------|-------------|----------|-------------------------------------|-------------------------------------|---------|
| 1  | Static              |                      |               |             |          | <input type="checkbox"/>            | <input type="checkbox"/>            |         |
| 2  | Measuring_point     | Array[1..10] of Real |               |             |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 3  | Measuring_point[1]  | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 4  | Measuring_point[2]  | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 5  | Measuring_point[3]  | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 6  | Measuring_point[4]  | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 7  | Measuring_point[5]  | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 8  | Measuring_point[6]  | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 9  | Measuring_point[7]  | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 10 | Measuring_point[8]  | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 11 | Measuring_point[9]  | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |
| 12 | Measuring_point[10] | Real                 | 0.0           | 0.0         |          | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> |         |

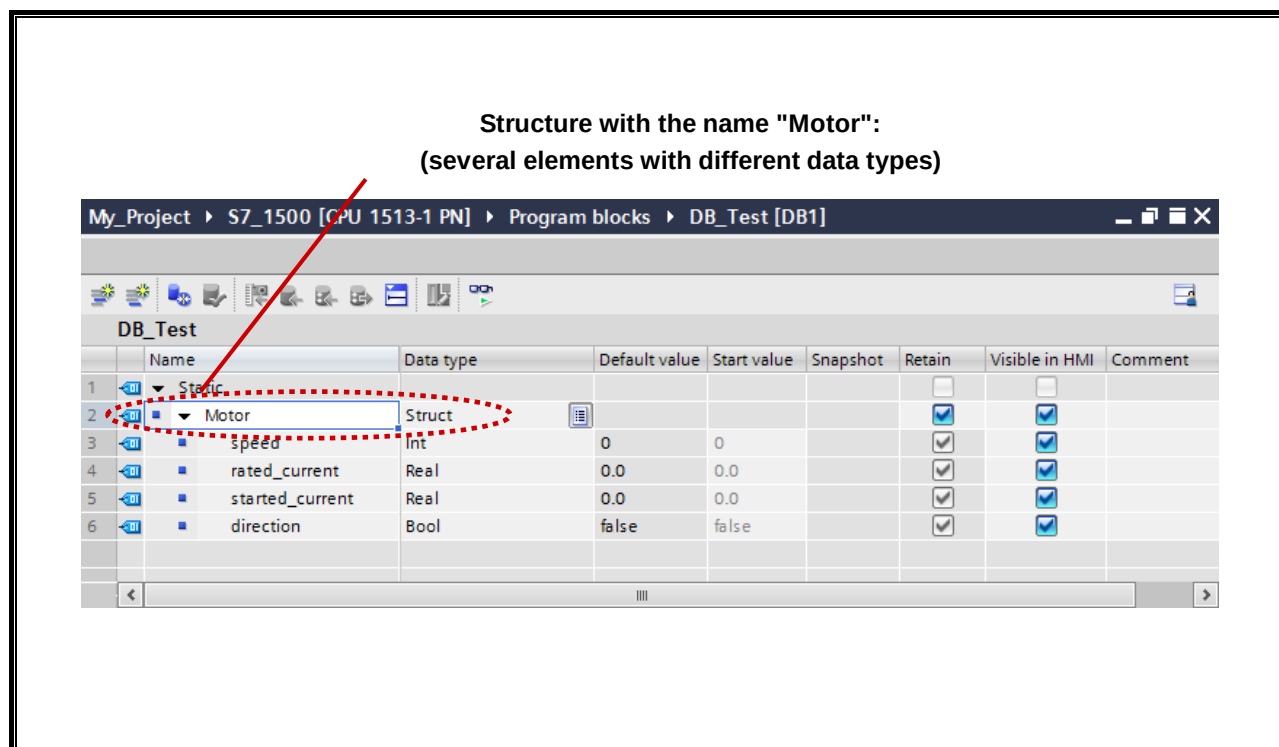
#### Array

An array consists of several elements of the same data type. In the picture above, you can see the array "Measuring\_point" with 10 elements of the data type REAL. Later, various measured values are to be stored in this array.

#### Declaring an Array in the DB

The data type of an array is called "ARRAY[n..m]". The first element (n) and the last element (m) are specified in the square brackets. In the example, [1..10] means 10 elements, whereby the first element is addressed with the index [1] and the last with the index [10]. Instead of [1..10] you could, for example, define [0..9]. This only influences the access to the elements.

## 9.6.2. Example of a Variable of the Data Type STRUCTURE



### Structure

The picture shows an example of a structure named "Motor". The structure consists of several elements of different data types. The individual elements of a structure can be elementary or complex data types.

The access to the individual elements of a structure contains the structure name and the name of the element. This makes the program easier to read.

In order to be able to access the elements symbolically, the data block must be given a symbol name, for example "Drive\_1".

Example: accessing individual elements of a structure:

- A "Drive\_1".Motor.Status
- L "Drive\_1".Motor.Speed

"Drive\_1" is the symbol name of the data block which contains the structure. After the symbol name, (separated by a dot) the structure name is specified. After the structure name (separated by a dot) an element name of the structure follows.

### Declaring a Structure in the DB

As a keyword for a structure, "STRUCT" is used. The end of the structure is identified with "END\_STRUCT". A name is defined for the structure (in the example: "Motor").

# Contents

|                                                                                                    |             |
|----------------------------------------------------------------------------------------------------|-------------|
| <b>10. Distributed I/O.....</b>                                                                    | <b>10-2</b> |
| 10.1. Task Description: Operating the Conveyor Model via the ET200SP Distributed I/O.....          | 10-3        |
| 10.2. Fieldbus Systems for SIMATIC S7 .....                                                        | 10-4        |
| 10.3. Identification of Distributed I/O Devices.....                                               | 10-5        |
| 10.4. Components of the PROFINET Standard.....                                                     | 10-6        |
| 10.4.1. PROFINET Communication Model.....                                                          | 10-7        |
| 10.4.2. PROFINET IO Device Types .....                                                             | 10-8        |
| 10.4.3. PROFINET Addresses.....                                                                    | 10-9        |
| 10.5. Inserting and Networking Distributed I/O .....                                               | 10-10       |
| 10.5.1. PROFINET IO Device ET200SP: Assigning the IP Address and Device Name <u>OFFLINE</u> .....  | 10-11       |
| 10.5.2. PROFINET IO Device ET200SP: Assigning the Device Name <u>ONLINE</u> .....                  | 10-12       |
| 10.5.3. Exercise 1: ET200SP: Reset to Factory Settings .....                                       | 10-13       |
| 10.5.4. Exercise 2: Reading-out the Firmware Version of the ET200SP.....                           | 10-14       |
| 10.5.5. Exercise 3: Offline Project: Adding the ET200SP.....                                       | 10-15       |
| 10.5.6. Exercise 4: Networking the ET200SP.....                                                    | 10-16       |
| 10.5.7. Exercise 5: Configuring and Parameterizing the ET200SP .....                               | 10-17       |
| 10.5.8. Exercise 6: Setting the Channel Parameters of the AI Modules (ET200SP and Central AI)..... | 10-18       |
| 10.5.9. Exercise 7: ET200SP: Assigning the IP Address / PROFINET Name <u>OFFLINE</u> .....         | 10-19       |
| 10.5.10. Exercise 8: ET200SP: Assigning the PROFINET Name <u>ONLINE</u> .....                      | 10-20       |
| 10.5.11. Exercise 9: Compiling the HW Configuration and Downloading it in the CPU .....            | 10-21       |
| 10.5.12. Exercise 10: Adjusting the S7 Program via "Rewiring".....                                 | 10-22       |
| 10.5.13. Exercise 11: Function Test with Conveyor Model via Distributed I/O.....                   | 10-23       |
| 10.6. Additional Information .....                                                                 | 10-24       |
| 10.6.1. Installing Distributed I/O Components Later On .....                                       | 10-25       |

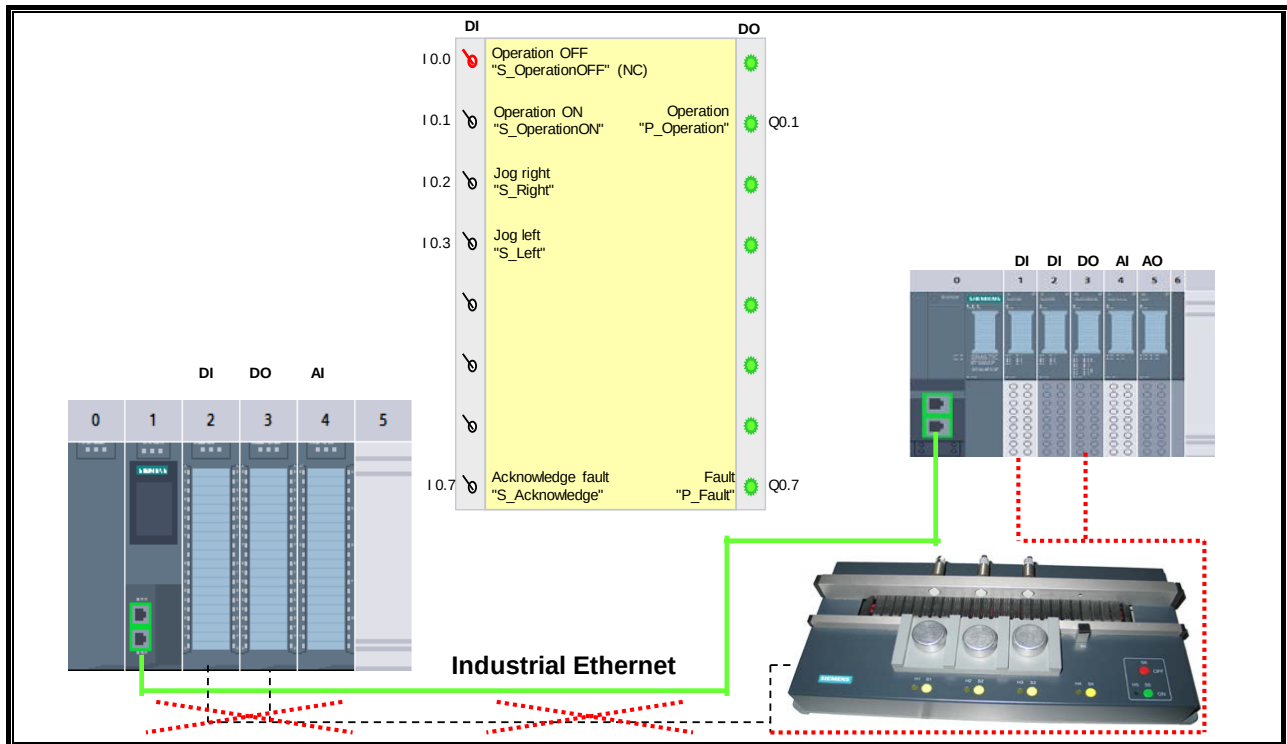
## 10. Distributed I/O

**At the end of the chapter the participant will ...**

- ... be familiar with the distributed I/O systems PROFIBUS and PROFINET
- ... be able to explain the functional principle of PROFINET
- ... be able to configure, network and commission a distributed PROFINET I/O station



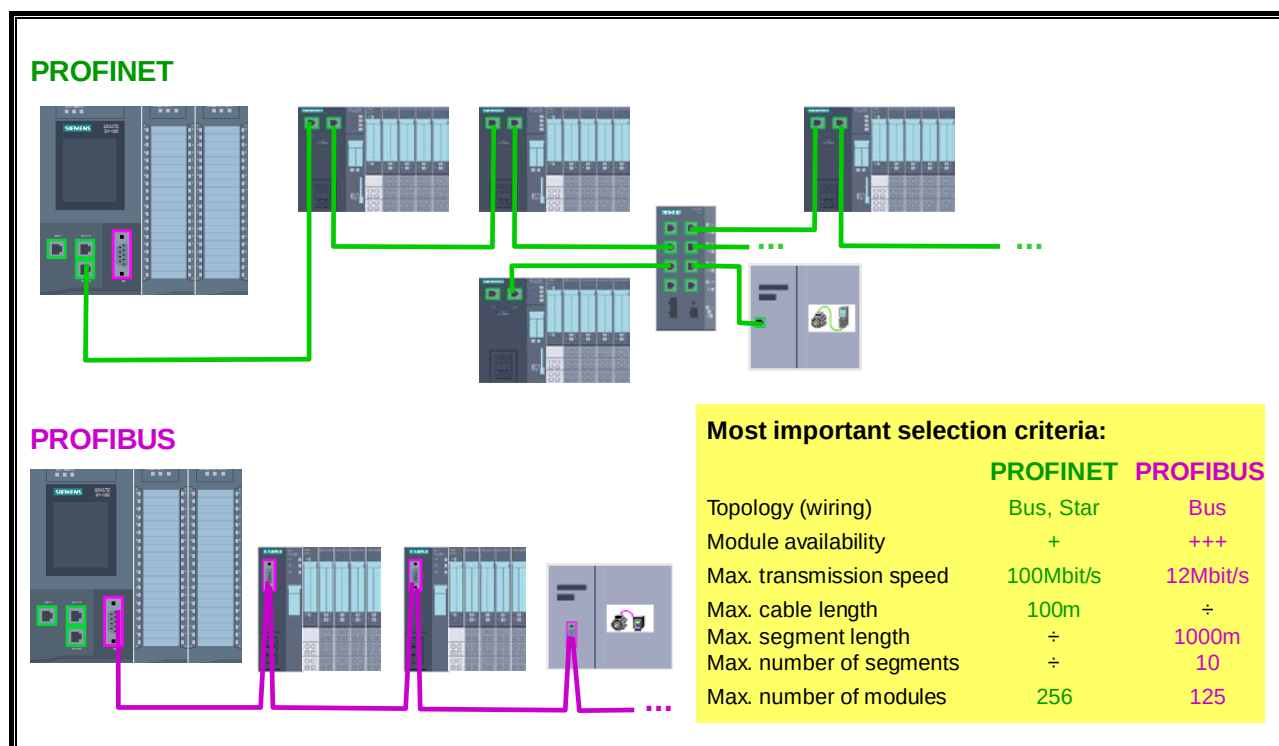
## 10.1. Task Description: Operating the Conveyor Model via the ET200SP Distributed I/O



### Task Description

- The conveyor model is no longer to be controlled via the DI/DO modules of the S7-1500 central device, but is to be controlled via the DI and DO modules of the distributed ET200SP station.
- For this, the ET200SP must be networked with the central S7-1500 station via PROFINET and the S7 program has to be adjusted.

## 10.2. Fieldbus Systems for SIMATIC S7



### Fieldbus Systems for SIMATIC S7

To connect distributed I/O, there are different bus systems.  
The most important for SIMATIC S7 are:

- PROFINET**  
 as the standard for communication applications at the field level enables the connection of distributed field devices via Industrial Ethernet.  
 The Industrial Ethernet network is a local area network (LAN) according to the international Standard IEEE 802.3 (Ethernet) and is designed for the industrial sector. It enables open and comprehensive network solutions with a high transmission performance.
- PROFIBUS**  
 is the bus system for local area networks (LANs) with only a few participants. Through its fulfillment of requirements according to EN 50170, PROFIBUS ensures openness for the connection of standard-conforming components of all manufacturers.

Due to the physical and communication-related differences of the two bus systems, there are various criteria which are used for the selection of the most suitable bus system.

#### Module Availability

Since PROFIBUS has already been established for a very long time in the area of fieldbus systems, a very wide spectrum of modules exists here.

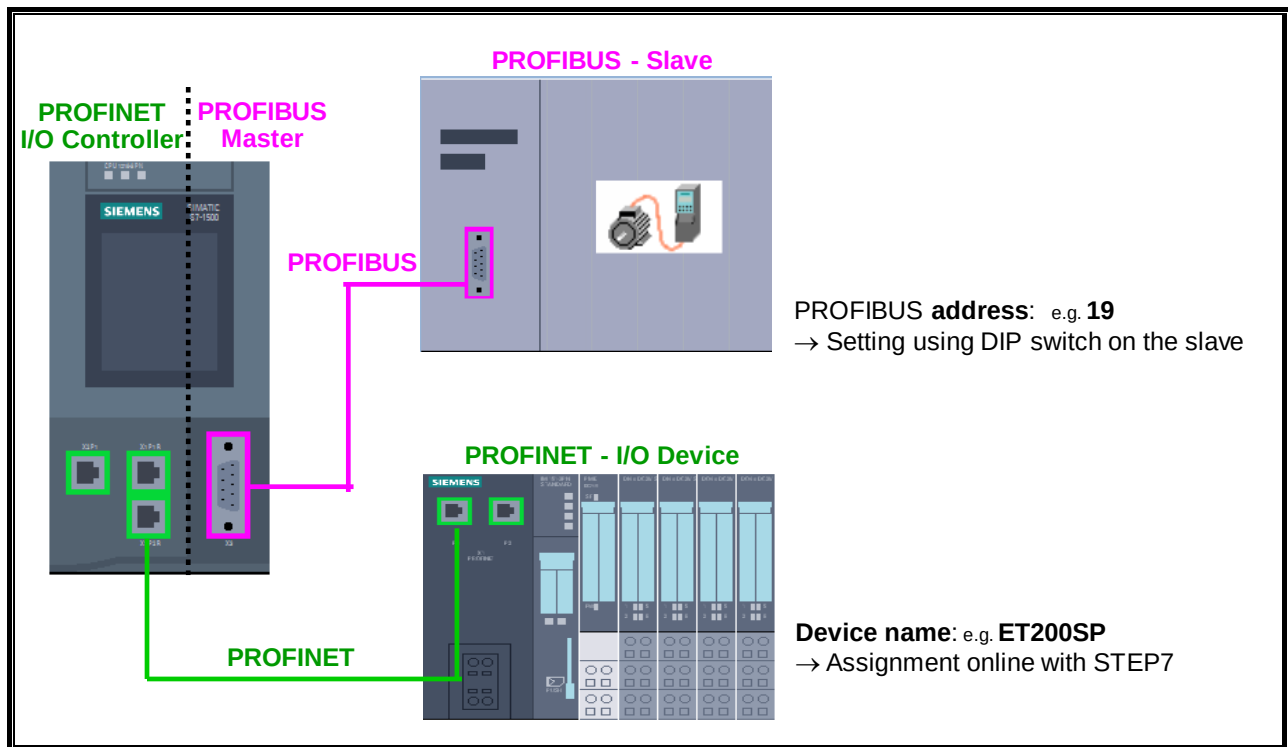
PROFINET is currently still a relatively "recent" fieldbus system where the diversity of modules is still evolving.

#### Cable Length, Segment Length

For PROFIBUS, a module line has to be reinforced after 100-1000m (depending on the transmission speed used); otherwise, the maximum bus length is reached.

For PROFINET, every connected component takes over this function. For that reason, only the cable length between two modules is relevant here.

### 10.3. Identification of Distributed I/O Devices



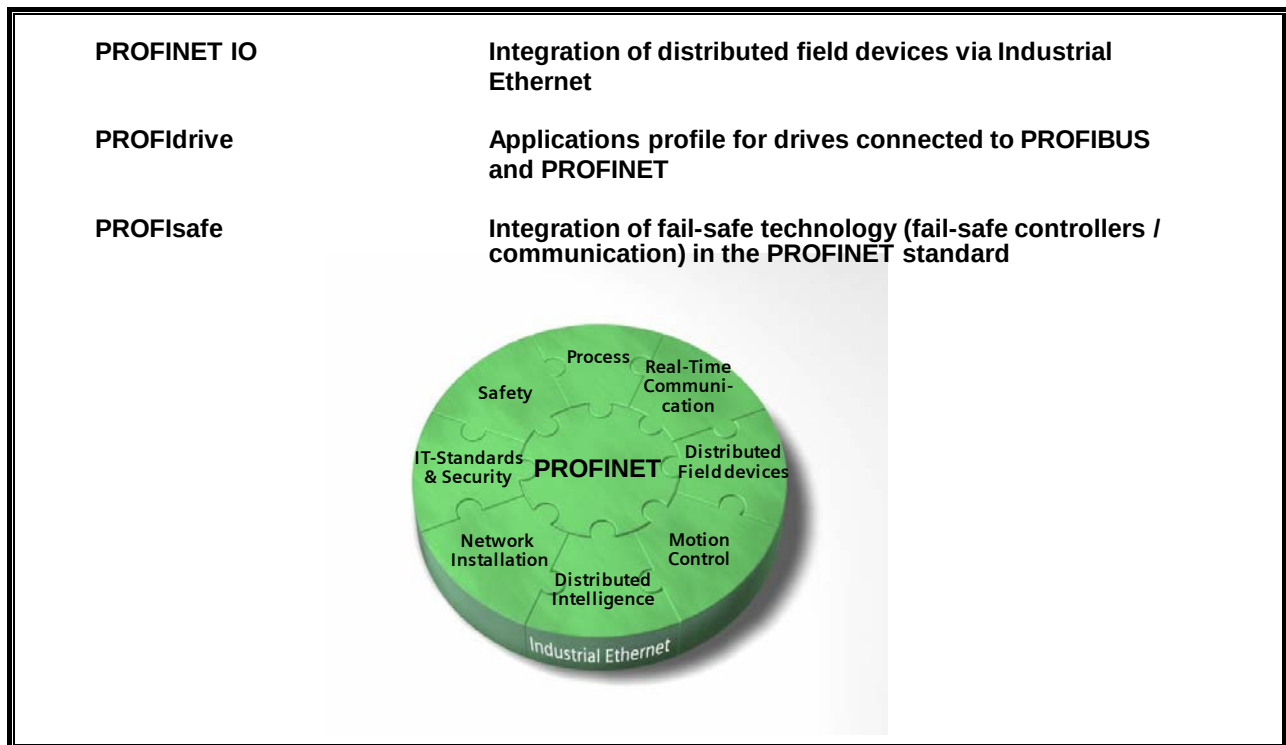
#### Distributed I/O Devices

During start-up, the CPU searches the configured PNIO devices or DP slaves and parameterizes these according to the loaded device configuration.

Both fieldbus systems use different methods for identifying I/O modules:

- **PROFIBUS**  
The set PROFIBUS address is used to search for the configured DP slave.  
The setting is typically made through the DIP switch on the slave.
- **PROFINET**  
The assigned device name is used to search for the configured PNIO device.  
The assignment of the device name (device initialization) is done from the STEP7 engineering through an online function.  
The parameterized IP address is then assigned to the PNIO device by the PNIO controller (CPU).

## 10.4. Components of the PROFINET Standard



### PROFINET

It completely covers the requirements of automation. PROFINET brings together the expertise of PROFIBUS and Industrial Ethernet. The utilization of the open standard, the easy handling and the integration of existing parts of a system (e.g. a plant) determined the definition of PROFINET right from the beginning. Today, PROFINET is integrated in IEC 61158.

### PROFINET IO

With PROFINET IO, the integration of distributed field devices takes place directly on the Ethernet. For that, the Master-Slave procedure from PROFIBUS DP is carried over into a Provider-Consumer model. From the communication point of view, all devices on the Ethernet have equal rights. Through the configuration, however, the field devices are assigned to a central controller. As a result, the familiar user view in PROFIBUS is transferred to the PROFINET I/Os: The distributed I/O device reads in the I/O signals and transfers them to the controller. The controller processes them and transfers the outputs back to the distributed I/O device.

### PROFIdrive

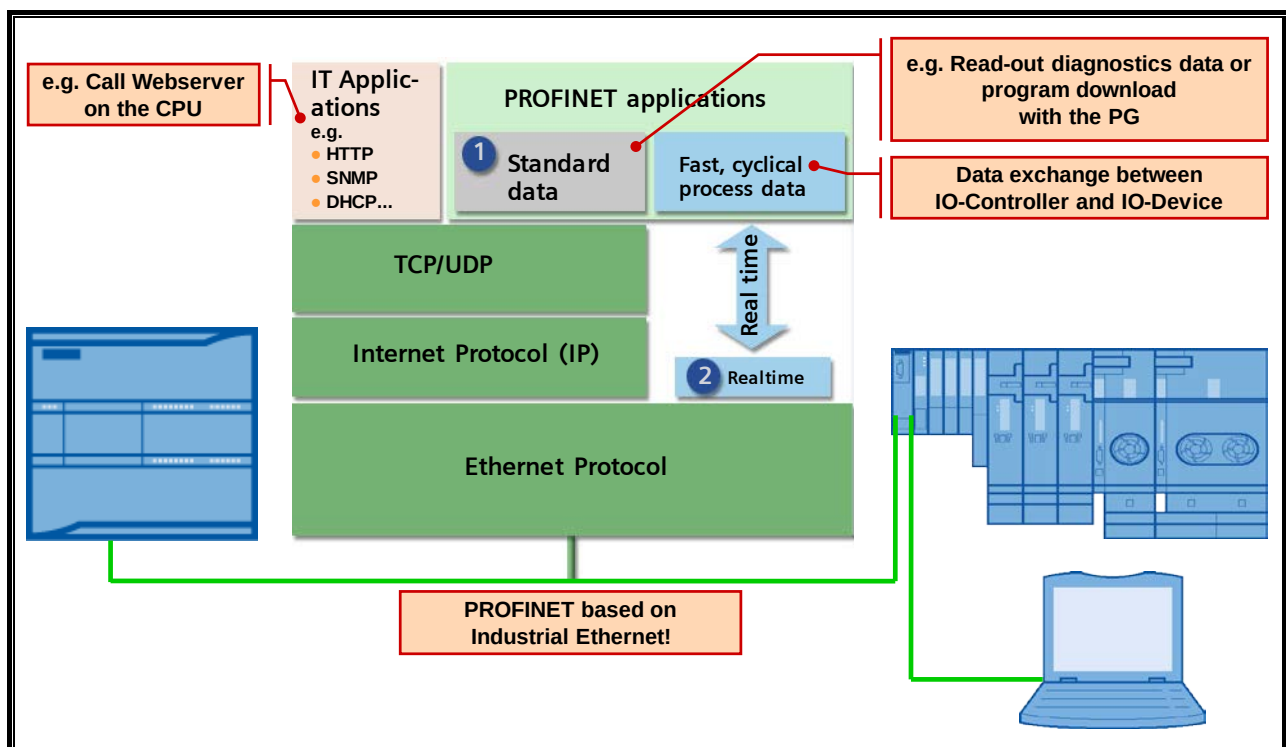
With PROFIdrive, very fast, clock-synchronous drive controls for high performance Motion Control applications will be implemented in the future.

### PROFIsafe

With PROFIsafe, the network infrastructure already existing for standard communication can also be used at the same time for fail-safe communication. Fail-safe and standard data is transmitted through the same bus line. The existing bus protocols, such as, PROFIBUS and PROFINET (so-called "black channel") are used to transport fail-safe data as additional data (so-called PROFIsafe layer). With that, the fail-safe communication is independent of the bus system and the lower-level networks.



### 10.4.1. PROFINET Communication Model



#### Real-time Channel

To be able to fulfill real-time requirements in automation, an optimized real-time communication channel, the Realtime Channel (RT Channel), was specified in PROFINET. It uses Ethernet (Layer 2) as a base.

The addressing of the data packets does not take place in this case via an IP address, rather by means of the MAC addresses of the participating devices. Such a solution minimizes the throughput times in the communications stack considerably and leads to an increase in performance with regards to the updating rate of automation data.

#### IRT Channel

Isochronous Real-time (IRT) as a further development with the following features:

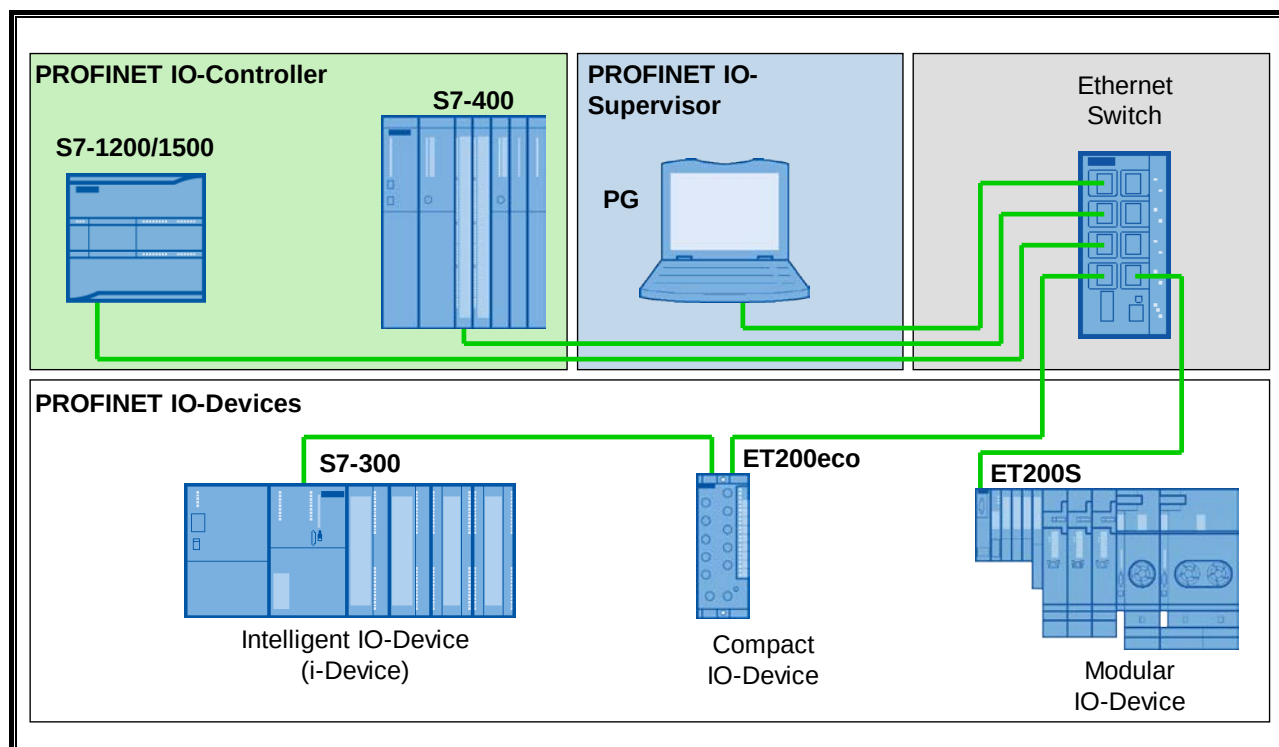
- Clock-synchronous data transmission
- Cycle times <1ms with jitter accuracy <1μs
- Typical field of application is Motion Control

#### IT Standards

The design of PROFINET WEB Integration focuses on commissioning and diagnostics. Access to a PROFINET device from the Internet or Intranet is done with standard protocols (for example, http). The data is transmitted in standard formats such as HTML or XML and can be presented with standard browsers such as Opera or Internet Explorer.

This worldwide accessibility makes it easy for the application manufacturer to support the user with commissioning, device diagnostics etc. Access to the data is done via Web servers which are integrated in the modules.

### 10.4.2. PROFINET IO Device Types



#### PROFINET IO-Controller

The IO-Controller (typically the PLC) establishes a logical connection to the connected IO-Devices after Power-On and subsequently parameterizes these (module parameters, address, etc.). (This corresponds to the function of a Class 1 Master in PROFIBUS).

#### PROFINET IO-Device

An IO-Device is a distributed IO device that is connected via PROFINET IO (this corresponds to the function of a slave in PROFIBUS).

Differentiation is made for the following IO-Device types:

- Compact IO-Device: Fixed degree of expansion.
- Modular IO-Device: Variable degree of expansion; can be expanded or reduced as required.
- Intelligent IO-Device: A PLC is configured not as an IO-Controller but as an IO-Device and provides a higher-level controller with I/O data.

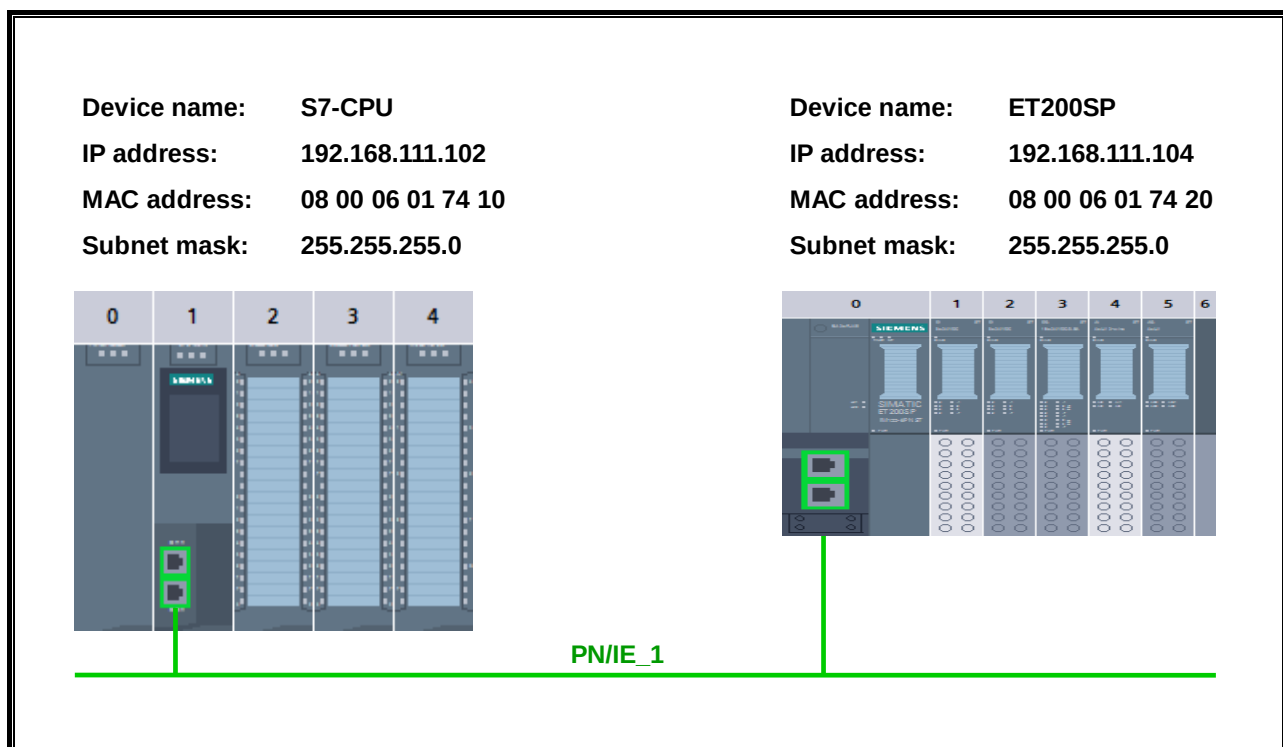
#### IO-Supervisor

This can be a programming device (PG), personal computer (PC) or Human Machine Interface (HMI) for commissioning or diagnostic purposes. (This corresponds to a Class 2 Master in PROFIBUS).

#### Ethernet Switch

PROFINET is based on Ethernet. For that reason, switches are always used as "network distributors". Every node is connected to a switch via a so-called "point-to-point" connection. This is also referred to as a **"Switched Ethernet"**. In most PROFINET devices, a 2 or multi-port switch is already integrated so that it is very easy to establish a line structure (comparable to PROFIBUS).

### 10.4.3. PROFINET Addresses



#### Internet Protocol

The Internet **P**rotocol (**IP**) forms the basis for all TCP/IP networks. It creates the so-called datagrams – specially tailored data packets for the Internet protocol - and takes care of their transport in the local sub-network or their "routing" to other subnets.

#### IP Addresses

IP addresses consist of 4 bytes. With the dot notation, each byte of the IP address is expressed as a decimal number between 0 and 255. The four decimal numbers are separated from one another through dots (see picture).

#### PROFINET Device Name

In PROFINET, each RT / IRT device must be assigned a unique device name that is retentively stored in the device. A module exchange without PG/PC is made possible through the device names.

#### MAC Address

Each Ethernet interface has been assigned a fixed and worldwide unique address by its manufacturer. This address is called hardware or MAC address (**M**edia **A**ccess **C**ontrol). It is stored on the network card and is used as the unique identification in a local network. The cooperation of the manufacturers guarantees that the address is unique worldwide.

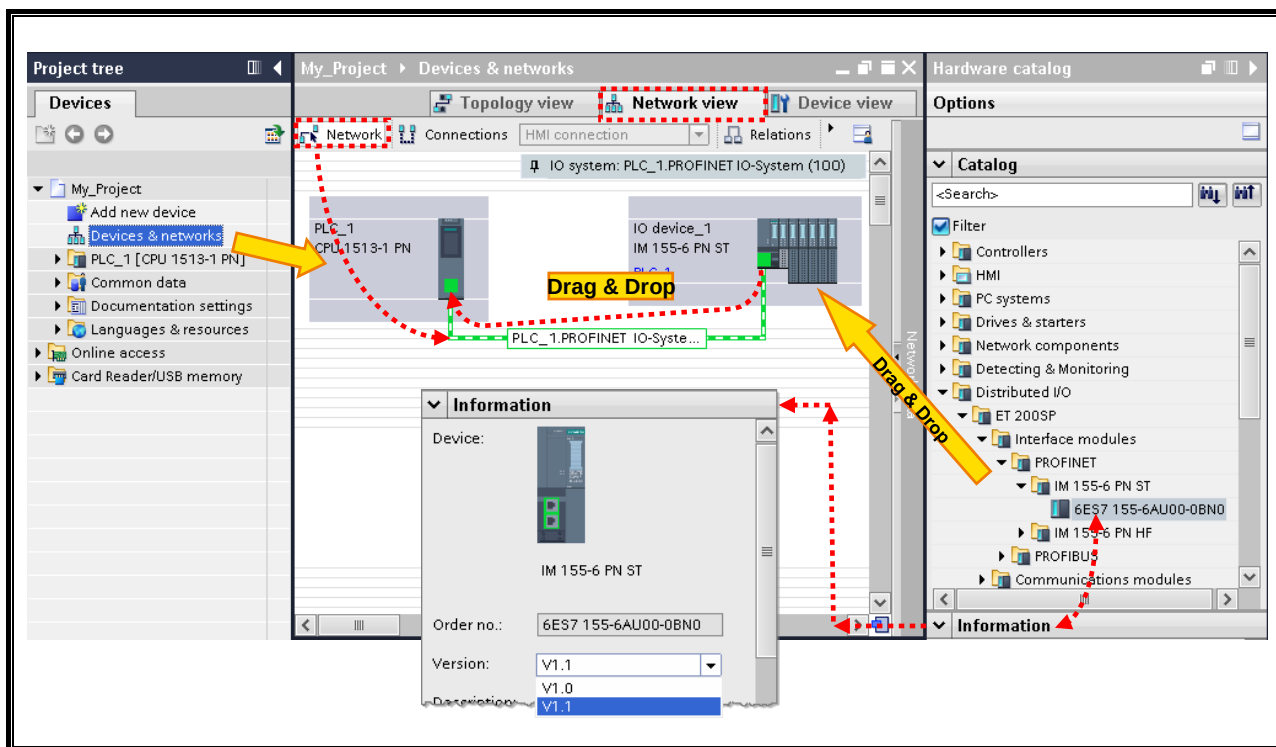
#### Subnet Mask

In the subnet mask you define which IP addresses can be accessed in your own network. It divides the IP address into network and device sections.

Only IP addresses whose network section is the same can be accessed.

e.g.: Subnet mask = **255.255.255.0** and IP address = **192.168.111.10**  
 accessible IP addresses: **192.168.111.1** to **192.168.111.254**

## 10.5. Inserting and Networking Distributed I/O



### Inserting Distributed I/O

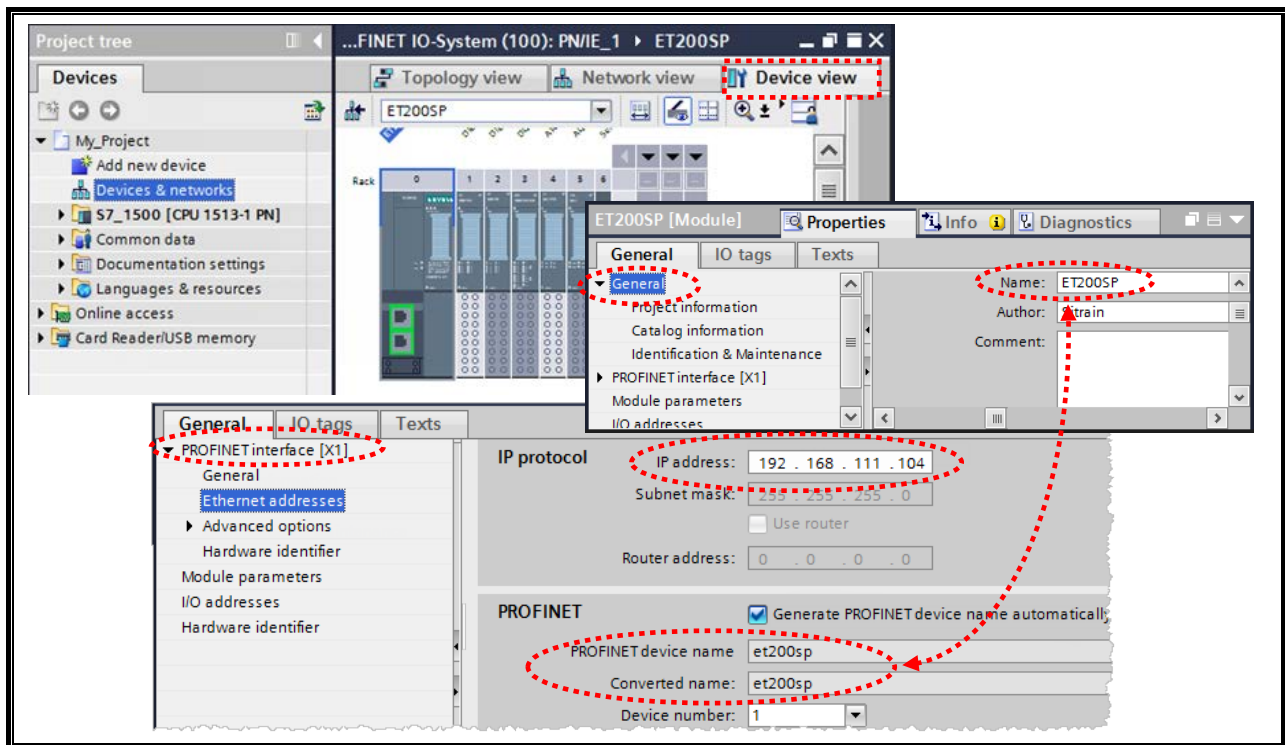
PROFINET IO-Devices are added in the Network view. Here, you can insert the relevant devices into the project by dragging & dropping them from the Hardware catalog.

In the beginning, the newly added ET200SP is not assigned to any controller and therefore appears in the Project tree as a "Not assigned device" in the same level as the PLCs and HMIs.

### Networking Distributed I/O

After the ET200SP IO-Device is added, it must now be assigned to an IO-Controller or networked with a CPU. In case there are several CPUs in the network, a co-ordination or monitoring of the I/O addresses by the IO-Controller and IO-Device can only be done through this unique assignment.

### 10.5.1. PROFINET IO Device ET200SP: Assigning the IP Address and Device Name OFFLINE



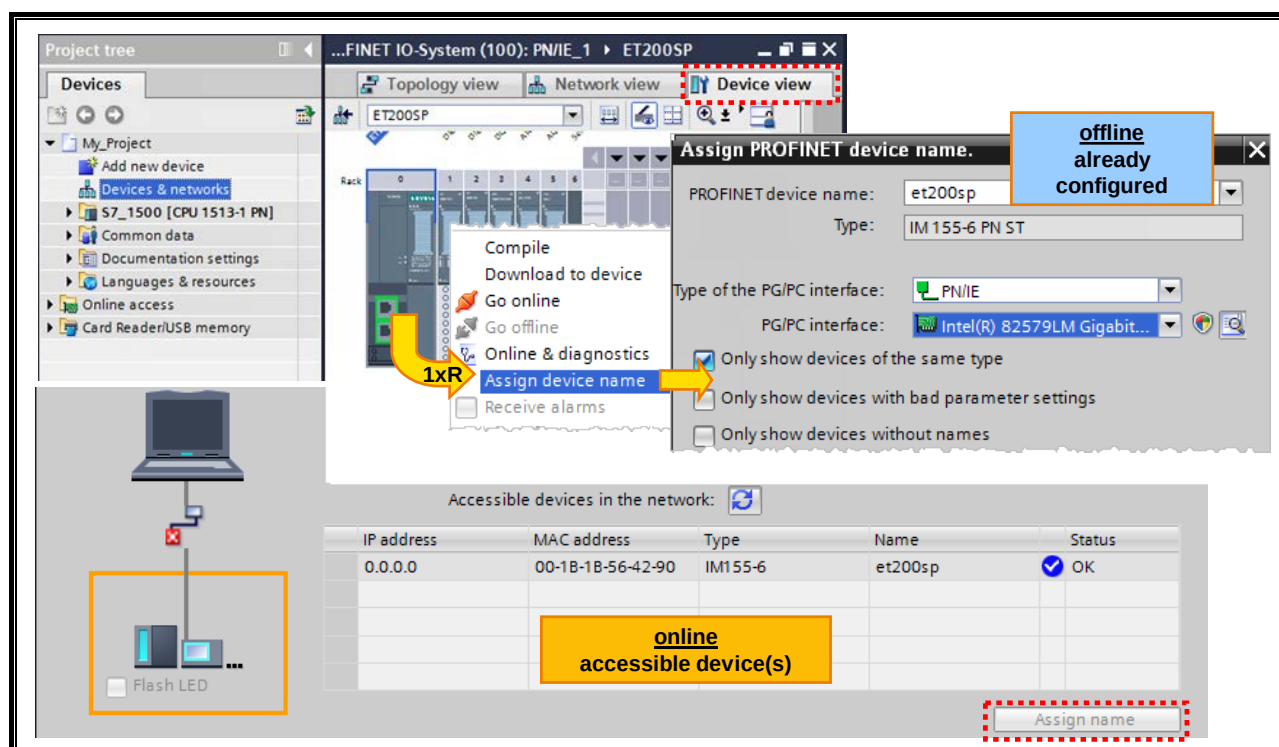
#### IP Address, Subnet Mask and PROFINET Device Name

For communication with the IO-Controller, a PROFINET device name must be assigned to the IO-Device (ET200SP) OFFLINE. The IO-Controller then assigns the IO-Device a valid IP address. If the IO-Device is assigned an IP address OFFLINE, this IP address is adopted. These parameters are downloaded to the IO-Controller (CPU) with the programming device. The IO-Controller (CPU) then transfers these and other parameters (such as, the I/O addresses) to the IO-Device (ET200SP).

#### Attention:

Only the PROFINET device name is relevant for the transmission of the offline configuration into the online device, not the IP address. The offline configured PROFINET device name and the online existing PROFINET device name must match. If the IO-Device has a different PROFINET device name or doesn't have a name at all, the IO-Controller cannot transfer the hardware configuration or the hardware parameter assignments to the IO-Device thus preventing a PROFINET system startup.

## 10.5.2. PROFINET IO Device ET200SP: Assigning the Device Name ONLINE



### IP Address and PROFINET Device Name

The PROFINET device name of the IO-Device configured offline and the device name existing online must match since the IO-Controller first checks the device names of the connected IO-Devices and then assigns the configured IP addresses during system startup. If an IO-Device is not accessible under its configured device name, the IO-Controller cannot establish a connection to the IO-Device.

The IP address of the IO-Device configured offline and the address existing online do not have to match. The PROFINET name is relevant for the downloading of the hardware configuration. If it exists, the online existing IP address is overwritten with the offline configured IP address.

### Ways of Assigning a Name Online

In principle, there are two ways of assigning a PROFINET device name to an IO-Device online:

Version 1 (safe, since there is no chance of typing errors)

The assignment of the device name is triggered from the device configuration of the IO-Device.

*Device configuration of IO-Device → Right-click on the Interface module (Slot 0) → Online & Diagnostics → Functions → Assign name (see picture)*

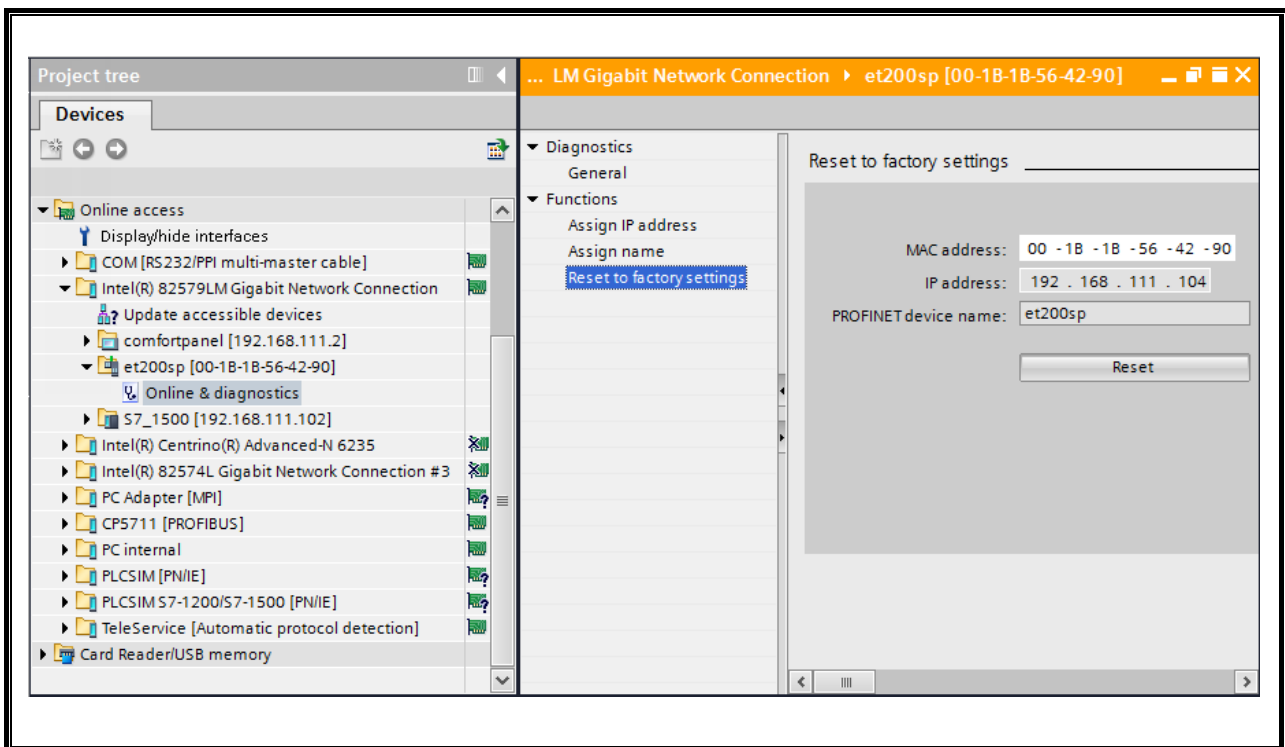
The advantage of this version is that the offline configured device name and the IP address are adopted 1:1 and so no typing errors can be made.

Version 2 (typing errors possible)

The assignment of the device name is triggered via "Online accesses":

*Project tree → Online accesses → Ethernet interface → IO-Device → Online & Diagnostics → Functions → Assign name*

### 10.5.3. Exercise 1: ET200SP: Reset to Factory Settings



#### Task

All settings so far (IP address, subnet mask and PROFINET name) of the Interface module and the memory card of the ET200SP station are to be deleted through a "Reset to factory settings". In the following exercises, you will then transfer your own settings onto the ET200SP station.

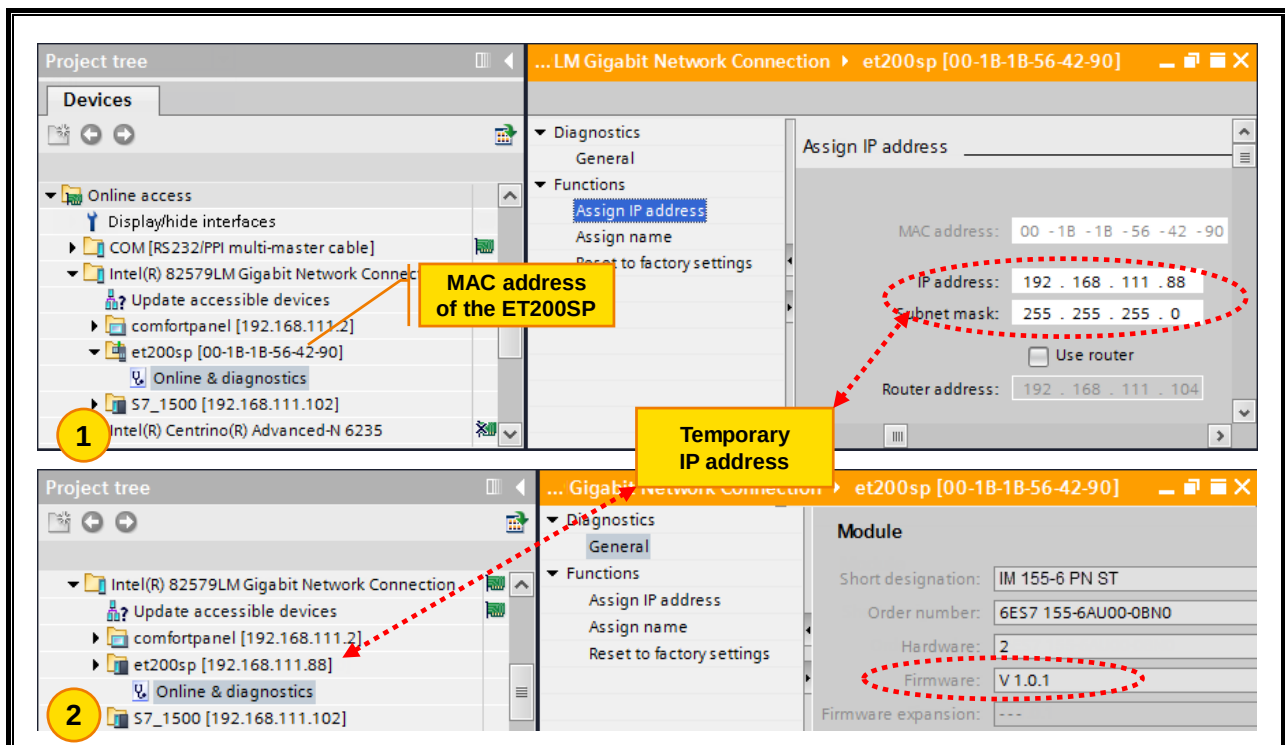
#### What to Do:

1. Open the Online accesses and there select the interface "Intel(R) 82574L Gigabit..."
2. There, activate "Update accessible devices" by double-clicking on it and wait until the list is completed.
3. Open the ET200SP and there activate the function "Online & diagnostics" by double-clicking on it.
4. In the "Online & diagnostics" window, open the "Functions" tab
5. There, activate "Reset to factory settings".
6. Close the "Online & diagnostics" window.
7. Check the success of the reset to factory settings by once again doing an "Update accessible devices" under  
Online accesses -> Intel(R) 82574L Gigabit...". The reset ET200SP should now only be displayed as "Device" with MAC address.

Leave all windows open for the next exercise.



### 10.5.4. Exercise 2: Reading-out the Firmware Version of the ET200SP



#### Task

In the following exercises, in order to be able to configure an ET200SP in the offline project which corresponds exactly to that of the training unit, you now have to read out the Firmware version of the ET200SP online.

#### Problem

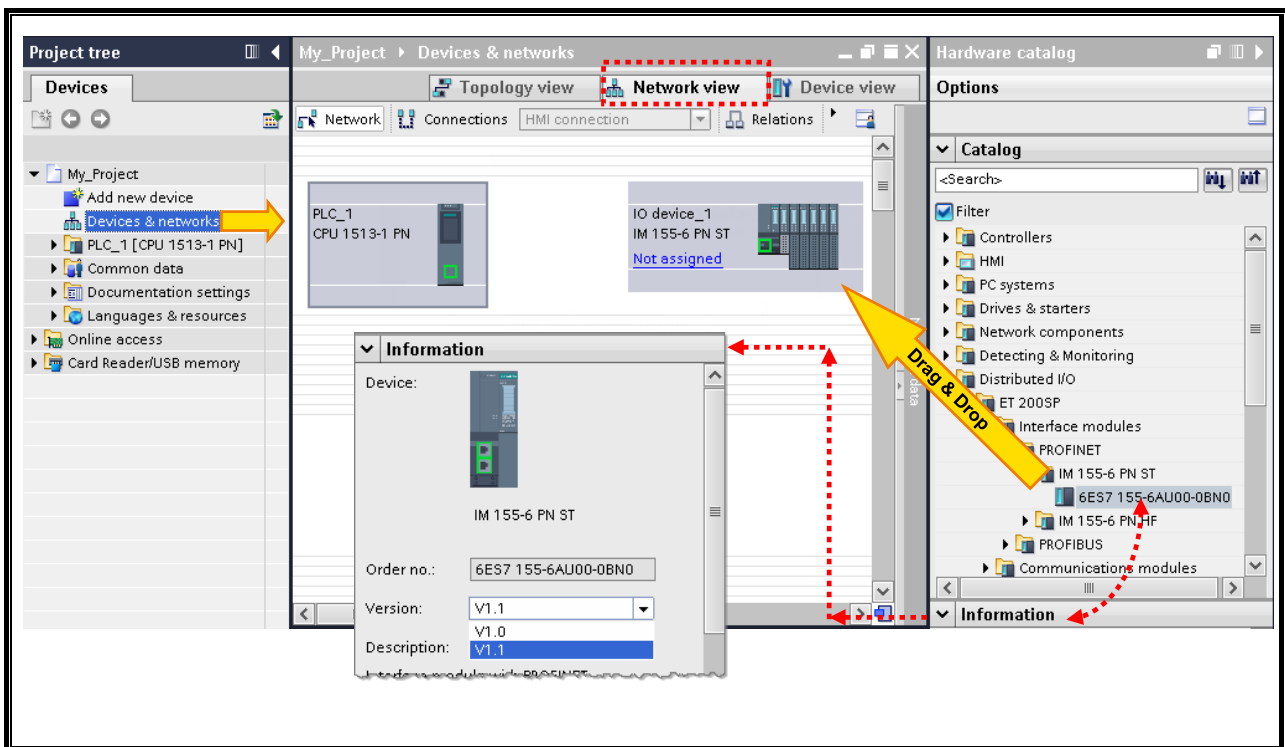
Due to the previous "Reset to factory settings", the ET200SP now no longer has a PROFINET device name nor an IP address, only a MAC address (see top picture). The Firmware version, however, cannot be read out via the MAC address, since an IP address is required for this diagnostic service.

#### What to Do

1. Open the ET200SP and, with a double-click, activate the "Online & diagnostics" function and there check whether the ET200SP Firmware version is displayed in the tab "Diagnostics -> General".
2. No Firmware version is displayed since the ET200SP doesn't have an IP address. To assign a temporary IP address, switch to the "Functions -> Assign IP address" tab. There enter the temporary IP address as well as the subnet mask shown in the picture and confirm via "Assign IP address" (see top picture).
3. In the Project tree, once again show the list of "Update accessible devices".
4. In the device list, the ET200SP is now displayed as a device with Order number and IP address. Once again activate "Online & diagnostics" (see bottom picture).
5. Make note of the Firmware version shown in the "Diagnostics -> General" tab.
6. Close the window and then the "Online accesses" in the Project tree.



### 10.5.5. Exercise 3: Offline Project: Adding the ET200SP



#### Task

An ET200SP is to be inserted into the project as a distributed I/O station.

PROFINET IO-Devices are added in the Network view. Here, you can insert the relevant devices into the project by dragging & dropping them from the Hardware catalog.

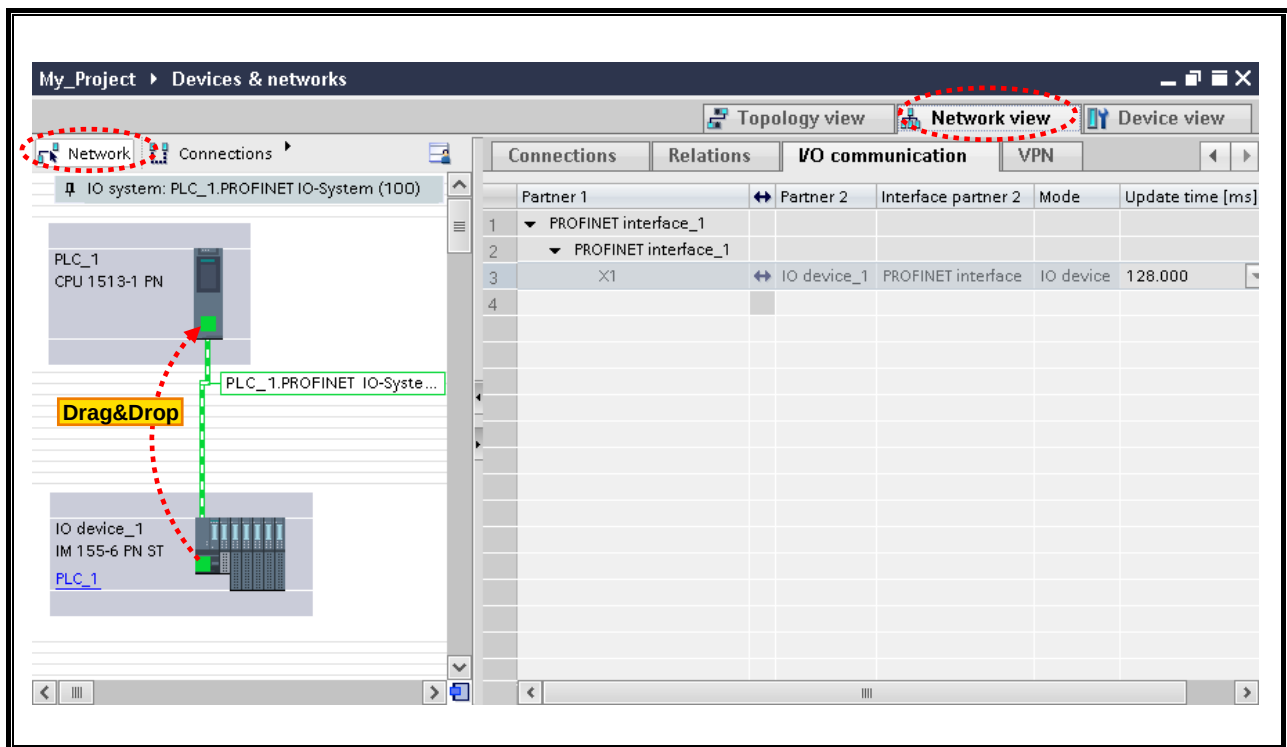
In the beginning, the newly added ET200SP is not assigned to any controller and therefore appears in the Project tree as a "Not assigned device" in the same level as the PLCs and HMIs.

#### What to Do

1. In the Project tree, open the "Devices & networks" editor by double-clicking on it.
2. Open the Hardware catalog Task Card and there Distributed I/O -> ET200SP -> Interface modules -> PROFINET -> IM155-6PN ST
3. Select the IM module used in your training unit, open the Information window and there select the Firmware version of your IM module.
4. Using drag & drop, drag the IM module into the "Devices & networks" editor (see picture).

**Leave all windows open because they are still needed for the next exercises!**

### 10.5.6. Exercise 4: Networking the ET200SP



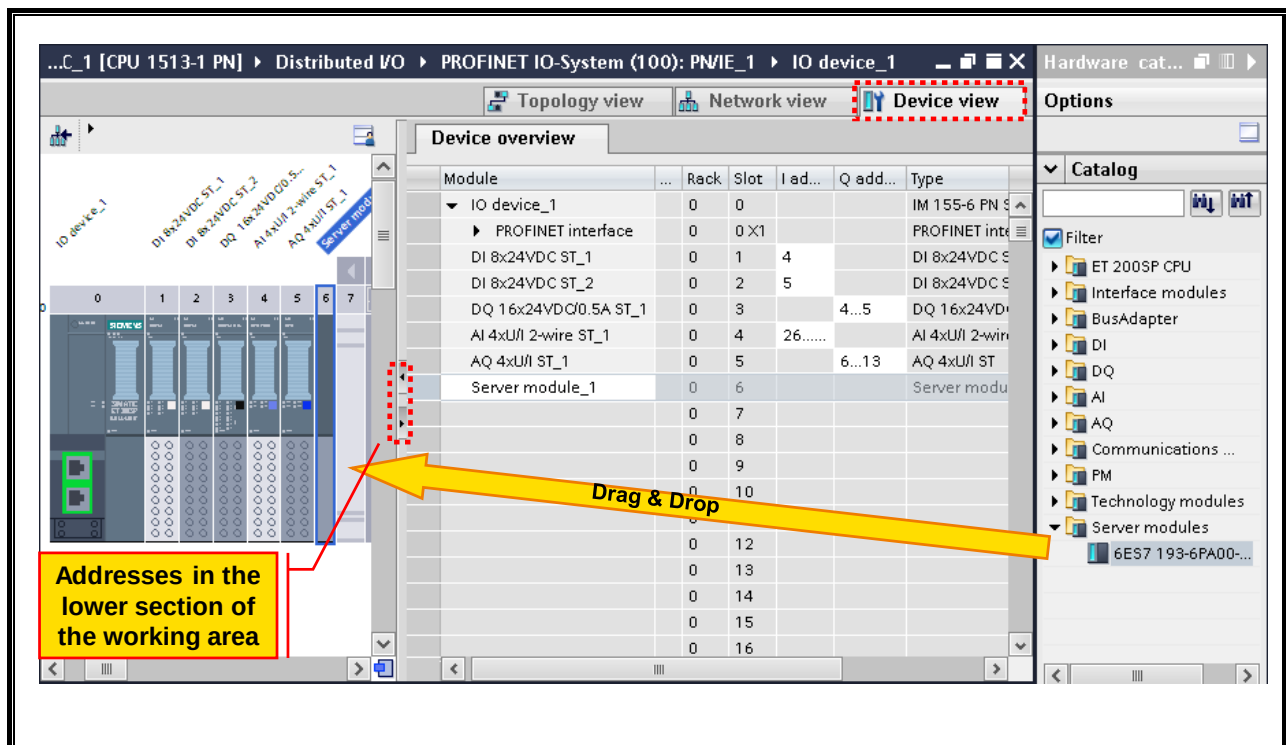
#### Task

After the ET200SP IO-Device is added, it must now be assigned to an IO-Controller or networked with a CPU. In case there are several CPUs in the network, a co-ordination or monitoring of the I/O addresses by the IO-Controller and IO-Device can only be done through this unique assignment.

#### What to Do

1. In the "Devices & networks" editor, select the Network view and there choose the "Network" tab.
2. Network the ET200SP with the CPU by connecting the Ethernet interface of the ET200SP with the Ethernet interface of the CPU using drag & drop.
3. Select the newly created PROFINET IO system and, in the Inspector window under "IO communication", check the generated communication partners.

### 10.5.7. Exercise 5: Configuring and Parameterizing the ET200SP



#### Task

The configuration of the ET200SP in the offline project must match exactly with the configuration of your training device. Attention should be given in particular to the order numbers and versions of the modules.

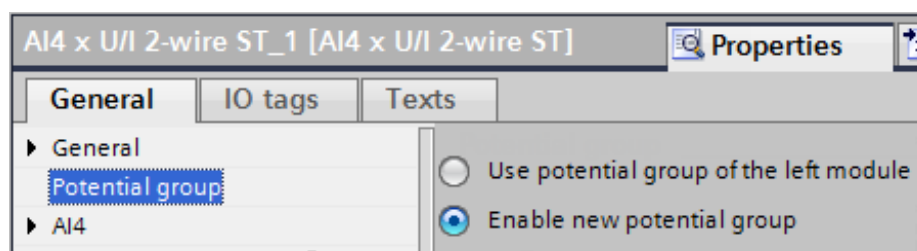
The ET200SP has digital and analog input and output modules to which the conveyor model is to be connected in the following. The I/O addresses used in the STEP 7 program must match the addresses of the DI/DO modules parameterized here.

The current address assignment is located in the lower section of the working area in the "Devices & networks" editor in the "Device view" tab of the module. The addresses can be changed in the table.

#### What to Do

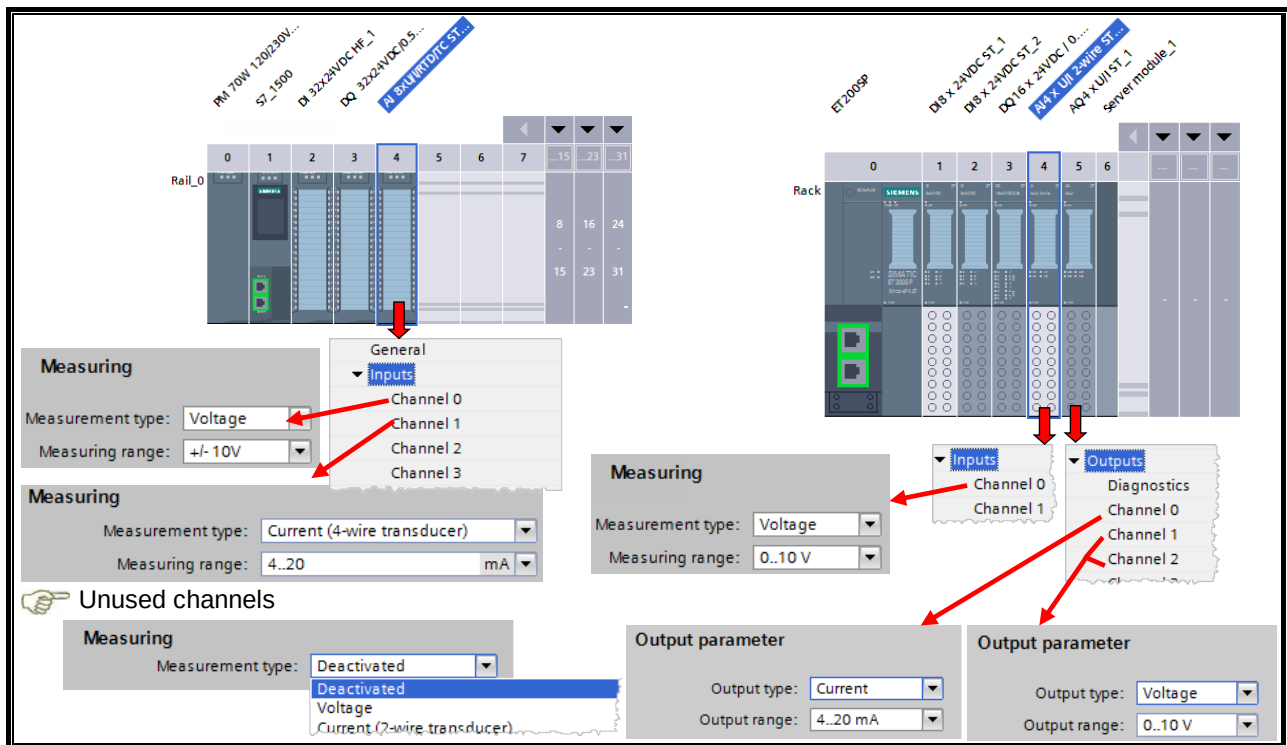
1. In the "Devices & networks" editor, select the "Device view" of the ET200SP.
2. In the Task Cards, open the "Hardware catalog".
3. Configure the ET200SP station according to your training unit.

Ensure that a new potential group is opened with the AI module on Slot 4:



4. Open the lower section of the working area of the "Devices & networks" editor (see picture) and, in the table, enter the I/O addresses shown in the picture.
5. Save your project.

### 10.5.8. Exercise 6: Setting the Channel Parameters of the AI Modules (ET200SP and Central AI)

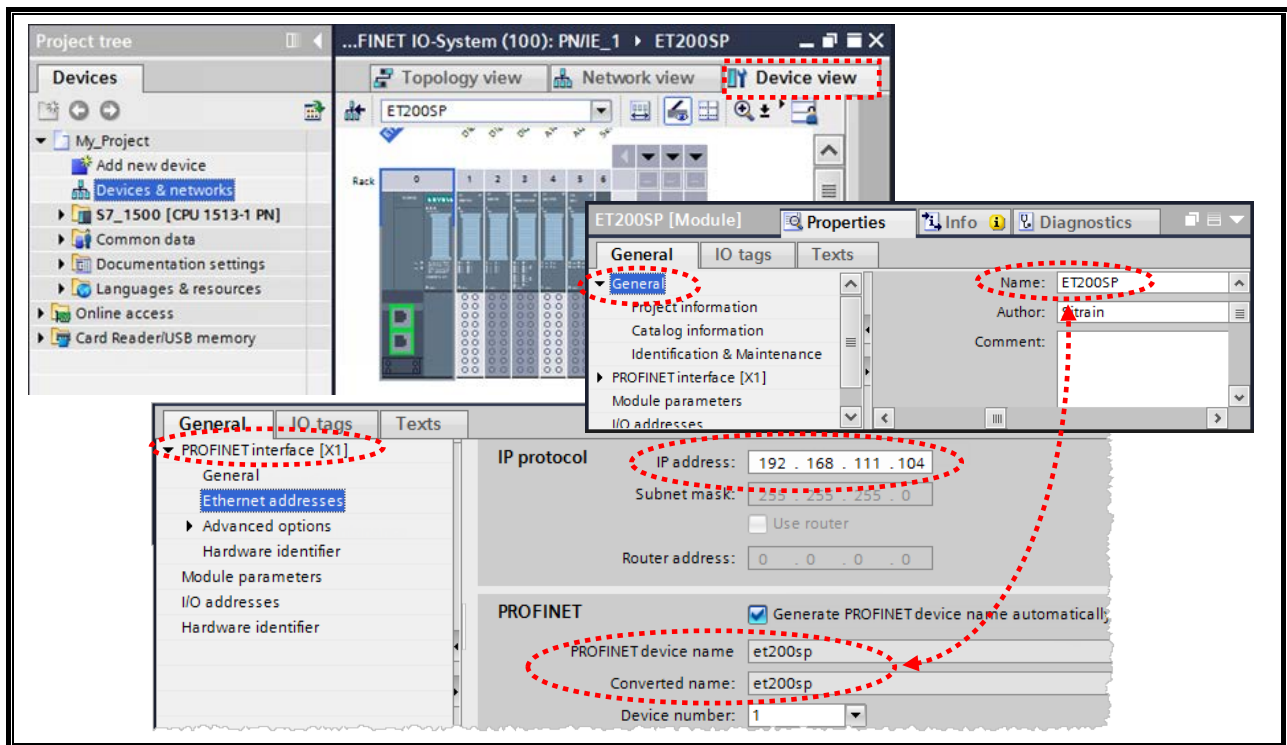


#### Task and What to Do:

Set the analog channels to the relevant parameters:

- Inputs central I/O
  - Channel 0 → Voltage +/-10V
  - Channel 1 → Current (4-wire transducer) 4 to 20mA
- Inputs distributed I/O (ET200SP)
  - Channel 0 → Voltage 0 to 10V
- Outputs distributed I/O (ET200SP)
  - Channel 0 → Current 4 to 20mA
  - Channel 1 → Voltage 0 to 10V
  - Channel 2 → Voltage 0 to 10V
- Deactivate all unused analog inputs and outputs.

### 10.5.9. Exercise 7: ET200SP: Assigning the IP Address / PROFINET Name OFFLINE



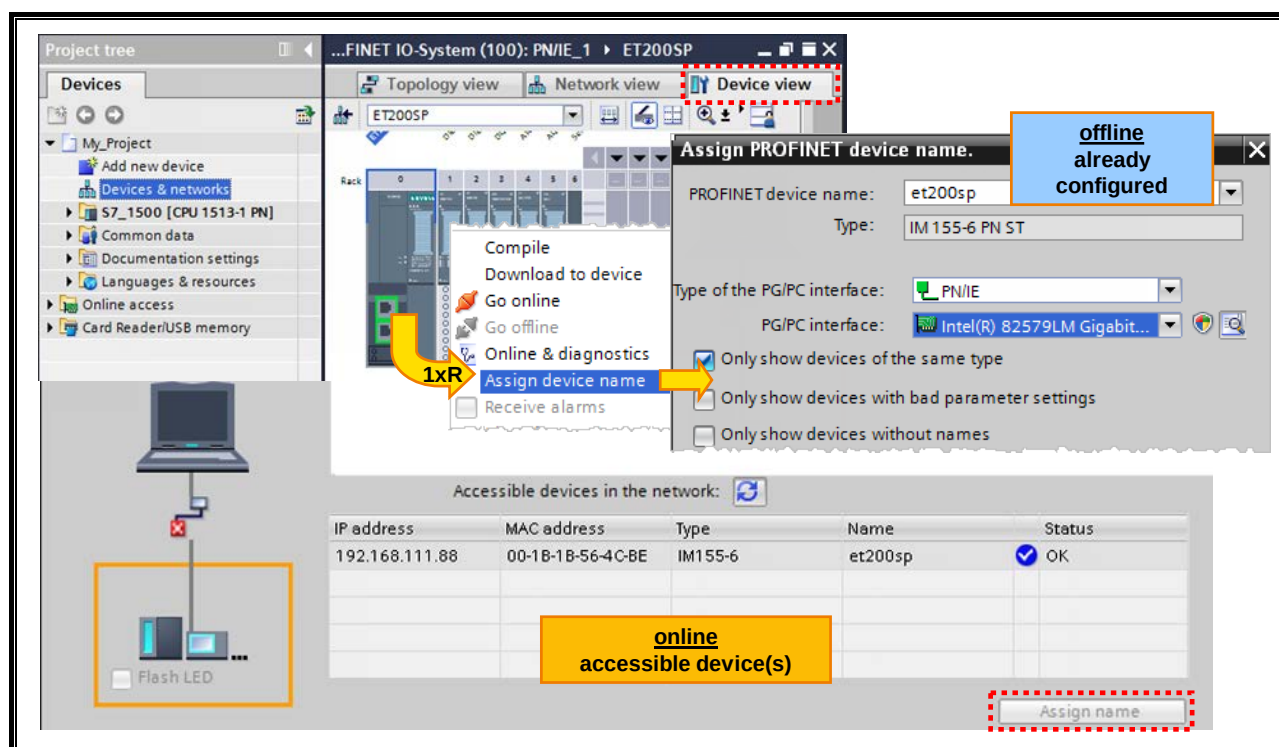
#### Task

The ET200SP station is later to work with the IP address, subnet mask and PROFINET device name shown in the pictures above.

#### What to Do

1. In the "Devices & networks" editor, select the "Device view" of the ET200SP.
2. Select the IM module on Slot 0 and open the "Properties" tab in the Inspector window.
3. There, select the "General" tab and under "Name" enter the PROFINET device name (see middle right picture).
4. Then select the "Ethernet addresses" tab and under "IP protocol" enter the IP address and subnet mask shown (see lower picture). In the same tab you will also find the PROFINET device name that you previously edited in the "General" tab.
5. Save your project.

### 10.5.10. Exercise 8: ET200SP: Assigning the PROFINET Name ONLINE



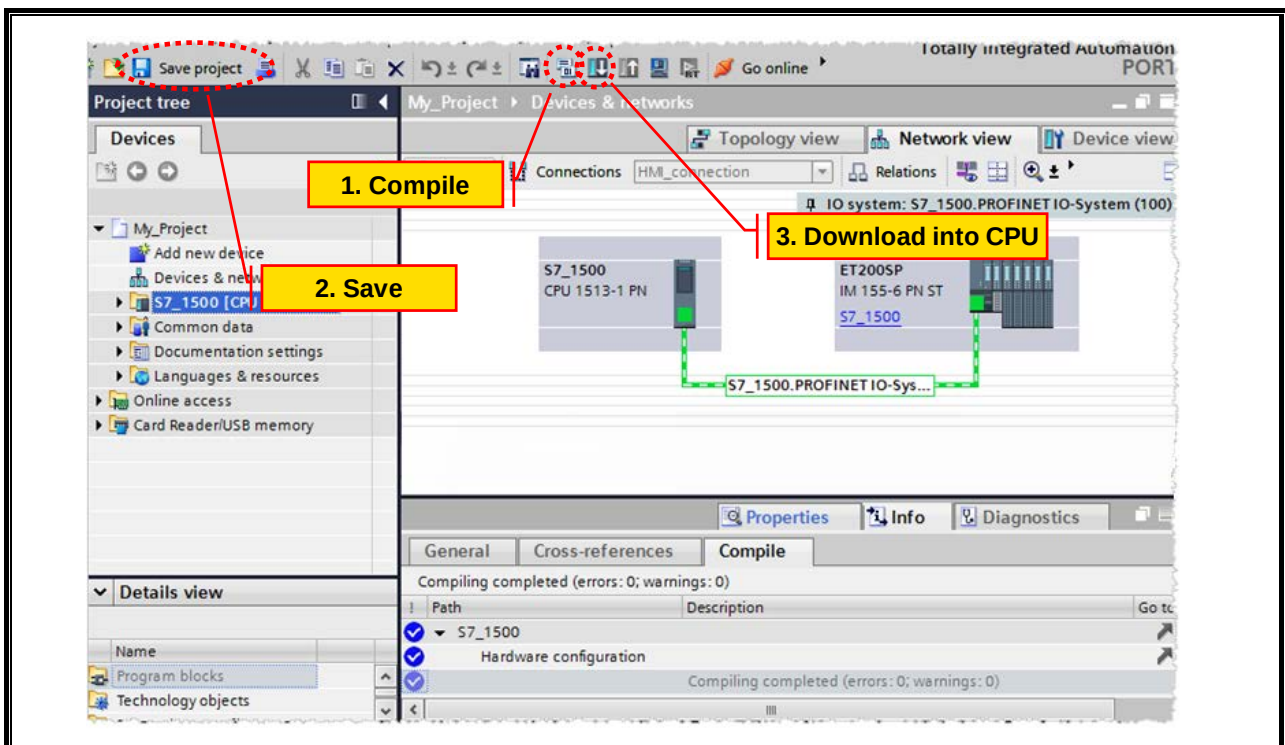
#### Task

The PROFINET device name previously assigned offline must now be assigned to the ET200SP online, so that the IO-Controller or the CPU can assign the offline-configured IP address during system startup of the ET200SP.

#### What to Do

1. In the "Devices & networks" editor, select the "Device view" of the ET200SP.
2. Right-click on the Interface module or the module on Slot 0 and in the menu that appears, activate the item "Assign device name".
3. In the dialog that appears, check the (offline) PROFINET device name.
4. Under "Type of the PG/PC interface", select the interface through which you are connected to the PROFINET (see picture).
5. In the lower part of the dialog, under the (online) "Accessible devices in the network", select the ET200SP or the Interface module IM156-6 and activate "Assign name".
6. Save your project.

### 10.5.11. Exercise 9: Compiling the HW Configuration and Downloading it in the CPU



#### Task

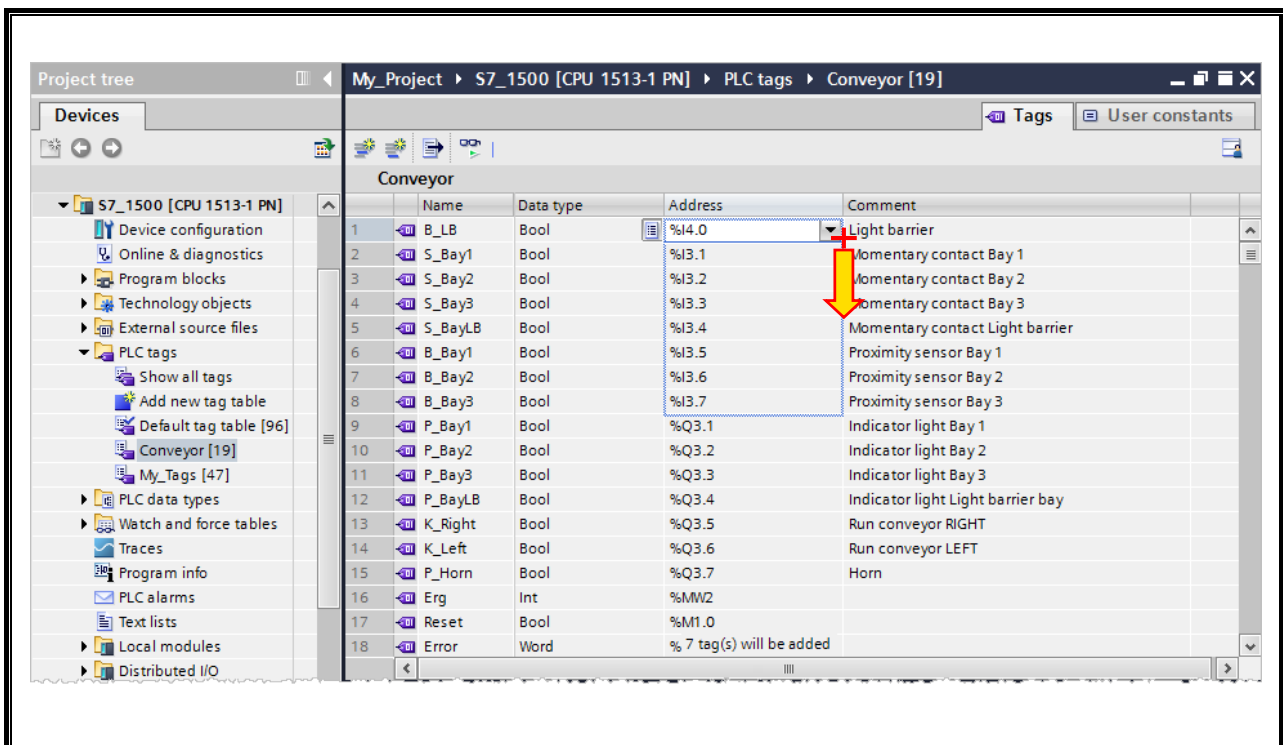
Now that the PROFINET I/O system is completely configured and parameterized, the project must be completely compiled, saved and downloaded into the CPU.

#### What to Do

1. Compile the hardware configuration by selecting the S7-1500 station in the Project tree and then clicking on the Compile button (see picture). In the Inspector window under "Info", check whether the compilation was successful. Should errors have occurred, correct them.
2. Save your project.
3. Download the entire station into the CPU by clicking on the Download button (see picture). In the Inspector window under "Info", check whether the loading was successful.
4. Check the module LEDs of your training unit: Only green LEDs should be lit and not flashing!!!
5. Save your project.



### 10.5.12. Exercise 10: Adjusting the S7 Program via "Rewiring"



#### Adjusting the S7 Program via "Rewiring"

So that the conveyor model can be operated via the ET200SP, the PLC tags (input and output byte 3) used in the S7 program must be "rewired" to the I/O addresses of the ET200SP modules (input and output byte 4).

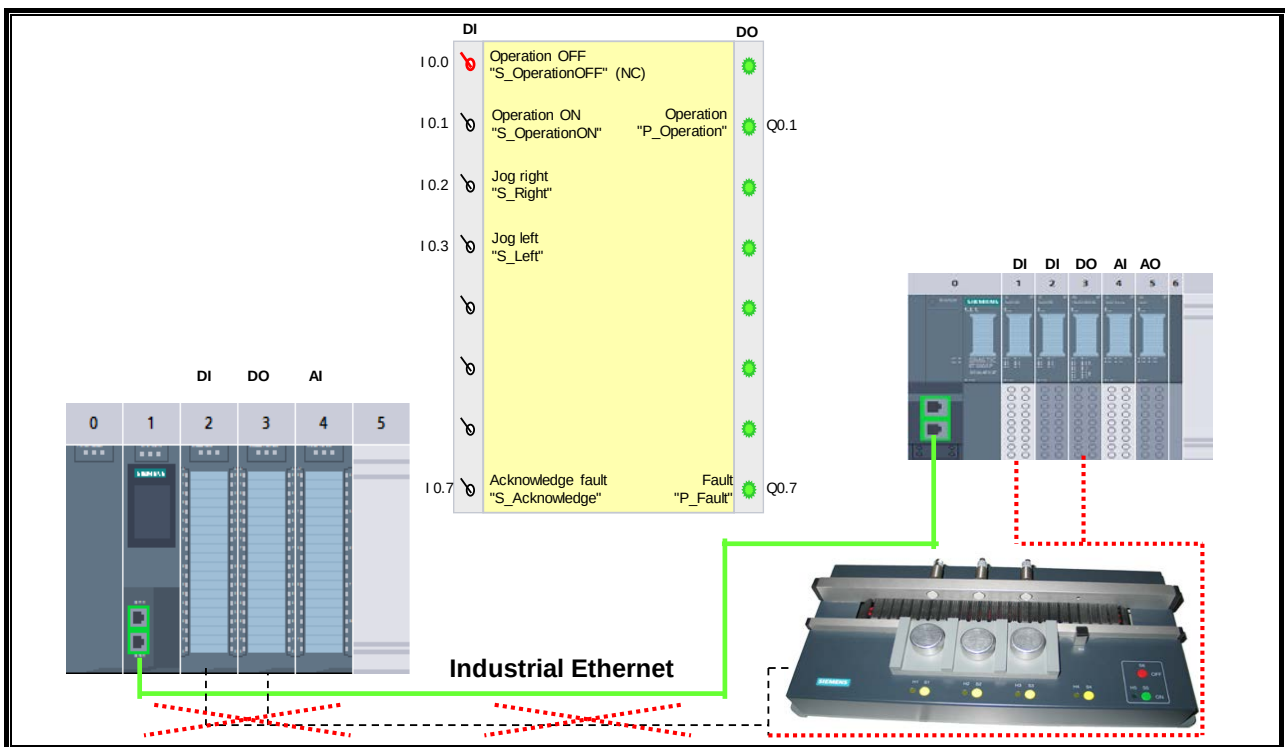
The "rewiring" can be carried out directly on the tag in the Blocks editor or, as shown in the picture, via the PLC tag table. All "rewiring" done here has immediate effect in the entire program.

#### What to Do

1. Open the PLC tag table "Conveyor".
2. Change the address of the PLC tag "B\_LB" from I 3.0 to I 4.0 and end the entry with Return.
3. Select the address field, whereby a small square appears in the lower right corner.
4. Position the mouse pointer on this square so that the square changes to a cross (shown in the picture in red).
5. With the mouse pointer pressed down, drag the small square onto the address fields below it so that the change of the byte address is also adopted there.
6. Do the same for rewiring the outputs.
7. Save your project.
8. Download the adjusted S7 program into the CPU.



### 10.5.13. Exercise 11: Function Test with Conveyor Model via Distributed I/O



#### Task:

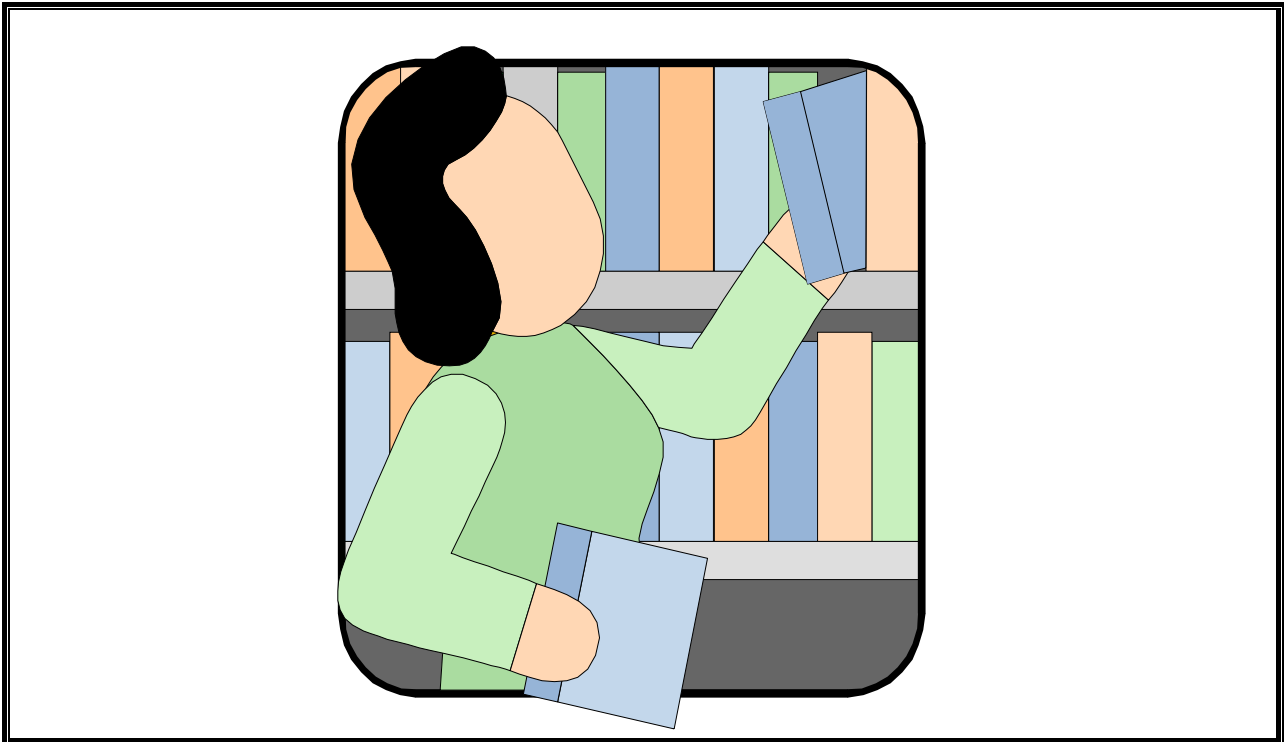
The conveyor model is now to be operated via the ET200SP station. For this, the conveyor model connector cable must be connected to the SUB-D connector of the ET200SP station on the backside of the training case.

With successful commissioning of the ET200SP IO-Device and rewiring of the S7 program, the control of the conveyor model should function unchanged.

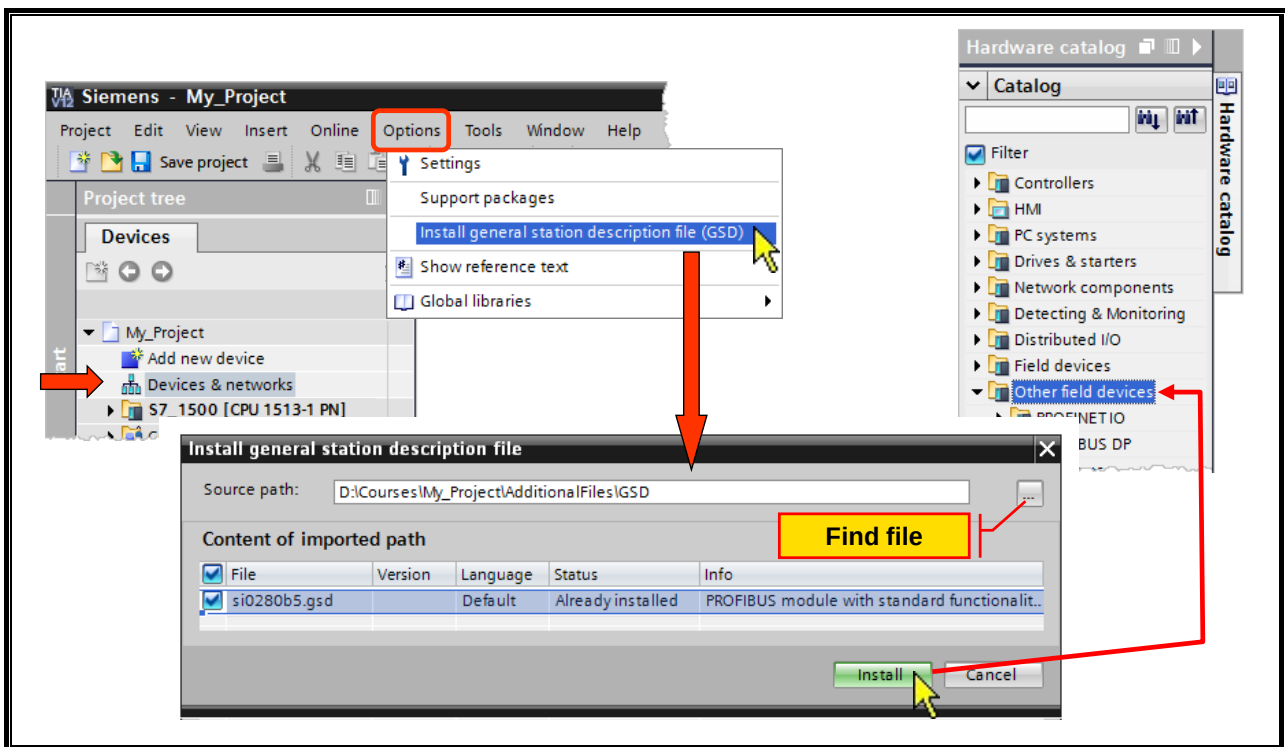
#### What to Do

1. Remove the conveyor model connector cable on the S7-1500 and insert it in the ET200SP.
2. Check whether the conveyor motor can still be jogged to the right and left and whether parts can be transported from Bay 1 or 2 through the light barrier.

## 10.6. Additional Information



### 10.6.1. Installing Distributed I/O Components Later On



#### GSD (General Station Description) File

STEP 7 requires a GSD or a type file for every distributed I/O slave so that the module can be configured from the Hardware catalog.

GSD files contain all necessary communication properties of a distributed I/O module according to the PROFIBUS/PROFINET standard.

A GSD file is normally supplied with distributed I/O modules or is available as a download from the Internet.

Many SIEMENS modules are already generally a part of the Hardware catalog.

#### Modules Added to the Hardware Catalog

Following installation, these are entered in the "Other field devices" section and are available there for the configuration.

# Contents

|            |                                                                                               |             |
|------------|-----------------------------------------------------------------------------------------------|-------------|
| <b>11.</b> | <b>Connecting an HMI Device .....</b>                                                         | <b>11-2</b> |
| 11.1.      | Task Description: Operating the 'Plant' via the Touchpanel .....                              | 11-3        |
| 11.2.      | Data Exchange between Touchpanel and CPU .....                                                | 11-4        |
| 11.3.      | Adjusting the S7 Program .....                                                                | 11-5        |
| 11.4.      | Adding an HMI Device .....                                                                    | 11-6        |
| 11.5.      | WinCC Configuration Interface .....                                                           | 11-7        |
| 11.5.1.    | Buttons and Input / Output Fields .....                                                       | 11-8        |
| 11.6.      | Configuring the IP Address of a Touchpanel .....                                              | 11-9        |
| 11.6.1.    | Networking a Touchpanel .....                                                                 | 11-10       |
| 11.6.2.    | Configuring an HMI Connection .....                                                           | 11-11       |
| 11.6.3.    | Setting the IP Address on the Touchpanel .....                                                | 11-12       |
| 11.7.      | Downloading the HMI Project into the Touchpanel.....                                          | 11-13       |
| 11.8.      | Exercise 1: Copying the Touchpanel Project and the Interface Data Block from the Library..... | 11-14       |
| 11.8.1.    | Exercise 2: Networking the Touchpanel .....                                                   | 11-15       |
| 11.8.2.    | Exercise 3: Configuring the HMI Connection .....                                              | 11-16       |
| 11.8.3.    | Exercise 4: Completing the HMI Tag Connection.....                                            | 11-17       |
| 11.8.4.    | Exercise 5: Configuring the SETPOINT Quantity Display .....                                   | 11-18       |
| 11.8.5.    | Exercise 6: Compiling and Saving the HMI Project .....                                        | 11-19       |
| 11.8.6.    | Exercise 7: Setting the IP Address on the TP.....                                             | 11-20       |
| 11.8.7.    | Exercise 8: Downloading the HMI Project into the Touchpanel.....                              | 11-21       |
| 11.8.8.    | Exercise 9: Adjusting the STEP 7 Program .....                                                | 11-22       |

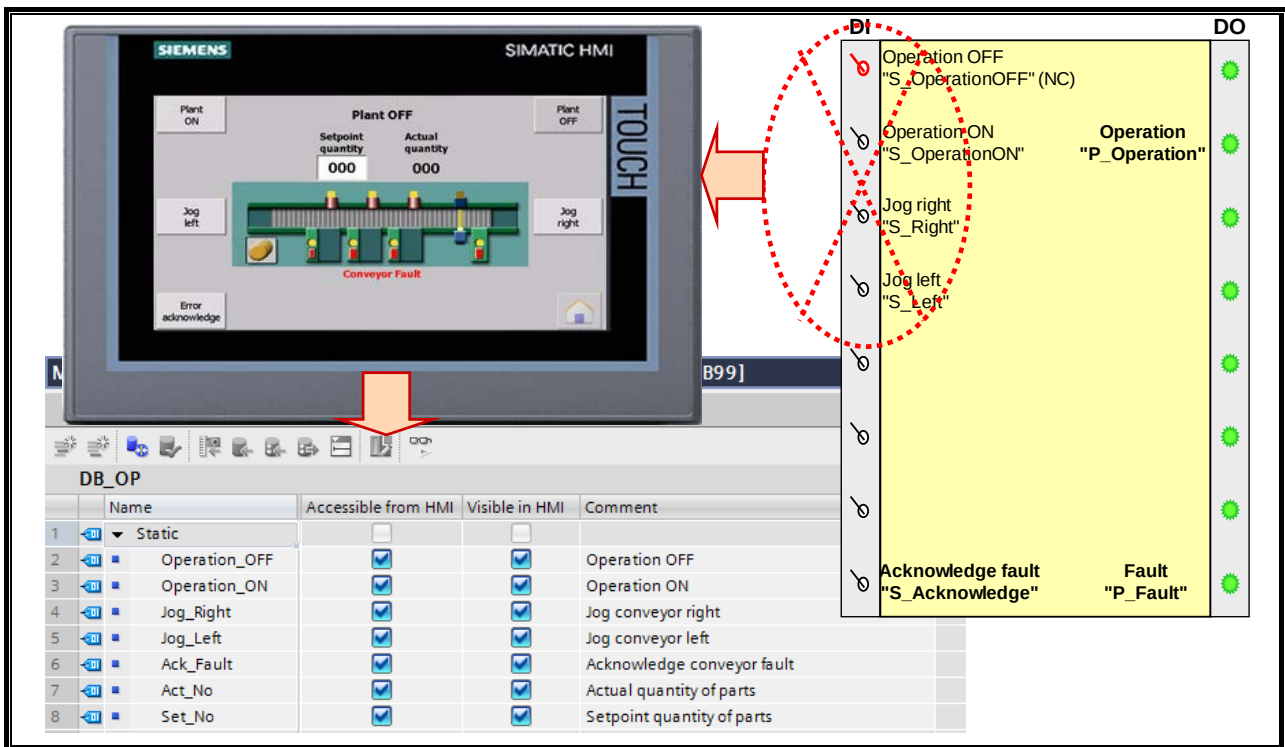
## 11. Connecting an HMI Device

At the end of the chapter the participant will ...

- ... be able to explain the principle of data exchange between touchpanel and CPU using tags
- ... be able to set the interface for the touchpanel
- ... be able to commission a touchpanel project
- ... be able to adjust a STEP 7 program



## 11.1. Task Description: Operating the 'Plant' via the Touchpanel

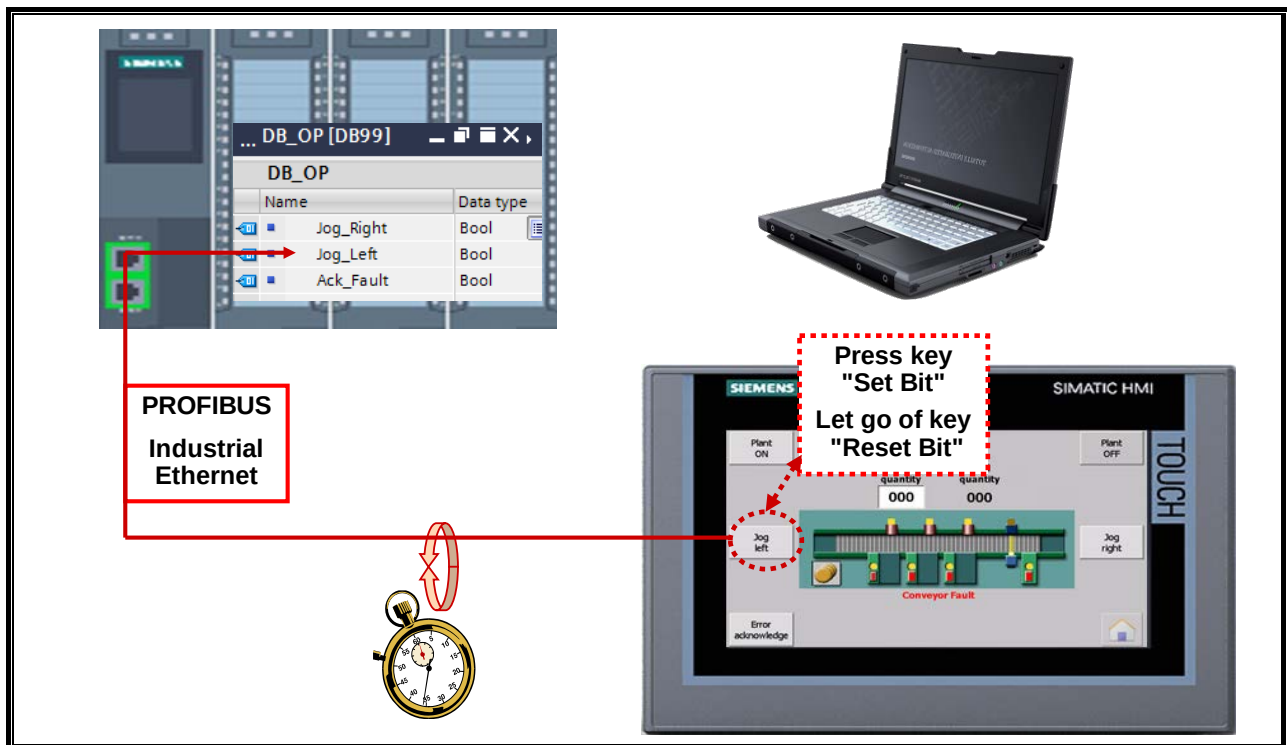


### Task Description

The touchpanel project is to be commissioned and the S7 program of the controller is to be adjusted in such a way that...

- the functions "Operation ON/OFF" and "Jog Right/Left" are no longer realized via the simulator pushbuttons but via the corresponding buttons on the touchpanel.
- the acknowledgement of a conveyor fault should still be possible via the simulator pushbutton "S\_Acknowledge" (I 0.7), and, in addition, also via the corresponding button on the touchpanel.
- the SETPOINT quantity is no longer constant 3, but can be specified via an input field on the touchpanel.

## 11.2. Data Exchange between Touchpanel and CPU



### Data Exchange between Touchpanel and CPU

Data is exchanged between SIMATIC S7 and the HMI system via tags. In the configuration of the HMI device, the screen objects, such as, buttons and input/output fields are linked to HMI tags which in turn are connected to PLC tags of the CPU. The HMI system cyclically exchanges the values between these tags. Data is transferred cyclically between SIMATIC S7 and the HMI system, that is, process variables are cyclically read by the HMI device depending on the configured update time.

### HMI Tags

HMI tags can be connected to the global PLC tags or to the following global data areas of the CPU:

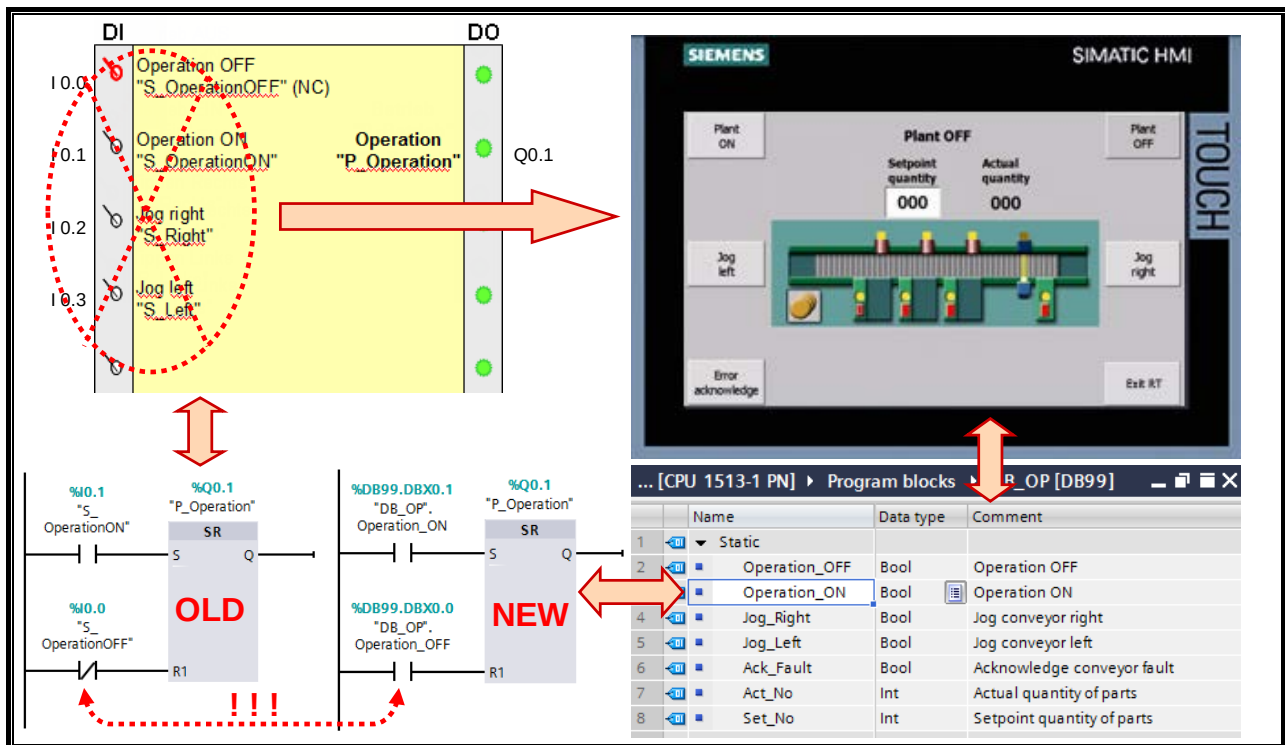
- Data blocks (DB)
- Bit memories (M)
- Inputs (I) and outputs (Q)
- I/O inputs and I/O outputs

HMI systems also recognize local tags without a link to the PLC, i.e. these tags are exclusively processed internally and also do not reserve any communication resources whatsoever.

### Communication

The operator panels can communicate with the (PLC) controller via the PROFIBUS or Industrial Ethernet bus systems. The S7 protocol is used for this purpose. Communication is handled by the operating systems of the S7 CPU and the HMI system. There is no user programming effort required. An operator panel can also exchange data with several (PLC) controllers.

### 11.3. Adjusting the S7 Program



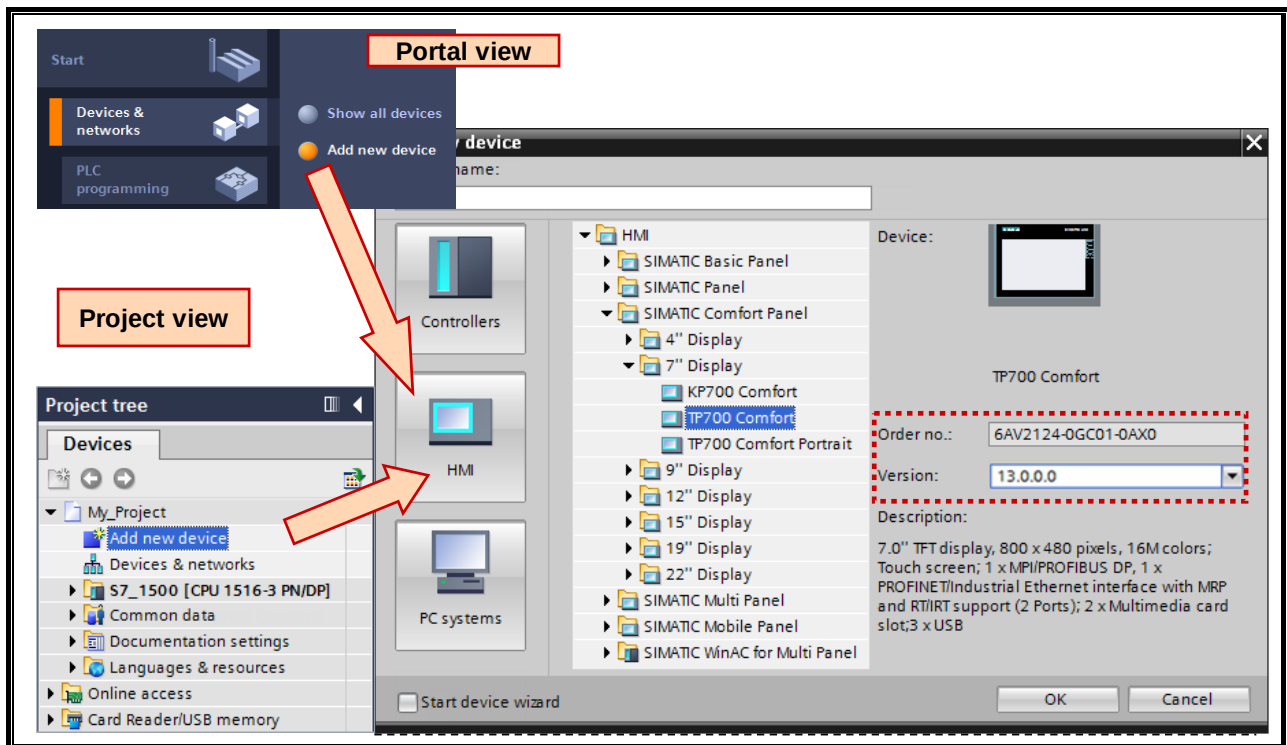
#### Adjusting the S7 Program

So that the plant can be operated via the touchpanel, PLC tags must be used in the S7 program which can be read or overwritten by the touchpanel.

In practice, data block variables (tags) are frequently used for this or data blocks are used as the "interface" between the PLC and the HMI device.



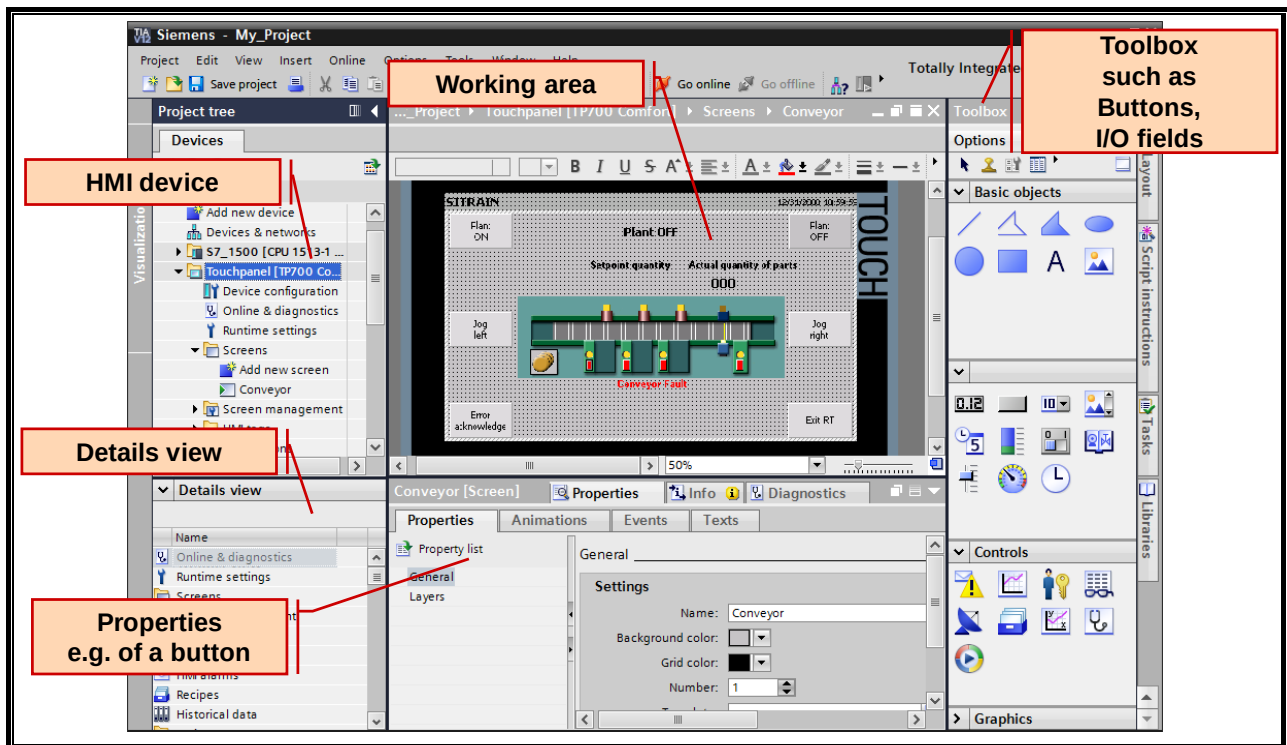
## 11.4. Adding an HMI Device



### Adding an HMI Device

New HMI devices can be added from both the Portal view and the Project view. More than anything else, attention has to be paid to the device data such as order number and version number.

## 11.5. WinCC Configuration Interface



### Project Window

In the Project tree, all devices and their configuration and parameter assignments are displayed in a tree structure. From there, the relevant editors can be opened. Furthermore, the "language support" and the "version management" can be found here.

### Working Area

This is the central configuration area in which objects of the operator panel are edited with the started editor. Several editors can be open at the same time.

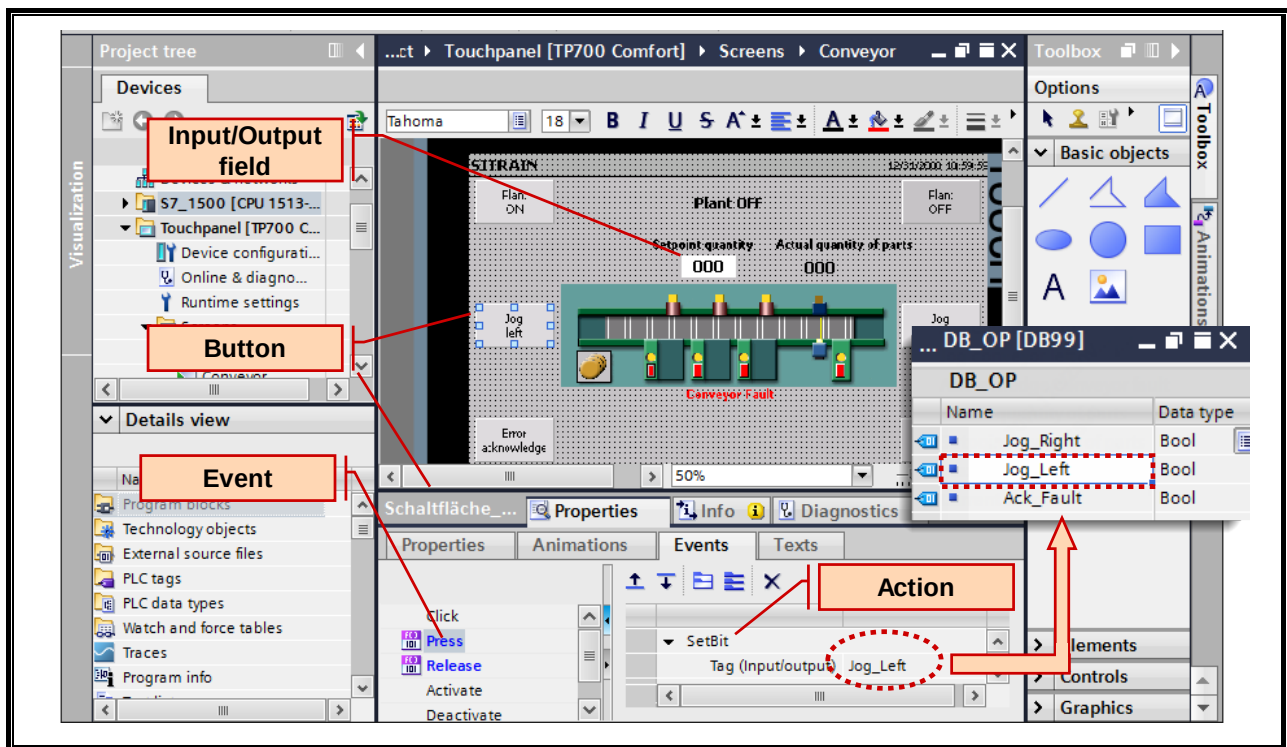
### Properties Window

The properties of selected objects (for example, of screens, screen objects, tags) can be edited in the Properties window. This window is only available in those editors where object properties have to be set.

### Toolbox Window

The toolbox window contains all configurable objects which can be configured in screens, and permits access to libraries.

### 11.5.1. Buttons and Input / Output Fields



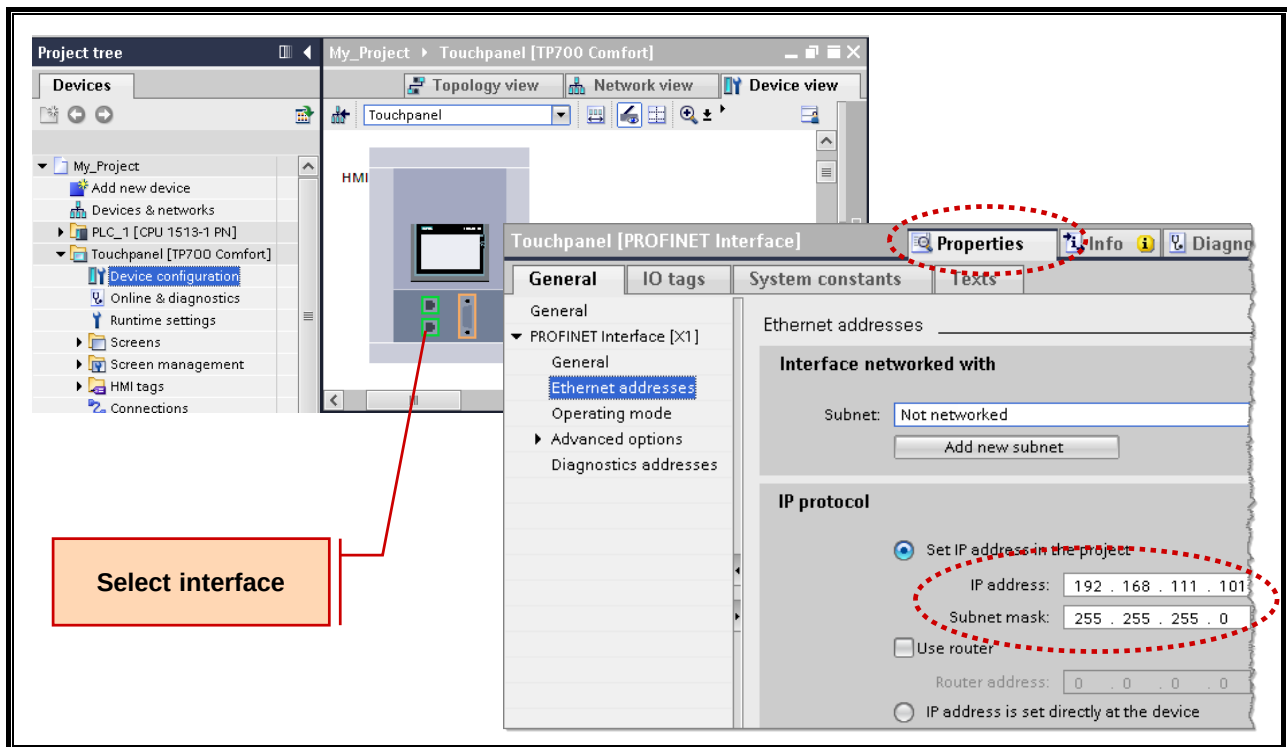
#### Buttons

System functions can be initiated by the operator via buttons, such as, the selection of a screen or the setting and resetting of a tag (shown in the picture above). The "Events" of a button are used to specify for which event which system function is to be executed.

#### I/O Fields

The values of tags are displayed via output fields; the values of tags can also be preset via input fields. The mode can be set in the Properties window.

## 11.6. Configuring the IP Address of a Touchpanel

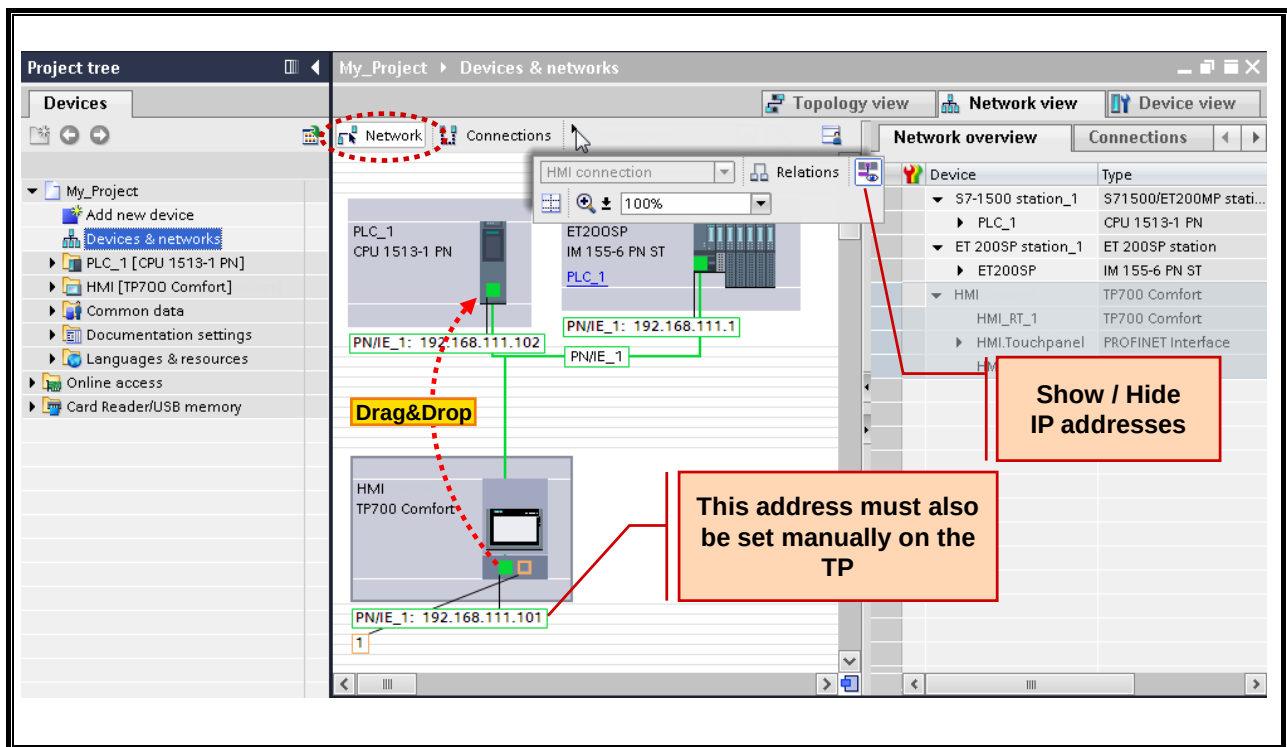


### PROFINET Interface of the Touchpanel

Regardless of whether the Devices & networks editor is in the Devices view or the Network view, the settings of the PROFINET interface (IP address and subnet mask) can be made in the "Properties" tab in the Inspector window for a selected HMI device interface.

If an online connection between the HMI device and the CPU is to be established, both devices must be assigned the same subnet mask and IP addresses that are in the same subnet.

### 11.6.1. Networking a Touchpanel

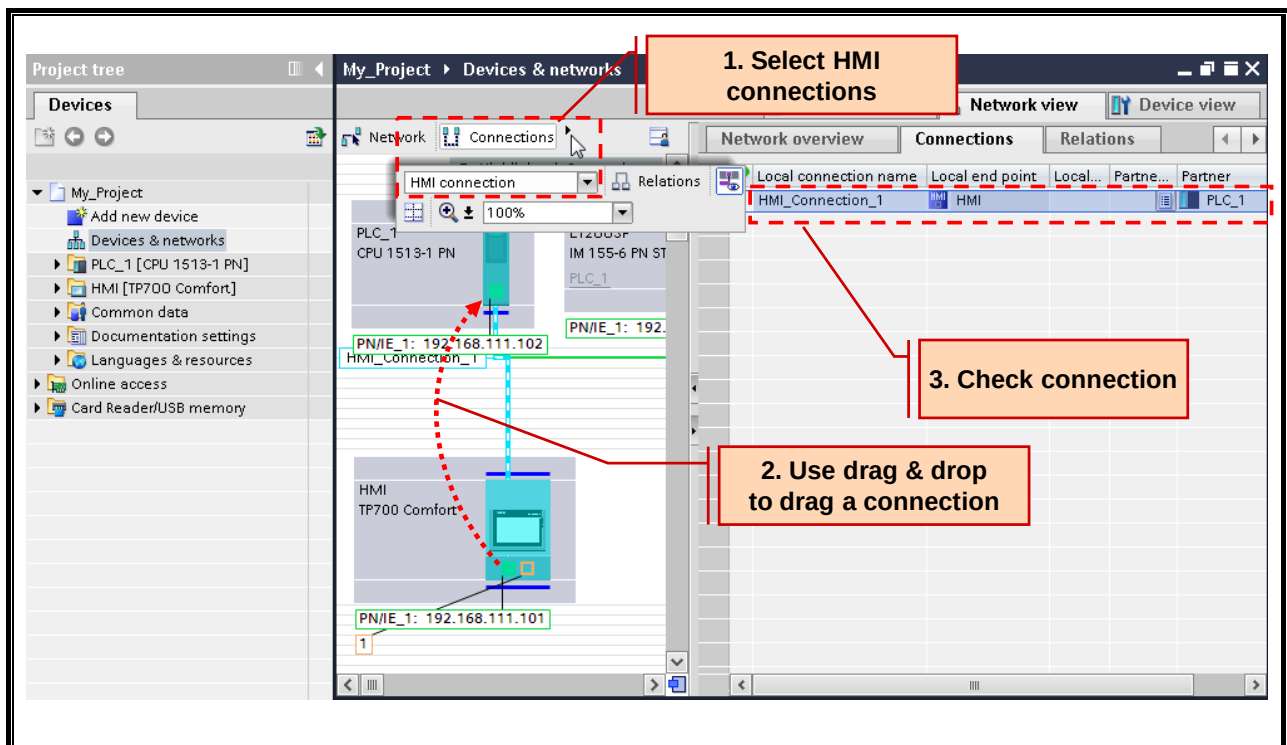


#### Networking a Touchpanel

During networking, devices are connected to a subnet. The device interface must be compatible with the type of network.

The devices are networked with the "Devices & networks" editor in the "Network" view by connecting the device interfaces using drag & drop.

### 11.6.2. Configuring an HMI Connection

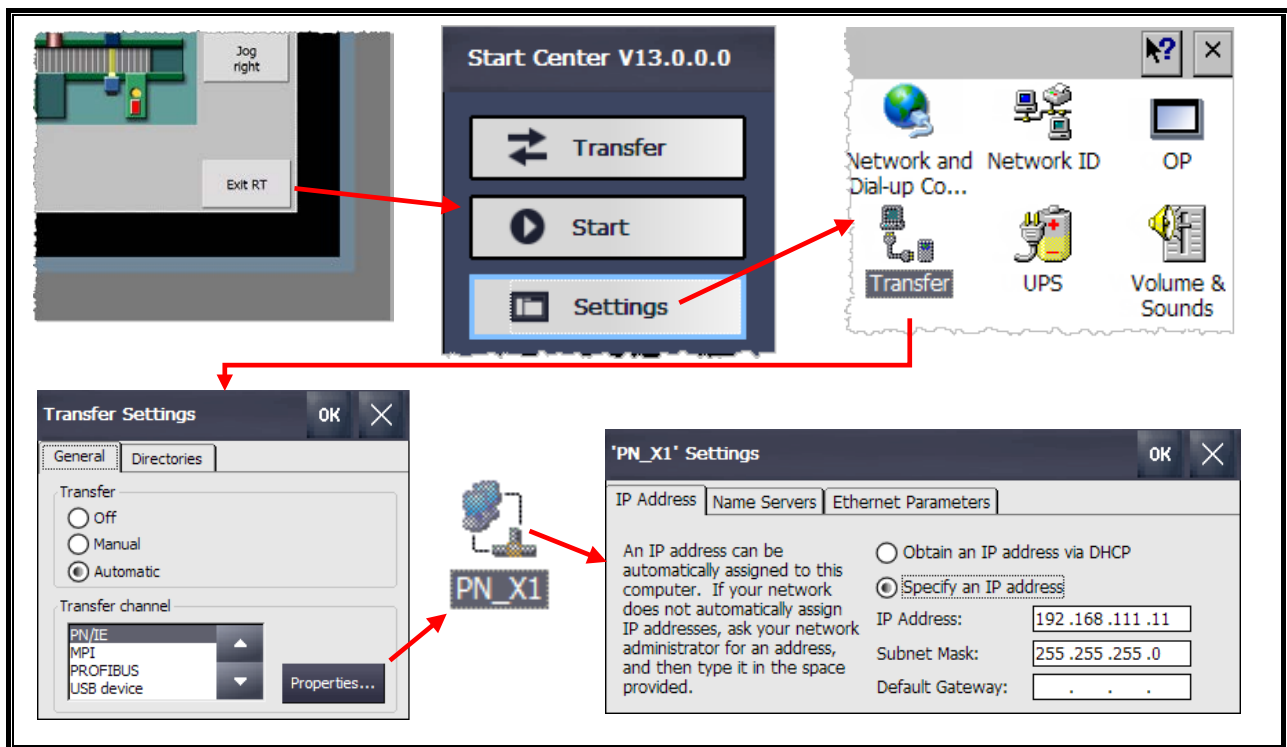


#### Configuring HMI Connections

In configuring HMI connection(s), the communications partners are defined with which the HMI device will later exchange data in the process control phase. The HMI device can also be connected to or exchange data with several controllers.

There can also be controllers in the same network with which the HMI device does not exchange data. Then, the HMI device is "networked" with these controllers but it is not "connected".

### 11.6.3. Setting the IP Address on the Touchpanel



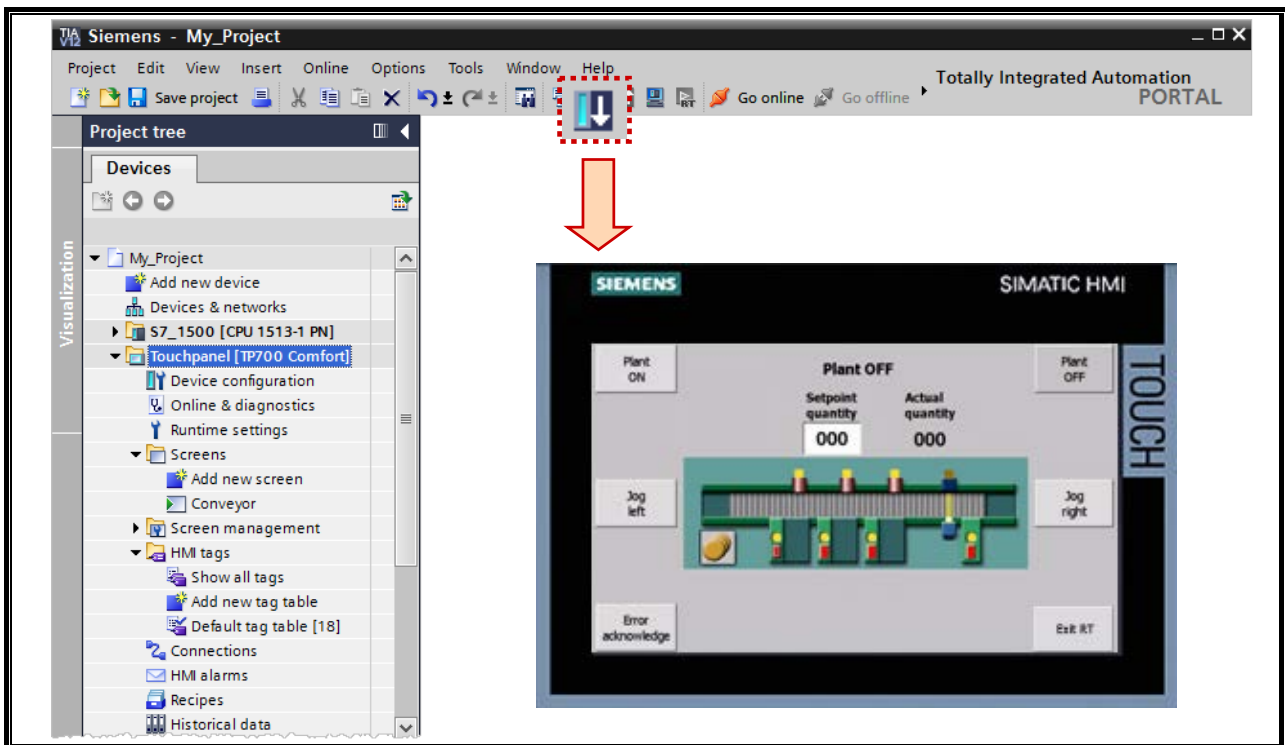
#### Setting the IP Address of the Touchpanel

The interface of the touchpanel must be set to the same IP address and subnet mask as it is also configured in the offline project.

#### Dialog "Transfer Settings" > Transfer > Automatic

You can initiate the loading of the WinCC flexible project without having to first manually end the Runtime of the HMI device. The Panel automatically ends the Runtime and switches to the Transfer Mode.

## 11.7. Downloading the HMI Project into the Touchpanel



### Downloading the HMI Project into the Touchpanel

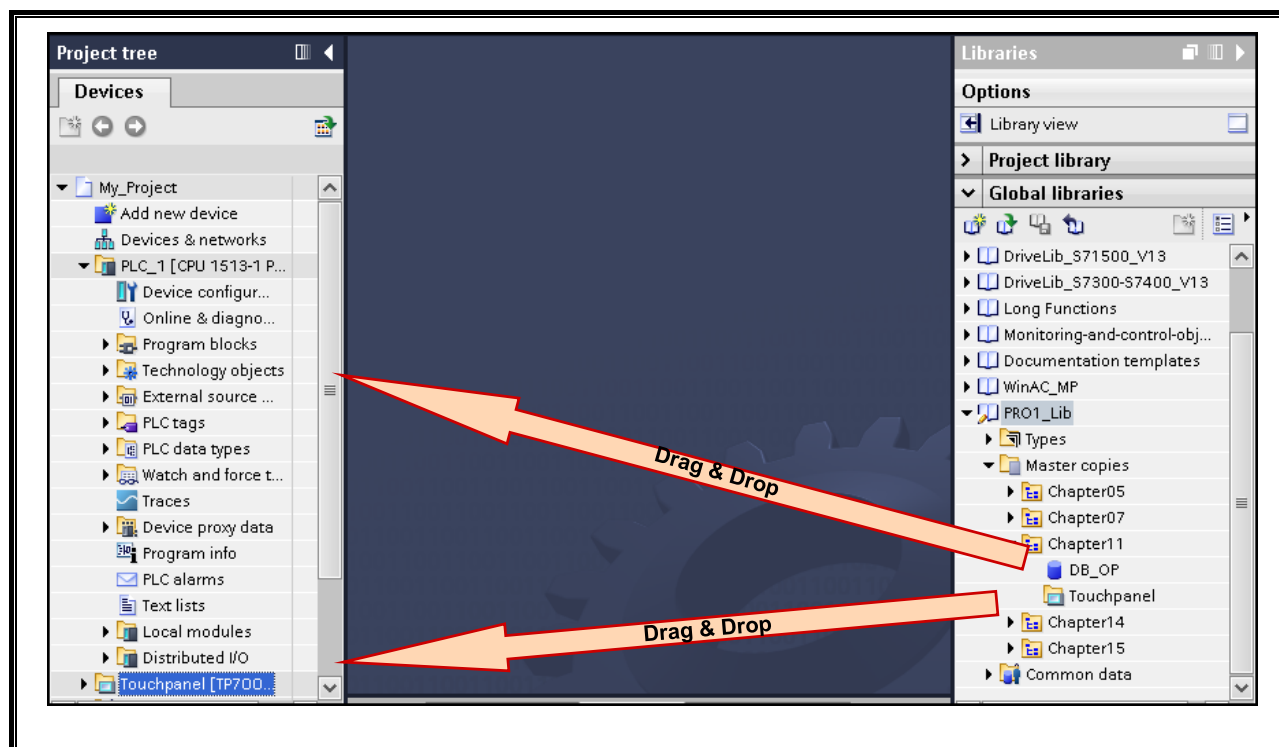
When you transfer an HMI project to one or more operator panels, the part of the project that has been changed since the last transfer is automatically compiled before downloading. This ensures that the current project status is always transferred. Beyond that, it is also possible to activate the option "Overwrite all" before loading starts.

For commissioning, the project should be completely compiled using the command "Compile > Software (complete)" in the context menu of the operator panel. If HMI tags that are linked to PLC tags are also used in the project, all modified STEP 7 blocks should also be compiled using the command "Compile > Software" in the context menu and then be downloaded into the CPU.

It is also advisable to execute the "Compile > Software (complete)" command occasionally to reduce the time required for compiling delta data in current engineering sessions.



## 11.8. Exercise 1: Copying the Touchpanel Project and the Interface Data Block from the Library

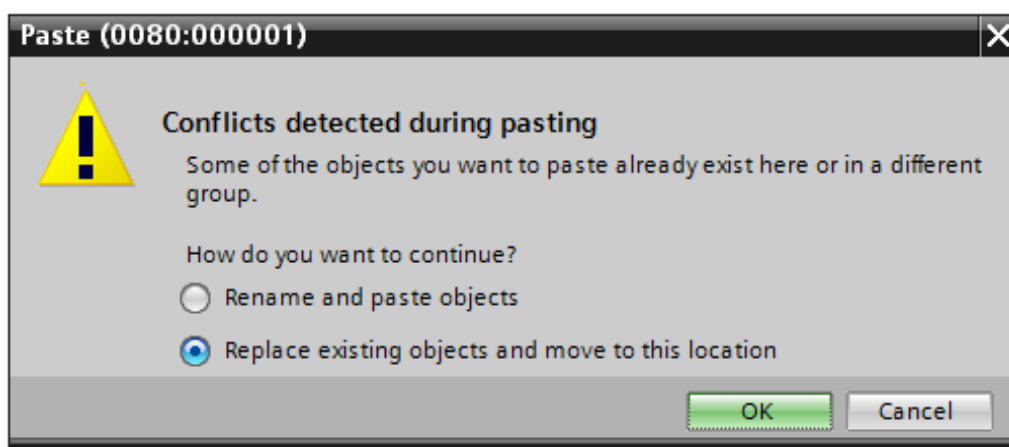


### Task

Up until now, your project doesn't contain an HMI device. Instead of a completely new configuration, you are to copy a prepared panel project and the data block "DB\_OP" (DB99) that is to serve as the interface between the controller and the touchpanel, from the global library "PRO1\_Lib" into your project.

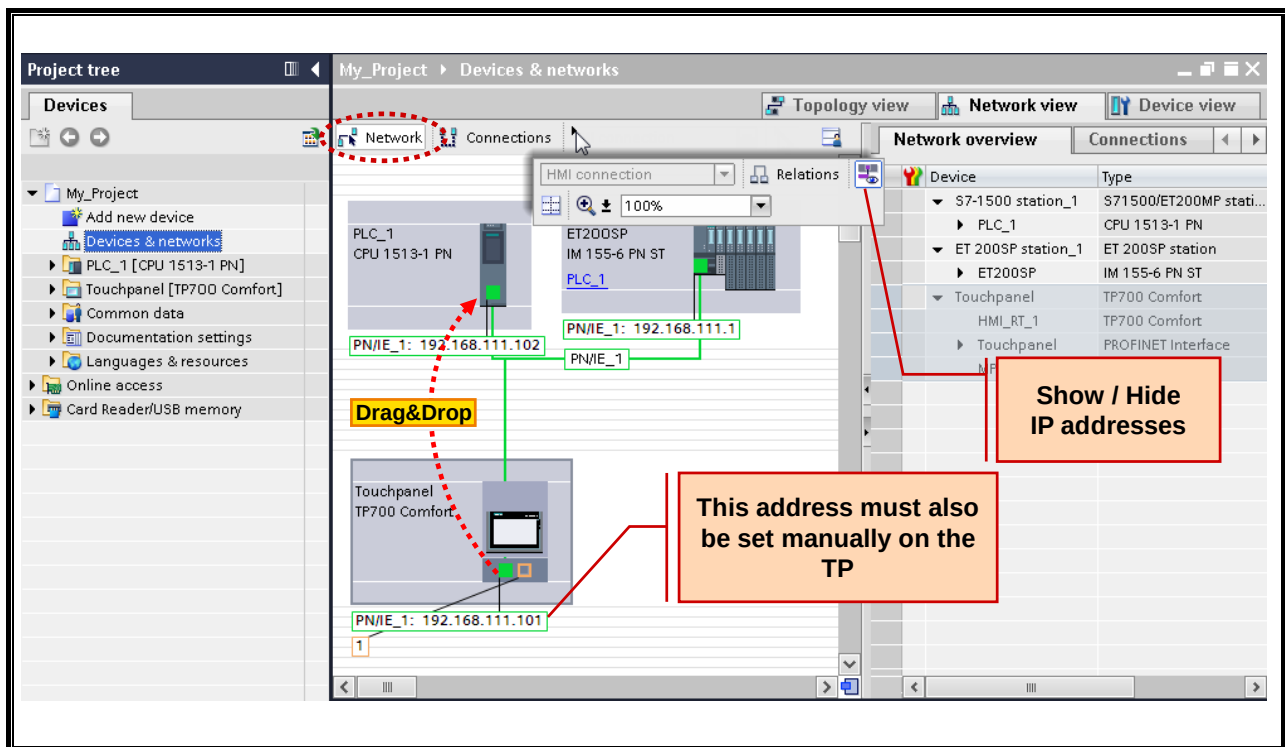
### What to Do

1. Open the Global libraries <Drive>:\Archives\TIA\PRO1\_Lib.
2. Using drag & drop, copy the library element "DB\_OP" (DB99) into your project (see picture) and, in the dialog that appears, confirm that objects that already exist in the project are to be overwritten.



3. Using drag & drop, copy the library element "Touchpanel" into your project (see picture).
4. Save your project.

### 11.8.1. Exercise 2: Networking the Touchpanel

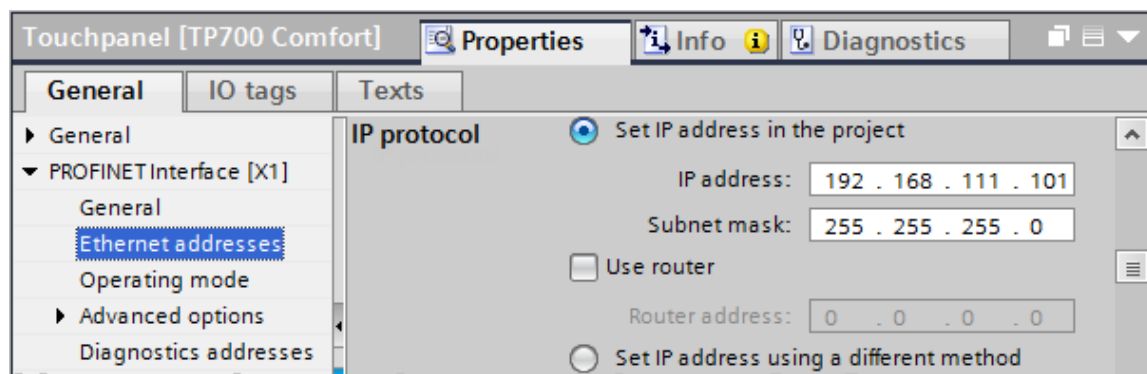


#### Task

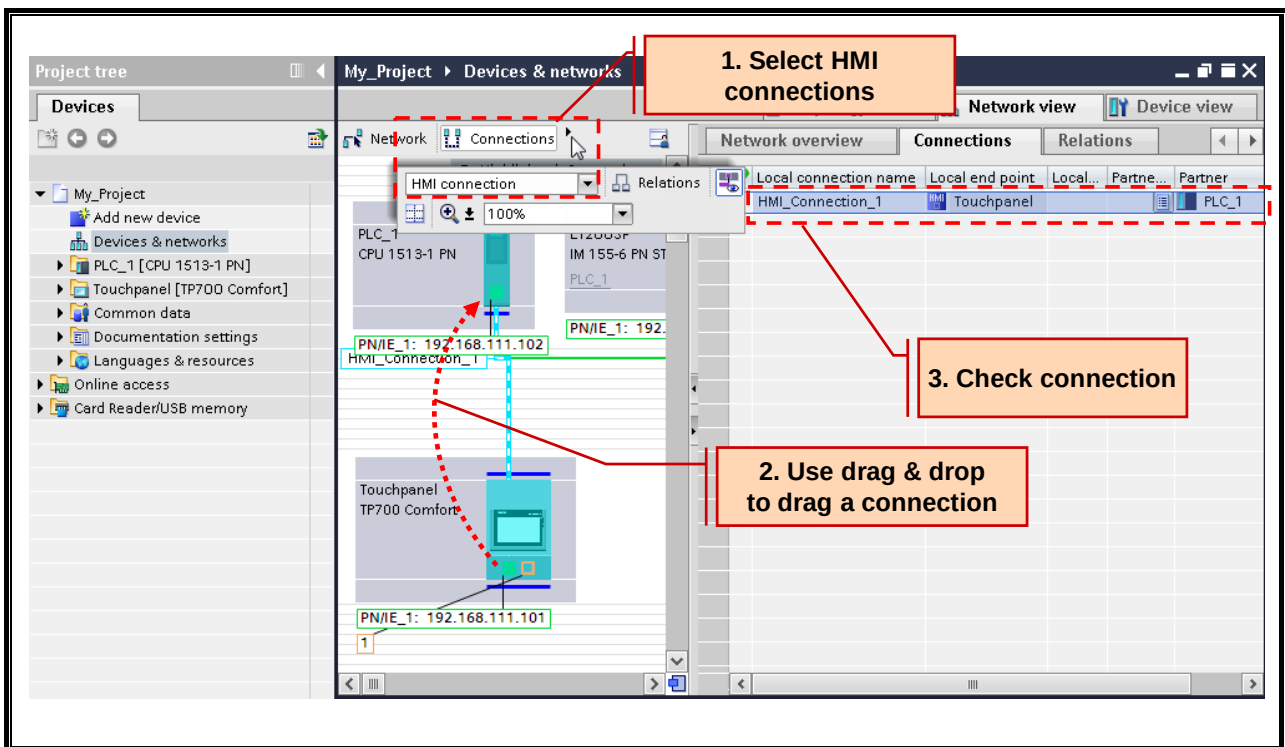
The added touchpanel is to be networked offline with the Ethernet network.

#### What to Do

1. In the Project tree, start the "Devices & networks" editor, switch to the "Network view" and there select "Network".
2. Position the mouse pointer on the small green square of the HMI device and, while keeping the left mouse button pressed down, drag a connection to the CPU. The network is created; the associated subnet and the parameters appropriate for the network (IP address and subnet mask) are automatically created.
3. Display the IP addresses of the devices via the button shown in the picture.
4. Select the touchpanel and switch to the "Device view".
5. In the Inspector window in the "Properties" of the touchpanel, check and correct the Ethernet address:



### 11.8.2. Exercise 3: Configuring the HMI Connection



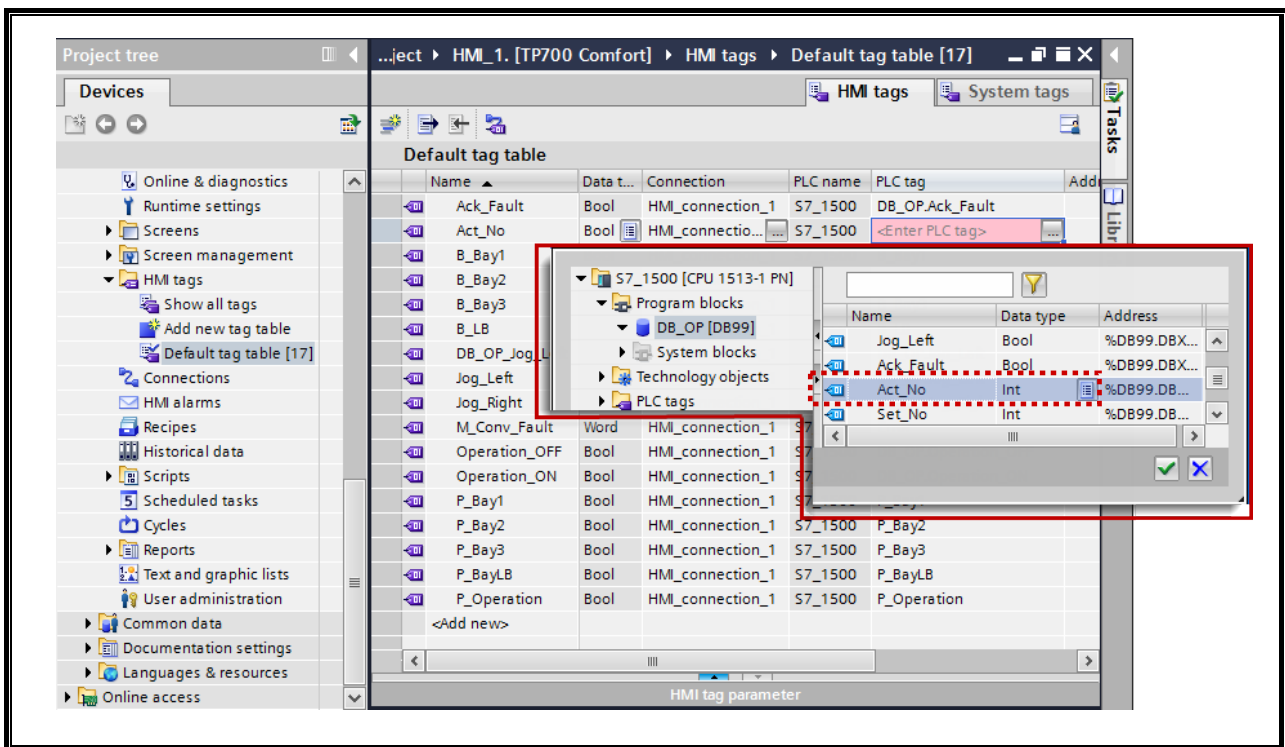
#### Task

Now that the TP is networked with the Ethernet network, an HMI connection between the TP and the CPU must be created. The automatically assigned name of the connection "HMI\_Connection\_1" can be adopted without changing it.

#### What to Do

1. In the Project tree, start the "Devices & networks" editor, switch to the "Network view" and there select "Connections" (see picture).
2. Position the mouse pointer on the small green square of the HMI device and, while keeping the left mouse button pressed down, drag a connection to the CPU. The connection is created and given the default name "HMI\_Connection\_1".
3. In the Details window, in the "Connections" tab, check whether the HMI connection was correctly created (see picture).
4. To display the type of connection between the S7-CPU and the HMI device, position the mouse pointer on the network and, in the dialog window that appears, select "HMI connection".

### 11.8.3. Exercise 4: Completing the HMI Tag Connection



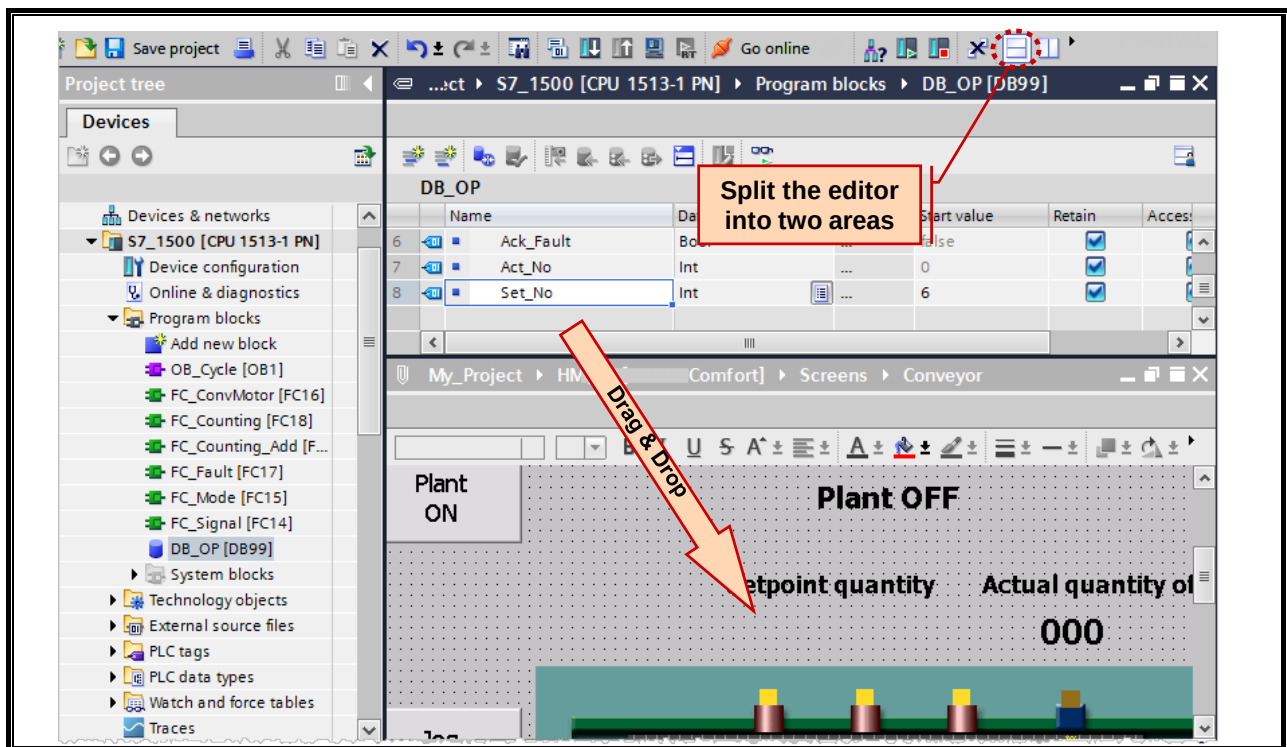
#### Task

The HMI tags "P\_Operation", "M\_Conv\_Fault" and "Act\_No" are not yet connected to PLC tags; for other tags, the "connection" may be highlighted in red. Your task is now to enter the missing tag connections and to correct any faulty ones.

#### What to Do

1. Open the "HMI tags" of the HMI device.
2. Connect the HMI tags that are not yet connected to the corresponding PLC tags. Proceed as shown in the picture.
3. Correct the still faulty connections so that there are no more red entries in the tag table.
4. Compile the entire HMI project.
5. Save your project.

### 11.8.4. Exercise 5: Configuring the SETPOINT Quantity Display

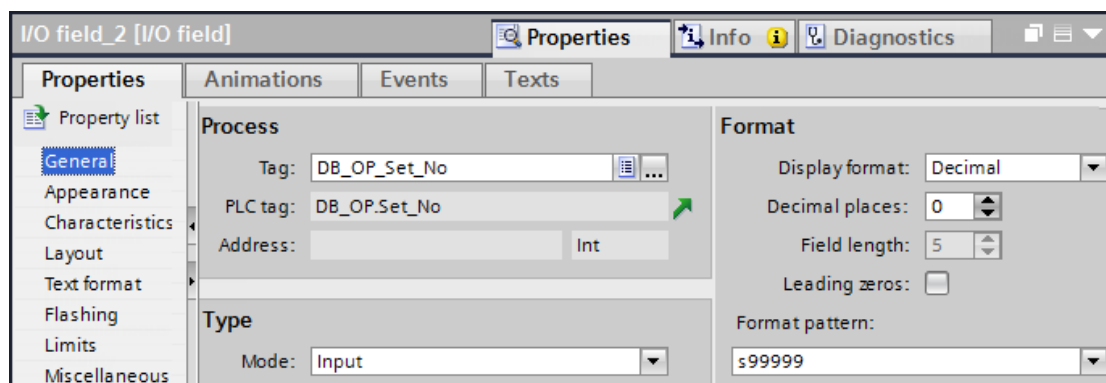


#### Task

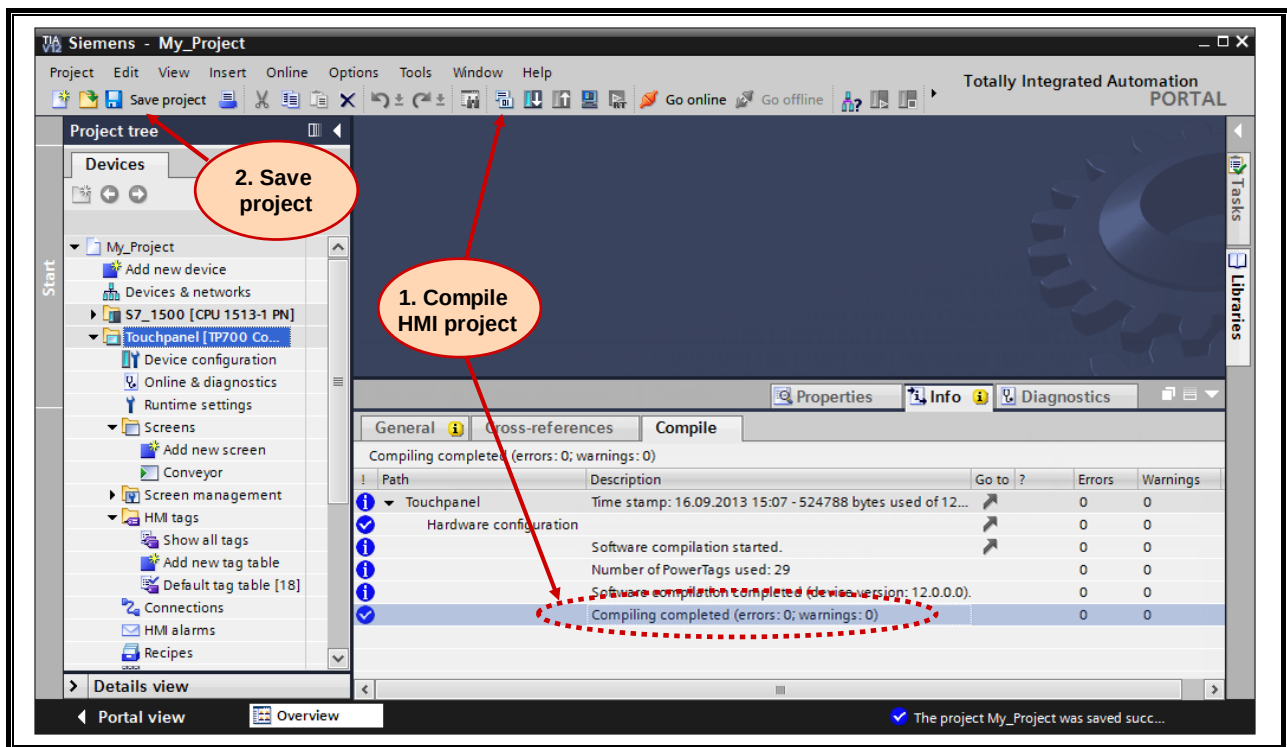
Up until now, the SETPOINT quantity of parts to be transported was preset with the value 3 using the tag DB\_OP.Set\_No. Now, instead, you should be able to preset the SETPOINT quantity via an input field on the touchpanel. For this, an additional input/output field must be configured in the touchpanel screen (see picture).

#### What to Do

1. Close all objects that are open in the editor.
2. Open the "DP\_OP" (DB99) data block in which the SETPOINT quantity is stored.
3. Open, as well, the touchpanel screen "Conveyor".
4. Split the editor area into two areas using the button shown in the picture.
5. Configure the input field for presetting the setpoint quantity by dragging the tag "Set\_No" from the data block into the touchpanel screen using drag & drop.
6. Select the newly created input/output field and in the Inspector window under "Properties -> General" for "Type" set "Input" as the Mode.



### 11.8.5. Exercise 6: Compiling and Saving the HMI Project



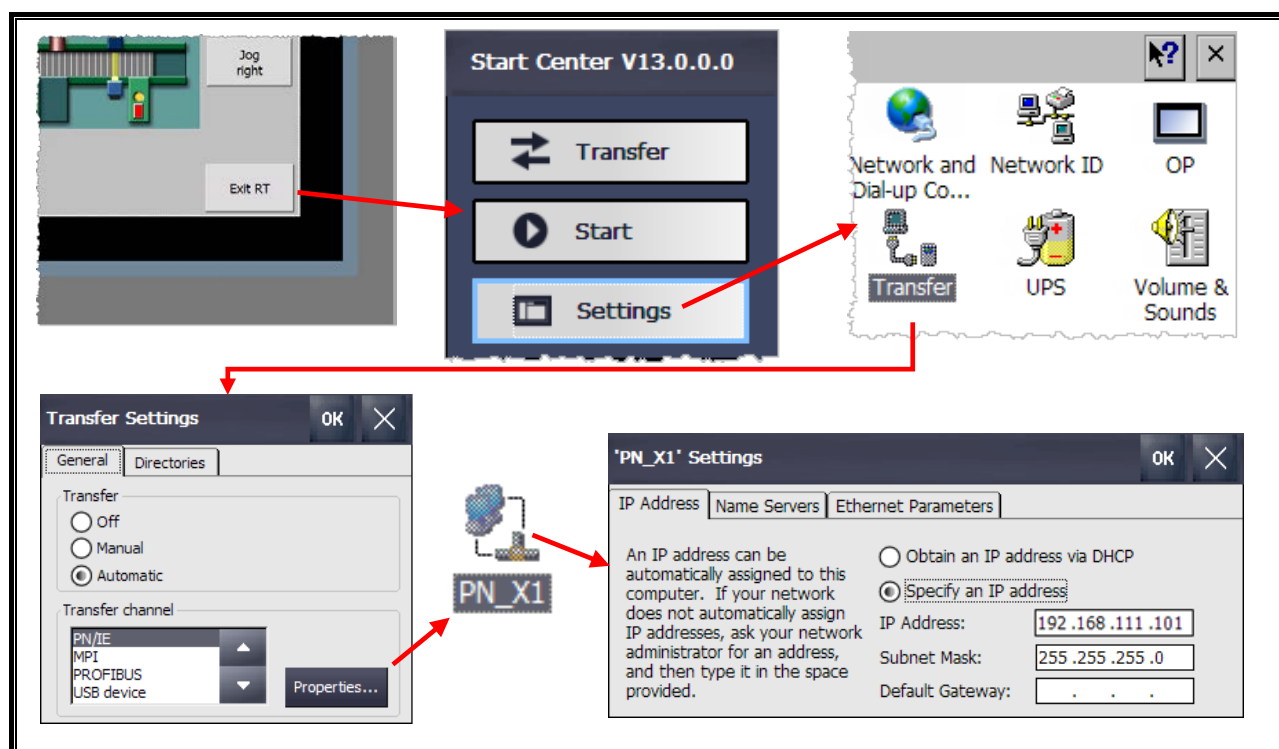
#### Task

The now complete HMI project is to be compiled and saved.

#### What to Do

1. Compile the HMI project by selecting the touchpanel in the Project tree and then clicking on the "Compile" button (see picture).
2. In the Inspector window under "Info", read the results of the compilation and eliminate any errors which may have occurred.
3. Save your project.

### 11.8.6. Exercise 7: Setting the IP Address on the TP



#### Task

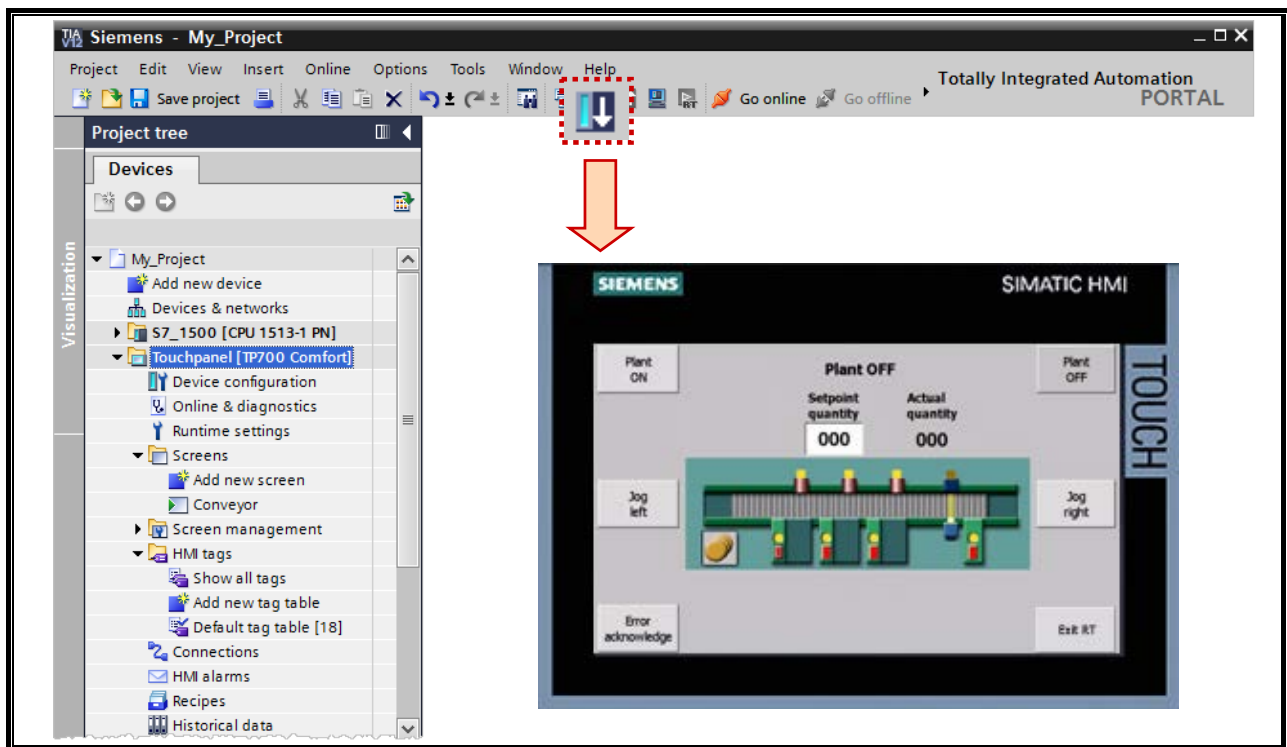
The interface of the touchpanel is to be set as shown in the picture so that the configuration can then be loaded into the panel.

Runtime must be exited before the interface can be assigned parameters. For this, a button for exiting Runtime is generally provided in the Start screen. When Runtime has been exited, the "Start Center" appears through which the "Settings" can be activated. The Start Center also appears every time power is restored.

#### What to Do

1. Exit the Runtime of the touchpanel.
2. Activate the Settings.
3. Select "Settings" by double-clicking on it.
4. Set the Transfer to Automatic
5. Activate the **"Properties..."** of the **PN/IE** interface **PN/X1** (see picture)
6. Implement the settings shown in the picture.
7. Go back to the Start Center by closing the windows with "OK".
8. Activate the "Transfer" button so that the touchpanel waits for a connection to be established by the PG.

### 11.8.7. Exercise 8: Downloading the HMI Project into the Touchpanel



#### Task

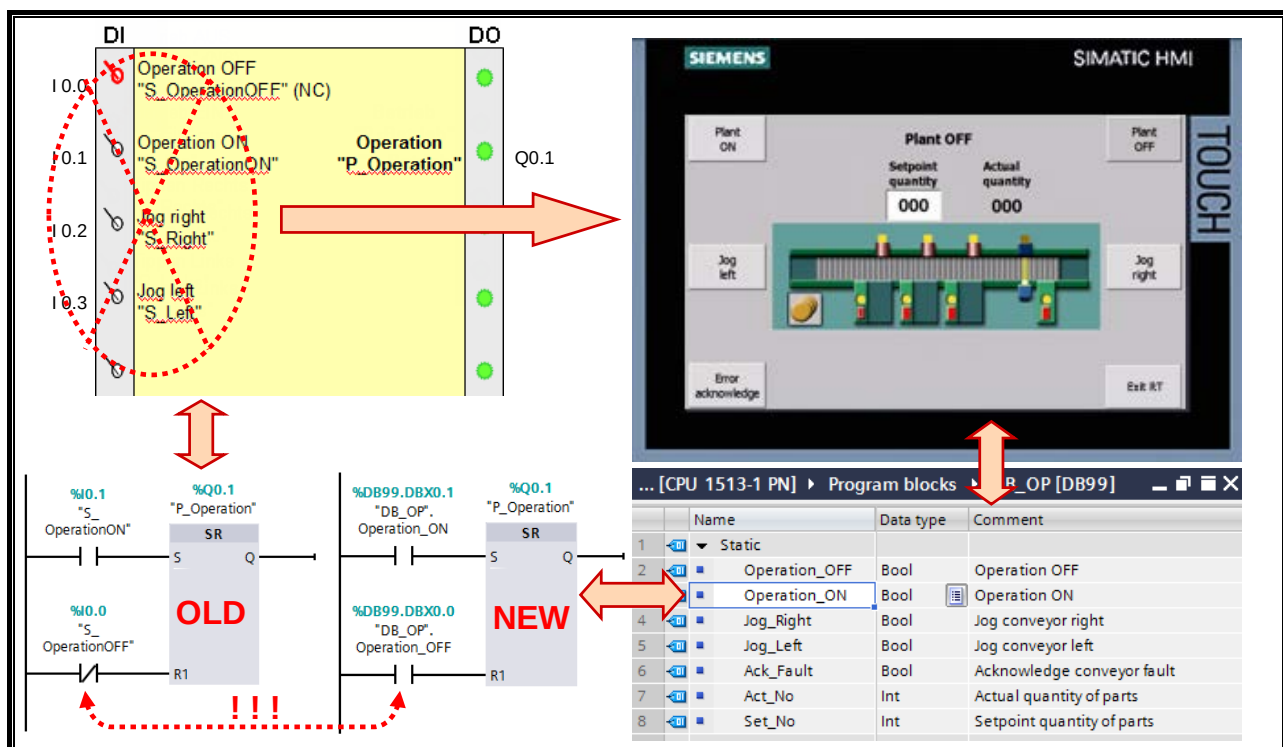
From the now complete project, all S7 blocks are now to be downloaded once more into the CPU and the entire panel project is to be transferred into the touchpanel.

#### What to Do

1. Completely compile the entire CPU program and the entire panel project.
2. In the Inspector window, in the "Compile" tab, check whether the compilation was successful. If need be, make the necessary corrections.
3. Download all S7 blocks into the CPU.
4. Transfer the Panel project to the touchpanel.
5. Carry out a function test by
  - Switching the operation ON and OFF.
  - When operation is OFF, jog the conveyor model to the right and left.
  - When operation is ON, preset a SETPOINT quantity and start part transportation.
  - Monitor how the ACTUAL quantity is updated.
  - Cause a conveyor fault, monitor its display on the touchpanel and acknowledge it on the touchpanel.
6. Correct your program – if necessary – and check the functions once more.
7. Save your project.



### 11.8.8. Exercise 9: Adjusting the STEP 7 Program



#### Task Description

The touchpanel project is to be commissioned and the S7 program of the controller is to be adjusted in such a way that...

- the functions "Operation ON/OFF" and "Jog Right/Left" are no longer realized via the simulator pushbuttons but via the corresponding buttons on the touchpanel.
- the acknowledgement of a conveyor fault should still be possible via the simulator pushbutton "S\_Acknowledge" (I 0.7), and, in addition, also via the corresponding button on the touchpanel.
- the SETPOINT quantity is no longer constant 3, but can be specified via an input/output field on the touchpanel.

#### What to Do

- In "FC\_Mode" (FC15), replace the variables "S\_OperationON" (I 0.1) and "S\_OperationOFF" (I 0.0) with the corresponding DB tags (see picture). Remember to also take into account that a hardware-based NC contact is connected to "S\_OperationOFF" (I 0.0).
- Expand "FC\_Fault" (FC17) in such a way that a fault acknowledgement is still possible via "S\_Acknowledge" (I 0.7) and, in addition, also via the touchpanel or the DB tag "DB\_OP".Ack\_Fault.
- In "FC\_ConvMotor" (FC16), replace the variables "S\_Right"(I 0.2) and "S\_Left" (I 0.3) with the corresponding DB tags.
- Check whether the setpoint quantity can be specified via the touchpanel.
- Check also all other functions and save your project.

# Contents

|                                                                                                    |             |
|----------------------------------------------------------------------------------------------------|-------------|
| <b>12. Functions and Function Blocks .....</b>                                                     | <b>12-2</b> |
| 12.1. Task Description: Fault Evaluation with Parameter-assignable Blocks.....                     | 12-3        |
| 12.2. Local and Global Operands .....                                                              | 12-4        |
| 12.2.1. Management of the Local Data Stack.....                                                    | 12-5        |
| 12.2.2. The Use of Local, Temporary Variables .....                                                | 12-6        |
| 12.3. Parameter-assignable Blocks: Example: Fault Evaluation .....                                 | 12-7        |
| 12.3.1. Solution with Parameter-assignable Block .....                                             | 12-8        |
| 12.3.2. Declaration of Formal Parameters .....                                                     | 12-9        |
| 12.3.3. Editing a Parameter-assignable Block.....                                                  | 12-10       |
| 12.3.4. Calling a Parameter-assignable Block.....                                                  | 12-11       |
| 12.4. Task Description: Fault Evaluation by means of a Function (FC) .....                         | 12-12       |
| 12.4.1. Exercise 1: Programming the Parameter-assignable Function "FC_FaultEvaluation" (FC20)..... | 12-13       |
| 12.4.2. Exercise 2: Calling and Parameterizing "FC_FaultEvaluation" (FC20) .....                   | 12-14       |
| 12.5. Task Description: Fault Evaluation by means of a Function Block (FB).....                    | 12-15       |
| 12.5.1. Instantiating FBs .....                                                                    | 12-16       |
| 12.5.2. FB - Declaration Part .....                                                                | 12-17       |
| 12.5.3. Generating Instance Data Blocks .....                                                      | 12-18       |
| 12.5.4. Exercise 3: Editing "FB_FaultEvaluation" (FB20) .....                                      | 12-19       |
| 12.5.5. Exercise 4: Calling "FB_FaultEvaluation" (FB20) .....                                      | 12-20       |
| 12.6. Adding Block Parameters Later On .....                                                       | 12-21       |
| 12.7. Deleting Block Parameters Later On .....                                                     | 12-22       |
| 12.8. Updating a Block Call.....                                                                   | 12-23       |

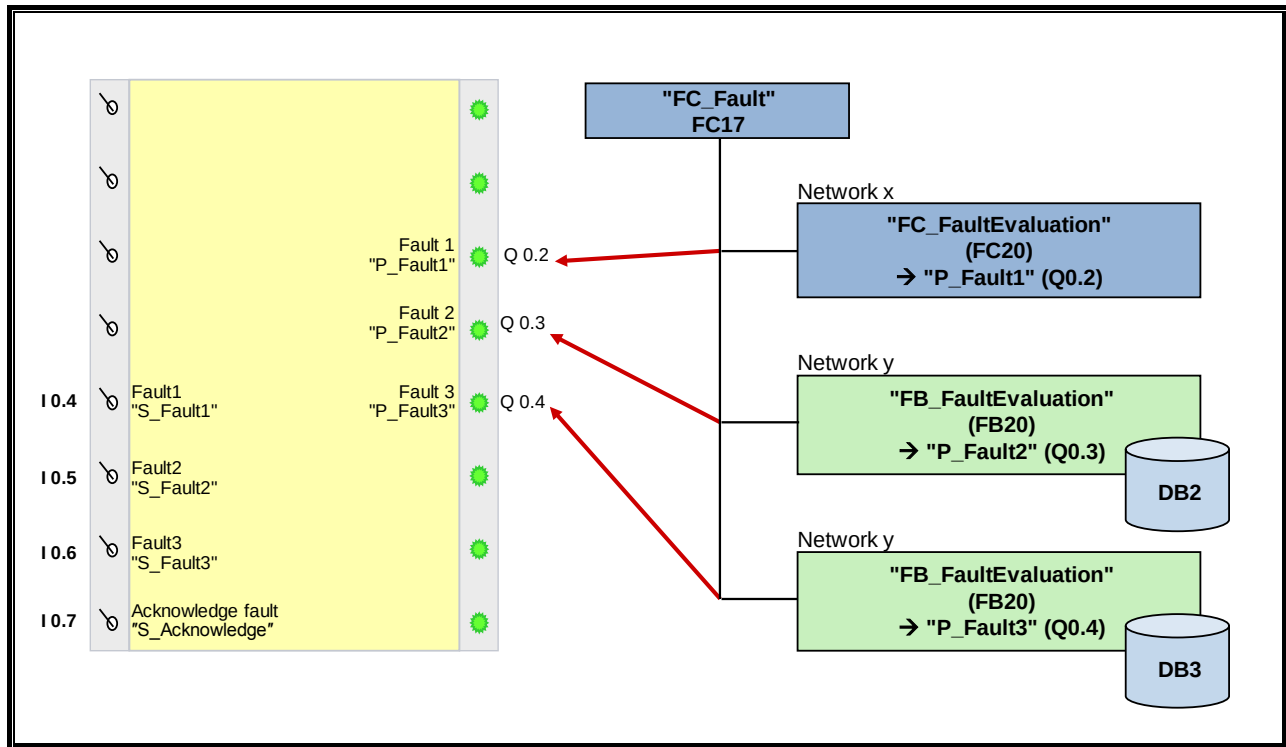
## 12. Functions and Function Blocks

At the end of the chapter the participant will ...

- ... be familiar with the purpose of temporary variables
- ... be familiar with the purpose of parameter-assignable blocks
- ... be able to program parameter-assignable functions and their calls
- ... know the difference between functions (FCs) and function blocks (FBs)
- ... be familiar with the purpose of static variables
- ... be able to declare static variables and apply them in the program
- ... be able to program parameter-assignable function blocks and call them



## 12.1. Task Description: Fault Evaluation with Parameter-assignable Blocks



### Task Description

Independent of the conveyor model functions so far, 3 different faults are to be evaluated as follows.

If a fault is triggered at the simulator inputs "S\_Fault1" (I 0.4) to "S\_Fault3" (I 0.6), the associated simulator LEDs "P\_Fault1" (Q0.2) to "P\_Fault3" (Q0.4) begin to flash.

A group acknowledgement for all faults takes place using the simulator input "S\_Acknowledge" (I 0.7). If the fault still exists after acknowledgement, the LED changes to continuous light; if the fault no longer exists, the LED goes dark.

## 12.2. Local and Global Operands

| Global Operands<br>(valid in the entire program)                                                                                                                                                                                                               | Local Operands<br>(only valid in one block)                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• PII / PIQ</li> <li>• I / O peripherals</li> <li>• Bit memories</li> <li>• Variables in DBs</li> <li>• S5-Timers *)</li> <li>• S5-Counters *)</li> <li>• Constants</li> </ul> <p>*) only for S7-300/400/1500/WinAC</p> | <b>Formal Parameters</b> <ul style="list-style-type: none"> <li>• interface for data exchange between calling and called block</li> <li>• temporary storage in the L-stack for FCs or storage in the IDB for FBs</li> <li>• can be used in FCs / FBs</li> </ul> |
|                                                                                                                                                                                                                                                                | <b>Temporary Variables</b> <ul style="list-style-type: none"> <li>• are overwritten after the block is executed</li> <li>• temporary storage in the L-stack</li> <li>• can be used in OBs / FCs / FBs</li> </ul>                                                |
|                                                                                                                                                                                                                                                                | <b>Static Variables</b> <ul style="list-style-type: none"> <li>• retain their value after the block is executed</li> <li>• permanent storage in IDBs</li> <li>• can <u>only</u> be declared in FBs</li> </ul>                                                   |
|                                                                                                                                                                                                                                                                | <b>Constants</b> <ul style="list-style-type: none"> <li>• read-only and symbolic access</li> <li>• no memory usage</li> <li>• can be used in OBs / FCs / FBs</li> </ul>                                                                                         |

### Global Operands

Global operands are valid throughout the entire S7 program. Accordingly, every logic block (OB, FC, FB) can access these operands.

Global operands include the traditional PLC operands - inputs, outputs, bit memories, SIMATIC timers, SIMATIC counters, constants and variables - which are declared in global data blocks.

### Local Operands

Local operands are only valid in the block in which they were declared in the declaration part. Accordingly, only this block can access them.

- Temporary Variables

Temporary variables can be declared in every logic block (OB, FC, FB) and are managed in the local data stack of the CPU. Accordingly, they only retain their values while the block is being executed. For that reason, it is important that in the current cycle, a write access must have taken place on the temporary variable in the block before a read access can take place. They are unsuitable, for example, as auxiliary variables for edge evaluations or to store quantities. They are, in fact, used to store intermediate results, such as, for complex calculations or format conversions.

- Static Variables

Static variables can only be declared in function blocks (FBs) and are stored in the associated instance data block (IDB). Accordingly, these variables retain their value even after the FB is executed.

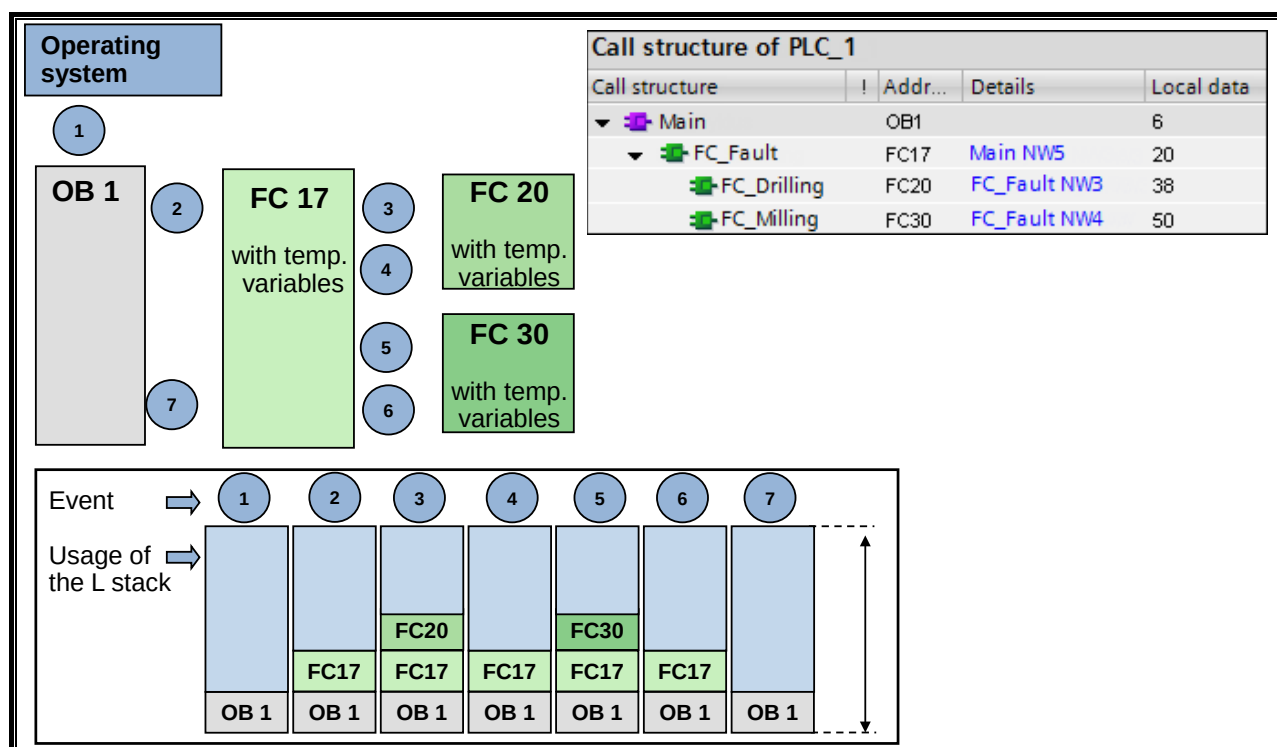
- Formal Parameters

Formal parameters form the interface between calling and called block. They are used to realize a data exchange between the calling and the called block.

- Constants

Constants are fixed values which have a read-only access and which do not take up any memory space.

### 12.2.1. Management of the Local Data Stack



#### Total Usage of the Local Data Stack (L-Stack)

For every program execution level or priority class (such as, OB 1 with all blocks that are called in it), a separate local data stack is reserved. That is, a segment of defined size is reserved on the L stack of the CPU (allocation or reservation of memory space).

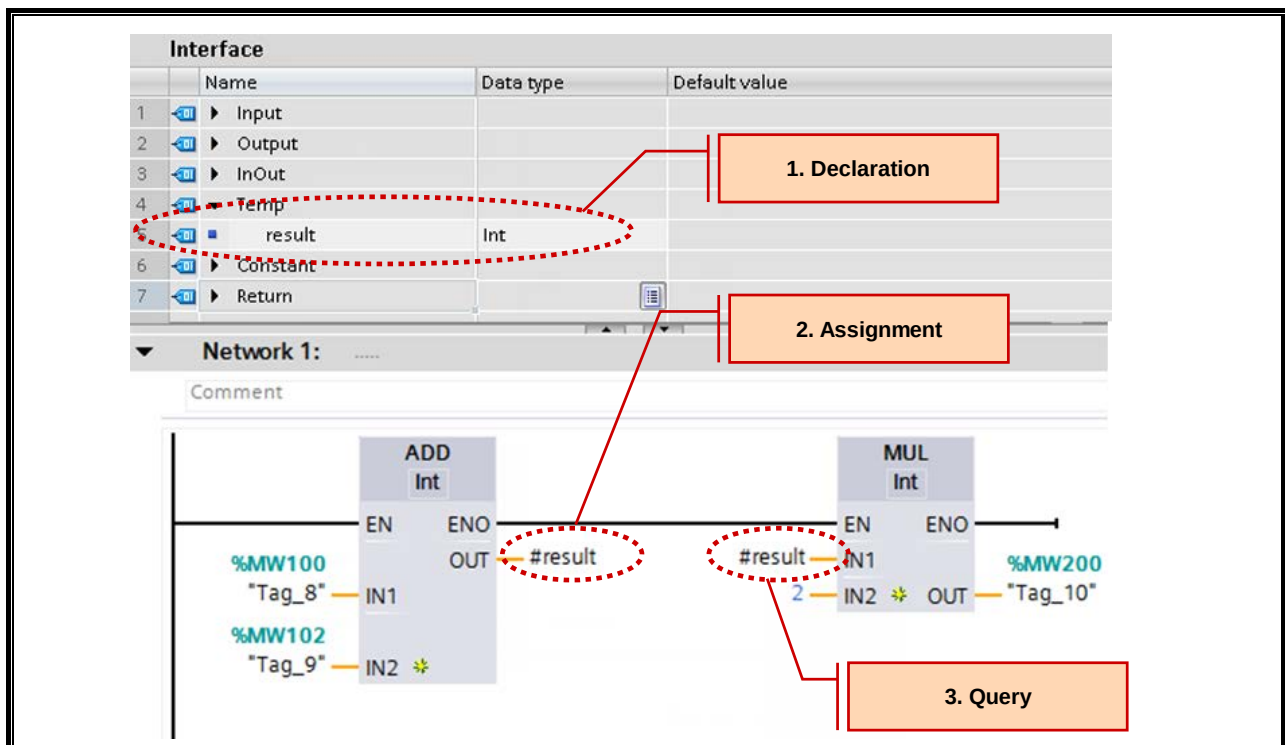
The local operands of OB 1 as well as the local, temporary variables of the blocks (FCs and FBS) called in or by OB 1 are stored in this local data stack.

You can use the "Reference Data" tool to display the "Program Structure" to see to what extent an S7 program puts a burden on the local data stack. The reference data are dealt with in more detail in the chapter "Troubleshooting".

#### Note

If the (CPU-specific) maximum number of local data is exceeded (overflow of the L stack) during program execution in the CPU, the CPU goes into the STOP state. "STOP caused by error when allocating local data" is entered as the cause of error in the diagnostics buffer.

### 12.2.2. The Use of Local, Temporary Variables



#### Declaration

The variables are defined in the declaration part of the block. The name of the variable and the data type must be specified.

You cannot predefine the variable with an initial value. After you have saved the block, the relative address (storage location) of the variable in the L stack is displayed in the "Address" column.

#### Access

With optimized blocks, all temporary variables are initialized with 0 at the beginning of block execution.

With non-optimized blocks, all temporary variables have an undefined value at the beginning of block execution. When working with temporary variables, you must therefore make sure that the variable is first of all assigned a defined value before it is queried.

In the example, the result of Add(ition) is assigned to the temporary variable #result before it is then queried at the Mul(tiplication).

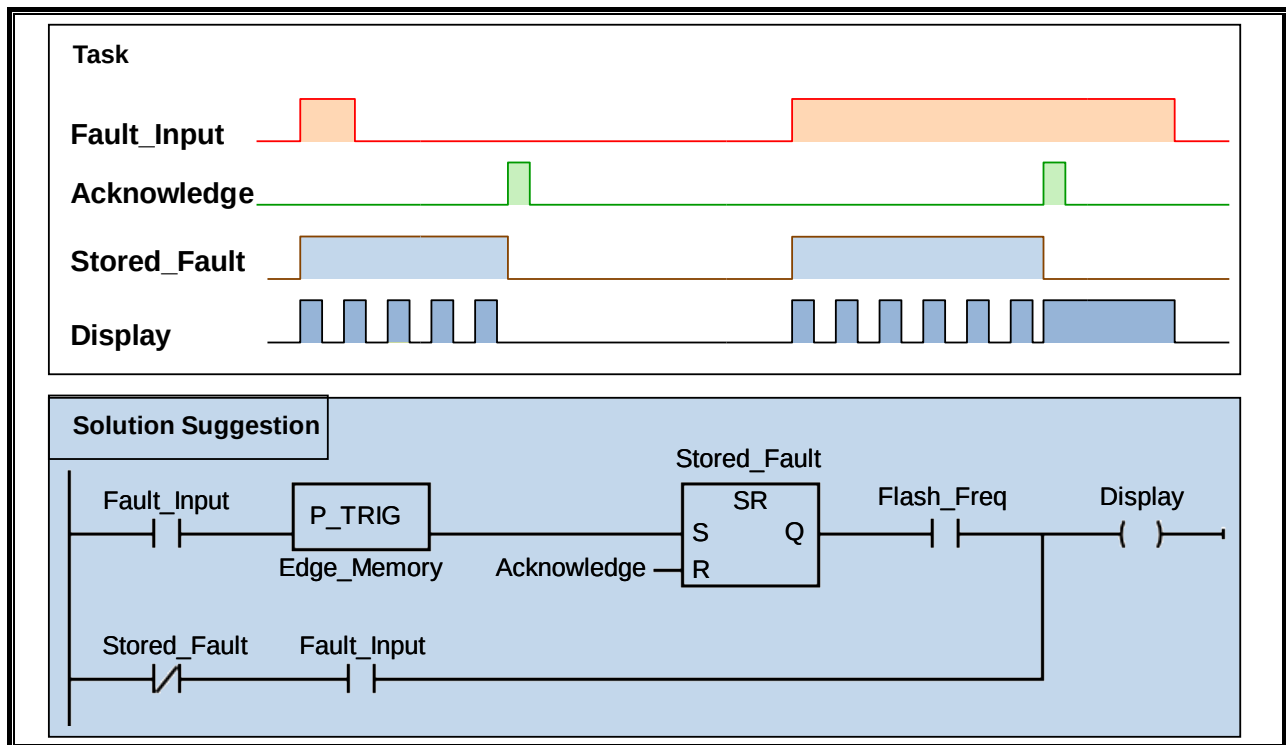
You can also access temporary variables absolutely (such as T LWx). You should, however, try to avoid this since the program is more difficult to read.

#### Note

Operands that begin with the # special character are local operands (parameters or local variables) that must be declared in the declaration part of the block. Local operands are only valid and usable in the block in which they were declared.

The Program Editor automatically inserts the # character.

### 12.3. Parameter-assignable Blocks: Example: Fault Evaluation



#### Task

Faults that occur are to be displayed by an indicator light on the operator console. When there is a signal change from 0 → 1 at "Fault\_Input" the "Display" shows a 2Hz flashing light.

After the fault is "Acknowledged" but still exists at "Fault\_Input", the "Display" switches to a continuous light. When the fault no longer exists at "Fault\_Input" the display (light) goes dark.

#### Solution Suggestion

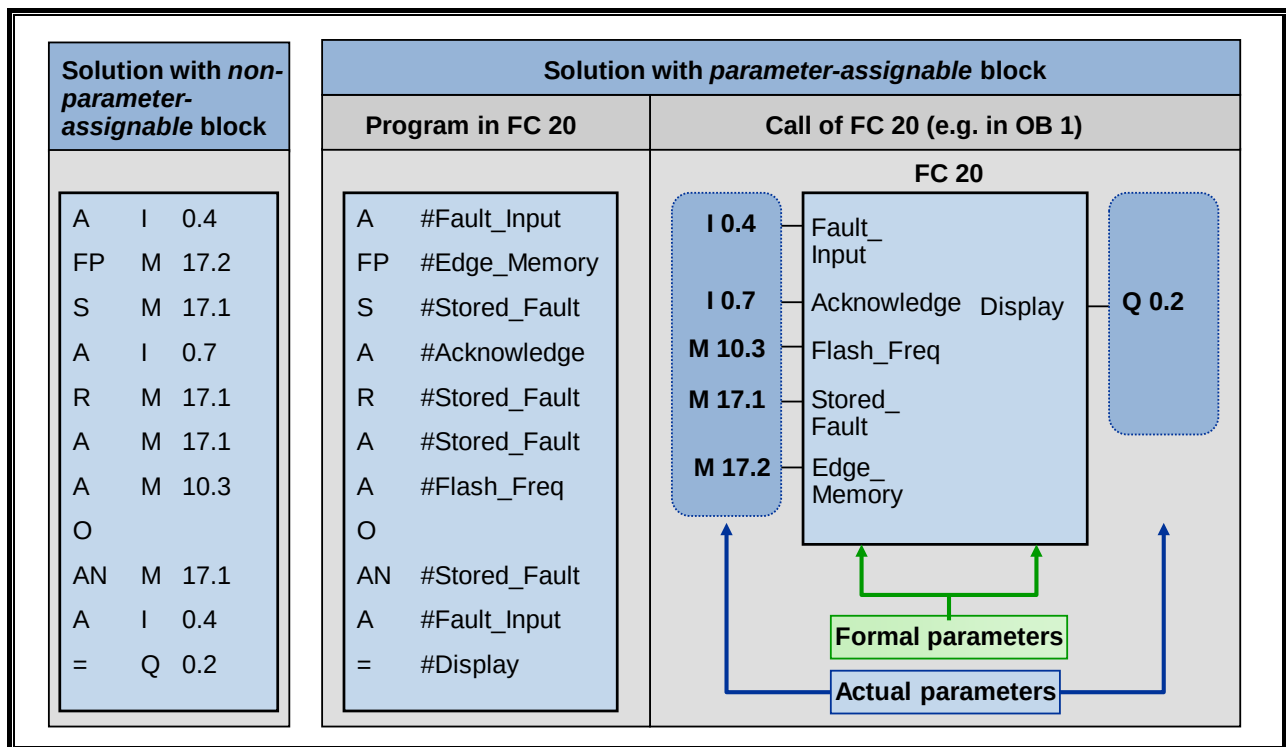
An edge evaluation of "Fault\_Input" is required since the "Stored\_Fault" would otherwise immediately be set again after an acknowledgement and a still existing "Fault\_Input" thus making the "Display" flash once more.

When "Acknowledge" has not yet occurred ("Stored\_Fault" is still set), the upper path (AND logic operation) with the "Flash\_Freq" causes the "Display" to flash.

When "Acknowledge" has already occurred ("Stored\_Fault" is no longer set), but "Fault\_Input" still exists, the lower path (AND logic operation) causes the "Display" to have a continuous light.



### 12.3.1. Solution with Parameter-assignable Block



#### Application

You can program parameter-assignable blocks for frequently recurring program functions. This has the following advantages:

- The program only has to be created once, which significantly reduces programming time.
- The block is only stored in the user memory once, which significantly reduces the amount of memory used.
- The block or the functionality implemented with the block can be called as often as you like, each time with different operands. For this, the formal parameters (input, output, or in/out parameters) are supplied with different actual parameters every time they are called.

#### Program Execution

When the block is executed, the parameter for the statement "A #Fault\_Input" is replaced with the actual parameter passed during the call.

If, during the call of the block, the input I 0.4 was passed as the actual parameter for the parameter #Fault\_Input, the statement "A #Fault\_Input" becomes the statement "A I 0.4" during runtime.

#### Parameter-assignability

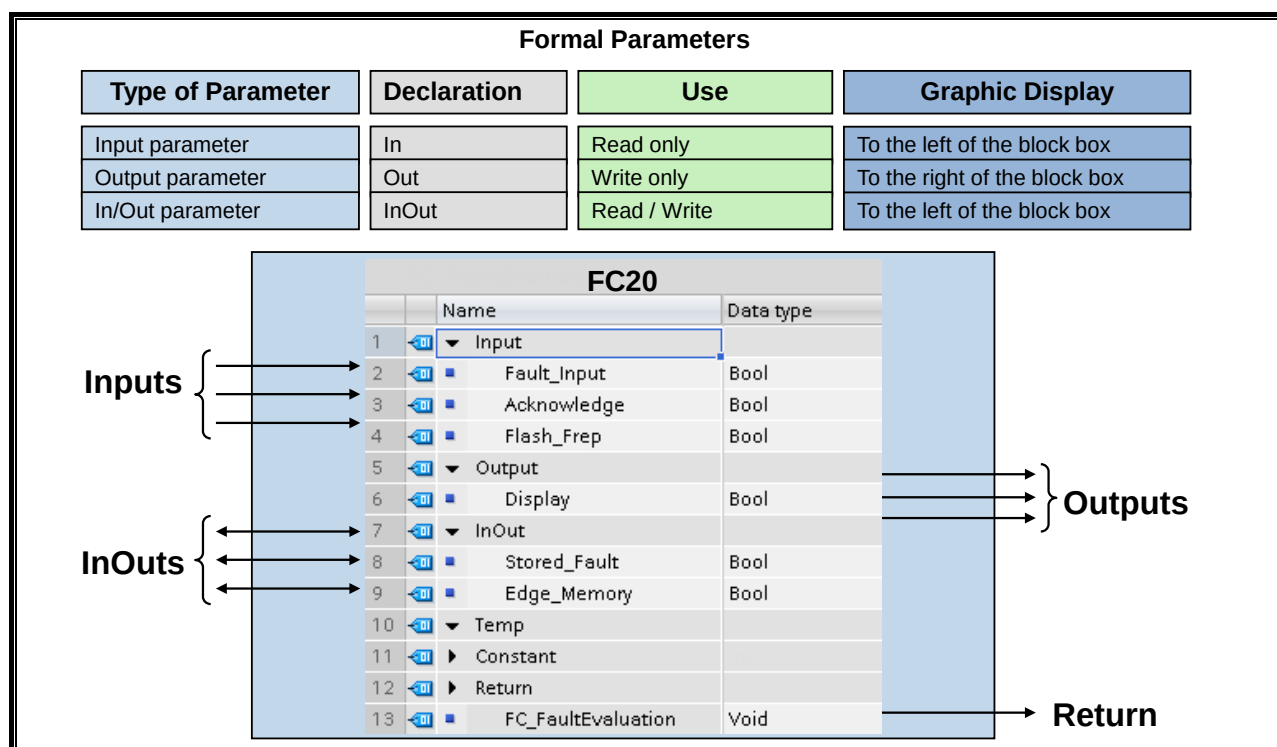
You can program FC or FB blocks as parameter-assignable. You cannot program organization blocks as parameter-assignable since they are called directly by the operating system. As no block call takes place in the user program, it is not possible to pass actual operands to an OB.

#### Our Example

Even if the fault evaluation or fault display is required repeatedly in the system, you only have to program FC 20 once as parameter-assignable.

The FC 20 is then called twice for the two different fault evaluations and is assigned a different actual address each time.

### 12.3.2. Declaration of Formal Parameters



#### Formal Operands

Before you can create the program in the parameter-assignable block, you have to define the formal parameters in the declaration part.

#### Type of Parameter

In the table in the picture, you can see the three possible types of parameters and their use. Please make sure that formal operands that have a reading and a writing access are declared as 'in/out' parameters.

#### Interface

The interface of a block forms the IN, OUT, and IN\_OUT parameters. The RETURN parameter is a defined, additional OUT parameter that has a specific name according to IEC 61131-3. This parameter only exists for FCs in the interface.

The TEMP variables and constants are - even though they are listed under "Interface" - not components of the block interface, since they do not become visible when the block is called.

#### Example FC20

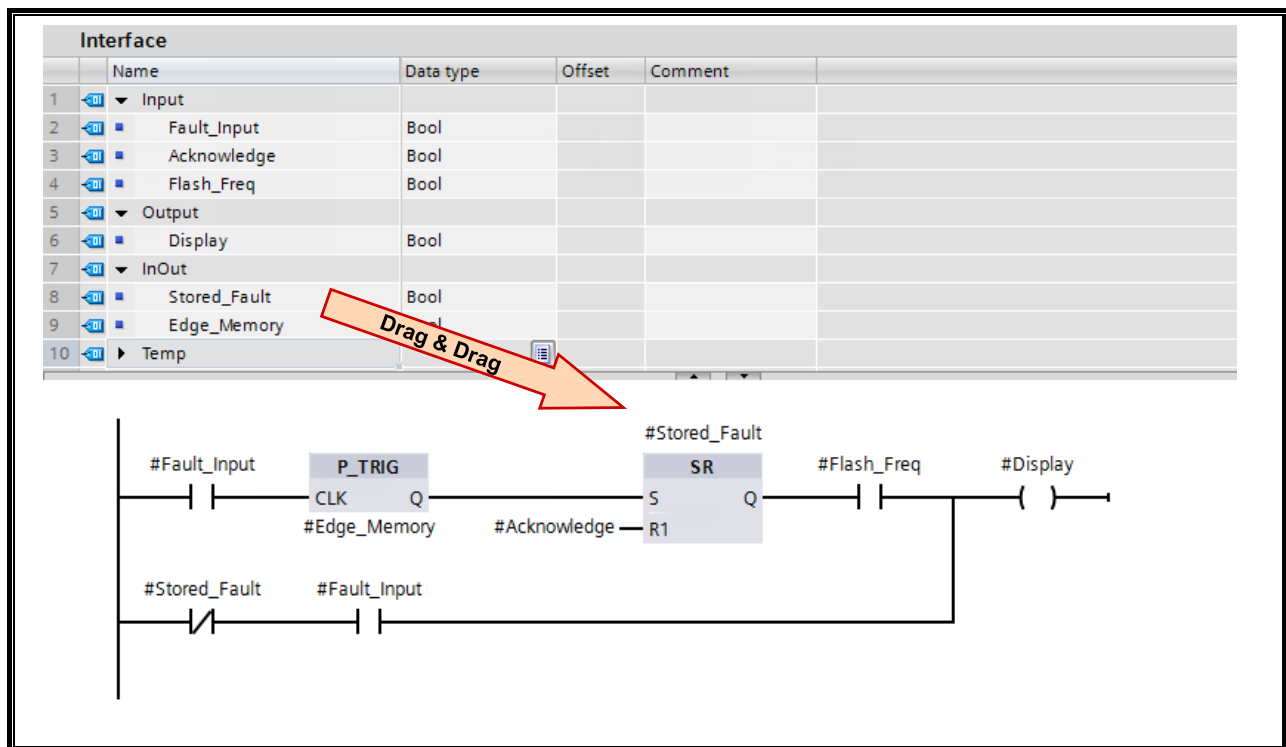
In the lower section of the picture, you can see the declaration part or the interface of the FC 20 block "FC\_FaultEvaluation" (see previous page). Since the formal parameters #Stored\_Fault and #Edge\_Memory are accessed both reading and writing in conjunction with the operation "P\_TRIG", you have to declare these as in/out parameters.



#### Caution!

The declared formal parameters (IN, OUT and IN\_OUT, not TEMP) of a block are its interface to the "outside". That is, they are "visible" or relevant to other blocks that call this block. If the interface of a block is changed by deleting or adding formal parameters later on, then the calls of the modified block have to be updated or corrected in all calling blocks.

### 12.3.3. Editing a Parameter-assignable Block



#### Notes

It doesn't matter whether the names of the formal parameters are written with capital or small letters. The "#" character in front of the name is automatically inserted by the PG. The character is used to indicate that the parameter is a local operand that was defined in the variable (tag) declaration table of this block.

It is possible, that when you write the program in LAD / FBD, that the name is not completely displayed in one line. This depends on how you have customized the settings in the Program Editor:

*Options → Settings → PLC programming → LAD / FBD → Operand field*

#### Symbols

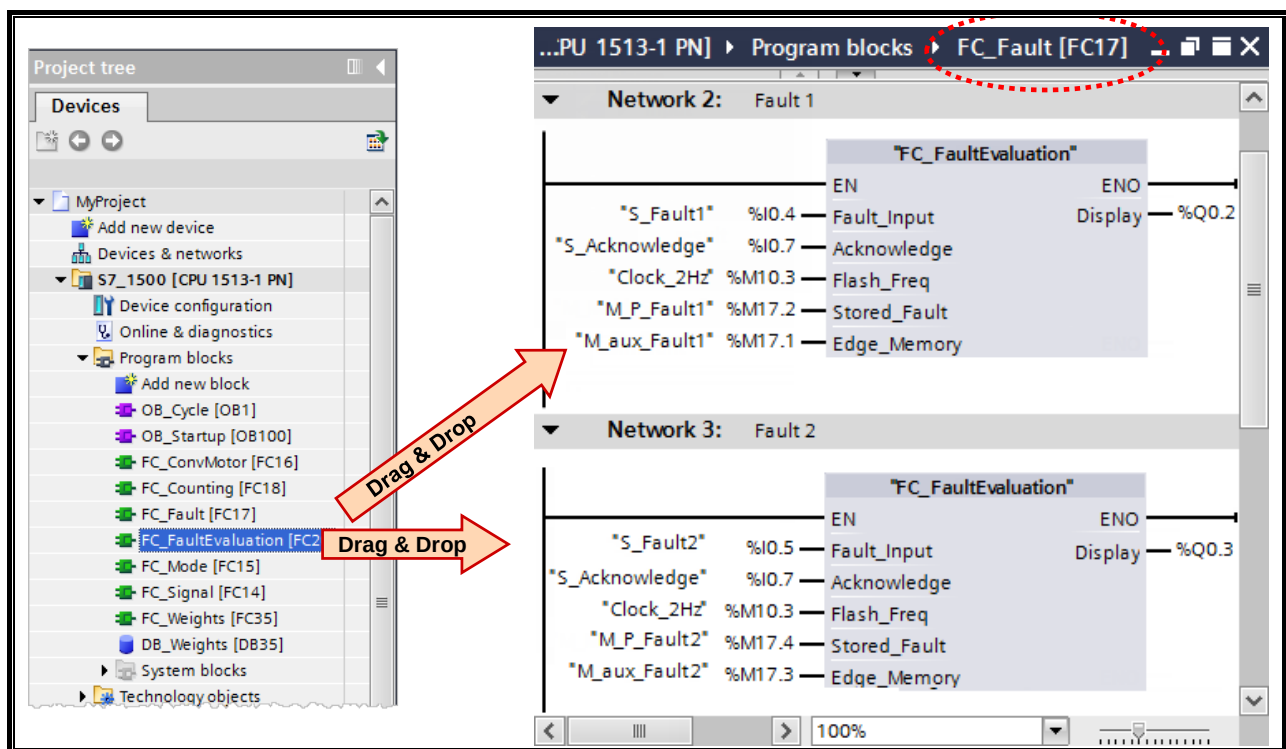
1. If you use a symbolic name when you edit a block, the Editor first of all searches through the interface of the block. If the symbolic name is there, the symbol with # in front of it is accepted in the program as a local operand.
2. If a symbol cannot be found as a local operand, the Editor searches through the PLC tags for the global symbol. If the symbol is found there, the symbol is placed in quotation marks and is accepted in the program as a global operand.
3. If you specified the same symbolic name globally (in the PLC tags) as well as locally (in the variable (tag) declaration table) the Editor will always insert the local operand.

If, however, you want to work with the global symbol, you must place the symbol name in quotation marks when you make the entry.

#### Drag & Drop

Just as with global operands (for example, from the PLC tags) local operands can be dragged into the program part of the Editor from the block interface using drag & drop and placed in the desired position there.

### 12.3.4. Calling a Parameter-assignable Block



#### Block Call

The block can be called by dragging it from the "Program blocks" container & dropping (inserting) it in the statement (code) part of the calling block.

#### Note

When a parameter-assignable function (FC) is called, an actual parameter must be passed for every formal parameter.

#### Exception:

In the graphic programming languages LAD and FBD, the assignment of the EN and ENO parameters, which are automatically added by the Editor, is optional.

#### Parameter Assignment

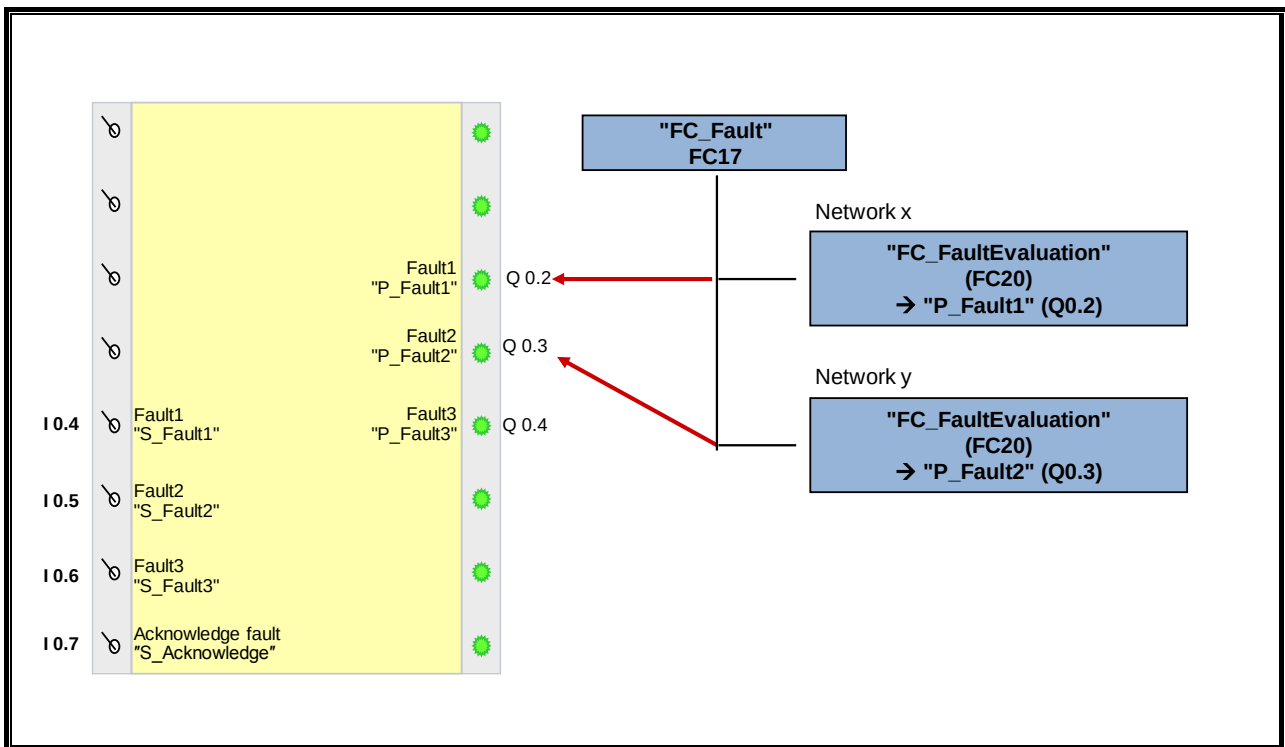
All global and local operands whose data type corresponds to the formal parameters of the called block can be passed as actual parameters.

The actual parameters can be passed with an absolute address or with a symbolic name - as declared in the PLC tags or in the declaration part of the calling block.

#### Passing On of Parameters

Basically, a "passing on of parameters" is also possible. That is, formal parameters of the calling block are passed on as actual parameters to the called block. For parameters of complex data types this is however only possible with limitations. It is dealt with in greater detail in another more advanced course.

## 12.4. Task Description: Fault Evaluation by means of a Function (FC)



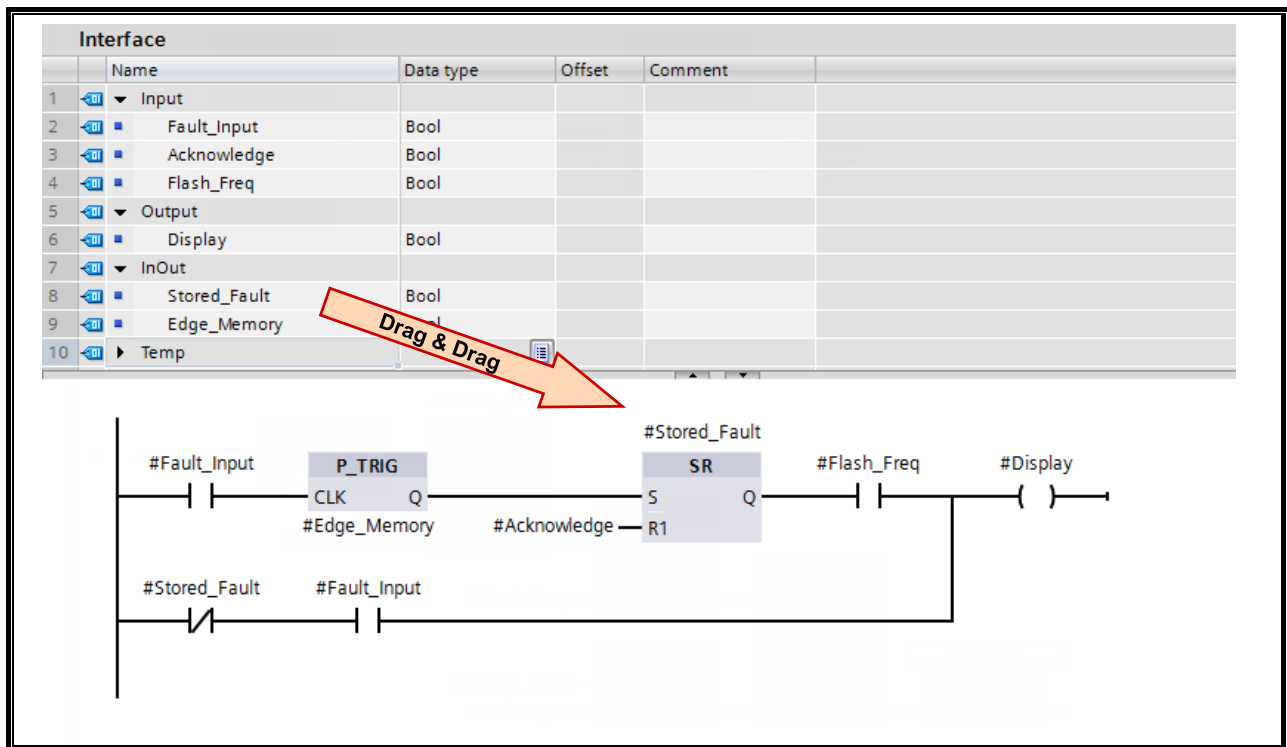
### Task Description

If a fault is triggered at the inputs I 0.4 or I 0.5, the associated LED Q 0.2 or Q 0.3 begins to flash.

The input I 0.7 ("S\_Acknowledge") is a group acknowledgement for all faults. If the fault still exists after acknowledgement, the LED changes to continuous light; if the fault no longer exists, the LED goes dark.

The required function is to be programmed in the parameter-assignable "FC\_FaultEvaluation" (FC20) which is to be called twice in "FC\_Fault" (FC17) for the evaluation of the two faults.

### 12.4.1. Exercise 1: Programming the Parameter-assignable Function "FC\_FaultEvaluation" (FC20)



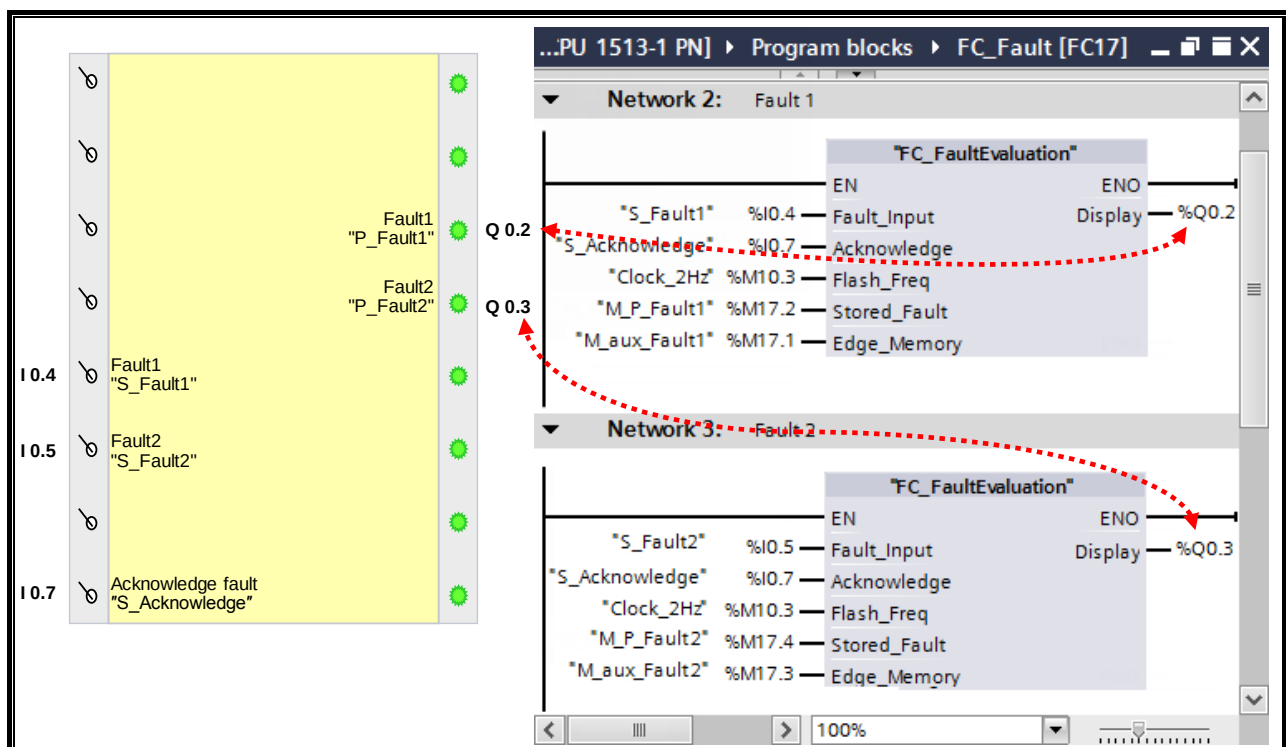
#### Task

Create the program for the fault evaluation in the parameter-assignable block "FC\_FaultEvaluation" (FC 20).

#### What to Do

1. Insert the "FC\_FaultEvaluation" (FC 20) block in the "Program blocks" folder.
2. Declare the formal parameters as shown in the picture.
3. Create the program as shown in the picture.
4. Save the block.

### 12.4.2. Exercise 2: Calling and Parameterizing "FC\_FaultEvaluation" (FC20)



#### Function Up Until Now

The (timely) monitoring of the transport sequences when P\_Operation is switched on is programmed in the block "FC\_Fault" (FC 17).

#### Task

In addition to the monitoring of the transport sequences, two faults from the process (signals of the two simulator switches) are to be evaluated. For this, the previously programmed "FC\_FaultEvaluation" (FC 20) block must be called twice.

#### What to Do

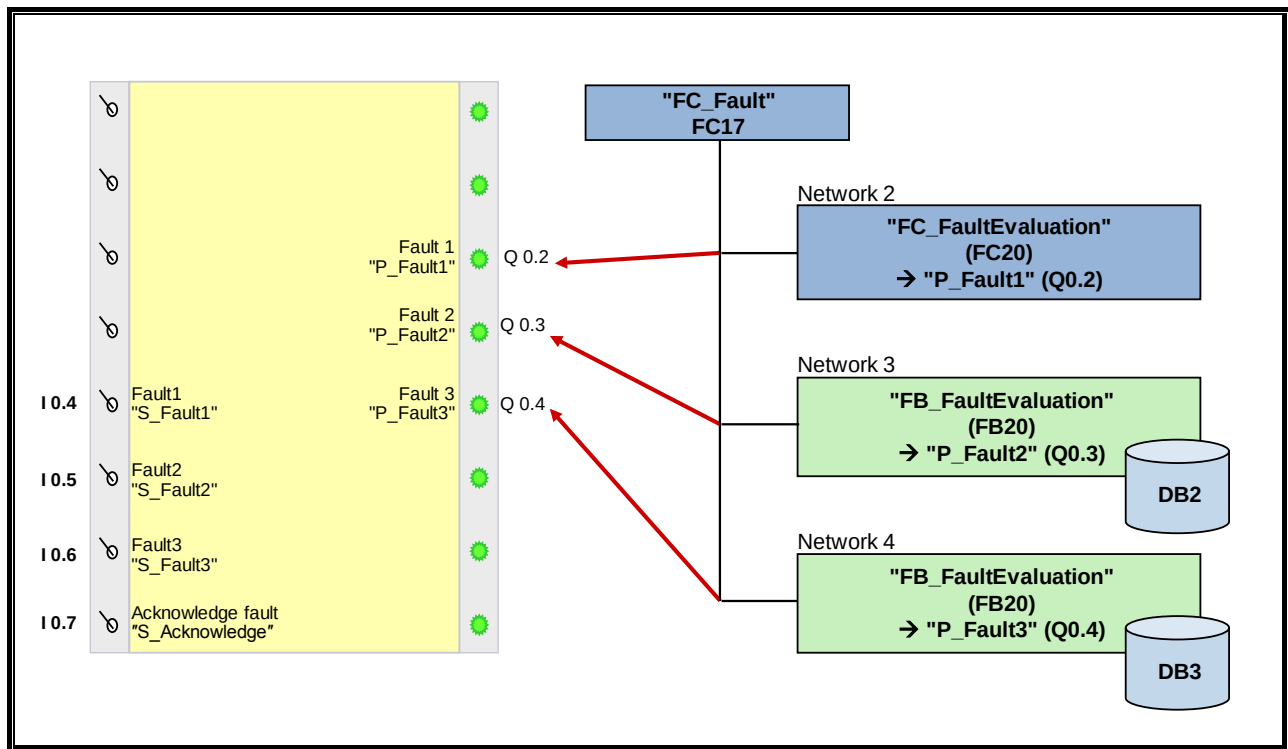
1. In "FC\_Fault" (FC 17), program the two calls of the new "FC\_FaultEvaluation" (FC 20) block as shown in the picture.
2. Save the modified "FC\_Fault" (FC 17) and download it into the CPU.
3. Check your program to see if it fulfills the described functions for fault evaluation.

#### Note

The MB 10 memory byte was already parameterized as a clock memory byte in the device configuration.

The "M\_2Hz" (M10.3) memory bit has a flashing frequency of 2Hz and was already created as a PLC tag.

## 12.5. Task Description: Fault Evaluation by means of a Function Block (FB)



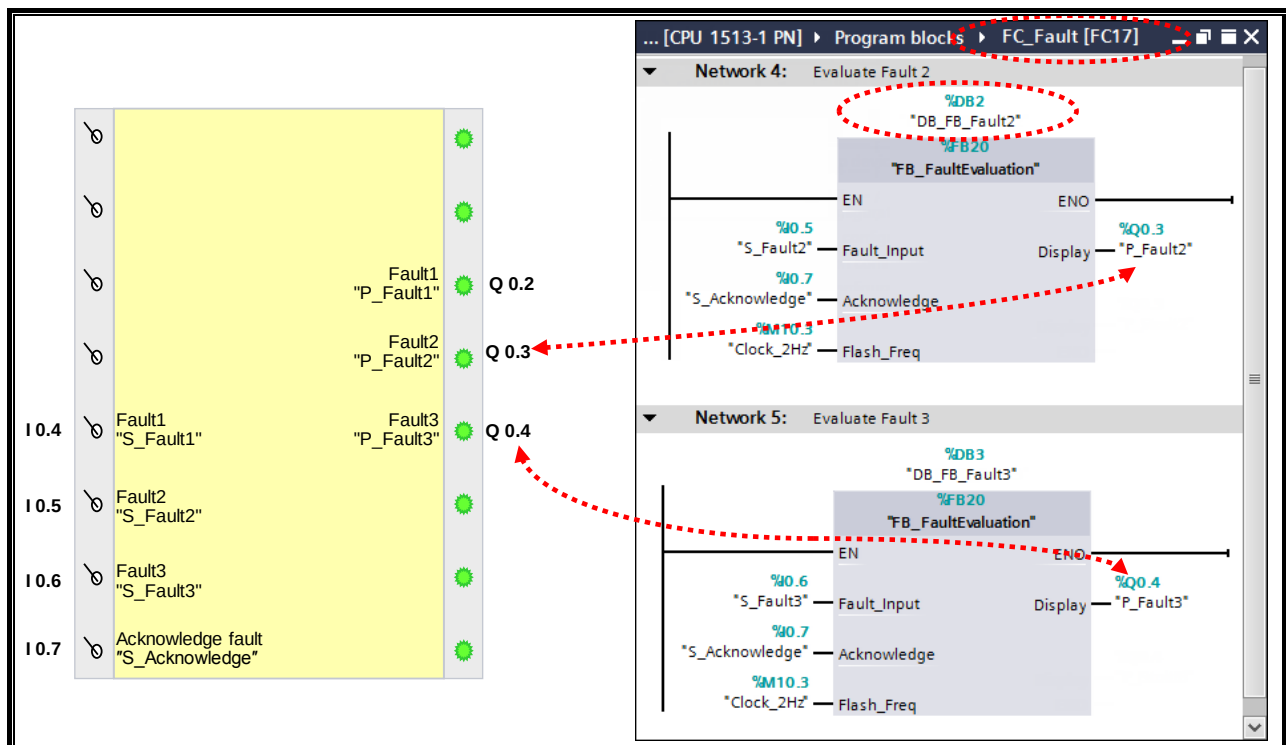
### Task Description

The previously described fault evaluation is now to be implemented with an FB instead of an FC. This offers the advantage that for the internally required edge evaluation of the fault and as stored fault, the FB doesn't have to be passed any global operands from outside since local, static variables can be used.

Fault 1 is to continue to be evaluated by the already existing "FC\_FaultEvaluation" (FC20). The evaluation of Fault 2 and 3 is to be carried out by the new "FB\_FaultEvaluation" (FB20).



### 12.5.1. Instantiating FBs



#### Special Features

Unlike functions (FCs), function blocks (FBs) have a (recall) memory. That means that a local data block is assigned to the function block. This data block is known as an instance data block. When you call an FB, you also have to specify the number of the instance DB, by which it is automatically opened.

An instance DB is used to save static variables, among other things. These local operands can only be used in the FB, in whose declaration table they were declared. When the block is exited, they are retained.

#### Parameters

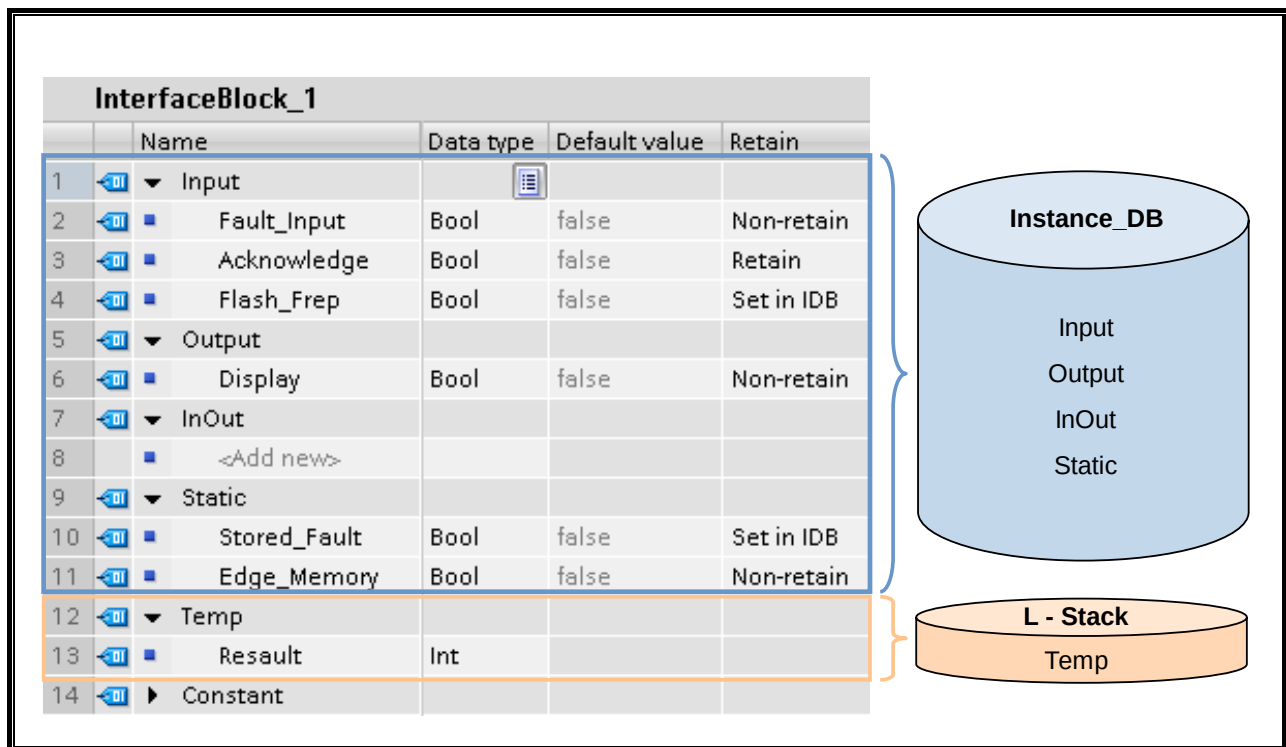
When the function block is called, the values of the actual parameters are stored in the instance data block. If no actual parameters were assigned to a formal parameter in a block call, then the last value stored in the instance DB for this parameter is used in the program execution.

You can specify different actual parameters with every FB call. When the function block is exited, the data in the data block is retained.

#### FB Advantages

- When you write a program for an FC, you must search for empty bit memory address areas or data areas and you must maintain them yourself. The static variables of an FB, on the other hand, are maintained by the STEP 7 software.
- When you use static variables you avoid the risk of assigning bit memory address areas or data areas twice (double assignment).
- Instead of the formal parameters "Stored\_Fault" and "Edge\_Memory" of the FC20, you use the static variables "Stored\_Fault" and "Edge\_Memory" in the FB. This makes the block call simpler since the two formal parameters are dropped.

### 12.5.2. FB - Declaration Part



#### Parameters

When the function block is called, the values of the actual parameters are stored in the instance data block. If no actual parameters were assigned to a formal parameter in a block call, then the last value stored in the instance DB for this parameter is used in the program execution.

You can specify different actual parameters with every FB call. When the function block is exited, the data in the data block is retained.

#### Static Variables

Unlike functions, function blocks have "static variables" (Static). These variables form the memory of the FB since they are not stored in the L-Stack but in their own memory area, the instance DB.

### 12.5.3. Generating Instance Data Blocks

**Automatically generating an instance DB during the FB call**

**Manually creating an instance DB**

#### Generating

There are two ways of generating an instance data block:

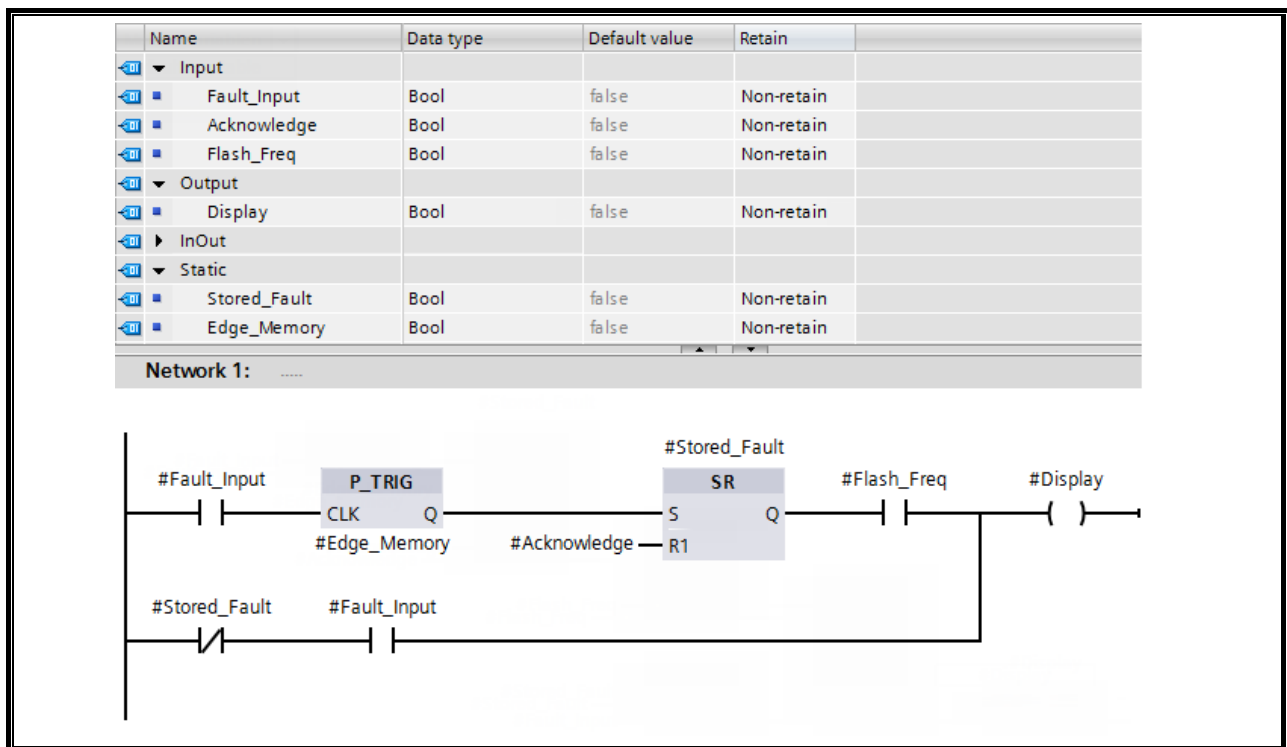
- When you call an FB, you specify with which instance DB the FB is to work. A dialog opens in which the symbolic name and, if desired, a manual number of the instance DB can be preset.
- When you create a new DB, you select the Type "Function block XY".



#### Caution!

If you modify the FB (by adding additional parameters or static variables), you must then also generate the instance DB again.

### 12.5.4. Exercise 3: Editing "FB\_FaultEvaluation" (FB20)



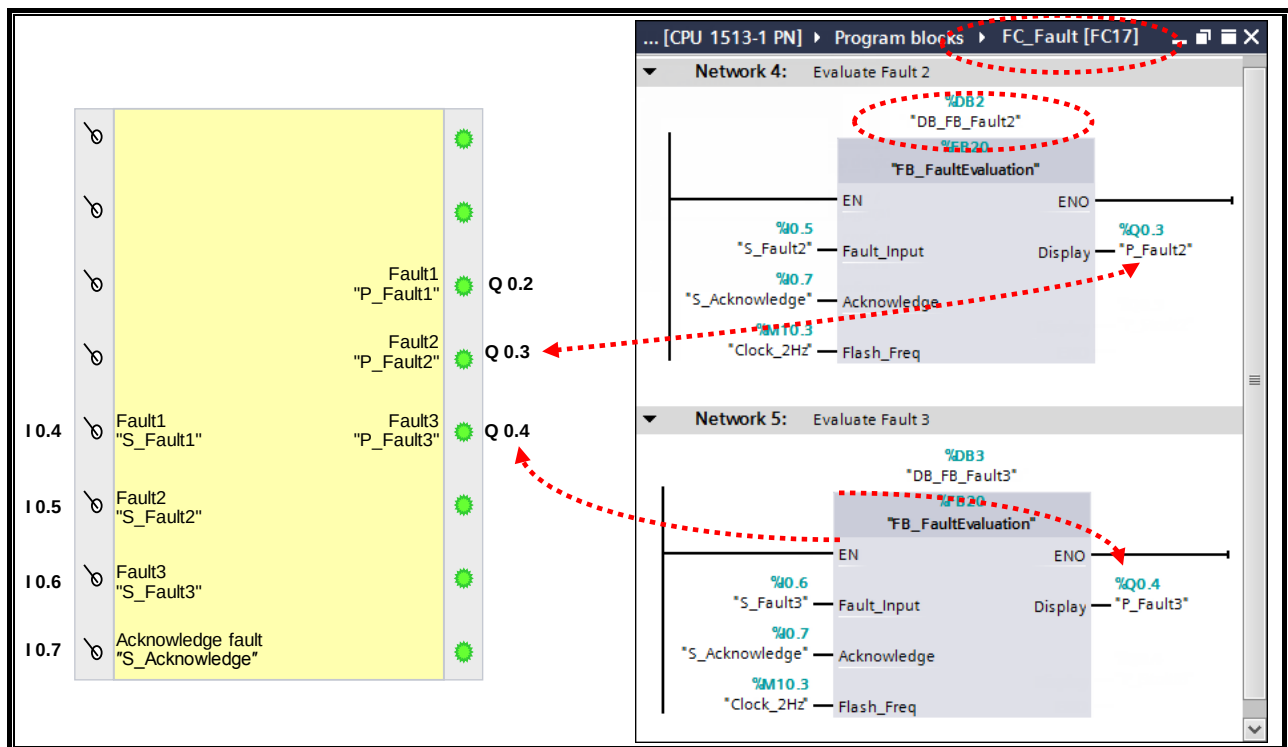
#### Task

Create the new "FB\_FaultEvaluation" block for the subsequent evaluation of Fault 2 and 3.

#### What to Do

1. Insert the new "FB\_FaultEvaluation" (FB 20) block.
2. Declare the formal parameters and the static variables of the block as shown in the picture.
3. Program "FB\_FaultEvaluation" by copying the necessary program parts from the already programmed "FC\_FaultEvaluation".
4. Save the block and download it into the CPU.

### 12.5.5. Exercise 4: Calling "FB\_FaultEvaluation" (FB20)



#### Task

The evaluation of the old Fault 2 (programmed up until now through the call of "FC\_FaultEvaluation" FC20) and the evaluation of the new Fault 3 is to be implemented with the newly created "FB\_FaultEvaluation" (FB 20).

For this, the parameter-assigned block "FB\_FaultEvaluation" must be called twice in "FC\_Fault" (FC 17), each time with a different instance data block.

#### What to Do

1. In FC 17 delete the second call of FC 20, since the evaluation of Fault 2 is to be subsequently implemented with FB 20.
2. Program both calls of FB 20 - as shown in the picture - in two new networks in FC 17. Let the Editor generate the instance DBs 2 and 3.
3. Save the modified "FC\_Fault" (FC 17).
4. Download the entire program into the CPU and check the program function.
5. Save your project.

## 12.6. Adding Block Parameters Later On

| Interface |             |           |               |            |
|-----------|-------------|-----------|---------------|------------|
|           | Name        | Data type | Default value | Retain     |
| 1         | ▼ Input     |           |               |            |
| 2         | Fault_Input | Bool      | false         | Non-retain |
| 3         | Acknowledge | Bool      | false         | Non-retain |
| 4         | Flash_Freq  | Bool      | false         | Non-retain |
| 5         | light_check | Bool      | false         | Non-ret... |

| Path                 |  | Description                                             |
|----------------------|--|---------------------------------------------------------|
| ▼ S7_1500            |  |                                                         |
| ▼ Program blocks     |  |                                                         |
| DB_FB20_Fault3 (DB3) |  | Block was successfully compiled.                        |
| DB_FB20_Fault2 (DB2) |  | Block was successfully compiled.                        |
| ▼ OB_Cycle (OB1)     |  |                                                         |
| Network 4            |  | Number of updated calls in network Evaluate Fault 2: 1. |
| Network 5            |  | Number of updated calls in network Evaluate Fault 3: 1. |
|                      |  | Block was successfully compiled.                        |
|                      |  | Compiling completed (errors: 0; warnings: 0)            |

Automatic update of the block call during compiling

### Problem

If you have to adjust or supplement the interfaces or the code of individual blocks during or after program creation, it can lead to time stamp conflicts. Time stamp conflicts can, in turn, lead to block inconsistencies between calling and called blocks or reference blocks and thus to a great degree of correction.

If block parameters are **added** later on to a block already called in the program, you also have to update the calls of the block in other blocks.

- Manual Update

In the open, calling block, the inconsistent calls of a block are highlighted in red. By right-clicking the inconsistent call, the function "Update" can be selected in the follow-up dialog box. A window then appears in which the old (faulty) and the new block call are displayed. With function blocks, the instance DB is subsequently regenerated.

- Automatic Update

Time stamp conflicts are also detected when the entire user program is compiled and in case of added parameters, affected block calls are automatically updated.

With functions, the added formal parameter still has to be assigned before downloading into the CPU, since this is a "Must Assign".

With function blocks, the default value out of the associated instance DB is used when the formal parameter is not assigned ("Can Assign").

## 12.7. Deleting Block Parameters Later On

The screenshot illustrates the process of deleting block parameters and the subsequent compilation results. At the top left, the 'FB\_FaultEvaluation' block is shown with its interface parameters: EN, ENO, Fault\_Input, Acknowledge, Flash\_Freq, and light\_check. The 'light\_check' parameter is circled in red. To the right, the 'Interface' table lists these parameters with their data types. The 'light\_check' row is crossed out with a red X.

|   | Name        | Data type |
|---|-------------|-----------|
| 1 | Input       |           |
| 2 | Fault_Input | Bool      |
| 3 | Acknowledge | Bool      |
| 4 | Flash_Freq  | Bool      |
| 5 | light_check | Bool      |

Below the interface table, a 'Program blocks' window shows the compilation status. The 'OB\_Cycle (OB1)' network is expanded, showing 'Network 4' and 'Network 5'. The 'light\_check' parameter is highlighted in red in the network call. A red box highlights the 'Automatic update of the block call during compiling' message.

Compilation status summary:

- FB\_FaultEvaluation (FB2... Block was successfully compiled.
- DB\_FB20\_Fault3 (DB3) Block was successfully compiled.
- DB\_FB20\_Fault2 (DB2) Block was successfully compiled.
- OB\_Cycle (OB1)
  - Network 4 Number of updated calls in network Evaluate Fault 2: 1.
  - Network 5 Number of updated calls in network Evaluate Fault 3: 1.
- Block was successfully compiled.
- Compiling completed (errors: 0; warnings: 0)

### Problem

If you have to adjust or supplement the interfaces or the code of individual blocks during or after program creation, it can lead to time stamp conflicts. Time stamp conflicts can, in turn, lead to block inconsistencies between calling and called blocks or reference blocks and thus to a great degree of correction.

If block parameters are **deleted** later on from a block already called in the program, you also have to update the calls of the block in the calling blocks.

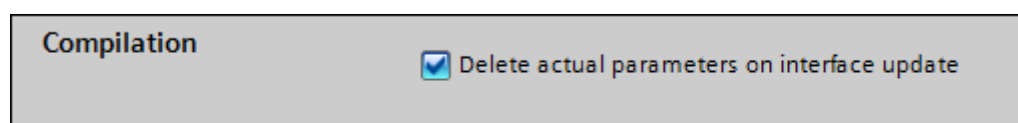
- Manual Update

In the open, calling block, the inconsistent calls of a block are highlighted in red. By right-clicking the inconsistent call, the function "Update" can be selected in the follow-up dialog box. A window then appears in which the old (faulty) and the new block call are displayed. With function blocks, the instance DB is subsequently regenerated.

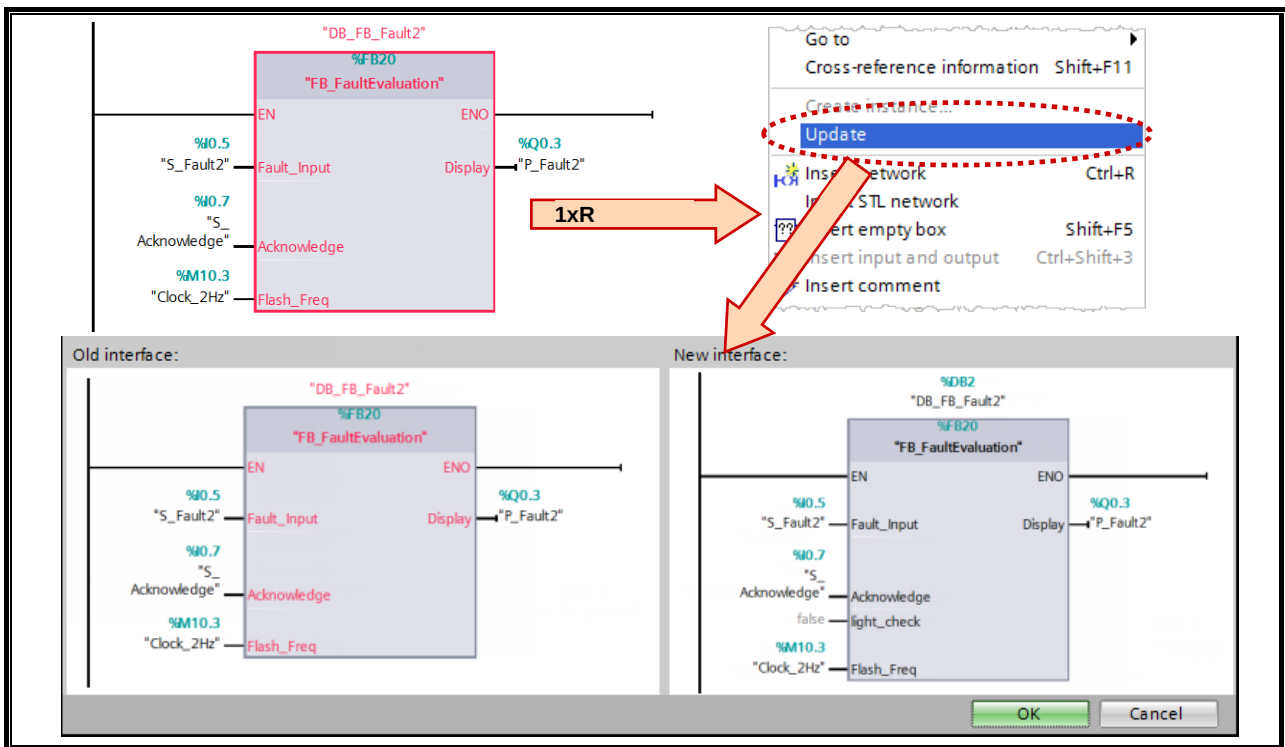
- Automatic Update

Time stamp conflicts are also detected when the entire user program is compiled and in case of deleted parameters, affected block calls are automatically updated.

**Attention/Caution:** If the deleted formal parameters have already been assigned with actual parameters, then this automatic update only occurs if the function "Delete actual parameters on interface update" is activated under Options > Settings > PLC-programming > General > Compilation.



## 12.8. Updating a Block Call



### Manual Update

In the open, calling block, the inconsistent calls of a block are highlighted in red. By right-clicking the inconsistent call, the function "Update" can be selected in the follow-up dialog box. A window then appears in which the old (faulty) and the new block call (in the picture with the additional parameter "light\_check") are displayed. With function blocks, the instance DB is subsequently regenerated.



Newly generated instance data blocks are not automatically downloaded into the CPU. You must do this explicitly.



# Contents

|                                                                                      |             |
|--------------------------------------------------------------------------------------|-------------|
| <b>13. Organization Blocks.....</b>                                                  | <b>13-2</b> |
| 13.1. Types of Program Blocks.....                                                   | 13-3        |
| 13.2. Organization Blocks of the S7-1500 .....                                       | 13-4        |
| 13.3. Creating a New OB .....                                                        | 13-5        |
| 13.4. OB Start Information using OB100 as an Example .....                           | 13-6        |
| 13.5. S7-1500 Startup .....                                                          | 13-7        |
| 13.6. Interrupting the Cyclic Program .....                                          | 13-8        |
| 13.6.1. Time-of-day Interrupt (OB 10).....                                           | 13-9        |
| 13.6.2. Cyclic Interrupt (OB35) .....                                                | 13-10       |
| 13.6.3. Phase Offset for Cyclic Interrupts .....                                     | 13-11       |
| 13.6.4. Hardware Interrupt (OB 40) .....                                             | 13-12       |
| 13.6.5. Exercise 1: Displaying the Type of Startup and Acknowledging It (OB100)..... | 13-13       |
| 13.7. OBs for Handling Asynchronous Errors.....                                      | 13-14       |
| 13.7.1. OBs for Handling Synchronous Errors.....                                     | 13-15       |

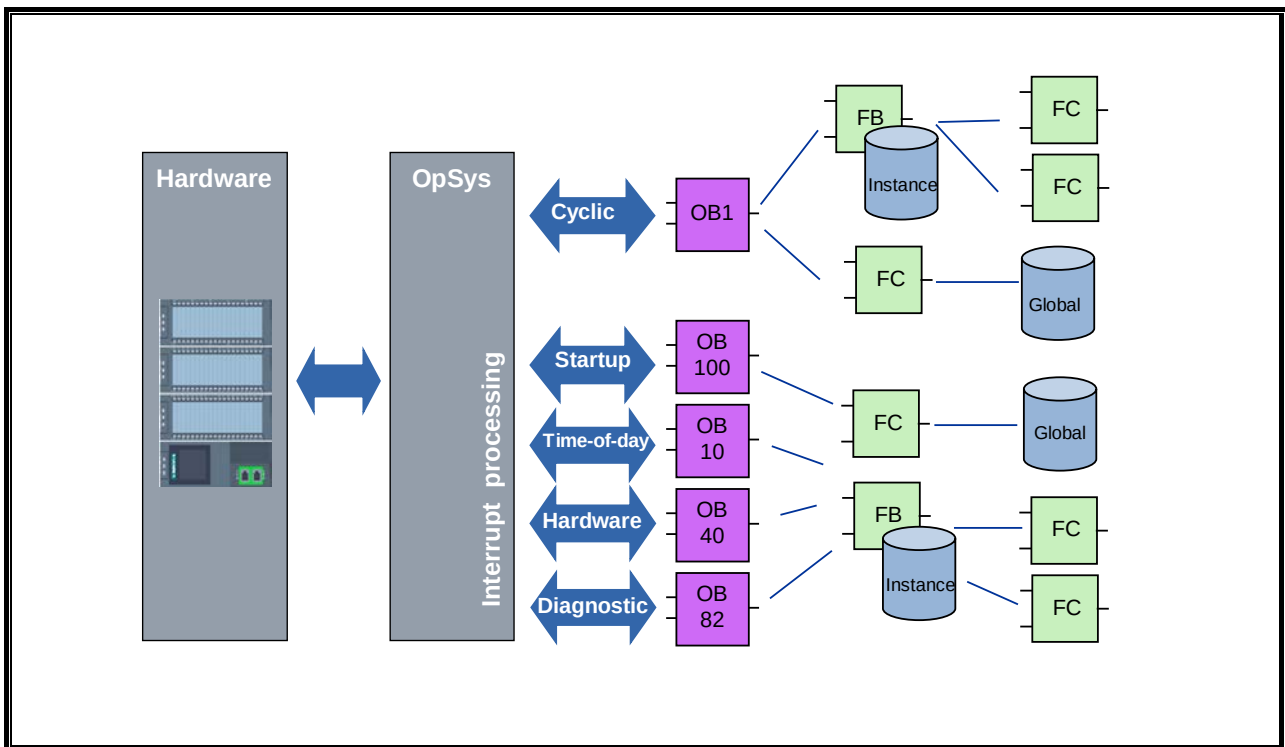
## 13. Organization Blocks

**At the end of the chapter the participant will ...**

- ... be familiar with the CPU startup
- ... understand the principle of interrupt processing
- ... be familiar with the "Time-of-day Interrupt", "Cyclic Interrupt", "Hardware Interrupt" and "Diagnostic Interrupt"
- ... know and be able to use the error OBs
- ... be able to interpret the start information of OBs



## 13.1. Types of Program Blocks



### Blocks

The automation system (PLC) provides various types of blocks in which the user program and the related data can be stored. Depending on the requirements of the process, the program can be structured in different blocks. You can use the entire operation set in all blocks (FB, FC and OB).

### Organization blocks (OBs)

Organization blocks (OBs) form the interface between the operating system and the user program. The entire program can be stored in OB1 that is cyclically called by the operating system (linear program) or the program can be divided and stored in several blocks (structured program).

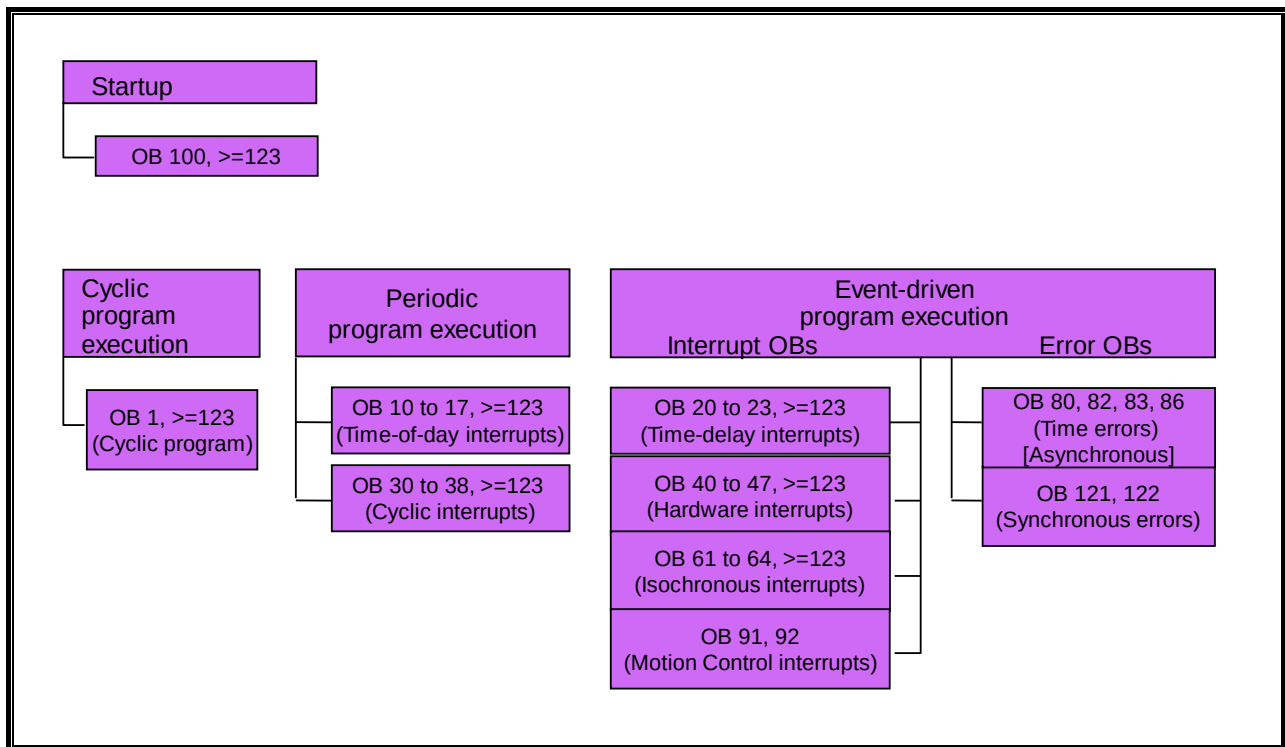
### Functions (FCs)

A function (FC) contains a partial functionality of the program. It is possible to program functions as parameter-assignable so that when the function is called it can be assigned parameters. As a result, functions are also suited for programming frequently recurring, complex partial functionalities such as calculations.

### Function Blocks (FBs)

Basically, function blocks offer the same possibilities as functions. In addition, function blocks have their own memory area in the form of instance data blocks. As a result, function blocks are suited for programming frequently recurring, complex functionalities such as closed-loop control tasks.

## 13.2. Organization Blocks of the S7-1500



### Startup Program

After a power recovery, or a change of operating mode (through the CPU's mode selector or through PG operation), a startup program is carried out before the cyclic program execution. In the startup OBs you can, for example, carry out a pre-assignment of communication connections.

### Cyclic Program Execution

The program stored in OB1 is executed cyclically, that is, after it is executed completely it is executed again. With this cyclic program execution, the reaction time results from the execution time for the CPU's operating system and the sum of the command runtimes of all executed instructions. The reaction time, that is, how fast an output can be switched in relation to an input signal, amounts to a minimum of one time and a maximum of two times the cycle time.

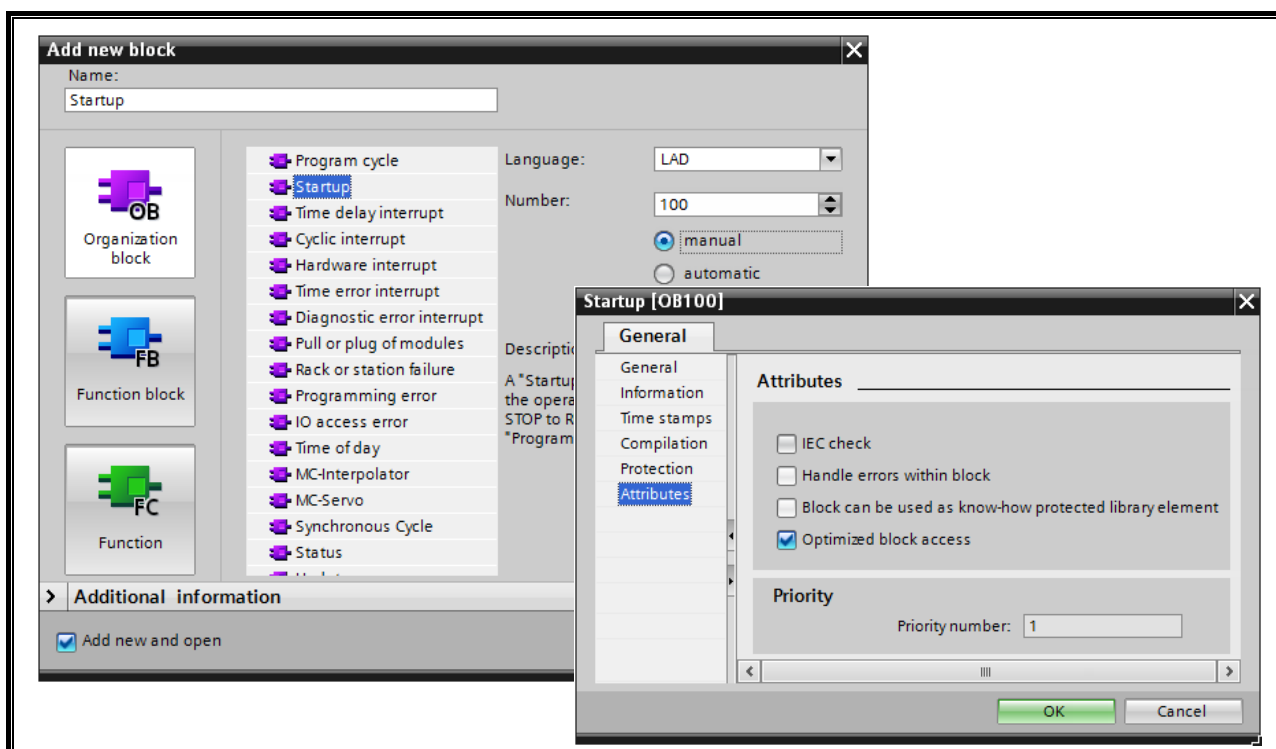
### Periodic Program Execution

This makes it possible to interrupt the cyclic program execution at fixed intervals. With the cyclic interrupts, an organization block (for example OB35) is executed after an adjustable time base (for example, every 100ms) has expired. In these blocks, closed-loop control blocks with their sampling time, for example, are called. With the time-of-day interrupts, an OB which carries out a data backup, for example, is executed at a specific time, for example, every day at 17:00 hours (5:00 p.m.).

### Event-driven Program Execution

In order to be able to react quickly to a process event, the hardware interrupt can be used. After an event occurs, the cycle is immediately interrupted and an interrupt program is executed. With time-delay interrupts, a freely definable event can be reacted to with a time-delay; with an error OB, the user can influence the behavior of the controller in case there is an error.

### 13.3. Creating a New OB



#### Creating a New OB

Basically, any number of OBs can be created from all OB types. Numbers < 123 are permanently assigned to specific OBs (see previous page), and numbers > 123 are freely selectable.

By default, new OBs are created with the attribute "Optimized block access" with the result that only reduced start information is available in the OBs (see next page).

## 13.4. OB Start Information using OB100 as an Example

The screenshot shows the SIMATIC Manager interface for OB100. On the left, a table lists the start information fields and their addresses:

| Address | Description                                             |
|---------|---------------------------------------------------------|
| 0 / 1   | Start event, Startup request                            |
| 2 / 3   | Priority, OB no.                                        |
| 4 / 5   | Reserved                                                |
| 6 / 7   | Number of the event that caused the CPU to go into STOP |
| 8 / 9   | Additional information on the current startup           |
| 10 / 11 |                                                         |
| 12 / 13 | Year, Month                                             |
| 14 / 15 | Day, Hours                                              |
| 16 / 17 | Minutes, Seconds                                        |
| 18 / 19 | 1/1000 seconds, Weekday                                 |

The main window shows the 'Interface' table for the 'Temp' block:

| Name       | Data type | Offset | Comment                                           |
|------------|-----------|--------|---------------------------------------------------|
| EV_CLASS   | Byte      | 0.0    | 16#13: Event class 2, Entering event state, Even. |
| STRTUP     | Byte      | 1.0    | Startup request                                   |
| PRIORITY   | Byte      | 2.0    | Priority of OB Execution                          |
| OB_NUMBR   | Byte      | 3.0    | OB number                                         |
| RESERVED_1 | Byte      | 4.0    | Reserved for system                               |
| RESERVED_2 | Byte      | 5.0    | Reserved for system                               |
| STOP       | Word      | 6.0    | Event that caused CPU to stop (16#4xxx)           |
|            |           | 8.0    | Information on                                    |
|            |           | 12.0   | Date and time                                     |

The 'Optimized block access' checkbox is checked, indicating that the start information is available for those OBs where this attribute is not activated.

### Start Information

When the operating system calls organization blocks, the user is provided with OB-specific start information on the local data stack.

This start information has a length of 20 bytes and is available after the OB starts execution.

The start information as well as their absolute L-stack addresses is only completely available for those OBs where the block attribute "Optimized block access" is not activated (as shown in the picture).

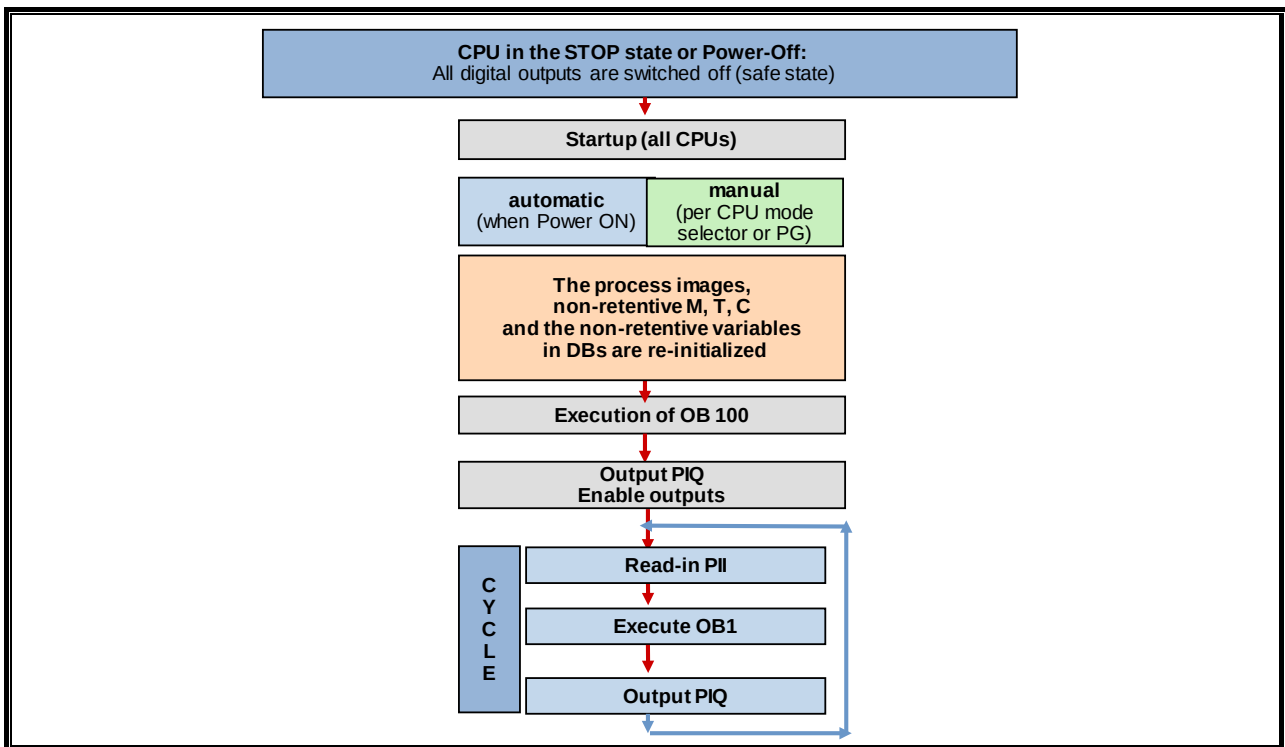
### Variables

An explanation of the meaning of the variables can be found in the online help.

### Note

In order to avoid errors, the structure of the standard declaration part must not be changed by the user. Following the standard declaration part, the user can declare further, additional temporary variables.

## 13.5. S7-1500 Startup



### General

Before the CPU begins with the execution of the cyclic user program, a startup program is executed.

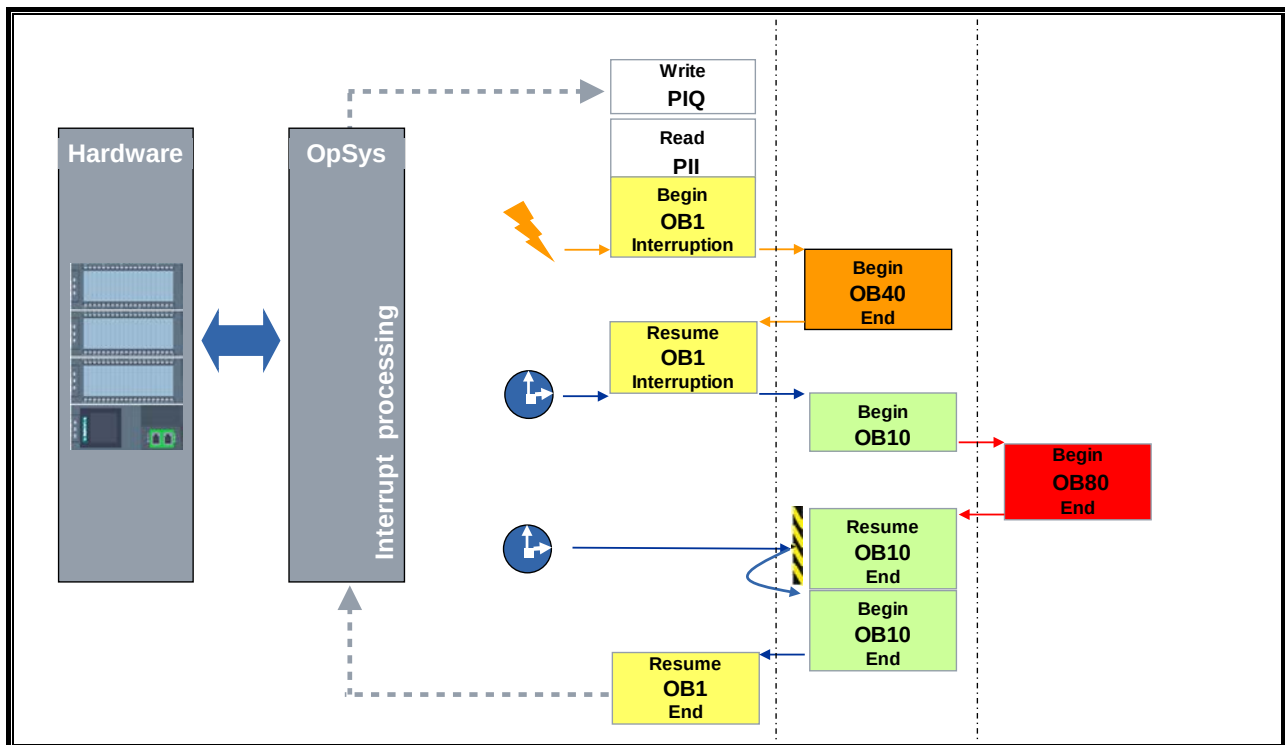
Through programming of startup OBs in the startup program, initialization variables can be set.

### Startup

With the startup program, the process images (PII, PIQ) and all non-retentive bit memories, timers and counters are deleted. Non-retentive DBs are reset to the start values of the load memory.

Retentive bit memories, timers and counters as well as retentive DB contents are retained.

## 13.6. Interrupting the Cyclic Program



### OB Calls

Organization blocks (OBs) form the interface between the CPU's operating system and the user program.

Organization blocks are called exclusively by the operating system. There are various start events (time-of-day interrupts, hardware interrupts - see picture) that each lead to the start of their associated organization block.

### Interrupting the Cyclic Program

When the operating system calls another OB, it interrupts the cyclic program execution because OB1 has the lowest priority. Any other OB can therefore interrupt the main program and execute its own program. Afterwards, OB1 resumes execution at the point of interruption.

### Priorities

The S7-1500 CPUs support the priorities 1 (lowest priority) to 26 (highest priority).

The OBs are executed on a purely priority-driven basis. This means that when several OB requests exist at the same time, the OB with the highest priority is executed first. When an event occurs that has a higher priority than the currently active OB, this OB is interrupted. Events of the same priority are executed in the order that they occur.

### Queue

If, during the execution of an interrupt OB, a further event with the same priority occurs, this event is placed in a queue according to its priority. The start events of a queue are processed at a later time in the order that they occur.

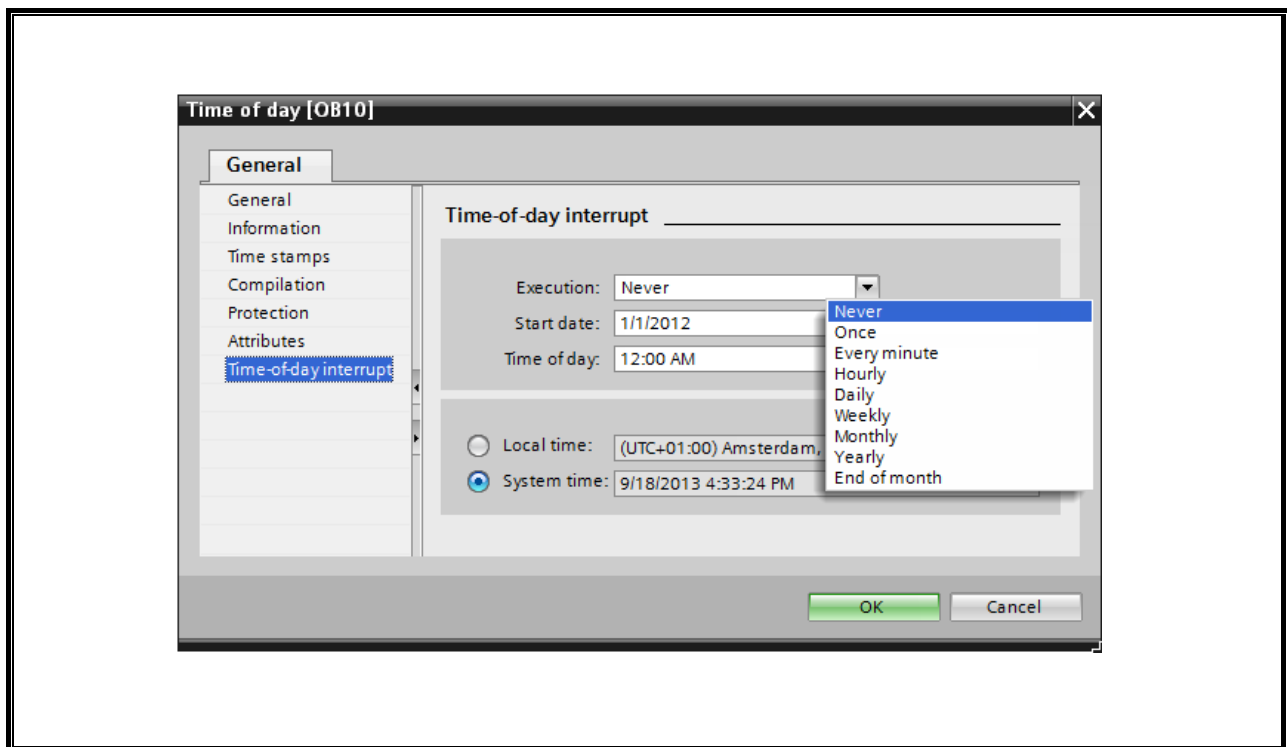
The number of pending events can be limited in order to keep temporary overload situations under control.

When, for example, the maximum number of start events for a cyclic interrupt OB is reached in the queue, every further start event is discarded.

At the input parameter "Event\_Count", the number of discarded start events is made available.



### 13.6.1. Time-of-day Interrupt (OB 10)



#### Time-of-Day Interrupts

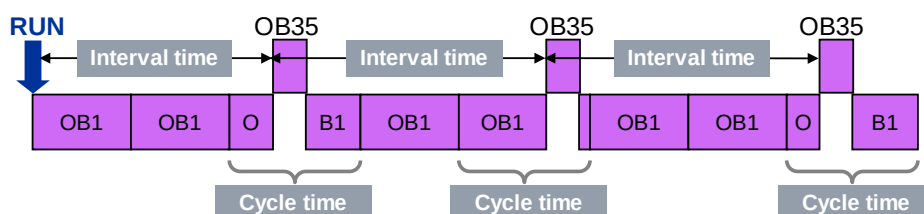
Time-of-day interrupts are used for executing a certain program called in OB 10 (as an example) either once only at a certain time or periodically (every minute, hourly, daily, weekly, monthly, yearly) starting at that time.

#### Note

In addition, the time-of-day interrupts can be controlled at runtime with the following "Extended instructions -> Interrupts":

- "SET\_TINT" Set start date, time and period
- "SET\_TINTL" Set start date, time and period
- "CAN\_TINT" Cancel time-of-day interrupt
- "ACT\_TINT" Activate time-of-day interrupt
- "QRY\_TINT" Query time-of-day interrupt

### 13.6.2. Cyclic Interrupt (OB35)



#### Cyclic Interrupt

With a cyclic interrupt, a block can be executed at fixed time intervals. The S7-1500 offers the OB 35, for example, as a cyclic interrupt OB. The default setting for its call interval is 100000µs; the selectable range is from 500µs to 60000000µs (60sec).

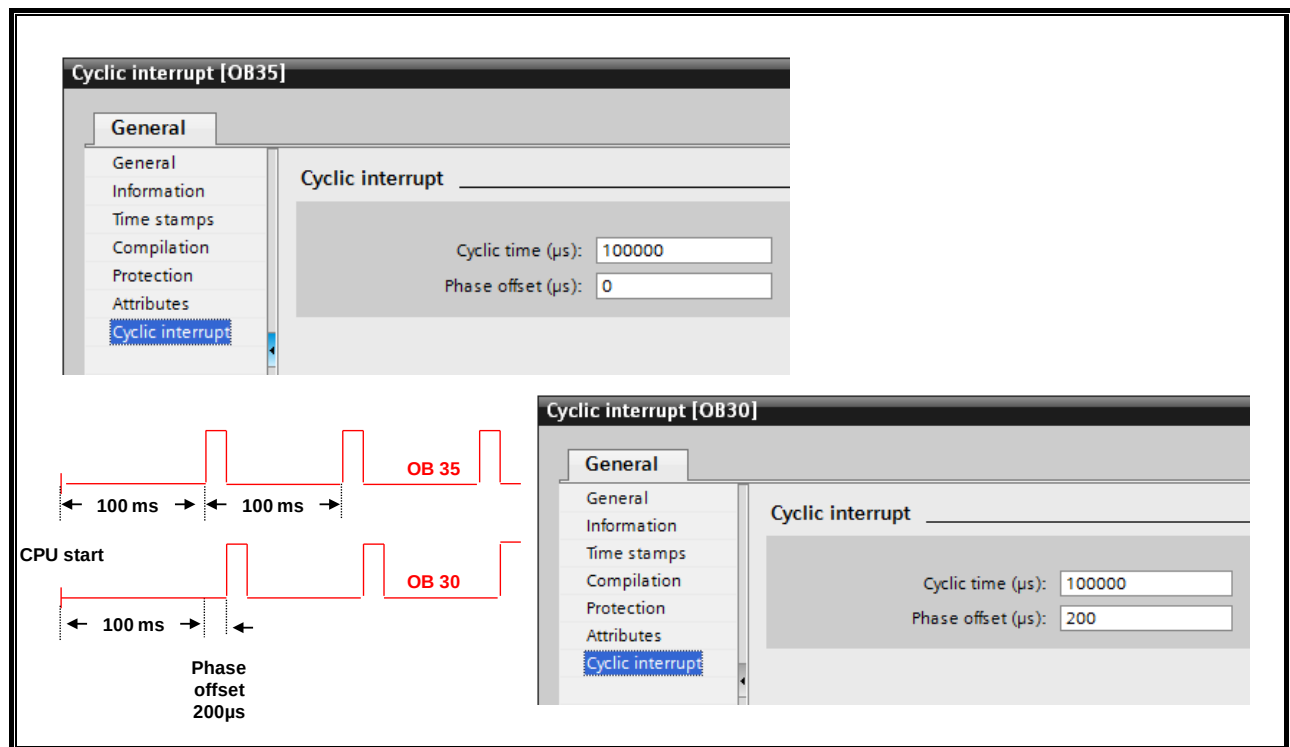
#### Interval Time

You must make sure that the interval you specify is longer than the time required for execution. The operating system calls OB35 at the specified time. If OB35 is still active at this time, the operating system calls OB80 (time error).

#### Note

Cyclic interrupts cannot be controlled at runtime with "Extended instructions".

### 13.6.3. Phase Offset for Cyclic Interrupts



#### Example for the Use of a Phase Offset

Two cyclic interrupt OBs are required in the program:

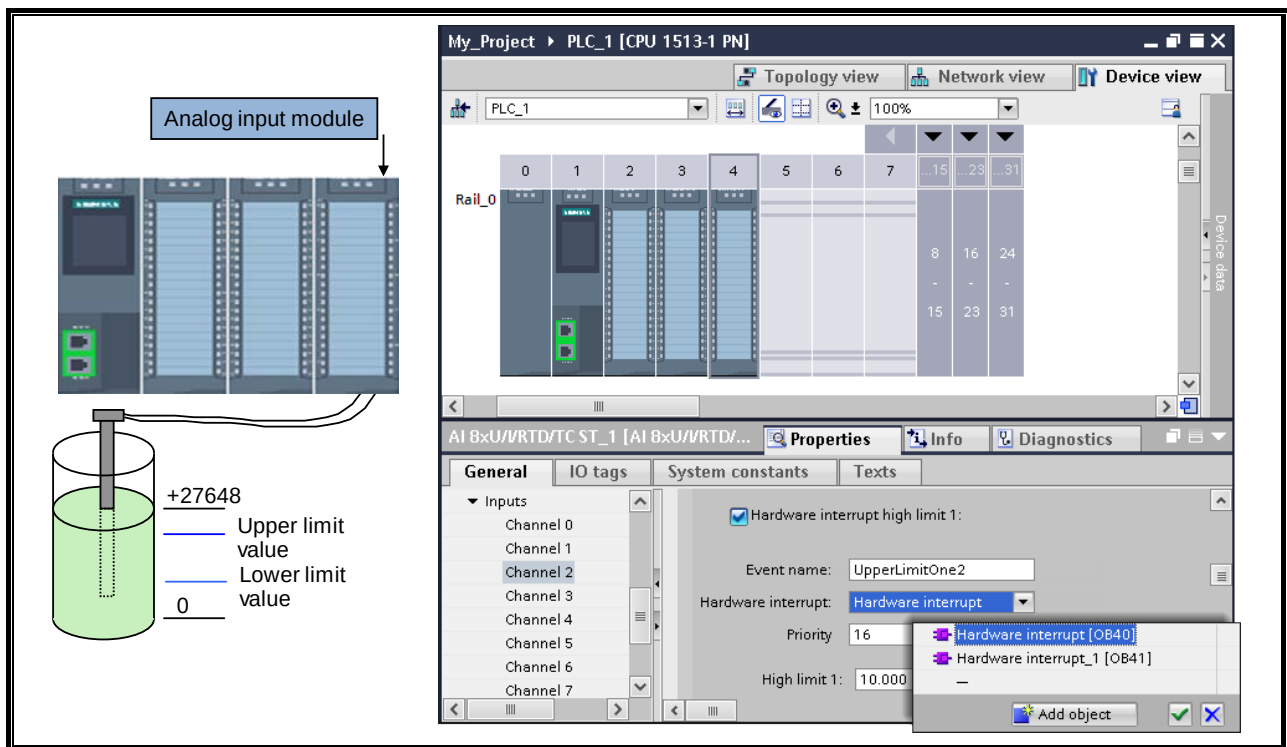
Cyclic interrupt OB30

Cyclic interrupt OB35

For cyclic interrupt OB30 and for cyclic interrupt OB35, a time period of 100 ms was set. After the time period of 100 ms has expired, both cyclic interrupt OBs are given their starting time.

However, in order to execute the cyclic interrupt OBs with a time lag, a phase offset is configured for one of the two cyclic interrupt OBs.

### 13.6.4. Hardware Interrupt (OB 40)



#### Hardware Interrupt

The program execution of a hardware interrupt OB (OB40) is started as soon as a certain event occurs.

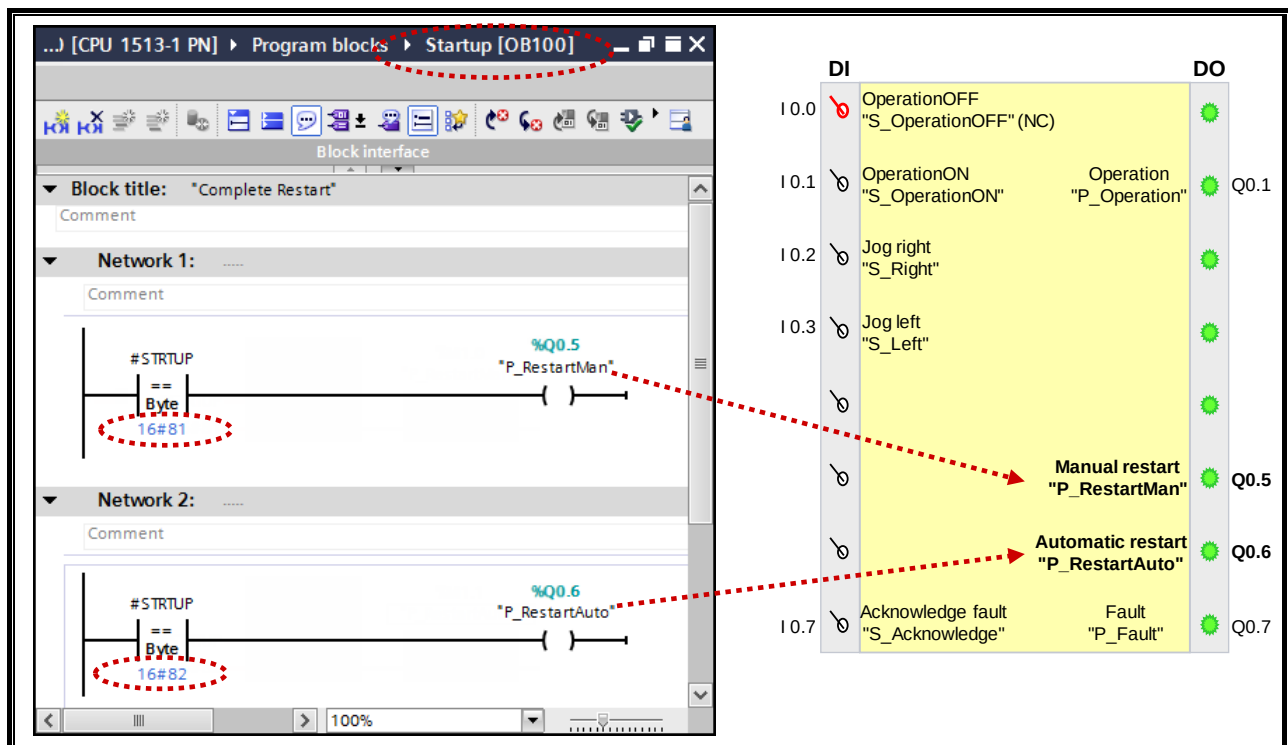
Hardware interrupts can be triggered by various module-specific signals:

For parameter-assignable signal modules (DI, DO, AI, AO) you use the "Devices & networks" editor (Device view) to specify which signal is to trigger the hardware interrupt.

#### Example

In configuring an analog input module, suitable limit values were specified in the above example. If the measured value then exceeds this limit, an interrupt is triggered on the CPU which causes the program to be interrupted and OB 40 to be called for execution.

### 13.6.5. Exercise 1: Displaying the Type of Startup and Acknowledging It (OB100)



#### Task

The CPU system writes the parameter "STRUP" with the hexadecimal value 81 for a manual restart, and, for an automatic restart, it writes it with the hexadecimal value 82.

A **manual restart** is to be displayed via the Simulator LED "P\_RestartMan" (Q0.5) and an **automatic restart** via the Simulator LED "P\_RestartAuto" (Q0.6).

Both LEDs are to be switched off in the moment when "P\_Operation" (Q0.1) is switched on.

#### What to Do

1. Insert the new block "Startup" (OB100) and deactivate the attribute "Optimized block access" in its Properties.
2. Program the switching on of the Simulator LEDs in OB 100 (see picture) and the switching off in "FC\_Mode" (FC15).
3. Download all blocks into the CPU and test the program function.
4. Save your project

#### Solution Hints

The OB 100 startup block is executed once both for **manual** (CPU STOP → RUN) and for **automatic restart** (power restoration). From a program point of view, the type of restart can be determined by evaluating the start information of OB 100. Information on the OB-specific start information can be found in the online help.

## 13.7. OBs for Handling Asynchronous Errors

| Type of Error                                                                                                                                                                                                              | Example                                                                | Error OB     | Priority                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|--------------|-------------------------------------|
| Time Error:<br>Max. allowed cycle time exceeded once<br>System reaction with OB: <b>RUN</b><br>without OB: <b>STOP</b><br><br>Max. allowed cycle time exceeded by more than double<br>System reaction with OB: <b>STOP</b> | Exceeding the maximum allowed cycle time,<br>Delayed call of a time OB | <b>OB 80</b> | <b>S7-1500:</b><br>Can be set: 2-26 |
| Diagnostic Interrupt<br>System reaction w/o OB: <b>RUN</b>                                                                                                                                                                 | Wirebreak at diagnostics-capable module, Power supply error            | <b>OB 82</b> | <b>S7-1500:</b><br>Can be set: 2-26 |
| Insert / Remove Interrupt<br>System reaction w/o OB: <b>RUN</b>                                                                                                                                                            | Insert / Remove a module                                               | <b>OB 83</b> | <b>S7-1500:</b><br>Can be set: 2-26 |
| Rack Failure<br>System reaction w/o OB: <b>RUN</b>                                                                                                                                                                         | Failure of a DP-slave or IO-Device                                     | <b>OB 86</b> | <b>S7-1500:</b><br>Can be set: 2-26 |

**Any event leads to an entry in the diagnostics buffer**

### Asynchronous Errors

Asynchronous errors occur asynchronous to the program execution and accordingly cannot be assigned to a defined program location.

### Time Errors

They occur when the current cycle time exceeds the cycle monitoring time set in "Devices & networks".

### Diagnostic Interrupts

They are triggered by diagnostics-capable modules, such as, analog modules in case of a fault (for example, wire break).

### Insert/Remove Interrupts

These interrupts are triggered when modules are inserted or removed (only S7-400). When a module is inserted, the operating system checks whether the correct module type was used. With this function, it is possible to insert/remove modules while the system is running.

### Rack Failure

A rack failure is detected with the failure of a rack, a subnet or a station of distributed I/O.

### 13.7.1. OBs for Handling Synchronous Errors

| Type of Error                                                                                               | Example                                                                            | OB                       | Priority                        |
|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|--------------------------|---------------------------------|
| <div>S7-1200</div> <div>Programming error<br/>Access error<br/>System reaction w/o OB: <div>RUN</div></div> | Access to non-existing DB<br>Direct access to non-existing or defective I/O module | no OB exists             | ⌘                               |
| <div>Programming error<br/>System reaction w/o OB: <div>STOP</div></div> <div>S7-1500</div>                 | Access to non-existing DB                                                          | OB 121<br>(only S7-1500) | S7-1500:<br>Can be set: 2 to 26 |
| <div>Access error<br/>System reaction w/o OB: <div>RUN</div></div>                                          | Direct access to non-existing or defective I/O module                              | OB 122<br>(only S7-1500) |                                 |

Any error leads to an entry in the diagnostics buffer

#### Synchronous Errors

Synchronous errors occur synchronously to the program execution and accordingly can be assigned to a defined program location.

With a programming error, OB121 is called; with an access error, OB122. If, in case of an error, the appropriate synchronous error OB does not exist in the CPU, the CPU switches to the STOP state.

#### S7-1500:

You can set the priority of the synchronous error OBs from 22 to 26. The register contents that the interrupted block has used are not available in the error OB and cannot be manipulated by means of system functions.

# Contents

|                                                                                        |             |
|----------------------------------------------------------------------------------------|-------------|
| <b>14. Troubleshooting .....</b>                                                       | <b>14-2</b> |
| 14.1. Categories of Errors.....                                                        | 14-3        |
| 14.2. STEP 7 - Test Functions, Overview.....                                           | 14-4        |
| 14.3. System Diagnostics - Overview .....                                              | 14-5        |
| 14.4. Hardware Diagnostics .....                                                       | 14-6        |
| 14.5. Online & Diagnostics.....                                                        | 14-7        |
| 14.5.1. Diagnostics: Diagnostics Buffer .....                                          | 14-8        |
| 14.5.2. Diagnostics Buffer: Interpreting Error Messages .....                          | 14-9        |
| 14.5.3. Diagnostics Buffer: Opening a Faulty Block .....                               | 14-10       |
| 14.6. Call Hierarchy.....                                                              | 14-11       |
| 14.6.1. Exercise 1: Creating a Program Backup in the Project Library .....             | 14-12       |
| 14.6.2. Exercise 2: Copying the Error Program .....                                    | 14-13       |
| 14.6.3. Exercise 3: STOP Troubleshooting .....                                         | 14-14       |
| 14.7. Monitor Block (Block Status).....                                                | 14-15       |
| 14.7.1. Monitor Block: Modify Tags.....                                                | 14-16       |
| 14.7.2. Monitor Block: Trigger Conditions / Call Environment .....                     | 14-17       |
| 14.8. Monitor / Modify Variables (Tags): Watch Tables.....                             | 14-18       |
| 14.8.1. Monitor / Modify Variables (Tags): Trigger Points .....                        | 14-19       |
| 14.8.2. Enable Peripheral Outputs (in Planning Stage) .....                            | 14-20       |
| 14.8.3. Force Variables (Tags) .....                                                   | 14-21       |
| 14.9. Reference Data: Cross-references of PLC Tags.....                                | 14-22       |
| 14.9.1. Reference Data: Cross-references of a Variable (Tag) in the Block Editor ..... | 14-23       |
| 14.9.2. Reference Data: Assignment of I, Q, M, T, C .....                              | 14-24       |
| 14.9.3. Reference Data: Call Structure .....                                           | 14-25       |
| 14.9.4. Reference Data: Dependency Structure.....                                      | 14-26       |
| 14.10. Comparing Blocks .....                                                          | 14-27       |
| 14.10.1. Comparing Blocks: Show Details.....                                           | 14-28       |
| 14.11. Exercise 4: Testing the Motor Jog .....                                         | 14-29       |
| 14.11.1. Exercise 5: Testing 'Counting' the Parts .....                                | 14-30       |
| 14.11.2. Exercise 6: Testing the Evaluation of Fault 3 .....                           | 14-31       |
| 14.12. TRACE Analyzer Function .....                                                   | 14-32       |
| 14.12.1. TRACES Editor .....                                                           | 14-33       |
| 14.12.2. Configuring a TRACE .....                                                     | 14-34       |
| 14.12.3. Downloading TRACE into the CPU and Activating It.....                         | 14-35       |
| 14.12.4. Looking at / Saving TRACE in STEP7 .....                                      | 14-36       |
| 14.13. Exercise 7: Creating, Looking At and Saving TRACE .....                         | 14-37       |



## 14. Troubleshooting

At the end of the chapter the participant will

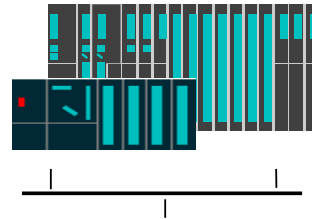
- ... be able to explain when to use which troubleshooting functions
- ... be able to check the wiring of digital input and output modules
- ... be able to apply troubleshooting functions for STOP troubleshooting and to be able to use them for error correction
- ... be able to apply troubleshooting functions for searching for logical errors and to be able to use them for error correction



## 14.1. Categories of Errors

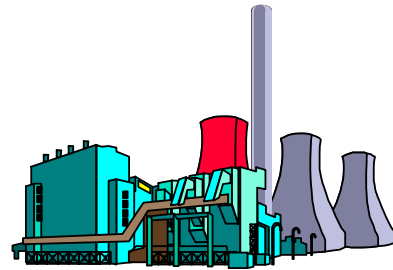
### Errors Detected by the System

- Acquiring, evaluating and indicating errors within a PLC (CPU STOP possible)
  - Module failure
  - Short-circuit in signal cables
  - Scan time overrun
  - Programming error (accessing a non-existent block)



### Functional Errors

- Desired function is either not executed at all or is not correctly executed
  - Process fault (sensor/actuator, cable defective)
  - Logical programming error (not detected during creation and commissioning)



### Monitoring Functions

Diagnosis is important in the operating phase of a system or machine. Diagnosis usually occurs when a problem (disturbance) leads to standstill or to the incorrect functioning of the system or machine. Due to the costs associated with downtimes or faulty functions, the associated cause of the disturbance has to be found quickly and then eliminated.

### Categories of Errors

Errors that occur can be divided into two categories, depending on whether or not they are detected by the PLC:

- Errors that are detected by the PLC's operating system and that can lead to the Stop state of the CPU.
- Functional errors, that is, the CPU executes the program as usual, but the desired function is either not executed at all or it is executed incorrectly. The search for these types of errors is much more difficult, since the cause of the error is initially hard to determine.

Possible causes could be:

- A logical programming error (software error) that was not detected during creation and commissioning of the user program and probably occurs only on extremely rare occasions.
- A process fault that was triggered by the faulty functioning of components directly associated with the process control, such as cables to sensors/actuators or by a defect in the sensor/actuator itself.

## 14.2. STEP 7 - Test Functions, Overview

| Errors Detected by the System                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Functional Errors                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Programming error (without OB121): <b>STOP</b></li> <li>• Scan time overrun <b>STOP</b></li> <li>• Access error <b>RUN</b></li> <li>• Diagnostics interrupts <b>RUN</b></li> <li>• <b>Online &amp; Diagnostics</b> <ul style="list-style-type: none"> <li>- Diagnostics buffer</li> </ul> </li> <li>• <b>Task Card "Testing"</b> <ul style="list-style-type: none"> <li>- Call hierarchy</li> <li>- Local data stack (in planning stage)</li> </ul> </li> <li>• <b>Diagnose Modules</b> <ul style="list-style-type: none"> <li>- Diagnostic status (all modules)</li> </ul> </li> </ul> | <ul style="list-style-type: none"> <li>• Process fault (e.g. wire break) <b>RUN</b></li> <li>• Logical programming error (e.g. double assignment) <b>RUN</b></li> <li>• <b>Monitor / Modify Variables</b> <ul style="list-style-type: none"> <li>→ Watch and Force tables</li> </ul> </li> <li>• <b>Monitor Blocks (Block Status)</b> <ul style="list-style-type: none"> <li>→ Monitor in the Blocks editor</li> <li>Additionally with Call environment</li> </ul> </li> <li>• <b>Tools</b> <ul style="list-style-type: none"> <li>- Cross references</li> <li>- Assignment list (I/Q/M/T/C)</li> <li>- Call structure</li> </ul> </li> </ul> |
| <ul style="list-style-type: none"> <li>• <b>"Trace" analyzer function</b></li> <li>• <b>Program comparison</b></li> <li>• <b>Set breakpoints</b></li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

in planning for S7-1200/1500

### Test Functions

There are various STEP 7 test functions for troubleshooting, depending on the type of error caused:

- **...when CPU in STOP**

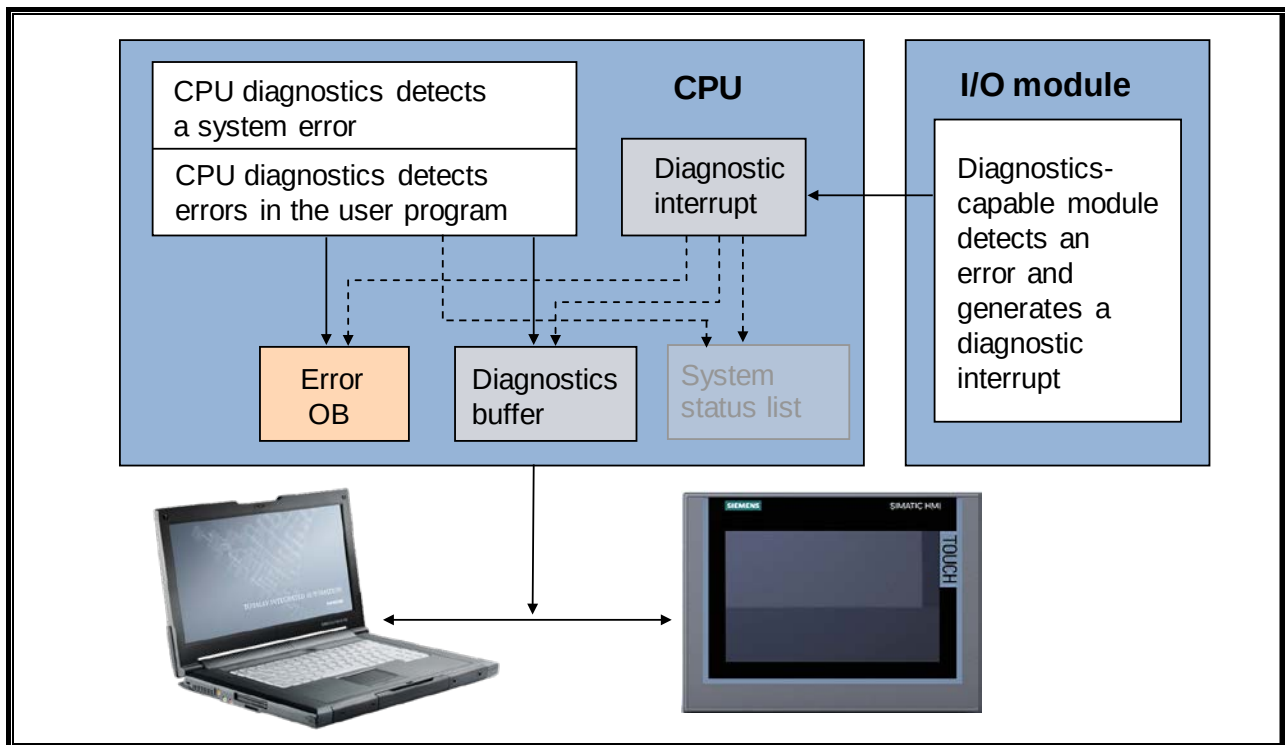
For errors that are detected by the system, the test functions Diagnostics buffer, Call hierarchy, Local data stack and Hardware diagnostics give detailed information on the cause of the error and the location of the interruption. By programming Error OBs (see the chapter on Organization Blocks), information on the error that occurred can be evaluated by program and the transition of the CPU into the STOP state can be prevented. If the CPU has stopped, the use of the test functions Monitor / Modify Variable and Monitor Blocks makes little sense since the CPU neither reads nor outputs process images while in the STOP state, and also no longer executes the program.

- **...when CPU in RUN**

Vice versa, it makes little sense, as a rule, to use test functions such as Local data stack for troubleshooting when the CPU is in RUN, since program execution has not been interrupted and the system does not provide any information on the error that occurred. The Module Information test function merely provides general information on the CPU's operating status or on errors that occurred in the past. Functional errors can be diagnosed as follows:

- **Process Fault** (such as a wiring error)
  - wiring test of the inputs: *Monitor Variable*
  - wiring test of the outputs: *Enable Peripheral Outputs* (only for CPU STOP)
- **Logical Programming Errors** (such as a double assignment)
  - All test functions listed, with the exception of *Enable Peripheral Outputs*, can be used for searching for logical program errors.
- **Force:** Forced control of operands regardless of the program logic
- **Breakpoints:** Program execution in single steps

### 14.3. System Diagnostics - Overview



#### System Diagnostics

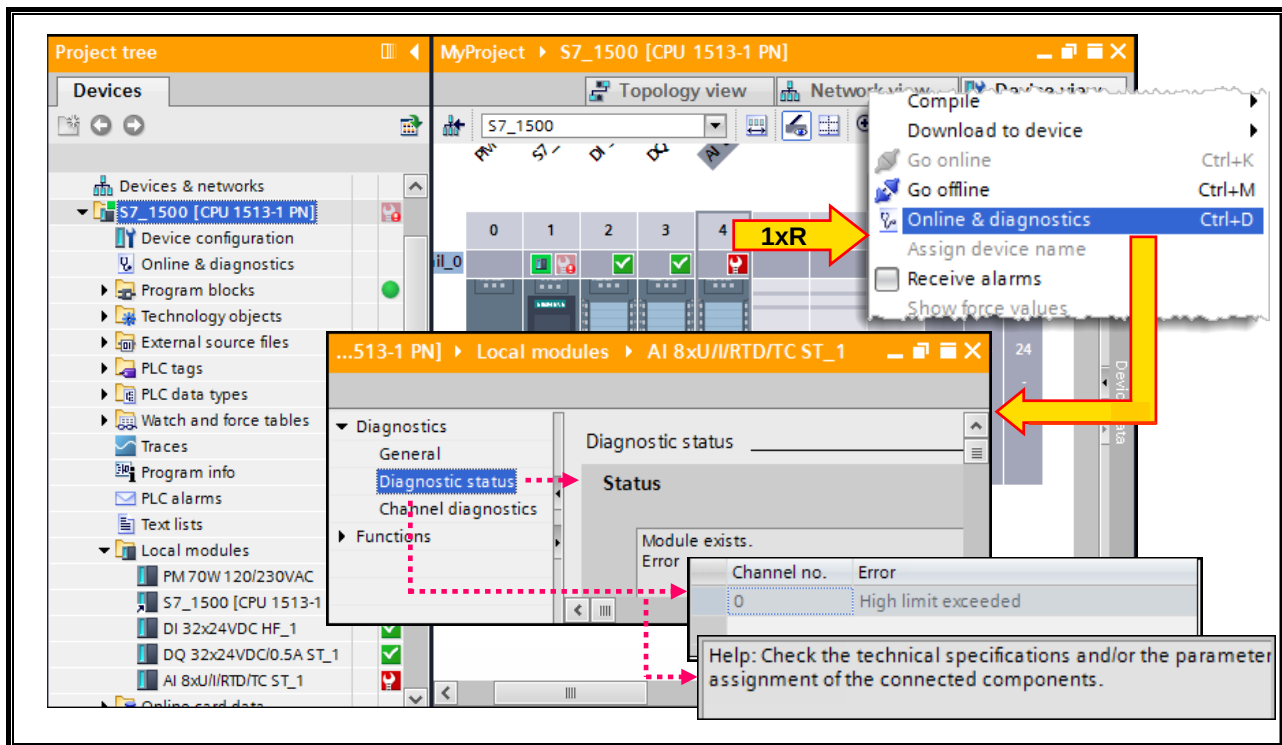
All those monitoring functions that deal with the correct functioning of the components of an automation system are grouped together under System Diagnostics. All S7-CPU have an intelligent diagnostics system. The acquisition of diagnostic data by the system diagnostics does not have to be programmed. It is integrated in the operating system of the CPU and in other diagnostics-capable modules and runs automatically. The CPU (temporarily) stores errors that occur in the diagnostics buffer and thus enables a fast and targeted error diagnosis by service personnel, even for sporadically occurring errors.

#### System Reaction

The operating system takes the following actions when it detects an error or a STOP event, such as an operating mode change (RUN → STOP):

- A message on the cause and the effect of the occurring error is entered in the diagnostics buffer, complete with the date and time. The diagnostics buffer is a FIFO (circular) buffer on the CPU module for storing error events. The size of the diagnostics buffer depends on the CPU (such as the CPU 315-2 DP = 100 entries). In the FIFO buffer structure, the most recently entered message overwrites the oldest diagnostics buffer entry. A CPU Memory Reset cannot delete the diagnostics buffer, only a "Reset to factory settings".
- System status lists that give information about the system status of the modules are updated.
- The Error OB associated with this error is called. This gives the user the opportunity of carrying out his own error handling.

## 14.4. Hardware Diagnostics



### Diagnosing Hardware

To use this function you must open the "Device configuration" and establish an online connection. The function "Diagnostic Hardware" gives information about the status or operating status of the modules. You can see that there is diagnostic information for a module when you see the diagnostic symbols that indicate the status of the associated module or the operating status of the CPU.

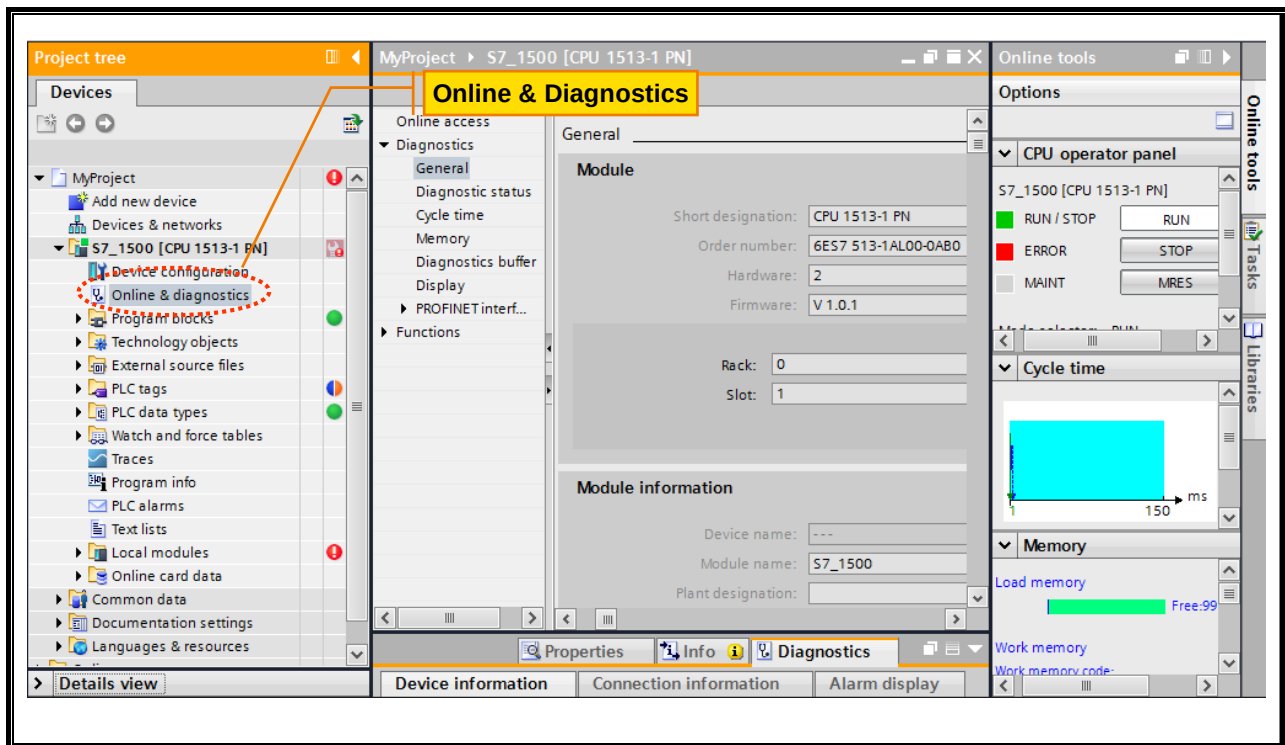
In the example shown, the analog input module (slot 7) has triggered a diagnostic interrupt. As a result, the CPU has gone into the STOP mode.

Both modules have been given symbols accordingly. With a single right-click on the CPU and then "Online & diagnostics", the CPU's diagnostic buffer is output; for the analog module, the associated diagnostic data (see picture)

### Symbol Meanings

| Icon | Meaning              | Icon | Meaning                                                                                                                                                    |
|------|----------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      | No fault             |      | Hardware error in lower-level component: The online and offline versions differ (only in the project tree) in at least one lower-level hardware component. |
|      | Maintenance required |      | Software error in lower-level component: The online and offline versions differ (only in the project tree) in at least one lower-level software component. |
|      | Maintenance demanded |      | Online and offline versions of the object are different                                                                                                    |
|      | Error                |      | Object only exists online                                                                                                                                  |
|      |                      |      | Object only exists offline                                                                                                                                 |
|      |                      |      | Online and offline versions of the object are the same                                                                                                     |

## 14.5. Online & Diagnostics

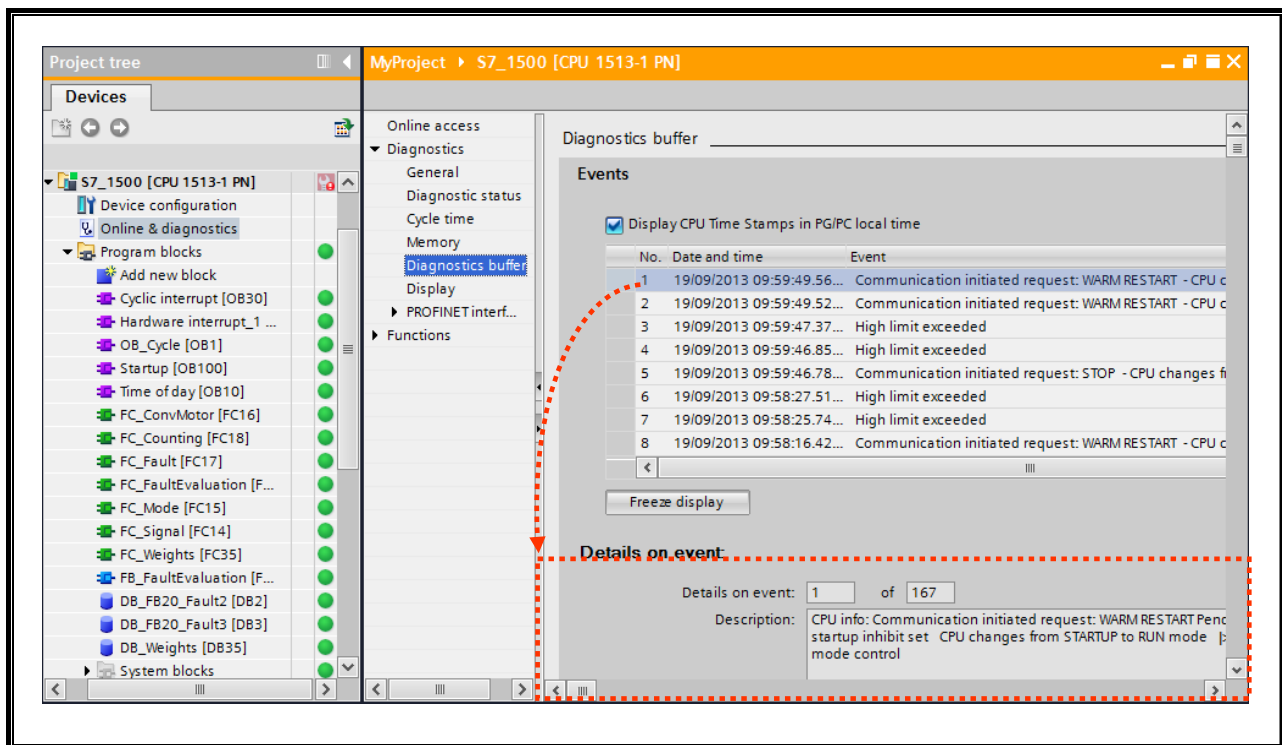


### Online and Diagnostics

The Module Information function reads the most important data from the directly connected module. You will find additional information in the individual tabs:

- **General:** Among other things, the module designation, hardware and firmware versions
- **Diagnostics Buffer:** It contains all diagnostic events in the order they occurred. In the display, all events are listed in plain language and in the order they occurred.
- **Memory:** Size and usage of the EPROM load memory, RAM load memory and the work memory.
- **Cycle Time:** Displays the monitoring time selected, the shortest, the longest and the current cycle time
- **Time System:** Displays the real-time clock and the integrated operating hours counter
- **Performance Data:** Displays the integrated system blocks and the available organization blocks as well as operand areas (I,Q,M,T,C,L)
- **Communication:** Displays the performance data of the communication interfaces and the connection overview

### 14.5.1. Diagnostics: Diagnostics Buffer



#### Diagnostics Buffer

The diagnostics buffer is a buffered memory area on the CPU organized as a circular buffer. It contains all diagnostics events (error messages, diagnostic interrupts, startup information etc.) of the CPU in the order in which they occurred. The highest entry is the last event to occur.

All events can be displayed on the programming device in plain language and in the order in which they occurred.



The size of the diagnostics buffer depends on the CPU. As well, not all of the diagnostics buffer is buffered with PowerOFF (only a part is retentive).

- Number of entries, 1000 to 3200
- Of that, 250 to 500 retentive

#### Details on Event

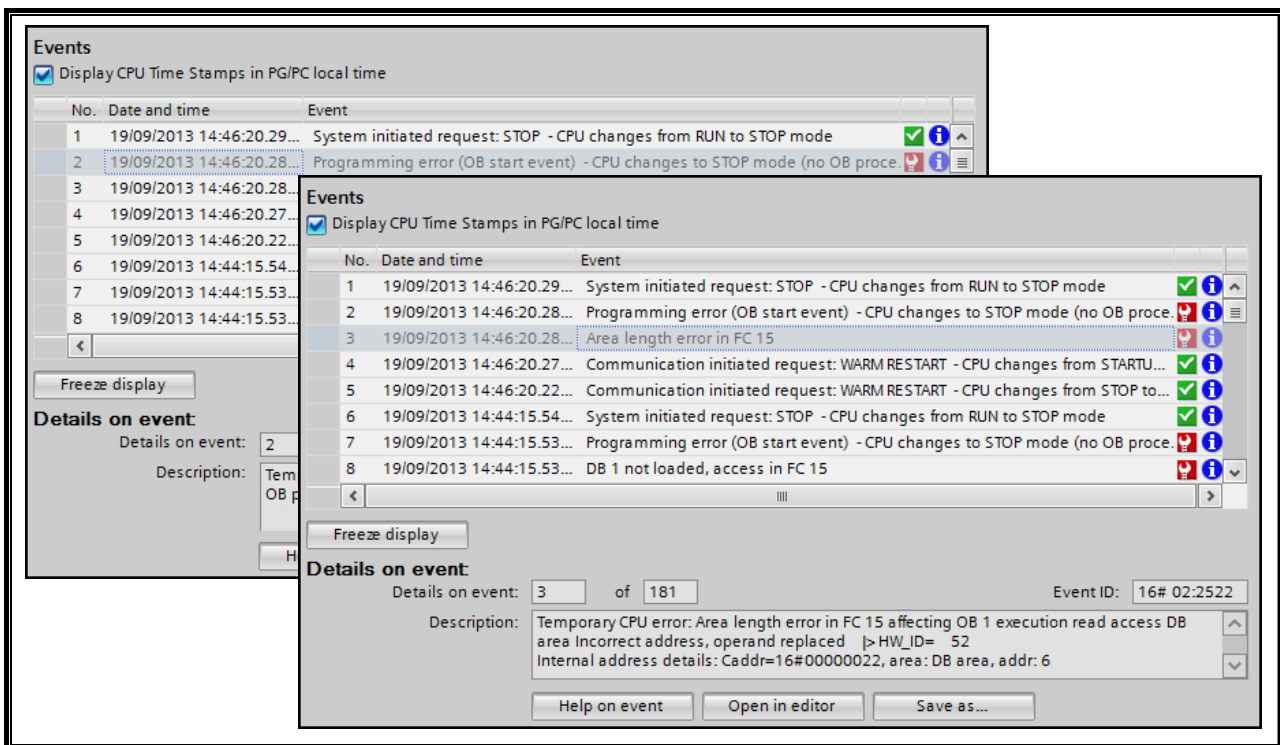
Some additional information is also provided for the selected event in the "Details on event" box:

- Event name and number
- Additional information depending on the event, such as, the address of the instruction that caused the event etc.

#### Help on Event

When you click on the button, help on the event selected in the list is opened. (In the example shown, a programming error has occurred for which the relevant error OB (OB121) is not programmed in the CPU.)

## 14.5.2. Diagnostics Buffer: Interpreting Error Messages



### Interpreting the Diagnostics Buffer

To interpret the diagnostics buffer, you have to look at the events that belong together in the sequence in which they occurred, in other words, from bottom to top:  
For orientation:

- In our example, a WARM RESTART was performed before the most recent error occurred (events no.4 and 5).

### Entries in the Diagnostics Buffer

The last error that occurred after this warm restart leads to the following entries in the diagnostics buffer:

- Event No. 3 :  
*Area length error in FC15*  
Details on event:
  - affects OB1 execution  
(FC15 is called in the cyclic program)
  - read access DB area incorrect address  
(the specified address could not be accessed)
- Event No. 2 :  
*Programming error (OB start event)*  
*(if it exists, the operating system calls an OB for a programming error)*  
Details on event:
  - CPU changes to STOP mode
  - no OB processing  
(because the programming error OB121 was not programmed)
- Event No. 1:
  - CPU changes to STOP mode



### 14.5.3. Diagnostics Buffer: Opening a Faulty Block

The screenshot shows the 'Diagnostics buffer' window with a list of events. Event 3 is selected: 'Area length error in FC 15'. The details for this event are shown below the list. A yellow arrow points from the event list to the 'Open in editor' button. Another yellow arrow points from the 'Open in editor' button to the 'FC\_Mode [FC15]' ladder logic diagram, which shows a network with a read access to %DB99.DBX100.1.

| No. | Date and time             | Event                                                                         |
|-----|---------------------------|-------------------------------------------------------------------------------|
| 1   | 19/09/2013 15:07:37.38... | System initiated request: STOP - CPU changes from RUN to STOP mode            |
| 2   | 19/09/2013 15:07:37.37... | Programming error (OB start event) - CPU changes to STOP mode (no OB proce... |
| 3   | 19/09/2013 15:07:37.37... | Area length error in FC 15                                                    |
| 4   | 19/09/2013 15:07:37.36... | Communication initiated request: WARM RESTART - CPU changes from STARTU...    |
| 5   | 19/09/2013 15:07:37.27... | Communication initiated request: WARM RESTART - CPU changes from STOP to...   |
| 6   | 19/09/2013 14:46:20.29... | System initiated request: STOP - CPU changes from RUN to STOP mode            |
| 7   | 19/09/2013 14:46:20.28... | Programming error (OB start event) - CPU changes to STOP mode (no OB proce... |
| 8   | 19/09/2013 14:46:20.28... | Area length error in FC 15                                                    |

Details on event: 3 of 186

Description: Temporary CPU error: Area length error in FC 15 affecting OB 1 execution rea...  
area incorrect address, operand replaced -> HW\_ID= 52  
Internal address details: Caddr=16#0000001E, area: DB area, addr: 100

Time stamp: 19/09/2013 15:07:37.371 PM

Module: S7\_1500

Rack/slot: Rack 0 / Slot 1

Priority: Error

Incoming/outgoing: Incoming event

Buttons: Help on event, Open in editor, Save as...

FC\_Mode [FC15]

Network 1: Operation ON/OFF

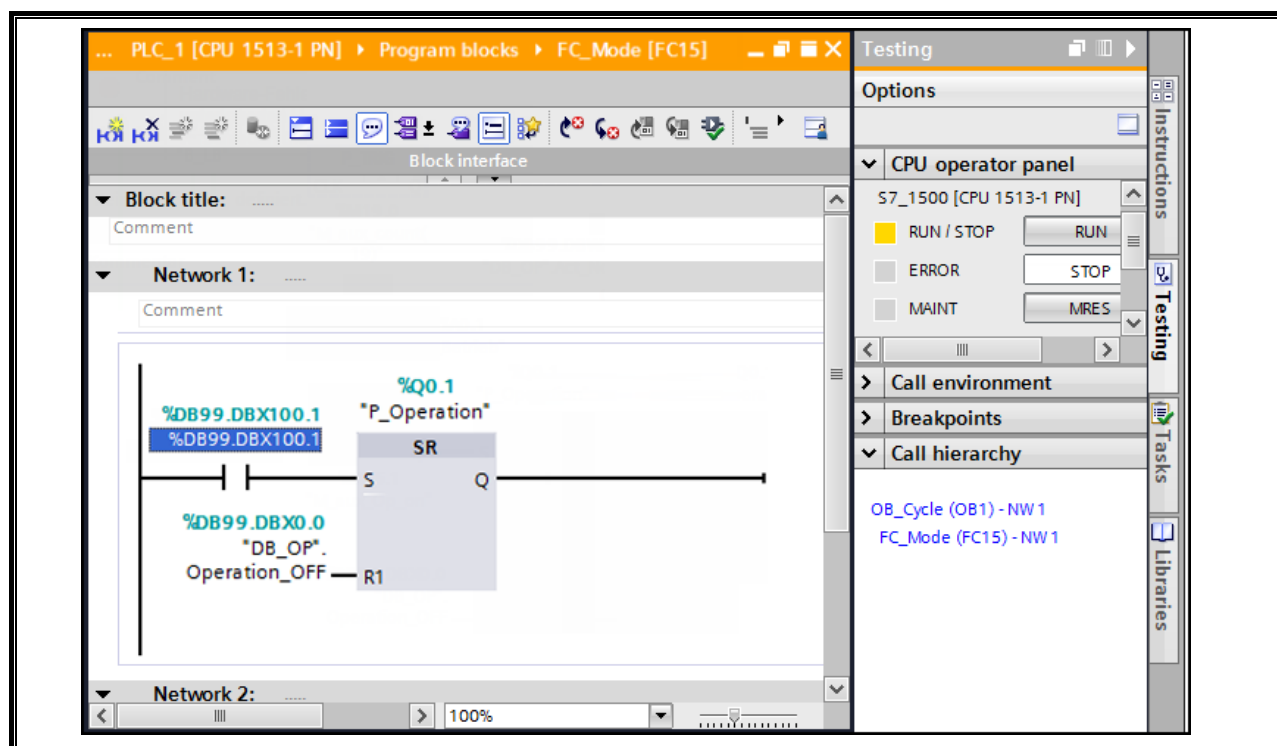
Logic diagram showing a network with a read access to %DB99.DBX100.1 and a set coil (S) for %Q0.1.

#### Opening a Block

For synchronous errors, that is, for errors that were triggered by a faulty instruction in the user program, you can open the block in which the interruption occurred by clicking on the "Open in editor" button.

If the STL language is selected, the cursor is positioned directly in front of the instruction that caused the interruption. In LAD/FBD, the network causing the interruption is highlighted. In the example shown, the DB99.DBX100.1 bit is read-accessed, which does not exist in the data block.

## 14.6. Call Hierarchy



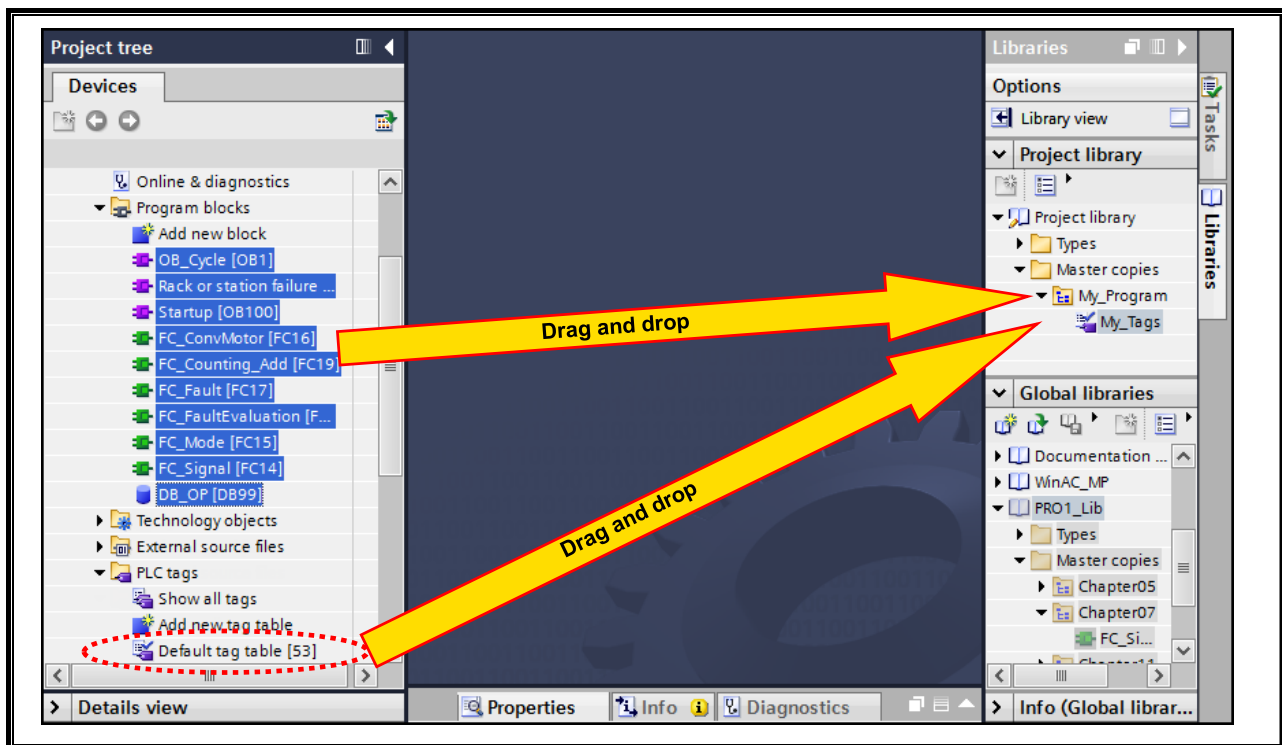
### Call Hierarchy

The "Call hierarchy" tells you in which call path the block is monitored.

If the block was opened via the "Open in editor" button from the Diagnostics buffer, you can also see in which path the error occurred via the entry in the call hierarchy.

If parameter-assignable blocks that are called multiple times in the program are monitored with the test function "Block status" (Monitor block), the "Block status" does not supply any unambiguous results. (see 14.7.2 Monitor Block: Trigger Conditions / Call Environment)

### 14.6.1. Exercise 1: Creating a Program Backup in the Project Library



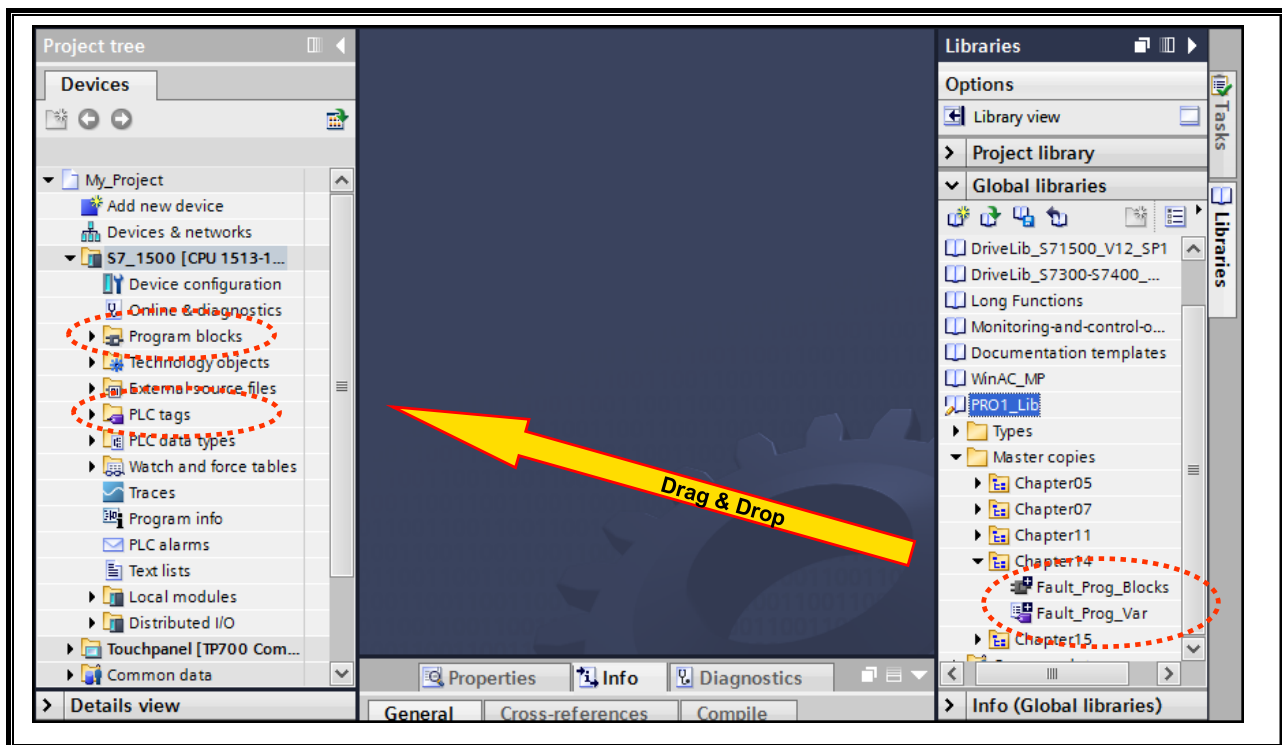
#### Task

You are to make a backup copy of your own program and the associated tags in the Project library of your project "My\_Project" since you will be working with a prepared error program afterwards.

#### What to Do

1. In the "Libraries", open the Project library.
2. Select all S7 blocks of the CPU program and copy them into your Project library using drag & drop.
3. Select all tag tables and copy them into your Project library using drag & drop.
4. Save your project and thus also the Project library.

## 14.6.2. Exercise 2: Copying the Error Program



### Task

You are to copy a prepared "error program" with the associated PLC tags from the global library "PRO1\_Lib" into your own project.

### What to Do

1. In your project, delete all S7 blocks and all PLC tag tables.
2. In the "Global libraries", open the library <Drive>:\Archives\TIA\PRO1\_Lib.
3. Using drag & drop, copy the library elements "Fault\_Prog\_Blocks" and "Fault\_Prog\_Var" into your project (see picture).
4. Download all blocks of the error program into the CPU by selecting the CPU in the Project tree and then clicking on the download button.
5. Carry out a CPU restart.
6. Save your project.

### 14.6.3. Exercise 3: STOP Troubleshooting

#### Task

The error program contains two STOP errors which are now to be corrected by you. If the CPU remains in RUN after error correction and subsequent restart, the exercise has been successfully completed.

In addition to the errors (STOP errors) detected by the system, the program also contains logical errors (RUN errors) so that the correct functioning of the program is still not established even after the STOP errors are eliminated. The logical errors will be eliminated in the next exercises.

#### What to Do

Please note that after every STOP error correction, a CPU restart must be carried out. If the CPU once again goes into the STOP mode after the restart, a further STOP error exists.

During error correction, answer the following questions on the errors that occur:

- First STOP error determined:



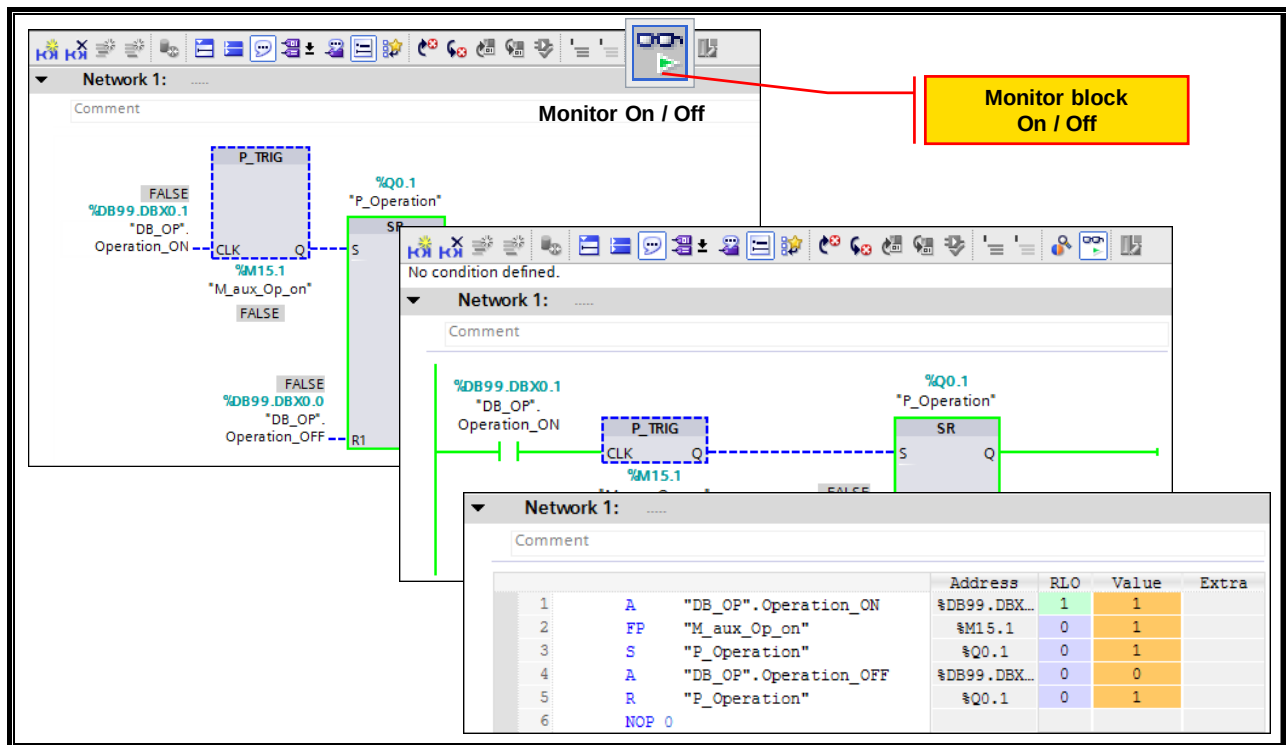
- Interrupted block: .....
- Error: .....
- Correction: (old instruction -> new instruction)
- .....

- Second STOP error determined:



- Interrupted block: .....
- Error: .....
- Correction: (old instruction -> new instruction)
- .....

## 14.7. Monitor Block (Block Status)



### Area of Use

The *Monitor Block* test function is used to be able to follow the program execution within a block. For this, the states or contents of the operands used in the block at the time of program execution are displayed on the screen. You can activate the "Monitor" ("Block Status") test mode for the block which is currently open in the LAD/STL/FBD Editor by clicking the *Glasses icon* or via *Online → Monitor*.

At the beginning of the test function, it is insignificant whether the block to be monitored is opened *online* or *offline* in the Editor. Should, however, the block opened offline not match the block saved online in the CPU, you first either have to open the block saved online or load the block opened offline into the CPU and then monitor it.

In the test mode, the states of the operands and LAD / FBD elements are displayed in different colors. You define these by selecting the menu option *Options → Settings*:

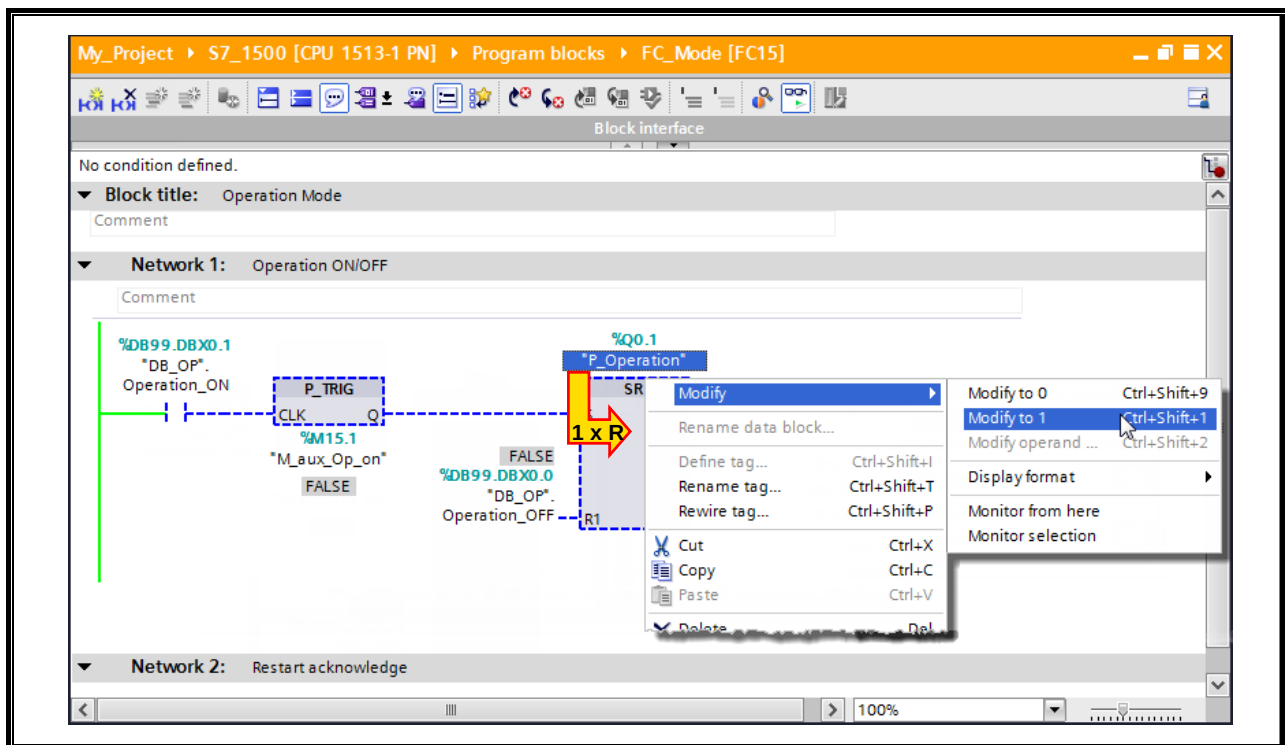
Examples:

- Status fulfilled → "Element is displayed in green"
- Status not fulfilled → "Element is displayed in blue"

### Notes

The Status display is only active when the CPU is in RUN mode and the instructions to be monitored are being processed!

### 14.7.1. Monitor Block: Modify Tags

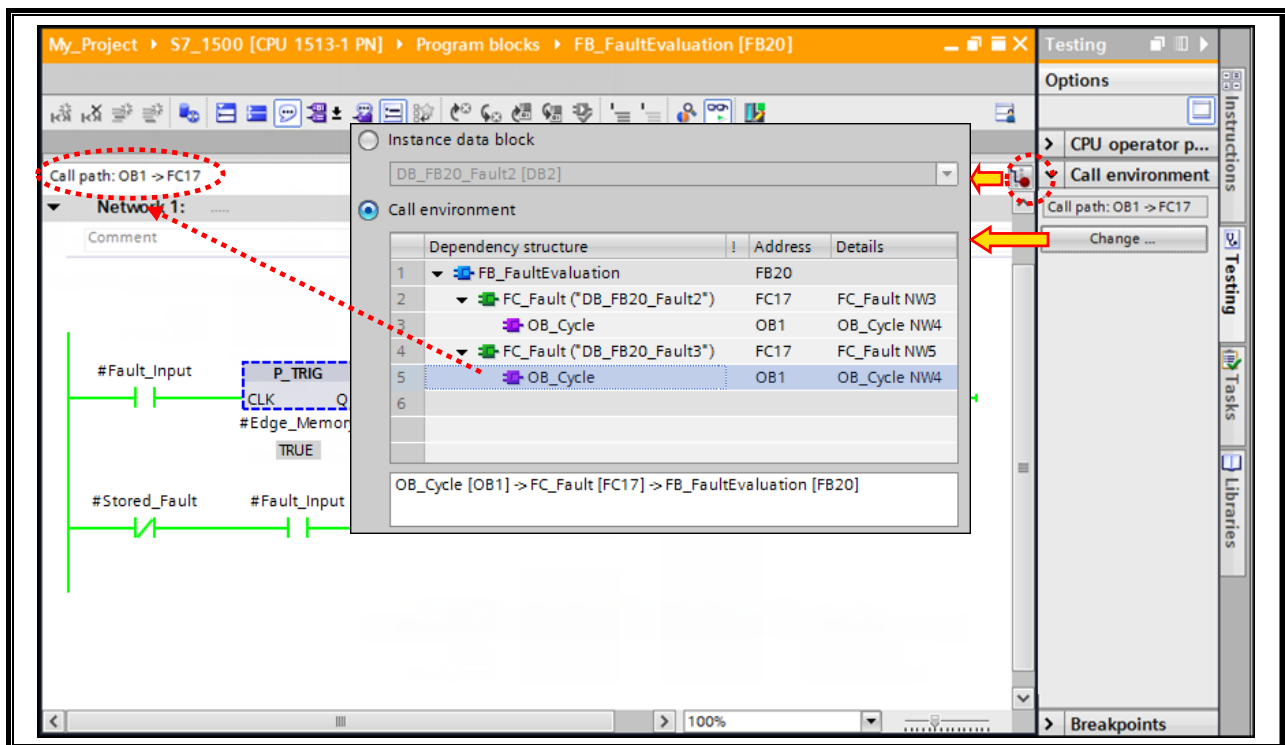


#### Modify Tags

When the "Monitor block" test function is activated, it is possible to modify tags to status '0' or '1'. The assignment of the status occurs once.

If the tag, whose status was modified to '0' or '1', is not overwritten by the program, it remains at the assigned status. If, for example, an output is modified to status '1' and this tag is not overwritten by the program, the output remains switched on or to status '1'.

## 14.7.2. Monitor Block: Trigger Conditions / Call Environment



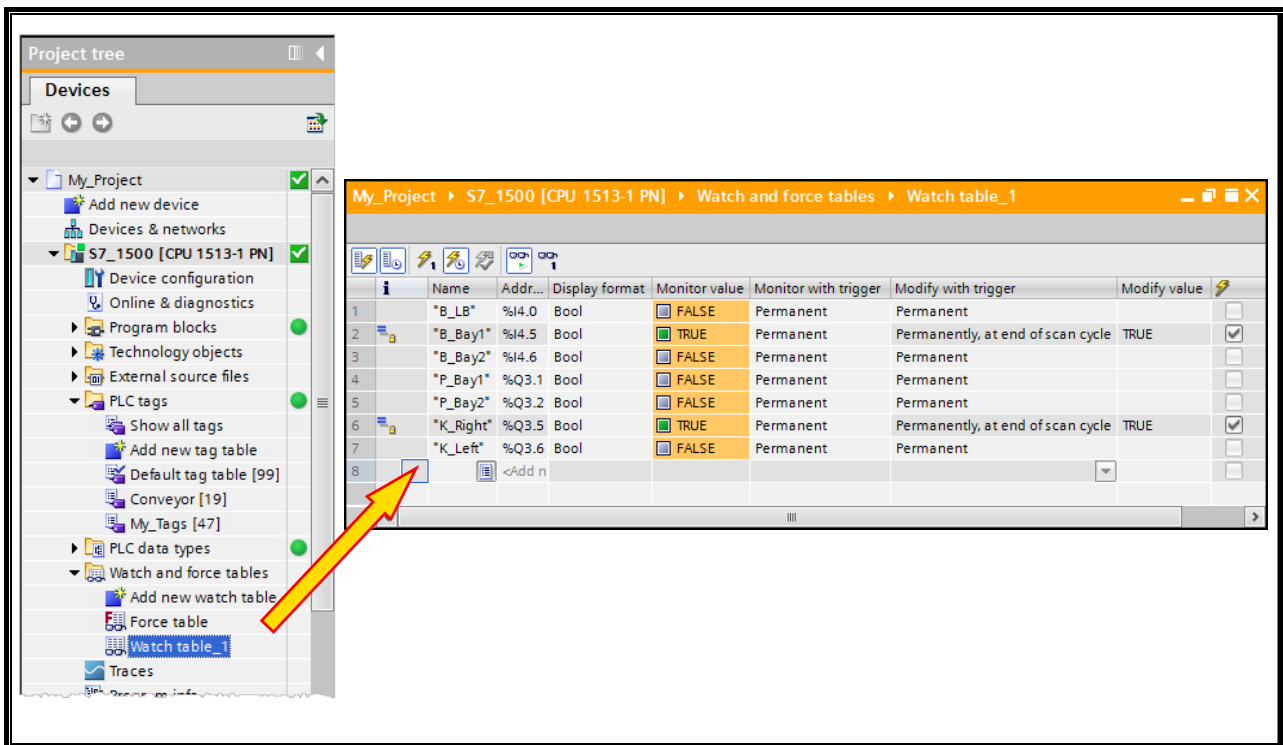
### Trigger Conditions / Call Environment

The call conditions for blocks and for breakpoints can be defined. Thereby it is determined under which conditions the program status of a block is displayed or the program execution is interrupted at a breakpoint. The following conditions can be selected:

- Global data block
  - The program status of a block is only displayed when the block is called with the selected global data block.
- Instance data block
  - The program status of a function block is only displayed when the function block is called with the selected instance data block.
- Call path
  - The program status of a block is only displayed when the block is called by a specific block or from a specific path.



## 14.8. Monitor / Modify Variables (Tags): Watch Tables



### Area of Use

The "Monitor/Modify Variables (Tags)" test function is used to monitor and / or modify variables (tags) in any format you choose. For this, the desired variables are entered in a watch table. With the exception of block-local, temporary variables, you can monitor and/or modify all variables (tags) or operands.

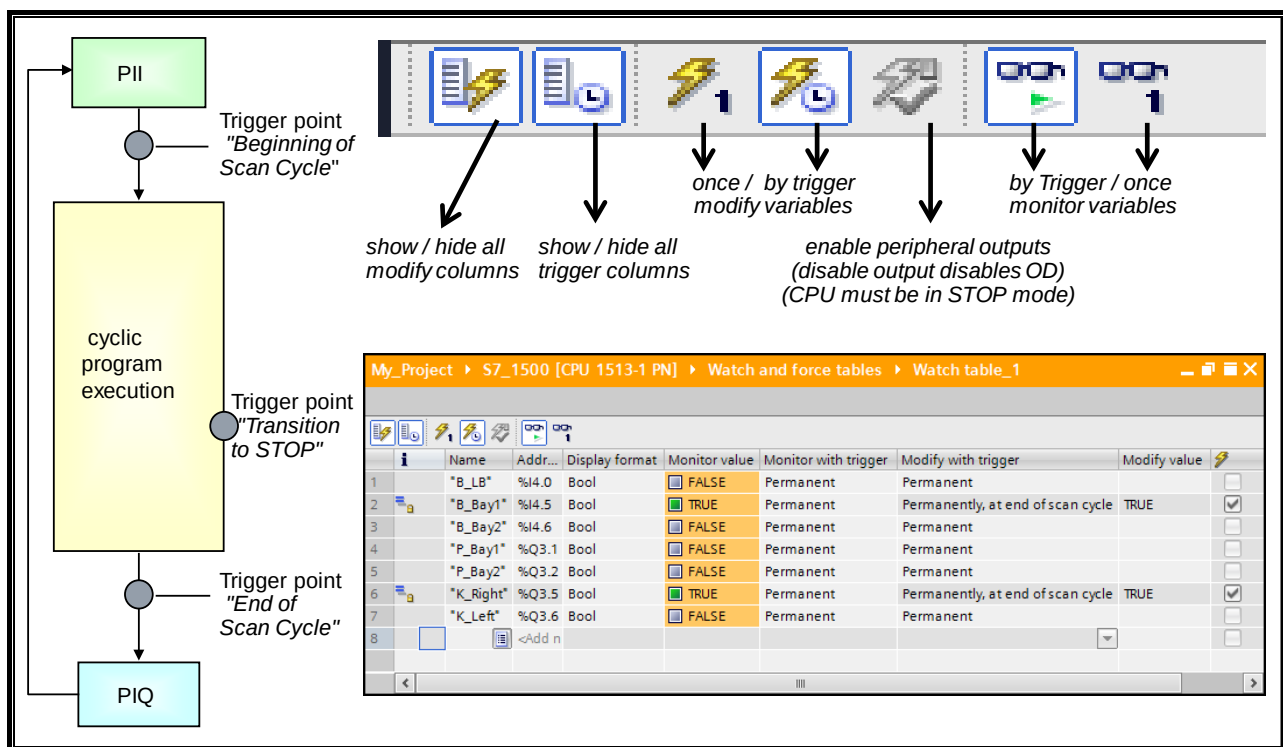
You can choose the columns displayed in the Watch table via the menu View. The columns have the following meanings:

- **Name:** symbolic name of the variable (tag)
- **Address:** absolute address of the variable (tag).
- **Symbol comment:** comment on the variable (tag) displayed
- **Display format:** a data format you can choose per mouse click (such as binary or decimal), in which the contents of the variable (tag) is displayed
- **Monitor value:** variable (tag) value in the selected status format
- **Modify value:** value to be assigned to the variable (tag)

### Watch Table

You can choose any name for the Watch table. Saved Watch tables can be reused to monitor and modify so that a renewed input of the variables to be monitored is no longer necessary.

### 14.8.1. Monitor / Modify Variables (Tags): Trigger Points



#### Trigger Points

Through the "Monitor with trigger or Modify with trigger" columns, you can define the trigger points for *monitoring* and *modifying*. The "Trigger Point for Monitoring" specifies when the values of the variables being monitored are to be updated on the screen. The "Trigger Point for Modifying" specifies when the given modify values are to be assigned to the variables being modified.

#### Trigger Condition

The "Trigger Condition for Monitoring" specifies whether the values are to be updated on the screen once only when the trigger point is reached or permanently (when the trigger point is reached).

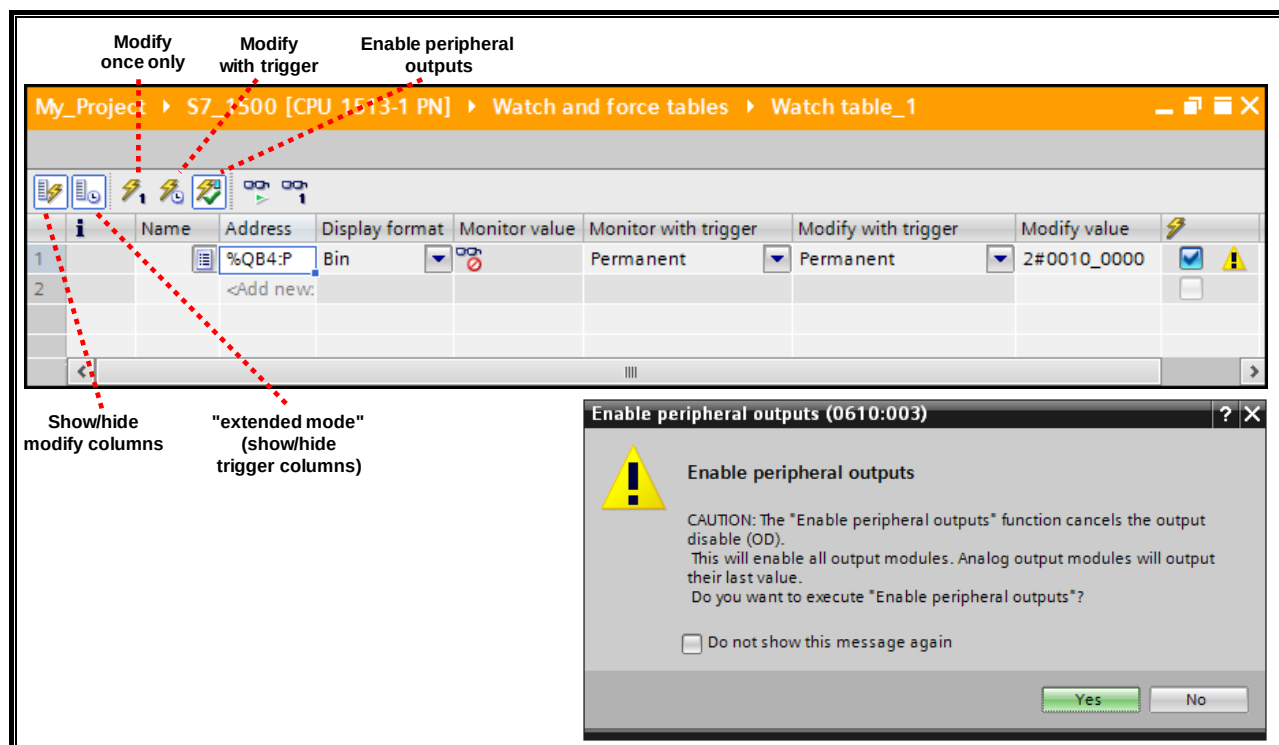
The "Trigger Condition for Modifying" specifies whether the given modify values are to be assigned to the variables being modified once only or permanently (every time the trigger point is reached).

#### Area of Use

The following tests, among others, can be implemented with the appropriate selection of trigger points and conditions:

- **Wiring test of the inputs:**  
*Monitor variables*, Trigger point: *Start of scan cycle*, Trigger condition: *Permanent*
- **Simulate input states** (user specified, independent of process):  
*Modify variables*, Trigger point: *Start of scan cycle*, Trigger condition: *Permanent*
- **Differentiation between hardware / software errors** (an actuator that should be activated in the process is not controlled)  
*Monitor variables*, in order to monitor the relevant output Trigger point: *End of scan cycle*, Trigger condition: *Permanent*  
(output state = '1' > program logic OK > process error (hardware))  
(output state = '0' > program logic error (such as double assignment))
- **Control Outputs** (independent of the program logic)  
*Modify variables*, Trigger point: *End of scan cycle*, Trigger condition: *Permanent*

## 14.8.2. Enable Peripheral Outputs (in Planning Stage)



### The Function "Enable Peripheral Outputs" (in Planning Stage)

The "Enable Peripheral Outputs" function is used to check the functioning of the output modules, the wiring of the digital output modules or it can be used to continue to control actuators in the process even though the CPU finds itself in the STOP state because of an error that has occurred.

The "Enable Peripheral Outputs" function cancels the output disable of the peripheral outputs (PQ), which enables you to control the outputs in spite of the CPU's STOP state.

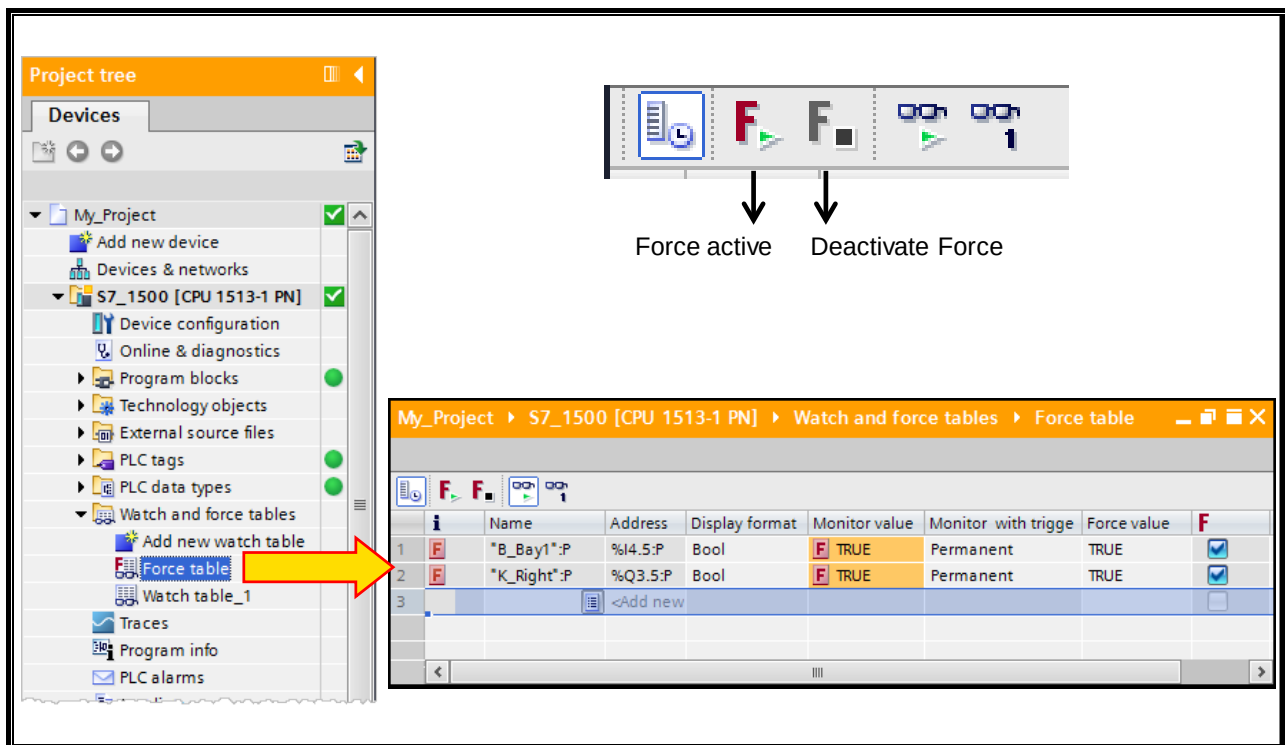
### Conditions

- The CPU must be in STOP mode
- A Force task must not be active in the CPU
- The Watch table must be displayed in "extended mode", in other words, displayed with trigger columns
- The peripheral outputs to be enabled are to be specified byte by byte, word by word or double-word by double-word with the suffix :P (for peripheral)
- After the peripheral outputs have been enabled, the modify values can be activated via the "Modify once only" button (not via "Modify with trigger").

### Note

When changing the CPU's operating status from STOP to RUN or STARTUP, *Enable Peripheral Outputs* is deactivated and a message pops up.

### 14.8.3. Force Variables (Tags)



#### Function and Area of Use

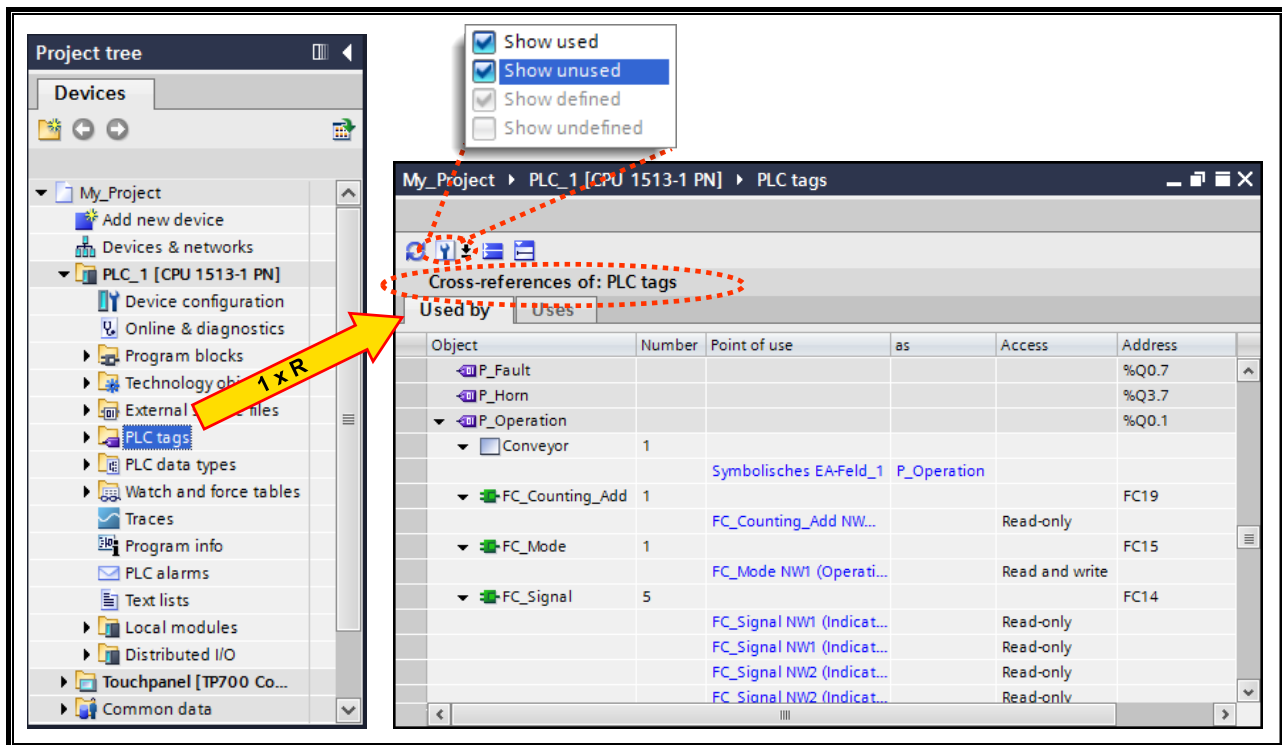
With Force, you can overwrite variables (tags) with any values you like, independent of the user program. Only one "Force Values" window can be open at a time for a CPU.

#### Notes on Forcing

Before your start the "Force" function, you should make sure that no one else is carrying out this function at the same time on the same CPU.

A force task can only be canceled through an explicit "stop forcing" [deactivating the icon] (not via *Edit* → *Undo!*). Closing the Force table or exiting the application does not cancel the force task on the CPU.

## 14.9. Reference Data: Cross-references of PLC Tags



### Introduction

The cross-references list offers an overview of the use of operands and variables (tags) within the user program. From the cross-references list, you can jump directly to the point of use.

The cross-references list contains the following information (depending on from which folder this was opened):

- Which operand and which HMI tag is used in which block with which instruction and in which HMI screen, (device and PLC tags, HMI tags)
- Which block is called by which other block (controller and program folder)

As part of the project documentation, the cross-references supply a comprehensive overview of all operands, memory areas, blocks, variables (tags) and screens used.

### Views

There are two views of the cross-references list which differentiate themselves by which objects are displayed in the first column:

- **Used by:** Displays the referenced objects  
Here, the locations (Point of use) where the object is used are displayed.
- **Used:** Displays the referencing objects.  
Here, the users of the object are displayed.  
The associated tooltips give further information on the respective objects.

### Show Unused

This is a list of tags which are declared in the PLC tag table but are not used in the S7 user program.

### 14.9.1. Reference Data: Cross-references of a Variable (Tag) in the Block Editor

The screenshot shows the SIMATIC Manager Block Editor interface. The main window displays a ladder logic network for 'Conveyor monitoring'. The network includes a timer TON (DB17) and a set coil (SR) for 'M\_Conv\_Fault' (M17.0). The output is connected to a coil for 'P\_Fault' (Q0.7). The 'Info' tab in the Properties window is selected, showing the 'Cross-references' section. The table below lists the cross-reference information for the selected object 'FC\_Fault'.

| Object           | Point of use                          | Access         | Address | Monitor value | Type |
|------------------|---------------------------------------|----------------|---------|---------------|------|
| ▼ "M_Conv_Fault" |                                       |                | %M17.0  |               | Bool |
| ▼ FC_Fault       | FC_Fault NW1 (Conveyor monitoring)    | Read and write | FC17    |               | FBD- |
| ▼ FC_Signal      | FC_Signal NW1 (Indicator light Bay 1) | Read-only      | FC14    |               | FBD- |
|                  | FC_Signal NW2 (Indicator light Bay 2) | Read-only      |         |               |      |
| ▼ "P_Fault"      |                                       |                | %Q0.7   |               | Bool |
| ▼ FC_Fault       | FC_Fault NW1 (Conveyor monitoring)    | Write          | FC17    |               | FBD- |
| ▼ DB_TON_Fault   |                                       |                | DB17    |               | Data |

#### Introduction

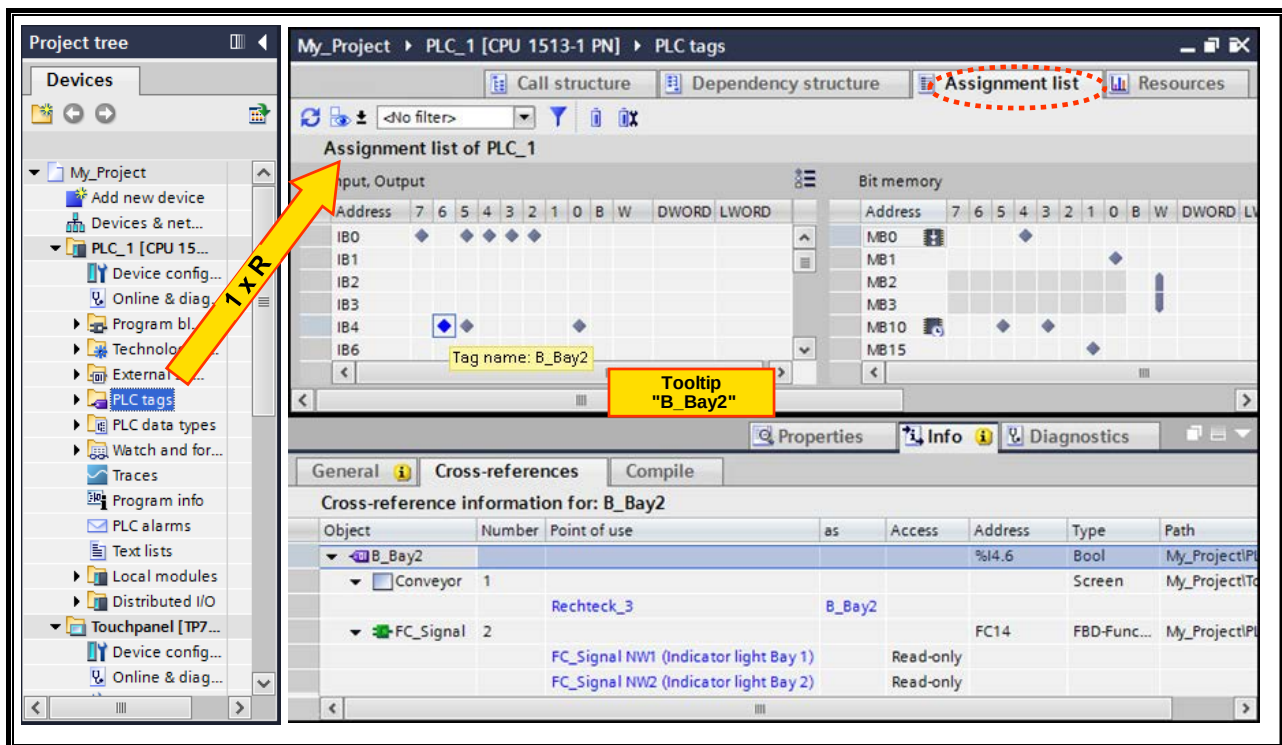
In the Inspector window, the cross-reference information for a selected object is displayed in the tabs "Info > Cross-references". In this tab, you will see at which locations (Point of use) and from which other objects every selected object is used.

In the Inspector window, cross-references are made even to those blocks which only exist online.

#### Structure

The cross-reference information is displayed in tabular form in the Inspector window. Each column contains specific detailed information on the selected object and its use.

### 14.9.2. Reference Data: Assignment of I, Q, M, T, C



#### Assignment of I/Q/M/T/C

You get to the assignment list for I/Q/M/T/C via "Right-click on PLC tags -> Assignment list".

This assignment list gives you an overview of which bit is used from which byte of the memory areas input (I), output (Q) and bit memory (M) and which SIMATIC timers and counters are used. The type of use (reading or writing) is not displayed.

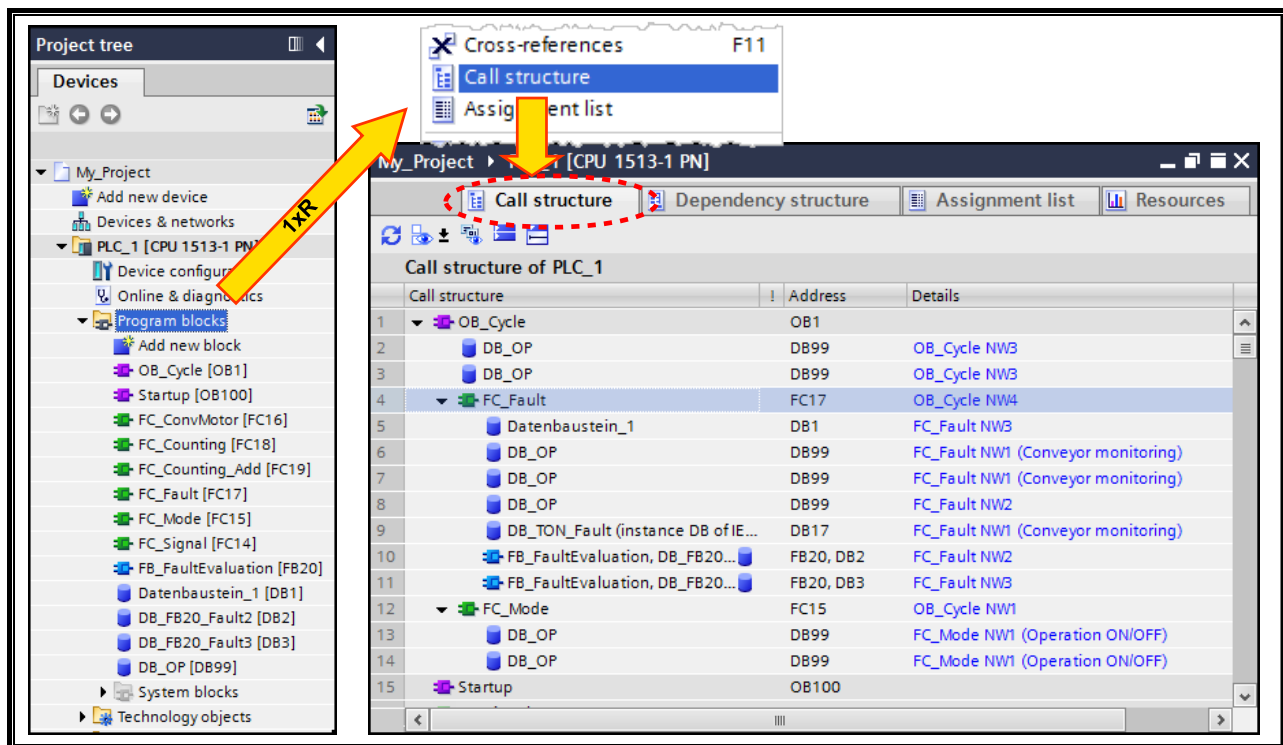
The memory areas inputs (I), outputs (Q) and bit memories (M) are displayed byte-by-byte in lines.

The bits identified with a small diamond or binary operands (in the picture, for example, I 1.0 or M 17.0) are used explicitly in the program.

The bits with a gray background (squares) together identify byte, word or double-word operands (in the picture, for example, MW20) that are used in the user program. The operand dimension (byte, word or double-word) comes from the vertical line in one of the columns "B" (Byte), "W" (Word) and "D" (Double word).

Bits that have both a small diamond and a gray background are used explicitly as a binary operand in the user program and are used through a byte, word, or double-word operand.

### 14.9.3. Reference Data: Call Structure



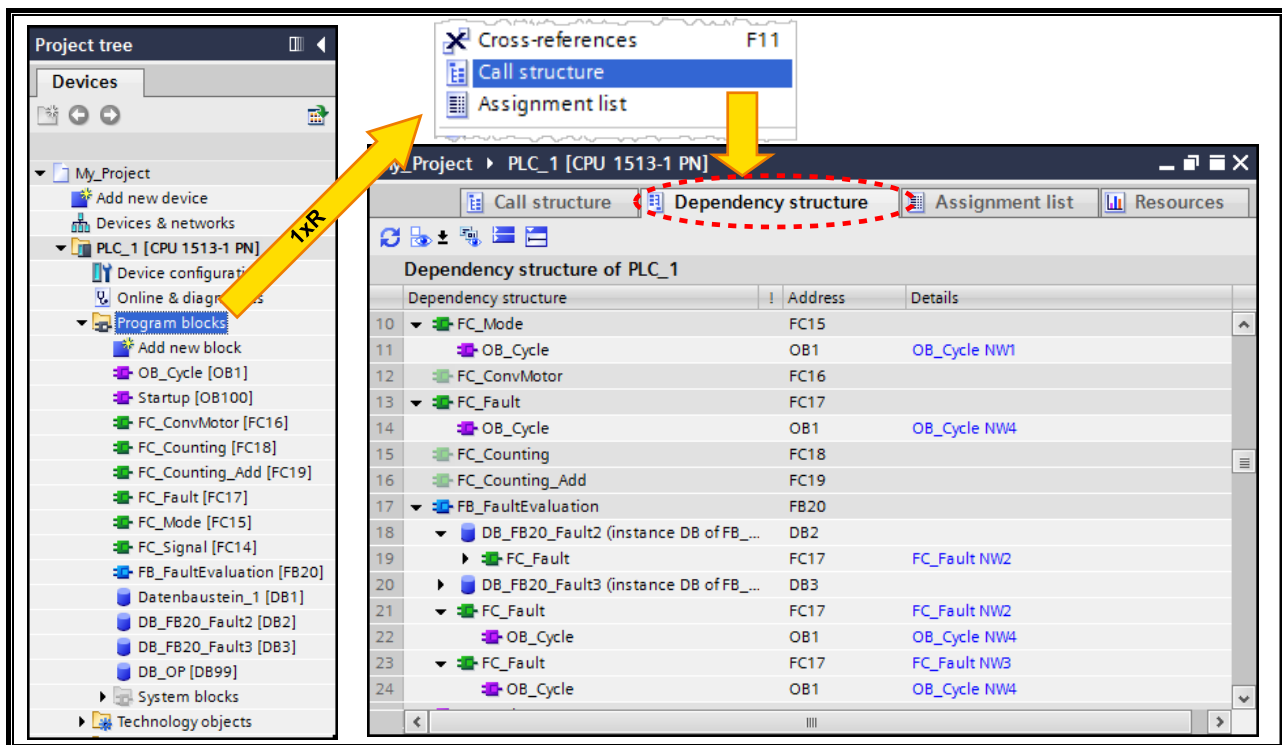
#### Call structure

The call structure shows (describes) the call hierarchy of the blocks within an S7 program. It gives you an overview of:

- The blocks used
- Jumps to the points of use of the blocks
- Dependencies between the blocks
- Local data requirements of the blocks
- Status of the blocks



#### 14.9.4. Reference Data: Dependency Structure



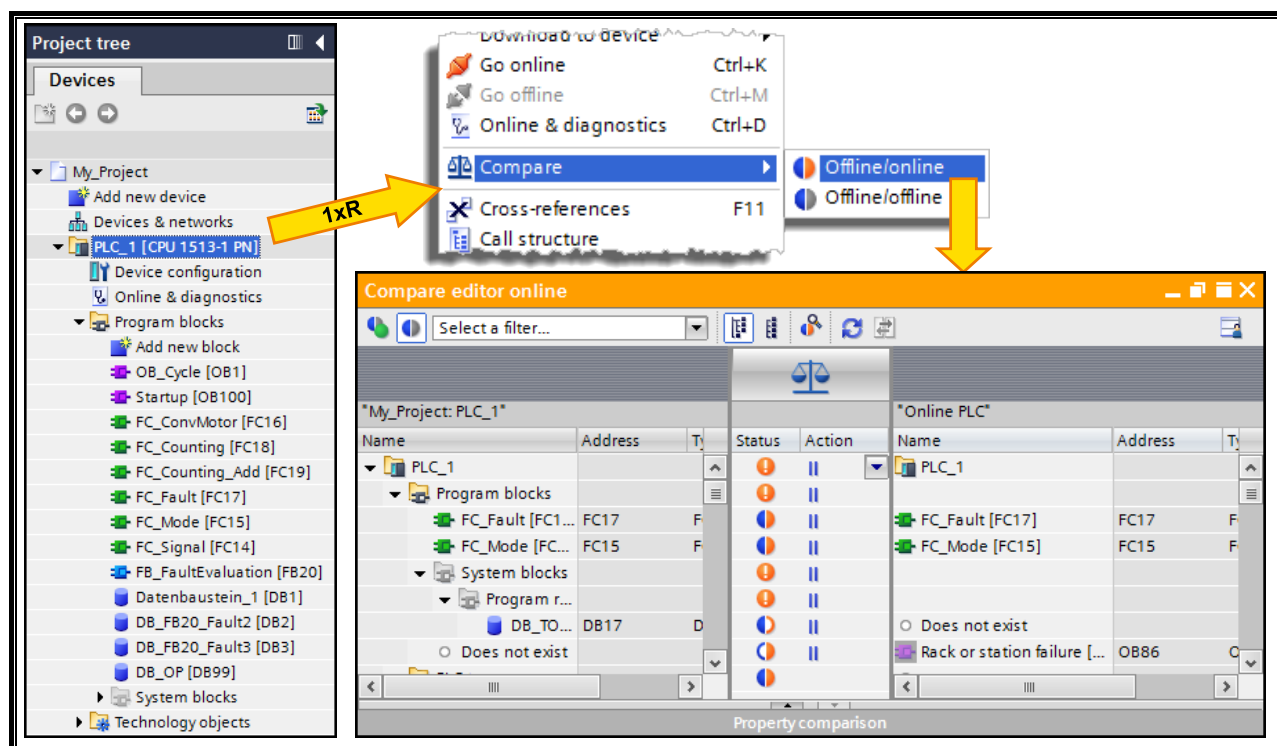
#### Dependency Structure

The dependency structure shows the list of blocks used in the user program. In the first level (to the very left) is the respective block and indented underneath it are the blocks which call this block or use it.

The dependency structure also shows the status of the individual blocks through the use of symbols. Objects which cause a time stamp conflict and which can lead to an inconsistency in the program are identified with different symbols.

The dependency structure represents an extension of the cross-references list for objects.

## 14.10. Comparing Blocks



### Types of Comparison

In principle, there are two different types of comparison:

Online/Offline comparison:

- The objects in the project are compared with the objects of the relevant device. For this, an online connection to the device is necessary.

Offline/Offline comparison:

- Either the objects of two devices within a project, from different projects or a device and a library are compared.

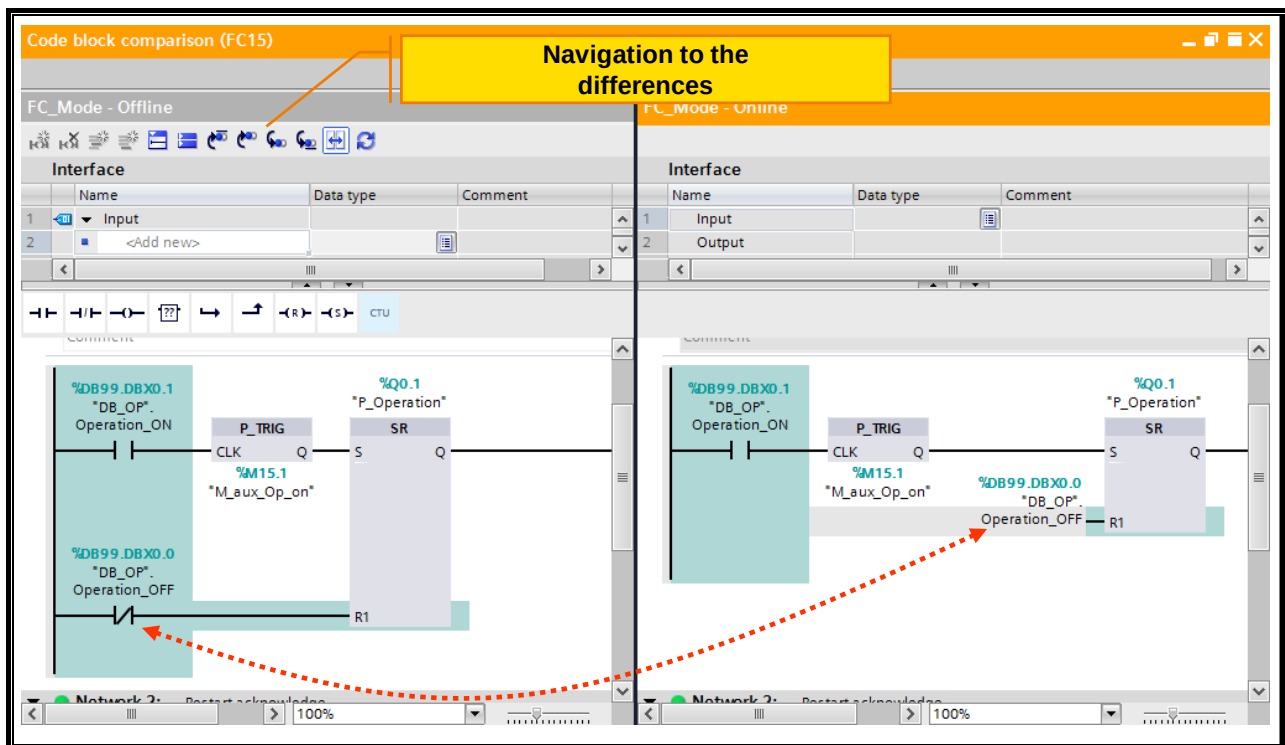
### Symbols of the Results Display

The result of the comparison is presented by means of symbols.

The following table shows the symbols for the comparison results of an Online/Offline comparison:

| Symbol | Meaning                                                                 |
|--------|-------------------------------------------------------------------------|
| !      | Folder contains objects whose online and offline versions are different |
| ?      | Comparison result is unknown                                            |
| ●      | Online and offline versions of the object are identical                 |
| ⦿      | Online and offline versions of the object are different                 |
| ⦿      | Object only exists offline                                              |
| ⦿      | Object only exists online                                               |

### 14.10.1. Comparing Blocks: Show Details

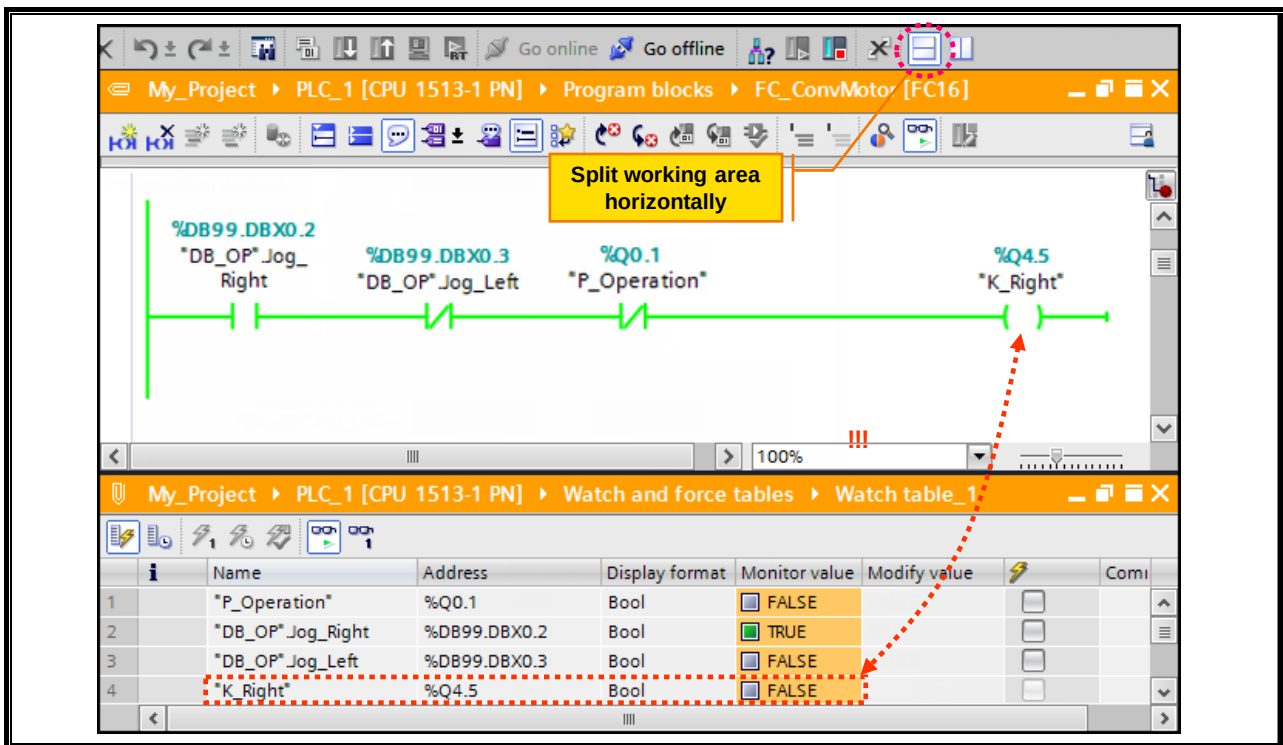


#### Detailed Comparison

Through the detailed comparison you can identify exactly those locations that are different in the online and offline version of a block. The following identifiers are used:

- Lines in which there are differences are highlighted in grey.
- Different operands and operations are highlighted in green.
- When the number of networks is different, pseudo networks are inserted so that a synchronized representation of identical networks is possible. These pseudo networks are highlighted in grey and contain the text "No corresponding network was found" in the title-bar of the network. Pseudo networks cannot be processed.
- If the sequence of the networks is mixed up, pseudo networks are inserted at the appropriate locations. These pseudo networks are highlighted in grey and contain the text "The networks are not synchronized" in the title-bar of the network. The pseudo network also contains a link "Go to network <No>", through which you can navigate to the associated network.

## 14.11. Exercise 4: Testing the Motor Jog



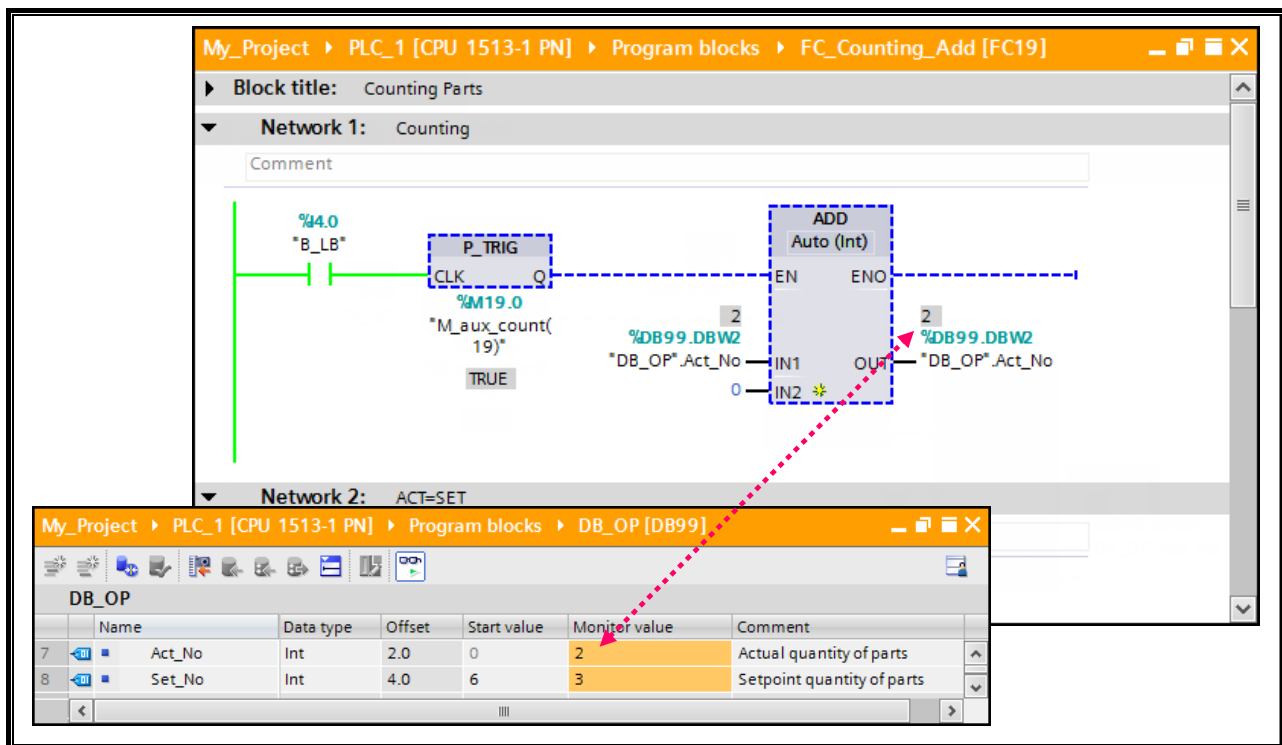
### Task

The function "Jog conveyor motor" does not work. The combined use of the PG functions "Monitor block" and "Watch table" (monitor tags [variables]) indicates that there must be a double assignment at output "K\_Right" (Q4.5). The task now is to find all instructions in the entire user program that write-access this output.

### What to Do:

1. Carry out a CPU restart.
2. On the touchpanel, switch off the "Plant".
3. Open the "FC\_ConvMotor" (FC 16) block and activate the "Monitor" test function.
4. In the Project tree, under "Watch and force tables" create a new Watch table and in it monitor the output "K\_Right" (Q4.5).
5. Display the Blocks Editor with the opened "FC\_ConvMotor" (FC16) and the Watch table one below the other by splitting the working area (see picture).
6. Interpret the different status displays of the two test functions.
7. Localize the double assignment at output "K\_Right" (Q4.5) with the help of the reference data and correct the error.
8. Download all modified blocks into the CPU and check the how the program functions.

### 14.11.1. Exercise 5: Testing 'Counting' the Parts



#### Task

In the search for this logical error, you are to use, among others, the test function "Monitor data block" and the test function "Monitor block".

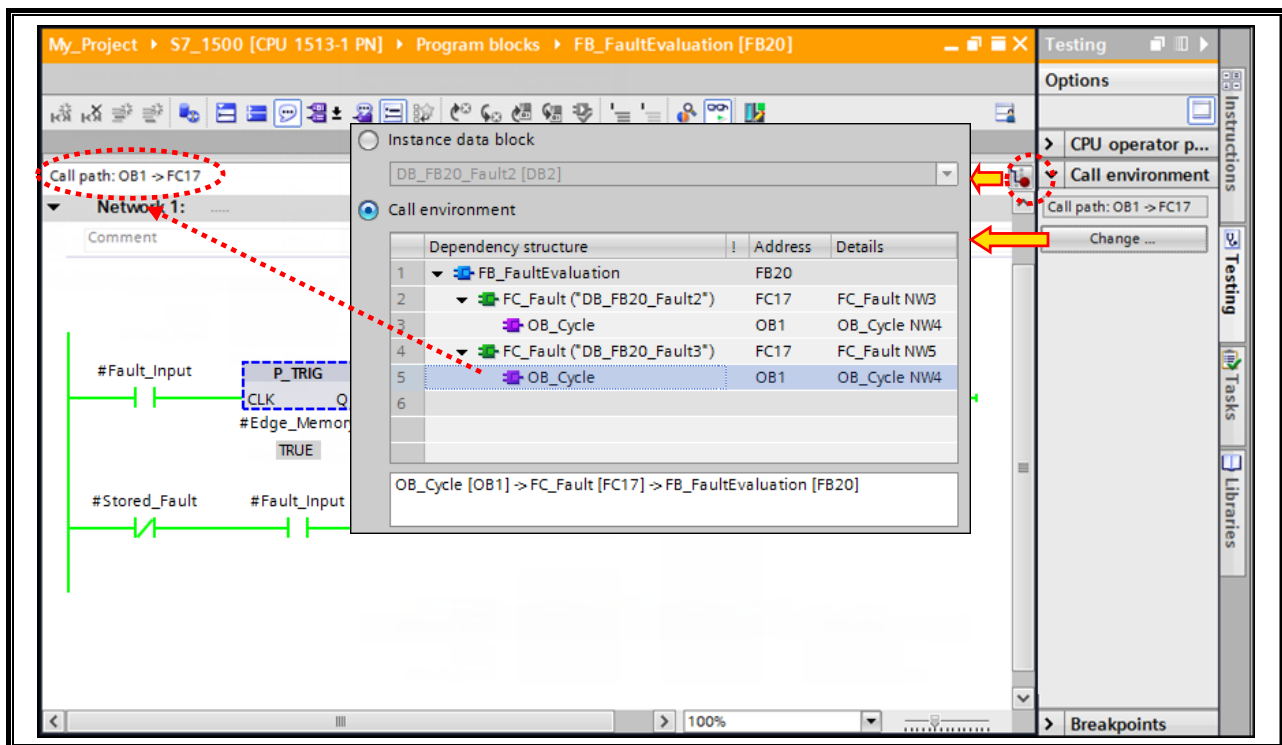
#### Function Test

Counting and displaying the quantity of transported parts is programmed in "FC\_Counting\_Add" (FC19) which determines the current quantity in the variable "DB\_OP".Act\_No (DB99.DBW2) by adding up. Check whether the number of transported parts is correctly recorded and displayed on the touchpanel.

#### What to Do:

1. First of all open "DB\_OP" (DB99) and monitor the contents of the variable "DB\_OP".Act\_No (DBW2).
2. It shows that "FC\_Counting\_Add" (FC19) does not store the correct quantity in the variable "DB\_Parts".Act\_No. For that reason, also open "FC\_Counting\_Add" (FC19) and monitor it with "Monitor block".
3. Correct "FC\_Counting\_Add" (FC19) and once again test the counting of the transported parts.

### 14.11.2. Exercise 6: Testing the Evaluation of Fault 3



#### Task

In the search for the last logical error, you are to use the test function 'Monitor block with trigger' on a certain block call.

Check whether the occurrence of *Fault 3* (switch I 0.6 on the simulator) is displayed on the simulator LED "P\_Fault3" (Q0.4) with a flashing light, and whether, after acknowledging with the simulator pushbutton "S\_Acknowledge" (I 0.7), the flashing light switches to continuous light.

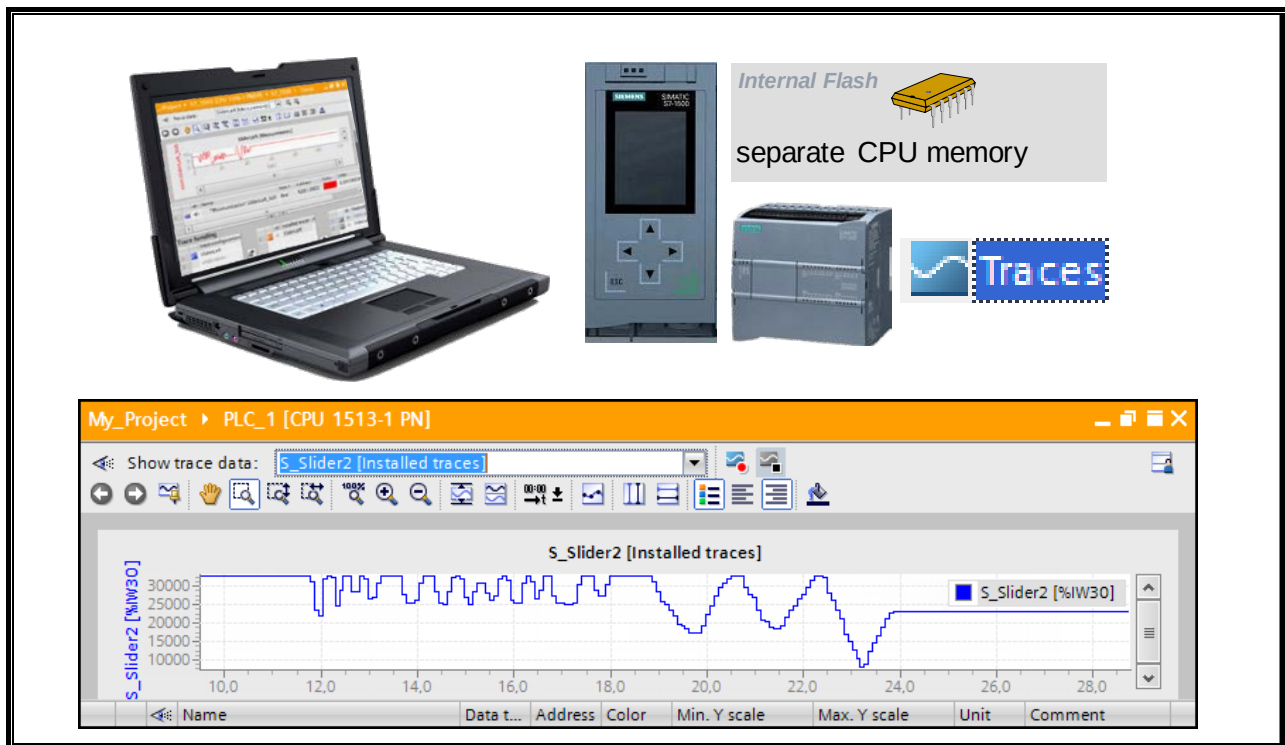
#### What to Do

1. To troubleshoot why no flashing light is displayed after *Fault 3* occurs, first of all monitor "FB\_FaultEvaluation" (FB20) with the test function "Monitor block".

You will see that the output parameter #Display is flashing, however not the simulator LED "P\_Fault3" (Q0.4). The cause of this lies in the fact that you are monitoring the execution of "FB\_FaultEvaluation" (FB20) for the evaluation of Fault 2 or the first FB call in "FC\_Fault" (FC17).

2. Even if you already know which correction is to be made:  
Change the call environment as shown in the picture in such a way that you specifically monitor the execution of "FB\_FaultEvaluation" (FB20) for the evaluation of Fault 3.
3. Correct the error and once again monitor the evaluation of Fault 3.
4. Download all modified blocks into the CPU and check the function.
5. Save your project.

## 14.12. TRACE Analyzer Function



### "Trace" Analyzer Function

The value-over-time of one or several CPU tags (max. 16) can be stored in a TRACE. In STEP7, a TRACE recording can be presented graphically.

- Recording start
  - manual (start from STEP7)
  - configurable (triggered by PLC tag; value or value exceeded)
- Max. 16 PLC tags per trace
- Look at the trace as a graph in STEP7 and, if necessary, save in the project

Trace size and number depends on the CPU.

In the S7-1500 CPU, a maximum of eight internal TRACE memories with 512 Kbyte each are available.



#### Caution!

A trace recording increases the runtime of the cyclic program execution which can possibly also lead to an exceeding of the cycle time.

### 14.12.1. TRACES Editor

The screenshot shows the SIMATIC TIA Portal Traces Editor interface, divided into three main sections:

- Trace Display:** The top section shows a graph of the trace data for 'S\_Slider2 [%IW30]'. The graph displays a fluctuating signal over time. A red arrow points to the 'Trace selection' button in the top toolbar.
- Trace Handling:** The middle section contains a table of configured traces and buttons for managing them. A red arrow points to the 'Configure' button, and another red arrow points to the 'Download in CPU' button. A third red arrow points to the 'Save in project' button.
- Trace Configuration:** The bottom section shows the 'Trace [Trace configuration]' dialog box, which includes tabs for 'General', 'Properties', 'Info', and 'Diagnostics'. The 'General' tab is active, showing a table of signals to be traced.

| Trace [Trace configuration] |             |           |           |       |         |
|-----------------------------|-------------|-----------|-----------|-------|---------|
| General                     |             |           |           |       |         |
| Signals                     |             |           |           |       |         |
|                             | Name        | Data type | Address   | Color | Comment |
| 1                           | *S_Slider2* | Word      | %IW30     | Blue  |         |
| 2                           |             |           | <Add new> |       |         |

#### Traces Editor

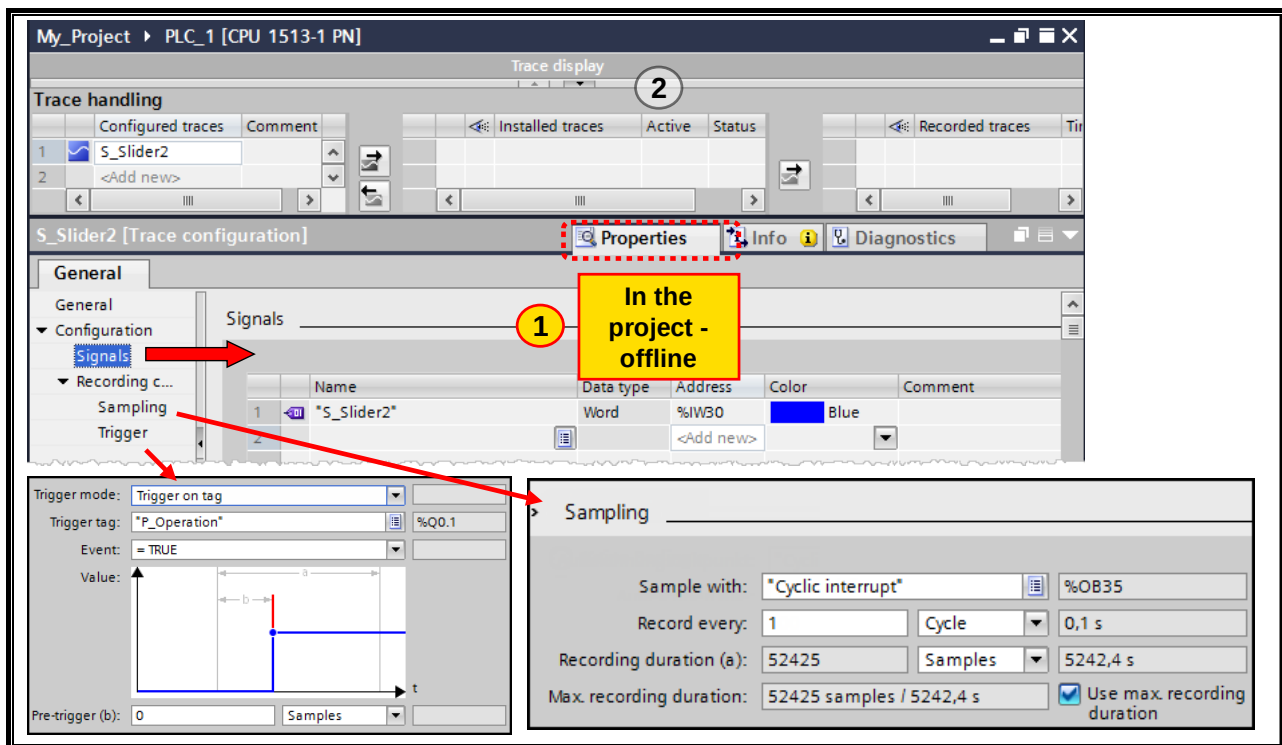
The Traces editor is divided into three sections.

- Selection and Diagram area (upper part of the trace display)
- Signal table (lower part of the trace display)
- Trace handling

The trace configuration is made via the "Properties" tab in the Inspector window.



### 14.12.2. Configuring a TRACE



#### Trace Signals

All elementary data types in the I/O (process image), M, DB

#### Pre-trigger

In order to already record a period of time before the trigger event, a value greater than 0 must be entered in the input field for the pre-trigger.

#### Sampling

Sampling is clocked via an associated cyclic or time-controlled OB.

→ here in the example, OB35 every 100ms

#### Retentiveness

Trace configurations in the device are retained with POWER-OFF; however, recorded values are lost with POWER-OFF.

### 14.12.3. Downloading TRACE into the CPU and Activating It

The screenshot illustrates the steps to download and activate TRACE in the SIMATIC Manager. The 'Trace handling' window shows the 'Installed traces' table with 'S\_Slider2' being downloaded. A yellow box labeled 'In CPU - online' indicates the connection status. A yellow box labeled 'Status then depends on configured trigger mode' points to the 'Status' column. A 'Tracing' dialog box (0604:000424) asks 'Do you want to enable tracing?' with a warning that tracing may increase cycle time. The 'Status' column shows the trace changing from 'Waiting' to 'Recording' and finally to 'Recording compl.'.

then activation necessary

when trigger condition is fulfilled or "Record immediately" and TRACE is not yet full

TRACE full

#### Requirements

- At least one trace is configured
- STEP7 is connected online with the CPU

#### Recording Start

- Trigger mode: Record immediately  
⇒ recording starts immediately after activation
- Trigger mode: Trigger on tag  
⇒ recording starts after activation only if trigger condition is fulfilled



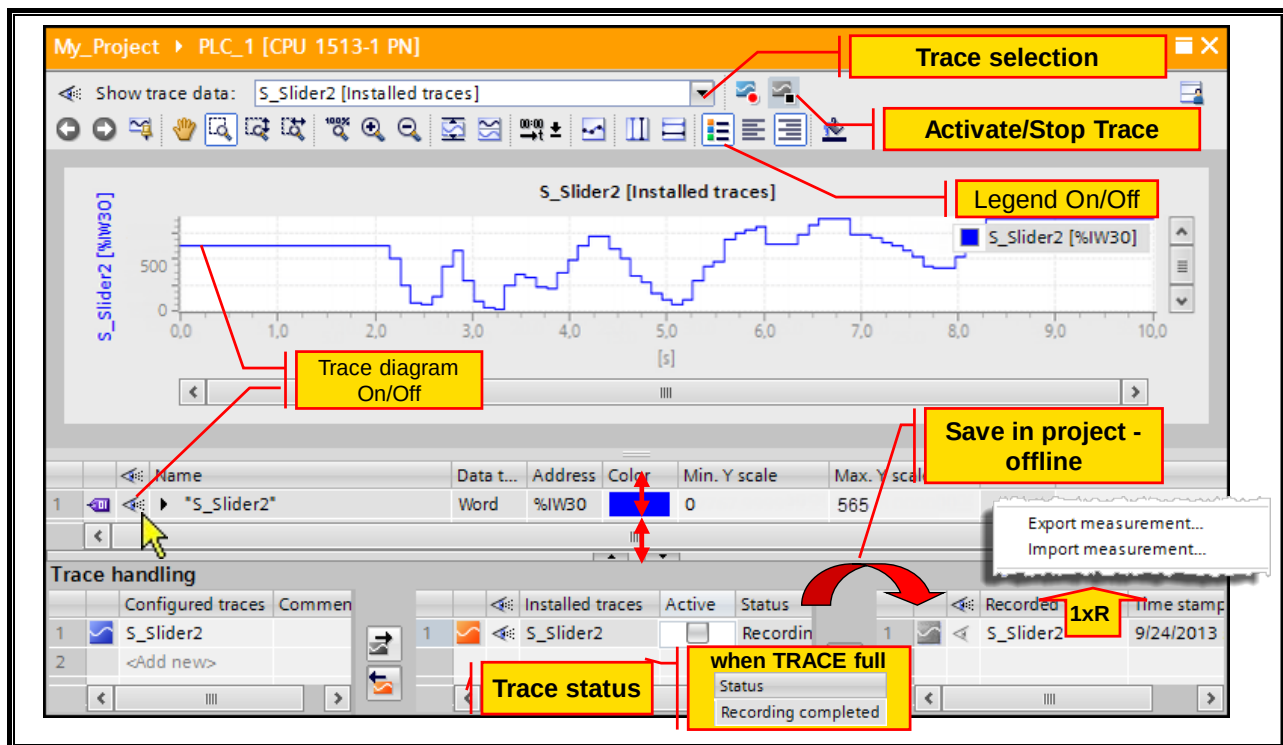
#### Caution!

If a trace is activated once again, the previous recording is overwritten.

#### Recording Duration

Trace memory is full, that is, configured number of measuring points is recorded.

#### 14.12.4. Looking at / Saving TRACE in STEP7



#### Looking at Trace

If a trace is selected, the real-time value is displayed in the Diagram area even if the trace is not yet started.

While the trace is recording, the diagram is updated.

If the trace is full, the recording stops automatically and the Diagram area displays the selected trace data.

If several tags are logged in one trace, the diagrams can be shown or hidden by clicking .



#### Caution!

If a trace is started / activated once again, the previous recording is overwritten.

The toolbar     contains various zoom functions for presenting the diagram of the displayed trace.

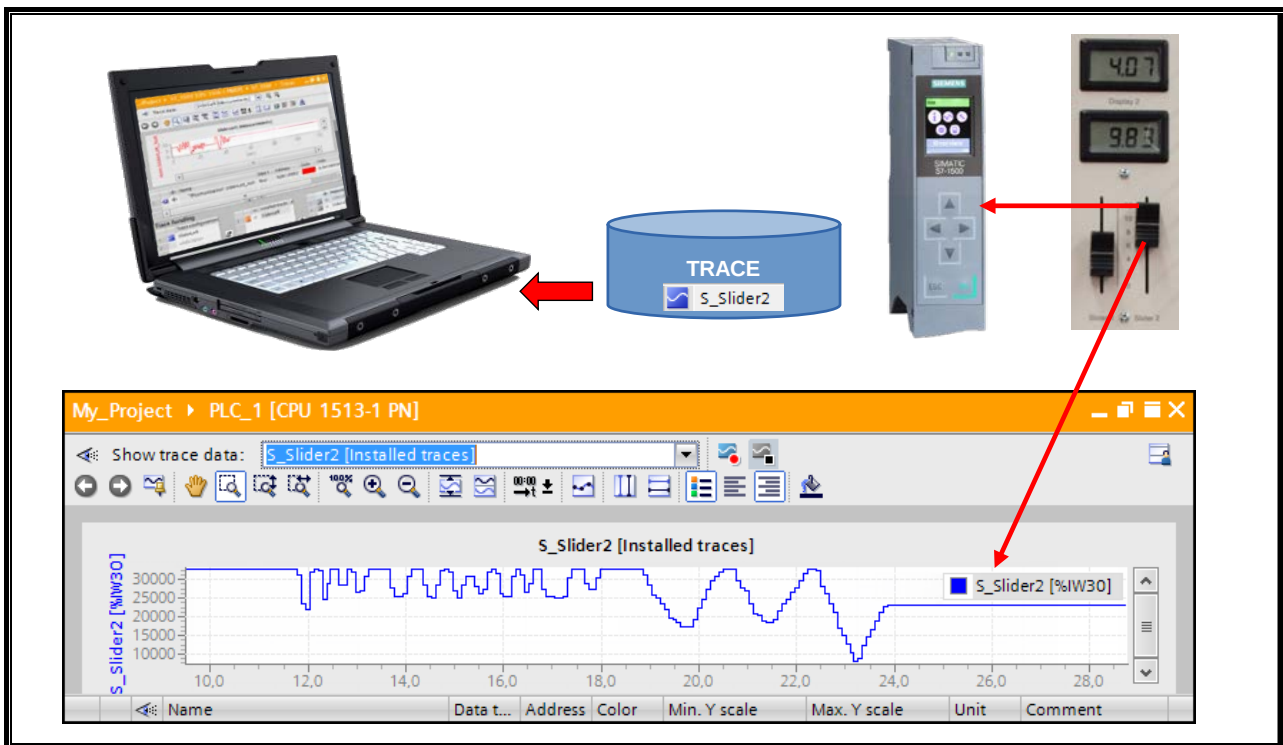
#### Saving Trace in the Project

Through the  icon in the Trace handling, traces saved in the CPU can be loaded into the offline project. A trace can only be saved in the project when it is full, or, if the recording was stopped.

#### Exporting a Measurement

To exchange measurements between projects, a trace (as \*.ttrec) can be exported and imported. For further processing, the CSV format can also be selected during export.

## 14.13. Exercise 7: Creating, Looking At and Saving TRACE



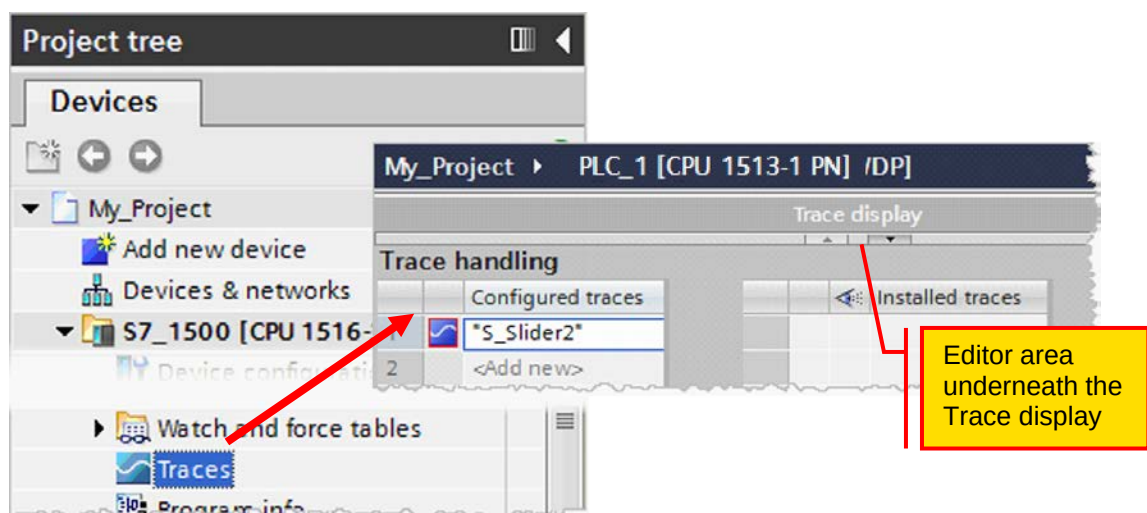
### Task

A weighing scale that is to weigh the workpieces is simulated with S\_Slider 2. The values are written into input word 30. The recording is to start when "P\_Operation" (Q0.1) is switched on and is then to be saved in the project.

### What to Do

#### 1. Configure a trace

Open the Traces editor and add a trace:



For now, you can close the upper editor area (Trace display) as it is not required for the configuration of the trace.

The signal of "S\_Slider2" (IW30) is to be recorded.

| Signals |             |           |         |       |         |
|---------|-------------|-----------|---------|-------|---------|
|         | Name        | Data type | Address | Color | Comment |
| 1       | "S_Slider2" | Word      | %IW30   | Blue  |         |

The recording is to take place every 100th OB1 cycle for the maximum possible recording duration.

> Sampling

Sample with: "OB\_Cycle" %OB1

Record every: 100 Cycle

Recording duration (a): 52425 Samples

Max. recording duration: 52425 samples ☒ Use max. recording duration

The recording is to start when "P\_Operation" (Q0.1) is switched on:

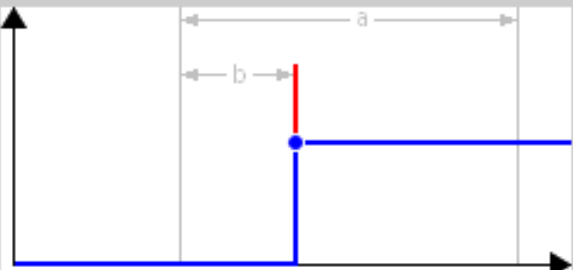
> Trigger

Trigger mode: Trigger on tag

Trigger tag: "P\_Operation" %Q0.1

Event: = TRUE

Value:



Pre-trigger (b): 0 Samples

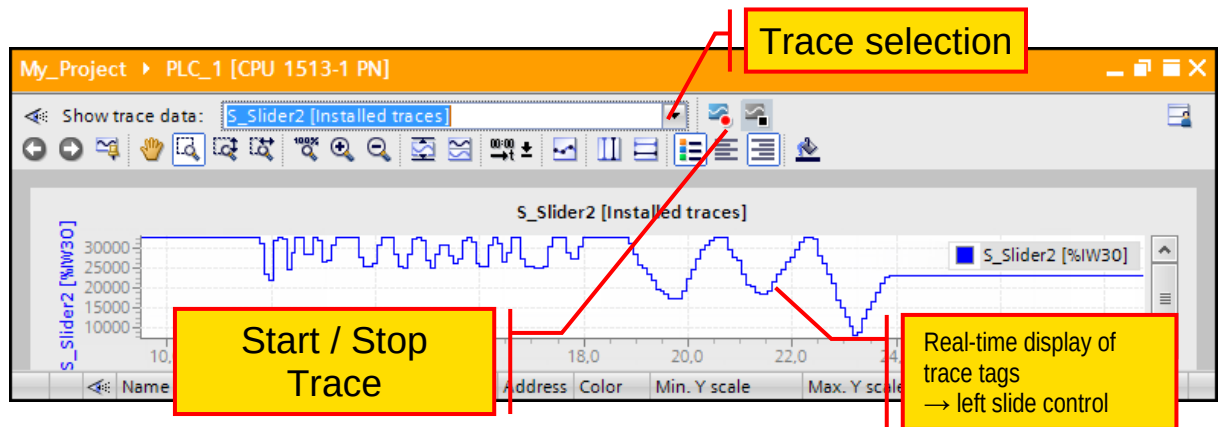
## 2. Transfer the trace to the CPU and activate it

'Go online' with the CPU, download the trace into the CPU and activate it:

| Trace handling |                                                                                               |         |                                                                                     |                                                                                                |                                     |           |
|----------------|-----------------------------------------------------------------------------------------------|---------|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------|-----------|
|                | Configured traces                                                                             | Comment |                                                                                     | Installed traces                                                                               | Active                              | Status    |
| 1              |  S_Slider2 |         |  |  S_Slider2 | <input checked="" type="checkbox"/> | Recording |
| 2              | <Add new>                                                                                     |         |  |                                                                                                |                                     |           |

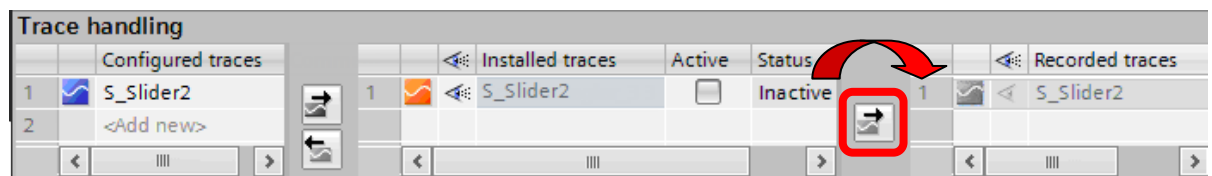
### 3. Start the recording and monitor the trace

On the touchpanel, switch on "P\_Operation" (Q0.1) and monitor your trace. Move the slide control on the operator panel and monitor the value changes in the trace.



### 4. Stop the recording and save the trace

Stop the recording (see picture above) and save the trace in the offline project.



### 5. Save your project.

# Contents

|                                                                                          |             |
|------------------------------------------------------------------------------------------|-------------|
| <b>15. Integrating and Commissioning a Drive with Startdrive .....</b>                   | <b>15-2</b> |
| Task Description: G120 as an Additional Conveyor Drive .....                             | 15-3        |
| 15.1. Communication Standard PROFIdrive .....                                            | 15-4        |
| 15.1.1. CPU - Drive Communication: CPU - G120 .....                                      | 15-5        |
| 15.1.2. Standard Telegrams .....                                                         | 15-6        |
| 15.1.3. Structure of the Control Word (CDS0 - ControlDataSet for Fieldbus Control) ..... | 15-7        |
| 15.1.4. Structure of the Status Word.....                                                | 15-8        |
| 15.1.5. Setpoint / Actual Value → Speed Values .....                                     | 15-9        |
| 15.2. Inserting a Drive into the Project.....                                            | 15-10       |
| 15.2.1. Networking a Drive.....                                                          | 15-11       |
| 15.2.2. Parameterizing the Module Address and Name .....                                 | 15-12       |
| 15.2.3. Configuring a Power Unit .....                                                   | 15-13       |
| 15.2.4. Parameterizing the Process Data Area (PZD).....                                  | 15-14       |
| 15.2.5. Assigning the Device Name ONLINE (Module Initialization) .....                   | 15-15       |
| 15.3. Parameterizing the Drive: with the "Commissioning Wizard" .....                    | 15-16       |
| 15.3.1. Exercise 1: Reading-out the Firmware Version of the Drive.....                   | 15-17       |
| 15.3.2. Exercise 2: Restoring the Factory Settings.....                                  | 15-18       |
| 15.3.3. Exercise 3: Inserting and Networking the Drive in the Offline Project .....      | 15-19       |
| 15.3.4. Exercise 4: Configuring and Parameterizing the Drive .....                       | 15-20       |
| 15.3.5. Exercise 5: Assigning the PROFINET Device Name ONLINE .....                      | 15-21       |
| 15.3.6. Exercise 6: Parameterizing the Drive OFFLINE with the Commissioning Wizard.....  | 15-22       |
| 15.3.7. Exercise 7: Downloading the Parameterization into the Drive.....                 | 15-27       |
| 15.3.8. Exercise 8: Commissioning the Program Expansion.....                             | 15-29       |
| 15.4. Additional Information .....                                                       | 15-30       |
| 15.4.1. Changing Parameters in the Inverter.....                                         | 15-31       |
| 15.4.2. Activating / Deactivating the Control Panel.....                                 | 15-32       |
| 15.4.3. Operating the Control Panel .....                                                | 15-33       |
| 15.4.4. Monitoring Control and Status Word Online .....                                  | 15-34       |
| 15.4.5. Monitoring Active Messages Online .....                                          | 15-34       |
| 15.4.6. G120 Reset to Factory Settings via BOP-2 .....                                   | 15-35       |

## 15. Integrating and Commissioning a Drive with Startdrive

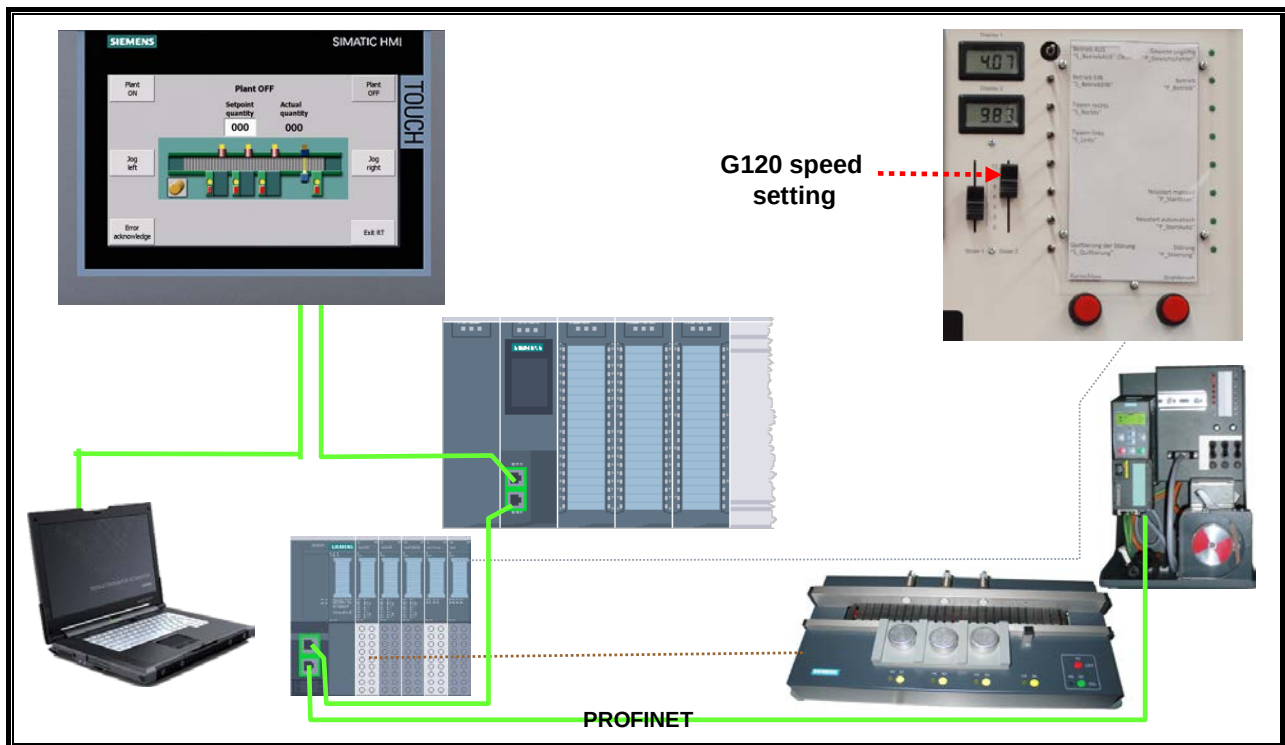
**At the end of the chapter the participant will ...**



- ... be able to parameterize and test a drive with Startdrive
- ... be able to integrate a drive in the device configuration
- ... be able to reset (restore) the inverter to factory settings
- ... be able to set basic parameters via Startdrive
- ... be able to control the drive via a PLC



## Task Description: G120 as an Additional Conveyor Drive



### Task Description

Imagine that the drive is being used as an additional drive for the conveyor model.

The drive is to be controlled as follows:

- When "P\_Operation" (Q0.1) is switched on (activated operation), the G120 drive is automatically also switched on and off parallel to the conveyor motor.
- The speed of the G120 can be set via the right slide control (potentiometer) on the operator panel.

## 15.1. Communication Standard PROFIdrive



- Consistent industrial communication
- Comprehensive range of applications
- PROFIdrive is the standard profile for drives technology in conjunction with the PROFIBUS and PROFINET communication systems
- Proven method for the easy and integrated connection of drives and controllers of different manufacturers

### Communication via PROFIBUS and PROFINET

PROFIBUS and PROFINET are the solutions which provide absolute consistency and are highly application-oriented. The main reason that PROFIBUS and PROFINET stand out from other industrial communication systems is because they span such an extraordinary breadth of applications.

One of the most important applications in industrial automation is drives technology.

Open technologies need, for their maintenance, ongoing development and market penetration, a company-independent institution that can serve as a working platform. For this purpose, the PROFIBUS Nutzerorganisation e.V. (PNO) [PROFIBUS User Organization] was founded in 1989 as a non-profit organization representing the interests of manufacturers, users and institutes (world's largest interest association in the field of industrial communication).

PNO is a member of the international umbrella organization PI (PROFIBUS & PROFINET International) founded in 1995.

### PROFIdrive

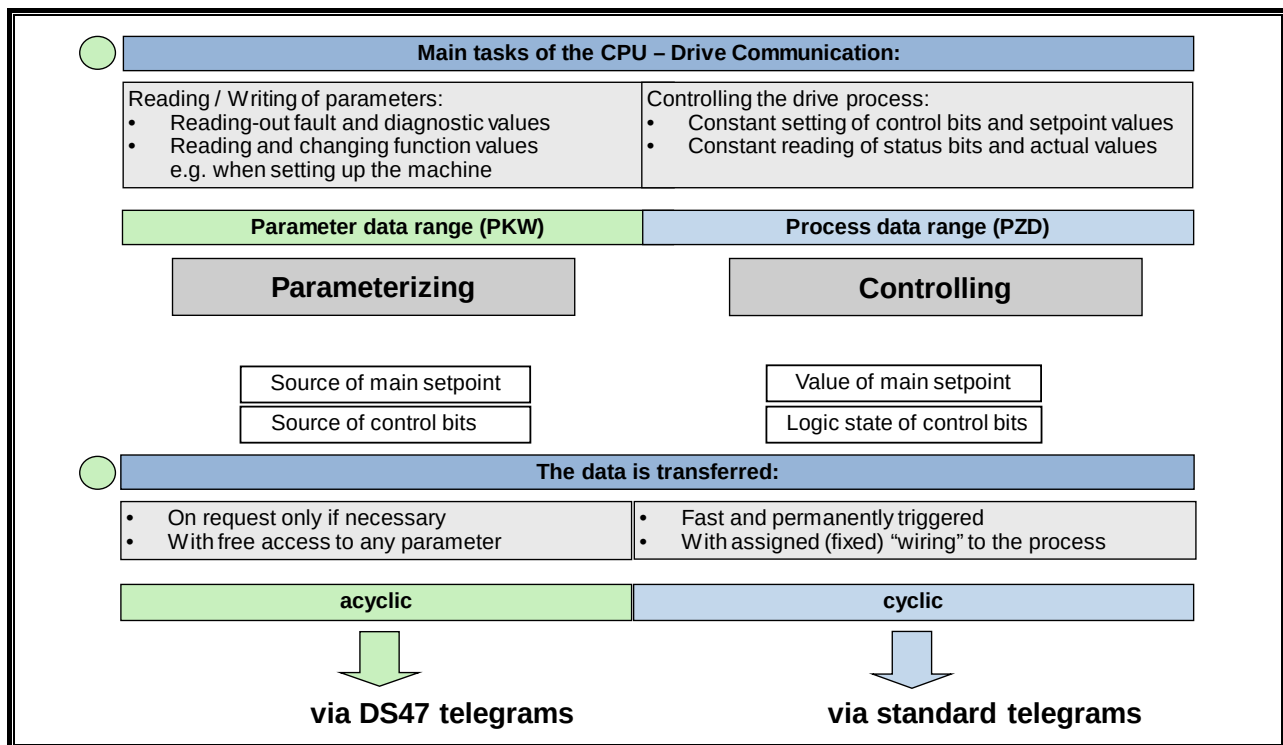
PROFIdrive is the standard profile for drives technology in conjunction with the communication systems PROFIBUS and PROFINET.

→ open "application profile" for connecting drives and controllers of different manufacturers via communication systems



The control word and status word of the G120 comply with the specifications of PROFIdrive profile Version 2.0 or Version 3.0 for the "speed control" operating mode.

### 15.1.1. CPU - Drive Communication: CPU - G120



#### Process Control

Fast data transmission of short data telegrams to all participating stations with the maximum speed available on the bus whereby all participants can receive different data. In drive systems these are typically setpoints, control commands, status replies and actual values (measured values).

#### Operating Control

In addition to this permanently available data, there is data that is only needed in particular cases. It would therefore be senseless to permanently put load on the bus if this data is only needed once per hour or per day, for example, when starting up the machine (Class 1 Master). Another reason for an expanded communication need could be the commissioning, optimization or diagnosis of machine components from a central location. In this case, (such as a fault) a detailed access of an Engineering Tool to the system components or the drive is enabled. Since, as a rule, only one affected device/component is directly addressed, this window is only made available once per bus cycle for one bus station and not simultaneously for every station.

#### Cyclic Data Exchange:

This is used to fulfill the demand for fast, permanent data exchange.

#### Acyclic Data Exchange:

The acyclic exchange of entire data sets (for example, DS47, DS100) between master and slave occurs according to standardized procedures which are implemented with FBs from the libraries (for example, Drive ES SIMATIC)..

The initiation of a data transmission for the acyclic communication of a Class 1 Master with a slave is always triggered by the user program. That is, the user program decides whether a data exchange with a slave is necessary or not based on further conditions. Only when there is a need, does a data transmission request get signaled to the DP master which then executes it.

### 15.1.2. Standard Telegrams

| Telegram | 1                                                                                 |                                                                                   | 20      |           | 352     |            | 354     |                                                                                                    | 999                                                                                                 |                                                                                                     |
|----------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------|-----------|---------|------------|---------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| PZD1     | STW1                                                                              | ZSW1                                                                              | STW1    | ZSW1      | STW1    | ZSW1       | STW1    | ZSW1                                                                                               | STW1 <4>                                                                                            | ZSW1 <4>                                                                                            |
| PZD2     | NSOLL_A                                                                           | NIST_A                                                                            | NSOLL_A | NIST_A_GL | NSOLL_A | NIST_A_GL  | NSOLL_A | NIST_A_GL                                                                                          | Receive telegram length freely<br>selectable via central PROFIdrive<br>configuration in the master  | Transmit telegram length freely<br>selectable via central PROFIdrive<br>configuration in the master |
| PZD3     |  |  |         | IAIST_GL  | <3>     | IAIST_GL   | <3>     | IAIST_GL                                                                                           |                                                                                                     |                                                                                                     |
| PZD4     |                                                                                   |                                                                                   |         | MIST_GL   | <3>     | MIST_GL    | <3>     | MIST_GL                                                                                            |                                                                                                     |                                                                                                     |
| PZD5     |                                                                                   |                                                                                   |         | PIST_GL   | <3>     | WARN_CODE  | <3>     | WARN_CODE                                                                                          |                                                                                                     |                                                                                                     |
| PZD6     |                                                                                   |                                                                                   |         | <2>       | <3>     | FAULT_CODE | <3>     | FAULT_CODE                                                                                         |                                                                                                     |                                                                                                     |
| PZD7     |                                                                                   |                                                                                   |         |           |         |            |         |                                                                                                    |                                                                                                     |                                                                                                     |
| PZD8     |                                                                                   |                                                                                   |         |           |         |            |         | Receive telegram length freely<br>selectable via central PROFIdrive<br>configuration in the master | Transmit telegram length freely<br>selectable via central PROFIdrive<br>configuration in the master |                                                                                                     |
| PZD9     |                                                                                   |                                                                                   |         |           |         |            |         |                                                                                                    |                                                                                                     |                                                                                                     |
| PZD10    |                                                                                   |                                                                                   |         |           |         |            |         |                                                                                                    |                                                                                                     |                                                                                                     |
| PZD11    |                                                                                   |                                                                                   |         |           |         |            |         |                                                                                                    |                                                                                                     |                                                                                                     |
| PZD12    |                                                                                   |                                                                                   |         |           |         |            |         |                                                                                                    |                                                                                                     |                                                                                                     |

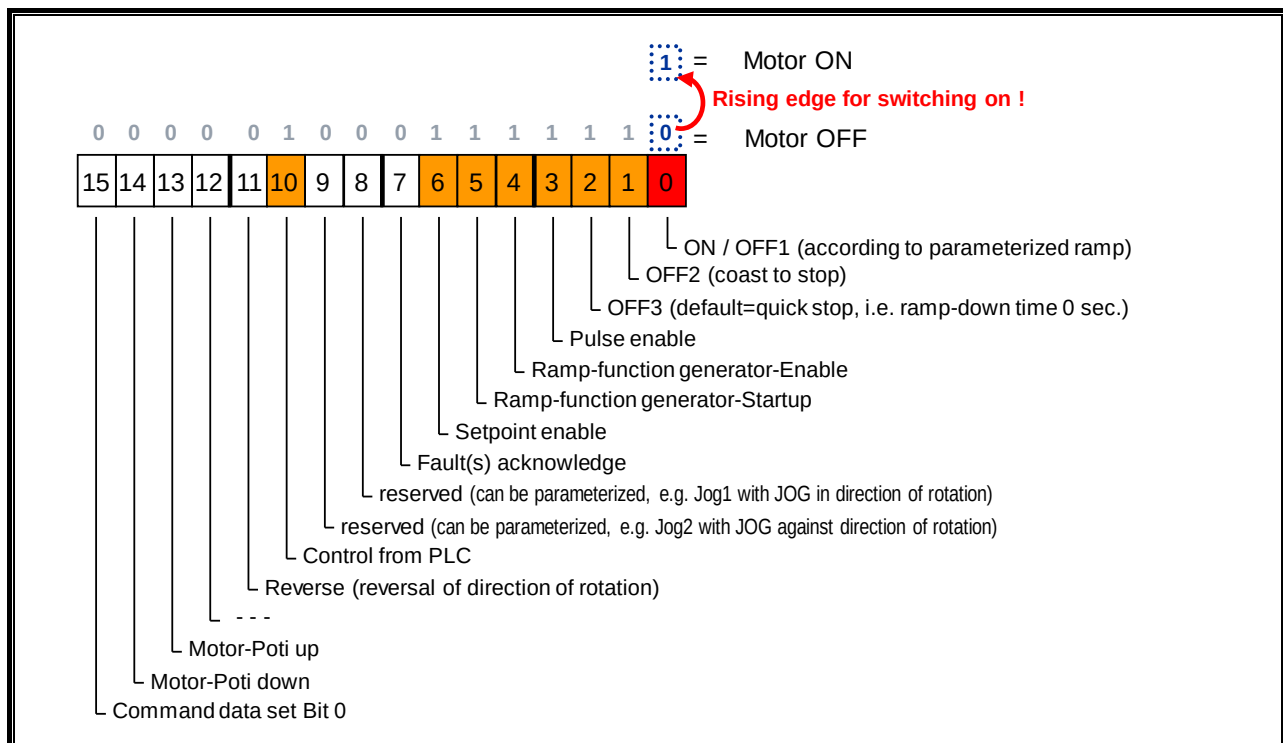
|                      |                                        |                                  |
|----------------------|----------------------------------------|----------------------------------|
| Telegrams 1, 20      | Manufacturer-independent Standard Tel. | automatic configuration in drive |
| Telegrams 352 to 391 | Siemens-specific Standard Telegrams    | automatic configuration in drive |
| Telegram 999         | Free telegram                          | manual configuration required    |

#### Master-Slave Communication via Standard Telegram

The best way to implement a cyclic data exchange between Master (CPU) and Slave (G120) is using a standard telegram, which typically is also used for simple speed controls. Between the Master and Slave, two input and output words with the following contents are exchanged:

- Output word 1: Control word
- Output word 2: Main setpoint value
- Input word 1: Status word
- Input word 2: Main actual value

### 15.1.3. Structure of the Control Word (CDS0 - ControlDataSet for Fieldbus Control)



The inverter can be controlled in various ways. The G120 can either be operated via the field bus or via the terminal strip.

This is controlled through a relevant Control Data Set (CDS).

default for G120 = two CDS:

- Control data set CDS0: Control of the inverter via field bus
- Control data set CDS1: Control of the inverter via terminal strip

#### Control Word

The control word (Bit 0 to 10) complies with the PROFIdrive profile standard. Bits 11 to 15 are inverter-specific.



For reasons of safety in the case of a wire break, the inverter is always switched off when the relevant OFF bit has signal 0.

#### ON/OFF1

A rising edge of this bit is generally required for switching on the inverter, and bits OFF2 and OFF3 must have status 1.

When it is switched off using this bit, the motor is braked by the ramp down of the ramp-function generator (Parameter 1121) and the inverter then switches itself off.

#### OFF2

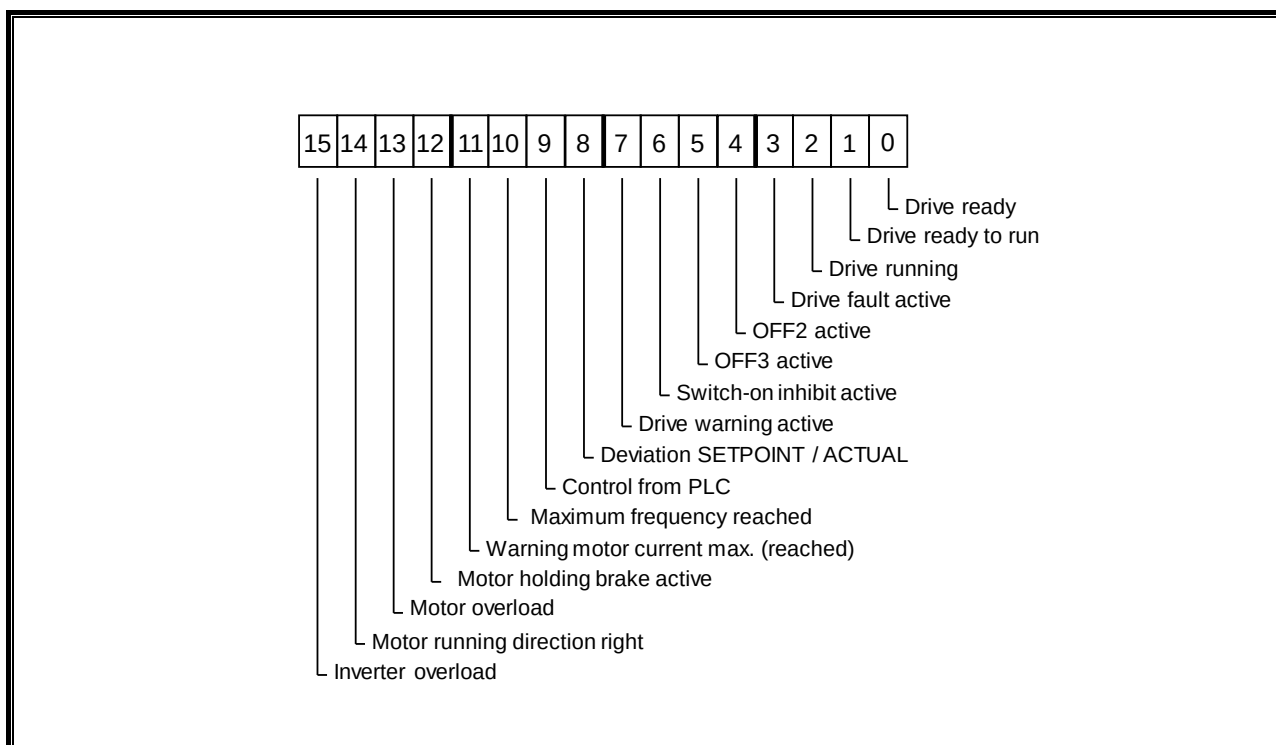
The inverter switches itself off immediately and as a result, the motor coasts down without braking (coasts to a standstill). To switch the inverter on again, this bit must be set to 1 again and a rising edge is necessary at ON/OFF1.

#### OFF3

The motor is braked by the OFF3 ramp down and the inverter remains on. This function is often used as an EMERGENCY STOP.

→ default for OFF3 = 0 seconds (in case of larger motors, this cannot always be achieved)

### 15.1.4. Structure of the Status Word



#### Status Word

The status word (Bit 0 to 10) complies with the PROFIdrive profile standard.

Bits 11 to 15 are inverter-specific.

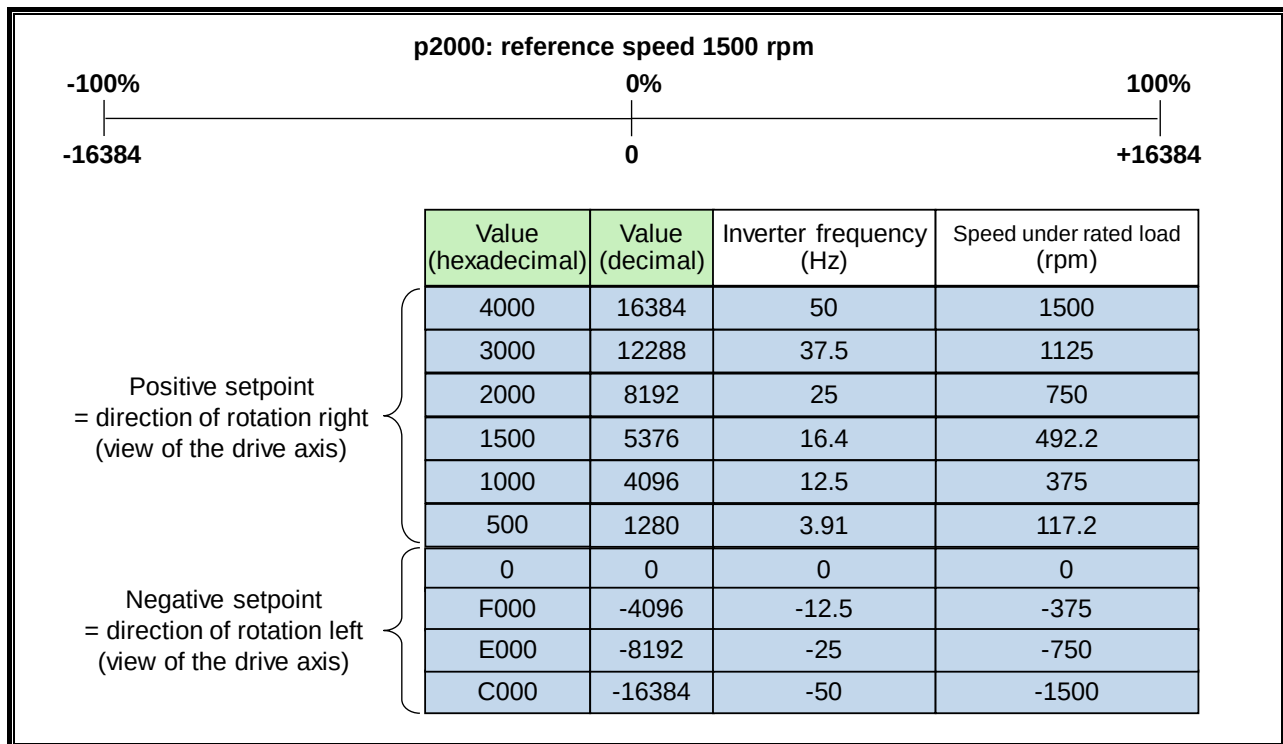
#### Switch-on Inhibit

This status is reached following the error elimination and acknowledgement of a drive fault. A subsequent switch on again is only possible with ON/OFF1 "0" → "1".

#### Warning, Message

Messages (bit 13, bit 15) and Warnings (bit 11)

### 15.1.5. Setpoint / Actual Value → Speed Values

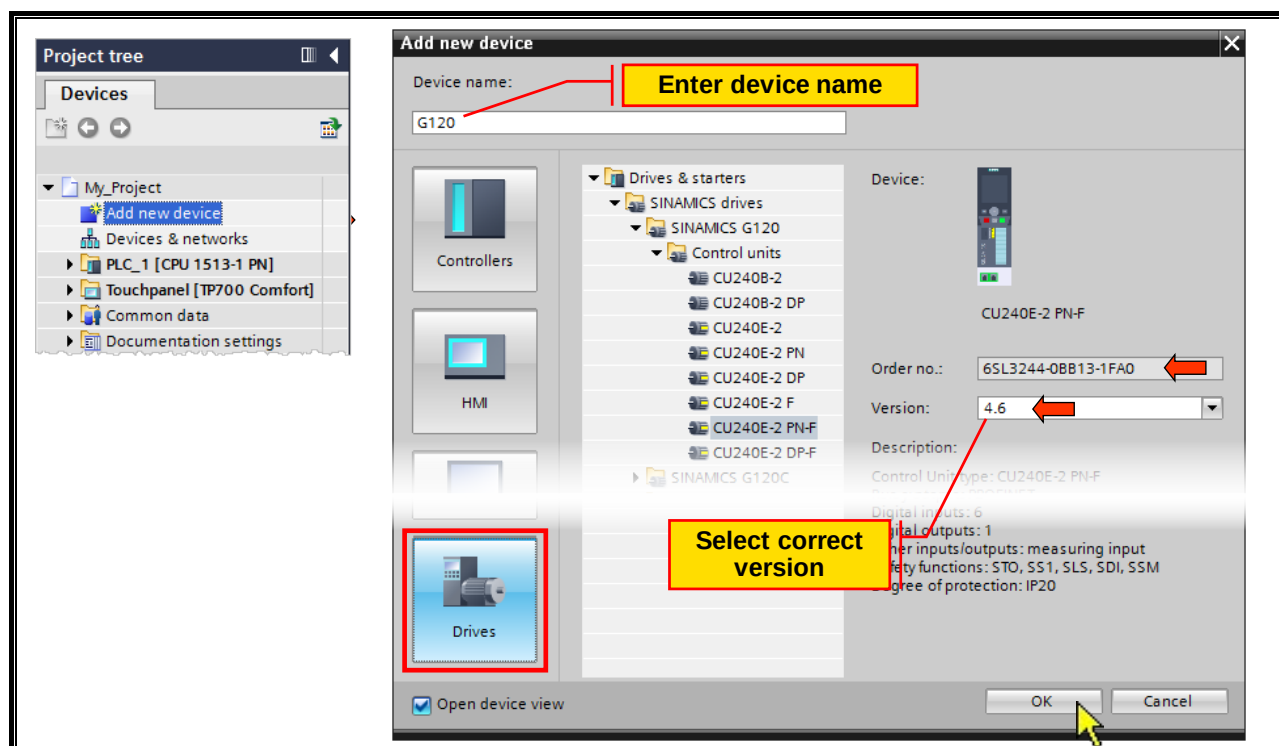


#### Scaling the Speed Values

Via the parameter p2000, a reference speed is defined which all setpoint and actual values use as a reference.

The range of -100% to 100% of the setpoint or actual value, in turn, is scaled to the range 16384 (CH00<sub>HEX</sub>) to +16384 (4000<sub>HEX</sub>).

## 15.2. Inserting a Drive into the Project



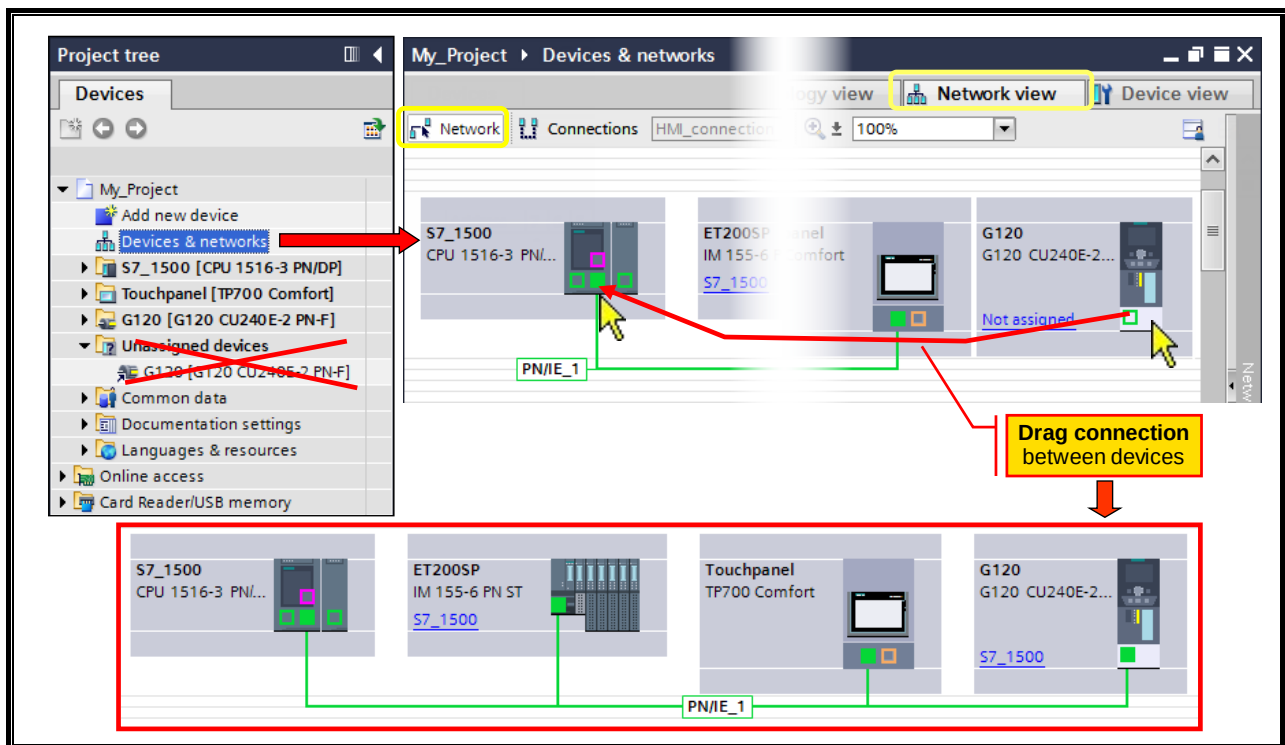
### "Startdrive" Option Package

If the "Startdrive" option package is installed in TIA Portal, drives can be added, configured and assigned parameters as a new device.

Otherwise, a drive can be inserted in the project via GSD file as a distributed I/O station using the Hardware catalog. However, then only the communication with the CPU can be configured in the TIA Portal, no hardware configuration, no parameter assignment.



### 15.2.1. Networking a Drive



#### Editor

"Devices & networks" → "Network view"

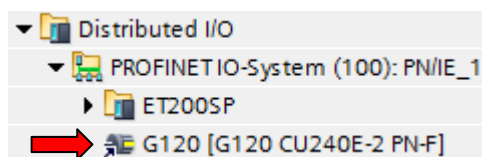
#### Networking

Click on the interface of the one module, hold the left mouse button and drag onto the interface of the other module.

→ here in the picture: PN interface of the G120 dragged onto PN interface of the CPU

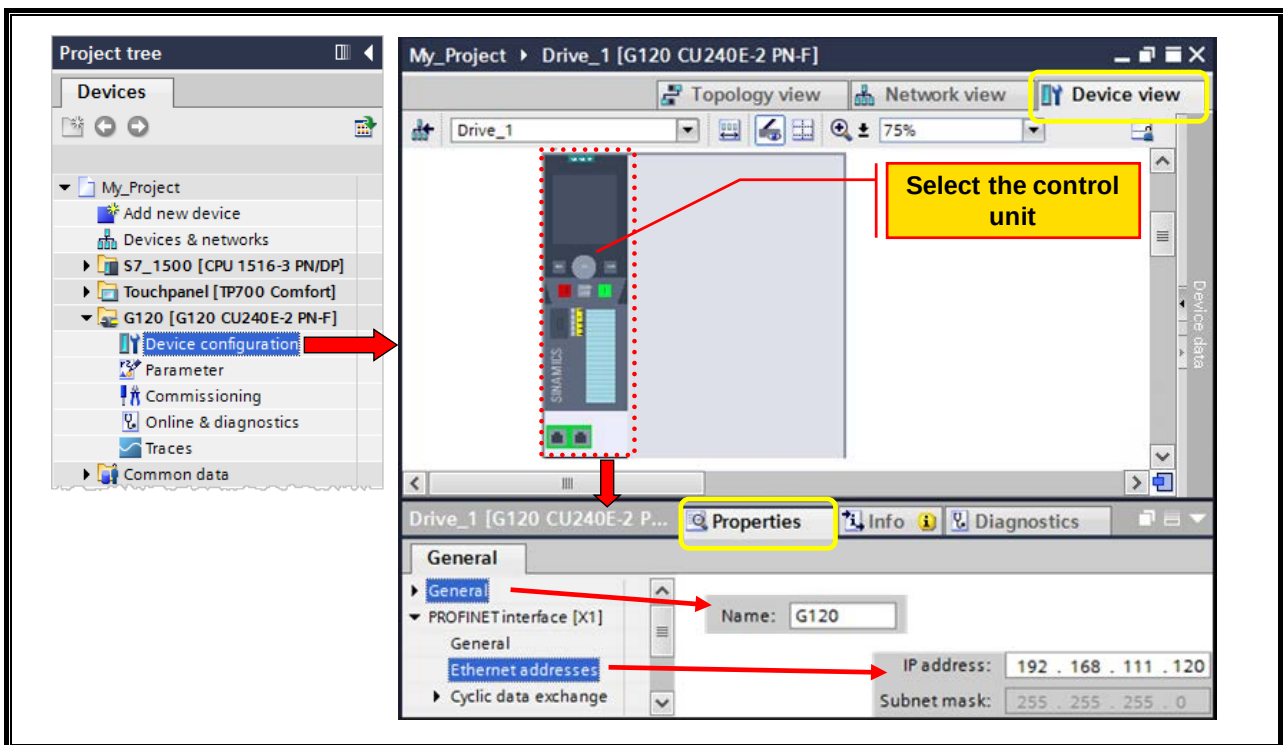
#### Not Assigned Devices

This section then disappears since the device is now assigned to the CPU.



The drive now also appears at the CPU.

### 15.2.2. Parameterizing the Module Address and Name



#### Name of the Module

The name is the assignment CPU ↔ module.

Through this, the PROFINET-IO is identified by the PROFINET-Controller.

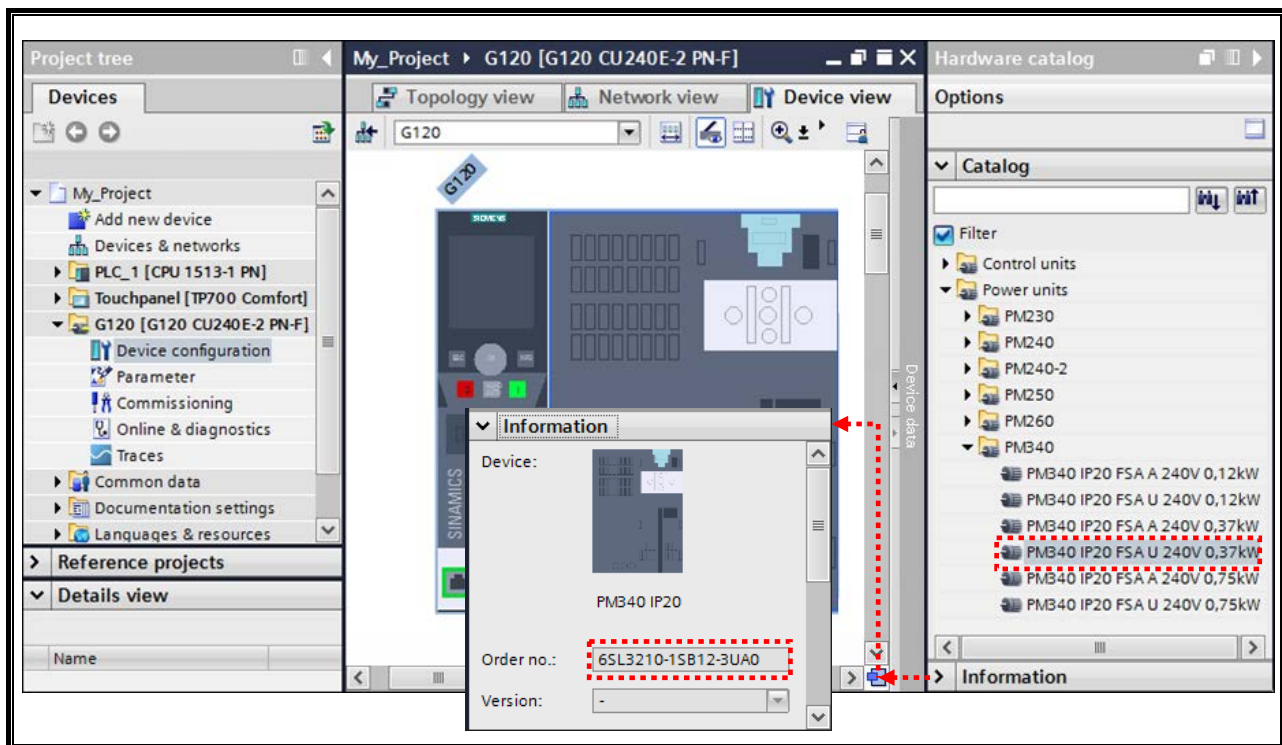


**This name must be assigned to the PROFINET-IO module online; otherwise, the CPU (PROFINET-I/O Controller) cannot identify the module.**

#### Address Assignment of the Distributed IO-Module

For PROFINET-IO, the parameterized IP address of the module is assigned by the CPU through the configured device name.

### 15.2.3. Configuring a Power Unit

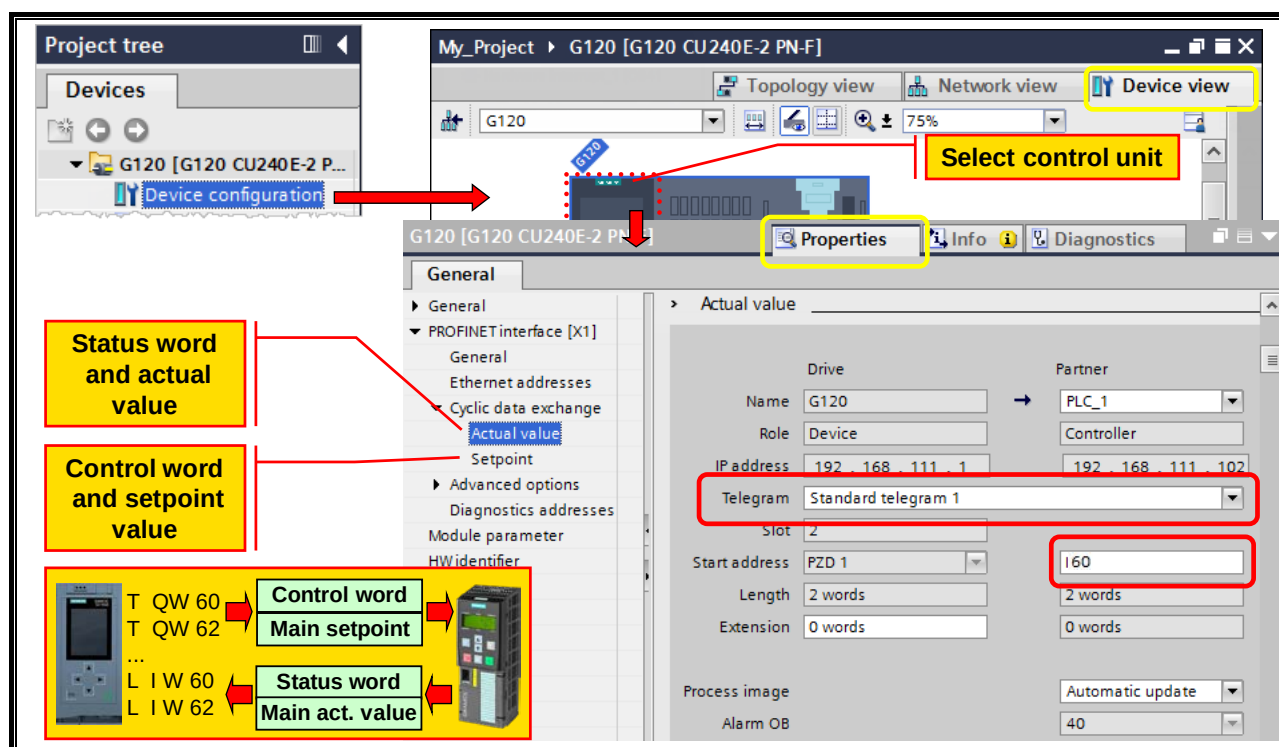


#### Configuring a Power Unit:

A drive consists of two components

- Control unit → already configured with the added device
- Power unit → must be configured

### 15.2.4. Parameterizing the Process Data Area (PZD)



#### Process Data Area

Communication using Standard telegram 1 is based on a cyclic process data exchange that is very easy to program.



The control word and status word comply with the specifications of PROFIdrive profile Version 2.0 or Version 3.0 for the "speed control" operating mode.

The control word and, if necessary, the main setpoint are sent from the CPU to the drive. In the response telegram, the drive returns the status word and the main actual value.

Since only two words are exchanged with the drive, simple load and transfer operations are all that are required in the program. When a double word is transferred, data consistency is also assured.

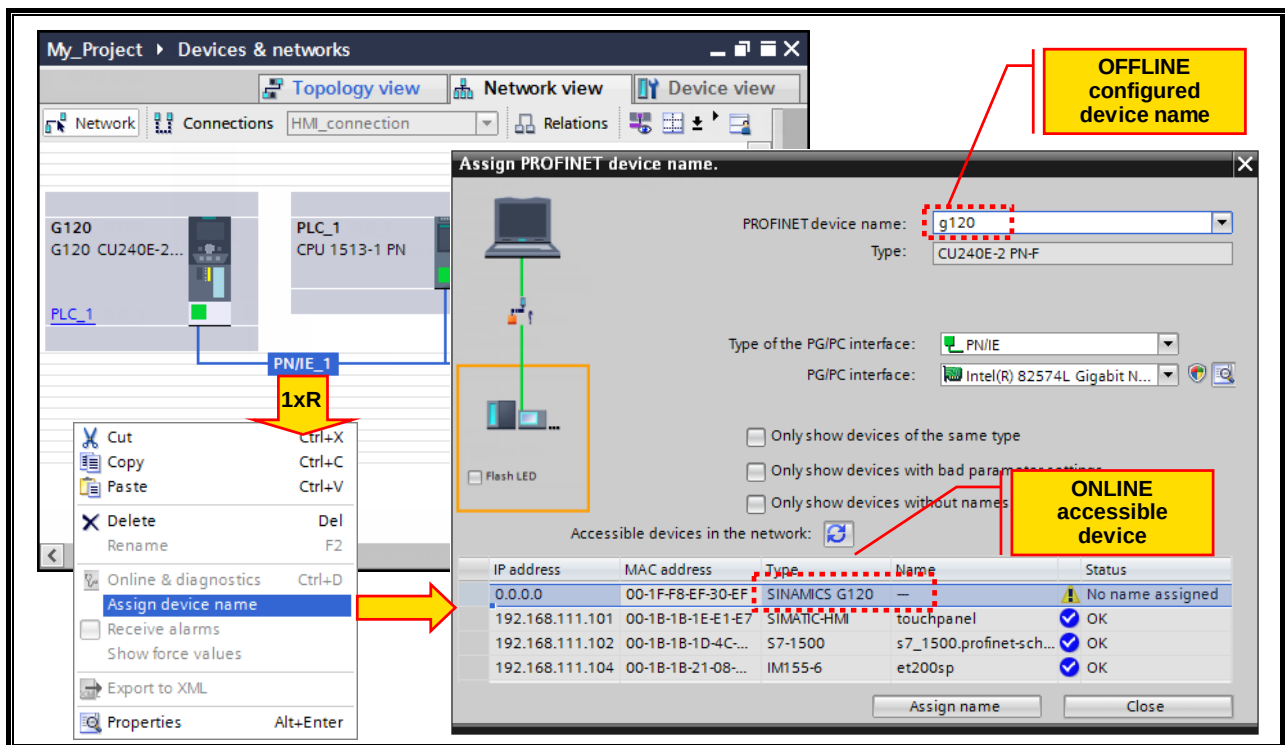
#### Control Word

The control word contains 16 binary signals for controlling (On/Off, direction of rotation) the drive.

#### Main Setpoint

According to standard parameterization of the drive, the main setpoint specifies the setpoint speed.

## 15.2.5. Assigning the Device Name ONLINE (Module Initialization)



### Device Name

The PROFINET device name that was configured OFFLINE must be assigned to the drive that actually exists ONLINE.

### Name of the Module

The name is the assignment CPU ↔ module.

Through this, the PROFINET-IO is identified by the PROFINET-Controller.

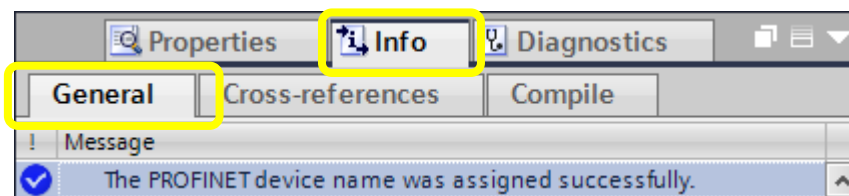


**This name must be assigned to the PROFINET-IO module online; otherwise, the CPU (PROFINET-I/O Controller) cannot identify the module.**

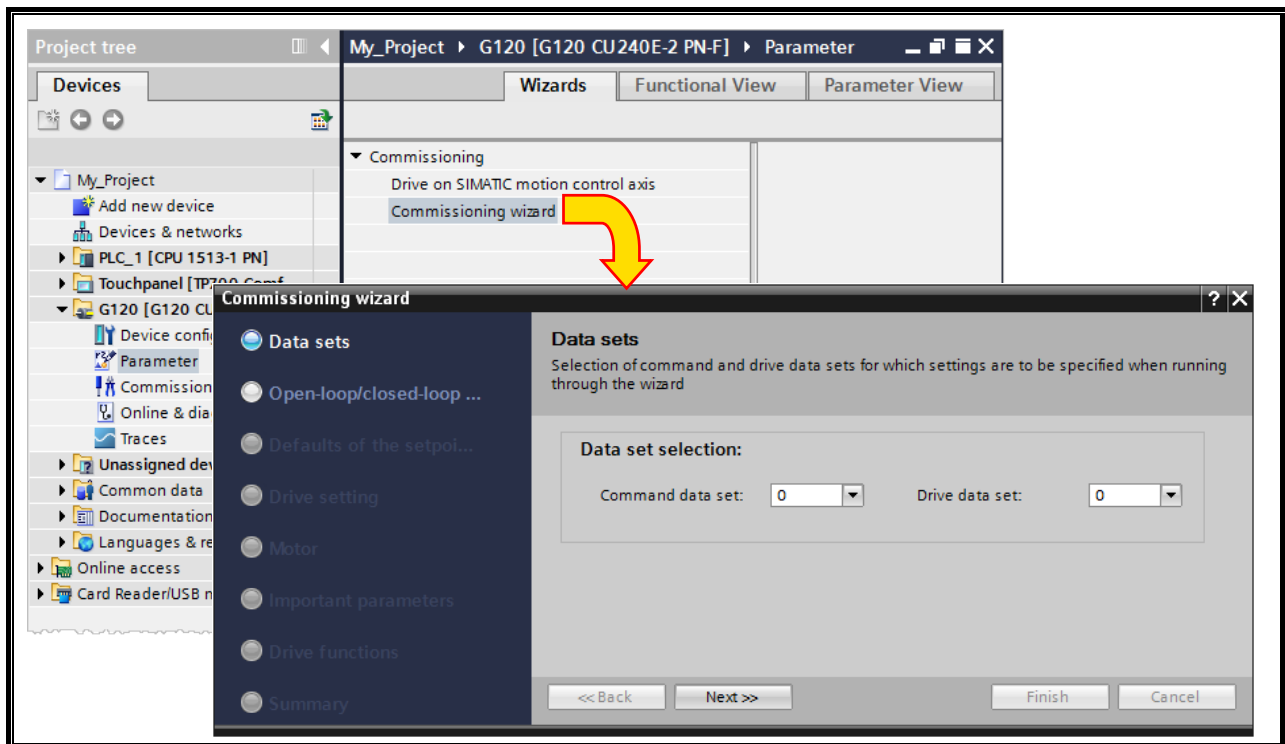
### Name Assignment

Currently, the menu item is only included in the context menu of the network connection; not with the drive.

### Checking the Result in the Inspector Window



### 15.3. Parameterizing the Drive: with the "Commissioning Wizard"

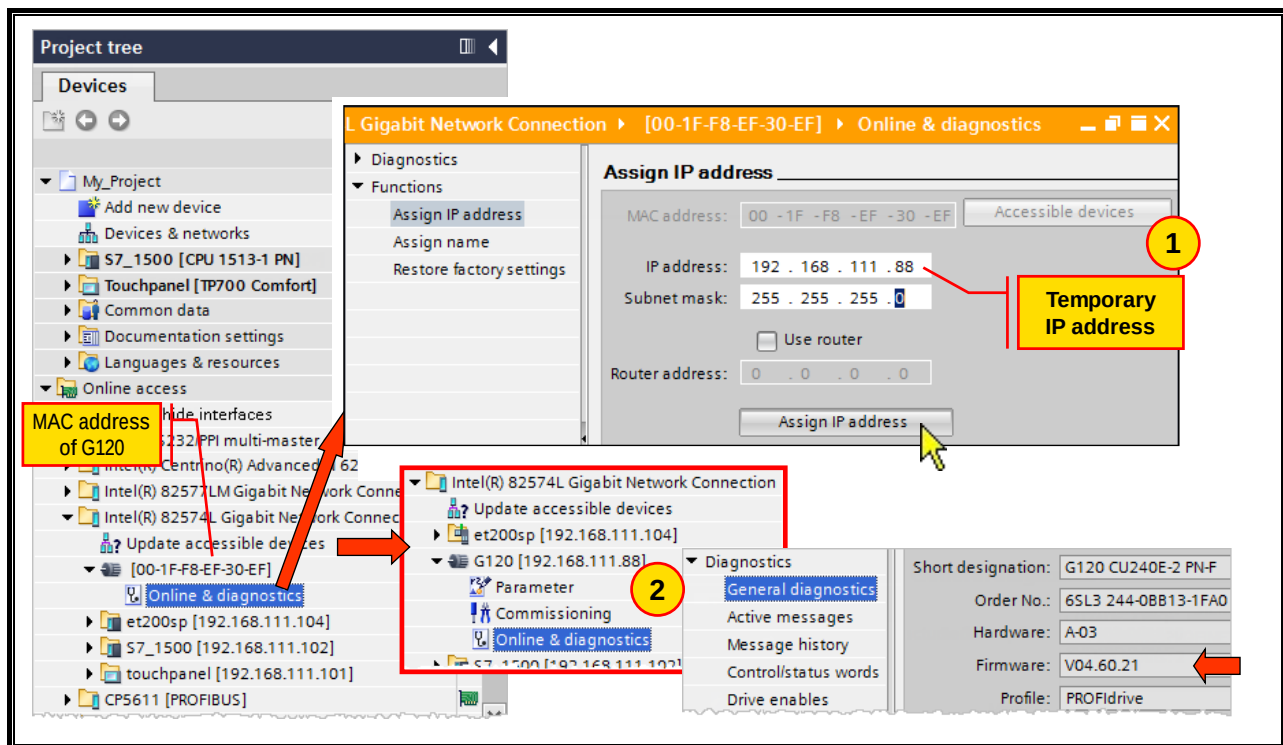


#### Commissioning Wizard

The most important settings for the drive can be made with the menu-driven Commissioning Wizard. These are:

- Data sets
- Open-loop/closed-loop parameters
- Defaults of the setpoints / command sources
- Drive settings
- Motor (data)
- Important parameters (dynamic data)
- Drive functions

### 15.3.1. Exercise 1: Reading-out the Firmware Version of the Drive



#### Task

In order to be able to configure the drive in the project in the next exercises, you must know the firmware version of the training unit. Since the firmware of the drive can be upgraded, the version of the firmware specified on the outside of the drive does not necessarily reflect reality. For that reason, the current version of the firmware is now to be read out online from the drive.

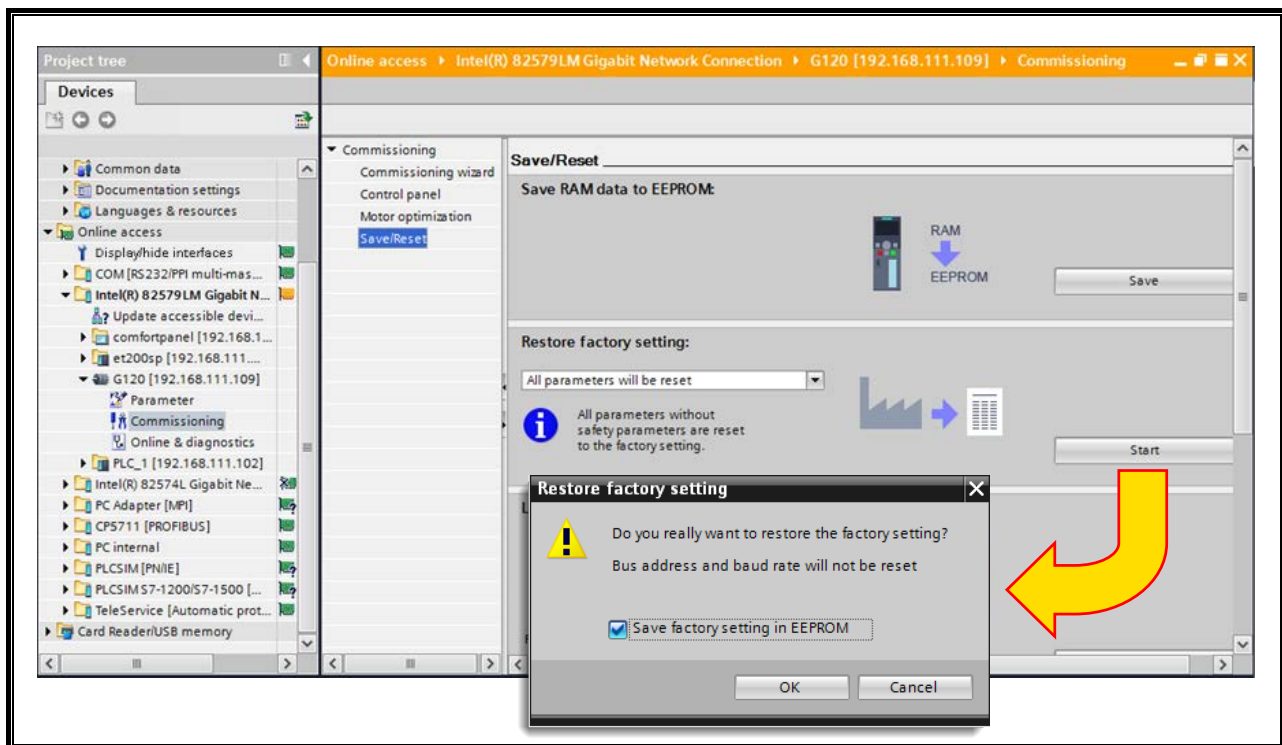
#### Problem

Should the drive only have a MAC address (see Project tree, to the left in the picture) and neither a PROFINET device name nor an IP address, the firmware version cannot be read out, since an IP address is required for this diagnostic service.

#### What to Do

1. In case the drive does not yet have an IP address:  
Assign a temporary IP address as shown in the picture and update the list "Update accessible devices".
2. In the list of "accessible devices", open the drive and activate "Online & diagnostics".
3. In the window that appears on the right, activate the function "General diagnostics" and make note of the firmware version of the inverter.
4. Close the window.

### 15.3.2. Exercise 2: Restoring the Factory Settings



#### Task

The drive is to be reset to its factory settings.

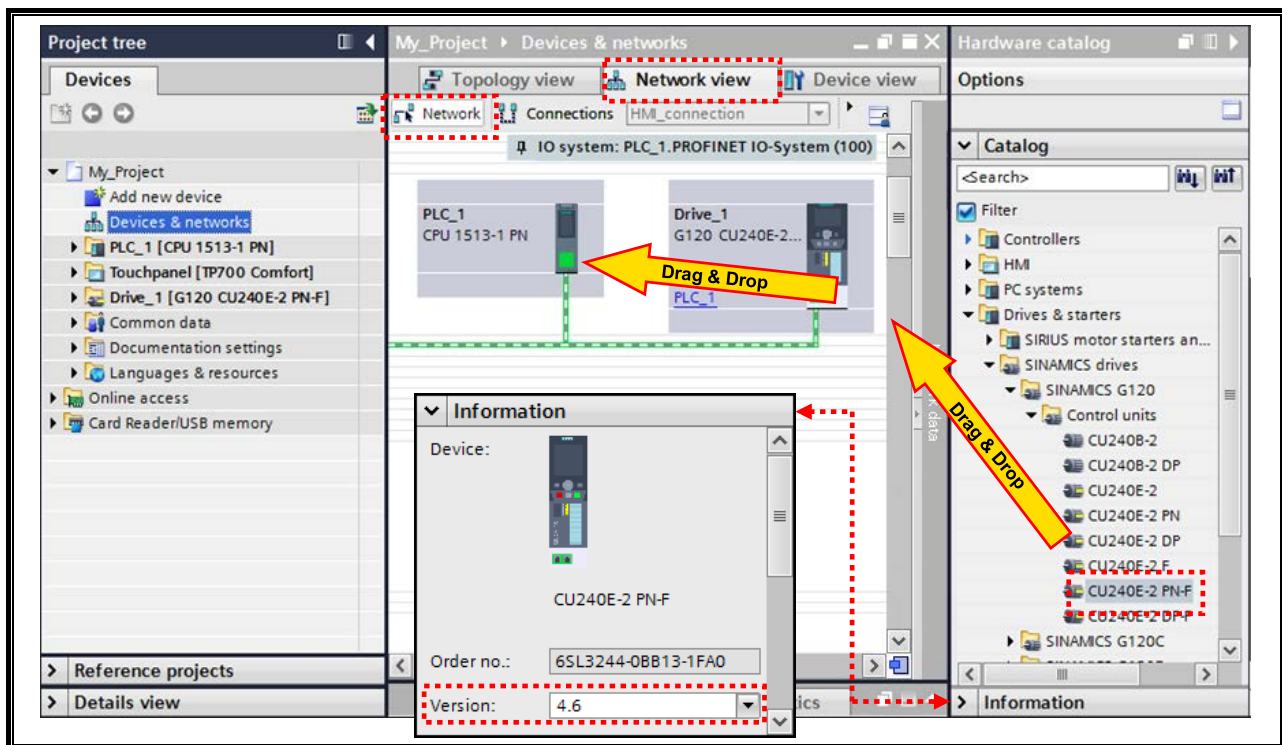
With the "Restore factory settings" function, the PROFINET device name and the IP address are retained.

#### What to Do

1. Under the Online access, select the interface Intel(R) 82574L Gigabit... and update the list "Update accessible devices".
2. Open the drive and there activate the function "Commissioning".
3. In the window that appears on the right, activate the function "Save/Reset" and there start the function "Restore factory setting".
4. In the dialog that then appears activate the option "Save factory setting in EEPROM" so that even after a power recovery, the drive retains the factory setting.
5. Close the window.



### 15.3.3. Exercise 3: Inserting and Networking the Drive in the Offline Project



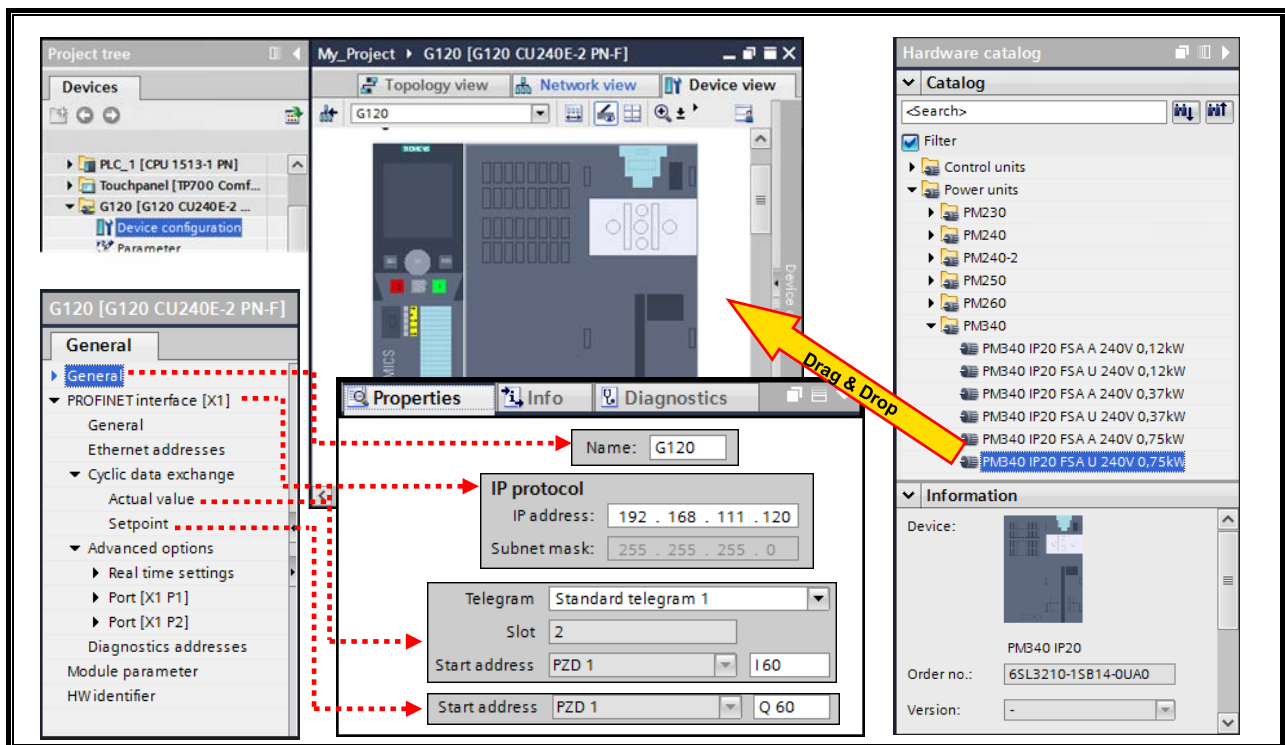
#### Task

The drive is to be inserted in the project and networked with the CPU via PROFINET.

#### What to Do

1. Open the "Devices & networks" editor and there activate the Network view.
2. In the Task Cards, open the Hardware catalog.
3. Insert the G120 drive by dragging the CU240E-2PN-F control unit into the Network view using drag & drop. Pay attention to the order number and the version in the "Information".
4. Network the drive with the CPU by dragging the PROFINET interface of the drive to the PROFINET interface of the CPU using drag & drop.
5. Save your project.

### 15.3.4. Exercise 4: Configuring and Parameterizing the Drive



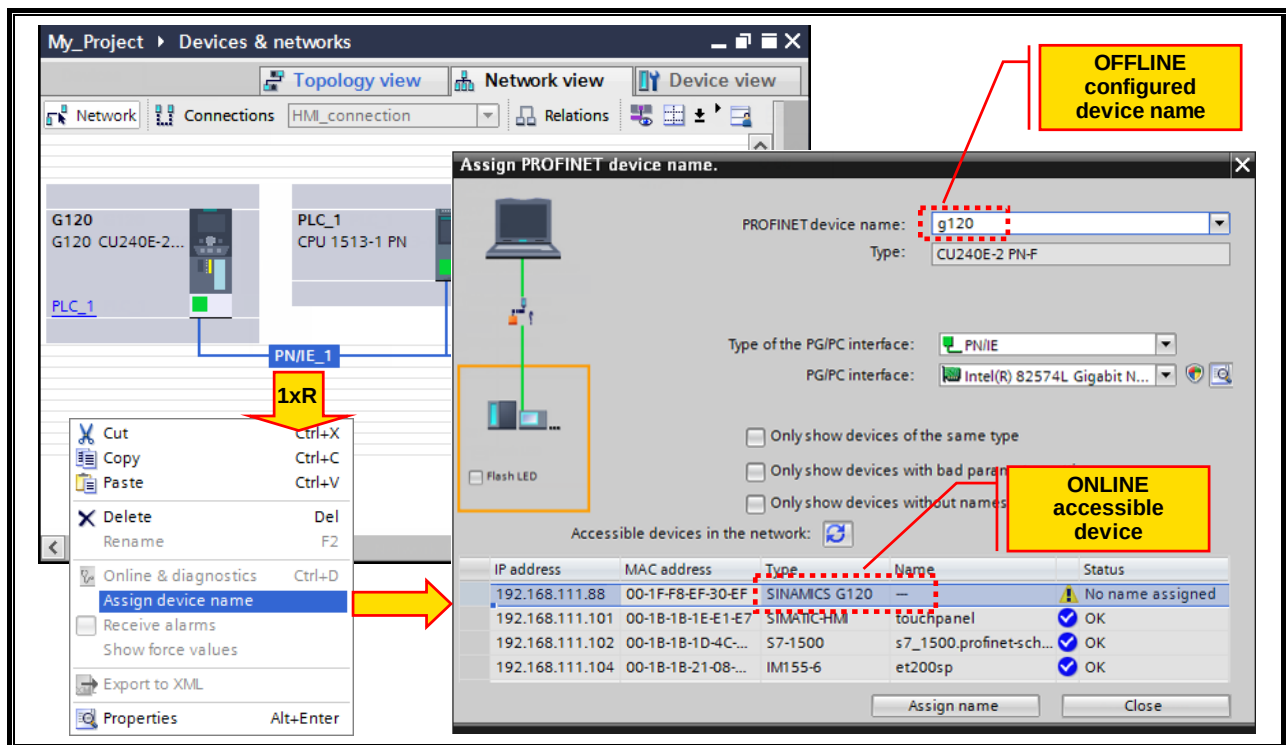
#### Task

So that the configuration of the drive is complete, a power unit still has to be added. Furthermore, the control unit still has to be parameterized.

#### What to Do:

1. Open the Device configuration of the drive.
2. Open the Hardware catalog and, using drag & drop, add the power unit. Pay attention to its order number and version in the "Information".
3. Select the control unit and, in the Inspector window under "Properties", parameterize the module with the parameters shown in the picture.
4. Save your project.

### 15.3.5. Exercise 5: Assigning the PROFINET Device Name ONLINE



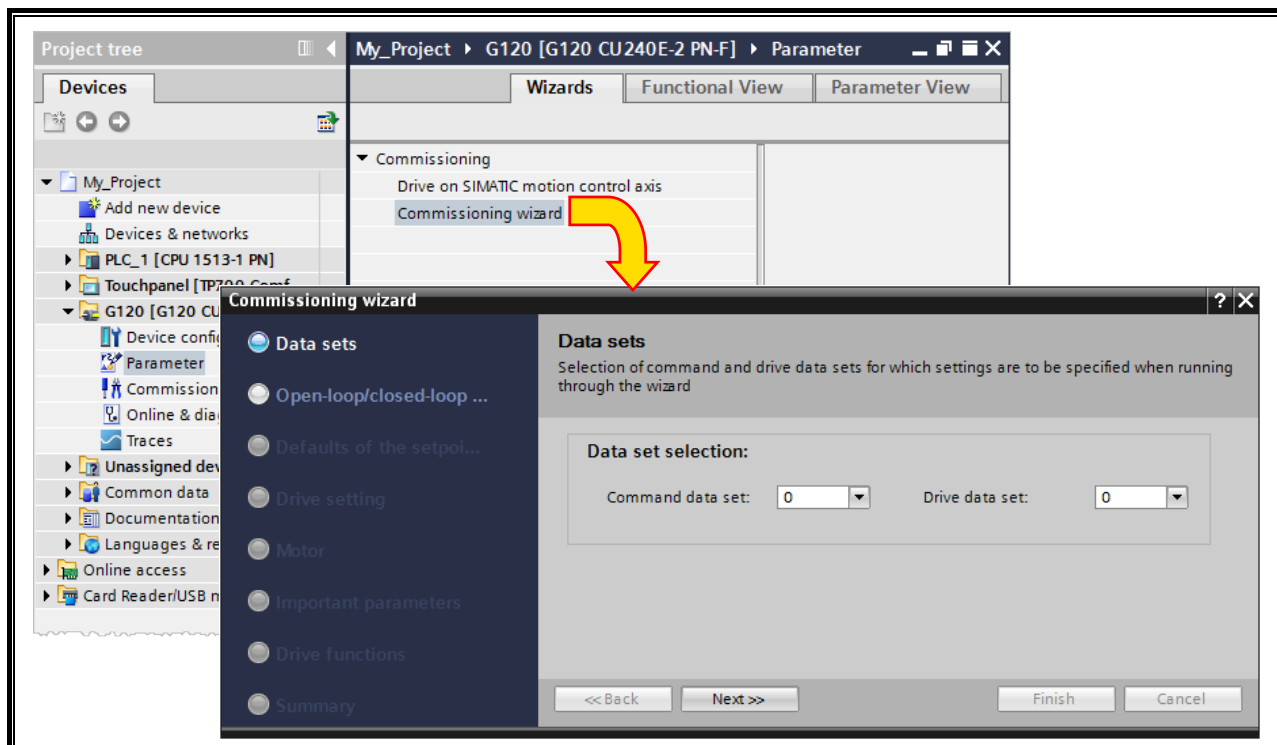
#### Task

ONLINE, assign the PROFINET device name and the IP address that was configured OFFLINE for the drive.

#### What to Do

1. Open the Devices & networks editor and there activate the Network view.
2. Select the PLC\_1 IO-System (100) and, with a right-click, activate the function "Assign device name".
3. Complete the dialog that follows with the information shown in the picture and assign the name.
4. Check whether the name assignment was successful by updating the "Update accessible devices" under 'Online access'.
5. Save your project.

### 15.3.6. Exercise 6: Parameterizing the Drive OFFLINE with the Commissioning Wizard



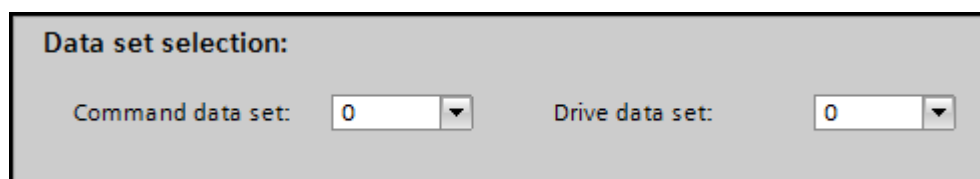
#### Task

OFFLINE set the most important drive parameters and use the "Commissioning Wizard" for this.

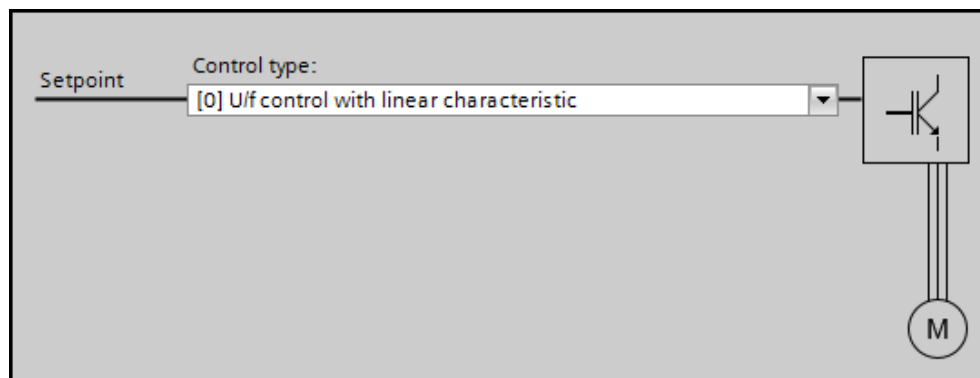
#### What to Do

1. Start the Commissioning Wizard as shown in the picture. Implement the parameterization as shown in the following.

- Data sets



- Open-loop and closed-loop control type



- Defaults of the setpoints / command sources

Current I/O configuration:

Select the default of the I/O configuration:

 Note: If changed, all the existing drive-internal interconnections to the I/O terminals are deleted and reconnected in accordance with the selected I/O configuration.

Current interconnections of the I/O terminals:

|       |                        |                                                                                     |
|-------|------------------------|-------------------------------------------------------------------------------------|
| DI 0: | p1055[1] BI: Jog bit 0 |  |
| DI 1: | p1056[1] BI: Jog bit 1 |  |
| AI 0: | -                      |  |
| AI 1: | -                      |  |

Current telegram configuration:

- Drive setting

Standard:

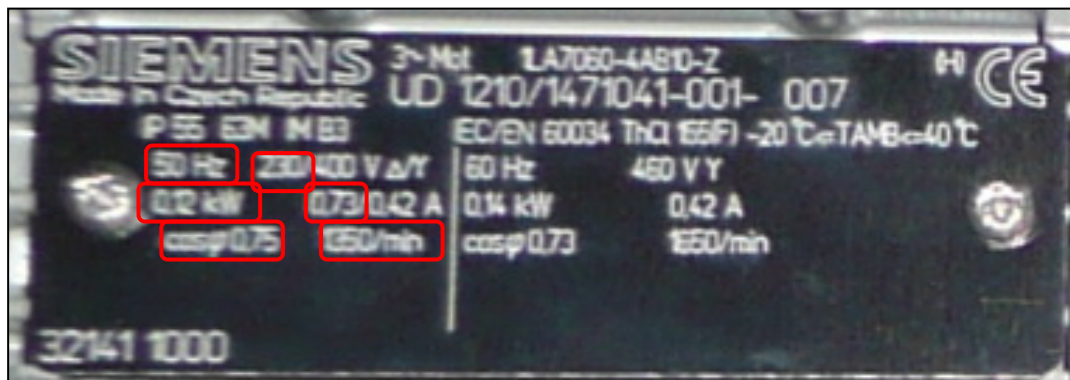
You can see which standard your motor complies with on the rating plate:

- [0] IEC-Motor (50 Hz, SI units)
- [1] NEMA motor (60 Hz, US units)
- [2] NEMA motor (60 Hz, SI units)

Power unit application:

- **Motor**

Take the motor data from the motor's nameplate and enter it in the Motor dialog:



Motor configuration

Enter motor data ▼

Select motor type

[17] 1LA7 standard induction motor series ▼

Motor data

| Parameter | Parameter text           | Value             | Unit |
|-----------|--------------------------|-------------------|------|
| p304[0]   | Rated motor voltage      | 230               | Vrms |
| p305[0]   | Rated motor current      | 0.73              | Arms |
| p307[0]   | Rated motor power        | 0.12              | kW   |
| p308[0]   | Rated motor power factor | 0.750             |      |
| p310[0]   | Rated motor frequency    | 50.00             | Hz   |
| p311[0]   | Rated motor speed        | 1350.0            | rpm  |
| p335[0]   | Motor cooling type       | [0] Non-ventil... |      |

Motor connection type

Delta ▼

☐ Motor 87 Hz operation

☐ Parallel motor connection

Number: 1

- **Important parameters**

Change the ramp-function generator times:

Set the values for the most important parameters:

|                                            |                                       |      |
|--------------------------------------------|---------------------------------------|------|
| Current limit:                             | <input type="text" value="1.10"/>     | Arms |
| Minimum speed:                             | <input type="text" value="0.000"/>    | rpm  |
| Maximum speed:                             | <input type="text" value="1500.000"/> | rpm  |
| Ramp-function generator<br>ramp-up time:   | <input type="text" value="1.000"/>    | s    |
| Ramp-function generator<br>ramp-down time: | <input type="text" value="1.000"/>    | s    |
| OFF3 ramp-down time:                       | <input type="text" value="0.000"/>    | s    |

- **Drive functions**

A motor identification is recommended for the first commissioning.  
Stationary measurement is recommended for V/f and vector control.  
Stationary measurement and rotating measurement are recommended  
for vector control (only rotating measurement is not sufficient).

Motor identification:

Calculation of the motor parameters

☐ No calculation

☒ Complete calculation

 Includes the calculation of the motor, open-loop control and closed-loop control parameters depending on the entered motor data.

- Summary

**Summary**

Please check the entered data and complete the configuration

The following drive data has been entered:

Open-loop/closed-loop control type:  
Open-loop/closed-loop control operating mode: [0] U/f control with linear characteristic

Defaults of the setpoints/command sources:  
Macro drive unit: [7] Fieldbus with data set changeover

Drive setting:  
IEC/NEMA mot stds: [0] IEC-Motor (50 Hz, SI units)  
Power unit application: [0] Load duty cycle with high overload for vector drives

Motor:  
Motor type selection: [17] 1LA7 standard induction motor series  
Number of motors connected in parallel: 1  
Rated motor voltage: 230 Vrms  
Rated motor current: 0.73 Arms  
Rated motor power: 0.12 kW  
Rated motor power factor: 0.750  
Rated motor frequency: 50.00 Hz  
Rated motor speed: 1350.0 rpm  
Motor cooling type: [0] Non-ventilated

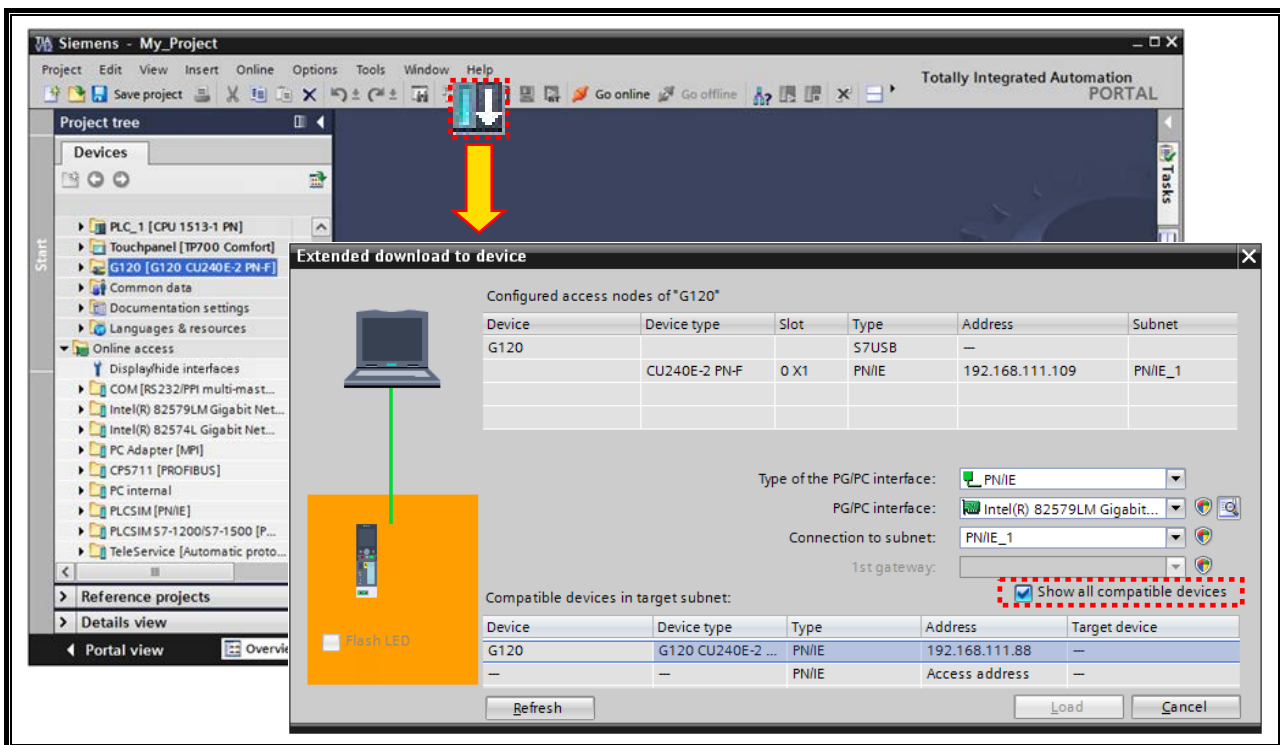
Important parameters:  
Current limit: 1.10 Arms  
Minimum speed: 0.000 rpm  
Maximum speed: 1500.000 rpm  
Ramp-function generator ramp-up time: 1.000 s  
Ramp-function generator ramp-down time: 1.000 s  
OFF3 ramp-down time: 0.000 s

<< Back    Next >>    **Finish**    Cancel

2. Close the Commissioning Wizard (with Finish) and save your project.



### 15.3.7. Exercise 7: Downloading the Parameterization into the Drive



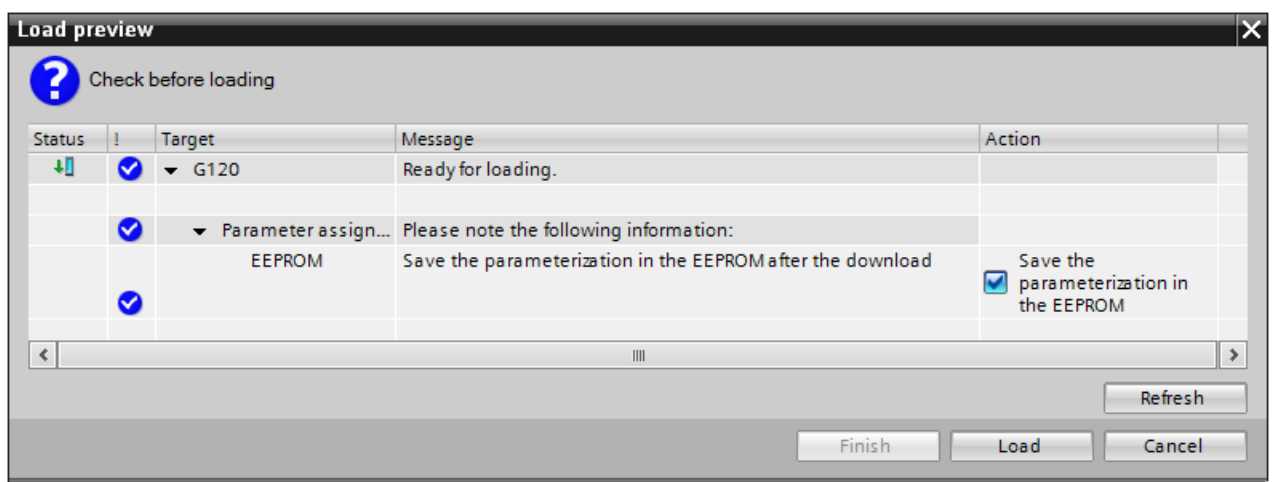
#### Task

Now that you have implemented the basic parameterization offline, it is now to be downloaded into the drive online.

#### What to Do

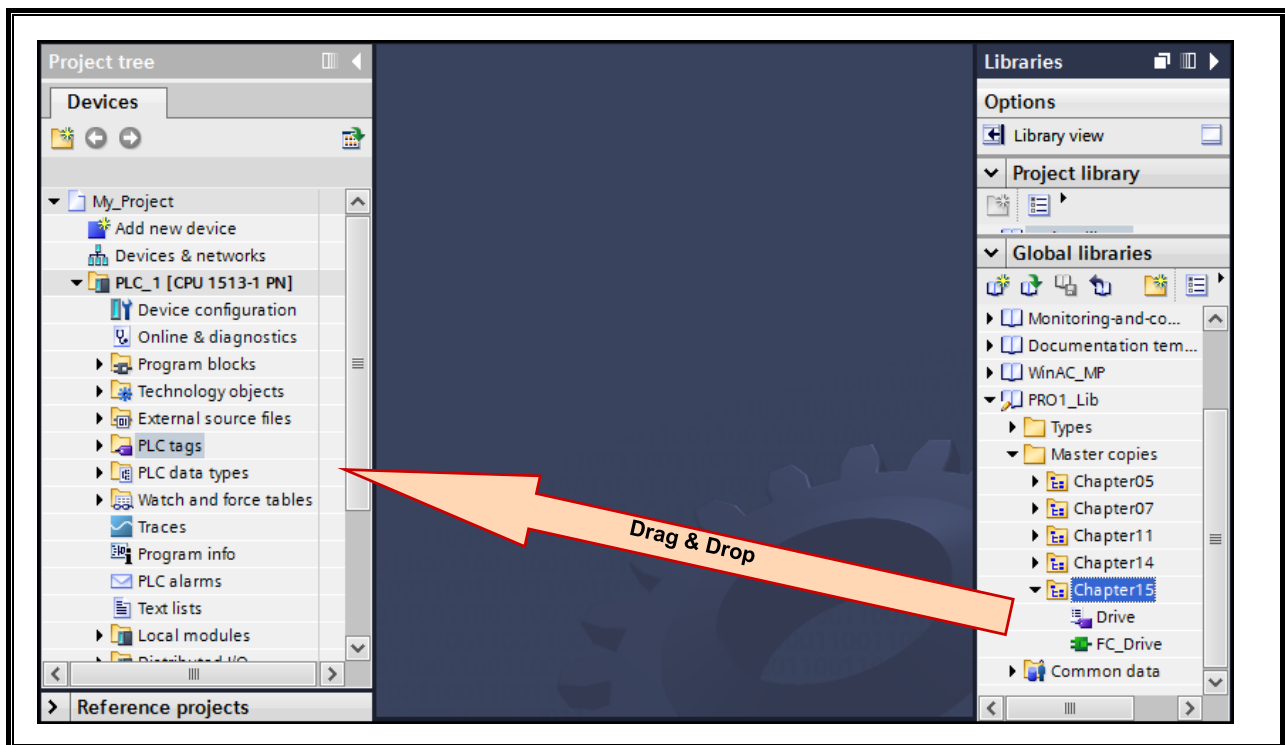
1. In the Project tree, select the drive and start the download of the hardware configuration and parameter assignment using the button shown in the picture.
2. The dialog "Extended download to device" appears which contains the list of "Compatible devices in target subnet" in the lower part. Should the drive of your training area not be listed there, activate the option "Show all compatible devices" (see picture).
3. In the list of "Compatible devices in target subnet", select the drive of your training area and start the "Download".

So that the drive doesn't lose the parameterization even after a power failure, answer the following dialog as follows:



4. Then, download the device configuration into the CPU so that the drive is passed the parameterized IP address.  
(With Profinet, the distributed I/O is recognized by the CPU by means of the device name and in this way is assigned the parameterized IP address. For that reason, when the IP address is changed, this information must also be passed to the CPU.)

### 15.3.8. Exercise 8: Commissioning the Program Expansion



#### Task

Imagine that the G120 serves as the drive for the conveyor model when parts are automatically transported from Bay1 or 2 through the light barrier.

Accordingly, the G120 must be switched on and off parallel to the conveyor model motor when "P\_Operation" (Q0.1) is switched on. The G120 speed can be set via the right slide control potentiometer.

The described function is already implemented in the block "FC\_Drive" (FC120) and is now to be copied from a library and commissioned.

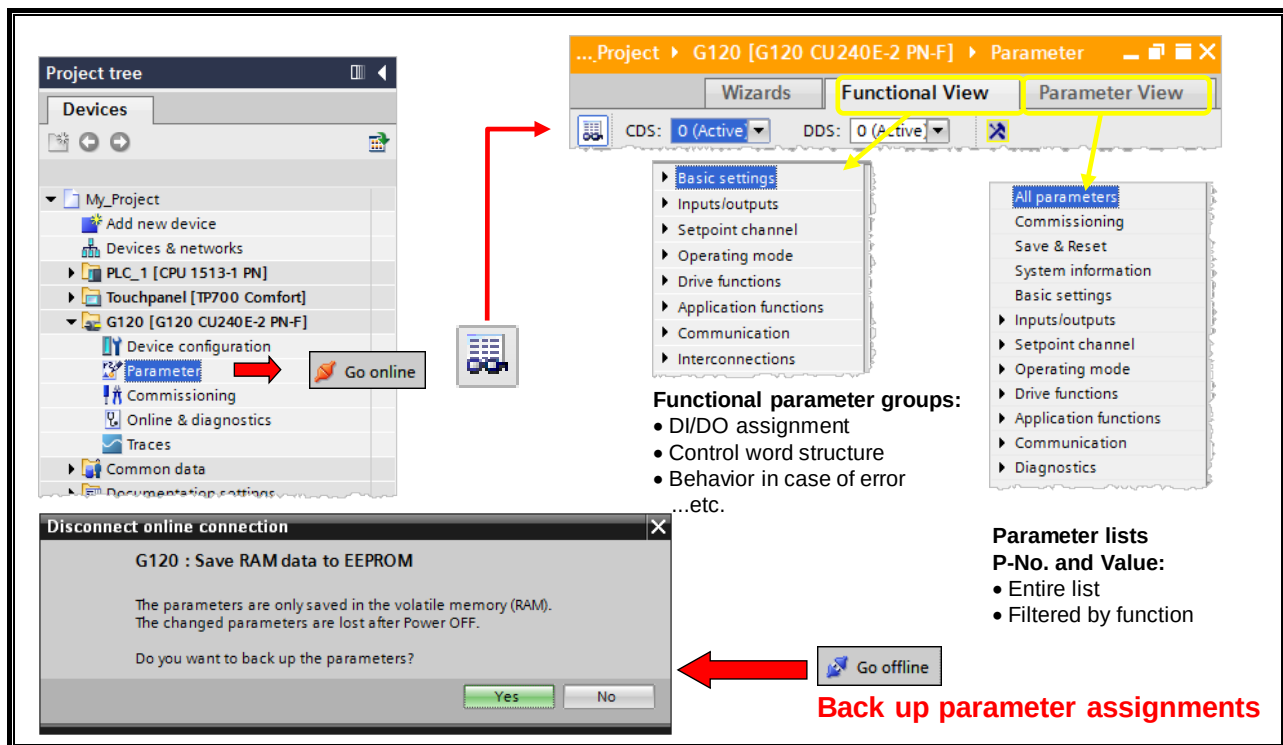
#### What to Do

1. Open the library C:\Archives\TIA\PRO1\_Lib
2. Using drag & drop, copy the library elements "FC\_Drive" (FC120) and "Drive" (PLC tag table) into your project.
3. Program the call of "FC\_Drive" (FC120) in OB1.
4. Download all blocks into the CPU and check whether the G120 is switched on and off parallel to the conveyor model motor when "P\_Operation" (Q0.1) is switched on.
5. Check whether you can change the speed of the G120 via the right slide control potentiometer of the operator panel.
6. Save your project.

## 15.4. Additional Information



### 15.4.1. Changing Parameters in the Inverter



Parameters can be changed through the "Parameter" editor.

#### Calling the "Parameter" Editor from the Device in the Project Tree



This function must be activated so that the online parameters of the drive are also visible. You can see this in that the parameters are displayed on an orange-colored background.

When the online connection is to be terminated, a prompt comes up questioning whether the data is to be copied to EEPROM (nonvolatile backup of the parameter changes).



If the change is only to be tried for test purposes, you can answer the query for copying with NO and after a PowerOFF/ON, the inverter then works with its previous values.

#### Calling the "Parameter" Editor from the Online Access

The change of these parameters is immediately written to the EEPROM.

### 15.4.2. Activating / Deactivating the Control Panel

**Prerequisite:**

- Online connection PG ↔ Drive exists
- No fault is active (if necessary, acknowledge first)

**Go online**

**Commissioning**

**Control panel**

**Activate**

**Acknowledge fault**

As soon as the drive has a valid IP address and thus is accessible online, the parameterization and functions of commissioning are available.

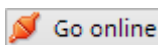
#### Online Accesses

As soon as the drive has been assigned a valid IP address in the subnet used, the functions of commissioning are available here.

#### Device in the Project Tree

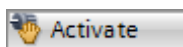


If the drive was inserted as a new device, it is assigned the default IP address 192.168.0.1.



**Go online** is only possible when the IP address of the device configuration corresponds to the subnet used and is also assigned to the drive.

#### Control Panel

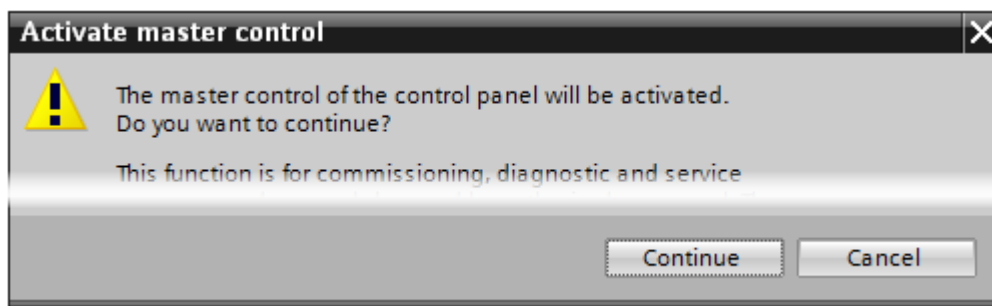


This can only be activated when an online connection exists and no fault exists.

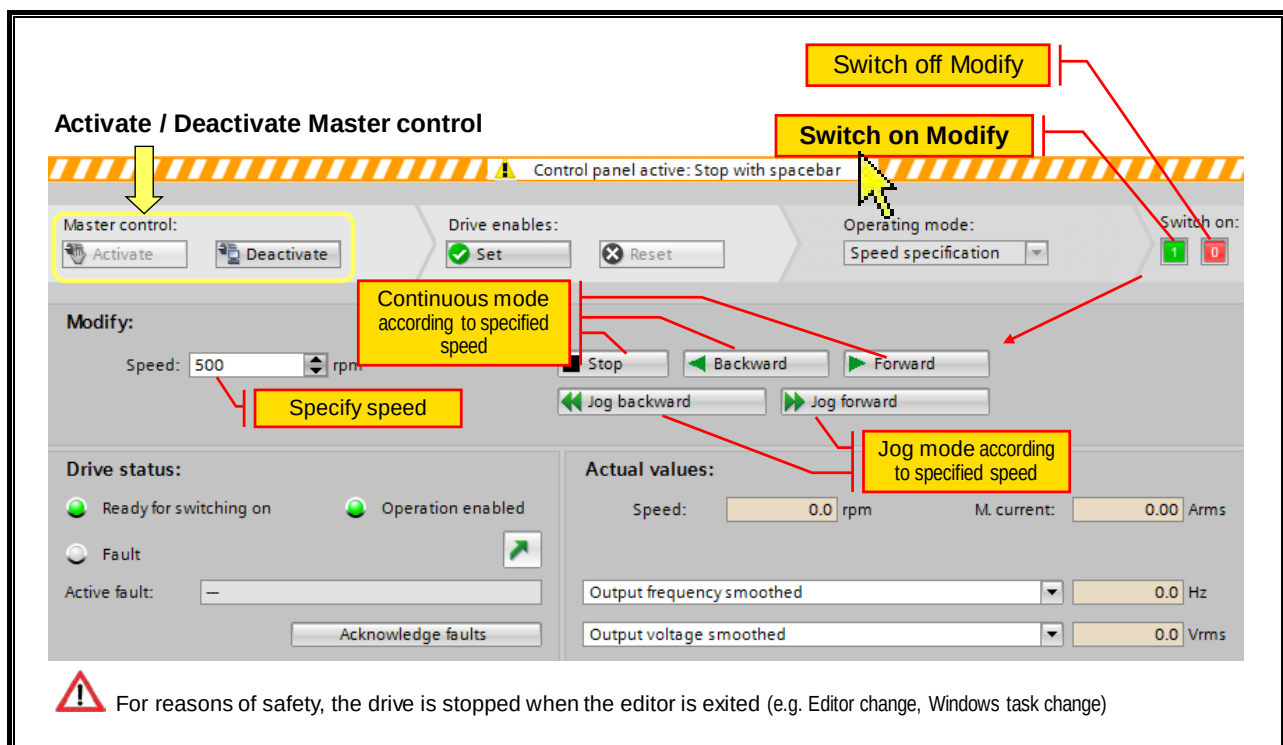


#### Caution!

Activation terminates a possibly existing connection to the CPU and gives the operator full control of the drive. Therefore it is important to have at your disposal a hardware solution for an Emergency Stop.



### 15.4.3. Operating the Control Panel



If the master control is "fetched to the PG", "Modify" can be switched on.

Operation is enabled and the buttons for modifying are active.

#### Safety Shutdown

The drive is stopped and...

- operating enables are cancelled, when the "Commissioning" editor is exited through a Windows task change.

⇒  **Set**

- the master control is deactivated with a fault message, when a switch is made to the editor of another device.

⇒  **Acknowledge faults** and  **Activate**

#### 15.4.4. Monitoring Control and Status Word Online

Select Control word 1 and Status word 1

Control word 1

|    |                                                |
|----|------------------------------------------------|
| 0  | ON/OFF1 [0=No, 1=Yes]                          |
| 1  | OC / OFF2 [0=No, 1=Yes]                        |
| 2  | OC / OFF3 [0=No, 1=Yes]                        |
| 3  | Operation enable [0=No, 1=Yes]                 |
| 4  | Ramp-function generator enable [0=No, 1=Yes]   |
| 5  | Continue ramp-function generator [0=No, 1=Yes] |
| 6  | Speed setpoint enable [0=No, 1=Yes]            |
| 7  | Acknowledge fault [0=No, 1=Yes]                |
| 8  | Jog bit 0 [0=No, 1=Yes]                        |
| 9  | Jog bit 1 [0=No, 1=Yes]                        |
| 10 | Master ctrl by PLC [0=No, 1=Yes]               |
| 11 | Direction reversal (setpoint) [0=No, 1=Yes]    |
| 12 | Motorized potentiometer raise [0=No, 1=Yes]    |
| 13 | Motorized potentiometer lower [0=No, 1=Yes]    |
| 14 |                                                |
| 15 |                                                |

Status word 1

|    |                                               |
|----|-----------------------------------------------|
| 0  | Rdy for switch on [0=No, 1=Yes]               |
| 1  | Ready [0=No, 1=Yes]                           |
| 2  | Operation enabled [0=No, 1=Yes]               |
| 3  | Fault present [0=No, 1=Yes]                   |
| 4  | Coast down active (OFF2) [0=Yes, 1=No]        |
| 5  | Quick Stop active (OFF3) [0=Yes, 1=No]        |
| 6  | Switching on inhibited active [0=No, 1=Yes]   |
| 7  | Alarm present [0=No, 1=Yes]                   |
| 8  | Deviation setpoint/actual speed [0=Yes, 1=No] |
| 9  | Control request [0=No, 1=Yes]                 |
| 10 | Maximum speed reached [0=No, 1=Yes]           |
| 11 | I, M, P limit reached [0=Yes, 1=No]           |
| 12 | Motor holding brake open [0=No, 1=Yes]        |
| 13 | Alarm motor overtemperature [0=Yes, 1=No]     |
| 14 | Motor rotates forwards [0=No, 1=Yes]          |
| 15 | Alarm drive converter overload [0=Yes, 1=No]  |

#### 15.4.5. Monitoring Active Messages Online

Active messages

Faults

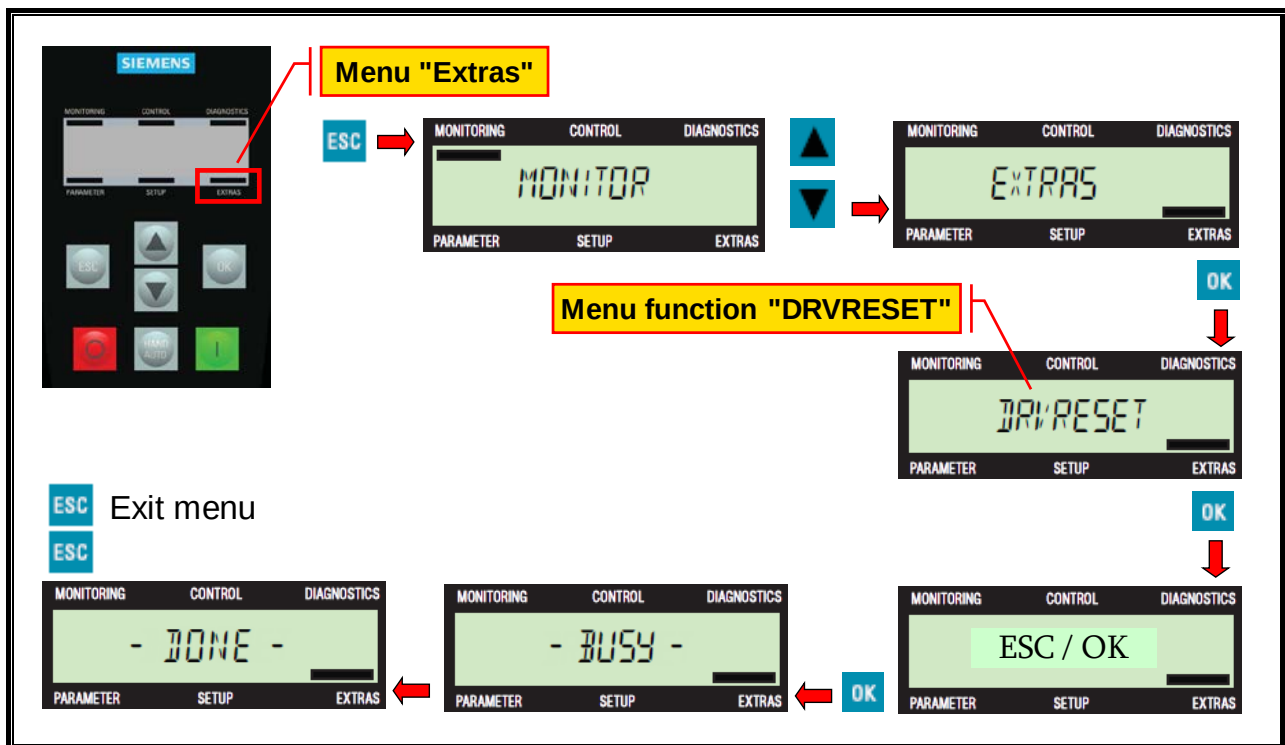
|   | Fault buffer | Fault code | Message                                 |
|---|--------------|------------|-----------------------------------------|
| 1 | Fault 1      | 1030       | Sign-of-life failure for master control |
| 2 |              |            |                                         |
| 3 |              |            |                                         |

Alarms

|   | Alarm buffer | Alarm code | Message                        |
|---|--------------|------------|--------------------------------|
| 1 | Alarm 1      | 8526       | PROFINET: No cyclic connection |
| 2 |              |            |                                |



### 15.4.6. G120 Reset to Factory Settings via BOP-2



# Contents

# 16

|            |                                                                            |             |
|------------|----------------------------------------------------------------------------|-------------|
| <b>16.</b> | <b>Training and Support .....</b>                                          | <b>16-2</b> |
| 16.1.      | Any Questions on our Training Courses Offered?? .....                      | 16-3        |
| 16.2.      | <a href="http://www.siemens.com/sitrain">www.siemens.com/sitrain</a> ..... | 16-4        |
| 16.3.      | Learning path: SIMATIC S7 Programming based on TIA Portal .....            | 16-5        |
| 16.4.      | Do You Need Service & Support? .....                                       | 16-6        |
| 16.5.      | Expertise Available in the Service & Support Portal .....                  | 16-7        |
| 16.6.      | Communication in the Service & Support Portal.....                         | 16-8        |
| 16.7.      | The Required Information – Just a Few Clicks Away 1/2 .....                | 16-9        |
| 16.8.      | The Required Information – Just a Few Clicks Away 2/2 .....                | 16-10       |
| 16.9.      | Documentation at Service & Support.....                                    | 16-11       |
| 16.10.     | Login .....                                                                | 16-12       |
| 16.11.     | What does “Configuring” Information mean? .....                            | 16-13       |

## 16. Training and Support



## 16.1. Any Questions on our Training Courses Offered??

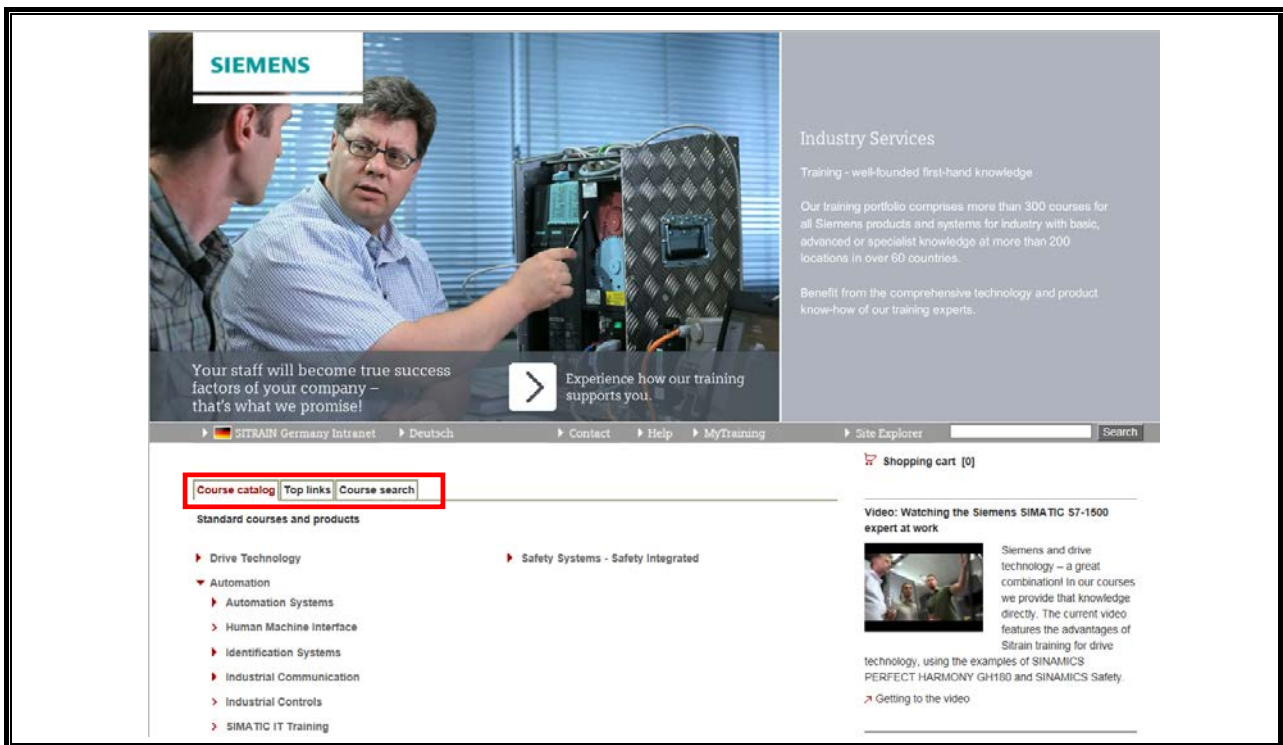


**We'll help you!**  
... on the Internet:  
[www.siemens.com/sitrain](http://www.siemens.com/sitrain)  
or with e-mail:  
[info@sitrain.com](mailto:info@sitrain.com)

### General Information

We'll be glad to help you regarding any questions on our training courses offered.

## 16.2. [www.siemens.com/sitrain](http://www.siemens.com/sitrain)



The complete range of courses offered can be accessed via the following links:

[www.siemens.de/sitrain](http://www.siemens.de/sitrain) or.

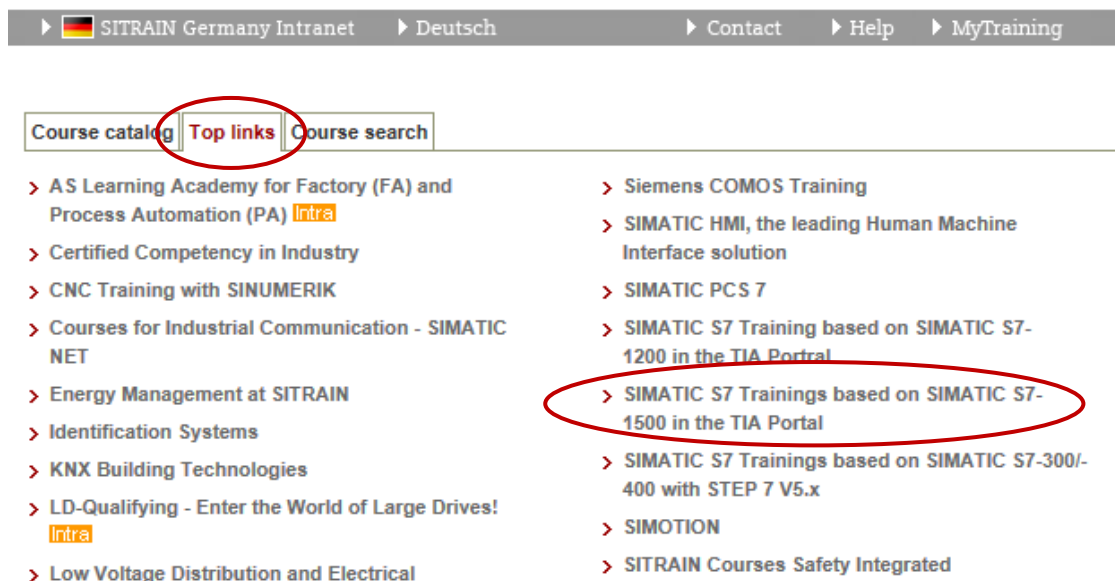
[www.siemens.com/sitrain](http://www.siemens.com/sitrain)

### Course Search

The course search permits the user to find the required courses by applying different search filters such as keyword, target group, etc. The filters can also be combined.

### Top Links

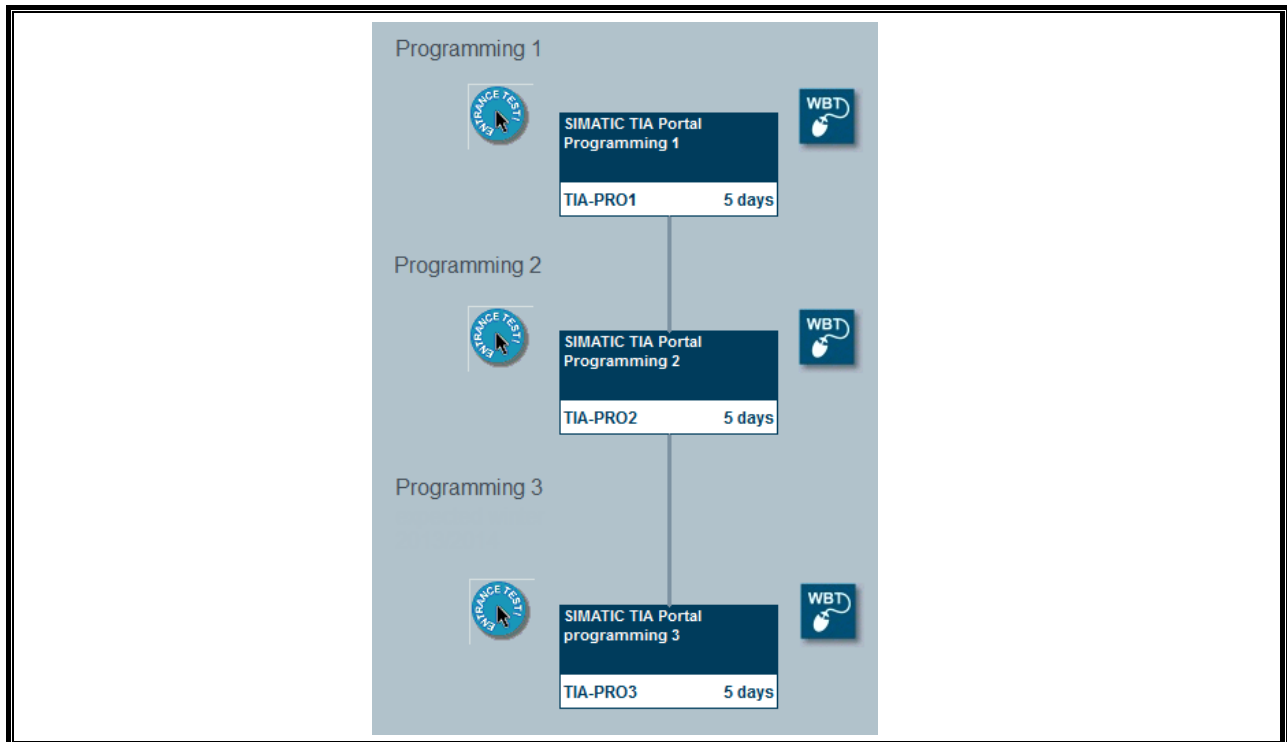
Various courses, e.g. SIMATIC S7-1500 solution line, etc., can be reached directly via the top links.



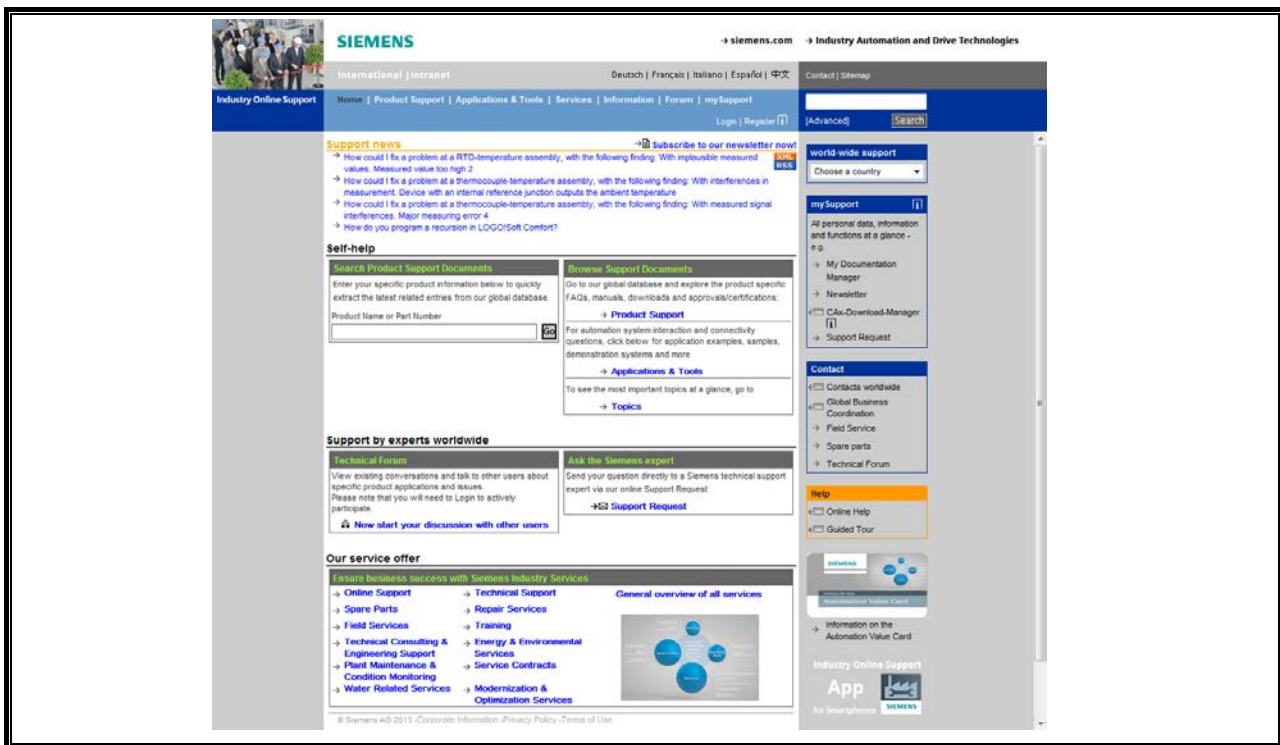
## Course Catalog

The course catalog permits you to find the required course via learning paths or via the Siemens Mall structure.

### 16.3. Learning path: SIMATIC S7 Programming based on TIA Portal



## 16.4. Do You Need Service & Support?



The Service & Support portal offers you at [www.siemens.com/automation/service&support](http://www.siemens.com/automation/service&support) online access whenever required to comprehensive information and services for the product range of Industry Automation and Drive Technology.

The online support is divided into two main topics:

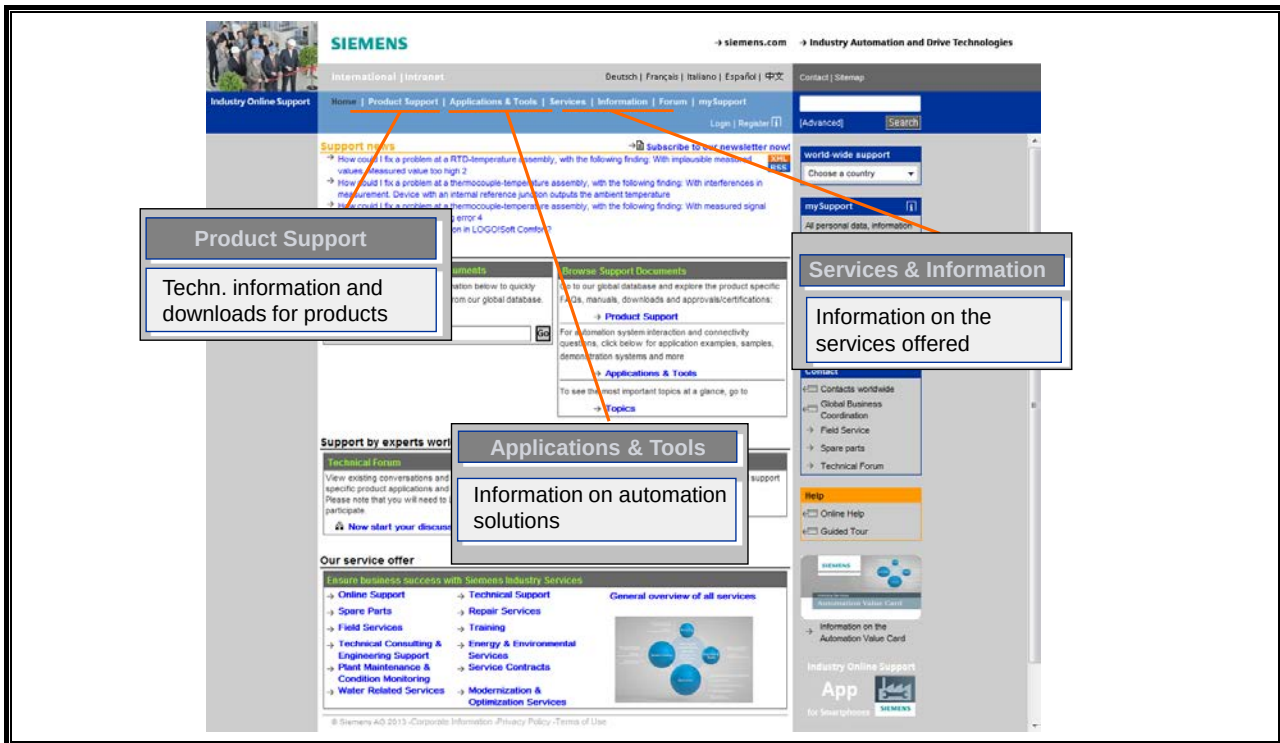
### Expertise

- **Product Support:**  
Technical information and downloads for products (manuals, FAQ, software updates, ...)
- **Applications & Tools:**  
Automation solutions (solution examples, system descriptions, demos, calculators, tools, etc.)
- **Services:**  
Information on our comprehensive range of Service & Support (contacts, repairs, ...)

### Communication

- **Forum:**  
Direct customer – customer communication in case of technical questions
- **Support Request:**  
Technical inquiries are sent direct to an expert

## 16.5. Expertise Available in the Service & Support Portal



### General Information

Online Support is a comprehensive information system with a wide range of services which is available to you whenever required. From product support to information on services all the way to interactive services, Online Support is always your first step to getting help fast.

### Product Support

Consolidated specialist knowledge with current information on every aspect of our products and systems is the basis for the expertise of Product Support in the form of Frequently Asked Questions, downloads of, for example, updates/upgrades, current product information from Marketing/Sales, electronic manuals in PDF format, test certificates and characteristics.

### Applications & Tools

Applications & Tools is the new platform for solutions and system descriptions in the Service & Support portal. Applications & Tools support you in implementing your automation tasks. This support is provided in the form of programming examples and tools, background knowledge, function descriptions, operating instructions, performance data, etc. The focus here is not on the individual product but rather on the interaction of the individual products.

### Services

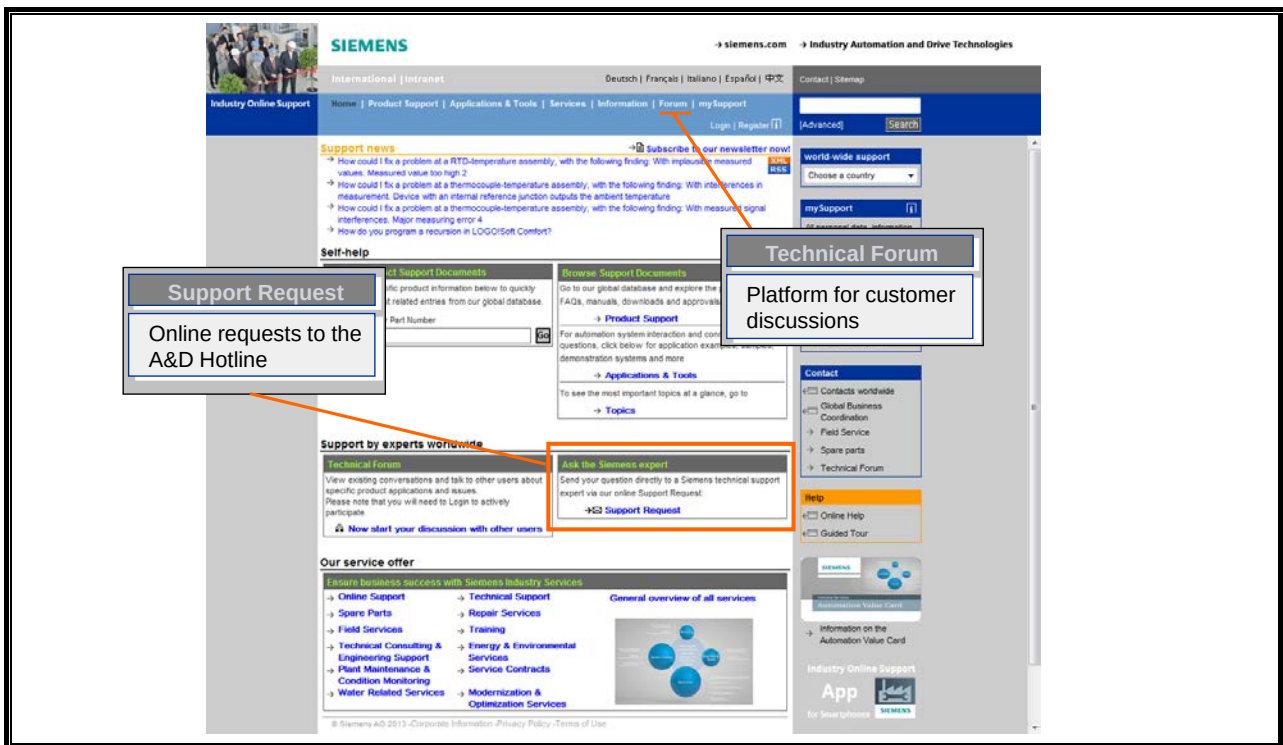
Shows you the complete range of services offered as well as detailed information on our Service & Support processes and contacts database.

### Information

Search for the latest Service & Support publications - or do you want to know at which trade fair you can meet us personally? All this and more is available under "Information".



## 16.6. Communication in the Service & Support Portal



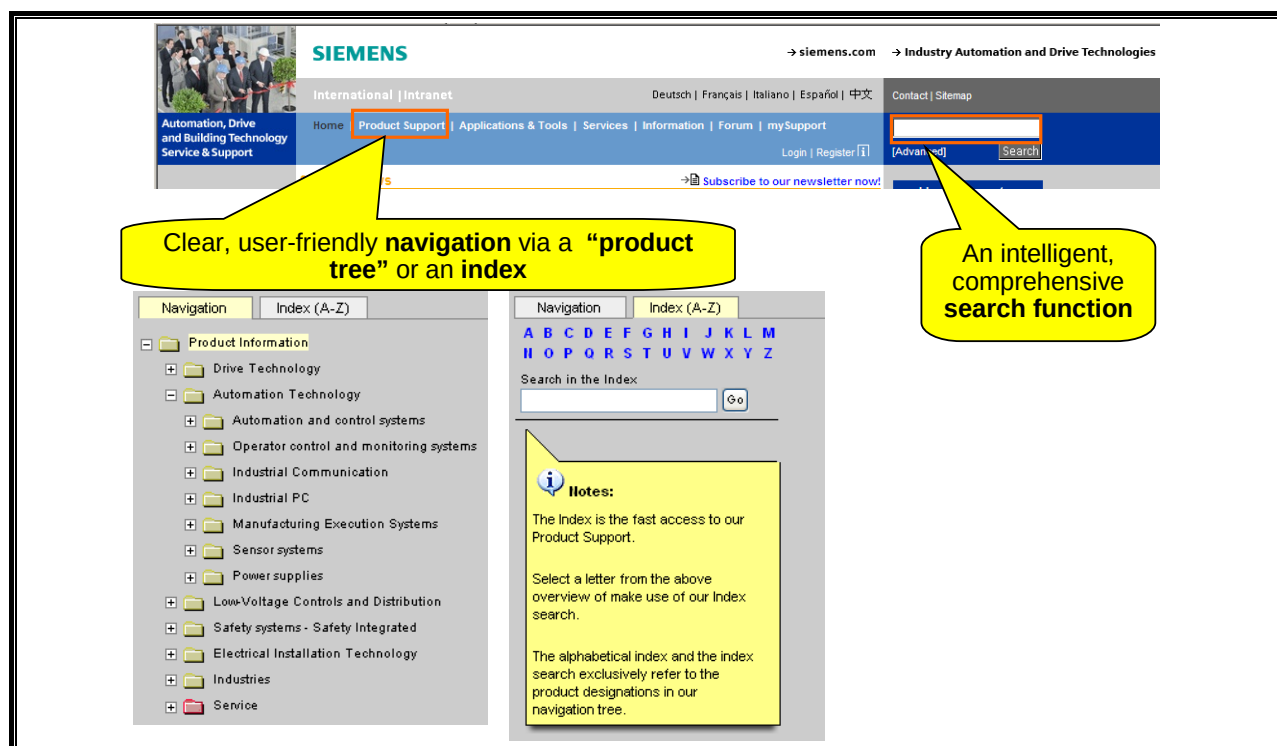
### Technical Forum

In order to supplement our comprehensive range of services and support, we promote the direct dialog between experts with the help of modern web technology. The new Technical Forum is an attractive discussion platform in German and English where users and experts can exchange information and ideas..

### Support Request

Use the online *Support Request* to always get a direct answer to your questions. You enter your inquiries in online screen forms. You immediately receive suitable solution options from the Knowledge Base. If the suggested solution does not solve your problem, send the web form direct to our team of experts at *Technical Support* (hotline).

## 16.7. The Required Information – Just a Few Clicks Away 1/2



An intelligent Search function and user-friendly Navigation function are available for finding the required information:

### Search

An intelligent and comprehensive search function is offered for finding relevant results.

- Automatic recognition of keywords
- Automatic spell-check
- Automatic sorting of the hit list entries according to up-to-dateness

### Navigation

You can also navigate via the product tree to the specific required information. An index is available to facilitate the selection of a product.

## 16.8. The Required Information – Just a Few Clicks Away 2/2

### Functions:

**Forum:** Experts can exchange information/ideas in a forum

**Newsletter:** Always up-to-date through daily news on preselected topics

**myTree:** The user can define a personal, clearly structured product tree Automatic filling in of forms with personal data, e.g. in the Support Request

### Support Request

The screenshot shows the 'Support Request' form with a progress bar at the top indicating six steps: 1. Select a product, 2. Select an application case, 3. Our solutions, 4. Describe a problem, 5. Specify contact data (highlighted), and 6. Summary & Send.

Below the progress bar, the 'Specify contact data' section is visible. It includes a 'Personal data (client)' section with a 'Form of address' dropdown and a 'Login' button. A red message states: 'You can trace the status of your request. Please login before the sending.'

The form contains several input fields:
 

- \* Surname
- \* First name
- \* e-mail
- \* Phone (Please complete with country code)
- \* Company
- Department
- \* Street
- \* Post code
- \* Town
- \* Country
- Escalation (Intranet)

At the bottom, there is a section 'How would you prefer to be contacted?' with radio buttons for 'Phone' and 'e-mail'. Below this is a text area for 'Other remarks regarding reachability' with a character count of 'remaining characters 200'.

### Registration

After registering with our service "mySupport", you can utilize our complete range of online services after entering your data only once.

**You will then have exclusive access to the following functions:**

#### Your personal newsletter

Through the Automation and Drives Newsletter system you receive daily, up-to-date news on selected topics - sent direct to your mailbox. As a mySupport user, you can individually adjust the mailing options.

#### myTree

Your personal product tree. As a mySupport user, you can adjust the product tree so that it only shows those products and services from the comprehensive A&D range that are important to you. Thus you can always maintain an overview and find the required information faster.

#### Online Support Request

You need to enter your personal data only once and they will be automatically entered in the Support Request forms. You don't have to repeatedly enter the same information in every form which saves you time for more important things.

#### myForum

Direct access to our Technical Forum. As mySupport user you also automatically have direct access to our Technical Forum

## 16.9. Documentation at Service & Support

The screenshot displays the Siemens Industry Online Support website. The header includes the Siemens logo, navigation links for International, Intranet, and various languages (Deutsch, Français, Italiano, Español, 中文), and a search bar. The main content area is divided into several sections:

- Support news:** A list of recent support articles with titles like "How could I fix a problem at a RTD-temperature assembly..." and "How could I fix a problem at a thermocouple-temperature assembly...".
- Self-help:** A section for searching product support documents, including a search bar and links to "Browse Support Documents" and "Product Support".
- Support by experts worldwide:** A section for technical forums and asking business experts, with links to "Technical Forum" and "Ask the Siemens expert".
- Our service offer:** A section detailing various services offered by Siemens, including Online Support, Technical Support, Spare Parts, Repair Services, Field Services, Training, Technical Consulting & Engineering Support, Energy & Environmental Services, Plant Maintenance & Condition Monitoring, Service Contracts, Water Related Services, Modernization & Optimization Services, and a General overview of all services.

On the right side of the page, there is a sidebar with additional navigation options:

- world-wide support:** A link to choose a country.
- mySupport:** A section for personal data, information, and functions, including links to "My Documentation Manager", "CAX-Download-Manager", and "Support Request".
- Contact:** A section for contacts worldwide, including links to "Global Business Coordination", "Field Service", "Spare parts", and "Technical Forum".
- Help:** A section for online help and guided tours.

The footer of the page includes the copyright notice: "© Siemens AG 2013 - Corporate Information - Privacy Policy - Terms of Use".

## 16.10. Login

For using the „My Documentation Manager“ you have to login.

**SIEMENS**

Login now at **My Documentation Manager**.

**Are you already registered?**

Login ⓘ

\_\_\_\_\_

Password

\_\_\_\_\_

☒ Stay permanently logged in.  
(except for A&D Mail, Extranet) ⓘ

→ [Forgotten your Password?](#) [Login](#)

**First time here?**

Register now to use the full range of functionality offered by the Automation and Drives Web. For each of the different access zones only one registration is required!

→ [Yes, I would like to register now.](#)

→ [More information](#)

## 16.11. What does “Configuring” Information mean?

